

Introduction to Computational Finance and Financial Econometrics with R

Eric Zivot

2026-09-07

Table of contents

Preface	3
Acknowledgments	7
1 Return Calculations	8
1.1 The Time Value of Money	8
1.1.1 Future value, present value and simple interest	8
1.1.2 Multiple compounding periods	11
1.1.3 Effective annual rate	12
1.2 Asset Return Calculations	14
1.2.1 Assets	14
1.2.2 Simple returns	16
1.2.3 Continuously compounded returns	28
1.3 Portfolios and Portfolio Returns	35
1.3.1 Multiperiod portfolio returns and rebalancing	37
1.3.2 Continuously Compounded Portfolio Returns	46
1.4 Return Calculations with Data in R	47
1.4.1 Representing time series data using <code>xts</code> objects	47
1.4.2 Calculating returns	57
1.4.3 Calculating portfolio returns from time series data	61
1.4.4 Downloading financial data from the internet	65
1.5 Further Reading: Return Calculations	65
1.6 Appendix: Properties of Exponentials and Logarithms	67
1.7 Exercises	68
Analytical Problems	68
Return Calculations using R Packages	72
2 Review of Random Variables	74
2.1 Random Variables	74
2.1.1 Discrete random variables	75
2.1.2 Continuous random variables	78
2.1.3 The cumulative distribution function	82
2.1.4 Quantiles of the distribution of a random variable	83
2.1.5 R functions for discrete and continuous distributions	85
2.1.6 Shape characteristics of probability distributions	88
2.1.7 Linear functions of a random variable	101
2.1.8 Value-at-Risk: An introduction	104
2.2 Bivariate Distributions	108
2.2.1 Discrete random variables	108
2.2.2 Bivariate distributions for continuous random variables	114

2.2.3	Independence	116
2.2.4	Covariance and correlation	118
2.2.5	Bivariate normal distributions	123
2.2.6	Expectation and variance of the sum of two random variables	125
2.3	Multivariate Distributions	127
2.3.1	Discrete random variables	127
2.3.2	Continuous random variables	128
2.3.3	Independence	129
2.3.4	Dependence concepts	129
2.3.5	Linear combinations of N random variables	130
2.3.6	Covariance between linear combinations of random variables	131
2.4	Further Reading: Review of Random Variables	132
2.5	Exercises	133
3	Matrix Algebra Review	137
3.1	Matrices and Vectors	137
3.2	Basic Matrix Operations	142
3.2.1	Addition and subtraction	142
3.2.2	Scalar multiplication	143
3.2.3	Matrix multiplication	144
3.2.4	The identity matrix	146
3.2.5	Diagonal, lower triangular and upper triangular matrices	147
3.3	Representing Summation Using Matrix Notation	149
3.4	Systems of Linear Equations	151
3.4.1	Rank of a matrix	156
3.4.2	Partitioned matrices and partitioned inverses	157
3.5	Positive Definite Matrices	159
3.5.1	Matrix square root	159
3.6	Multivariate Probability Distributions Using Matrix Algebra	161
3.6.1	Random vectors	161
3.6.2	Covariance matrix	161
3.6.3	Correlation matrix	162
3.6.4	Variance of linear combination of random vectors	163
3.6.5	Covariance between linear combination of two random vectors	164
3.6.6	Variance of a vector of linear functions of a random vector	164
3.6.7	Multivariate normal distribution	164
3.7	Portfolio Math Using Matrix Algebra	166
3.8	Derivatives of Simple Matrix Functions	169
3.9	Further Reading: Matrix Algebra Review	171
3.10	Problems: Matrix Algebra Review	172
4	Time Series Concepts	174
4.1	Stochastic Processes	175
4.1.1	Stationary stochastic processes	175
4.1.2	Non-Stationary processes	183
4.1.3	Ergodicity	186
4.2	Multivariate Time Series	187
4.2.1	Stationary and ergodic multivariate time series	187

4.3	Time Series Models	190
4.3.1	Moving average models	191
4.3.2	Autoregressive Models	198
4.3.3	Autoregressive Moving Average Models	203
4.3.4	Vector Autoregressive Models	204
4.4	Forecasting	205
4.5	Further Reading: Time Series Concepts	205
4.6	Problems: Time Series Concepts	205
5	Descriptive Statistics for Financial Data	208
5.1	Univariate Descriptive Statistics	208
5.1.1	Example data	209
5.1.2	Time plots	210
5.1.3	Descriptive statistics for the distribution of returns	215
5.1.4	QQ-plots	225
5.1.5	Shape characteristics of the empirical distribution	230
5.1.6	Outliers	233
5.1.7	Box plots	236
5.2	Time Series Descriptive Statistics	238
5.2.1	Sample autocovariances and autocorrelations	238
5.2.2	Time dependence in volatility	240
5.3	Bivariate Descriptive Statistics	243
5.3.1	Scatterplots	243
5.3.2	Sample covariance and correlation	245
5.3.3	Sample cross-lag covariances and correlations	249
5.4	Rolling descriptive statistics	251
5.4.1	Practical issues associated with rolling estimates	258
5.5	Stylized facts for daily and monthly asset returns	259
5.6	Further Reading: Descriptive Statistics	259
5.7	Problems: Descriptive Statistics	260
6	The Gaussian White Noise Return Model	265
6.1	GWN Model Assumptions	265
6.1.1	Regression model representation	266
6.1.2	Location-Scale model representation	268
6.1.3	The GWN model in matrix notation	268
6.1.4	The GWN model for continuously compounded returns	269
6.1.5	GWN model for simple returns	273
6.2	Monte Carlo Simulation of the GWN Model	276
6.2.1	Simulating returns on more than one asset	284
6.3	Scenario Analysis	289
6.4	Further Reading: The GWN Return Model	289
6.5	Exercises	289
7	Estimation of The GWN Model	291
7.1	Estimators and Estimates	292
7.2	Finite Sample Properties of Estimators	293
7.2.1	Bias	294

7.2.2	Precision	295
7.2.3	Estimated standard errors	297
7.3	Asymptotic Properties of Estimators	297
7.3.1	Consistency	297
7.3.2	Asymptotic Normality	298
7.3.3	Central Limit Theorem	299
7.3.4	Asymptotic Confidence Intervals	300
7.4	Estimators for the Parameters of the GWN Model	300
7.5	Statistical Properties of the GWN Model Estimates	305
7.5.1	Bias	305
7.5.2	Precision	306
7.5.3	Sampling Distributions and Confidence Intervals	310
7.6	Using Monte Carlo Simulation to Understand the Statistical Properties of Estimators	316
7.6.1	Evaluating the Statistical Properties of $\hat{\mu}$ Using Monte Carlo Simulation . . .	316
7.6.2	Evaluating the Statistical Properties of $\hat{\sigma}_i^2$ and $\hat{\sigma}_i$ Using Monte Carlo simulation	320
7.6.3	Evaluating the Statistical Properties of $\hat{\sigma}_{ij}$ and $\hat{\rho}_{ij}$ by Monte Carlo simulation	323
7.7	Sampling distributions of $\hat{\mu}$ and $\hat{\Sigma}$	327
7.7.1	Sampling distribution of $\hat{\mu}$	327
7.7.2	Sampling distribution of the elements of $\hat{\Sigma}$	328
7.7.3	Joint sampling distribution between $\hat{\mu}$ and $\text{vech}(\hat{\Sigma})$	329
7.8	Further Reading: Estimation of The GWN Model	330
7.9	Problems: Estimation of The GWN Model	331
8	Estimating Functions of GWN Model Parameters	332
8.1	Functions of GWN Model Parameters	332
8.2	Estimation of Functions of GWN Model Parameters	334
8.2.1	Bias	335
8.2.2	Consistency	336
8.3	Asymptotic normality and the delta method	337
8.3.1	The numerical delta method	343
8.3.2	The delta method for vector valued functions (advanced)	345
8.3.3	The numerical delta method for vector valued functions (advanced)	347
8.3.4	The delta method explained	347
8.4	Resampling	348
8.5	The Jackknife	348
8.5.1	The jackknife for vector-valued estimates	354
8.5.2	Pros and Cons of the Jackknife	356
8.6	The Nonparametric Bootstrap	356
8.6.1	Bootstrap bias estimate	357
8.6.2	Bootstrap standard error estimate	358
8.6.3	Bootstrap confidence intervals	359
8.6.4	The bootstrap for vector-valued estimates	360
8.6.5	Performing the nonparametric bootstrap in R	360
8.7	Brute force nonparametric bootstrap procedure	360
8.7.1	Pros and cons of the nonparametric bootstrap	373
8.8	Using Monte Carlo Simulation to Evaluate Delta Method, Jackknife and Bootstrap Standard Errors	374
8.9	Further Reading: Delta Method and Resampling	379

8.10 Exercises	379
9 Hypothesis Testing in the GWN Model	381
9.1 Review of General Principles	382
9.1.1 Steps for hypothesis testing	382
9.1.2 Hypothesis tests and decisions	383
9.1.3 Significance level and power	384
9.2 Hypothesis Testing in the GWN Model	385
9.2.1 Coefficient tests	385
9.2.2 Model specification tests	387
9.2.3 Data for hypothesis testing examples	388
9.3 Tests for Individual Parameters: t-tests and z-scores	389
9.3.1 Exact tests under normality of data	389
9.3.2 Exact tests with σ unknown	395
9.3.3 Z-scores under asymptotic normality of estimators	397
9.3.4 Relationship between hypothesis tests and confidence intervals	401
9.4 Coefficient tests between model coefficients for two assets	403
9.4.1 Test for equal means between two assets	403
9.4.2 Test for equal Sharpe ratios between two assets	405
9.5 Wald tests for general linear hypotheses (advanced)	407
9.6 Model Specification Tests	407
9.6.1 Tests for normality	407
9.6.2 Tests for no autocorrelation	412
9.6.3 Tests for constant parameters	414
9.6.4 Tests for constant parameters based on rolling estimators	415
9.7 Using Monte Carlo Simulation to Understand Hypothesis Testing	418
9.8 Using the Bootstrap for Hypothesis Testing	418
9.9 Further Reading: Hypothesis Testing in the GWN Model	418
9.10 Exercises	418
References	419

Preface

I am a Professor of Economics at the University of Washington (UW) where I teach the class *Introductory to Computational Finance and Financial Econometrics* (ECON 424). This course started as an experiment. In the winter quarter of 1998 I was on leave visiting the economics department at the University of California Los Angeles (UCLA). To help pay for my leave I, I arranged to teach a standard undergraduate econometrics course. My research at the time was moving toward financial econometrics and I thought it would be fun to teach an econometrics course with most of the examples motivated by financial applications. I thought that combining econometrics and statistics topics with financial applications would be a very effective and popular way of getting students interested in learning and applying econometrics. My intuition was right. My students at UCLA loved the version of the course I offered. After returning to the University of Washington, I created the upper division undergraduate course *Introduction to Computational and Financial Econometrics*. The course was an immediate hit. This book evolved from my lecture notes for the course.

Who is the book written for? What background is required?

My ECON 424 course is taken mainly by senior economics majors who are pursuing the more quantitative Bachelor of Science Degree in Economics at UW. These students typically have at least one year of calculus, intermediate microeconomics with some calculus (so they are exposed to optimization under constraints using Lagrange multipliers), and an introductory course in probability and statistics for economics majors. Some students have taken a course in linear algebra, some have taken an introductory econometrics course, and some have taken a financial economics course. I find that the typical introductory probability and statistics course is at a lower level than is needed for much of the material in my course. Ideally, students should have an intermediate probability and statistics course that uses calculus¹. As a result, I spend a week at the beginning of the course doing a review of random variables and probability that uses basic calculus. A key part of the review is the manipulation of random variables, computing expectations of functions of random variables, etc., so that the students have the necessary tools to work with simple probabilistic models for asset returns. After the review of probability theory I introduce some basic time series concepts (e.g., stochastic processes and stationarity), as these concepts are often not taught in introductory probability and statistics classes, and I also do a quick review of basic matrix algebra that I use throughout the course. The use of matrix algebra is important for two reasons. First,

¹Insert reference to recent MIT Press book here

most introductory textbook treatments of portfolio theory show calculations for the simple case of two assets and then talk about how the calculations extend to the general case of many assets but do not show how this is done. Hence, students are not capable of implementing the theory for real world portfolios. I emphasize that with knowledge of just a little bit of matrix algebra - essentially matrix addition, multiplication, inversion, and some matrix differential calculus - the general analysis of portfolio theory can be easily learned and implemented. Second, R is a vectorized and matrix programming language and so basic knowledge of matrix algebra allows students to easily understand and implement the R code for portfolio analysis.

Main topics covered in the book

After the review material, I jump into descriptive statistical analysis of historical financial time series data. I emphasize that my approach to building models for asset returns is based on first looking at the historical data and studying its properties. I start by telling students to pretend that observed returns on assets are realizations from some unknown covariance stationary stochastic process. I review common graphical and numerical descriptive statistics (e.g., histograms, QQ-plots, sample mean, variance, correlation, etc.). The goal of descriptive statistical analysis is to compile a collection of “stylized facts” that can be used as building blocks for probabilistic models for asset returns. For example, I show that monthly asset returns are typically uncorrelated over time, have empirical distributions (histograms) that look like normal bell curves, have means close to zero, are positively correlated with other assets, etc. I also emphasize that some “stylized facts” of asset returns appear to stay the same for different investment horizons (daily, weekly, and monthly returns are uncorrelated) and some do not (e.g., daily and weekly returns have fatter tails than the normal and show evidence of nonlinear time dependence in higher moments)

Why us R?

My early versions of *Introduction to Computational and Financial Econometrics* used Microsoft Excel as the main software tool as R was not yet around then. My decision to use Excel was for the students’ benefit. Students having experience with Excel was, and still is, in high demand by employers in the finance industry. It is often said, “the financial world runs out of an Excel spreadsheet”. However, Excel is not a good tool for doing statistics. Even though you *can* do statistical analysis in Excel, does not mean that you *should* do statistical analysis in Excel. I searched for Excel add-ins to make doing statistical analysis in Excel easier but I did not like what was available. Everything changed when I met Professor Doug Martin from the UW statistics department. Doug founded the company who created S-PLUS, the commercial version of the S statistical programming language developed by John Chambers at Bell Labs (R became the open source version of S in 2000). S is a beautiful language for exploratory data and statistical analysis and S-PLUS made it easily available to students on windows PCs. In the early 2000s I used Excel together with S-PLUS for my course. This hybrid approach worked well but the difficulties of getting student licenses at the beginning of the quarter was always a headache. As R matured in the mid to late 2000s, I made the switch from S-PLUS to R and never looked back. As the

course evolved and R became more commonly used in the financial world, and Rstudio became the integrated developer environment of choice for R, I eventually dropped the use of Excel and focused solely on R for the applications and examples in the book.

Many economics students lack experience with programming languages and some find R a difficult language to learn and use. My approach to dealing with this reality is to provide all of the R code to do calculations in the class lectures, in the textbook examples, and on homework assignments. In this way students do not have to worry about the coding aspect of the class and can more easily concentrate on the concepts of computational finance and financial econometrics. I find that students learn to use R quite well this way – learning by doing – and most students are quite proficient in R by the end of the course.

There are a lot of free resources on the web for learning R. In my Coursera course I partnered with DataCamp to provide a web-based learning environment for R tailored to the course material. I have found that this has substantially lowered the learning curve for many students. Student mostly use either a MAC or PC for their work (a few geeks use UNIX). I found that some things didn't always work well across platforms (usually issues with the file system). In 2014, I required students to use Rstudio as the environment for using R. This makes learning and working with R even easier. Another advantage of using Rstudio is that my lecture notes and homework assignments are created using Rmarkdown.

Longer print book

(Shorter ebook). This book is based on my University of Washington sponsored Coursera course Introduction to Computational Finance and Financial Econometrics that has been running every quarter on Coursera since 2013. This Coursera course is based on the Summer 2013 offering of my University of Washington advanced undergraduate economics course of the same name. At the time, my UW course was part of a three course summer certificate in Fundamentals of Quantitative Finance offered by the Professional Masters Program in Computational Finance & Risk Management that was video-recorded and available for online students. An edited version of this course became the Coursera course. The popularity of the course encouraged me to convert the class notes for the course into a short book.

The book was typeset using Lyx with all of the R code examples embedded into the document and evaluated using knitr. I am extremely grateful to Yihui Xie for writing his wonderful book Dynamics Documents with R and knitr as this book would not have been possible without it. I am a firm believer in the “reproducible research paradigm”.

The book's accompanying R package `introcompfinr` contains all of the financial data used for the examples presented in the book as well as a number of R functions written by me for estimating some simple models of asset returns, and doing portfolio and risk analysis. The package is freely available on GitHub at <https://github.com/ezivot/introcompfinr> and can be installed using:

```
install.packages("pak")  
pak::pak("ezivot/introcompfinr").
```

How is this book different from other introductory book on financial econometrics?

There are now many good books written on the topic of computational/quantitative finance and financial econometrics with examples in R at various levels of sophistication. Books written for advanced U.S. undergraduates include Carmona (2014) and Tsay (2012). Books written for Masters students in computational finance or financial engineering courses include Fan and Yao (20xx), Lai and Xing (2008), Ruppert and Matteson (2015) and Tsay (2010). All of these books assume the readers have a broader mathematical and statistical background than what I assume here. Hence, this book is truly an “introduction” to the methods of computational finance and financial econometrics and is appropriate for undergraduate economics and finance majors at Universities worldwide. Another feature that distinguishes this book from others is the extensive use of R and illustrated R code throughout the book. In this regard, this book is a “hands on” book in which the student can readily execute the R examples on their own computer while reading the book. This allows the student to also learn R as well as the concepts in the book.

Eric Zivot
Seattle, Washington
August 2026

Acknowledgments

Many people have helped with the development of this book. First and foremost, I would like to thank the students at UW who have taken my course over the last fifteen years. I have received countless comments, corrections, and suggestions that have greatly improved the exposition and material in the book.

A lot of how I think about the statistical analysis of financial data has been shaped by many discussions and interactions with my long-term colleague and good friend R. Douglas Martin. Doug introduced me to the S language and I had the pleasure of working with him at Insightful where we developed the S+FinMetrics module for S-PLUS and I wrote the book *Modeling Financial Time Series with S-PLUS* with my former Ph.D. student Jiahui (Jeffrey) Wang.

My very patient wife Nina Sidneva carefully proof-read many early versions of the chapters as they were under development and caught many of my stupid errors. Xiao Yin proof-read most of the chapters after I converted my Latex notes to Lyx and incorporated the R examples with `knitr`. I could not have written the book without the expert technical help from Bethany Yollin and the wonderful book *Dynamics Documents with R and knitr* by Yihui Xie. Bethany helped with the conversion of my Latex notes to Lyx and the incorporation of the R examples in the text using `knitr`. She also helped to develop and maintain an early version the book's R package, `IntroCompfinR`. Any problem I found with Lyx and `knitr` was solved by referring to Yihui's book. In 2020 I decided to migrate the book from Lyx to Rmarkdown so that everything could be managed within Rstudio and the book could be generated using `bookdown`. My UW PhD RAs Yumming (Herbert) Liu, Pushpak Sarkar and Kovid Puria made this happen. In 2026, I moved from `bookdown` to Quarto.

My interactions with the R community, especially those involved with R/Finance, have been invaluable for learning and using R for the analysis of financial data. In particular, Brian Peterson and Peter Carl have been incredibly helpful and supportive of my book projects. I use their wonderful R package `PerformanceAnalytics` throughout the book. Working with time series data is made so much easier with Achiem Zieles' `zoo` package and Jeff Ryan's `xts` package. Many University of Washington (UW) econ PhD students (Brian Donhauser, Ming He, Wan-Jung Hsu, Kara Ng, Anthony Sanford, Galib Sultan, Cindy Huang) acted as TAs for my undergraduate course and gave constructive feedback. I also thank Coursera for helping me convert my undergraduate course into Coursera format.

1 Return Calculations

Updated: September 1, 2026 Copyright © Eric Zivot 2026

Asset returns are the fundamental object of study in this book and in this chapter I review many important definitions and properties of asset returns with an emphasis on equity returns. It is important to understand the different ways in which asset returns are calculated as they impact model choices for asset returns. For example, simple returns are most commonly used for money and portfolio calculations in practice. However, simple returns have some undesirable properties for time aggregation that make mathematical and statistical modeling difficult. Continuously compounded returns, in contrast, have nicer aggregation properties that make mathematical and statistical modeling easier.

This chapter is organized as follows. Section Section 1.1 covers basic time value of money calculations. Section Section 1.2 covers asset return calculations, including both simple and continuously compounded returns. Section Section 1.3 defines portfolios and details portfolio return calculations. Section Section 1.4 illustrates asset return calculations using R.

In this chapter I use the following R packages: **ggplot2**, **introcompfinr**, **methods**, **PerformanceAnalytics**, **quantmod**, **xts**, and **zoo**. Make sure these packages are installed and loaded before replicating the chapter examples.

1.1 The Time Value of Money

In this section I review basic time value of money calculations. The concepts of future value, present value and the compounding of interest are defined and discussed.

1.1.1 Future value, present value and simple interest

I start with the definition of the *future value* of an investment.

Definition 1.1 (Future value). Consider an amount $\$V$ invested for n years at a *simple interest rate* of R per annum (where R is expressed as a decimal). If compounding takes place only at the end of the year, the *future value* after n years is defined as:

$$FV_n = \$V(1 + R) \times \cdots \times (1 + R) = \$V(1 + R)^n. \quad (1.1)$$

Over the first year, $\$V$ grows to $\$V(1 + R) = \$V + \$V \times R$ which represents the initial principal $\$V$ plus the payment of simple interest $\$V \times R$ for the year. Over the second year, the new principal $\$V(1 + R)$ grows to $\$V(1 + R)(1 + R) = \$V(1 + R)^2$, and so on.

Example 1.1 (Future value with simple interest). Consider putting \$1,000 in an interest checking account that pays a simple annual percentage rate of 3%. The future value after $n = 1, 5$ and 10 years is, respectively,

$$FV_1 = \$1,000 \cdot (1.03)^1 = \$1,030,$$

$$FV_5 = \$1,000 \cdot (1.03)^5 = \$1,159.27,$$

$$FV_{10} = \$1,000 \cdot (1.03)^{10} = \$1,343.92.$$

Over the first year, \$30 in interest is accrued; over three years, \$159.27 in interest is accrued; over five years, \$343.92 in interest is accrued. These calculations are easy to perform in R:

```
V = 1000
R = 0.03
FV1 = V*(1 + R)
FV5 = V*(1 + R)^5
FV10 = V*(1 + R)^10
c(FV1, FV5, FV10)
#> [1] 1030 1159 1344
```

■

The future value formula Equation 1.1 defines a relationship between four variables: FV_n , V , R and n . Given three variables, the fourth variable can be determined. For example, given FV_n , R and n and solving for V gives the *present value* formula:

$$V = \frac{FV_n}{(1 + R)^n}. \quad (1.2)$$

Here, V represents the present value (or current value) of the future value FV_n that is to be received n years from today. The future value is discounted by the factor $(1 + R)^{-n}$ to reflect the fact that dollars to be received in n years are worth less than current dollars because current dollars can be invested today returning R per year. The discount factor $(1 + R)^{-n}$ is less than 1, and represents the price today of \$1 to be received in n years when the interest rate is R per year.

Taking FV_n , n and V as given, the *compound annual return* on the investment is defined as:

$$R = \left(\frac{FV_n}{V} \right)^{1/n} - 1. \quad (1.3)$$

Here, the simple interest rate R also represents a type of average rate of return on an investment. Specifically, R is the rate of return which compounded annually over n years transforms the present value V into the future value FV_n .

Finally, given FV_n , V and R you can solve for n :

$$n = \frac{\ln(FV_n/V)}{\ln(1+R)}. \quad (1.4)$$

The expression Equation 1.4 is called the *horizon period*. The horizon period is the number of years it takes for an investment of $\$V$ to grow to a future value of $\$FV_n$ at a simple interest rate of R per year.

The horizon period formula can be used to determine the number of years it takes for an investment of $\$V$ to double. Setting $FV_n = 2V$ in Equation 1.4 gives:

$$n = \frac{\ln(2)}{\ln(1+R)} \approx \frac{0.7}{R},$$

which uses the approximations $\ln(2) = 0.6931 \approx 0.7$ and $\ln(1+R) \approx R$ for R close to zero. The approximation $n \approx 0.7/R$ is called the *rule of 70*. For example, if $R = 0.07$ (7% per year) then $n \approx 0.7/0.07 = 70/7 = 10$ years for the investment to double.

Example 1.2 (Using the rule of 70). The rule of 70 gives a good approximation as long as the interest rate is not too high.

```
R = seq(0.01, 0.10, by=0.01)
nExact = log(2)/log(1 + R)
nRule70 = 0.7/R
cbind(R, nExact, nRule70)
#>      R nExact nRule70
#> [1,] 0.01  69.66   70.00
#> [2,] 0.02  35.00   35.00
#> [3,] 0.03  23.45   23.33
#> [4,] 0.04  17.67   17.50
#> [5,] 0.05  14.21   14.00
#> [6,] 0.06  11.90   11.67
#> [7,] 0.07  10.24   10.00
#> [8,] 0.08   9.01    8.75
#> [9,] 0.09   8.04    7.78
#> [10,] 0.10   7.27    7.00
```

When $R = 0.01$ it takes approximately 70 years for your money to double. When $R = 0.10$ it takes approximately 7 years. For R values between 1% and 10% the approximation error in the rule of 70 is never more than half a year.

■

1.1.2 Multiple compounding periods

Often interest is paid more than once per year. In the case, the future value formula needs to be adjusted.

Definition 1.2 (Future value with multiple compounding periods). Consider an amount $\$V$ invested for n years at a simple interest rate of R per year. If interest is paid m times per year, then the future value after n years is:

$$FV_n^m = \$V \cdot \left(1 + \frac{R}{m}\right)^{m \cdot n}.$$

The value R/m is called the *periodic interest rate*. As $m \rightarrow \infty$, the periodic rate becomes *continuously compounded* and the future value becomes:

$$FV_n^c = \lim_{m \rightarrow \infty} \$V \cdot \left(1 + \frac{R}{m}\right)^{m \cdot n} = \$V \cdot e^{R \cdot n},$$

where $e^{(\cdot)}$ is the exponential function and $e^1 = 2.71828$.

Example 1.3 (Future value with different compounding frequencies). For a simple annual percentage rate of 10%, the value of \$1000 at the end of one year ($n = 1$) for different values of m is calculated below.

```
V = 1000
R = 0.1
m = c(1, 2, 4, 356, 10000)
FV = V*(1 + R/m)^(m)
print(cbind(m, FV), digits=7)
#>           m      FV
#> [1,]      1 1100.000
#> [2,]      2 1102.500
#> [3,]      4 1103.813
#> [4,]    356 1105.155
#> [5,] 10000 1105.170
```

The result with continuous compounding is

```
print(V*exp(R), digits=7)
#> [1] 1105.171
```

You get more interest as the number of compounding periods increases but the amount of extra interest earned essentially peaks at daily compounding. The difference between daily and continuous compounding is very small for a \$1,000 investment at 10% simple interest rate. ■

The continuously compounded return analogues to the present value, annual return and horizon period formulas Equation 1.2, Equation 1.3 and Equation 1.4 are:

$$V = e^{-Rn} FV_n, \quad (1.5)$$

$$R = \frac{1}{n} \ln \left(\frac{FV_n}{V} \right), \quad (1.6)$$

$$n = \frac{1}{R} \ln \left(\frac{FV_n}{V} \right). \quad (1.7)$$

1.1.3 Effective annual rate

I now consider the relationship between simple interest rates, periodic rates, effective annual rates and continuously compounded rates. Suppose an investment pays a periodic interest rate of 2% each quarter. This gives rise to a simple annual rate of 8% ($2\% \times 4$ quarters). At the end of the year, \$1,000 invested accrues to:

$$\$1000 \cdot \left(1 + \frac{0.08}{4} \right)^{4 \cdot 1} = \$1082.40.$$

Here, compounding the periodic rate 2% for 4 quarters produces \$82.20 of interest on a \$1,000 investment. What simple interest rate R_A , compounded once per year, would produce the same amount of interest? This *effective annual rate*, R_A , is determined by the relationship:

$$\$1,000 \cdot (1 + R_A) = \$1,082.40.$$

Solving for R_A gives:

$$R_A = \frac{\$1,082.40}{\$1,000} - 1 = 0.0824,$$

or $R_A = 8.24\%$. Here, the effective annual rate 8.4% is the simple interest rate with annual compounding that gives the same future value as the simple rate 8% compounded four times per year. The effective annual rate is greater than the simple annual to account for the payment of interest on interest.

The general relationship between the simple annual rate R with interest compounded m times per year and the effective annual rate, R_A , compounded once per year is given in the following

definition.

Definition 1.3 (Effective annual rate). Let R be a simple annual interest rate with interest compounded m times per year and let R_A be the effective annual rate. The relationship between the simple annual rate and the effective annual rate is given by:

$$(1 + R_A) = \left(1 + \frac{R}{m}\right)^m.$$

Given the simple rate R , you can solve for the effective annual rate using:

$$R_A = \left(1 + \frac{R}{m}\right)^m - 1. \quad (1.8)$$

Given the effective annual rate R_A , you can solve for the corresponding simple rate using:

$$R = m \left[(1 + R_A)^{1/m} - 1\right]. \quad (1.9)$$

The relationship between the effective annual rate and the simple rate that is compounded continuously is:

$$(1 + R_A) = e^R.$$

The continuously compounded effective annual rate is then

$$R_A = e^R - 1. \quad (1.10)$$

Given a continuously compounded effective annual rate, the implied simple rate is

$$R = \ln(1 + R_A). \quad (1.11)$$

Example 1.4 (Determine effective annual rates). The effective annual rates associated with the investments in example [Example 1.2](#) are calculated below.

```
RA = FV/V - 1
print(cbind(m, FV, RA), digits=7)
#>      m      FV      RA
#> [1,]  1 1100.000 0.1000000
#> [2,]  2 1102.500 0.1025000
#> [3,]  4 1103.813 0.1038129
#> [4,] 356 1105.155 0.1051554
#> [5,]10000 1105.170 0.1051704
```

The effective annual rate with continuous compounding is

```
print(exp(R) - 1, digits=7)
#> [1] 0.1051709
```

■

Example 1.5 (Determine continuously compounded rate from effective annual rate). Suppose an investment pays a periodic interest rate of 5% every six months ($m = 2$, $R/2 = 0.05$). In the market this would be quoted as having an annual percentage rate, R_A , of 10%. An investment of \$100 yields $\$100 \cdot (1.05)^2 = \110.25 after one year. The effective annual rate, R_A , is then 10.25%. To find the continuously compounded simple rate that gives the same future value as investing at the effective annual rate you solve: $R = \ln(1.1025) = 0.09758$. That is, if interest is compounded continuously at a simple annual rate of 9.758% then \$100 invested today would grow to $\$100 \cdot e^{0.09758} = \110.25

■

1.2 Asset Return Calculations

In this section, I review asset return calculations given initial and future prices associated with an investment. I first summarize the main type of assets used for investment purposes. Next, I cover simple return calculations, which are typically reported in practice but are often not convenient for statistical modeling purposes. I then describe continuously compounded return calculations, which are more convenient for statistical modeling purposes. I finish the section with a discussion of portfolio return calculations.

1.2.1 Assets

When I talk about assets in this book, I mean the typical investment assets available for purchase to individuals through brokerage and retirement accounts. These include bonds, stocks, mutual funds, exchange traded funds and notes, and derivative securities. Investment assets can be purchased through online brokerage accounts at firms like Ally Invest, Charles Schwab, E*TRADE, Fidelity, Interactive Brokers, Merrill Lynch, Robinhood, TD Ameritrade, and Vanguard. Due to the growth and competition of online investment accounts, the cost of purchasing investment assets has dropped significantly over time.

Bonds

Bonds are assets that represent debt obligations from corporations or the government used to raise capital (money) to finance operations. A typical bond is characterized by (1) a principal amount, representing initial capital borrowed; (2) maturity date, representing when the principal must be repaid, (3) periodic coupon payments, representing interest paid on specific dates. Bonds issued by

the U.S. government are called *Treasury bonds*, and are generally thought to be the safest bonds as it is very unlikely that the U.S. government will default on its debt obligations. Other government agencies as well as states and municipalities issue bonds to raise money. Corporate bonds are bonds issued by private and public corporations. Bonds other than Treasury bonds have the risk, called *default risk*, that the principal or coupon payments may not be made.

Stocks

A *stock* represents an ownership share in a corporation. Public stock represents ownership in a publicly traded corporation, where the shares can be purchased or sold (traded) by the public in organized stock exchanges (markets). In the U.S. the main stock exchanges are the New York Stock Exchange (NYSE) and the National Association of Securities Dealers Automated Quotations (NASDAQ) exchange. Private stock represents ownership in a private corporation whose stock is not traded in an exchange. Private corporations become public companies when they decide to raise money (capital) by issuing new stock in a public exchange through an *initial public offering* (IPO). This new stock purchased by individuals can then be sold to others in the exchange. Ownership of stock often provides voting rights which allows stock holders to have a say in the management of the company. Owners of stock have a right to receive dividends, which are cash payments representing corporate profits. Corporations may or may not pay dividends. Typically, young growing companies reinvest their profits instead of paying dividends. The stocks of these companies are called *growth stocks*. More established companies more often pay dividends instead of reinvesting all profits.

Mutual Funds

A *mutual fund* is a professionally managed investment fund that pools money from many investors to purchase a collection of assets called a *portfolio*. Mutual funds typically invest in bonds, stocks, and money market instruments. Mutual funds are commonly used in *individual retirement accounts* (IRAs) and employee sponsored retirement accounts. Because mutual funds are managed portfolios they charge fees for this service. The amount of the fee depends on the activity of the portfolio manager. Actively managed funds, where the portfolio manager actively picks the assets in the portfolio, have the highest fees. Actively managed mutual funds are offered by many firms such as the Capital Group, Fidelity, Janice, Putnam, and T. Rowe Price. Passively managed funds have lower fees because the managers follow a rule to match a particular asset index such as the S&P 500 index. Passive mutual funds were pioneered by Vanguard. Mutual funds are traded on exchanges but are only traded at the end of each day.

Exchange Traded Funds and Notes

An *exchange traded fund* (ETF) is similar in many ways to a mutual fund, except that an ETF can be bought and sold throughout the day on stock exchanges. Most ETFs are index funds, that is, they hold the same securities in the same proportions as a certain stock market index or bond market index. The most popular ETFs in the U.S. replicate the S&P 500 Index, the total market

index, the NASDAQ-100 index, the price of gold, the “growth” stocks in the Russell 1000 Index, or the index of the largest technology companies. ETNs are similar to ETFs. The main companies issuing ETFs and ETNs are Barclays Capital Inc. (iPath), BlackRock (iShares), Credit Suisse, Direxion, StateStreet (SPDR), Invesco (PowerShares), and ProShares. Each of these companies provide online information on their ETFs and ETNs. You can easily find ETFs and ETNs for particular asset categories and investment strategies at www.etf.com.

Derivative Securities

A *derivative security* is an asset whose value is derived from the behavior of other underlying assets such as bonds, stocks, ETFs, etc. Common derivative securities are forwards, futures, options and swaps. I will not consider derivative securities in this book.

Information and historical price data on many publicly traded assets is freely available at <https://finance.yahoo.com/>.

1.2.2 Simple returns

Consider purchasing one share of an asset (e.g., stock, bond, ETF, mutual fund, option, etc.) at time t_0 for the price P_{t_0} , and then selling that share at time t_1 for the price P_{t_1} . If there are no intermediate cash flows (e.g., dividends) between t_0 and t_1 , the rate of return over the period t_0 to t_1 is defined as the percentage change in price:

$$R(t_0, t_1) = \frac{P_{t_1} - P_{t_0}}{P_{t_0}}. \quad (1.12)$$

The time between t_0 and t_1 is called the *holding period* and equation Equation 1.12 is called the *holding period return*. In principle, the holding period can be any amount of time: one second, five minutes, eight hours, two days six minutes and two seconds, fifteen years, etc.. To simplify matters, in this chapter and throughout the book I will assume that the holding period is some increment of calendar time; e.g., one day, one month or one year. In particular, I will often assume a default holding period of one month.

Definition 1.4 (Simple net return). Let P_t denote the price at the end of month t of an asset that pays no dividends and let P_{t-1} denote the price at the end of month $t - 1$. Then the one-month *simple net return* on an investment in the asset between months $t - 1$ and t is defined as:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \Delta P_t. \quad (1.13)$$

Writing $(P_t - P_{t-1})/P_{t-1} = P_t/P_{t-1} - 1$, the *simple gross return* is defined as:

$$1 + R_t = \frac{P_t}{P_{t-1}}. \quad (1.14)$$

The one-month gross return has the interpretation of the future value of \$1 invested in the asset for one-month with simple interest rate R_t . Solving Equation 1.14 for P_t gives

$$P_t = P_{t-1}(1 + R_t),$$

which shows P_t can be thought of as the future value of P_{t-1} invested for one month with simple return R_t . Unless otherwise stated, when I refer to returns I mean net returns. Since asset prices must always be non-negative (a long position in an asset is a limited liability investment), the smallest value for R_t is -1 or -100%.

Example 1.6 (Simple return calculation). Consider a one-month investment in Microsoft stock. Suppose you buy the stock in month $t - 1$ (e.g., end of February) at $P_{t-1} = \$85$ and sell the stock the next month (e.g., end of March) for $P_t = \$90$. Further assume that Microsoft does not pay a dividend between months $t - 1$ and t . The one-month simple net and gross returns (over the month of March) are then:

$$R_t = \frac{\$90 - \$85}{\$85} = \frac{\$90}{\$85} - 1 = 1.0588 - 1 = 0.0588,$$

$$1 + R_t = 1.0588.$$

The one-month investment in Microsoft yielded a 5.88% per month return. Alternatively, \$1 invested in Microsoft stock in month $t - 1$ grew to \$1.0588 in month t . The calculations using R are:

```
P0 = 90
Pm1 = 85
R0 = (P0 - Pm1)/Pm1
c(R0, 1 + R0)
#> [1] 0.0588 1.0588
```

■

The return calculation works the same way if multiple shares of an asset are initially purchased. Let $s \geq 1$ denote the number of shares of an asset purchased at time $t - 1$. The initial value of the investment is then $V_{t-1} = s \times P_{t-1}$. The value of the investment at time t is $V_t = s \times P_t$. The rate of return on the investment is the percentage change in value over the holding period:

$$R_t = \frac{V_t - V_{t-1}}{V_{t-1}} = \frac{s \times P_t - s \times P_{t-1}}{s \times P_{t-1}} \quad (1.15)$$

$$= \frac{s \times (P_t - P_{t-1})}{s \times P_{t-1}} = \frac{P_t - P_{t-1}}{P_{t-1}}. \quad (1.16)$$

As a result, the future value of the investment is equal to the current value times the gross return:

$$V_t = V_{t-1} \times (1 + R_t)$$

1.2.2.1 Two-period returns

Definition 1.5 (Two-period return). The *simple two-month return* on an investment in an asset between months $t - 2$ and t is defined as the percentage change in price over two months:

$$R_t(2) = \frac{P_t - P_{t-2}}{P_{t-2}} = \frac{P_t}{P_{t-2}} - 1. \quad (1.17)$$

The *two-month gross return* is defined as:

$$1 + R_t(2) = \frac{P_t}{P_{t-2}}. \quad (1.18)$$

Writing

$$\frac{P_t}{P_{t-2}} = \frac{P_t}{P_{t-1}} \times \frac{P_{t-1}}{P_{t-2}}$$

the two-month return can be expressed as:

$$\begin{aligned} R_t(2) &= \frac{P_t}{P_{t-1}} \cdot \frac{P_{t-1}}{P_{t-2}} - 1 \\ &= (1 + R_t)(1 + R_{t-1}) - 1. \end{aligned}$$

Then the simple two-month gross return becomes:

$$1 + R_t(2) = (1 + R_t)(1 + R_{t-1}) = 1 + R_{t-1} + R_t + R_{t-1}R_t,$$

which is the *product* of the two simple one-month gross returns and not one plus the sum of the two one-month returns. Hence, the two-month net return is

$$R_t(2) = R_{t-1} + R_t + R_{t-1}R_t,$$

which is not the sum of the two one-month returns unless one of the one-month returns is zero. If, however, either R_{t-1} or R_t is close to zero then $R_{t-1}R_t \approx 0$ and $1 + R_t(2) \approx 1 + R_{t-1} + R_t$ so that $R_t(2) \approx R_{t-1} + R_t$.

Example 1.7 (Computing two-period returns). Continuing with the previous example, suppose that the price of Microsoft stock in month $t - 2$ is \$80 and no dividend is paid between months $t - 2$ and t . The two-month net return is:

$$R_t(2) = \frac{\$90 - \$80}{\$80} = \frac{\$90}{\$80} - 1 = 1.1250 - 1 = 0.1250,$$

or 12.50% per two months. The two one-month returns are:

$$R_{t-1} = \frac{\$85 - \$80}{\$80} = 1.0625 - 1 = 0.0625,$$

$$R_t = \frac{\$90 - 85}{\$85} = 1.0588 - 1 = 0.0588,$$

and the geometric average of the two one-month gross returns is: $1 + R_t(2) = 1.0625 \times 1.0588 = 1.1250$.

The calculations can be *vectorized* in R as follows:

```
Pm2 = 80
# create vector of prices
P = c(Pm2, Pm1, P0)
# calculate vector of 1-month returns from vector of prices
R = (P[2:3] - P[1:2])/P[1:2]
R
#> [1] 0.0625 0.0588
# calculate 2-month return from 2 1-month returns
(1 + R[1])*(1 + R[2]) - 1
#> [1] 0.125
# same calculation vectorized using R function cumprod()
cumprod(1+R) - 1
#> [1] 0.0625 0.1250
```

In the above, `P[2:3]` represents the 2nd and 3rd elements of the price vector `P`, and `P[1:2]` represents the 1st and 2nd elements of `P`.

■

It is important to recognize that two-period simple returns are not symmetric in the following sense. Suppose $R_{t-1} = -0.5$ and $R_t = 0.5$ and initial wealth is \$1. After two-periods \$1 falls to

$$\$1 \times (1 + R_{t-1})(1 + R_t) = \$1 \times (1 - 0.5)(1 + 0.5) = \$0.75$$

That is, a 50% drop followed by a 50% gain does not return wealth to its initial level. In the second period, the return has to be 100% in order for wealth to return to its initial level.

1.2.2.2 Multi-period returns

Definition 1.6 (Multi-period return). The k -month simple return on an investment in an asset between months $t - k$ and t is defined as the percentage change in price over k months:

$$R_t(k) = \frac{P_t - P_{t-k}}{P_{t-k}} = \frac{P_t}{P_{t-k}} - 1. \quad (1.19)$$

Writing

$$\frac{P_t}{P_{t-k}} = \frac{P_t}{P_{t-1}} \times \frac{P_{t-1}}{P_{t-2}} \times \cdots \times \frac{P_{t-k+1}}{P_{t-k}} = (1 + R_t)(1 + R_{t-1}) \cdots (1 + R_{t-k+1}),$$

the k -month gross return is defined as the product of k one-month gross returns:

$$1 + R_t(k) = (1 + R_t)(1 + R_{t-1}) \cdots (1 + R_{t-k+1}) = \prod_{j=0}^{k-1} (1 + R_{t-j}). \quad (1.20)$$

If all returns are close to zero then

$$R_t(k) \approx \sum_{j=0}^{k-1} R_{t-j},$$

which is a useful approximation when considering returns over short horizons (e.g., daily returns).

As with the one-month return, the multi-period return can also be derived in the same way if multiple shares of an asset are initially purchased. Using $V_{t-j} = s \times P_{t-j}$ you get

$$\frac{V_t}{V_{t-k}} = \frac{V_t}{V_{t-1}} \times \frac{V_{t-1}}{V_{t-2}} \cdots \frac{V_{t-k+1}}{V_{t-k}} \quad (1.21)$$

$$= \frac{s \times P_t}{s \times P_{t-1}} \times \frac{s \times P_{t-1}}{s \times P_{t-2}} \cdots \frac{s \times P_{t-k+1}}{s \times P_{t-k}} \quad (1.22)$$

$$= (1 + R_t)(1 + R_{t-1}) \cdots (1 + R_{t-k+1}). \quad (1.23)$$

As a result, V_t can be interpreted as the future value of V_{t-k} based on the returns $R_{t-k+1}, \dots, R_t$:

$$V_t = V_{t-k} \times (1 + R_{t-k+1})(1 + R_{t-k+2}) \cdots (1 + R_t).$$

1.2.2.3 Adjusting for dividends

If an asset pays a *dividend* (or any cash payment), D_t , sometime between months $t - 1$ and t , the *total net return* calculation becomes:

$$R_t^{\text{total}} = \frac{P_t + D_t - P_{t-1}}{P_{t-1}} = \frac{P_t - P_{t-1}}{P_{t-1}} + \frac{D_t}{P_{t-1}}, \quad (1.24)$$

where $(P_t - P_{t-1})/P_{t-1}$ is referred as the *capital gain* and D_t/P_{t-1} is referred to as the *dividend yield*. The *total gross return* is:

$$1 + R_t^{\text{total}} = \frac{P_t + D_t}{P_{t-1}}. \quad (1.25)$$

The formula Equation 1.19 for computing multi-period return remains the same except that one-period gross returns are computed using Equation 1.25.

High dividend paying stocks are often called *income stocks* and have characteristics similar to bonds where the regular dividends play a similar role as coupon interest payments.

Example 1.8 (Compute total return when dividends are paid). Consider a one-month investment in Microsoft stock. Suppose you buy the stock in month $t - 1$ at $P_{t-1} = \$85$ and sell the stock the next month for $P_t = \$90$. Further assume that Microsoft pays a \$1 dividend between months $t - 1$ and t . The capital gain, dividend yield and total return are then:

$$\begin{aligned} R_t &= \frac{\$90 + \$1 - \$85}{\$85} = \frac{\$90 - \$85}{\$85} + \frac{\$1}{\$85} \\ &= 0.0588 + 0.0118 \\ &= 0.0707. \end{aligned}$$

The one-month investment in Microsoft yields a 7.07% per month total return. The capital gain component is 5.88% and the dividend yield component is 1.18%

■

1.2.2.4 Adjusting for inflation

The return calculations considered so far are based on the *nominal* or current prices of assets. Returns computed from nominal prices are nominal returns. The *real return* on an asset over a particular horizon takes into account the growth rate of the general price level over the horizon. If the nominal price of the asset grows faster than the general price level, then the nominal return will be greater than the inflation rate and the real return will be positive. Conversely, if the nominal price of the asset increases less than the general price level, then the nominal return will be less than the inflation rate and the real return will be negative. Real returns reflect the increase or decrease in purchasing power from an investment.

The computation of real returns on an asset is a two-step process:

1. Deflate the nominal price of the asset by the general price level
2. Compute returns in the usual way using the deflated prices

To illustrate, consider computing the real simple one-period return on an asset. Let P_t denote the nominal price of the asset at time t and let CPI_t denote an index of the general price level (e.g. consumer price index) at time t ¹. The deflated or real price at time t is:

$$P_t^{\text{Real}} = \frac{P_t}{CPI_t},$$

and the real one-period return is:

$$\begin{aligned} R_t^{\text{Real}} &= \frac{P_t^{\text{Real}} - P_{t-1}^{\text{Real}}}{P_{t-1}^{\text{Real}}} = \frac{\frac{P_t}{CPI_t} - \frac{P_{t-1}}{CPI_{t-1}}}{\frac{P_{t-1}}{CPI_{t-1}}} \\ &= \frac{P_t}{P_{t-1}} \cdot \frac{CPI_{t-1}}{CPI_t} - 1. \end{aligned} \quad (1.26)$$

The one-period gross real return return is:

$$1 + R_t^{\text{Real}} = \frac{P_t}{P_{t-1}} \cdot \frac{CPI_{t-1}}{CPI_t}. \quad (1.27)$$

If you define inflation between periods $t-1$ and t as:

$$\pi_t = \frac{CPI_t - CPI_{t-1}}{CPI_{t-1}} = \% \Delta CPI_t, \quad (1.28)$$

then $1 + \pi_t = CPI_t/CPI_{t-1}$ and Equation 1.26 may be re-expressed as:

$$R_t^{\text{Real}} = \frac{1 + R_t}{1 + \pi_t} - 1. \quad (1.29)$$

Example 1.9 (Compute real return). Consider, again, a one-month investment in Microsoft stock. Suppose the CPI in months $t-1$ and t is 1 and 1.01, respectively, representing a 1% monthly growth rate in the overall price level. The real prices of Microsoft stock are:

$$\begin{aligned} P_{t-1}^{\text{Real}} &= \frac{\$85}{1} = \$85, \\ P_t^{\text{Real}} &= \frac{\$90}{1.01} = \$89.1089, \end{aligned}$$

and the real monthly return is:

$$R_t^{\text{Real}} = \frac{\$89.1089 - \$85}{\$85} = 0.0483.$$

¹The CPI is usually normalized to 1 or 100 in some base year. I assume that the CPI is normalized to 1 in the base year for simplicity.

The nominal return and inflation over the month are:

$$R_t = \frac{\$90 - \$85}{\$85} = 0.0588,$$

$$\pi_t = \frac{1.01 - 1}{1} = 0.01.$$

Then the real return computed using Equation 1.29 is:

$$R_t^{\text{Real}} = \frac{1.0588}{1.01} - 1 = 0.0483.$$

Notice that simple real return is almost, but not quite, equal to the simple nominal return minus the inflation rate: $R_t^{\text{Real}} \approx R_t - \pi_t = 0.0588 - 0.01 = 0.0488$. ■

1.2.2.5 Adjusting for taxes

Any investment that generates a capital gain is potentially subject to taxation by the government. This taxation reduces the investment return. To see how this works, let V_t denote an initial investment amount, let $R_t > 0$ denote the one-period rate of return on the investment, and let τ denote the tax rate on investment income (capital gain). In the U.S. the tax rate on long-term (holding period more than one year) capital gains is either 0%, 15%, or 20% (depending on income) and the tax rate on short-term capital gains (holding period less than one year) is the tax rate on ordinary income (which could be as high as 35%). The future value without taxes is $V_t(1 + R_t)$, and $V_t R_t$ is the capital gain. The amount owed in taxes is $V_t R_t \tau$ and the amount remaining after taxes is $V_t R_t (1 - \tau)$. The after tax return is

$$\begin{aligned} R_t^{\text{Atax}} &= \frac{V_t + V_t R_t (1 - \tau) - V_t}{V_t} \\ &= \frac{V_t (1 + R_t (1 - \tau) - 1)}{V_t} \\ &= R_t (1 - \tau). \end{aligned} \tag{1.30}$$

Hence, the after tax return is the before tax return multiplied by one minus the tax rate.

Example 1.10 (Compute after tax return). Let $V_t = \$100$, $R_t = 0.05$, and $\tau = 0.20$. Then the after tax return is

$$R_t^{\text{Atax}} = (0.05)(1 - 0.20) = (0.05)(0.80) = 0.04.$$

The amount paid in taxes is $\$100 \times (0.05) \times (0.2) = \10 , and the after tax profit is $\$50 - \$10 = \$40$. ■

1.2.2.6 Return on foreign investment

It is common for an investor in one country to buy an asset in another country as an investment. If the two countries have different currencies then the exchange rate between the two currencies influences the return on the asset. To illustrate, consider an investment in a foreign stock (e.g., a stock trading on the London stock exchange) by a U.S. national (domestic investor). The domestic investor takes U.S. dollars, converts them to the foreign currency (e.g., British Pound) via the exchange rate (price of foreign currency in U.S. dollars) and then purchases the foreign stock using the foreign currency. When the stock is sold, the proceeds in the foreign currency must then be converted back to the domestic currency.

To be more precise, consider the information in the table below:

Month	U.S. \$ Cost of 1 Pound £, $E_t^{US/UK}$	Value of British Shares, P_t^{UK}	Value in U.S. \$, P_t^{US}
t - 1	\$ 1.50	£40	$\$1.50 \times £40 = \60
t	\$ 1.30	£45	$\$1.30 \times £45 = \58.5

Let P_t^{US} , P_t^{UK} , and $E_t^{US/UK}$ denote the U.S. dollar price of the asset, British Pound price of the asset, and currency exchange rate between U.S. dollars and British Pounds, at the end of month t , respectively. The U.S. dollar cost of one share of British stock in month $t - 1$ is

$$P_{t-1}^{US} = E_{t-1}^{US/UK} \times P_{t-1}^{UK} = \$1.50 \times 40 = \$60,$$

and the U.S. dollar cost at time t is

$$P_t^{US} = E_t^{US/UK} \times P_t^{UK} = \$1.30 \times £45 = \$58.5.$$

The U.S. dollar rate of return is then

$$R_t^{US} = \frac{P_t^{US} - P_{t-1}^{US}}{P_{t-1}^{US}} = \frac{\$58.5 - \$60}{\$60} = -0.025.$$

Notice that

$$\begin{aligned}
R_t^{US} &= \frac{E_t^{US/UK} \times P_t^{UK} - E_{t-1}^{US/UK} \times P_{t-1}^{UK}}{E_{t-1}^{US/UK} \times P_{t-1}^{UK}} \\
&= \frac{E_t^{US/UK} \times P_t^{UK}}{E_{t-1}^{US/UK} \times P_{t-1}^{UK}} - 1 \\
&= (1 + R_t^{US/UK})(1 + R_t^{UK}) - 1,
\end{aligned} \tag{1.31}$$

where

$$\frac{E_t^{US/UK}}{E_{t-1}^{US/UK}} = 1 + R_t^{US/UK}, \quad \frac{P_t^{UK}}{P_{t-1}^{UK}} = 1 + R_t^{UK}.$$

Hence,

$$1 + R_t^{US} = (1 + R_t^{US/UK})(1 + R_t^{UK}).$$

Here, $R_t^{US/UK}$ is the simple rate of return on foreign currency and R_t^{UK} is the simple rate of return on the British stock in British Pounds:

$$\begin{aligned} R_t^{US/UK} &= \frac{E_t^{US/UK} - E_{t-1}^{US/UK}}{E_{t-1}^{US/UK}} = \frac{\$1.30 - \$1.50}{\$1.50} = -0.133, \\ R_t^{UK} &= \frac{P_t^{UK} - P_{t-1}^{UK}}{P_{t-1}^{UK}} = \frac{\pounds 45 - \pounds 40}{\pounds 40} = 0.125. \end{aligned}$$

Hence, the gross return, $1 + R_t^{US}$, for the U.S. investor has two parts: (1) the gross return from an investment in foreign currency, $1 + R_t^{US/UK}$; and (2) the gross return of the foreign asset in the foreign currency, $1 + R_t^{UK}$:

$$\begin{aligned} 1 + R_t^{US} &= (1 + R_t^{US/UK})(1 + R_t^{UK}) \\ &= (0.867)(1.125) \\ &= 0.975. \end{aligned}$$

In this example, the U.S. investor loses money even though the British investment has a positive rate of return because of the large drop in the exchange rate. This example illustrates the importance of considering *exchange rate risk* when investing in foreign assets.

1.2.2.7 Annualizing returns

Very often returns over different horizons are annualized, i.e., converted to an annual return, to facilitate comparisons with other investments. The annualization process depends on the holding period of the investment and an implicit assumption about compounding. I illustrate with several examples using monthly returns. The same techniques work with different holding periods.

Example 1.11 (Compute annualized return from 12 one-month returns). Suppose you have 12 one-month returns and you want to report an annual return on the investment. Then the annual gross and net returns are computed directly from the 12 one-month returns:

$$\begin{aligned} 1 + R_A &= 1 + R_t(12) = \frac{P_t}{P_{t-12}} = (1 + R_t)(1 + R_{t-1}) \cdots (1 + R_{t-11}), \\ R_A &= R_t(12). \end{aligned}$$

In this case, no compounding is required to create an annual return. ■

Example 1.12 (Compute annualized return from a single one-month return). Suppose you have a one-month investment in an asset with return R_t . What is the annualized return on this investment?

To answer this you must make an assumption about the future returns. For example, if you assume that you receive the same return $R = R_t$ every month for the year, then the gross annual return is:

$$1 + R_A = 1 + R_t(12) = (1 + R)^{12}.$$

That is, the *annual gross return* on the one-month investment is defined as the assumed common monthly return compounded for 12 months. Accordingly, the *annual net return* is then:

$$R_A = (1 + R)^{12} - 1.$$

■

In the above example, it is important to recognize that this annualization calculation assumes the same monthly return R for each of the 12 months. This can be highly inaccurate and misleading if the actual future monthly returns are much different than R . The following examples illustrate.

Example 1.13 (Compute annualized return from one-month return). In the first example, the one-month return, R_t , on Microsoft stock was 5.88%. If you assume that you can get this return for 12 months then the annualized return is: $R_A = (1.0588)^{12} - 1 = 1.9850 - 1 = 0.9850$, or 98.50% per year. Pretty good! However, this is likely to be highly inaccurate since getting 5% every month for a year is highly unlikely for a risky stock like Microsoft.

■

Now, consider a two-month investment with return $R_t(2)$. If you assume that you receive the same two-month return $R(2) = R_t(2)$ for the next six two-month periods, then the gross and net annual returns are:

$$\begin{aligned} 1 + R_A &= (1 + R(2))^6, \\ R_A &= (1 + R(2))^6 - 1. \end{aligned}$$

Here the annual gross return is defined as the two-month return compounded for 6 months.

Example 1.14 (Compute annualized return from two-month return). Suppose the two-month return, $R_t(2)$, on Microsoft stock is 12.5%. If you assume that you can get this two-month return for the next 6 two-month periods then the annualized return is: $R_A = (1.1250)^6 - 1 = 2.0273 - 1 = 1.0273$, or 102.73% per year. As with the previous example, this result is likely to be inaccurate as it is highly unlikely to get 12.5% every two months for the rest of the year.

■

Example 1.15 (Compute annualized return from 6 two-month returns). Suppose your investment horizon is two years. That is, you start your investment at month $t - 24$ and cash out at month t . The two-year gross return is then $1 + R_t(24) = P_t/P_{t-24}$. What is the annual return on this two-year investment? The process is the same as computing the effective annual rate. To determine the annual return you solve the following relationship for R_A :

$$(1 + R_A)^2 = 1 + R_t(24) \Rightarrow \\ R_A = (1 + R_t(24))^{1/2} - 1.$$

In this case, the annual return is compounded twice to get the two-year return and the relationship is then solved for the annual return. ■

Example 1.16 (Compute annualized return from two-year return). Suppose that the price of Microsoft stock 24 months ago is $P_{t-24} = \$50$ and the price today is $P_t = \$90$. The two-year gross return is $1 + R_t(24) = \$90/\$50 = 1.8000$ which yields a two-year net return of $R_t(24) = 0.80 = 80\%$. The annual return for this investment is defined as:

$$R_A = (1.800)^{1/2} - 1 = 1.3416 - 1 = 0.3416$$

or 34.16% per year. ■

1.2.2.8 Average returns

For investments over a given horizon, it is often of interest to compute a measure of the average rate of return over the horizon. To illustrate, consider a sequence of monthly investments over n months with monthly returns $R_1, R_2, \dots, R_n$. The n -month return is

$$R(n) = (1 + R_1)(1 + R_2) \cdots (1 + R_n) - 1.$$

What is the average monthly return? There are two possibilities. The first is the *arithmetic average return*:

$$\bar{R}^{Arith} = \frac{1}{n} (R_1 + R_2 + \cdots + R_n). \quad (1.32)$$

The second is the *geometric average return*:

$$\bar{R}^{Geo} = [(1 + R_1)(1 + R_2) \cdots (1 + R_n)]^{1/n} - 1. \quad (1.33)$$

Notice that the geometric average return is the monthly return which compounded monthly for n months gives the gross n -month return

$$(1 + \bar{R}^{Geo})^n = 1 + R(n).$$

As a measure of average investment performance, the geometric average return is preferred to the arithmetic average return. To see why, consider a two-month investment horizon with monthly returns $R_1 = 0.5$ and $R_2 = -0.5$. One dollar invested over these two months grows to

$$(1 + R_1)(1 + R_2) = (1.5)(0.5) = 0.75,$$

for a two-period return of $R(2) = 0.75 - 1 = -0.25$. Hence, the two-month investment loses 25%. The arithmetic average return is

$$\bar{R}^{Arith} = \frac{1}{2}(0.5 + (-0.5)) = 0.$$

As a measure of investment performance, this is misleading because the investment loses money over the two-month horizon. The average return indicates the investment breaks even. The geometric average return is

$$\bar{R}^{Geo} = [(1.5)(0.5)]^{1/2} - 1 = -0.134.$$

This is a more accurate measure of investment performance because it indicates that the investment eventually loses money. In addition, the compound two-month return using $\bar{R}^{Geo} = -0.134$ is equal to $R(2) = -0.25$.

While the arithmetic average return may not be the most appropriate measure of actual average investment performance, you shall see in the following chapters that it is a useful estimate of expected performance.

1.2.3 Continuously compounded returns

In this section I define continuously compounded returns from simple returns, and describe their properties.

1.2.3.1 One-period returns

Definition 1.7 (Continuously compounded return). Let R_t denote the simple monthly return on an investment. The *continuously compounded monthly return*, r_t , is defined as:

$$r_t = \ln(1 + R_t) = \ln\left(\frac{P_t}{P_{t-1}}\right), \quad (1.34)$$

where $\ln(\cdot)$ is the natural log function².

To see why r_t is called the continuously compounded return, take the exponential of both sides of Equation 1.34 to give:

$$e^{r_t} = 1 + R_t = \frac{P_t}{P_{t-1}}.$$

Rearranging and solving for P_t gives

$$P_t = P_{t-1}e^{r_t} = P_{t-1}(1 + R_t),$$

so that r_t is the continuously compounded growth rate in prices between months $t - 1$ and t . This is to be contrasted with R_t , which is the simple growth rate in prices between months $t - 1$ and t without any compounding. As a result, r_t is always smaller than R_t .

Using the result $\ln(x/y) = \ln(x) - \ln(y)$ you can alternatively compute the continuously compounded return as:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(P_t) - \ln(P_{t-1}) = p_t - p_{t-1}, \quad (1.35)$$

where $p_t = \ln(P_t)$. Hence, the continuously compounded monthly return, r_t , can be computed simply by taking the first difference of the natural logarithms of monthly prices.

Although r_t is always smaller than R_t , it is close to R_t when R_t is small. This is stated in the following proposition. `:: {#prp-ccReturnApprox name="Continuously compounded return approximation"}` Suppose the simple return R_t is close to zero ($R_t \approx 0$). Then

$$r_t = \ln(1 + R_t) \approx R_t$$

. `::` The above result can be derived using a first order Taylor series approximation to the function $f(x) = \ln(1 + x)$. Recall, for a continuous and differentiable function $f(x)$ the *first order Taylor series approximation* of $f(x)$ about the point $x = x_0$ is

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \text{remainder}, \quad (1.36)$$

where $f'(x_0)$ denotes the derivative of $f(x)$ evaluated at x_0 . Let $f(x) = \ln(1 + x)$ and set $x_0 = 0$. Then

Using the first order Taylor series approximation Equation 1.36, the remainder term is small when $x \approx 0$ and $\ln(1 + x) \approx x$ when $x \approx 0$.

Example 1.17 (Compute continuously compounded returns). Using the price and return data from Example 1.6, the continuously compounded monthly return on Microsoft stock can be computed

²The continuously compounded return is always defined since asset prices, P_t , are always non-negative. Properties of logarithms and exponentials are discussed in the appendix to this chapter.

in two ways:

$$\begin{aligned} r_t &= \ln(1.0588) = 0.0571 \\ r_t &= \ln(90) - \ln(85) = 4.4998 - 4.4427 = 0.0571 \end{aligned}$$

Notice that r_t is slightly smaller than R_t . Using R, the calculations are:

```
c(log(1 + R0), log(P0) - log(Pm1))
#> [1] 0.0572 0.0572
```

Notice that the R function `log()` computes the natural logarithm $\ln(x)$.

■

Given a monthly continuously compounded return r_t , it is straightforward to solve back for the corresponding simple net return R_t :

$$R_t = e^{r_t} - 1. \quad (1.37)$$

Hence, nothing is lost by considering continuously compounded returns instead of simple returns. Continuously compounded returns are very similar to simple returns as long as the return is relatively small, which it generally will be for monthly or daily returns. Since R_t is bounded from below by -1, the smallest value for r_t is $r_t = \ln(1 - 1) = -\infty$. This, however, does not mean that you could lose an infinite amount of money on an investment. The actual amount of money lost is determined by the simple return Equation 1.37.

Example 1.18 (Determine simple returns from continuously compounded returns). In the previous example, the continuously compounded monthly return on Microsoft stock is $r_t = 5.71\%$. The simple net return is then: $R_t = e^{0.0571} - 1 = 0.0588$.

■

For asset return modeling and statistical analysis it is often more convenient to use continuously compounded returns than simple returns. One reason is that continuously compounded returns take values on the entire real line, $-\infty < r_t < \infty$, whereas simple returns are only defined for values larger than -1, $-1 \leq R_t < \infty$. Another reason is due to the additivity property of multi-period continuously compounded returns discussed in the next sub-section.

1.2.3.2 Multi-period returns

The relationship between multi-period continuously compounded returns and one-period continuously compounded returns is more simple than the relationship between multi-period simple returns

and one-period simple returns. To illustrate, consider the two-month continuously compounded return defined as:

$$r_t(2) = \ln(1 + R_t(2)) = \ln\left(\frac{P_t}{P_{t-2}}\right) = p_t - p_{t-2}.$$

Taking exponentials of both sides and rearranging gives:

$$P_t = P_{t-2}e^{r_t(2)}$$

so that $r_t(2)$ is the continuously compounded growth rate of prices between months $t - 2$ and t . Using $P_t/P_{t-2} = (P_t/P_{t-1}) \cdot (P_{t-1}/P_{t-2})$ and the fact that $\ln(x \cdot y) = \ln(x) + \ln(y)$ it follows that:

$$\begin{aligned} r_t(2) &= \ln\left(\frac{P_t}{P_{t-1}} \cdot \frac{P_{t-1}}{P_{t-2}}\right) \\ &= \ln\left(\frac{P_t}{P_{t-1}}\right) + \ln\left(\frac{P_{t-1}}{P_{t-2}}\right) \\ &= r_t + r_{t-1}. \end{aligned}$$

Hence the continuously compounded two-month return is just the sum of the two continuously compounded one-month returns. Recall, with simple returns the two-month return is a multiplicative (geometric) sum of two one-month returns.

Example 1.19 (Compute two-month continuously compounded returns). Using the data from Example 1.7, the continuously compounded two-month return on Microsoft stock can be computed in two equivalent ways. The first way uses the difference in the logs of P_t and P_{t-2} :

$$r_t(2) = \ln(90) - \ln(80) = 4.4998 - 4.3820 = 0.1178.$$

The second way uses the sum of the two continuously compounded one-month returns. Here $r_t = \ln(90) - \ln(85) = 0.0571$ and $r_{t-1} = \ln(85) - \ln(80) = 0.0607$ so that:

$$r_t(2) = 0.0571 + 0.0607 = 0.1178.$$

Notice that $r_t(2) = 0.1178 < R_t(2) = 0.1250$. The vectorized calculations in R are:

```
p = log(P)
r = log(1 + R)
c(r[1] + r[2], sum(r), p[3] - p[1], diff(p, 2))
#> [1] 0.118 0.118 0.118 0.118
```

■

Previously, you saw that simple two-period returns were not symmetric. Continuously compounded

two-period returns, however, are symmetric. To see this, suppose $r_{t-1} = -0.5$ and $r_t = 0.5$. Notice that the implied simple returns are not symmetric: $R_{t-1} = e^{-0.5} - 1 = -0.3935$ and $R_t = e^{0.5} - 1 = 0.6487$. Then one dollar invested over two periods becomes:

$$\$1 \times (1 - 0.3935)(1 + 0.6487) = \$1$$

The continuously compounded k -month return is defined by:

$$r_t(k) = \ln(1 + R_t(k)) = \ln\left(\frac{P_t}{P_{t-k}}\right) = p_t - p_{t-k}.$$

Using similar manipulations to the ones used for the continuously compounded two-month return, you can express the continuously compounded k -month return as the sum of k continuously compounded monthly returns:

$$r_t(k) = \sum_{j=0}^{k-1} r_{t-j}. \quad (1.38)$$

The additivity of continuously compounded returns to form multiperiod returns is an important property for statistical modeling purposes.

1.2.3.3 Adjusting for dividends

The continuously compounded one-period return adjusted for dividends is defined by Equation 1.34, where R_t is computed using Equation 1.24³.

Example 1.20 (Compute continuously compounded total return). From example Example 1.8, the total simple return using Equation 1.24 is $R_t = 0.0707$. The continuously compounded total return is then:

$$r_t = \ln(1 + R_t) = \ln(1.0707) = 0.0683.$$

■

1.2.3.4 Adjusting for inflation

Adjusting continuously compounded nominal returns for inflation is particularly simple. The continuously compounded one-period real return is defined as:

$$r_t^{\text{Real}} = \ln(1 + R_t^{\text{Real}}). \quad (1.39)$$

³Show formula from Campbell, Lo and MacKinlay.

Using Equation 1.27, it follows that:

$$\begin{aligned}
r_t^{\text{Real}} &= \ln \left(\frac{P_t}{P_{t-1}} \cdot \frac{CPI_{t-1}}{CPI_t} \right) \\
&= \ln \left(\frac{P_t}{P_{t-1}} \right) + \ln \left(\frac{CPI_{t-1}}{CPI_t} \right) \\
&= \ln(P_t) - \ln(P_{t-1}) + \ln(CPI_{t-1}) - \ln(CPI_t) \\
&= \ln(P_t) - \ln(P_{t-1}) - (\ln(CPI_t) - \ln(CPI_{t-1})) \\
&= r_t - \pi_t^c,
\end{aligned} \tag{1.40}$$

where $r_t = \ln(P_t) - \ln(P_{t-1}) = \ln(1 + R_t)$ is the nominal continuously compounded one-period return and $\pi_t^c = \ln(CPI_t) - \ln(CPI_{t-1}) = \ln(1 + \pi_t)$ is the one-period continuously compounded growth rate in the general price level (continuously compounded one-period inflation rate). Hence, the real continuously compounded return is simply the nominal continuously compounded return minus the the continuously compounded inflation rate.

Example 1.21 (Compute continuously compounded real returns). From Example Example 1.9, the nominal simple return is $R_t = 0.0588$, the monthly inflation rate is $\pi_t = 0.01$, and the real simple return is $R_t^{\text{Real}} = 0.0483$. Using Equation 1.39, the real continuously compounded return is:

$$r_t^{\text{Real}} = \ln(1 + R_t^{\text{Real}}) = \ln(1.0483) = 0.047.$$

Equivalently, using (Equation 1.40) the real return may also be computed as:

$$r_t^{\text{Real}} = r_t - \pi_t^c = \ln(1.0588) - \ln(1.01) = 0.047.$$

■

1.2.3.5 Adjusting for taxes

Using Equation 1.30, the continuously compounded return adjusted for taxes is

$$r_t^{\text{Atax}} = \ln(R_t(1 - \tau)) = \ln(R_t) + \ln(1 - \tau) = r_t + \ln(1 - \tau) \tag{1.41}$$

1.2.3.6 Return on a foreign investment

Consider again, the US/UK investment example from sub-section Section 1.2.2.6. Using Equation 1.31, the continuously compounded US return is

$$\begin{aligned} r_t^{US} &= \ln \left((1 + R_t^{US/UK})(1 + R_t^{UK}) \right) \\ &= \ln(1 + R_t^{US/UK}) + \ln(1 + R_t^{UK}) \\ &= r_t^{US/UK} + r_t^{UK} \end{aligned} \quad (1.42)$$

where $r_t^{US/UK} = \ln(1 + R_t^{US/UK})$ is the continuously compounded currency return and $r_t^{UK} = \ln(1 + R_t^{UK})$ is the continuously compounded UK return.

1.2.3.7 Annualizing continuously compounded returns

Just as you annualized simple monthly returns, you can also annualize continuously compounded monthly returns. I illustrate using monthly continuously compounded returns. First, if your investment horizon is one year and you observe 12 monthly continuously compounded returns then the annual continuously compounded return is just the sum of the twelve monthly continuously compounded returns:

$$\begin{aligned} r_A &= r_t(12) = r_t + r_{t-1} + \cdots + r_{t-11} \\ &= \sum_{j=0}^{11} r_{t-j}. \end{aligned}$$

The average continuously compounded monthly return is defined as:

$$\bar{r}_m = \frac{1}{12} \sum_{j=0}^{11} r_{t-j}.$$

Notice that:

$$12 \cdot \bar{r}_m = \sum_{j=0}^{11} r_{t-j}$$

so that you may alternatively express r_A as:

$$r_A = 12 \cdot \bar{r}_m.$$

That is, the continuously compounded annual return is twelve times the average of the continuously compounded monthly returns.

Next, consider a one-month investment in an asset with continuously compounded return $r_t = r$. If you assume that you receive the same return r every month for the year, then the annual

continuously compounded return is just 12 times the monthly continuously compounded return:

$$r_A = r_t(12) = r + r + \dots + r = 12 \cdot r.$$

If you have a two-month continuously compounded return of $r(2)$, and you assume that you get this same return every two months for the next year, then the annualized continuously compounded return is

$$r_A = 6 \cdot r(2).$$

Finally, suppose you have a 2-year continuously compounded return $r(24)$. Then the continuously compounded annual return can be defined using

$$2 \cdot r_A = r(24) \implies r_A = \frac{r(24)}{2}.$$

1.3 Portfolios and Portfolio Returns

Consider an investment in two assets named asset A and asset B (e.g. Amazon and Boeing stock). Let V_A and V_B denote the dollar amounts invested in assets A and B , respectively. you call such an investment in multiple assets a *portfolio*. The total dollar amount invested in the portfolio is $V_p = V_A + V_B$. The shares of dollars invested in assets A and B are:

$$x_A = \frac{V_A}{V_p}, x_B = \frac{V_B}{V_p}.$$

Notice that $x_A + x_B = 1$ by construction (100% of all wealth is invested in the two assets).

In this book, the collection of investment shares (x_A, x_B) and initial wealth invested V_p defines a portfolio. If V_p is not specified then assume that $V_p = \$1$. For example, one portfolio may be $(x_A = 0.5, x_B = 0.5)$ (equally weighted portfolio) and another may be $(x_A = 0, x_B = 1)$ (everything invested in asset B). Negative values for x_A or x_B are possible and represent short sales (to be discussed in later chapters). Given x_A , x_B , and V_p , the dollar amounts invested in assets A and B are:

$$V_A = x_A \times V_p, V_B = x_B \times V_p.$$

The rate of return on a portfolio can be represented as a share weighted average of the rates of returns on the assets in the portfolio. To see this, let R_A and R_B denote the simple one-period returns on assets A and B . You want to determine the simple one-period return on the portfolio defined by (x_A, x_B) and initial wealth V_p . To do this, note that at the end of the holding period, the investments in assets A and B are worth $V_p \times x_A(1 + R_A)$ and $V_p \times x_B(1 + R_B)$, respectively.

Hence, at the end of the holding period the portfolio is worth:

$$V_p \times [x_A(1 + R_A) + x_B(1 + R_B)] = V_p \times (1 + R_p).$$

Hence, $x_A(1 + R_A) + x_B(1 + R_B) = 1 + R_p$ defines the *portfolio gross return*. The portfolio gross return is equal to a weighted average of the gross returns on assets A and B , where the weights are the portfolio shares x_A and x_B .

To determine the portfolio rate of return, re-write the portfolio gross return as:

$$1 + R_p = x_A + x_B + x_A R_A + x_B R_B = 1 + x_A R_A + x_B R_B,$$

since $x_A + x_B = 1$ by construction. Then the *portfolio rate of return* is:

$$R_p = x_A R_A + x_B R_B,$$

which is equal to a weighted average of the simple returns on assets A and B , where the weights are the portfolio shares x_A and x_B .

In the portfolio rate of return, the components $x_A R_A$ and $x_B R_B$ are the *contributions* of assets A and B to the portfolio return, respectively. These contributions add up to the total portfolio return R_p .

Example 1.22 (Compute portfolio return). Consider a portfolio of Microsoft and Starbucks stock in which you initially purchase ten shares of each stock at the end of month $t - 1$ at the prices $P_{\text{msft}, t-1} = \$85$ and $P_{\text{sbux}, t-1} = \$30$, respectively. The initial value of the portfolio is $V_{t-1} = 10 \times \$85 + 10 \times \$30 = \$1,150$. The portfolio shares are

$$x_{\text{msft}} = \frac{\$850}{\$1,150} = 0.7391, \quad x_{\text{sbux}} = \frac{\$300}{\$1,150} = 0.2609.$$

Suppose at the end of month t , $P_{\text{msft}, t} = \$90$ and $P_{\text{sbux}, t} = \$28$. Assuming that Microsoft and Starbucks do not pay a dividend between periods $t - 1$ and t , the one-period returns on the two stocks are:

$$R_{\text{msft}, t} = \frac{\$90 - \$85}{\$85} = 0.0588, \quad R_{\text{sbux}, t} = \frac{\$28 - \$30}{\$30} = -0.0667.$$

The one-month rate of return on the portfolio is then:

$$R_{p, t} = (0.7391)(0.0588) + (0.2609)(-0.0667) = 0.02609,$$

The contributions of assets A and B to the portfolio return are:

$$x_A R_A = (0.7391)(0.0588) = 0.0435, \quad x_B R_B = (0.2609)(-0.0667) = -0.0174.$$

Asset A contributes positively and asset B contributes negatively to the portfolio return. The portfolio value at the end of month t is:

$$V_t = V_{t-1}(1 + R_{p,t}) = \$1,150 \times (1.02609) = \$1,180$$

In R, the portfolio calculations are:

```
P.msft = c(85, 90)
P.sbox = c(30, 28)
V = P.msft[1]*10 + P.sbox[1]*10
x.msft = 10*P.msft[1]/V
x.sbox = 10*P.sbox[1]/V
R.msft = (P.msft[2] - P.msft[1])/P.msft[1]
R.sbox = (P.sbox[2] - P.sbox[1])/P.sbox[1]
R.p = x.msft*R.msft + x.sbox*R.sbox
V1 = V*(1 + R.p)
c(R.p, x.msft*R.msft, x.sbox*R.sbox, V1)
#> [1] 0.0261 0.0435 -0.0174 1180.0000
```

■

In general, for a portfolio of n assets with investment shares x_i such that $x_1 + \dots + x_n = 1$, the one-period portfolio gross and simple returns are defined as:

$$1 + R_{p,t} = \sum_{i=1}^n x_i(1 + R_{i,t}) \quad (1.43)$$

$$R_{p,t} = \sum_{i=1}^n x_i R_{i,t}. \quad (1.44)$$

where $R_{i,t}$ is the one-period simple return on asset i . Asset i 's contribution to the portfolio return is

$$\text{ctr}_{i,p} = x_i R_{i,t}. \quad (1.45)$$

and

$$\sum_{i=1}^n \text{ctr}_{i,p} = \sum_{i=1}^n x_i R_{i,t} = R_{p,t}.$$

1.3.1 Multiperiod portfolio returns and rebalancing

How to compute multiperiod portfolio returns depends on the assumptions made about the portfolio weights each period. Several scenarios exist:

1. Portfolio weights from the initial portfolio are allowed to change over time as prices of the underlying assets change over time. In this case no rebalancing of the portfolio is done. This is called a *buy-and-hold* portfolio.
2. Portfolio weights remain constant over time, which implies that the portfolio is rebalanced at every time period (associated with holding period) to maintain constant weights.
3. Portfolio weights from the initial portfolio are rebalanced at specific time intervals that are different than the holding period (e.g. rebalance quarterly for monthly holding periods). This is a hybrid of scenarios 1 and 2.
4. Portfolio weights are actively changed at each time period associated with the holding period. This is called an *actively managed portfolio*.

How to deal with the above scenarios is best illustrated through examples.

1.3.1.1 No rebalancing of portfolio weights

Prices and shares framework

I illustrate the buy-and-hold scenario 1 using a simple portfolio consisting of two assets A and B invested for three months starting at time $t - 3$ in a framework where I know the asset prices each period and the initial amounts invested in each asset⁴. In particular, I assume that the initial portfolio consists of $s_A = 100$ shares of asset A and $s_B = 50$ shares of asset B . The end-of-month prices and values ($V_{i,t} = s_i \times P_{i,t}, i = A, B$) of the assets as well as the end-of-month values of the portfolio are given by

```
s.A = 100
s.B = 50
P.A = c(5,7,6,7)
V.A = s.A*P.A
P.B = c(10, 11, 12, 8)
V.B = s.B*P.B
V.p = V.A + V.B
table.vals = cbind(P.A, V.A, P.B, V.B, V.p)
rownames(table.vals) = c("t-3", "t-2", "t-1", "t")
table.vals
#>      P.A V.A P.B V.B V.p
#> t-3    5 500  10 500 1000
#> t-2    7 700  11 550 1250
#> t-1    6 600  12 600 1200
#> t      7 700   8 400 1100
```

⁴This example is inspired by the `return.portfolio` vignette in **PerformanceAnalytics**).

In the above table the time index t represents the end-of-month t . Slightly abusing notation, I can also say that the time index $t - 1$ represents the beginning of month t . This is important because for the portfolio calculations the end-of-month values $V_{A,t-1}$, $V_{B,t-1}$ and $V_{p,t-1}$ are used to calculate the portfolio weights to be applied in month t . That is,

$$x_{A,t-1} = \frac{V_{A,t-1}}{V_{p,t-1}}, \quad x_{B,t-1} = \frac{V_{B,t-1}}{V_{p,t-1}}$$

represent the portfolio weights for assets A and B at the beginning of month t . Then, the portfolio return over month t is computed as

$$R_{p,t} = \% \Delta V_{p,t} = \frac{V_{p,t} - V_{p,t-1}}{V_{p,t-1}} \quad (1.46)$$

$$= x_{A,t-1} R_{A,t} + x_{B,t-1} R_{B,t}, \quad (1.47)$$

$$(1.48)$$

where

$$R_{A,t} = \% \Delta V_{A,t} = \frac{V_{A,t} - V_{A,t-1}}{V_{A,t-1}} \quad (1.49)$$

$$= \% \Delta P_{A,t} = \frac{P_{A,t} - P_{A,t-1}}{P_{A,t-1}} \quad (1.50)$$

$$R_{B,t} = \% \Delta V_{B,t} = \frac{V_{B,t} - V_{B,t-1}}{V_{B,t-1}} \quad (1.51)$$

$$= \% \Delta P_{B,t} = \frac{P_{B,t} - P_{B,t-1}}{P_{B,t-1}}. \quad (1.52)$$

The portfolio return, $R_{p,t}$, is still a share weighted average of the individual asset returns but with time varying portfolio weights $x_{A,t-1}$ and $x_{B,t-1}$.

The R calculations for returns are:

```
R.A = (V.A[2:4] - V.A[1:3])/V.A[1:3]
R.B = (V.B[2:4] - V.B[1:3])/V.B[1:3]
R.p = (V.p[2:4] - V.p[1:3])/V.p[1:3]
table.returns = cbind(R.A, R.B, R.p)
rownames(table.returns) = c("t-2", "t-1", "t")
table.returns
#>      R.A      R.B      R.p
#> t-2  0.400  0.1000  0.2500
#> t-1 -0.143  0.0909 -0.0400
#> t    0.167 -0.3333 -0.0833
```

The R calculations for the weights are:

```
x.A = V.A/V.p
x.B = V.B/V.p
table.weights = cbind(x.A, x.B)
rownames(table.weights) = c("t-3", "t-2", "t-1", "t")
table.weights
#>      x.A    x.B
#> t-3 0.500 0.500
#> t-2 0.560 0.440
#> t-1 0.500 0.500
#> t    0.636 0.364
```

Here, the initial portfolio formed at the end of month $t-3$ is an equally weighted portfolio of assets A and B . However, the portfolio weights at the end of month $t-2$ are not 0.5 and 0.5 as the prices of assets A and B have changed over the month. Asset A 's price has gone up more in percentage terms ($R_{A,t-2} = 40\%$) than asset B 's price ($R_{B,t-2} = 10\%$) so the portfolio weight in asset A at the beginning of $t-1$ ($x_{A,t-2} = 0.56$), has become bigger than the portfolio weight in asset B ($x_{B,t-2} = 0.44$).

For the calculation of the portfolio returns, the asset weights at time $t-3$ apply to the asset returns at time $t-2$, the weights at time $t-2$ apply to the returns at time $t-1$, and the weights at time $t-1$ apply to the returns at time t :

```
R.p = x.A[1:3]*R.A + x.B[1:3]*R.B
R.p
#> [1] 0.2500 -0.0400 -0.0833
```

The asset contributions to the portfolio return at time t are

$$\text{ctr}_{A,t} = \frac{\Delta V_{A,t}}{V_{p,t-1}} = x_{A,t-1} \times R_{A,t} \quad (1.53)$$

$$\text{ctr}_{B,t} = \frac{\Delta V_{B,t}}{V_{p,t-1}} = x_{B,t-1} \times R_{B,t} \quad (1.54)$$

The R calculations for the contributions derived from values are:

```
ctr.A = (V.A[2:4] - V.A[1:3])/V.p[1:3]
ctr.B = (V.B[2:4] - V.B[1:3])/V.p[1:3]
table.ctr = cbind(ctr.A, ctr.B, ctr.A+ctr.B)
colnames(table.ctr)[3] = "R.p"
rownames(table.ctr) = c("t-2", "t-1", "t")
```

```
table.ctr
#>      ctr.A  ctr.B    R.p
#> t-2  0.2000  0.050  0.2500
#> t-1 -0.0800  0.040 -0.0400
#> t    0.0833 -0.167 -0.0833
```

The R calculations for the contributions derived from returns are:

```
ctr.A = x.A[1:3]*R.A
ctr.B = x.B[1:3]*R.B
table.ctr = cbind(ctr.A, ctr.B, R.p)
rownames(table.ctr) = c("t-2", "t-1", "t")
table.ctr
#>      ctr.A  ctr.B    R.p
#> t-2  0.2000  0.050  0.2500
#> t-1 -0.0800  0.040 -0.0400
#> t    0.0833 -0.167 -0.0833
```

Returns and weights framework

The above calculations are based on knowing asset prices and initial shares purchased for each asset. In many situations, you only have return information and initial portfolio weights on each asset and the initial value of the portfolio. Using the above example, the initial portfolio weights are $x_{A,t-3} = x_{B,t-3} = 0.5$ and the initial value of the portfolio is $V_{p,t-3} = 1000$. You also know the returns, $R_{i,t}$, each period for the two assets:

```
table.returns[, c("R.A", "R.B")]
#>      R.A    R.B
#> t-2  0.400  0.1000
#> t-1 -0.143  0.0909
#> t    0.167 -0.3333
```

You can deduce the initial asset values using

$$V_{A,t-3} = x_{A,t-3} \times V_{p,t-3} = 0.5 \times 1000 = 500, \quad (1.55)$$

$$V_{B,t-3} = x_{B,t-3} \times V_{p,t-3} = 0.5 \times 1000 = 500. \quad (1.56)$$

The end-of-month asset values for $i = A, B$ are computed using

$$\begin{aligned} V_{i,t-2} &= V_{i,t-3} \times (1 + R_{i,t-2}), \\ V_{i,t-1} &= V_{i,t-2} \times (1 + R_{i,t-1}) = V_{i,t-3} \times (1 + R_{i,t-2})(1 + R_{i,t-1}), \\ V_{i,t} &= V_{i,t-1} \times (1 + R_{i,t}) = V_{i,t-3} \times (1 + R_{i,t-2})(1 + R_{i,t-1})(1 + R_{i,t}). \end{aligned}$$

The end-of-month portfolio values are the sum of the end-of-month asset values. The R calculations are:

```
V.A = V.B = V.p = rep(0, 4)
V.p[1] = 1000
V.A[1] = V.B[1] = 0.5*V.p[1]
V.A[2:4] = cumprod(1+R.A)*V.A[1]
V.B[2:4] = cumprod(1+R.B)*V.B[1]
V.p[2:4] = V.A[2:4] + V.B[2:4]
table.vals = cbind(V.A, V.B, V.p)
rownames(table.vals) = c("t-3", "t-2", "t-1", "t")
table.vals
#>      V.A V.B  V.p
#> t-3  500  500 1000
#> t-2  700  550 1250
#> t-1  600  600 1200
#> t    700  400 1100
```

The portfolio weights at the end-of-month t are computed as

$$x_{i,t} = \frac{V_{i,t}}{V_{p,t}},$$

and the R calculations are:

```
x.A = V.A/V.p
x.B = V.B/V.p
table.weights = cbind(x.A, x.B)
rownames(table.weights) = c("t-3", "t-2", "t-1", "t")
table.weights
#>      x.A  x.B
#> t-3 0.500 0.500
#> t-2 0.560 0.440
#> t-1 0.500 0.500
```

```
#> t      0.636 0.364
```

The portfolio return over month t can be computed using

$$R_{p,t} = \% \Delta V_{p,t} = \frac{V_{p,t} - V_{p,t-1}}{V_{p,t-1}} \quad (1.57)$$

$$= x_{A,t-1} \times R_{A,t} + x_{B,t-1} \times R_{B,t} \quad (1.58)$$

The R calculations for returns based on values are:

```
diff(table.vals[, "V.p"])/table.vals[1:3, "V.p"]
#>      t-2      t-1      t
#> 0.2500 -0.0400 -0.0833
```

The calculation for returns based on weights is

```
x.A[1:3]*R.A + x.B[1:3]*R.B
#> [1] 0.2500 -0.0400 -0.0833
```

1.3.1.2 Constant portfolio weights

If the initial portfolio weights are to be held constant over time then the portfolio typically needs to be *rebalanced* at each time period to adjust the portfolio weights back to the initial weights. Given the constant portfolio weights x_A and x_B the return on the portfolio at every time period is given by

$$R_{p,t} = x_A R_{A,t} + x_B R_{B,t}.$$

The rebalancing of portfolio weights can be illustrated with the example from the previous subsection. The initial portfolio is an equally weighted portfolio, $x_{A,t-3} = x_{B,t-3} = 0.5$. To have constant portfolio weights the portfolio will need to be rebalanced each period so that $x_{A,t-2} = x_{A,t-1} = x_{A,t} = 0.5$ and $x_{B,t-2} = x_{B,t-1} = x_{B,t} = 0.5$. Consider the portfolio at the end-of-month $t - 2$ where

$$V_{A,t-2} = V_{A,t-3} \times (1 + R_{A,t-2}) = 500 \times (1 + 0.40) = 700 \quad (1.59)$$

$$V_{B,t-2} = V_{B,t-3} \times (1 + R_{B,t-2}) = 500 \times (1 + 0.10) = 550 \quad (1.60)$$

$$V_{p,t-2} = V_{A,t-2} + V_{B,t-2} = 700 + 550 = 1250 \quad (1.61)$$

$$x_{A,t-2} = \frac{V_{A,t-2}}{V_{p,t-2}} = \frac{700}{1250} = 0.56 \quad (1.62)$$

$$x_{B,t-2} = \frac{V_{B,t-2}}{V_{p,t-2}} = \frac{550}{1250} = 0.44 \quad (1.63)$$

The end-of-month portfolio weights are $x_{A,t-2} = 0.56$ and $x_{B,t-2} = 0.44$, which are not equal to 0.5. Since $V_{p,t-2} = 1250$, to rebalance the portfolio so that $x_{A,t-2} = x_{B,t-2} = 0.5$ you need $V_{A,t-2} = V_{B,t-2} = V_{p,t-2}/2 = 625$. This requires selling $700 - 625 = 75$ of asset A and purchasing 75 of asset B at the end-of-month $t-2$. Then, at the beginning of month $t-1$ you have $V_{A,t-2} = V_{B,t-2} = 625$ and $x_{A,t-2} = x_{B,t-2} = 0.5$.

At the end-of-month $t-1$, for the rebalanced portfolio you have

$$V_{A,t-1} = V_{A,t-2} \times (1 + R_{A,t-1}) = 625 \times (1 - 0.1429) = 535.69 \quad (1.64)$$

$$V_{B,t-1} = V_{B,t-2} \times (1 + R_{B,t-1}) = 625 \times (1 + 0.0909) = 681.81 \quad (1.65)$$

$$V_{p,t-1} = V_{A,t-2} + V_{B,t-2} = 535.69 + 681.81 = 1217.50 \quad (1.66)$$

$$x_{A,t-1} = \frac{V_{A,t-1}}{V_{p,t-1}} = \frac{535.69}{1217.50} = 0.44 \quad (1.67)$$

$$x_{B,t-1} = \frac{V_{B,t-1}}{V_{p,t-1}} = \frac{681.81}{1217.50} = 0.56 \quad (1.68)$$

The end-of-month portfolio weights are $x_{A,t-1} = 0.44$ and $x_{B,t-1} = 0.56$. Since $V_{p,t-1} = 1217.50$, to rebalance the portfolio you need $V_{A,t-1} = V_{B,t-1} = 608.75$. This requires selling 73.06 of asset B and purchasing 73.06 of asset A and the end-of-month $t-1$. Then, at the beginning of month t you have $V_{A,t-1} = V_{B,t-1} = 608.75$ and $x_{A,t-1} = x_{B,t-1} = 0.5$.

Finally, at the end-of-month t , for the rebalanced portfolio you have

$$V_{A,t} = V_{A,t-1} \times (1 + R_{A,t}) = 608.75 \times (1 + 0.1667) = 710.23 \quad (1.69)$$

$$V_{B,t} = V_{B,t-1} \times (1 + R_{B,t}) = 608.75 \times (1 - 0.3333) = 405.85 \quad (1.70)$$

$$V_{p,t} = V_{A,t-1} + V_{B,t-1} = 710.23 + 405.85 = 1116.08 \quad (1.71)$$

$$x_{A,t} = \frac{V_{A,t}}{V_{p,t}} = \frac{710.23}{1116.08} = 0.64 \quad (1.72)$$

$$x_{B,t} = \frac{V_{B,t}}{V_{p,t}} = \frac{405.85}{1116.08} = 0.36 \quad (1.73)$$

The end-of-month portfolio weights are $x_{A,t} = 0.64$ and $x_{B,t} = 0.36$. Since $V_{p,t} = 1116.08$, to rebalance the portfolio you need $V_{A,t} = V_{B,t} = 558.04$. This requires selling 152.19 of asset A and purchasing 152.19 of asset B and the end-of-month t . Then, at the beginning of month $t+1$ you have $V_{A,t} = V_{B,t} = 558.04$ and $x_{A,t} = x_{B,t} = 0.5$.

The above calculations in R are:

```
value.table = matrix(0, 4, 3)
rownames(value.table) = c("t-3", "t-2", "t-1", "t")
colnames(value.table) = c("V.A", "V.B", "V.p")
value.table["t-3", ] = c(V.A[1], V.B[1], V.p[1])
```



```

for (t in 2:4) {
  value.table[t, "V.A"] = value.table[t-1, "V.A"]*(1 + R.A[t-1])
  value.table[t, "V.B"] = value.table[t-1, "V.B"]*(1 + R.B[t-1])
  value.table[t, "V.p"] = value.table[t, "V.A"] + value.table[t, "V.B"]
  RB = max(value.table[t, "V.A"], value.table[t, "V.B"]) - value.table[t, "V.p"]/2
  if (value.table[t, "V.A"] > value.table[t, "V.B"]) {
    value.table[t, "V.A"] = value.table[t, "V.A"] - RB
    value.table[t, "V.B"] = value.table[t, "V.B"] + RB
  }
  else{
    value.table[t, "V.A"] = value.table[t, "V.A"] + RB
    value.table[t, "V.B"] = value.table[t, "V.B"] - RB
  }
  value.table[t, "V.p"] = value.table[t, "V.A"] + value.table[t, "V.B"]
}
value.table
#>      V.A V.B  V.p
#> t-3 500 500 1000
#> t-2 625 625 1250
#> t-1 609 609 1218
#> t   558 558 1116

```

Notice that the rebalanced portfolio slightly outperforms the buy-and-hold portfolio.

The portfolio return each period can be computed using $R_{p,t} = \% \Delta V_{p,t}$:

```

diff(value.table[, "V.p"])/value.table[1:3, "V.p"]
#>      t-2      t-1      t
#> 0.2500 -0.0260 -0.0833

```

It can also be computed using $R_{p,t} = 0.5 \times R_{A,t} + 0.5 \times R_{B,t}$:

```

0.5*R.A + 0.5*R.B
#> [1] 0.2500 -0.0260 -0.0833

```

Here, you see that the rebalanced portfolio loses less money over month $t-1$ than the buy-and-hold portfolio.

In the above example, at each rebalancing date you sell the asset that has the best performance (sell high) and you buy the asset that has the worst performance (buy low). Hence, rebalancing forces the sage investing advice: “buy low and sell high”.

1.3.1.3 Rebalance portfolio at specified dates

When the portfolio is rebalanced to a fixed initial portfolio, such as an equally weighted portfolio, on specific dates the portfolio return calculations become a hybrid of those used for scenarios 1 and 2. The portfolio return is computed as

$$R_{p,t} = x_{A,t-1} \times R_{A,t} + x_{B,t-1} \times R_{B,t},$$

where $x_{i,t-1}$ ($i = A, B$) are equal to the initial portfolio weights on the rebalancing dates and is computed like the buy-and-hold weights in between the rebalancing dates.

1.3.1.4 Actively managed portfolio

In an actively managed portfolio, the portfolio weights are chosen by the portfolio manager at each time period. Many mutual funds are managed in this way. The return on the actively managed portfolio is computed as

$$R_{p,t} = x_{A,t-1} \times R_{A,t} + x_{B,t-1} \times R_{B,t}$$

where $x_{A,t-1}$ and $x_{B,t-1}$ are the portfolio weights chosen by the portfolio manager at the beginning of time t .

1.3.2 Continuously Compounded Portfolio Returns

The continuously compounded portfolio return is defined by Equation 1.34, where R_t is computed using the portfolio return Equation 1.44. However, notice that:

$$r_{p,t} = \ln(1 + R_{p,t}) = \ln \left(1 + \sum_{i=1}^n x_i R_{i,t} \right) \neq \sum_{i=1}^n x_i r_{i,t}, \quad (1.74)$$

where $r_{i,t}$ denotes the continuously compounded one-period return on asset i . Hence, the continuously compounded portfolio return is not a share weighted average of the individual asset continuously compounded returns. However, if the portfolio return $R_{p,t} = \sum_{i=1}^n x_i R_{i,t}$ is not too large then $r_{p,t} \approx R_{p,t}$ otherwise, $R_{p,t} > r_{p,t}$.

Example 1.23 (Compute continuously compounded portfolio returns). Consider a portfolio of Microsoft and Starbucks stock with $x_{\text{msft}} = 0.25$, $x_{\text{sbux}} = 0.75$, $R_{\text{msft},t} = 0.0588$, $R_{\text{sbux},t} = -0.0503$ and $R_{p,t} = -0.02302$. Using (Equation 1.74), the continuously compounded portfolio return is: $r_{p,t} = \ln(1 - 0.02302) = \ln(0.977) = -0.02329$. Using $r_{\text{msft},t} = \ln(1 + 0.0588) = 0.0572$ and $r_{\text{sbux},t} = \ln(1 - 0.0503) = -0.05161$, notice that: $x_{\text{msft}} r_{\text{msft}} + x_{\text{sbux}} r_{\text{sbux}} = -0.02442 \neq r_{p,t}$

■

Because the continuously compounded portfolio return is not a share weighted average of the individual asset continuously compounded returns, the analysis of portfolios is typically performed using simple returns and not continuously compounded returns.

1.4 Return Calculations with Data in R

This section discusses representing time series data in R using `xts` objects, the calculation of returns from historical prices in R, as well as the graphical display of prices and returns.

1.4.1 Representing time series data using `xts` objects

The examples in this section are based on the daily adjusted closing price data for Microsoft and Starbucks stock over the period January 4, 1993 through December 31, 2014⁵. These data are available as the `xts` objects `msftDailyPrices` and `sbuxDailyPrices` in the R package `introcompfinr`⁶

```
head(cbind(msftDailyPrices,sbuxDailyPrices), 3)
#>           MSFT SBUX
#> 1993-01-04 1.89 1.08
#> 1993-01-05 1.92 1.10
#> 1993-01-06 1.98 1.12

class(msftDailyPrices)
#> [1] "xts" "zoo"
```

There are many different ways of representing a time series of data in R. For financial time series `xts` (extensible time series) objects from the `xts` package are especially convenient and useful. An `xts` object consists of two pieces of information: (1) a `matrix` of numeric data with different time series in the columns, (2) an R object representing the common time indexes associated with the rows of the data. `xts` objects extend and enhance the `zoo` class of time series objects from the `zoo` package written by Achim Zeileis (zoo stands for Z's ordered observations).

The `matrix` of time series data can be extracted from the `xts` object using the `xts` function `coredata()`:

⁵These are prices that are adjusted for dividends and stock splits. That is, any dividend payments have been included in the prices and historical prices have been divided by the split ratio associated with any stock splits.

⁶See the Appendix Working with Time Series Data in R for an overview of `xts` objects and Zivot (2016) for a comprehensive coverage.

```

matrix.data = coredata(msftDailyPrices)
head(matrix.data, 3)
#>      MSFT
#> [1,] 1.89
#> [2,] 1.92
#> [3,] 1.98

```

The resulting `matrix` of data does not have any date information. The date index can be extracted using the `xts` function `index()`:

```

date.index = index(msftDailyPrices)
head(date.index, 3)
#> [1] "1993-01-04" "1993-01-05" "1993-01-06"

```

The object `date.index` is of class `Date`:

```

class(date.index)
#> [1] "Date"

```

When a `Date` object is printed, the date is displayed in the format `yyyy-mm-dd`. However, internally each date represents the number of days since January 1, 1970:

```

as.numeric(date.index[1])
#> [1] 8404

```

Here, `1993-01-04` (January 4, 1993) is 8,404 days after January 1, 1970. This allows for simple *date arithmetic* (e.g. adding and subtracting dates, etc). More sophisticated date arithmetic is available using the functions in the **lubridate** package.

There are several advantages of using `xts` objects to represent financial time series data. Most financial time series do not follow an equally spaced regular calendar. The time index of a `xts` object can be any strictly increasing date sequence which can match the times and dates for which assets trade on an exchange. For example, assets traded on the main US stock exchanges (e.g. NYSE and NASDAQ) trade only on weekdays between 9:30AM and 4:00PM Eastern Standard Time and do not trade on a number of holidays.

Another advantage of `xts` objects is that you can easily extract observations at or between specific dates. For example, to extract the daily price on January 3, 2014 use

```

msftDailyPrices["2014-01-03"]
#>      MSFT
#> 2014-01-03 35.7

```

To extract prices between January 3, 2014 and January 7, 2014, use

```
msftDailyPrices["2014-01-03::2014-01-07"]
#>           MSFT
#> 2014-01-03 35.7
#> 2014-01-06 34.9
#> 2014-01-07 35.2
```

Or to extract all of the prices for January, 2014 use:

```
msftDailyPrices["2014-01"]
#>           MSFT
#> 2014-01-02 35.9
#> 2014-01-03 35.7
#> 2014-01-06 34.9
#> 2014-01-07 35.2
#> 2014-01-08 34.6
#> 2014-01-09 34.3
#> 2014-01-10 34.8
#> 2014-01-13 33.8
#> 2014-01-14 34.6
#> 2014-01-15 35.5
#> 2014-01-16 35.6
#> 2014-01-17 35.2
#> 2014-01-21 35.0
#> 2014-01-22 34.7
#> 2014-01-23 34.9
#> 2014-01-24 35.6
#> 2014-01-27 34.8
#> 2014-01-28 35.0
#> 2014-01-29 35.4
#> 2014-01-30 35.6
#> 2014-01-31 36.6
```

Two or more `xts` objects can be merged together and aligned to a common date index using the `xts` function `merge()`:

```
msftSbuxDailyPrices = merge(msftDailyPrices, sbuxDailyPrices)
head(msftSbuxDailyPrices, n=3)
#>           MSFT SBUX
```

```
#> 1993-01-04 1.89 1.08
#> 1993-01-05 1.92 1.10
#> 1993-01-06 1.98 1.12
```

If two or more `xts` objects have a common date index then they can be combined using `cbind()`.

1.4.1.1 Basic calculations on `xts` objects.

Because `xts` objects typically contain a matrix of numeric data, many R functions that operate on matrices also operate on `xts` objects. For example, to transform `msftDailyPrices` to log prices use

```
msftDailyLogPrices = log(msftDailyPrices)
head(msftDailyLogPrices, 3)
#>           MSFT
#> 1993-01-04 0.637
#> 1993-01-05 0.652
#> 1993-01-06 0.683
```

If two or more `xts` objects have the same time index then they can be added, subtracted, multiplied, and divided. For example,

```
msftPlusSbuxDailyPrices = msftDailyPrices + sbuxDailyPrices
colnames(msftPlusSbuxDailyPrices) = "MSFT+SBUX"
head(msftPlusSbuxDailyPrices, 3)
#>           MSFT+SBUX
#> 1993-01-04      2.97
#> 1993-01-05      3.02
#> 1993-01-06      3.10
```

To compute the average of the Microsoft daily prices use

```
mean(msftDailyPrices)
#> [1] 19.9
```

If an R function, for some unknown reason, is not operating correctly on an `xts` object first extract the data using `coredata()` and then call the R function. For example,

```
mean(coredata(msftDailyPrices))
#> [1] 19.9
```

1.4.1.2 Changing the frequency of an `xts` object

Data at the daily frequency is the highest frequency of data considered in this book. However, in many of the examples you want to use data at the weekly or monthly frequency. The conversion of data at a daily frequency to data at a monthly frequency is easy to do with `xts` objects. For example, end-of-month prices can be extracted from the daily prices using the `xts` function `to.monthly()`:

```
msftMonthlyPrices = to.monthly(msftDailyPrices, OHLC=FALSE)
sbuxMonthlyPrices = to.monthly(sbuxDailyPrices, OHLC=FALSE)
msftSbuxMonthlyPrices = to.monthly(msftSbuxDailyPrices, OHLC=FALSE)
head(msftMonthlyPrices, 3)
#>           MSFT
#> Jan 1993 1.92
#> Feb 1993 1.85
#> Mar 1993 2.06
```

By default, `to.monthly()` extracts the data for the last day of the month and creates a `zoo` `yearmon` date index. For monthly data, the `yearmon` date index is convenient for printing and plotting as the month and year are nicely printed. To preserve the `Date` class of the time index and show the end-of-month date, use the optional argument `indexAt = "lastof"` in the call to `to.monthly()`:

```
head(to.monthly(msftDailyPrices, OHLC=FALSE, indexAt="lastof"), 3)
#>           MSFT
#> 1993-01-31 1.92
#> 1993-02-28 1.85
#> 1993-03-31 2.06
```

In the above calls to `to.monthly()`, the optional argument `OHLC=FALSE` prevents the creation of open, high, low, and closing prices for the month.

In a similar fashion, you can extract end-of-week prices using the `xts` function `to.weekly()`:

```
msftWeeklyPrices = to.weekly(msftDailyPrices, OHLC=FALSE)
head(msftWeeklyPrices, 3)
#>           MSFT
#> 1993-01-08 1.94
#> 1993-01-15 2.00
#> 1993-01-22 1.99
```

Here, the weekly data are the closing prices on each Friday of the week:

```
head(weekdays(index(msftWeeklyPrices)),3)
#> [1] "Friday" "Friday" "Friday"
```

1.4.1.3 Plotting xts objects with plot.xts() and plot.zoo

Time plots of `xts` objects can be created with the generic `plot()` function as there is a method function in the `xts` package for objects of class `xts`. For example, Figure Figure 1.1 shows a basic time plot of the monthly closing prices of Microsoft and Starbucks created with:

```
plot(msftSbuxMonthlyPrices, main="Monthly Closing Prices",
     legend.loc="topleft")
```

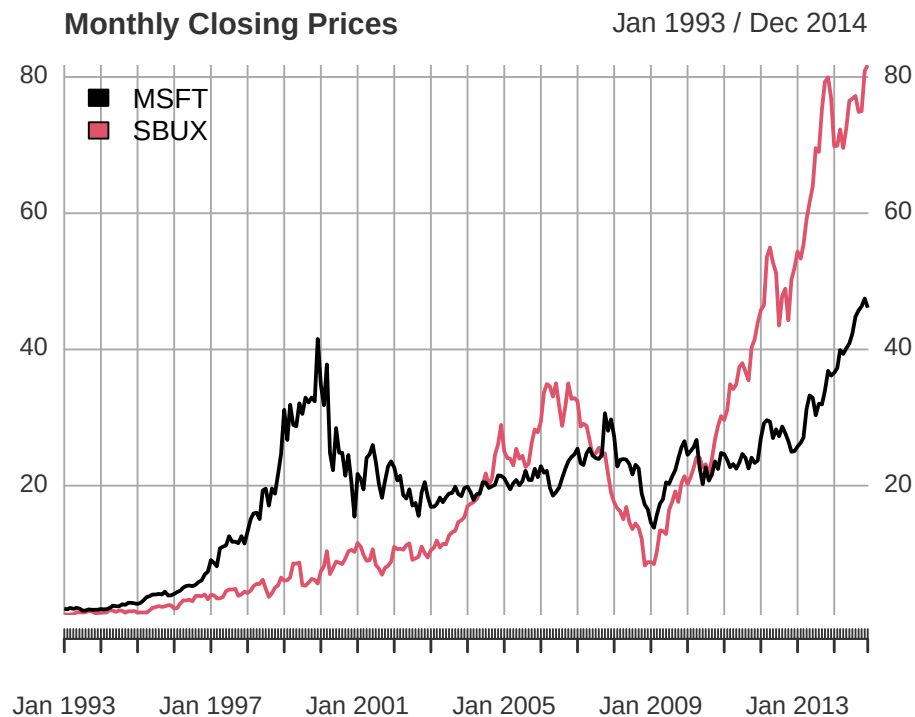


Figure 1.1: Single panel plot (default) with `plot.xts()`

The default plot style in `plot.xts()` is a single-panel plot with multiple series. You can also create multi-panel plots, as in Figure Figure 1.2, by setting the optional argument `multi.panel = TRUE` in the call to `plot.xts()`

```
plot(msftSbuxMonthlyPrices, main="Monthly Closing Prices",
     multi.panel=TRUE)
```

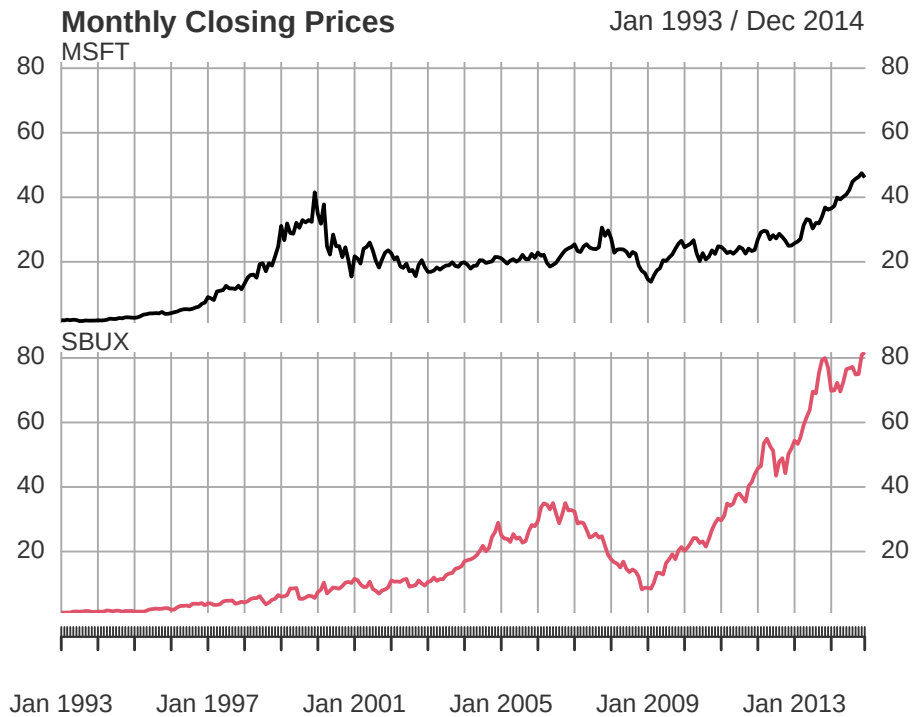



Figure 1.2: Multi-panel time series plot with `plot.xts()`

See the help file for `plot.xts()` for more examples of plotting `xts` objects.

Because the `xts` class inherits from the `zoo` class, the `zoo` method function `plot.zoo()` can also be used to plot `xts` objects. Figure Figure 1.3 is created with

```
plot.zoo(msftSbuxMonthlyPrices, plot.type="single",
        main="Monthly Closing Prices",
        lwd = 2, col=c("black", "red"), ylab="Price")
grid()
legend(x="topleft", legend=colnames(msftSbuxMonthlyPrices),
       lwd = 2, col=c("black", "red"))
```

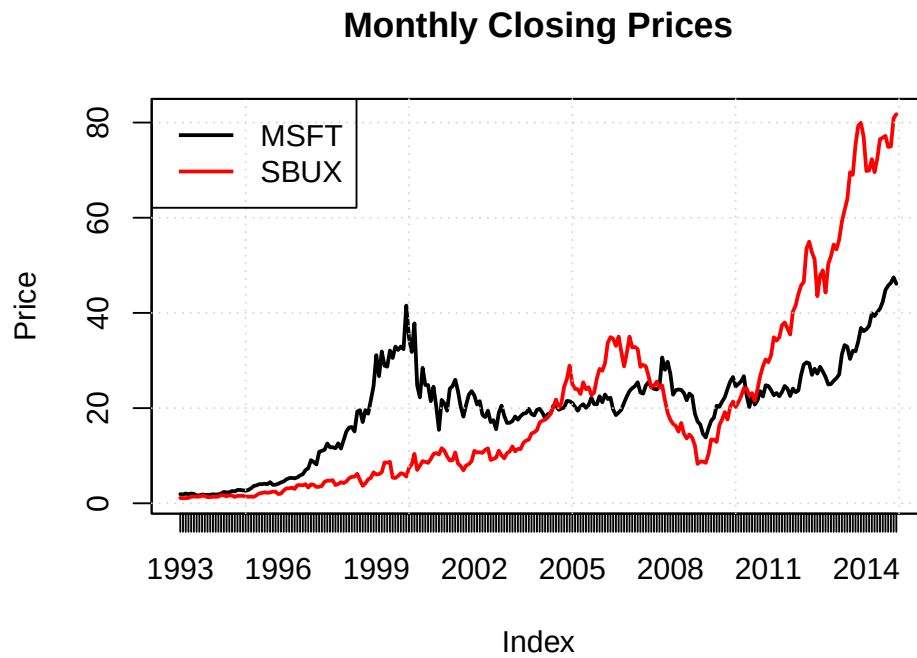


Figure 1.3: Single panel time series plot (default) with `plot.zoo()`

A multi-panel plot, shown in Figure Figure 1.4, can be created using

```
plot.zoo(msftSbuxMonthlyPrices, plot.type="multiple", main="",  
        lwd = 2, col=c("black", "red"), cex.axis=0.8)
```

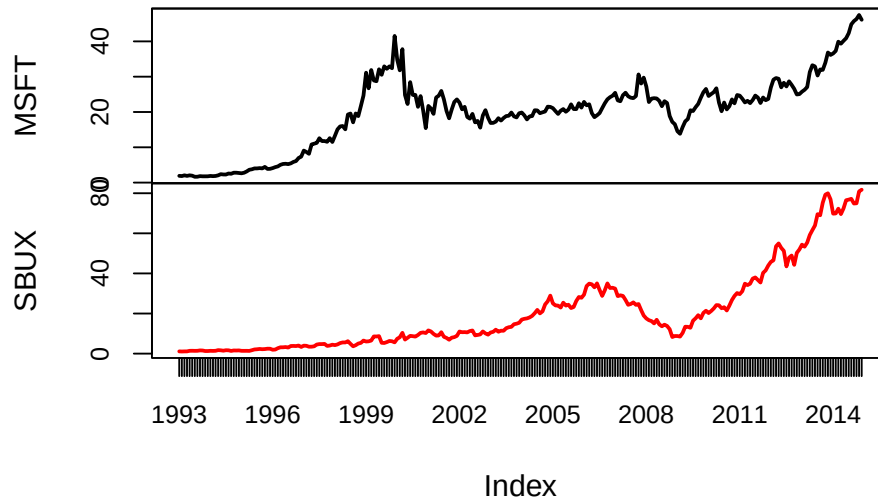


Figure 1.4: Multi-panel time series plot with `plot.zoo()`

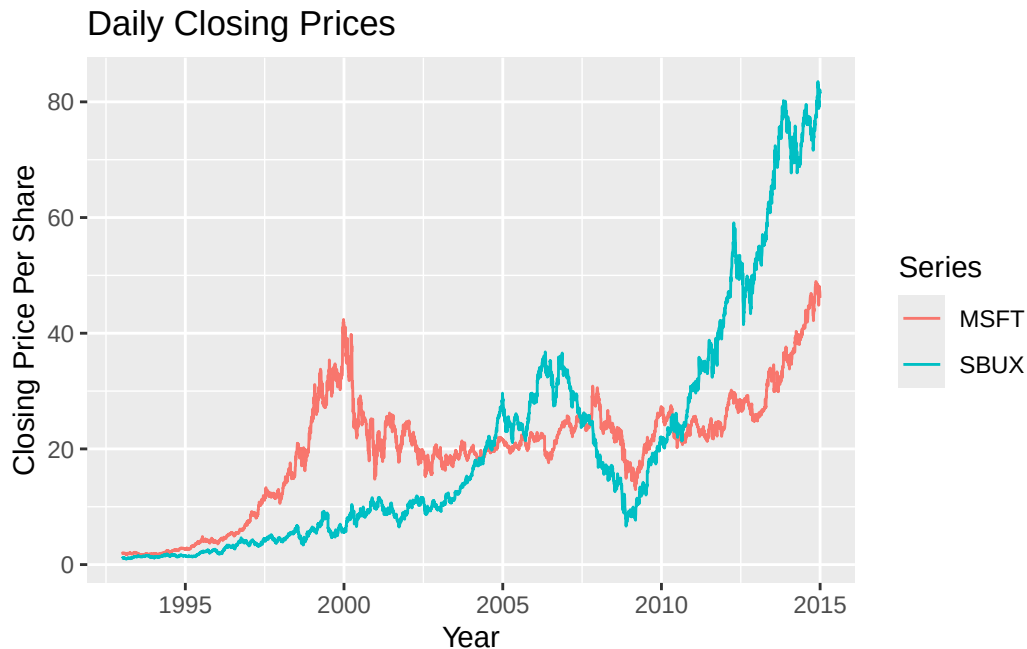
Creating plots with `plot.zoo()` is a bit more involved than with `plot.xts()`, as `plot.xts()` is newer. See the help file for `plot.zoo()` for more examples.

1.4.1.4 Plotting `xts` objects using `autoplot()` from `ggplot2`

The plots created using `plot.xts()` and `plot.zoo()` are created using the base graphics system in R. This allows for a lot of flexibility but often the resulting plots do not look very exciting or modern. In addition, these functions are not well suited for plotting many time series together in a single plot.

Another very popular graphics system in R is provided by the **ggplot2** package by Hadley Wickham of RStudio. For plotting `xts` objects, especially with multiple columns (data series), the **ggplot2** function `autoplot()` is especially convenient and easy:

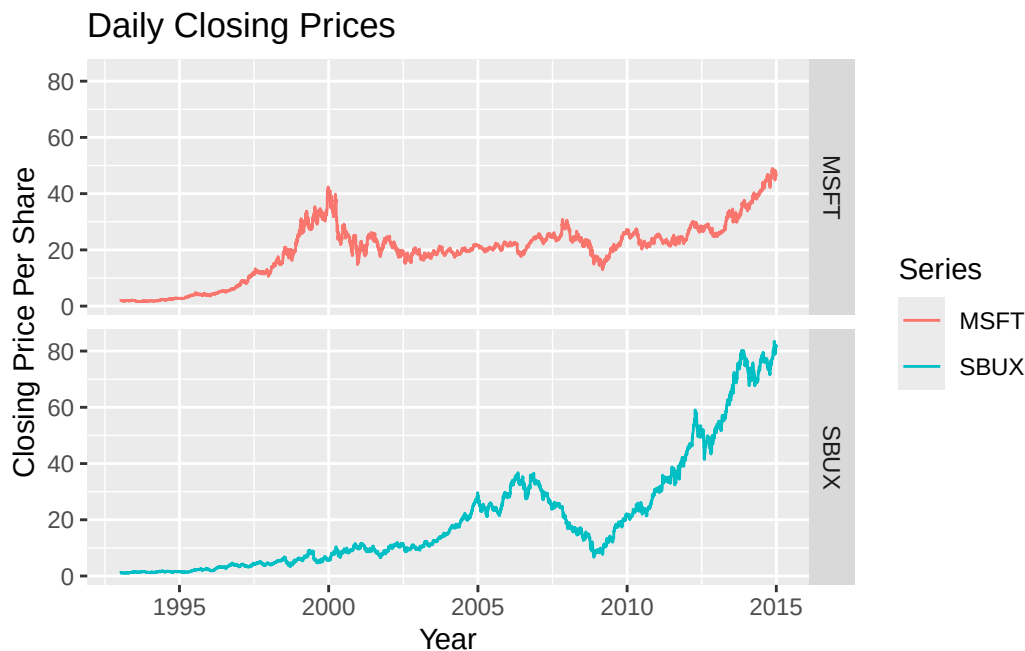
```
autoplot(msftSbuxDailyPrices, facets = NULL) +
  ggtitle("Daily Closing Prices") +
  ylab("Closing Price Per Share") +
  xlab("Year")
```



The syntax for creating graphs with `autoplot()` uses the *grammar of graphics* syntax from the **ggplot2** package. See the [R Graphics Cookbook](#) for more details and examples on using the **ggplot2** package. The call to `autoplot()` first invokes the method function `autoplot.zoo()` from the **zoo** package and creates a basic time series plot. Additional layers showing a main title and axes labels are added using `+`. The optional argument `facets = NULL` specifies that all series are plotted together on a single plot with different colors.

To produce a multi-panel plot call `autoplot()` with `facets = Series ~ .:`

```
autoplot(msftSbuxDailyPrices, facets = Series ~ .) +  
  ggtitle("Daily Closing Prices") +  
  ylab("Closing Price Per Share") +  
  xlab("Year")
```



See the help file for `autoplot.zoo()` for more examples.

1.4.2 Calculating returns

In this sub-section, I illustrate how to calculate a time series of returns from a time series of prices.

1.4.2.1 Brute force return calculations

Consider computing simple monthly returns, $R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$, from historical prices using the `xts` object `sbuxMonthlyPrices`. The R code for a brute force calculation is:

```
sbuxMonthlyReturns = diff(sbuxMonthlyPrices)/stats::lag(sbuxMonthlyPrices)
head(sbuxMonthlyReturns, n=3)
#>           SBUX
#> Jan 1993      NA
#> Feb 1993 -0.0625
#> Mar 1993  0.0476
```

Here, the `diff()` function computes the first difference in the prices, $P_t - P_{t-1}$, and the `lag()` function computes the lagged price, P_{t-1} .⁷ An equivalent calculation is $R_t = \frac{P_t}{P_{t-1}} - 1$:

⁷The method function `lag.xts()` works differently than generic `lag()` function and the method function `lag.zoo()`. In particular, `lag.xts()` with optional argument `k=1` gives the same result as `lag()` and `lag.zoo()` with `k=-1`.

```
head(sboxMonthlyPrices/stats::lag(sboxMonthlyPrices) - 1, n=3)
#>           SBUX
#> Jan 1993      NA
#> Feb 1993 -0.0625
#> Mar 1993  0.0476
```

Notice that the return for January, 1993 is NA (missing value). To automatically remove this missing value, use the R function `na.omit()` or explicitly exclude the first observation:

```
head(na.omit(sboxMonthlyReturns), n=3)
#>           SBUX
#> Feb 1993 -0.0625
#> Mar 1993  0.0476
#> Apr 1993  0.0182
```

```
head(sboxMonthlyReturns[-1], n=3)
#>           SBUX
#> Feb 1993 -0.0625
#> Mar 1993  0.0476
#> Apr 1993  0.0182
```

To compute continuously compounded returns from simple returns, use:

```
sboxMonthlyReturnsC = na.omit(log(1 + sboxMonthlyReturns))
head(sboxMonthlyReturnsC, n=3)
#>           SBUX
#> Feb 1993 -0.0645
#> Mar 1993  0.0465
#> Apr 1993  0.0180
```

Or, equivalently, to compute continuously compounded returns directly from prices, use:

```
sboxMonthlyReturnsC = na.omit(diff(log(sboxMonthlyPrices)))
head(sboxMonthlyReturnsC, n=3)
#>           SBUX
#> Feb 1993 -0.0645
#> Mar 1993  0.0465
#> Apr 1993  0.0180
```

The above calculations used an `xts` object with a single column. If an `xts` object has multiple

columns the same calculations work for each column. For example,

```
msftSbuxMonthlyReturns = diff(msftSbuxMonthlyPrices)/stats::lag(msftSbuxMonthlyPrices)
head(msftSbuxMonthlyReturns, n=3)
#>           MSFT      SBUX
#> Jan 1993      NA      NA
#> Feb 1993 -0.0365 -0.0625
#> Mar 1993  0.1135  0.0476
```

Also,

```
msftSbuxMonthlyReturnsC = diff(log(msftSbuxMonthlyPrices))
head(msftSbuxMonthlyReturnsC, n=3)
#>           MSFT      SBUX
#> Jan 1993      NA      NA
#> Feb 1993 -0.0371 -0.0645
#> Mar 1993  0.1075  0.0465
```

These are examples of *vectorized* calculations. That is, the calculations are performed on all columns simultaneously. This is computationally more efficient than looping the same calculation for each column. In R, you should try to vectorize calculations whenever possible.

1.4.2.2 Calculating returns using `Return.calculate()`

Simple and continuously compounded returns can be computed using the **PerformanceAnalytics** function `Return.calculate()`:

```
# Simple returns
sbuxMonthlyReturns = Return.calculate(sbuxMonthlyPrices)
head(sbuxMonthlyReturns, n=3)
#>           SBUX
#> Jan 1993      NA
#> Feb 1993 -0.0625
#> Mar 1993  0.0476

# CC returns
sbuxMonthlyReturnsC = Return.calculate(sbuxMonthlyPrices, method="log")
head(sbuxMonthlyReturnsC, n=3)
#>           SBUX
#> Jan 1993      NA
```

```
#> Feb 1993 -0.0645
#> Mar 1993  0.0465
```

`Return.calculate()` also works with `xts` objects with multiple columns:

```
msftSbuxMonthlyReturns = Return.calculate(msftSbuxMonthlyPrices)
head(msftSbuxMonthlyReturns, 3)
#>           MSFT      SBUX
#> Jan 1993      NA      NA
#> Feb 1993 -0.0365 -0.0625
#> Mar 1993  0.1135  0.0476
```

The advantages of using `Return.calculate()` instead of brute force calculations are: (1) clarity of code (you know what is being computed), (2) the same function is used for both simple and continuously compounded returns, (3) built-in error checking.

1.4.2.3 Equity curves

To directly compare the investment performance of two or more assets, plot the simple multi-period cumulative returns of each asset on the same graph. This type of graph, sometimes called an *equity curve*, shows how a one dollar investment amount in each asset grows over time. Better performing assets have higher equity curves. For simple returns, the k -period returns are $R_t(k) = \prod_{j=0}^{k-1} (1 + R_{t-j})$ and represent the growth of one dollar invested for k periods. For continuously compounded returns, the k -period returns are $r_t(k) = \sum_{j=0}^{k-1} r_{t-j}$. However, this continuously compounded k -period return must be converted to a simple k -period return, using $R_t(k) = \exp(r_t(k)) - 1$, to properly represent the growth of one dollar invested for k periods.

Example 1.24 (Equity curves for Microsoft and Starbucks monthly returns). To create the equity curves for Microsoft and Starbucks based on simple returns use (You can also use the **PerformanceAnalytics** function `chart.CumReturns()`)

```
msftMonthlyReturns = na.omit(Return.calculate(msftMonthlyPrices))
sbuxMonthlyReturns = na.omit(Return.calculate(sbuxMonthlyPrices))
equityCurveMsft = cumprod(1 + msftMonthlyReturns)
equityCurveSbux = cumprod(1 + sbuxMonthlyReturns)
dataToPlot = merge(equityCurveMsft, equityCurveSbux)
plot(dataToPlot, main="Monthly Equity Curves", legend.loc="topleft")
```

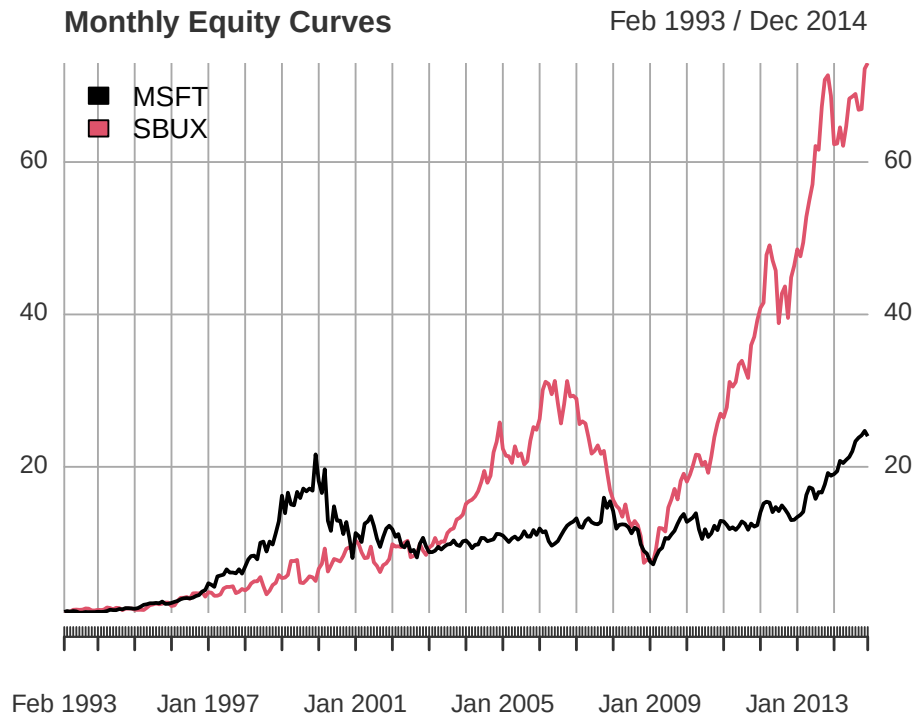



Figure 1.5: Monthly equity curve for Microsoft and Starbucks.

The R function `cumprod()` creates the cumulative products needed for the equity curves. Figure 1.5 shows that a one dollar investment in Starbucks dominated a one dollar investment in Microsoft over the given period. In particular, \$1 invested in Microsoft grew to about only \$22 (over about 20 years) whereas \$1 invested in Starbucks grew over \$70. Notice the huge increases and decreases in value of Microsoft during the dot-com bubble and bust over the period 1998 - 2001, and the enormous increase in value of Starbucks from 2009 - 2014.

■

1.4.3 Calculating portfolio returns from time series data

As discussed in sub-section Section 1.3, the calculation of portfolio returns depends on the assumptions made about the portfolio weights over time.

1.4.3.1 Constant portfolio weights

The easiest case assumes that portfolio weights are constant over time. To illustrate, consider an equally weighted portfolio of Microsoft and Starbucks stock. A time series a portfolio monthly returns can be created directly from the monthly returns on Microsoft and Starbucks stock:

```

x.msft = x.sbx = 0.5
msftMonthlyReturns = na.omit(Return.calculate(msftMonthlyPrices))
sbxMonthlyReturns = na.omit(Return.calculate(sbxMonthlyPrices))
equalWeightPortMonthlyReturns = x.msft*msftMonthlyReturns +
  x.sbx*sbxMonthlyReturns
head(equalWeightPortMonthlyReturns, 3)
#>           MSFT
#> Feb 1993 -0.0495
#> Mar 1993  0.0806
#> Apr 1993 -0.0297

```

The same calculation can be performed using the **PerformanceAnalytics** function `Return.portfolio()`:

```

equalWeightPortMonthlyReturns =
  Return.portfolio(na.omit(msftSbxMonthlyReturns),
    weights = c(x.msft, x.sbx),
    rebalance_on = "months")
head(equalWeightPortMonthlyReturns, 3)
#>           portfolio.returns
#> Feb 1993          -0.0495
#> Mar 1993           0.0806
#> Apr 1993          -0.0297

```

Here, the optional argument `rebalance_on = "months"` specifies that the portfolio is to be rebalanced to the specified weights at the beginning of every month.

If you also want the asset contributions to the portfolio return set the optional argument `contribution = TRUE` (recall, asset contributions sum to the portfolio return):

```

equalWeightPortMonthlyReturns =
  Return.portfolio(na.omit(msftSbxMonthlyReturns),
    weights = c(x.msft, x.sbx),
    rebalance_on = "months",
    contribution = TRUE)
head(equalWeightPortMonthlyReturns, 3)
#>           portfolio.returns    MSFT    SBUX
#> Feb 1993          -0.0495 -0.0182 -0.03125
#> Mar 1993           0.0806  0.0568  0.02381
#> Apr 1993          -0.0297 -0.0388  0.00909

```

You can see the rebalancing calculations by setting the optional argument `verbose = TRUE`:

```

equalWeightPortMonthlyReturns =
  Return.portfolio(na.omit(msftSbuxMonthlyReturns),
    weights = c(x.msft, x.sbx),
    rebalance_on = "months",
    verbose = TRUE)
names(equalWeightPortMonthlyReturns)
#> [1] "returns"      "contribution" "BOP.Weight"   "EOP.Weight"   "BOP.Value"
#> [6] "EOP.Value"

```

In this case `Return.portfolio()` returns a list with the above components. The component `BOP.Weight` shows the beginning-of-period portfolio weights, and the component `EOP.Weight` shows the end-of-period portfolio weights (before any rebalancing).

```

head(equalWeightPortMonthlyReturns$BOP.Weight, 3)
#>      MSFT  SBUX
#> Feb 1993  0.5  0.5
#> Mar 1993  0.5  0.5
#> Apr 1993  0.5  0.5

```

With monthly returns and with `rebalance_on = "months"`, the beginning-of-period weights are always rebalanced to the initially supplied weights.

```

head(equalWeightPortMonthlyReturns$EOP.Weight, 3)
#>      MSFT  SBUX
#> Feb 1993 0.507 0.493
#> Mar 1993 0.515 0.485
#> Apr 1993 0.475 0.525

```

Because the returns for Microsoft and Starbux are different every month the end-of-period weights drift away from the beginning-of-period weights.

1.4.3.2 Buy-and-hold portfolio

In the buy-and-hold portfolio, no rebalancing of the initial portfolio is done and the portfolio weights are allowed to evolve over time based on the returns of the assets. Hence, the computation of the portfolio return each month requires knowing the weights on each asset each month. You can use `Return.portfolio()` to compute the buy-and-hold portfolio returns as follows:

```

buyHoldPortMonthlyReturns =
  Return.portfolio(na.omit(msftSbuxMonthlyReturns),
                  weights = c(x.msft, x.sbx))
head(buyHoldPortMonthlyReturns, 3)
#>           portfolio.returns
#> Feb 1993          -0.0495
#> Mar 1993           0.0810
#> Apr 1993          -0.0319

```

Omitting the optional argument `rebalance_on` tells `Return.portfolio()` to not rebalance the portfolio at all (default behavior of the function).

1.4.3.3 Periodic rebalancing

In practice, a portfolio may be rebalanced at a specific frequency that is different than the frequency of the returns. For example, with monthly returns the portfolio might be rebalanced quarterly or annually. For example, to rebalance the equally weighted portfolio of Microsoft and Starbucks stock each quarter (every three months) use

```

equalWeightQtrPortMonthlyReturns =
  Return.portfolio(na.omit(msftSbuxMonthlyReturns),
                  weights = c(x.msft, x.sbx),
                  rebalance_on = "quarters")
head(equalWeightQtrPortMonthlyReturns, 3)
#>           portfolio.returns
#> Feb 1993          -0.0495
#> Mar 1993           0.0810
#> Apr 1993          -0.0297

```

In between quarters, the portfolio evolves as a buy-and-hold portfolio. To see the details of the rebalancing set `verbose = TRUE` in the call to `Return.Portfolio()`.

1.4.3.4 Active portfolio weights

If the weights are actively chosen at specific time periods, these weights can be supplied to the function `Return.portfolio()` to compute the portfolio return.

1.4.4 Downloading financial data from the internet

The data for the examples in this book, available in the **introcompFinr** package, were downloaded from finance.yahoo.com using the `getSymbols()` function from the **quantmod** package.

For example, to download daily data on Amazon and Google stock (ticker symbols AMZN and GOOG) from January 3, 2007 through January 3, 2020 use `getSymbols()` as follows:

```
options("getSymbols.warning4.0"=FALSE)
suppressPackageStartupMessages(library(quantmod))
getSymbols(Symbols = c("AMZN", "GOOG"), from="2007-01-03", to="2020-01-03",
          auto.assign=TRUE, warnings=FALSE)
#> [1] "AMZN" "GOOG"
class(AMZN)
#> [1] "xts" "zoo"
```

The `xts` objects `AMZN` and `GOOG` each have six columns containing the daily open price for the day, high price for the day, low price for the day, close price for the day, volume for the day, and (dividend and split) adjusted closing price for the day:

```
head(AMZN, 3)
#>           AMZN.Open AMZN.High AMZN.Low AMZN.Close AMZN.Volume AMZN.Adjusted
#> 2007-01-03      1.93      1.95      1.90      1.93      2.48e+08          1.93
#> 2007-01-04      1.93      1.96      1.91      1.95      1.26e+08          1.95
#> 2007-01-05      1.94      1.94      1.88      1.92      1.32e+08          1.92
```

```
head(GOOG, 3)
#>           GOOG.Open GOOG.High GOOG.Low GOOG.Close GOOG.Volume GOOG.Adjusted
#> 2007-01-03      11.6      11.9      11.5      11.6      3.09e+08          11.5
#> 2007-01-04      11.7      12.1      11.7      12.0      3.17e+08          11.9
#> 2007-01-05      12.0      12.1      11.9      12.1      2.76e+08          12.0
```

1.5 Further Reading: Return Calculations

This chapter describes basic asset return calculations with an emphasis on equity calculations. Similar material is covered in Chapter 2 of (Ruppert and Matteson 2015). Comprehensive coverage of general return calculations is given in (Bacon 2008) and (Christopherson, Carino, and Ferson 2009). The virtues of portfolio rebalancing are discussed in (Ang 2014) and (Qian 2019).

This chapter introduced you to the R packages **PerformanceAnalytics**, **quantmod**, and **xts** for working with financial time series. It is recommended to read the vignettes included in each of these

packages. Jeff Ryan's original [quantmod page](#), while a bit outdated, has many useful examples of working with **quantmod** and **xts**. A good [online tutorial](#) for the **xts** package is provided by Joshua Ulrich, one of the creators of **xts**.

There are many useful R packages for the analysis of financial data. An up-to-date list of R packages relevant for finance is given on the [Empirical Finance Task View](#) on the comprehensive R archive network (CRAN).

1.6 Appendix: Properties of Exponentials and Logarithms

The computation of continuously compounded returns requires the use of natural logarithms. The natural logarithm function, $\ln(\cdot)$, is the inverse of the exponential function, $e^{(\cdot)} = \exp(\cdot)$, where $e^1 = 2.718$. That is, $\ln(x)$ is defined such that $x = \ln(e^x)$. Figure Figure 1.6 plots e^x and $\ln(x)$. Notice that e^x is always positive and increasing in x . $\ln(x)$ is monotonically increasing in x and is only defined for $x > 0$. Also note that $\ln(1) = 0$ and $\ln(0) = -\infty$.

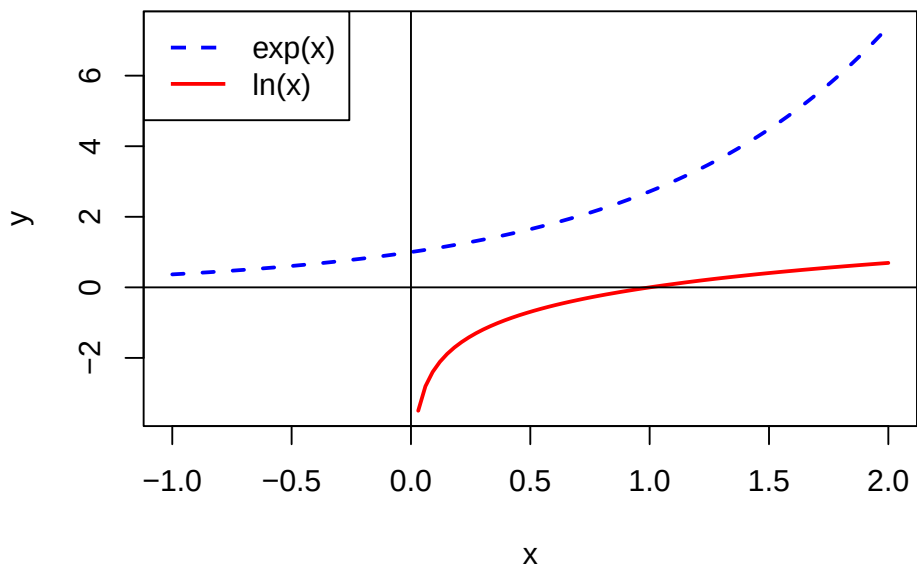


Figure 1.6: Exponential and natural logarithm functions.

The exponential and natural logarithm functions have the following properties:

1. $\ln(x \cdot y) = \ln(x) + \ln(y)$, $x, y > 0$
2. $\ln(x/y) = \ln(x) - \ln(y)$, $x, y > 0$
3. $\ln(x^y) = y \ln(x)$, $x > 0$
4. $\frac{d \ln(x)}{dx} = \frac{1}{x}$, $x > 0$
5. $\frac{d}{dx} \ln(f(x)) = \frac{1}{f(x)} \frac{d}{dx} f(x)$ (chain-rule)
6. $e^x e^y = e^{x+y}$
7. $e^x e^{-y} = e^{x-y}$
8. $(e^x)^y = e^{xy}$
9. $e^{\ln(x)} = x$
10. $\frac{d}{dx} e^x = e^x$
11. $\frac{d}{dx} e^{f(x)} = e^{f(x)} \frac{d}{dx} f(x)$ (chain-rule)

1.7 Exercises

Analytical Problems

Exercise 1.1. In this question you will see how the future value of an investment, FV_n varies with the interest rate, R , and the investment horizon, n . Let the initial amount invested be $V = \$1,000$. For a grid of values $R = 0.01, 0.02, \dots, 0.10$ and $n = 1, 2, \dots, 10$, compute and plot FV_n .

Exercise 1.2. Consider the following credit card offer from a hypothetical bank. The card advertises an annual percentage rate (APR) of 19.34%. However, the bank actually compounds interest charges on a daily basis.

1. Assuming 365 days in the year, what is the daily periodic interest rate being charged?
2. If you carried a \$1,000 balance throughout the year, how much would you owe at the end of the year?

Exercise 1.3. Consider “payday loans”. Payday loans are short-term loans made to consumers, often for less than two weeks, and are offered by companies such as AmeriCash Advance and National Payday. The loan works like this: You write a check today that is postdated and the company gives you an amount of money less than the check. When the check date arrives, you go to the store and pay cash for the check, or the company cashes the check. For example, suppose AmeriCash Advance allows you to write a postdated check for \$125 for 15 days later in return for a \$100 loan today.

1. What is the annual percentage rate (APR) on this loan?
2. What is the effective annual rate (EAR)?

Exercise 1.4. Consider the following (actual) monthly adjusted closing price data for Starbucks stock over the period December 2004 through December 2005

Table 1.2: End of Month Price Data for Starbucks Stock

Date	Price
December, 2004	31.2
January, 2005	27.0
February, 2005	25.9
March, 2005	25.8
April, 2005	24.8
May, 2005	27.4
June, 2005	25.8
July, 2005	26.3

Date	Price
August, 2005	24.5
September, 2005	25.1
October, 2005	28.3
November, 2005	30.4
December, 2005	30.5

For the following questions, first do the return calculations by hand. That is, give the appropriate return calculation formula and evaluate the result. Then, check your hand calculations using the corresponding R calculations.

1. Using the data in the table, what is the simple monthly return between the end of December 2004 and the end of January 2005? If you invested \$10,000 in Starbucks at the end of December 2004, how much would the investment be worth at the end of January 2005?
2. Using the data in the table, what is the continuously compounded monthly return between December 2004 and January 2005? Convert this continuously compounded return to a simple return (you should get the same answer as in part 1).
3. Assuming that the simple monthly return you computed in part 1 is the same for 12 months, what is the annual return with monthly compounding?
4. Assuming that the continuously compounded monthly return you computed in part 2 is the same for 12 months, what is the continuously compounded annual return?
5. Using the data in the table, compute the actual simple annual return between December 2004 and December 2005. If you invested \$10,000 in Starbucks at the end of December 2004, how much would the investment be worth at the end of December 2005? Compare with your result in part 3.
6. Using the data in the table, compute the actual annual continuously compounded return between December 2004 and December 2005. Compare with your result in part 4. Convert this continuously compounded return to a simple return (you should get the same answer as in part 5).

Exercise 1.5. Consider a one month investment in two Northwest stocks: Amazon and Costco. Suppose you buy Amazon and Costco at the end of September at $P_{A,t-1} = \$38.23$, $P_{C,t-1} = \$41.11$ and then sell at the end of the October for $P_{A,t} = \$41.29$ and $P_{C,t} = \$41.74$. (Note: these are actual closing prices for 2004 taken from Yahoo!)

For the following questions, first do the return calculations by hand. That is, give the appropriate return calculation formula and evaluate the result. Then, check your hand calculations using the corresponding R calculations.

1. What are the simple monthly returns for the two stocks?

2. What are the continuously compounded returns for the two stocks?
3. Suppose Costco paid a \$0.10 per share cash dividend at the end of October. What is the monthly simple total return on Costco? What is the monthly dividend yield?
4. Suppose the monthly returns on Amazon and Costco from part 1 above are the same every month for 1 year. Compute the simple annual returns as well as the continuously compounded annual returns for the two stocks.
5. At the end of September 2004, suppose you have \$10,000 to invest in Amazon and Costco over the next month. If you invest \$8000 in Amazon and \$2000 in Costco, what are your portfolio shares, x_A and x_C ?
6. Continuing with part 5, compute the monthly simple return and the monthly continuously compounded return on the portfolio. Assume that Costco does not pay a dividend.

Exercise 1.6. Consider a 48-month (4 year) investment in two assets: the Vanguard S&P 500 index (VFINX) and Amazon stock (AMZN). Suppose you buy one share of the S&P 500 fund and one share of Amazon stock at the end of January, 2015 for $P_{VFINX,t-48} = 165$, $P_{AMZN,t-48} = 355$, and then sell these shares at the end of January, 2019 for $P_{VFINX,t} = 241$, $P_{AMZN,t} = 1719$ (Note: these are actual adjusted closing prices taken from Yahoo!). In this question, you will see how much money you could have made if you invested in these assets right before the COVID-19 pandemic.

1. What are the simple 48-month (4-year) returns for the two investments?
2. What are the continuously compounded (cc) 48-month (4-year) returns for the two investments? Why are the cc returns smaller?
3. Suppose you invested \$10,000 in each asset at the end of January, 2015. How much would each investment be worth at the end of January, 2019?
4. What are the compound annual returns on the two 4-year investments?
5. At the end of January, 2015, suppose you plan to invest in a portfolio of VFINX and AMZN over the next 48 months (4 years). Suppose you purchase 5 shares of the VFINX mutual fund (at \$165/share) and 10 shares of AMZN stock (at \$355/share). What is the value of your initial investment? What are the portfolio weights in the two assets as of the end of January, 2015?
6. Using the results from part 1, compute the 4-year simple and cc portfolio returns.
7. What is the value of your portfolio at the end of January, 2019? What is the value of your investments in VFINX and AMZN? What is share of wealth invested in VFINX and AMZN (i.e., your portfolio weights)?

Exercise 1.7. Consider a 60-month (5 year) investment in two assets: the Vanguard S&P 500 index (VFINX) and Apple stock (AAPL). Suppose you buy one share of the S&P 500 fund and one share of Apple stock at the end of January, 2010 for $P_{VFINX,t-60} = \$89.91$, $P_{AAPL,t-60} = \$25.88$,

and then sell these shares at the end of January, 2015 for $P_{VFINX,t} = \$184.2$, $P_{AAPL,t} = \$116.7$. (Note: these are actual adjusted closing prices taken from Yahoo!). In this question, you will see how much money you could have made if you invested in these assets right after the 2008 financial crisis.

1. What are the simple 60-month (5-year) returns for the two investments?
2. What are the continuously compounded 60-month (5-year) returns for the two investments?
3. Are the simple and continuously compounded returns close to each other?
4. Suppose you invested \$1,000 in each asset at the end of January,
5. How much would each investment be worth at the end of January, 2015?
6. What is the compound annual return on the two 5 year investments?
7. At the end of January, 2010, suppose you have \$1,000 to invest in VFINX and AAPL over the next 60 months (5 years). Suppose you purchase \$400 worth of VFINX and the remainder in AAPL. What are the portfolio weights in the two assets? Using the results from parts 1 and 2, compute the 5-year simple and continuously compounded portfolio returns.

Exercise 1.8. Consider a one month investment in two Northwest stocks: Amazon and Costco. Suppose you buy Amazon and Costco at the end of September at $P_{A,t-1} = \$38.23$, $P_{C,t-1} = \$41.11$ and then sell at the end of the October for $P_{A,t} = \$41.29$, $P_{C,t} = \$41.74$. (Note: these are actual closing prices for 2004 taken from Yahoo!)

1. What are the simple monthly returns for the two stocks?
2. What are the continuously compounded returns for the two stocks?
3. Suppose Costco paid a \$0.10 per share cash dividend at the end of October. What is the monthly simple total return on Costco? What is the monthly dividend yield?
4. Suppose the monthly returns on Amazon and Costco from question (a) above are the same every month for 1 year. Compute the simple annual returns as well as the continuously compounded annual returns for the two stocks.
5. At the end of September, 2004, suppose you have \$10,000 to invest in Amazon and Costco over the next month. If you invest \$8,000 in Amazon and \$2,000 in Costco, what are your portfolio shares, x_A and x_C . (Assume partial share purchases are possible)
6. Continuing with the previous question, compute the monthly simple return and the monthly continuously compounded return on the portfolio. Assume that Costco does not pay a dividend.

Exercise 1.9. Consider an investment in a foreign stock (e.g., a stock trading on the London stock exchange) by a U.S. national (domestic investor). The domestic investor takes U.S. dollars, converts them to the foreign currency (e.g. British Pound) via the exchange rate (price of foreign currency in U.S. dollars) and then purchases the foreign stock using the foreign currency. When the stock is sold, the proceeds in the foreign currency must then be converted back to the domestic currency. To be more precise, consider the information in the table below:

Time	Cost of 1 Pound	Value of UK Shares	Value in U.S. \$
0	\$1.50	£40	$1.5 \times 40 = 60$
1	\$1.30	£45	$1.3 \times 45 = 58.5$

1. Compute the simple rate of return, R_e , from the prices of foreign currency. This is the return to the domestic investor of investing in the foreign currency.
2. Compute the simple rate of return, R_{UK} , from the UK stock prices.
3. Compute the simple rate of return, R_{US} , from the prices in US dollars.
4. What is the mathematical relationship between R_{US} , R_{UK} and R_e ?

Exercise 1.10. Consider an investing in a long-only portfolio of two assets A and B for one period between months $t-1$ and t . The beginning of period investment values in A and B are $V_{A,t-1} > 0$ and $V_{B,t-1} > 0$, and the one-month returns are $R_{A,t-1}$ and $R_{B,t-1}$. The initial value of the portfolio is $V_{p,t-1} = V_{A,t-1} + V_{B,t-1}$. The initial investment weights are $x_{A,t-1} = V_{A,t-1}/V_{p,t-1} > 0$ and $x_{B,t-1} = V_{B,t-1}/V_{p,t-1} > 0$ such that $x_{A,t-1} + x_{B,t-1} = 1$.

1. Show that

$$x_{A,t} = x_{A,t-1} \times \frac{1 + R_{A,t-1}}{1 + R_{p,t-1}}, \quad x_{B,t} = x_{B,t-1} \times \frac{1 + R_{B,t-1}}{1 + R_{p,t-1}},$$

and that $x_{A,t} + x_{B,t} = 1$.

2. Show that the change in portfolio weights are

$$\Delta x_{A,t} = x_{A,t} - x_{A,t-1} = \frac{x_{A,t-1}(R_{A,t-1} - R_{p,t-1})}{1 + R_{p,t-1}},$$

$$\Delta x_{B,t} = x_{B,t} - x_{B,t-1} = \frac{x_{B,t-1}(R_{B,t-1} - R_{p,t-1})}{1 + R_{p,t-1}}.$$

3. Under what conditions will $\Delta x_{A,t} > 0$, $\Delta x_{A,t} < 0$ and $\Delta x_{A,t} = 0$?
4. Using $R_{p,t-1} = x_{A,t-1}R_{A,t-1} + x_{B,t-1}R_{B,t-1}$ show that

$$\Delta x_{A,t} = \frac{x_{A,t-1}x_{B,t-1}(R_{A,t-1} - R_{B,t-1})}{1 + x_{A,t-1}R_{A,t-1} + x_{B,t-1}R_{B,t-1}}$$

5. Under what conditions will $\Delta x_{A,t} > 0$, $\Delta x_{A,t} < 0$ and $\Delta x_{A,t} = 0$?
6. To rebalance the portfolio at the end-of-month, you need to adjust the portfolio weights such that $x_{A,t} = x_{A,t-1}$ and $x_{B,t} = x_{B,t-1}$. If $R_{A,t-1} > R_{B,t-1}$ do you buy or sell asset A to rebalance the portfolio?

Return Calculations using R Packages

- Give examples of using the **PerformanceAnalytics** package

- Download different asset data from Yahoo!. Extract adjusted closing price data. Compute returns and plot returns and equity curves.

2 Review of Random Variables

Updated: September 1, 2026 Copyright © Eric Zivot 2026

In this chapter I give a review of probability concepts that are necessary for the probability modeling of asset returns and for the statistical analysis of financial data presented in this book. Much of this material is typically covered in an introductory probability and statistics course using calculus. In the course of the review, I will present many examples related to finance and risk management and some important risk concepts, such as value-at-risk, will be introduced. In the examples, I will also show how to use R for probability calculations and for working with probability distributions.

The chapter is outlined as follows. Section [2.1](#) reviews univariate random variables and properties of univariate probability distributions. Important univariate distributions, such as the normal distribution, that are used throughout the book for modeling financial data, are described in detail. Section [2.2](#) covers bivariate probability distributions for two random variables. The concepts of dependence and independence between two random variables are discussed. The measures of linear dependence between two random variables, covariance and correlation, are defined and the bivariate normal distribution is introduced. Properties of linear combinations of two random variables are given and illustrated using an example to describe the return and risk properties of a portfolio of two assets. More than two random variables and Multivariate distributions are briefly discussed in section [2.3](#).

The R packages I use in this chapter are **mvtnorm** and **sn**. Make sure these packages are downloaded and installed prior to replicating the R examples in this chapter.

2.1 Random Variables

I start with the basic definition of a *random variable*:

Definition 2.1 (Random variable). A *random variable*, X , is a variable that can take on a given set of values, called the *sample space* and denoted S_X , where the likelihood of the values in S_X is determined by X 's *probability distribution function* (pdf).

Example 2.1 (Future price of Microsoft stock). Consider the price of Microsoft stock next month, P_{t+1} . Since P_{t+1} is not known with certainty today (t), you can consider it as a random variable.

P_{t+1} must be positive, and realistically it can't get too large. Therefore, the sample space is the set of positive real numbers bounded above by some large number: $S_{P_{t+1}} = \{P_{t+1} : P_{t+1} \in [0, M], M > 0\}$. It is an open question as to what is the best characterization of the probability distribution of stock prices. The log-normal distribution is one possibility.¹ ■

Example 2.2 (Return on Microsoft stock). Consider a one-month investment in Microsoft stock. That is, you buy one share of Microsoft stock at the end of month $t - 1$ (e.g., end of February) and plan to sell it at the end of month t (e.g., end of March). The simple return over month t on this investment, $R_t = (P_t - P_{t-1})/P_{t-1}$, is a random variable because you do not know what the price will be at the end of the month. In contrast to prices, returns can be positive or negative and are bounded from below by -100%. You can express the sample space as $S_{R_t} = \{R_t : R_t \in [-1, M], M > 0\}$. The normal distribution is often a good approximation to the distribution of simple monthly returns, and is a better approximation to the distribution of continuously compounded monthly returns. ■

Example 2.3 (Up-down indicator variable). As a final example, consider a variable X_{t+1} defined to be equal to one if the monthly price change on Microsoft stock, $P_{t+1} - P_t$, is positive, and is equal to zero if the price change is zero or negative. Here, the sample space is the set $S_{X_{t+1}} = \{0, 1\}$. If it is equally likely that the monthly price change is positive or negative (including zero) then the probability that $X_{t+1} = 1$ or $X_{t+1} = 0$ is 0.5. This is an example of a Bernoulli random variable. ■

The next sub-sections define discrete and continuous random variables.

2.1.1 Discrete random variables

Consider a random variable generically denoted X and its set of possible values or sample space denoted S_X .

Definition 2.2 (Discrete random variable). A *discrete random variable* X is one that can take on a finite number of n different values $S_X = \{x_1, x_2, \dots, x_n\}$ or, at most, a countably infinite number of different values $S_X = \{x_1, x_2, \dots\}$.

Definition 2.3 (Probability density function). The pdf of a discrete random variable, denoted $p(x)$, is a function such that $p(x) = \Pr(X = x)$. The pdf must satisfy (i) $p(x) \geq 0$ for all $x \in S_X$; (ii) $p(x) = 0$ for all $x \notin S_X$; and (iii) $\sum_{x \in S_X} p(x) = 1$.

¹If P_{t+1} is a positive random variable such that $\ln P_{t+1}$ is normally distributed then P_{t+1} has a log-normal distribution.

Example 2.4 (Annual return on Microsoft stock).

Table 2.1: Probability distribution for the annual return on Microsoft.

State of Economy	Sample Space	$\Pr(X = x)$
Depression	-0.30	0.05
Recession	0.0	0.20
Normal	0.10	0.50
Mild Boom	0.20	0.20
Major Boom	0.50	0.05

Let X denote the annual return on Microsoft stock over the next year. You might hypothesize that the annual return will be influenced by the general state of the economy. Consider five possible states of the economy: depression, recession, normal, mild boom and major boom. A stock analyst might forecast different values of the return for each possible state. Hence, X is a discrete random variable that can take on five different values. Table 2.1 describes such a probability distribution of the return. A graphical representation of the probability distribution is created with the following R code:

```
r.msft = c(-0.3, 0, 0.1, 0.2, 0.5)
prob.vals = c(0.05, 0.20, 0.50, 0.20, 0.05)
plot(r.msft, prob.vals, type="h", lwd=4, xlab="annual return",
     ylab="probability", xaxt="n")
points(r.msft, prob.vals, pch=16, cex=2, col="blue")
axis(1, at=r.msft)
```

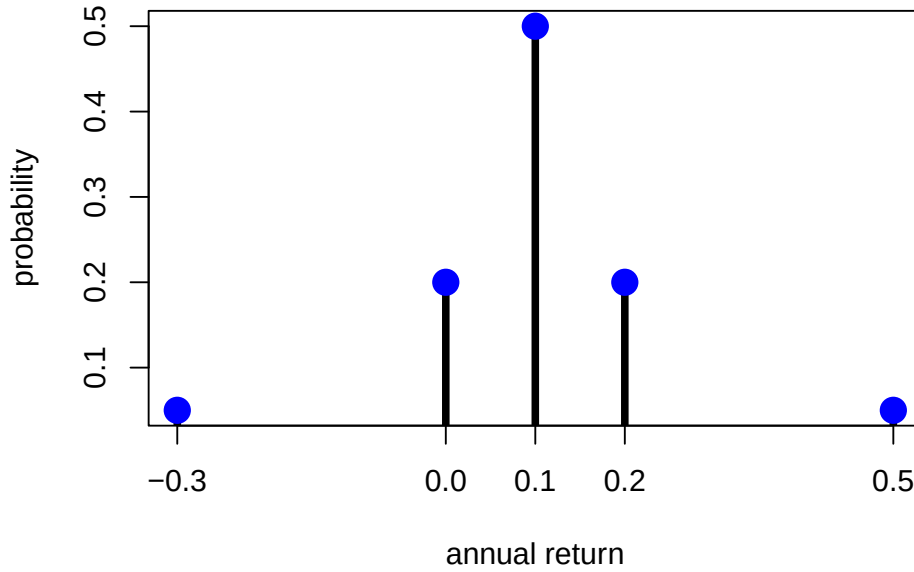



Figure 2.1: Discrete distribution for Microsoft stock.

This distribution is presented in Figure Figure 2.1. The shape of the distribution is revealing. The center and most likely value is at 0.1 and the distribution is symmetric about this value (has the same shape to the left and to the right). Most of the probability is concentrated between 0 and 0.2 and there is very little probability associated with very large or small returns. ■

2.1.1.1 The Bernoulli distribution

Let $X = 1$ if the price next month of Microsoft stock goes up and $X = 0$ if the price goes down (assuming it cannot stay the same). Then X is clearly a discrete random variable with sample space $S_X = \{0, 1\}$. If the probability of the stock price going up or down is the same then $p(0) = p(1) = 1/2$ and $p(0) + p(1) = 1$.

The probability distribution described above can be given an exact mathematical representation known as the *Bernoulli* distribution. Consider two mutually exclusive events generically called “success” and “failure”. For example, a success could be a stock price going up or a coin landing heads and a failure could be a stock price going down or a coin landing tails. The process creating the success or failure is called a *Bernoulli trial*. In general, let $X = 1$ if success occurs and let $X = 0$ if failure occurs. Let $\Pr(X = 1) = \pi$, where $0 < \pi < 1$, denote the probability of success. Then $\Pr(X = 0) = 1 - \pi$ is the probability of failure. A mathematical model describing this distribution

is:

$$p(x) = \Pr(X = x) = \pi^x(1 - \pi)^{1-x}, \quad x = 0, 1. \quad (2.1)$$

When $x = 0$, $p(0) = \pi^0(1 - \pi)^{1-0} = 1 - \pi$ and when $x = 1$, $p(1) = \pi^1(1 - \pi)^{1-1} = \pi$.

2.1.1.2 The binomial distribution

Consider a sequence of independent Bernoulli trials with success probability π generating a sequence of 0-1 variables indicating failures and successes. A *binomial* random variable X counts the number of successes in n Bernoulli trials, and is denoted $X \sim B(n, \pi)$. The sample space is $S_X = \{0, 1, \dots, n\}$ and:

$$\Pr(X = k) = \binom{n}{k} \pi^k (1 - \pi)^{n-k}.$$

The term $\binom{n}{k}$ is the binomial coefficient, and counts the number of ways k objects can be chosen from n distinct objects. It is defined by:

$$\binom{n}{k} = \frac{n!}{(n-k)!k!},$$

where $n!$ is the factorial of n , or $n(n-1) \cdots 2 \cdot 1$.

2.1.2 Continuous random variables

Definition 2.4 (Continuous random variable). A *continuous random variable* X is one that can take on any real value. That is, $S_X = \{x : x \in \mathbb{R}\}$.

Definition 2.5 (Probability density function for continuous random variable). The probability density function (pdf) of a continuous random variable X is a nonnegative function f , defined on the real line, such that for any interval A :

$$\Pr(X \in A) = \int_A f(x) dx.$$

That is, $\Pr(X \in A)$ is the area under the probability curve over the interval A . The pdf $f(x)$ must satisfy (i) $f(x) \geq 0$; and (ii) $\int_{-\infty}^{\infty} f(x) dx = 1$.

A typical “bell-shaped” pdf is displayed in Figure 2.2 and the shaded area under the curve between -2 and 1 represents $\Pr(-2 \leq X \leq 1)$. For a continuous random variable, $f(x) \neq \Pr(X = x)$ but rather gives the height of the probability curve at x . In fact, $\Pr(X = x) = 0$ for all values of x . That is, probabilities are not defined over single points. They are only defined over intervals. As a result, for a continuous random variable X :

$$\Pr(a \leq X \leq b) = \Pr(a < X \leq b) = \Pr(a < X < b) = \Pr(a \leq X < b).$$

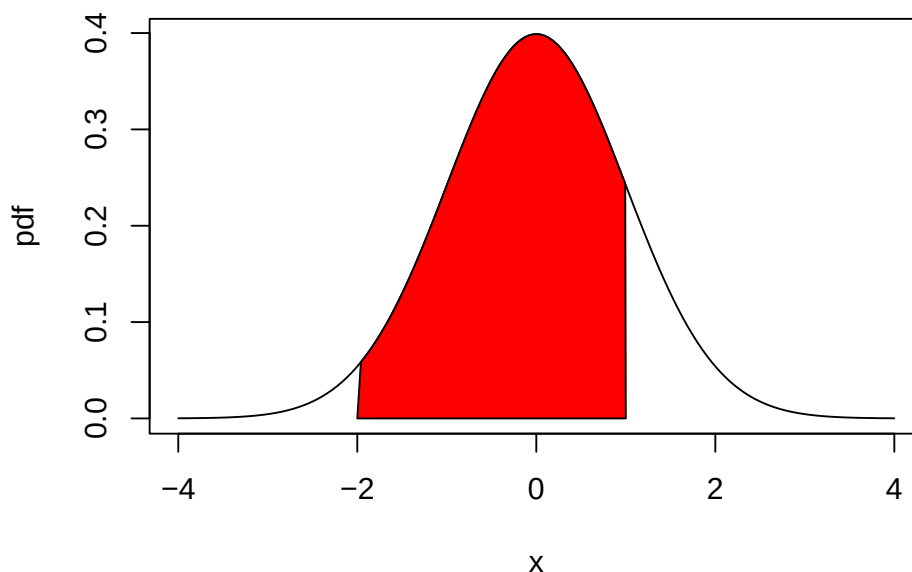


Figure 2.2: $Pr(-2 \leq X \leq 1)$ is represented by the area under the probability curve.

2.1.2.1 The uniform distribution on an interval

Let X denote the annual return on Microsoft stock and let a and b be two real numbers such that $a < b$. Suppose that the annual return on Microsoft stock can take on any value between a and b . That is, the sample space is restricted to the interval $S_X = \{x \in \mathcal{R} : a \leq x \leq b\}$. Further suppose that the probability that X will belong to any subinterval of S_X is proportional to the length of the interval. In this case, it is said that X is *uniformly distributed* on the interval $[a, b]$. The pdf of X has a very simple mathematical form:

$$f(x) = \begin{cases} \frac{1}{b-a} & \text{for } a \leq x \leq b, \\ 0 & \text{otherwise,} \end{cases}$$

and is presented graphically as a rectangle. It is easy to see that the area under the curve over the interval $[a, b]$ (area of rectangle) integrates to one:

$$\int_a^b \frac{1}{b-a} dx = \frac{1}{b-a} \int_a^b dx = \frac{1}{b-a} [x]_a^b = \frac{1}{b-a} [b-a] = 1.$$

Example 2.5 (Uniform distribution on $[-1, 1]$). Let $a = -1$ and $b = 1$, so that $b - a = 2$. Consider

computing the probability that X will be between -0.5 and 0.5. You must solve:

$$\Pr(-0.5 \leq X \leq 0.5) = \int_{-0.5}^{0.5} \frac{1}{2} dx = \frac{1}{2} [x]_{-0.5}^{0.5} = \frac{1}{2} [0.5 - (-0.5)] = \frac{1}{2}.$$

Next, consider computing the probability that the return will fall in the interval $[0, \delta]$ where δ is some small number less than $b = 1$:

$$\Pr(0 \leq X \leq \delta) = \frac{1}{2} \int_0^{\delta} dx = \frac{1}{2} [x]_0^{\delta} = \frac{1}{2} \delta.$$

As $\delta \rightarrow 0$, $\Pr(0 \leq X \leq \delta) \rightarrow \Pr(X = 0)$.

Using the above result notice that

$$\lim_{\delta \rightarrow 0} \Pr(0 \leq X \leq \delta) = \Pr(X = 0) = \lim_{\delta \rightarrow 0} \frac{1}{2} \delta = 0.$$

Hence, for continuous random variables probabilities are defined on intervals but not at distinct points. ■

2.1.2.2 The standard normal distribution

The normal or Gaussian distribution is perhaps the most famous and most useful continuous distribution in all of statistics. The shape of the normal distribution is the familiar “bell curve”. As you shall see, it can be used to describe the probabilistic behavior of stock returns although other distributions may be more appropriate.

If a random variable X follows a *standard normal distribution* then you often write $X \sim N(0, 1)$ as short-hand notation. This distribution is centered at zero and has inflection points at ± 1 .² The pdf of a standard normal random variable is given by:

$$f(x) = \frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{1}{2}x^2}, \quad -\infty \leq x \leq \infty. \quad (2.2)$$

It can be shown via the change of variables formula in calculus that the area under the standard normal curve is one:

$$\int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{1}{2}x^2} dx = 1.$$

The standard normal distribution is illustrated in Figure Figure 2.3. Notice that the distribution is *symmetric* about zero; i.e., the distribution has exactly the same form to the left and right of zero. Because the standard normal pdf formula is used so often it is given its own special symbol $\phi(x)$.

²An inflection point is a point where a curve goes from concave up to concave down, or vice versa.

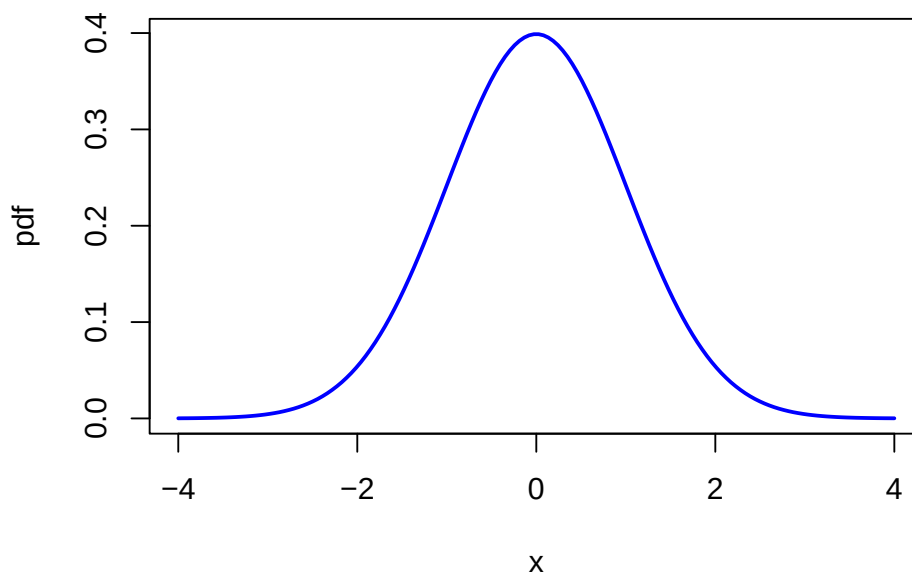


Figure 2.3: Standard normal density.

The normal distribution has the annoying feature that the area under the normal curve cannot be evaluated analytically. That is:

$$\Pr(a \leq X \leq b) = \int_a^b \frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{1}{2}x^2} dx,$$

does not have a closed form solution. The above integral must be computed by numerical approximation. Areas under the normal curve, in one form or another, are given in tables in almost every introductory statistics book and standard statistical software can be used to find these areas. Some useful approximate results are:

$$\Pr(-1 \leq X \leq 1) \approx 0.67,$$

$$\Pr(-2 \leq X \leq 2) \approx 0.95,$$

$$\Pr(-3 \leq X \leq 3) \approx 0.99.$$

2.1.3 The cumulative distribution function

Definition 2.6 (Cumulative distribution function). The *cumulative distribution function* (cdf) of a random variable X (discrete or continuous), denoted F_X , is the probability that $X \leq x$:

$$F_X(x) = \Pr(X \leq x), \quad -\infty \leq x \leq \infty.$$

The cdf has the following properties:

- (i) If $x_1 < x_2$ then $F_X(x_1) \leq F_X(x_2)$
- (ii) $F_X(-\infty) = 0$ and $F_X(\infty) = 1$
- (iii) $\Pr(X > x) = 1 - F_X(x)$
- (iv) $\Pr(x_1 < X \leq x_2) = F_X(x_2) - F_X(x_1)$
- (v) $F'_X(x) = \frac{d}{dx}F_X(x) = f(x)$ if X is a continuous random variable and $F_X(x)$ is continuous and differentiable.

Example 2.6 (CDF for a discrete random variable). The cdf for the discrete distribution of Microsoft from Table Table 2.1 is given by:

$$F_X(x) = \begin{cases} 0, & x < -0.3 \\ 0.05, & -0.3 \leq x < 0 \\ 0.25, & 0 \leq x < 0.1 \\ 0.75, & 0.1 \leq x < 0.2 \\ 0.95, & 0.2 \leq x < 0.5 \\ 1, & x > 0.5 \end{cases}$$

and is illustrated in Figure Figure 2.4. ■

Example 2.7 (CDF for a uniform random variable). The cdf for the uniform distribution over $[a, b]$ can be determined analytically:

$$\begin{aligned} F_X(x) &= \Pr(X < x) = \int_{-\infty}^x f(t) dt \\ &= \frac{1}{b-a} \int_a^x dt = \frac{1}{b-a} [t]_a^x = \frac{x-a}{b-a}. \end{aligned}$$

You can determine the pdf of X directly from the cdf via,

$$f(x) = F'_X(x) = \frac{d}{dx}F_X(x) = \frac{1}{b-a}.$$
■

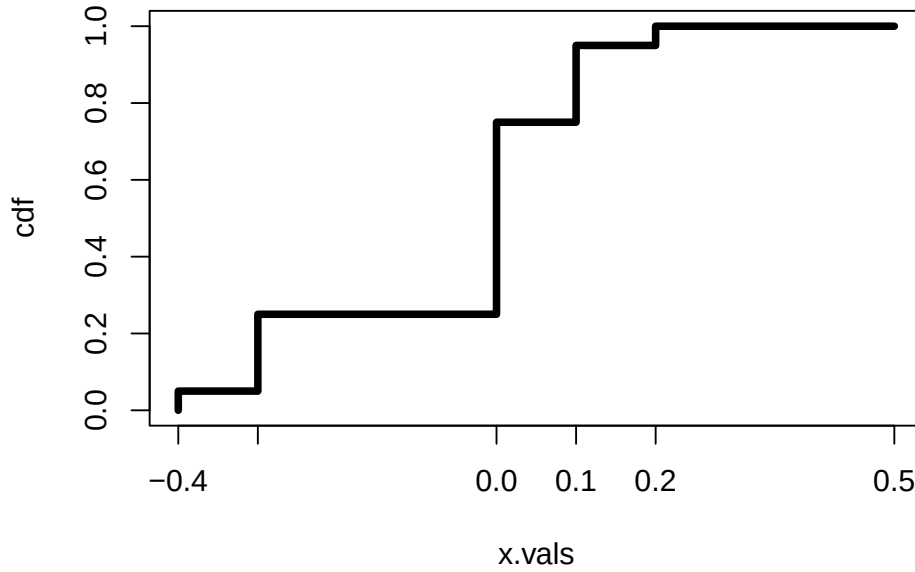


Figure 2.4: CDF of Discrete Distribution for Microsoft Stock Return.

Example 2.8 (CDF for a standard normal random variable). The cdf of standard normal random variable X is used so often in statistics that it is given its own special symbol:

$$\Phi(x) = F_X(x) = \int_{-\infty}^x \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2} dz. \quad (2.3)$$

The cdf $\Phi(x)$, however, does not have an analytic representation like the cdf of the uniform distribution and so the integral in Equation 2.3 must be approximated using numerical techniques. A graphical representation of $\Phi(x)$ is given in Figure Figure 2.5. Because the standard normal pdf, $\phi(x)$, is symmetric about zero and bell-shaped the standard normal CDF, $\Phi(x)$, has an “S” shape where the middle of the “S” is at zero. ■

2.1.4 Quantiles of the distribution of a random variable

Definition 2.7 (Quantiles of the distribution of a random variable). Consider a random variable X with continuous cdf $F_X(x)$. For $0 \leq \alpha \leq 1$, the $100 \cdot \alpha\%$ *quantile* of the distribution for X is the value q_α that satisfies:

$$F_X(q_\alpha) = \Pr(X \leq q_\alpha) = \alpha.$$

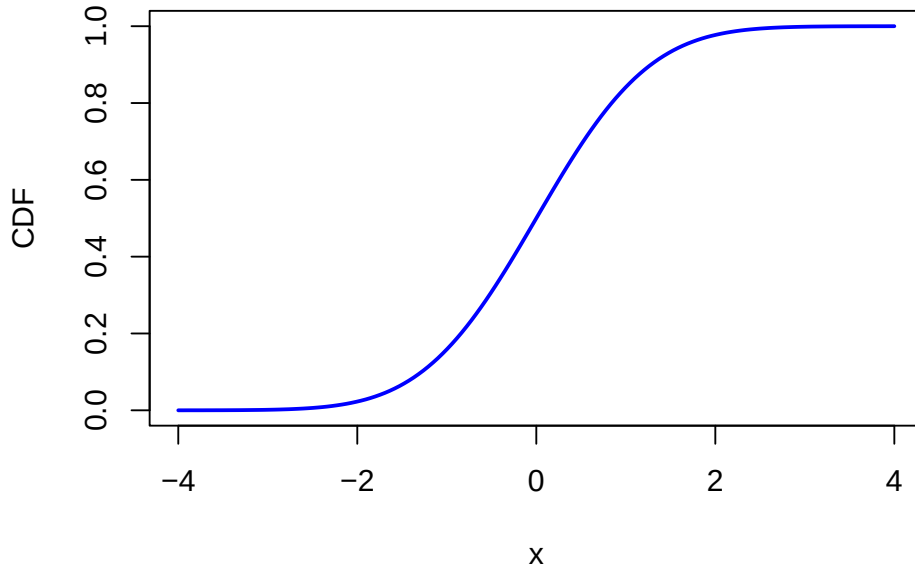


Figure 2.5: Standard normal cdf $\Phi(x)$.

For example, the 5% quantile of X , $q_{0.05}$, satisfies,

$$F_X(q_{0.05}) = \Pr(X \leq q_{0.05}) = 0.05.$$

The *median* of the distribution is the 50% quantile. That is, the median, $q_{0.5}$, satisfies,

$$F_X(q_{0.5}) = \Pr(X \leq q_{0.5}) = 0.5.$$

If F_X is invertible then q_α may be determined analytically as:

$$q_\alpha = F_X^{-1}(\alpha) \tag{2.4}$$

where F_X^{-1} denotes the inverse function of F_X .³ Hence, the 5% quantile and the median may be determined from

$$q_{0.05} = F_X^{-1}(.05), q_{0.5} = F_X^{-1}(.5).$$

The inverse cdf F_X^{-1} is sometimes called the *quantile function*.

³The inverse function of F_X , denoted F_X^{-1} , has the property that $F_X^{-1}(F_X(x)) = x$. F_X^{-1} will exist if F_X is strictly increasing and is continuous.

Example 2.9 (Quantiles from a uniform distribution). Let $X \sim U[a, b]$ where $b > a$. Recall, the cdf of X is given by:

$$F_X(x) = \frac{x-a}{b-a}, \quad a \leq x \leq b,$$

which is continuous and strictly increasing. Given $\alpha \in [0, 1]$ such that $F_X(x) = \alpha$, solving for x gives the inverse cdf:

$$x = F_X^{-1}(\alpha) = \alpha(b-a) + a. \quad (2.5)$$

Using Equation 2.5, the 5% quantile and median, for example, are given by:

$$\begin{aligned} q_{0.05} &= F_X^{-1}(.05) = .05(b-a) + a = .05b + .95a, \\ q_{0.5} &= F_X^{-1}(.5) = .5(b-a) + a = .5(a+b). \end{aligned}$$

If $a = 0$ and $b = 1$, then $q_{0.05} = 0.05$ and $q_{0.5} = 0.5$. ■

Example 2.10 (Quantiles from a standard normal distribution). Let $X \sim N(0, 1)$. The quantiles of the standard normal distribution are determined by solving

$$q_\alpha = \Phi^{-1}(\alpha), \quad (2.6)$$

where Φ^{-1} denotes the inverse of the cdf Φ . This inverse function must be approximated numerically and is available in most spreadsheets and statistical software. Using the numerical approximation to the inverse function, the 1%, 2.5%, 5%, 10% quantiles and median are given by:

$$\begin{aligned} q_{0.01} &= \Phi^{-1}(.01) = -2.33, \quad q_{0.025} = \Phi^{-1}(.025) = -1.96, \\ q_{0.05} &= \Phi^{-1}(.05) = -1.645, \quad q_{0.10} = \Phi^{-1}(.10) = -1.28, \\ q_{0.5} &= \Phi^{-1}(.5) = 0. \end{aligned}$$

Often, the standard normal quantile is denoted z_α . ■

2.1.5 R functions for discrete and continuous distributions

R has built-in functions for a number of discrete and continuous distributions that are commonly used in probability modeling and statistical analysis. These are summarized in Table 2.2. For each distribution, there are four functions starting with **d**, **p**, **q**, and **r** that compute density (pdf) values, cumulative probabilities (cdf), quantiles (inverse cdf) and random draws, respectively. Consider, for example, the functions associated with the normal distribution. The functions **dnorm()**, **pnorm()** and **qnorm()** evaluate the standard normal density Equation 2.2, the cdf Equation 2.3, and the inverse cdf or quantile function Equation 2.6, respectively, with the default values **mean** = 0 and **sd** = 1. The function **rnorm()** returns a specified number of simulated values from the normal distribution.

Table 2.2: Probability distributions in base R.

Distribution	Function (root)	Parameters	Defaults
beta	beta	shape1, shape2	—, —
binomial	binom	size, prob	—, —
cauchy	cauchy	location, scale	0, 1
chi-squared	chisq	df, ncp	—, 1
F	f	df1, df2	—, —
gamma	gamma	shape, rate, scale	—, 1, 1/rate
geometric	geom	prob	—
hyper-geometric	hyper	m, n, k	—, —, —
log-normal	lnorm	meanlog, sdlog	0, 1
logistic	logis	location, scale	0, 1
negative binomial	nbinom	size, prob, mu	—, —, —
normal	norm	mean, sd	0, 1
Poisson	pois	Lambda	1
Student's t	t	df, ncp	—, 1
uniform	unif	min, max	0, 1
Weibull	weibull	shape, scale	—, 1
Wilcoxon	wilcoxon	m, n	—, —

Commonly used distributions used in finance application are the binomial, chi-squared, log normal, normal, and Student's t.

2.1.5.1 Finding areas under the standard normal curve

Let X denote a standard normal random variable. Given that the total area under the normal curve is one and the distribution is symmetric about zero, the following results hold:

- $\Pr(X \leq z) = 1 - \Pr(X \geq z)$ and $\Pr(X \geq z) = 1 - \Pr(X \leq z)$
- $\Pr(X \geq z) = \Pr(X \leq -z)$
- $\Pr(X \geq 0) = \Pr(X \leq 0) = 0.5$

The following examples show how to do probability calculations with standard normal random variables.

Example 2.11 (Finding areas under the normal curve using R). First, consider finding $\Pr(X \geq 2)$. By the symmetry of the normal distribution, $\Pr(X \geq 2) = \Pr(X \leq -2) = \Phi(-2)$. In R use:

```
pnorm(-2)
#> [1] 0.0228
```

Next, consider finding $\Pr(-1 \leq X \leq 2)$. Using the cdf, you compute $\Pr(-1 \leq X \leq 2) = \Pr(X \leq 2) - \Pr(X \leq -1) = \Phi(2) - \Phi(-1)$. In R use:

```
pnorm(2) - pnorm(-1)
#> [1] 0.819
```

Finally, using R the exact values for $\Pr(-1 \leq X \leq 1)$, $\Pr(-2 \leq X \leq 2)$ and $\Pr(-3 \leq X \leq 3)$ are:

```
pnorm(1) - pnorm(-1)
#> [1] 0.683
pnorm(2) - pnorm(-2)
#> [1] 0.954
pnorm(3) - pnorm(-3)
#> [1] 0.997
```

■

2.1.5.2 Plotting distributions

When working with a probability distribution, it is a good idea to make plots of the pdf or cdf to reveal important characteristics. The following examples illustrate plotting distributions using R.

Example 2.12 (Plotting the standard normal curve). The graphs of the standard normal pdf and cdf in Figures Figure 2.3 and Figure 2.5 were created using the following R code:

```
# plot pdf
x.vals = seq(-4, 4, length=150)
plot(x.vals, dnorm(x.vals), type="l", lwd=2, col="blue",
      xlab="x", ylab="pdf")
# plot cdf
plot(x.vals, pnorm(x.vals), type="l", lwd=2, col="blue",
      xlab="x", ylab="CDF")
```

■

Example 2.13 (Shading a region under the standard normal curve). Figure Figure 2.2 showing $\Pr(-2 \leq X \leq 1)$ as a red shaded area is created with the following code:

```

lb = -2
ub = 1
x.vals = seq(-4, 4, length=150)
d.vals = dnorm(x.vals)
# plot normal density
plot(x.vals, d.vals, type="l", xlab="x", ylab="pdf")
i = x.vals >= lb & x.vals <= ub
# add shaded region between -2 and 1
polygon(c(lb, x.vals[i], ub), c(0, d.vals[i], 0), col="red")

```



2.1.6 Shape characteristics of probability distributions

Very often you would like to know certain shape characteristics of a probability distribution. You might want to know where the distribution is centered, and how spread out the distribution is about the central value. You might want to know if the distribution is symmetric about the center or if the distribution has a long left or right tail. For stock returns you might want to know about the likelihood of observing extreme values for returns representing market crashes. This means that you would like to know about the amount of probability in the extreme tails of the distribution. In this section I will discuss four important shape characteristics of a probability distribution:

1. *expected value* (mean): measures the center of mass of a distribution
2. *variance* and *standard deviation* (volatility): measures the spread about the mean
3. *skewness*: measures symmetry about the mean
4. *kurtosis*: measures “tail thickness”

2.1.6.1 Expected Value

The *expected value* of a random variable X , denoted $E[X]$ or μ_X , measures the center of mass of the pdf.

Definition 2.8 (Expected value of discrete random variable). For a discrete random variable X with sample space S_X , the expected value is defined as:

$$\mu_X = E[X] = \sum_{x \in S_X} x \cdot \Pr(X = x). \quad (2.7)$$

Equation Equation 2.7 shows that $E[X]$ is a probability weighted average of the possible values of X .

Example 2.14 (Expected value of discrete random variable). Using the discrete distribution for the return on Microsoft stock in Table 2.1, the expected return is computed as:

$$\begin{aligned} E[X] &= (-0.3) \cdot (0.05) + (0.0) \cdot (0.20) + (0.1) \cdot (0.5) \\ &\quad + (0.2) \cdot (0.2) + (0.5) \cdot (0.05) \\ &= 0.10. \end{aligned}$$

■

Example 2.15 (Expected value of Bernoulli and binomial random variables). Let X be a Bernoulli random variable with success probability π . Then,

$$E[X] = 0 \cdot (1 - \pi) + 1 \cdot \pi = \pi.$$

That is, the expected value of a Bernoulli random variable is its probability of success. Now, let $Y \sim B(n, \pi)$. It can be shown that:

$$E[Y] = \sum_{k=0}^n k \binom{n}{k} \pi^k (1 - \pi)^{n-k} = n\pi. \quad (2.8)$$

■

Definition 2.9 (Expected value of a continuous random variable). For a continuous random variable X with pdf $f(x)$, the *expected value* is defined as:

$$\mu_X = E[X] = \int_{-\infty}^{\infty} x \cdot f(x) \, dx. \quad (2.9)$$

Example 2.16 (Expected value of a uniform random variable). Suppose X has a uniform distribution over the interval $[a, b]$. Then,

$$\begin{aligned} E[X] &= \frac{1}{b-a} \int_a^b x \, dx = \frac{1}{b-a} \left[\frac{1}{2} x^2 \right]_a^b \\ &= \frac{1}{2(b-a)} [b^2 - a^2] \\ &= \frac{(b-a)(b+a)}{2(b-a)} = \frac{b+a}{2}. \end{aligned}$$

If $b = -1$ and $a = 1$, then $E[X] = 0$.

■

Example 2.17 (Expected value of a standard normal random variable). Let $X \sim N(0, 1)$. Then it can be shown that:

$$E[X] = \int_{-\infty}^{\infty} x \cdot \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} dx = 0.$$

Hence, the standard normal distribution is centered at zero. ■

2.1.6.2 Expectation of a function of a random variable

The other shape characteristics of the distribution of a random variable X are based on expectations of certain functions of X .

Definition 2.10 (Expected value of a function of a random variable). Let $g(X)$ denote some function of the random variable X . If X is a discrete random variable with sample space S_X then:

$$E[g(X)] = \sum_{x \in S_X} g(x) \cdot \Pr(X = x), \quad (2.10)$$

and if X is a continuous random variable with pdf f then,

$$E[g(X)] = \int_{-\infty}^{\infty} g(x) \cdot f(x) dx. \quad (2.11)$$

2.1.6.3 Variance and standard deviation

The variance of a random variable X , denoted $\text{var}(X)$ or σ_X^2 , measures the spread of the distribution about the mean using the function $g(X) = (X - \mu_X)^2$. If most values of X are close to μ_X then on average $(X - \mu_X)^2$ will be small. In contrast, if many values of X are far below and/or far above μ_X then on average $(X - \mu_X)^2$ will be large. Squaring the deviations about μ_X guarantees a positive value.

Definition 2.11 (Variance and standard deviation). For a discrete or continuous random variable X , the variance and standard deviation of X are defined as:

$$\sigma_X^2 = \text{var}(X) = E[(X - \mu_X)^2], \quad (2.12)$$

$$\sigma_X = \text{sd}(X) = \sqrt{\sigma_X^2} = \sigma_X. \quad (2.13)$$

Because σ_X^2 represents an average squared deviation, it is not in the same units as X . The standard deviation of X is the square root of the variance and is in the same units as X . For “bell-shaped” distributions, σ_X measures the typical size of a deviation from the mean value.

The computation of Equation 2.12 can often be simplified by using the result:

$$\text{var}(X) = E[(X - \mu_X)^2] = E[X^2] - \mu_X^2 \quad (2.14)$$

You will learn how to derive this result later in the chapter.

Example 2.18 (Variance and standard deviation for a discrete random variable). Using the discrete distribution for the return on Microsoft stock in Table 2.1 and the result that $\mu_X = 0.1$, it follows that:

$$\begin{aligned} \text{var}(X) &= (-0.3 - 0.1)^2 \cdot (0.05) + (0.0 - 0.1)^2 \cdot (0.20) + (0.1 - 0.1)^2 \cdot (0.5) \\ &\quad + (0.2 - 0.1)^2 \cdot (0.2) + (0.5 - 0.1)^2 \cdot (0.05) \\ &= 0.020. \end{aligned}$$

Alternatively, you can compute $\text{var}(X)$ using Equation 2.14:

$$\begin{aligned} E[X^2] - \mu_X^2 &= (-0.3)^2 \cdot (0.05) + (0.0)^2 \cdot (0.20) + (0.1)^2 \cdot (0.5) \\ &\quad + (0.2)^2 \cdot (0.2) + (0.5)^2 \cdot (0.05) - (0.1)^2 \\ &= 0.020. \end{aligned}$$

The standard deviation is $\sqrt{0.020} = 0.141$. Given that the distribution is fairly bell-shaped you can say that typical values deviate from the mean value of 10% by about $\pm 14.1\%$. ■

Example 2.19 (Variance and standard deviation of a Bernoulli and binomial random variables). Let X be a Bernoulli random variable with success probability π . Given that $\mu_X = \pi$ it follows that:

$$\begin{aligned} \text{var}(X) &= (0 - \pi)^2 \cdot (1 - \pi) + (1 - \pi)^2 \cdot \pi \\ &= \pi^2(1 - \pi) + (1 - \pi)^2\pi \\ &= \pi(1 - \pi)[\pi + (1 - \pi)] \\ &= \pi(1 - \pi), \\ \text{sd}(X) &= \sqrt{\pi(1 - \pi)}. \end{aligned}$$

Now, let $Y \sim B(n, \pi)$: It can be shown that:

$$\text{var}(Y) = n\pi(1 - \pi)$$

and so $\text{sd}(Y) = \sqrt{n\pi(1 - \pi)}$. ■

Example 2.20 (Variance and standard deviation of a uniform random variable). Let $X \sim U[a, b]$.

Using Equation 2.14 and $\mu_X = \frac{a+b}{2}$, after some algebra, it can be shown that:

$$\text{var}(X) = E[X^2] - \mu_X^2 = \frac{1}{b-a} \int_a^b x^2 dx - \left(\frac{a+b}{2}\right)^2 = \frac{1}{12}(b-a)^2,$$

and $\text{sd}(X) = (b-a)/\sqrt{12}$. ■

Example 2.21 (Variance and standard deviation of a standard normal random variable). Let $X \sim N(0, 1)$. Here, $\mu_X = 0$ and it can be shown that:

$$\sigma_X^2 = \int_{-\infty}^{\infty} x^2 \cdot \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} dx = 1.$$

It follows that $\text{sd}(X) = 1$. ■

2.1.6.4 The general normal distribution

Recall, if X has a standard normal distribution then $E[X] = 0$, $\text{var}(X) = 1$. A *general normal* random variable X has $E[X] = \mu_X$ and $\text{var}(X) = \sigma_X^2$ and is denoted $X \sim N(\mu_X, \sigma_X^2)$. Its pdf is given by:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma_X^2}} \exp\left\{-\frac{1}{2\sigma_X^2}(x - \mu_X)^2\right\}, \quad -\infty \leq x \leq \infty. \quad (2.15)$$

Showing that $E[X] = \mu_X$ and $\text{var}(X) = \sigma_X^2$ is a bit of work and is good calculus practice. As with the standard normal distribution, areas under the general normal curve cannot be computed analytically. Using numerical approximations, it can be shown that:

$$\Pr(\mu_X - \sigma_X < X < \mu_X + \sigma_X) \approx 0.67,$$

$$\Pr(\mu_X - 2\sigma_X < X < \mu_X + 2\sigma_X) \approx 0.95,$$

$$\Pr(\mu_X - 3\sigma_X < X < \mu_X + 3\sigma_X) \approx 0.99.$$

Hence, for a general normal random variable about 95% of the time you expect to see values within ± 2 standard deviations from its mean. Observations more than three standard deviations from the mean are very unlikely.

Example 2.22 (Normal distribution for monthly returns). Let R denote the monthly return on an investment in Microsoft stock, and assume that it is normally distributed with mean $\mu_R = 0.01$ and standard deviation $\sigma_R = 0.10$. That is, $R \sim N(0.01, (0.10)^2)$. Notice that $\sigma_R^2 = 0.01$ and is not in units of return per month. Figure 2.6 illustrates the distribution and is created using:

```
mu.r = 0.01
sd.r = 0.1
```



```
x.vals = seq(-4, 4, length=150)*sd.r + mu.r
plot(x.vals, dnorm(x.vals, mean=mu.r, sd=sd.r), type="l", lwd=2,
     col="blue", xlab="x", ylab="pdf")
```

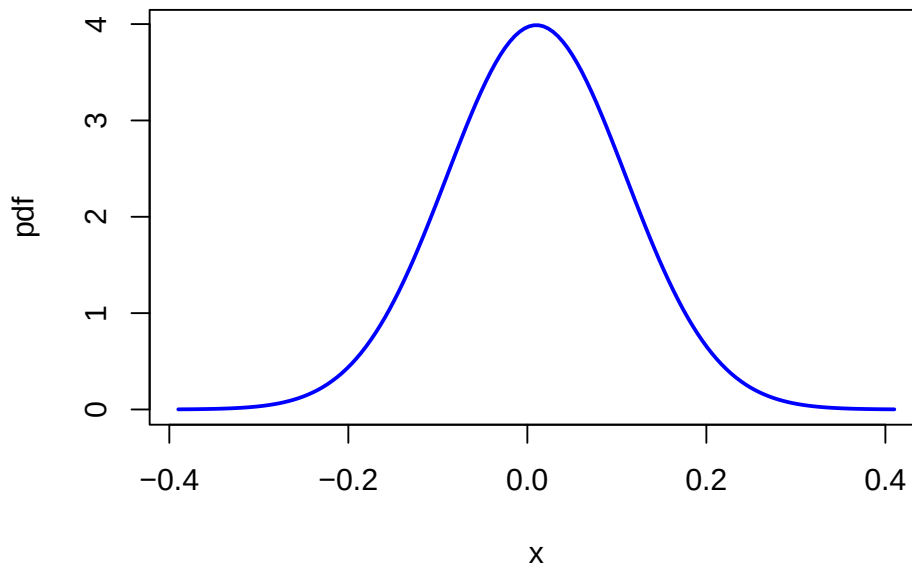


Figure 2.6: Normal distribution for the monthly returns on Microsoft: $R \sim N(0.01, (0.10)^2)$.

Notice that essentially all of the probability lies between -0.3 and 0.3 . Using the R function `pnorm()`, you can easily compute the probabilities $\Pr(R < -0.5)$, $\Pr(R < 0)$, $\Pr(R > 0.5)$ and $\Pr(R > 1)$:

```
pnorm(-0.5, mean=0.01, sd=0.1)
#> [1] 1.7e-07
pnorm(0, mean=0.01, sd=0.1)
#> [1] 0.46
1 - pnorm(0.5, mean=0.01, sd=0.1)
#> [1] 4.79e-07
1 - pnorm(1, mean=0.01, sd=0.1)
#> [1] 0
```

Using the R function `qnorm()`, you can find the quantiles $q_{0.01}$, $q_{0.05}$, $q_{0.95}$ and $q_{0.99}$:

```
a.vals = c(0.01, 0.05, 0.95, 0.99)
qnorm(a.vals, mean=0.01, sd=0.10)
#> [1] -0.223 -0.154  0.174  0.243
```

Hence, over the next month, there are 1% and 5% chances of losing more than 22.2% and 15.5%, respectively. In addition, there are 5% and 1% chances of gaining more than 17.5% and 24.3%, respectively. ■

Example 2.23 (Risk-return tradeoff). Consider the following investment problem. You can invest in two non-dividend paying stocks, Amazon and Boeing, over the next month. Let R_A denote the monthly return on Amazon and R_B denote the monthly return on Boeing. Assume that $R_A \sim N(0.02, (0.10)^2)$ and $R_B \sim N(0.01, (0.05)^2)$. Figure 2.7 shows the pdfs for the two returns. Notice that $\mu_A = 0.02 > \mu_B = 0.01$ but also that $\sigma_A = 0.10 > \sigma_B = 0.05$. The return you expect on Amazon is bigger than the return you expect on Boeing but the variability of Amazon's return is also greater than the variability of Boeing's return. The high return variability (volatility) of Amazon reflects the higher risk associated with investing in Amazon compared to investing in Boeing. If you invest in Boeing you get a lower expected return, but you also get less return variability or risk. For example, the probability of losing more than 10% in one month for Amazon is

```
pnorm(-0.1, 0.02, 0.1)
#> [1] 0.115
```

and the probability of losing more than 10% in one month for Boeing is

```
pnorm(-0.1, 0.01, 0.05)
#> [1] 0.0139
```

Of course, there is also a higher probability for upside return for Amazon than for Boeing. This example illustrates the fundamental “no free lunch” principle of economics and finance: you can't get something for nothing. In general, to get a higher expected return you must be prepared to take on higher risk. ■

Example 2.24 (Why the normal distribution may not be appropriate for simple returns). Let R_t denote the simple annual return on an asset, and suppose that $R_t \sim N(0.05, (0.50)^2)$. Because asset prices must be non-negative, R_t must always be larger than -1 . However, the normal distribution is defined for $-\infty \leq R_t \leq \infty$ and based on the assumed normal distribution $\Pr(R_t < -1) = 0.018$. That is, there is a 1.8% chance that R_t is smaller than -1 . This implies that there is a 1.8% chance that the asset price at the end of the year will be negative! This is one reason why the normal distribution may not be appropriate for simple returns. ■

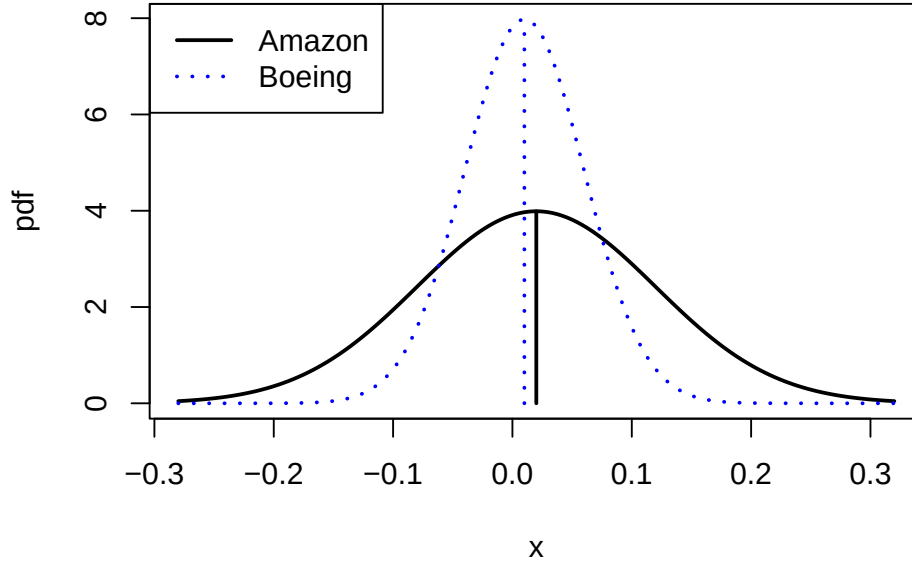


Figure 2.7: Risk-return tradeoff between one-month investments in Amazon and Boeing stock.

Example 2.25 (The normal distribution is more appropriate for continuously compounded returns). Let $r_t = \ln(1 + R_t)$ denote the continuously compounded annual return on an asset, and suppose that $r_t \sim N(0.05, (0.50)^2)$. Unlike the simple return, the continuously compounded return can take on values less than -1 . In fact, r_t is defined for $-\infty \leq r_t \leq \infty$. For example, suppose $r_t = -2$. This implies a simple return of $R_t = e^{-2} - 1 = -0.865$.⁴ Then $\Pr(r_t \leq -2) = \Pr(R_t \leq -0.865) = 0.00002$. Although the normal distribution allows for values of r_t smaller than -1 , the implied simple return R_t will always be greater than -1 . ■

2.1.6.5 The Log-Normal Distribution

Let $X \sim N(\mu_X, \sigma_X^2)$, which is defined for $-\infty < X < \infty$. The *log-normally* distributed random variable Y is determined from the normally distributed random variable X using the transformation $Y = e^X$. Here, Y is log-normally distributed and you write:

$$Y \sim \ln N(\mu_X, \sigma_X^2), \quad 0 < Y < \infty. \quad (2.16)$$

⁴If $r_t = -\infty$ then $R_t = e^{r_t} - 1 = e^{-\infty} - 1 = 0 - 1 = -1$.

The pdf of the log-normal distribution for Y can be derived from the normal distribution for X using the change-of-variables formula from calculus and is given by:

$$f(y) = \frac{1}{y\sigma_X\sqrt{2\pi}} e^{-\frac{(\ln y - \mu_X)^2}{2\sigma_X^2}}. \quad (2.17)$$

Due to the exponential transformation, Y is only defined for non-negative values. It can be shown that:

$$\begin{aligned} \mu_Y &= E[Y] = e^{\mu_X + \sigma_X^2/2}, \\ \sigma_Y^2 &= \text{var}(Y) = e^{2\mu_X + \sigma_X^2}(e^{\sigma_X^2} - 1). \end{aligned}$$

Example 2.26 (Log-normal distribution for simple returns). Let $r_t = \ln(P_t/P_{t-1})$ denote the continuously compounded monthly return on an asset and assume that $r_t \sim N(0.05, (0.50)^2)$. That is, $\mu_r = 0.05$ and $\sigma_r = 0.50$. Let $R_t = (P_t - P_{t-1})/P_t$ denote the simple monthly return. The relationship between r_t and R_t is given by $r_t = \ln(1 + R_t)$ and $1 + R_t = e^{r_t}$. Since r_t is normally distributed, $1 + R_t$ is log-normally distributed. Notice that the distribution of $1 + R_t$ is only defined for positive values of $1 + R_t$. This is appropriate since the smallest value that R_t can take on is -1 . Using Equation 2.18, the mean and variance for $1 + R_t$ are given by:

$$\begin{aligned} \mu_{1+R} &= e^{0.05 + (0.5)^2/2} = 1.191 \\ \sigma_{1+R}^2 &= e^{2(0.05) + (0.5)^2}(e^{(0.5)^2} - 1) = 0.563 \end{aligned}$$

The pdfs for r_t and R_t are shown in Figure Figure 2.8. ■

2.1.6.6 Skewness

Definition 2.12 (Skewness). The *skewness* of a random variable X , denoted $\text{skew}(X)$, measures the symmetry of a distribution about its mean value using the function $g(X) = (X - \mu_X)^3/\sigma_X^3$, where σ_X^3 is just $\text{sd}(X)$ raised to the third power:

$$\text{skew}(X) = \frac{E[(X - \mu_X)^3]}{\sigma_X^3} \quad (2.18)$$

When X is far below its mean, $(X - \mu_X)^3$ is a big negative number. When X is far above its mean, $(X - \mu_X)^3$ is a big positive number. Hence, if there are more big values of X below μ_X then $\text{skew}(X) < 0$. Conversely, if there are more big values of X above μ_X then $\text{skew}(X) > 0$. If X has a symmetric distribution, then $\text{skew}(X) = 0$ since positive and negative values in Equation 2.18 cancel out. If $\text{skew}(X) > 0$, then the distribution of X has a *long right tail* and if $\text{skew}(X) < 0$ the distribution of X has a *long left tail*. These cases are illustrated in Figure Figure 2.9.

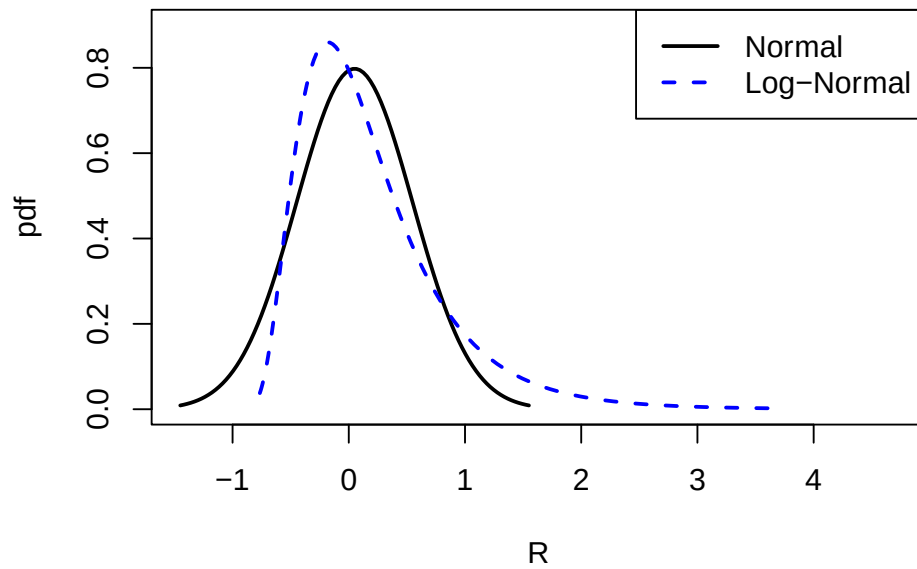


Figure 2.8: Normal distribution for r_t and log-normal distribution for $R_t = e^{r_t} - 1$.

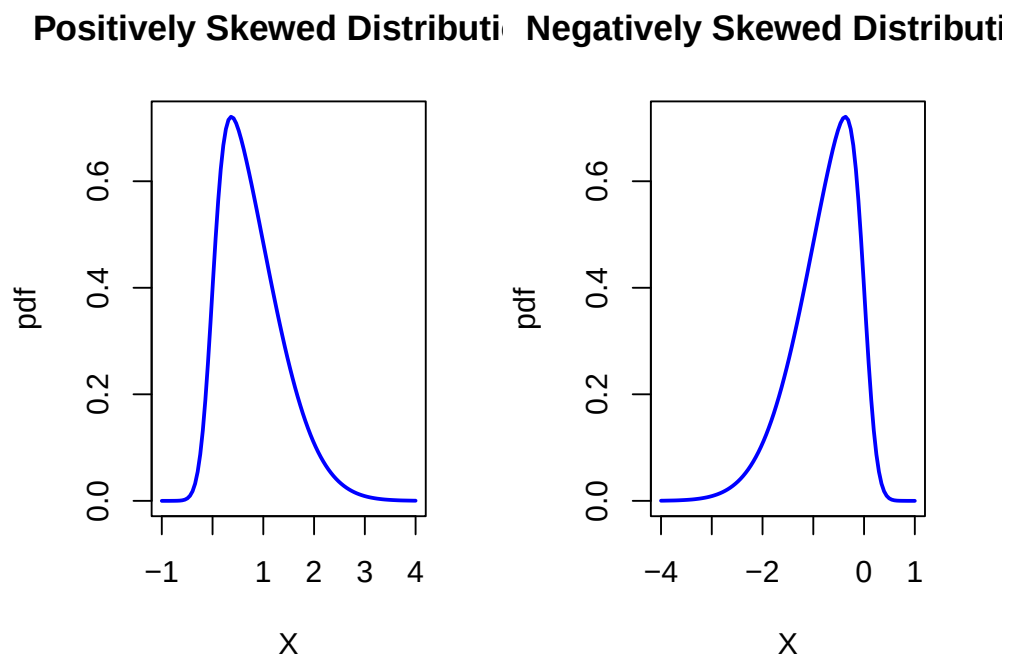


Figure 2.9: Skewed distributions.

Example 2.27 (Skewness for a discrete random variable). From the discrete distribution for the return on Microsoft stock in Table 2.1, and the results $\mu_X = 0.1$ and $\sigma_X = 0.141$, it follows that:

$$\begin{aligned}\text{skew}(X) &= [(-0.3 - 0.1)^3 \cdot (0.05) + (0.0 - 0.1)^3 \cdot (0.20) + (0.1 - 0.1)^3 \cdot (0.5) \\ &\quad + (0.2 - 0.1)^3 \cdot (0.2) + (0.5 - 0.1)^3 \cdot (0.05)] / (0.141)^3 \\ &= 0.0.\end{aligned}$$

■

Example 2.28 (Skewness for a normal random variable). Suppose X has a general normal distribution with mean μ_X and variance σ_X^2 . Then it can be shown that:

$$\text{skew}(X) = \int_{-\infty}^{\infty} \frac{(x - \mu_X)^3}{\sigma_X^3} \cdot \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma_X^2}(x - \mu_X)^2} dx = 0.$$

This result is expected since the normal distribution is symmetric about its mean value μ_X . ■

Example 2.29 (Skewness for a log-Normal random variable). Let $Y = e^X$, where $X \sim N(\mu_X, \sigma_X^2)$, be a log-normally distributed random variable with parameters μ_X and σ_X^2 . Then it can be shown that:

$$\text{skew}(Y) = (e^{\sigma_X^2} + 2) \sqrt{e^{\sigma_X^2} - 1} > 0.$$

Notice that $\text{skew}(Y)$ is always positive, indicating that the distribution of Y has a long right tail, and that it is an increasing function of σ_X^2 . This positive skewness is illustrated in Figure 2.8. ■

2.1.6.7 Kurtosis

Definition 2.13 (Kurtosis). The *kurtosis* of a random variable X , denoted $\text{kurt}(X)$, measures the thickness in the tails of a distribution and is based on $g(X) = (X - \mu_X)^4 / \sigma_X^4$:

$$\text{kurt}(X) = \frac{E[(X - \mu_X)^4]}{\sigma_X^4}, \quad (2.19)$$

where σ_X^4 is just $\text{sd}(X)$ raised to the fourth power.

Since kurtosis is based on deviations from the mean raised to the fourth power, large deviations get lots of weight. Hence, distributions with large kurtosis values are ones where there is the possibility of extreme values. In contrast, if the kurtosis is small then most of the observations are tightly clustered around the mean and there is very little probability of observing extreme values.

Example 2.30 (Kurtosis for a discrete random variable). From the discrete distribution for the return on Microsoft stock in Table Table 2.1, and the results that $\mu_X = 0.1$ and $\sigma_X = 0.141$, it follows that:

$$\begin{aligned}\text{kurt}(X) &= [(-0.3 - 0.1)^4 \cdot (0.05) + (0.0 - 0.1)^4 \cdot (0.20) + (0.1 - 0.1)^4 \cdot (0.5) \\ &\quad + (0.2 - 0.1)^4 \cdot (0.2) + (0.5 - 0.1)^4 \cdot (0.05)] / (0.141)^4 \\ &= 6.5.\end{aligned}$$

■

Example 2.31 (Kurtosis for a normal random variable). Suppose X has a general normal distribution with mean μ_X and variance σ_X^2 . Then it can be shown that:

$$\text{kurt}(X) = \int_{-\infty}^{\infty} \frac{(x - \mu_X)^4}{\sigma_X^4} \cdot \frac{1}{\sqrt{2\pi\sigma_X^2}} e^{-\frac{1}{2}\left(\frac{x - \mu_X}{\sigma_X}\right)^2} dx = 3.$$

Hence, a kurtosis of 3 is a benchmark value for tail thickness of bell-shaped distributions. If a distribution has a kurtosis greater than 3, then the distribution has thicker tails than the normal distribution. If a distribution has kurtosis less than 3, then the distribution has thinner tails than the normal. ■

Sometimes the kurtosis of a random variable is described relative to the kurtosis of a normal random variable. This relative value of kurtosis is referred to as *excess kurtosis* and is defined as:

$$\text{ekurt}(X) = \text{kurt}(X) - 3 \quad (2.20)$$

If the excess kurtosis of a random variable is equal to zero then the random variable has the same kurtosis as a normal random variable. If excess kurtosis is greater than zero, then kurtosis is larger than that for a normal; if excess kurtosis is less than zero, then kurtosis is less than that for a normal.

2.1.6.8 The Student's t Distribution

The kurtosis of a random variable gives information on the tail thickness of its distribution. The normal distribution, with kurtosis equal to three, gives a benchmark for the tail thickness of symmetric distributions. A distribution similar to the standard normal distribution but with fatter tails, and hence larger kurtosis, is the Student's t distribution. If X has a Student's t distribution with degrees of freedom parameter v , denoted $X \sim t_v$, then its pdf has the form:

$$f(x) = \frac{\Gamma\left(\frac{v+1}{2}\right)}{\sqrt{v\pi}\Gamma\left(\frac{v}{2}\right)} \left(1 + \frac{x^2}{v}\right)^{-\left(\frac{v+1}{2}\right)}, \quad -\infty < x < \infty, \quad v > 0. \quad (2.21)$$

where $\Gamma(z) = \int_0^\infty t^{z-1} e^{-t} dt$ denotes the gamma function. It can be shown that:

$$\begin{aligned} E[X] &= 0, \quad v > 1 \\ \text{var}(X) &= \frac{v}{v-2}, \quad v > 2, \\ \text{skew}(X) &= 0, \quad v > 3, \\ \text{kurt}(X) &= \frac{6}{v-4} + 3, \quad v > 4. \end{aligned}$$

The parameter v controls the scale and tail thickness of the distribution. If v is close to four, then the kurtosis is large and the tails are thick. If $v < 4$, then $\text{kurt}(X) = \infty$. As $v \rightarrow \infty$ the Student's t pdf approaches that of a standard normal random variable and $\text{kurt}(X) = 3$. Figure Figure 2.10 shows plots of the Student's t density for various values of v .

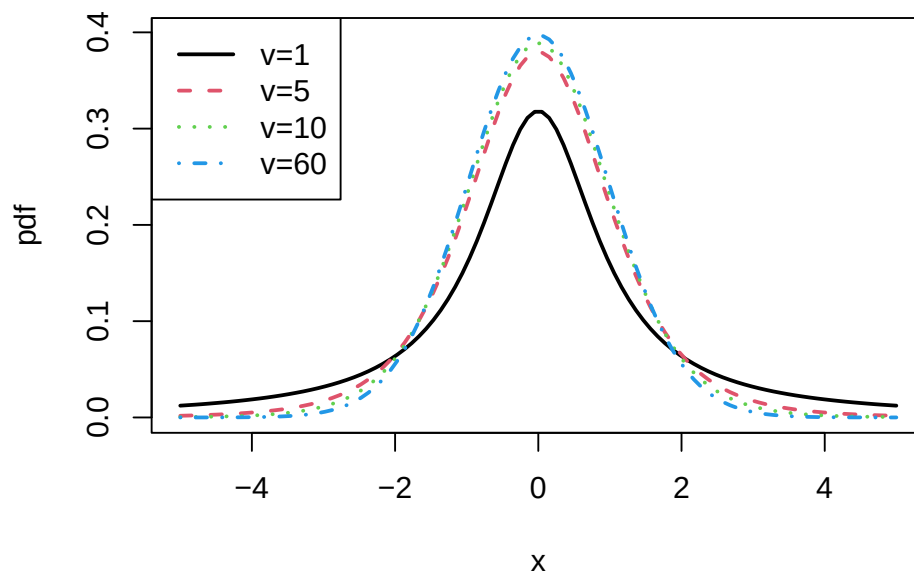


Figure 2.10: Student's t density with $v = 1, 5, 10$ and 60 .

Example 2.32 (Computing tail probabilities and quantiles from the Student's t distribution). The R functions `pt()` and `qt()` can be used to compute the cdf and quantiles of a Student's t random variable. For $v = 1, 2, 5, 10, 60, 100$ and ∞ the 1% quantiles can be computed using:

```
v = c(1, 2, 5, 10, 60, 100, Inf)
qt(0.01, df=v)
#> [1] -31.82 -6.96 -3.36 -2.76 -2.39 -2.36 -2.33
```


For $v = 1, 2, 5, 10, 60, 100$ and ∞ the values of $\Pr(X < -3)$ are:

```
pt(-3, df=v)
#> [1] 0.10242 0.04773 0.01505 0.00667 0.00196 0.00170 0.00135
```

■

2.1.7 Linear functions of a random variable

Let X be a random variable either discrete or continuous with $E[X] = \mu_X$, $\text{var}(X) = \sigma_X^2$, and let a and b be known constants. Define a new random variable Y via the linear function of X :

$$Y = g(X) = aX + b.$$

Then the following results hold:

$$\begin{aligned}\mu_Y &= E[Y] = a \cdot E[X] + b = a \cdot \mu_X + b, \\ \sigma_Y^2 &= \text{var}(Y) = a^2 \text{var}(X) = a^2 \sigma_X^2, \\ \sigma_Y &= \text{sd}(Y) = a \cdot \text{sd}(X) = a \cdot \sigma_X.\end{aligned}$$

The first result shows that expectation is a linear operation. That is,

$$E[aX + b] = aE[X] + b.$$

The second result shows that adding a constant to X does not affect its variance, and that the effect of multiplying X by the constant a increases the variance of X by the square of a . These results will be used often enough that it is instructive to go through the derivations.

Consider the first result. Let X be a discrete random variable. By the definition of $E[g(X)]$, with $g(X) = b + aX$, you can write:

$$\begin{aligned}E[Y] &= \sum_{x \in S_X} (ax + b) \cdot \Pr(X = x) \\ &= a \sum_{x \in S_X} x \cdot \Pr(X = x) + b \sum_{x \in S_X} \Pr(X = x) \\ &= a \cdot E[X] + b \cdot 1 \\ &= a \cdot \mu_X + b.\end{aligned}$$

If X is a continuous random variable then by the linearity of integration:

$$\begin{aligned} E[Y] &= \int_{-\infty}^{\infty} (ax + b)f(x) dx = a \int_{-\infty}^{\infty} xf(x) dx + b \int_{-\infty}^{\infty} f(x) dx \\ &= aE[X] + b. \end{aligned}$$

Next consider the second result. Since $\mu_Y = a\mu_X + b$:

$$\begin{aligned} \text{var}(Y) &= E[(Y - \mu_Y)^2] \\ &= E[(aX + b - (a\mu_X + b))^2] \\ &= E[(a(X - \mu_X) + (b - b))^2] \\ &= E[a^2(X - \mu_X)^2] \\ &= a^2 E[(X - \mu_X)^2] \text{ (by the linearity of } E[\cdot]) \\ &= a^2 \text{var}(X) \end{aligned}$$

Notice that the derivation of the second result works for discrete and continuous random variables.

A normal random variable has the special property that a linear function of it is also a normal random variable. The following proposition states the result.

Proposition 2.1 (Linear function of a normal random variable). *Let $X \sim N(\mu_X, \sigma_X^2)$ and let a and b be constants. Let $Y = aX + b$. Then $Y \sim N(a\mu_X + b, a^2\sigma_X^2)$.*

The above property is special to the normal distribution and may or may not hold for a random variable with a distribution that is not normal.

Example 2.33 (Standardizing a random variable). Let X be a random variable with $E[X] = \mu_X$ and $\text{var}(X) = \sigma_X^2$. Define a new random variable Z as:

$$Z = \frac{X - \mu_X}{\sigma_X} = \frac{1}{\sigma_X}X - \frac{\mu_X}{\sigma_X},$$

which is a linear function $aX + b$ where $a = \frac{1}{\sigma_X}$ and $b = -\frac{\mu_X}{\sigma_X}$. This transformation is called *standardizing* the random variable X since,

$$\begin{aligned} E[Z] &= \frac{1}{\sigma_X}E[X] - \frac{\mu_X}{\sigma_X} = \frac{1}{\sigma_X}\mu_X - \frac{\mu_X}{\sigma_X} = 0, \\ \text{var}(Z) &= \left(\frac{1}{\sigma_X}\right)^2 \text{var}(X) = \frac{\sigma_X^2}{\sigma_X^2} = 1. \end{aligned}$$

Hence, standardization creates a new random variable with mean zero and variance 1. In addition, if X is normally distributed then $Z \sim N(0, 1)$. ■

Example 2.34 (Computing probabilities using standardized random variables). Let $X \sim N(2, 4)$ and suppose you want to find $\Pr(X > 5)$ but you only know probabilities associated with a standard normal random variable $Z \sim N(0, 1)$. You can solve the problem by standardizing X as follows:

$$\Pr(X > 5) = \Pr\left(\frac{X - 2}{\sqrt{4}} > \frac{5 - 2}{\sqrt{4}}\right) = \Pr\left(Z > \frac{3}{2}\right) = 0.06681.$$

■

Standardizing a random variable is often done in the construction of test statistics. For example, the so-called *t-statistic* or *t-ratio* used for testing simple hypotheses on coefficients is constructed by the standardization process.

A non-standard random variable X with mean μ_X and variance σ_X^2 can be created from a standard random variable via the linear transformation:

$$X = \mu_X + \sigma_X \cdot Z. \quad (2.22)$$

This result is useful for modeling purposes as illustrated in the next examples.

Example 2.35 (Quantile of general normal random variable). Let $X \sim N(\mu_X, \sigma_X^2)$. Quantiles of X can be conveniently computed using:

$$q_\alpha = \mu_X + \sigma_X z_\alpha, \quad (2.23)$$

where $\alpha \in (0, 1)$ and z_α is the $\alpha \times 100\%$ quantile of a standard normal random variable. This formula is derived as follows. By the definition z_α and Equation 2.22

$$\alpha = \Pr(Z \leq z_\alpha) = \Pr(\mu_X + \sigma_X \cdot Z \leq \mu_X + \sigma_X \cdot z_\alpha) = \Pr(X \leq \mu_X + \sigma_X \cdot z_\alpha),$$

which implies Equation 2.23. ■

Example 2.36 (Generalized Student's t distribution). The Student's t distribution defined in Equation 2.21 has zero mean and variance equal to $v/(v - 2)$ where v is the degrees of freedom parameter. You can define a Student's t random variable with non-zero mean μ and variance σ^2 from a Student's t random variable X as follows:

$$Y = \mu + \sigma \left(\frac{v-2}{v}\right)^{1/2} X = \mu + \sigma Z_v,$$

where $Z_v = \left(\frac{v-2}{v}\right)^{1/2} X$. The random variable Z_v is called a *standardized Student's t* random variable with v degrees of freedom since $E[Z_v] = 0$ and $\text{var}(Z_v) = \left(\frac{v-2}{v}\right) \text{var}(X) = \left(\frac{v-2}{v}\right) \left(\frac{v}{v-2}\right) = 1$.

Then

$$\begin{aligned} E[Y] &= \mu + \sigma E[Z] = \mu, \\ \text{var}(Y) &= \sigma^2 \text{var}(Z) = \sigma^2. \end{aligned}$$

I use the notation $Y \sim t_v(\mu, \sigma^2)$ to denote that Y has a *generalized Student's t* distribution with mean μ , variance σ^2 , and degrees of freedom v . ■

Example 2.37 (Location scale return model for asset returns). Let r denote the monthly continuously compounded return on an asset with $E[r] = \mu$ and $\text{var}(R) = \sigma^2$. If r has a normal distribution or a general Student's t distribution, then r can be expressed as:

$$r = \mu_r + \sigma_r \cdot \varepsilon,$$

where $\varepsilon \sim N(0, 1)$ or $\varepsilon \sim t_v(0, 1)$. The random variable ε can be interpreted as representing the random news arriving in a given month that makes the observed return differ from its expected value μ . The fact that ε has mean zero means that news, on average, is neutral. The value of σ_r represents the typical size of a news shock. The bigger is σ_r , the larger is the impact of a news shock and vice-versa. The distribution of ε impacts the likelihood of extreme new events (like stock market crashes). If $\varepsilon \sim N(0, 1)$ then extreme news events are not very likely. If $\varepsilon \sim t_v(0, 1)$ with v small (but bigger than 4) then extreme news events may happen from time-to-time. ■

2.1.8 Value-at-Risk: An introduction

As an example of working with linear functions of a normal random variable, and to illustrate the concept of *Value-at-Risk* (VaR), consider an investment of \$10,000 in Microsoft stock over the next month. Let R denote the monthly simple return on Microsoft stock and assume that $R \sim N(0.05, (0.10)^2)$. That is, $E[R] = \mu_R = 0.05$ and $\text{var}(R) = \sigma_R^2 = (0.10)^2$. Let W_0 denote the investment value at the beginning of the month and W_1 denote the investment value at the end of the month. In this example, $W_0 = \$10,000$. Consider the following questions:

- (i) What is the probability distribution of end-of-month wealth, W_1 ?
- (ii) What is the probability that end-of-month wealth is less than \$9,000, and what must the return on Microsoft be for this to happen?
- (iii) What is the loss in dollars that would occur if the return on Microsoft stock is equal to its 5% quantile, $q_{0.05}$? That is, what is the monthly 5% VaR on the \$10,000 investment in Microsoft?

To answer (i), note that end-of-month wealth, W_1 , is related to initial wealth W_0 and the return on Microsoft stock R via the linear function:

$$W_1 = W_0(1 + R) = W_0 + W_0 R = \$10,000 + \$10,000 \cdot R.$$

Using the properties of linear functions of a random variable you get:

$$E[W_1] = W_0 + W_0 E[R] = \$10,000 + \$10,000(0.05) = \$10,500,$$

and,

$$\begin{aligned}\text{var}(W_1) &= (W_0)^2 \text{var}(R) = (\$10,000)^2 (0.10)^2, \\ \text{sd}(W_1) &= (\$10,000)(0.10) = \$1,000.\end{aligned}$$

Further, since R is assumed to be normally distributed it follows that W_1 is normally distributed:

$$W_1 \sim N(\$10,500, (\$1,000)^2).$$

To answer (ii), you can use the above normal distribution for W_1 to get:

$$\Pr(W_1 < \$9,000) = 0.067.$$

To find the return that produces end-of-month wealth of \$9,000, or a loss of \$10,000-\$9,000=\$1,000, you solve

$$R = \frac{\$9,000 - \$10,000}{\$10,000} = -0.10.$$

If the monthly return on Microsoft is -10% or less, then end-of-month wealth will be \$9,000 or less. Notice that $R = -0.10$ is the 6.7% quantile of the distribution of R :

$$\Pr(R < -0.10) = 0.067.$$

Question (iii) can be answered in two equivalent ways. First, use $R \sim N(0.05, (0.10)^2)$ and solve for the 5% quantile of Microsoft Stock using Equation 2.23⁵:

$$\begin{aligned}\Pr(R < q_{.05}^R) &= 0.05 \\ \Rightarrow q_{.05}^R &= \mu_R + \sigma_R \cdot z_{.05} = 0.05 + 0.10 \cdot (-1.645) = -0.114.\end{aligned}$$

That is, with 5% probability the return on Microsoft stock is -11.4% or less. Now, if the return on Microsoft stock is -11.4% the loss in investment value that occurs with 5% probability is

$$\text{VaR}_{0.05} = W_0 \cdot q_{.05}^R = \$10,000 \cdot (-0.114) = -\$1,144.$$

Since losses are typically reported as positive numbers, $\text{VaR}_{0.05} = \$1,144$ is the 5% VaR over the next month on the \$10,000 investment in Microsoft stock.

For the second method, use $W_1 \sim N(\$10,500, (\$1,000)^2)$ and solve for the 5% quantile of end-of-

⁵Using R, $z_{.05} = \text{qnorm}(0.05) = -1.645$.

month wealth directly:

$$\Pr(W_1 < q_{.05}^{W_1}) = 0.05 \Rightarrow q_{.05}^{W_1} = \$8,856.$$

This corresponds to a loss of investment value of $\$10,000 - \$8,856 = \$1,144$. Hence, if W_0 represents the initial wealth and $q_{.05}^{W_1}$ is the 5% quantile of the distribution of W_1 then the 5% VaR is:

$$\text{VaR}_{.05} = W_0 - q_{.05}^{W_1} = \$10,000 - \$8,856 = \$1,144.$$

The following is the usual definition of VaR.

Definition 2.14 (Value-at-Risk). If W_0 represents the initial wealth in dollars and q_α^R is the $\alpha \times 100\%$ quantile of distribution of the simple return R for small values of α , then the $\alpha \times 100\%$ VaR is defined as:

$$\text{VaR}_\alpha = -W_0 \cdot q_\alpha^R. \quad (2.24)$$

In words, VaR_α represents the dollar loss that could occur with probability α . By convention, it is reported as a positive number (hence the multiplication by minus one as $q_\alpha^R < 0$ for small values of α).

2.1.8.1 Value-at-Risk calculations for continuously compounded returns

The above calculations illustrate how to calculate VaR using the normal distribution for simple returns. However, as argued in Example 31, the normal distribution may not be appropriate for characterizing the distribution of simple returns and may be more appropriate for characterizing the distribution of continuously compounded returns. Let R denote the simple monthly return, let $r = \ln(1 + R)$ denote the continuously compounded return and assume that $r \sim N(\mu_r, \sigma_r^2)$. The $\alpha \times 100\%$ monthly VaR on an investment of $\$W_0$ is computed using the following steps:

1. Compute the $\alpha \cdot 100\%$ quantile, q_α^r , from the normal distribution for the continuously compounded return r :

$$q_\alpha^r = \mu_r + \sigma_r z_\alpha,$$

where z_α is the $\alpha \cdot 100\%$ quantile of the standard normal distribution.

2. Convert the continuously compounded return quantile, q_α^r , to a simple return quantile using the transformation:

$$q_\alpha^R = e^{q_\alpha^r} - 1$$

3. Compute VaR using the simple return quantile Equation 2.24.

Step 2 works because quantiles are preserved under monotonically increasing transformations of random variables.

Example 2.38 (Computing VaR from simple and continuously compounded returns using R). Let R denote the simple monthly return on Microsoft stock and assume that $R \sim N(0.05, (0.10)^2)$. Consider an initial investment of $W_0 = \$10,000$. To compute the 1% and 5% VaR values over the next month use:

```
mu.R = 0.05
sd.R = 0.10
w0 = 10000
q.01.R = mu.R + sd.R*qnorm(0.01)
q.05.R = mu.R + sd.R*qnorm(0.05)
VaR.01 = -q.01.R*w0
VaR.05 = -q.05.R*w0
VaR.01
#> [1] 1826
VaR.05
#> [1] 1145
```

Hence with 1% and 5% probability the loss over the next month is at least \$1,826 and \$1,145, respectively.

Let r denote the continuously compounded return on Microsoft stock and assume that $r \sim N(0.05, (0.10)^2)$. To compute the 1% and 5% VaR values over the next month use:

```
mu.r = 0.05
sd.r = 0.10
q.01.R = exp(mu.r + sd.r*qnorm(0.01)) - 1
q.05.R = exp(mu.r + sd.r*qnorm(0.05)) - 1
VaR.01 = -q.01.R*w0
VaR.05 = -q.05.R*w0
VaR.01
#> [1] 1669
VaR.05
#> [1] 1082
```

Notice that when $1 + R = e^r$ has a log-normal distribution, the 1% and 5% VaR values (losses) are slightly smaller than when R is normally distributed. This is due to the positive skewness of the log-normal distribution. ■

2.2 Bivariate Distributions

So far I have only considered probability distributions for a single random variable. In many situations, you want to be able to characterize the probabilistic behavior of two or more random variables simultaneously. For example, you might want to know about the probabilistic behavior of the returns on a two asset portfolio. In this section, I discuss bivariate distributions and important concepts related to the analysis of two random variables.

2.2.1 Discrete random variables

Let X and Y be discrete random variables with sample spaces S_X and S_Y , respectively. The likelihood that X and Y takes values in the joint sample space $S_{XY} = S_X \times S_Y$ (Cartesian product of the individual sample spaces) is determined by the *joint probability distribution* $p(x, y) = \Pr(X = x, Y = y)$. The function $p(x, y)$ satisfies:

- (i) $p(x, y) > 0$ for $x, y \in S_{XY}$;
- (ii) $p(x, y) = 0$ for $x, y \notin S_{XY}$;
- (iii) $\sum_{x, y \in S_{XY}} p(x, y) = \sum_{x \in S_X} \sum_{y \in S_Y} p(x, y) = 1$.

Example 2.39 (Bivariate discrete distribution for stock returns). Let X denote the monthly return (in percent) on Microsoft stock and let Y denote the monthly return on Starbucks stock. For simplicity suppose that the sample spaces for X and Y are $S_X = \{0, 1, 2, 3\}$ and $S_Y = \{0, 1\}$ so that the random variables X and Y are discrete. The joint sample space is the two dimensional grid $S_{XY} = \{(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1), (3, 0), (3, 1)\}$. Table 2.3 illustrates the joint distribution for X and Y . From the table, $p(0, 0) = \Pr(X = 0, Y = 0) = 1/8$. Notice that the sum of all the entries in the table sum to unity. ■

Table 2.3: Discrete bivariate distribution for Microsoft and Starbucks stock prices.

		Y		
X	%	0	1	Pr(X)
	0	1/8	0	1/8
	1	2/8	1/8	3/8
	2	1/8	2/8	3/8
	3	0	1/8	1/8
	Pr(Y)	4/8	4/8	1

2.2.1.1 Marginal Distributions

The joint probability distribution tells us the probability of X and Y occurring together. What if you only want to know about the probability of X occurring, or the probability of Y occurring?

Example 2.40 (Find $\Pr(X = 0)$ and $\Pr(Y = 1)$ from joint distribution). Consider the joint distribution in Table Table 2.3. What is $\Pr(X = 0)$ regardless of the value of Y ? Now X can occur if $Y = 0$ or if $Y = 1$ and since these two events are mutually exclusive you get:

$$\Pr(X = 0) = \Pr(X = 0, Y = 0) + \Pr(X = 0, Y = 1) = 0 + 1/8 = 1/8.$$

Notice that this probability is equal to the horizontal (row) sum of the probabilities in the table at $X = 0$. You can find $\Pr(Y = 1)$ in a similar fashion:

$$\begin{aligned}\Pr(Y = 1) &= \Pr(X = 0, Y = 1) + \Pr(X = 1, Y = 1) + \Pr(X = 2, Y = 1) + \Pr(X = 3, Y = 1) \\ &= 0 + 1/8 + 2/8 + 1/8 = 4/8.\end{aligned}$$

This probability is the vertical (column) sum of the probabilities in the table at $Y = 1$. ■

Definition 2.15 (Marginal distribution). The probability $\Pr(X = x)$ is called the *marginal probability* of X and is given by:

$$\Pr(X = x) = \sum_{y \in S_Y} \Pr(X = x, Y = y). \quad (2.25)$$

Similarly, the marginal probability of $Y = y$ is given by:

$$\Pr(Y = y) = \sum_{x \in S_X} \Pr(X = x, Y = y). \quad (2.26)$$

Example 2.41 (Marginal probabilities of discrete bivariate distribution). The marginal probabilities of $X = x$ are given in the last column of Table Table 2.3, and the marginal probabilities of $Y = y$ are given in the last row of Table Table 2.3. Notice that these probabilities sum to 1. For future reference, note that $E[X] = 3/2$, $\text{var}(X) = 3/4$, $E[Y] = 1/2$, and $\text{var}(Y) = 1/4$. ■

2.2.1.2 Conditional Distributions

For random variables in Table Table 2.3, suppose you know that the random variable Y takes on the value $Y = 0$. How does this knowledge affect the likelihood that X takes on the values 0, 1, 2 or 3? For example, what is the probability that $X = 0$ *given that* you know $Y = 0$? To find this

probability, you can use Bayes' law and compute the *conditional probability*:

$$\Pr(X = 0|Y = 0) = \frac{\Pr(X = 0, Y = 0)}{\Pr(Y = 0)} = \frac{1/8}{4/8} = 1/4.$$

The notation $\Pr(X = 0|Y = 0)$ is read as “the probability that $X = 0$ given that $Y = 0$ ” . Notice that $\Pr(X = 0|Y = 0) = 1/4 > \Pr(X = 0) = 1/8$. Hence, knowledge that $Y = 0$ increases the likelihood that $X = 0$. Clearly, X *depends* on Y .

Now suppose that you know that $X = 0$. How does this knowledge affect the probability that $Y = 0$? To find out compute:

$$\Pr(Y = 0|X = 0) = \frac{\Pr(X = 0, Y = 0)}{\Pr(X = 0)} = \frac{1/8}{1/8} = 1.$$

Notice that $\Pr(Y = 0|X = 0) = 1 > \Pr(Y = 0) = 1/2$. That is, knowledge that $X = 0$ makes it certain that $Y = 0$.

The conditional probability that $X = x$ given that $Y = y$ (provided $\Pr(Y = y) \neq 0$) is,

$$p(x|y) = \Pr(X = x|Y = y) = \frac{\Pr(X = x, Y = y)}{\Pr(Y = y)} = \frac{p(x, y)}{p(y)}, \quad (2.27)$$

and the conditional probability that $Y = y$ given that $X = x$ (provided $\Pr(X = x) \neq 0$) is,

$$p(y|x) = \Pr(Y = y|X = x) = \frac{\Pr(X = x, Y = y)}{\Pr(X = x)} = \frac{p(x, y)}{p(x)}. \quad (2.28)$$

Example 2.42 (Conditional distributions). For the bivariate distribution in Table Table 2.3, the conditional probabilities along with marginal probabilities are summarized in Tables Table 2.4 and Table 2.5. Notice that the marginal distribution of X is centered at $x = 3/2$ whereas the conditional distribution of $X|Y = 0$ is centered at $x = 1$ and the conditional distribution of $X|Y = 1$ is centered at $x = 2$. ■

Table 2.4: Conditional probability distribution of X from bivariate discrete distribution.

x	$\Pr(X = x)$	$\Pr(X Y = 0)$	$\Pr(X Y = 1)$
0	1/8	2/8	0
1	3/8	4/8	2/8
2	3/8	4/8	4/8
3	1/8	0	2/8

Table 2.5: Conditional distribution of Y from bivariate discrete distribution.

y	Pr(Y = y)	Pr(Y X = 0)	Pr(Y X = 1)	Pr(Y X = 2)	Pr(Y X = 3)
0	1/2	1	2/3	1/3	0
1	1/2	0	1/3	2/3	1

2.2.1.3 Conditional expectation and conditional variance

Just as I defined shape characteristics of the marginal distributions of X and Y , I can also define shape characteristics of the conditional distributions of $X|Y = y$ and $Y|X = x$. The most important shape characteristics are the *conditional expectation* (*conditional mean*) and the *conditional variance*. The conditional mean of $X|Y = y$ is denoted by $\mu_{X|Y=y} = E[X|Y = y]$, and the conditional mean of $Y|X = x$ is denoted by $\mu_{Y|X=x} = E[Y|X = x]$.

Definition 2.16 (Conditional expectation). For discrete random variables X and Y , the conditional expectations are defined as:

$$\mu_{X|Y=y} = E[X|Y = y] = \sum_{x \in S_X} x \cdot \Pr(X = x|Y = y), \quad (2.29)$$

$$\mu_{Y|X=x} = E[Y|X = x] = \sum_{y \in S_Y} y \cdot \Pr(Y = y|X = x). \quad (2.30)$$

Similarly, the conditional variance of $X|Y = y$ is denoted by $\sigma_{X|Y=y}^2 = \text{var}(X|Y = y)$ and the conditional variance of $Y|X = x$ is denoted by $\sigma_{Y|X=x}^2 = \text{var}(Y|X = x)$.

Definition 2.17 (Conditional variance). For discrete random variables X and Y , the conditional variances are defined as:

$$\sigma_{X|Y=y}^2 = \text{var}(X|Y = y) = \sum_{x \in S_X} (x - \mu_{X|Y=y})^2 \cdot \Pr(X = x|Y = y), \quad (2.31)$$

$$\sigma_{Y|X=x}^2 = \text{var}(Y|X = x) = \sum_{y \in S_Y} (y - \mu_{Y|X=x})^2 \cdot \Pr(Y = y|X = x). \quad (2.32)$$

The conditional volatilities are defined as:

$$\sigma_{X|Y=y} = \sqrt{\sigma_{X|Y=y}^2}, \quad (2.33)$$

$$\sigma_{Y|X=x} = \sqrt{\sigma_{Y|X=x}^2}. \quad (2.34)$$

Then conditional mean is the center of mass of the conditional distribution, and the conditional variance and volatility measure the spread of the conditional distribution about the conditional mean.

Example 2.43 (Compute conditional expectation and conditional variance). For the random variables in Table 2.3, you can derive the following conditional moments for X :

$$\begin{aligned} E[X|Y=0] &= 0 \cdot 1/4 + 1 \cdot 1/2 + 2 \cdot 1/4 + 3 \cdot 0 = 1, \\ E[X|Y=1] &= 0 \cdot 0 + 1 \cdot 1/4 + 2 \cdot 1/2 + 3 \cdot 1/4 = 2, \\ \text{var}(X|Y=0) &= (0-1)^2 \cdot 1/4 + (1-1)^2 \cdot 1/2 + (2-1)^2 \cdot 1/2 + (3-1)^2 \cdot 0 = 1/2, \\ \text{var}(X|Y=1) &= (0-2)^2 \cdot 0 + (1-2)^2 \cdot 1/4 + (2-2)^2 \cdot 1/2 + (3-2)^2 \cdot 1/4 = 1/2. \end{aligned}$$

Compare these values to $E[X] = 3/2$ and $\text{var}(X) = 3/4$. Notice that $\text{var}(X|Y) = 1/2 < \text{var}(X) = 3/4$ so that conditioning on information reduces variability. Also, notice that as y increases, $E[X|Y=y]$ increases.

For Y , similar calculations gives:

$$\begin{aligned} E[Y|X=0] &= 0, E[Y|X=1] = 1/3, E[Y|X=2] = 2/3, E[Y|X=3] = 1, \\ \text{var}(Y|X=0) &= 0, \text{var}(Y|X=1) = 0.2222, \text{var}(Y|X=2) = 0.2222, \text{var}(Y|X=3) = 0. \end{aligned}$$

Compare these values to $E[Y] = 1/2$ and $\text{var}(Y) = 1/4$. Notice that as x increases $E[Y|X=x]$ increases. ■

2.2.1.4 Conditional expectation and the regression function

Consider the problem of predicting the value Y given that you know $X = x$. A natural predictor to use is the conditional expectation $E[Y|X = x]$. In this prediction context, the conditional expectation $E[Y|X = x]$ is called the *regression function*. The graph with $E[Y|X = x]$ on the vertical axis and x on the horizontal axis gives the *regression line*. The relationship between Y and the regression function may be expressed using the trivial identity:

$$\begin{aligned} Y &= E[Y|X = x] + Y - E[Y|X = x] \\ &= E[Y|X = x] + \varepsilon, \end{aligned}$$

where $\varepsilon = Y - E[Y|X]$ is called the *regression error*.

Example 2.44 (Regression line for bivariate discrete distribution). For the random variables in Table 2.3, the regression line is plotted in Figure 2.11. Notice that there is a linear relationship between $E[Y|X = x]$ and x . When such a linear relationship exists you call the regression function a *linear regression*. Linearity of the regression function, however, is not guaranteed. It may be the case that there is a non-linear (e.g., quadratic) relationship between $E[Y|X = x]$ and x . In this case, you call the regression function a *non-linear regression*. ■

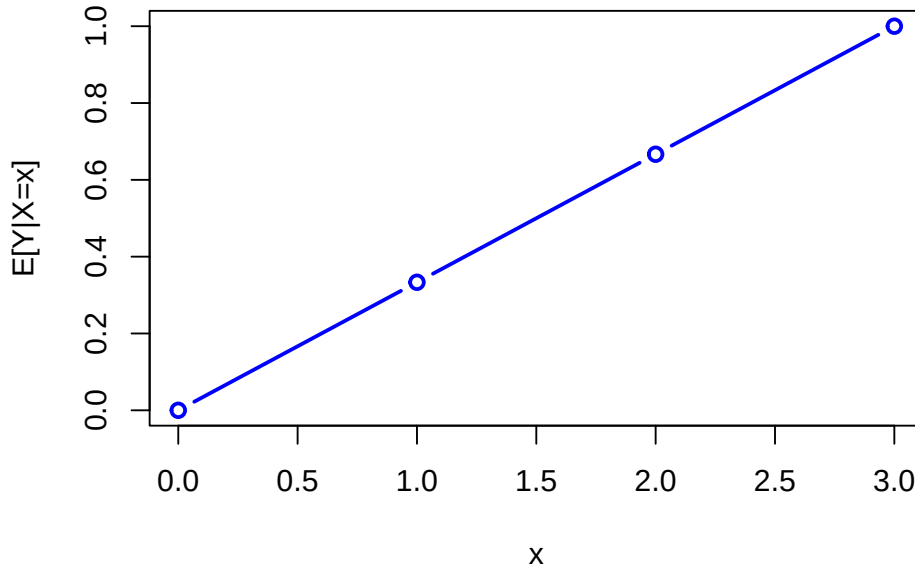


Figure 2.11: Regression function $E[Y|X = x]$ from discrete bivariate distribution.

2.2.1.5 Law of Total Expectations

Before Y is observed, $E[X|Y]$ is a random variable because its value depends on the value of Y which is a random variable. Similarly, $E[Y|X]$ is also a random variable. Since $E[X|Y]$ and $E[Y|X]$ are random variables you can consider their mean values. To illustrate, for the random variables in Table 2.3 notice that:

$$\begin{aligned}
 E[X] &= \sum_{y \in S_Y} E[X|Y = y] \cdot \Pr(Y = y) \\
 &= E[X|Y = 0] \cdot \Pr(Y = 0) + E[X|Y = 1] \cdot \Pr(Y = 1) \\
 &= 1 \cdot 1/2 + 2 \cdot 1/2 = 3/2,
 \end{aligned}$$

and,

$$\begin{aligned}
 E[Y] &= \sum_{x \in S_X} E[Y|X = x] \cdot \Pr(X = x) \\
 &= E[Y|X = 0] \cdot \Pr(X = 0) + E[Y|X = 1] \cdot \Pr(X = 1) \\
 &\quad + E[Y|X = 2] \cdot \Pr(X = 2) + E[Y|X = 3] \cdot \Pr(X = 3) = 1/2.
 \end{aligned}$$

This result is known as the *law of iterated expectations* or the *law of total expectations*.

Proposition 2.2. For two random variables X and Y (discrete or continuous) it can be shown that:

$$E[X] = E[E[X|Y]],$$

$$E[Y] = E[E[Y|X]],$$

where the expectation on the first line is taken with respect to the random variable Y , and the expectation on the second line is taken with respect to the random variable X .

In words, the law of iterated expectations says that the mean of the conditional mean is the unconditional mean.

2.2.2 Bivariate distributions for continuous random variables

Let X and Y be continuous random variables defined over the real line. You can characterize the joint probability distribution of X and Y using the joint probability function (pdf) $f(x, y)$ such that $f(x, y) \geq 0$ and,

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) \, dx \, dy = 1.$$

The three-dimensional plot of the joint probability distribution gives a probability surface whose total volume is unity. To compute joint probabilities of $x_1 \leq X \leq x_2$ and $y_1 \leq Y \leq y_2$, you need to find the volume under the probability surface over the grid where the intervals $[x_1, x_2]$ and $[y_1, y_2]$ overlap. Finding this volume requires solving the double integral:

$$\Pr(x_1 \leq X \leq x_2, y_1 \leq Y \leq y_2) = \int_{x_1}^{x_2} \int_{y_1}^{y_2} f(x, y) \, dx \, dy.$$

Example 2.45 (Standard bivariate normal distribution). A *standard bivariate normal* pdf for X and Y has the form:

$$f(x, y) = \frac{1}{2\pi} e^{-\frac{1}{2}(x^2+y^2)}, \quad -\infty \leq x, y \leq \infty \quad (2.35)$$

and has the shape of a symmetric bell (think Liberty Bell) centered at $x = 0$ and $y = 0$. To find $\Pr(-1 < X < 1, -1 < Y < 1)$ you must solve:

$$\int_{-1}^1 \int_{-1}^1 \frac{1}{2\pi} e^{-\frac{1}{2}(x^2+y^2)} \, dx \, dy,$$

which, unfortunately, does not have an analytical solution. Numerical approximation methods are required to evaluate the above integral. The function `pmvnorm()` in the R package **mvtnorm** can be used to evaluate areas under the bivariate standard normal surface. To compute $\Pr(-1 < X < 1, -1 < Y < 1)$ use:

```
pmvnorm(lower=c(-1, -1), upper=c(1, 1))
#> [1] 0.466
#> attr("error")
#> [1] 1e-15
#> attr("msg")
#> [1] "Normal Completion"
```

Here, $\Pr(-1 < X < 1, -1 < Y < 1) = 0.4661$. The attribute **error** gives the estimated absolute error of the approximation, and the attribute **message** tells the status of the algorithm used for the approximation. See the online help for **pmvnorm** for more details. ■

2.2.2.1 Marginal and conditional distributions

The marginal pdf of X is found by integrating y out of the joint pdf $f(x, y)$ and the marginal pdf of Y is found by integrating x out of the joint pdf $f(x, y)$:

$$f(x) = \int_{-\infty}^{\infty} f(x, y) \, dy, \quad (2.36)$$

$$f(y) = \int_{-\infty}^{\infty} f(x, y) \, dx. \quad (2.37)$$

The conditional pdf of X given that $Y = y$, denoted $f(x|y)$, is computed as

$$f(x|y) = \frac{f(x, y)}{f(y)}, \quad (2.38)$$

and the conditional pdf of Y given that $X = x$ is computed as,

$$f(y|x) = \frac{f(x, y)}{f(x)}. \quad (2.39)$$

The conditional means are computed as

$$\mu_{X|Y=y} = E[X|Y = y] = \int x \cdot p(x|y) \, dx, \quad (2.40)$$

$$\mu_{Y|X=x} = E[Y|X = x] = \int y \cdot p(y|x) \, dy, \quad (2.41)$$

and the conditional variances are computed as,

$$\sigma_{X|Y=y}^2 = \text{var}(X|Y = y) = \int (x - \mu_{X|Y=y})^2 p(x|y) dx, \quad (2.42)$$

$$\sigma_{Y|X=x}^2 = \text{var}(Y|X = x) = \int (y - \mu_{Y|X=x})^2 p(y|x) dy. \quad (2.43)$$

Example 2.46 (Conditional and marginal distributions from bivariate standard normal). Suppose X and Y are distributed bivariate standard normal. To find the marginal distribution of X you use Equation 2.36 and solve:

$$f(x) = \int_{-\infty}^{\infty} \frac{1}{2\pi} e^{-\frac{1}{2}(x^2+y^2)} dy = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}y^2} dy = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}.$$

Hence, the marginal distribution of X is standard normal. Similiar calculations show that the marginal distribution of Y is also standard normal. To find the conditional distribution of $X|Y = y$ use Equation 2.38 and solve:

$$\begin{aligned} f(x|y) &= \frac{f(x, y)}{f(y)} = \frac{\frac{1}{2\pi} e^{-\frac{1}{2}(x^2+y^2)}}{\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}y^2}} \\ &= \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(x^2+y^2)+\frac{1}{2}y^2} \\ &= \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} \\ &= f(x) \end{aligned}$$

So, for the standard bivariate normal distribution $f(x|y) = f(x)$ which does not depend on y . Similar calculations show that $f(y|x) = f(y)$. ■

2.2.3 Independence

Let X and Y be two discrete random variables. Intuitively, X is independent of Y if knowledge about Y does not influence the likelihood that $X = x$ for all possible values of $x \in S_X$ and $y \in S_Y$. Similarly, Y is independent of X if knowledge about X does not influence the likelihood that $Y = y$ for all values of $y \in S_Y$. I represent this intuition formally for discrete random variables as follows.

Definition 2.18 (Independence of discrete random variables). Let X and Y be discrete random variables with sample spaces S_X and S_Y , respectively. X and Y are independent random variables if and only if (iff):

$$\Pr(X = x|Y = y) = \Pr(X = x), \text{ for all } x \in S_X, y \in S_Y$$

$$\Pr(Y = y|X = x) = \Pr(Y = y), \text{ for all } x \in S_X, y \in S_Y$$

Example 2.47 (Check independence of bivariate discrete random variables). For the data in Table 2.3, you know that $\Pr(X = 0|Y = 0) = 1/4 \neq \Pr(X = 0) = 1/8$ so X and Y are not independent. ■

Proposition 2.3 (Independence of discrete random variables). *Let X and Y be discrete random variables with sample spaces S_X and S_Y , respectively. X and Y are independent iff:*

$$\Pr(X = x, Y = y) = \Pr(X = x) \cdot \Pr(Y = y), \text{ for all } x \in S_X, y \in S_Y.$$

Intuition for the above result follows from:

$$\begin{aligned} \Pr(X = x|Y = y) &= \frac{\Pr(X = x, Y = y)}{\Pr(Y = y)} = \frac{\Pr(X = x) \cdot \Pr(Y = y)}{\Pr(Y = y)} \\ &= \Pr(X = x), \\ \Pr(Y = y|X = x) &= \frac{\Pr(X = x, Y = y)}{\Pr(X = x)} = \frac{\Pr(X = x) \cdot \Pr(Y = y)}{\Pr(X = x)} \\ &= \Pr(Y = y), \end{aligned}$$

which shows that X and Y are independent.

For continuous random variables, you have the following definition of independence.

Definition 2.19 (Independence of continuous random variables). Let X and Y be continuous random variables with marginal pdfs $f(x)$ and $f(y)$, and conditional pdfs $f(x|y)$ and $f(y|x)$. Then X and Y are independent iff:

$$\begin{aligned} f(x|y) &= f(x), \text{ for } -\infty < x, y < \infty, \\ f(y|x) &= f(y), \text{ for } -\infty < x, y < \infty. \end{aligned}$$

As with discrete random variables, you have the following result for continuous random variables.

Proposition 2.4 (Independence of continuous random variables). *Let X and Y be continuous random variables with marginal pdfs $f(x)$ and $f(y)$, and joint pdf $f(x, y)$. Then X and Y are independent iff:*

$$f(x, y) = f(x)f(y)$$

This result is extremely useful in practice because it gives us an easy way to compute the joint pdf for two independent random variables: you simply compute the product of the marginal distributions.

Example 2.48 (Constructing the bivariate standard normal distribution). Let $X \sim N(0, 1)$, $Y \sim N(0, 1)$ and let X and Y be independent. Then,

$$f(x, y) = f(x)f(y) = \frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2} \frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}y^2} = \frac{1}{2\pi}e^{-\frac{1}{2}(x^2+y^2)}.$$

This result is a special case of the general bivariate normal distribution to be discussed in the next sub-section. ■

A useful property of the independence between two random variables is the following Proposition.

Proposition 2.5 (Property of independent random variables). *If X and Y are independent then $g(X)$ and $h(Y)$ are independent for any functions $g(\cdot)$ and $h(\cdot)$.*

For example, if X and Y are independent then X^2 and Y^2 are also independent.

2.2.4 Covariance and correlation

Let X and Y be two discrete random variables. Figure Figure 2.12 displays several bivariate probability scatterplots (where equal probabilities are given on the dots). In panel (a) you see no linear relationship between X and Y . In panel (b) you see a perfect positive linear relationship between X and Y and in panel (c) you see a perfect negative linear relationship. In panel (d) you see a positive, but not perfect, linear relationship; in panel (e) you see a negative, but not perfect, linear relationship. Finally, in panel (f) you see no systematic linear relationship but you see a strong nonlinear (parabolic) relationship. The *covariance* between X and Y measures the *direction* of the linear relationship between the two random variables. The *correlation* between X and Y measures the *direction* and the *strength* of the linear relationship between the two random variables.

Let X and Y be two random variables with $E[X] = \mu_X$, $\text{var}(X) = \sigma_X^2$, $E[Y] = \mu_Y$ and $\text{var}(Y) = \sigma_Y^2$.

Definition 2.20 (Covariance). The covariance between two random variables X and Y is given by:

$$\begin{aligned} \sigma_{XY} &= \text{cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)] \\ &= \sum_{x \in S_X} \sum_{y \in S_Y} (x - \mu_X)(y - \mu_Y) \Pr(X = x, Y = y) \text{ for discrete } X, Y, \\ &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} (x - \mu_X)(y - \mu_Y)p(x, y) dx dy \text{ for continuous } X, Y. \end{aligned}$$

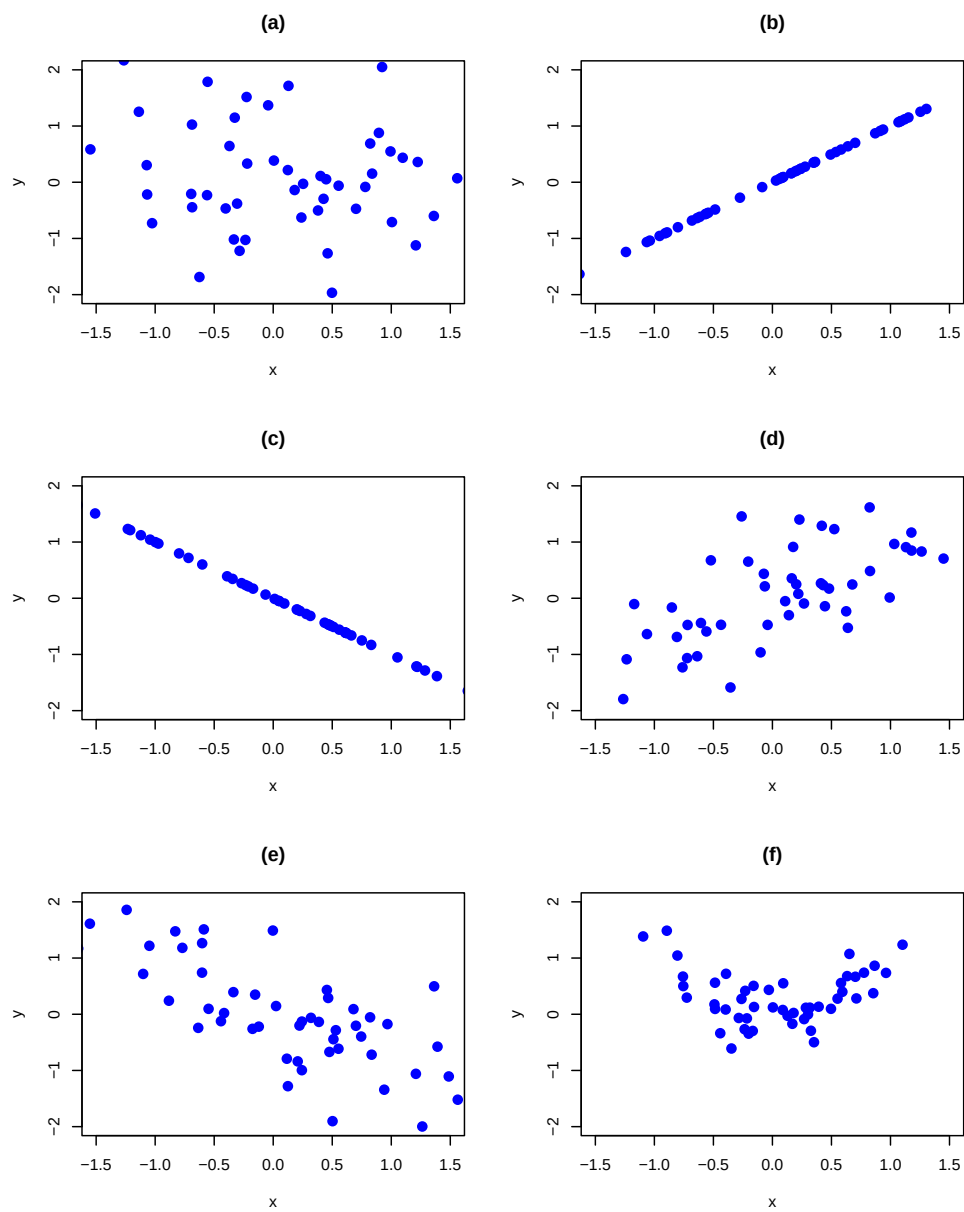


Figure 2.12: Probability scatterplots illustrating dependence between X and Y .

Definition 2.21 (Correlation). The correlation between two random variables X and Y is given by:

$$\rho_{XY} = \text{cor}(X, Y) = \frac{\text{cov}(X, Y)}{\sqrt{\text{var}(X)\text{var}(Y)}} = \frac{\sigma_{XY}}{\sigma_X \sigma_Y}.$$

The correlation coefficient, ρ_{XY} , is a scaled version of the covariance.

To see how covariance measures the direction of linear association, consider the probability scatterplot in Figure Figure 2.13. In the figure, each pair of points occurs with equal probability. The plot is separated into quadrants (right to left, top to bottom). In the first quadrant (black circles), the realized values satisfy $x < \mu_X, y > \mu_Y$ so that the product $(x - \mu_X)(y - \mu_Y) < 0$. In the second quadrant (blue squares), the values satisfy $x > \mu_X$ and $y > \mu_Y$ so that the product $(x - \mu_X)(y - \mu_Y) > 0$. In the third quadrant (red triangles), the values satisfy $x < \mu_X$ and $y < \mu_Y$ so that the product $(x - \mu_X)(y - \mu_Y) > 0$. Finally, in the fourth quadrant (green diamonds), $x > \mu_X$ but $y < \mu_Y$ so that the product $(x - \mu_X)(y - \mu_Y) < 0$. Covariance is then a probability weighted average all of the product terms in the four quadrants. For the values in Figure Figure 2.13, this weighted average is positive because most of the values are in the second and third quadrants.

Example 2.49 (Calculate covariance and correlation for discrete random variables). For the data in Table Table 2.3, you get:

$$\begin{aligned}\sigma_{XY} = \text{cov}(X, Y) &= (0 - 3/2)(0 - 1/2) \cdot 1/8 + (0 - 3/2)(1 - 1/2) \cdot 0 \\ &\quad + \dots + (3 - 3/2)(1 - 1/2) \cdot 1/8 = 1/4 \\ \rho_{XY} = \text{cor}(X, Y) &= \frac{1/4}{\sqrt{(3/4) \cdot (1/2)}} = 0.577\end{aligned}$$

■

2.2.4.1 Properties of covariance and correlation

Let X and Y be random variables and let a and b be constants. Some important properties of $\text{cov}(X, Y)$ are summarized in the following Proposition:

Proposition 2.6 (Properties of covariance).

1. $\text{cov}(X, X) = \text{var}(X)$
2. $\text{cov}(X, Y) = \text{cov}(Y, X)$
3. $\text{cov}(X, Y) = E[XY] - E[X]E[Y] = E[XY] - \mu_X \mu_Y$
4. $\text{cov}(aX, bY) = a \cdot b \cdot \text{cov}(X, Y)$
5. If X and Y are independent then $\text{cov}(X, Y) = 0$ (no association $\implies$ no linear association). However, if $\text{cov}(X, Y) = 0$ then X and Y are not necessarily independent (no linear association $\nRightarrow$ no association).

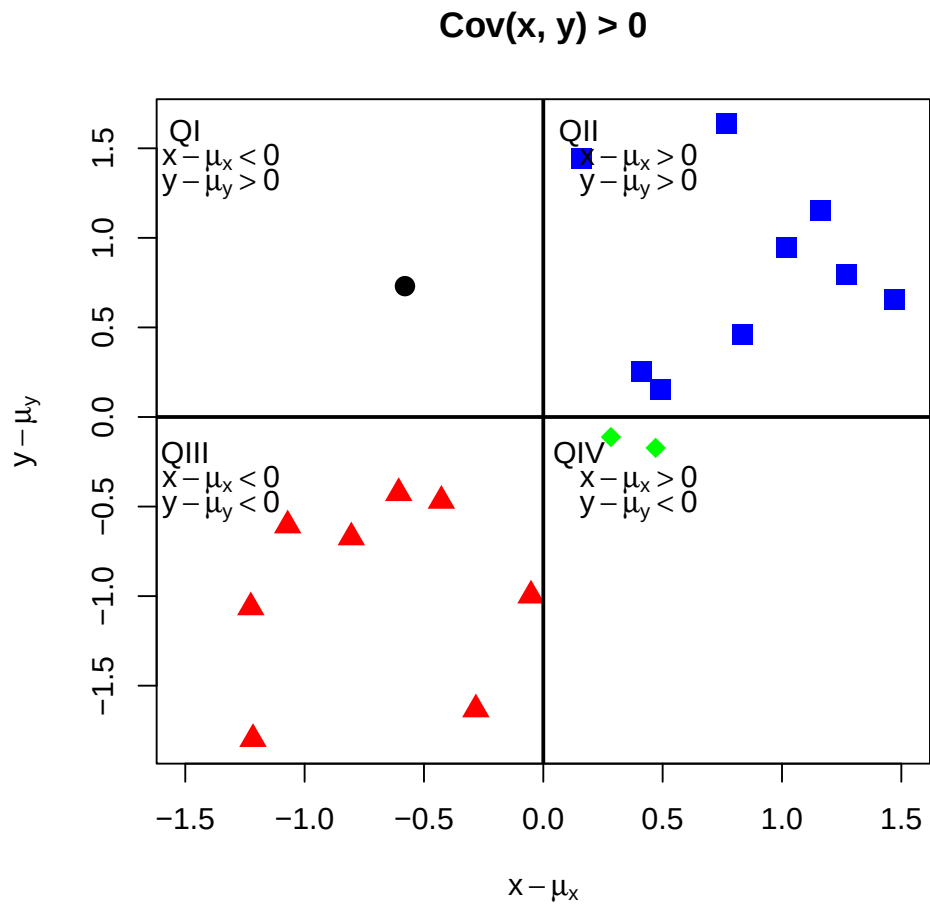


Figure 2.13: Probability scatterplot of discrete distribution with positive covariance. Each pair (X, Y) occurs with equal probability.

6. If X and Y are jointly normally distributed and $\text{cov}(X, Y) = 0$, then X and Y are independent.

The first two properties are intuitive. The third property results from expanding the definition of covariance:

$$\begin{aligned}
 \text{cov}(X, Y) &= E[(X - \mu_X)(Y - \mu_Y)] \\
 &= E[XY - X\mu_Y - \mu_X Y + \mu_X \mu_Y] \\
 &= E[XY] - E[X]\mu_Y - \mu_X E[Y] + \mu_X \mu_Y \\
 &= E[XY] - 2\mu_X \mu_Y + \mu_X \mu_Y \\
 &= E[XY] - \mu_X \mu_Y
 \end{aligned}$$

The fourth property follows from the linearity of expectations:

$$\begin{aligned}
 \text{cov}(aX, bY) &= E[(aX - a\mu_X)(bY - b\mu_Y)] \\
 &= a \cdot b \cdot E[(X - \mu_X)(Y - \mu_Y)] \\
 &= a \cdot b \cdot \text{cov}(X, Y)
 \end{aligned}$$

The fourth property shows that the value of $\text{cov}(X, Y)$ depends on the scaling of the random variables X and Y . By simply changing the scale of X or Y you can make $\text{cov}(X, Y)$ equal to any value that you want. Consequently, the numerical value of $\text{cov}(X, Y)$ is not informative about the strength of the linear association between X and Y . However, the sign of $\text{cov}(X, Y)$ is informative about the direction of linear association between X and Y .

The fifth property should be intuitive. Independence between the random variables X and Y means that there is no relationship, linear or nonlinear, between X and Y . However, the lack of a linear relationship between X and Y does not preclude a nonlinear relationship.

The last result illustrates an important property of the normal distribution: lack of covariance implies independence.

Some important properties of $\text{cor}(X, Y)$ are:

Proposition 2.7.

1. $-1 \leq \rho_{XY} \leq 1$
2. If $\rho_{XY} = 1$ then X and Y are perfectly positively linearly related. That is, $Y = aX + b$ where $a > 0$.
3. If $\rho_{XY} = -1$ then X and Y are perfectly negatively linearly related. That is, $Y = aX + b$ where $a < 0$.
4. If $\rho_{XY} = 0$ then X and Y are not linearly related but may be nonlinearly related.
5. $\text{cor}(aX, bY) = \text{cor}(X, Y)$ if $a > 0$ and $b > 0$; $\text{cor}(X, Y) = -\text{cor}(X, Y)$ if $a > 0, b < 0$ or $a < 0, b > 0$.

TBD: discuss the results briefly

2.2.5 Bivariate normal distributions

Let X and Y be distributed bivariate normal. The joint pdf is given by:

$$f(x, y) = \frac{1}{2\pi\sigma_X\sigma_Y\sqrt{1-\rho_{XY}^2}} \times \exp \left\{ -\frac{1}{2(1-\rho_{XY}^2)} \left[\left(\frac{x-\mu_X}{\sigma_X} \right)^2 + \left(\frac{y-\mu_Y}{\sigma_Y} \right)^2 - \frac{2\rho_{XY}(x-\mu_X)(y-\mu_Y)}{\sigma_X\sigma_Y} \right] \right\}$$

where $E[X] = \mu_X$, $E[Y] = \mu_Y$, $\text{sd}(X) = \sigma_X$, $\text{sd}(Y) = \sigma_Y$, and $\rho_{XY} = \text{cor}(X, Y)$. The correlation coefficient ρ_{XY} describes the dependence between X and Y . If $\rho_{XY} = 0$ then the pdf collapses to the pdf of the standard bivariate normal distribution. In this case, X and Y are independent random variables since $f(x, y) = f(x)f(y)$.

It can be shown that the marginal distributions of X and Y are normal: $X \sim N(\mu_X, \sigma_X^2)$, $Y \sim N(\mu_Y, \sigma_Y^2)$. In addition, it can be shown that the conditional distributions $f(x|y)$ and $f(y|x)$ are also normal with means given by:

$$\begin{aligned}\mu_{X|Y=y} &= \alpha_X + \beta_X \cdot y, \\ \mu_{Y|X=x} &= \alpha_Y + \beta_Y \cdot x,\end{aligned}$$

where,

$$\begin{aligned}\alpha_X &= \mu_X - \beta_X\mu_Y, \quad \beta_X = \sigma_{XY}/\sigma_Y^2, \\ \alpha_Y &= \mu_Y - \beta_Y\mu_X, \quad \beta_Y = \sigma_{XY}/\sigma_X^2,\end{aligned}$$

and variances given by,

$$\begin{aligned}\sigma_{X|Y=y}^2 &= \sigma_X^2 - \sigma_{XY}^2/\sigma_Y^2, \\ \sigma_{Y|X=x}^2 &= \sigma_Y^2 - \sigma_{XY}^2/\sigma_X^2.\end{aligned}$$

Notice that the conditional means (regression functions) are linear functions of x and y , respectively.

Example 2.50 (Expressing the bivariate normal distribution using matrix algebra). The formula for the bivariate normal distribution Equation 2.44 is a bit messy. you can greatly simplify the formula by using matrix algebra. Define the 2×1 vectors $\mathbf{x} = (x, y)'$ and $\boldsymbol{\mu} = (\mu_X, \mu_Y)'$, and the 2×2 matrix:

$$\Sigma = \begin{pmatrix} \sigma_X^2 & \sigma_{XY} \\ \sigma_{XY} & \sigma_Y^2 \end{pmatrix}$$

Then the bivariate normal distribution Equation 2.44 may be compactly expressed as:

$$f(\mathbf{x}) = \frac{1}{2\pi \det(\Sigma)^{1/2}} e^{-\frac{1}{2}(\mathbf{x}-\mu)'\Sigma^{-1}(\mathbf{x}-\mu)} \quad (2.44)$$

where,

$$\det(\Sigma) = \sigma_X^2 \sigma_Y^2 - \sigma_{XY}^2 = \sigma_X^2 \sigma_Y^2 - \sigma_X^2 \sigma_Y^2 \rho_{XY}^2 = \sigma_X^2 \sigma_Y^2 (1 - \rho_{XY}^2)$$

$$\Sigma^{-1} = \frac{1}{\det(\Sigma)} \begin{pmatrix} \sigma_Y^2 & -\sigma_{XY} \\ -\sigma_{XY} & \sigma_X^2 \end{pmatrix}$$

■

Example 2.51 (Plotting the bivariate normal distribution). The R package **mvtnorm** contains the functions `dmvnorm()`, `pmvnorm()`, and `qmvnorm()` which can be used to compute the bivariate normal pdf, cdf and quantiles, respectively. Plotting the bivariate normal distribution over a specified grid of x and y values in R can be done with the `persp()` function. First, I specify the parameter values for the joint distribution. Here, I choose $\mu_X = \mu_Y = 0$, $\sigma_X = \sigma_Y = 1$ and $\rho = 0.5$. I will use the `dmvnorm()` function to evaluate the joint pdf at these values. To do so, I must specify a covariance matrix of the form:

$$\Sigma = \begin{pmatrix} \sigma_X^2 & \sigma_{XY} \\ \sigma_{XY} & \sigma_Y^2 \end{pmatrix} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}.$$

In R this matrix can be created using:

```
sigma = matrix(c(1, 0.5, 0.5, 1), 2, 2)
```

Next I specify a grid of x and y values between -3 and 3 :

```
x = seq(-3, 3, length=25)
y = seq(-3, 3, length=25)
```

To evaluate the joint pdf over the two-dimensional grid I can use the `outer()` function:

```
# function to evaluate bivariate normal pdf on grid
bv.norm <- function(x, y, sigma) {
  z = cbind(x,y)
  return(dmvnorm(z, sigma=sigma))
}

# use outer function to evaluate pdf on 2D grid of x-y values
fxy = outer(x, y, bv.norm, sigma)
```


To create the 3D plot of the joint pdf, I use the `persp()` function:

```
persp(x, y, fxy, theta=60, phi=30, expand=0.5, ticktype="detailed",
      zlab="f(x,y)", col="grey")
```

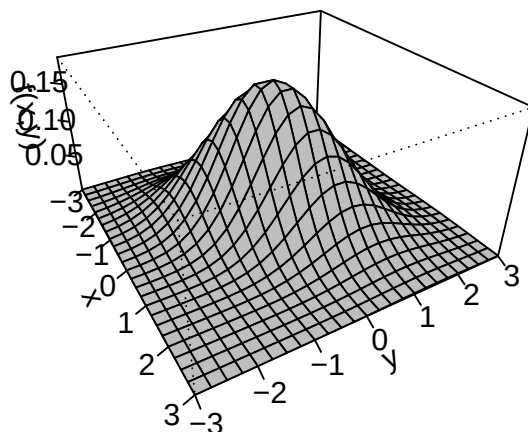


Figure 2.14: Bivariate normal pdf with $\mu_X = \mu_Y = 0$, $\sigma_X\sigma_Y = 1$ and $\rho = 0.5$.

The resulting plot is given in Figure Figure 2.14. ■

2.2.6 Expectation and variance of the sum of two random variables

Proposition 2.8 (Expectation and variance of the sum of two random variables). *Let X and Y be two random variables with well defined means, variances and covariance and let a and b be constants. Then the following results hold:*

$$\begin{aligned} E[aX + bY] &= aE[X] + bE[Y] = a\mu_X + b\mu_Y \\ \text{var}(aX + bY) &= a^2\text{var}(X) + b^2\text{var}(Y) + 2 \cdot a \cdot b \cdot \text{cov}(X, Y) \\ &= a^2\sigma_X^2 + b^2\sigma_Y^2 + 2 \cdot a \cdot b \cdot \sigma_{XY} \end{aligned}$$

The first result states that the expected value of a linear combination of two random variables is equal to a linear combination of the expected values of the random variables. This result indicates

that the expectation operator is a linear operator. In other words, expectation is additive. The second result states that variance of a linear combination of random variables is not a linear combination of the variances of the random variables. In particular, notice that covariance comes up as a term when computing the variance of the sum of two (not independent) random variables. Hence, the variance operator is not, in general, a linear operator. That is, variance, in general, is not additive.

It is instructive to go through the derivation of these results. Let X and Y be discrete random variables. Then,

$$\begin{aligned}
E[aX + bY] &= \sum_{x \in S_X} \sum_{y \in S_Y} (ax + by) \Pr(X = x, Y = y) \\
&= \sum_{x \in S_X} \sum_{y \in S_Y} ax \Pr(X = x, Y = y) + \sum_{x \in S_X} \sum_{y \in S_Y} by \Pr(X = x, Y = y) \\
&= a \sum_{x \in S_X} x \sum_{y \in S_Y} \Pr(X = x, Y = y) + b \sum_{y \in S_Y} y \sum_{x \in S_X} \Pr(X = x, Y = y) \\
&= a \sum_{x \in S_X} x \Pr(X = x) + b \sum_{y \in S_Y} y \Pr(Y = y) \\
&= aE[X] + bE[Y] = a\mu_X + b\mu_Y.
\end{aligned}$$

The result for continuous random variables is similar. Effectively, the summations are replaced by integrals and the joint probabilities are replaced by the joint pdf.

Next, let X and Y be discrete or continuous random variables. Then, using the definition of variance you get:

$$\begin{aligned}
\text{var}(aX + bY) &= E[(aX + bY - E[aX + bY])^2] \\
&= E[(aX + bY - a\mu_X - b\mu_Y)^2] \\
&= E[(a(X - \mu_X) + b(Y - \mu_Y))^2] \\
&= a^2 E[(X - \mu_X)^2] + b^2 E[(Y - \mu_Y)^2] + 2 \cdot a \cdot b \cdot E[(X - \mu_X)(Y - \mu_Y)] \\
&= a^2 \text{var}(X) + b^2 \text{var}(Y) + 2 \cdot a \cdot b \cdot \text{cov}(X, Y).
\end{aligned}$$

2.2.6.1 Linear combination of two normal random variables

The following proposition gives an important result concerning a linear combination of normal random variables.

Proposition 2.9 (Linear combination of normal random variables). *Let $X \sim N(\mu_X, \sigma_X^2)$, $Y \sim N(\mu_Y, \sigma_Y^2)$, $\sigma_{XY} = \text{cov}(X, Y)$ and a and b be constants. Define the new random variable Z as:*

$$Z = aX + bY.$$

Then,

$$Z \sim N(\mu_Z, \sigma_Z^2),$$

where $\mu_Z = a\mu_X + b\mu_Y$ and $\sigma_Z^2 = a^2\sigma_X^2 + b^2\sigma_Y^2 + 2ab\sigma_{XY}$.

This important result states that a linear combination of two normally distributed random variables is itself a normally distributed random variable. The proof of the result relies on the change of variables theorem from calculus and is omitted. Not all random variables have the nice property that their distributions are closed under addition.

Example 2.52 (Portfolio of two assets). Consider a portfolio of two stocks A (Amazon) and B (Boeing) with investment shares x_A and x_B with $x_A + x_B = 1$. Let R_A and R_B denote the simple monthly returns on these assets, and assume that $R_A \sim N(\mu_A, \sigma_A^2)$ and $R_B \sim N(\mu_B, \sigma_B^2)$. Furthermore, let $\sigma_{AB} = \rho_{AB}\sigma_A\sigma_B = \text{cov}(R_A, R_B)$. The portfolio return is $R_p = x_AR_A + x_BR_B$, which is a linear function of two random variables. Using the properties of linear functions of random variables, you get:

$$\begin{aligned}\mu_p &= E[R_p] = x_A E[R_A] + x_B E[R_B] = x_A \mu_A + x_B \mu_B \\ \sigma_p^2 &= \text{var}(R_p) = x_A^2 \text{var}(R_A) + x_B^2 \text{var}(R_B) + 2x_A x_B \text{cov}(R_A, R_B) \\ &= x_A^2 \sigma_A^2 + x_B^2 \sigma_B^2 + 2x_A x_B \sigma_{AB}.\end{aligned}$$

■

2.3 Multivariate Distributions

Multivariate distributions are used to characterize the joint distribution of a collection of N random variables $X_1, X_2, \dots, X_N$ for $N > 1$. The mathematical formulation of this joint distribution can be quite complex and typically makes use of matrix algebra. Here, I summarize some basic properties of multivariate distributions without the use of matrix algebra. In chapter Chapter 3, I will show how matrix algebra can greatly simplify the description of multivariate distributions.

2.3.1 Discrete random variables

Let $X_1, X_2, \dots, X_N$ be N discrete random variables with sample spaces $S_{X_1}, S_{X_2}, \dots, S_{X_N}$. The likelihood that these random variables take values in the joint sample space $S_{X_1} \times S_{X_2} \times \dots \times S_{X_N}$ is given by the joint probability function:

$$p(x_1, x_2, \dots, x_N) = \Pr(X_1 = x_1, X_2 = x_2, \dots, X_N = x_N).$$

For $N > 2$ it is not easy to represent the joint probabilities in a table like Table 2.3 or to visualize the distribution.

Marginal distributions for each variable X_i can be derived from the joint distribution as in Equation 2.25 by summing the joint probabilities over the other variables $j \neq i$. For example,

$$p(x_1) = \sum_{x_2 \in S_{X_2}, \dots, x_N \in S_{X_N}} p(x_1, x_2, \dots, x_N).$$

With N random variables, there are numerous conditional distributions that can be formed. For example, the distribution of X_1 given $X_2 = x_2, \dots, X_N = x_N$ is determined using:

$$\Pr(X_1 = x_1 | X_2 = x_2, \dots, X_N = x_N) = \frac{\Pr(X_1 = x_1, X_2 = x_2, \dots, X_N = x_N)}{\Pr(X_2 = x_2, \dots, X_N = x_N)}.$$

Similarly, the joint distribution of X_1 and X_2 given $X_3 = x_3, \dots, X_N = x_N$ is given by:

$$\begin{aligned} & \Pr(X_1 = x_1, X_2 = x_2 | X_3 = x_3, \dots, X_N = x_N) \\ &= \frac{\Pr(X_1 = x_1, X_2 = x_2, \dots, X_N = x_N)}{\Pr(X_3 = x_3, \dots, X_N = x_N)}. \end{aligned}$$

2.3.2 Continuous random variables

Let $X_1, X_2, \dots, X_N$ be N continuous random variables each taking values on the real line. The joint pdf is a function $f(x_1, x_2, \dots, x_N) \geq 0$ such that:

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \dots \int_{-\infty}^{\infty} f(x_1, x_2, \dots, x_N) dx_1 dx_2 \dots dx_N = 1.$$

Joint probabilities of $x_{11} \leq X_1 \leq x_{12}, x_{21} \leq X_2 \leq x_{22}, \dots, x_{N1} \leq X_N \leq x_{N2}$ are computed by solving the integral equation:

$$\int_{x_{11}}^{x_{12}} \int_{x_{21}}^{x_{22}} \dots \int_{x_{N1}}^{x_{N2}} f(x_1, x_2, \dots, x_N) dx_1 dx_2 \dots dx_N. \quad (2.45)$$

For most multivariate distributions, the integral in Equation 2.45 cannot be solved analytically and must be approximated numerically.

The marginal pdf for x_i is found by integrating the joint pdf with respect to the other variables. For example, the marginal pdf for x_1 is found by solving:

$$f(x_1) = \int_{-\infty}^{\infty} \dots \int_{-\infty}^{\infty} f(x_1, x_2, \dots, x_N) dx_2 \dots dx_N.$$

Conditional pdf for a single random variable or a collection of random variables are defined in the obvious way.

2.3.3 Independence

A collection of N random variables are independent if their joint distribution factors into the product of all of the marginal distributions:

$$\begin{aligned} p(x_1, x_2, \dots, x_N) &= p(x_1)p(x_2) \cdots p(x_N) \text{ for } X_i \text{ discrete,} \\ f(x_1, x_2, \dots, x_N) &= f(x_1)f(x_2) \cdots f(x_N) \text{ for } X_i \text{ continuous.} \end{aligned}$$

In addition, if N random variables are independent then any functions of these random variables are also independent.

2.3.4 Dependence concepts

In general, it is difficult to define dependence concepts for collections of more than two random variables. Dependence is typically only defined between pairwise random variables. Hence, covariance and correlation are also useful concepts when dealing with more than two random variables.

For N random variables $X_1, X_2, \dots, X_N$, with mean values $\mu_i = E[X_i]$ and variances $\sigma_i^2 = \text{var}(X_i)$, the pairwise covariances and correlations are defined as:

$$\begin{aligned} \text{cov}(X_i, X_j) &= \sigma_{ij} = E[(X_i - \mu_i)(X_j - \mu_j)], \\ \text{cov}(X_i, X_j) &= \rho_{ij} = \frac{\sigma_{ij}}{\sigma_i \sigma_j}, \end{aligned}$$

for $i \neq j$. There are $N(N-1)/2$ pairwise covariances and correlations. Often, these values are summarized using matrix algebra in an $N \times N$ covariance matrix Σ and an $N \times N$ correlation matrix $\mathbf{C}$:

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1N} \\ \sigma_{12} & \sigma_2^2 & \cdots & \sigma_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{1N} & \sigma_{2N} & \cdots & \sigma_N^2 \end{pmatrix}, \quad (2.46)$$

$$\mathbf{C} = \begin{pmatrix} 1 & \rho_{12} & \cdots & \rho_{1N} \\ \rho_{12} & 1 & \cdots & \rho_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \rho_{1N} & \rho_{2N} & \cdots & 1 \end{pmatrix}. \quad (2.47)$$

These matrices are formally defined in chapter [Chapter 3](#).

2.3.5 Linear combinations of N random variables

Many of the results for manipulating a collection of random variables generalize in a straightforward way to the case of more than two random variables. The details of the generalizations are not important for our purposes. However, the following results will be used repeatedly throughout the book.

Proposition 2.10 (Linear combination of random variables). *Let $X_1, X_2, \dots, X_N$ denote a collection of N random variables (discrete or continuous) with means μ_i , variances σ_i^2 and covariances σ_{ij} . Define the new random variable Z as a linear combination:*

$$Z = a_1X_1 + a_2X_2 + \dots + a_NX_N$$

where $a_1, a_2, \dots, a_N$ are constants. Then the following results hold:

$$\begin{aligned}\mu_Z &= E[Z] = a_1E[X_1] + a_2E[X_2] + \dots + a_NE[X_N] \\ &= \sum_{i=1}^N a_iE[X_i] = \sum_{i=1}^N a_i\mu_i. \\ \sigma_Z^2 &= \text{var}(Z) = a_1^2\sigma_1^2 + a_2^2\sigma_2^2 + \dots + a_N^2\sigma_N^2 \\ &\quad + 2a_1a_2\sigma_{12} + 2a_1a_3\sigma_{13} + \dots + a_1a_N\sigma_{1N} \\ &\quad + 2a_2a_3\sigma_{23} + 2a_2a_4\sigma_{24} + \dots + a_2a_N\sigma_{2N} \\ &\quad + \dots + \\ &\quad + 2a_{N-1}a_N\sigma_{(N-1)N} \\ &= \sum_{i=1}^N a_i^2\sigma_i^2 + 2 \sum_{i=1}^N \sum_{j \neq i}^N a_i a_j \sigma_{ij}.\end{aligned}$$

The derivation of these results is very similar to the bivariate case and so is omitted. In addition, if all of the X_i are normally distributed then Z is also normally distributed with mean μ_Z and variance σ_Z^2 as described above.

The variance of a linear combination of N random variables contains N variance terms and $N(N-1)$ covariance terms. For $N = 2, 5, 10$ and 100 the number of covariance terms in $\text{var}(Z)$ is 2, 20, 90 and 9,900, respectively. Notice that when N is large there are many more covariance terms than variance terms in $\text{var}(Z)$.

The expression for $\text{var}(Z)$ is messy. It can be simplified using matrix algebra notation, as explained in detail in chapter Chapter 3. To preview, define the $N \times 1$ vectors $\mathbf{X} = (X_1, \dots, X_N)'$ and $\mathbf{a} = (a_1, \dots, a_N)'$. Then $Z = \mathbf{a}'\mathbf{X}$ and $\text{var}(Z) = \text{var}(\mathbf{a}'\mathbf{X}) = \mathbf{a}'\Sigma\mathbf{a}$, where Σ is the $N \times N$ covariance matrix, which is much more compact than Equation 3.13.

Example 2.53 (Square-root-of-time rule for multi-period continuously compounded returns). Let r_t denote the continuously compounded monthly return on an asset at times $t = 1, \dots, 12$. Assume that $r_1, \dots, r_{12}$ are independent and identically distributed (iid) $N(\mu, \sigma^2)$. Recall, the annual continuously compounded return is equal to the sum of twelve monthly continuously compounded returns: $r_A = r(12) = \sum_{t=1}^{12} r_t$. Since each monthly return is normally distributed, the annual return is also normally distributed. The mean of $r(12)$ is:

$$\begin{aligned} E[r(12)] &= E\left[\sum_{t=1}^{12} r_t\right] \\ &= \sum_{t=1}^{12} E[r_t] \text{ (by linearity of expectation),} \\ &= \sum_{t=1}^{12} \mu \text{ (by identical distributions),} \\ &= 12 \cdot \mu. \end{aligned}$$

Hence, the expected 12-month (annual) return is equal to 12 times the expected monthly return. The variance of $r(12)$ is:

$$\begin{aligned} \text{var}(r(12)) &= \text{var}\left(\sum_{t=1}^{12} r_t\right) \\ &= \sum_{t=1}^{12} \text{var}(r_t) \text{ (by independence),} \\ &= \sum_{j=0}^{11} \sigma^2 \text{ (by identical distributions),} \\ &= 12 \cdot \sigma^2, \end{aligned}$$

so that the annual variance is also equal to 12 times the monthly variance⁶. Hence, the annual standard deviation is $\sqrt{12}$ times the monthly standard deviation: $\text{sd}(r(12)) = \sqrt{12}\sigma$ (this result is known as the *square-root-of-time rule*). Therefore, $r(12) \sim N(12\mu, 12\sigma^2)$. ■

2.3.6 Covariance between linear combinations of random variables

Consider the linear combinations of two random variables:

$$\begin{aligned} Y &= aX_1 + bX_2, \\ Z &= cX_3 + dX_4, \end{aligned}$$

⁶This result often causes some confusion. It is easy to make the mistake and say that the annual variance is $(12)^2 = 144$ times the monthly variance. This result would occur if $r_A = 12r_t$, so that $\text{var}(r_A) = (12)^2 \text{var}(r_t) = 144 \text{var}(r_t)$.

where a, b, c and d are constants. The covariance between Y and Z is,

$$\begin{aligned}
\text{cov}(Y, Z) &= \text{cov}(aX_1 + bX_2, cX_3 + dX_4) \\
&= E[((aX_1 + bX_2) - (a\mu_1 + b\mu_2))((cX_3 + dX_4) - (c\mu_3 + d\mu_4))] \\
&= E[(a(X_1 - \mu_1) + b(X_2 - \mu_2))(c(X_3 - \mu_3) + d(X_4 - \mu_4))] \\
&= acE[(X_1 - \mu_1)(X_3 - \mu_3)] + adE[(X_1 - \mu_1)(X_4 - \mu_4)] \\
&\quad + bcE[(X_2 - \mu_2)(X_3 - \mu_3)] + bdE[(X_2 - \mu_2)(X_4 - \mu_4)] \\
&= accov(X_1, X_3) + adcov(X_1, X_4) + bccov(X_2, X_3) + bdcov(X_2, X_4).
\end{aligned}$$

Hence, covariance is additive for linear combinations of random variables. The result above extends in an obvious way to arbitrary linear combinations of random variables.

2.4 Further Reading: Review of Random Variables

This material in this chapter is a selected survey of topics, with applications to finance, typically covered in an introductory probability and statistics course using calculus. A good textbook for such a course is (DeGroot and Schervish 2011). Chapter 1 of (Carmona 2014) and the appendix of (Ruppert and Matteson 2015) give a slightly more advanced review than is presented here. Two slightly more advanced books, oriented towards economics students, are (Amemiya 1994) and (Goldberger 1991). Good introductory treatments of probability and statistics using R include (Dalgaard 2002) and (Verzani 2005). (Pfaff 2013) discusses some useful financial asset return distributions and their implementation in R.

2.5 Exercises

Exercise 2.1. Suppose X is a normally distributed random variable with mean 0.05 and variance $(0.10)^2$. That is, $X \sim N(0.05, (0.10)^2)$. Use the `pnorm()` and `qnorm()` functions in R to compute the following:

1. $\Pr(X \geq 0.10)$, $\Pr(X \leq -0.10)$.
2. $\Pr(-0.05 \leq X \leq 0.15)$.
3. 1% and 5% quantiles, $q_{0.01}^R$ and $q_{0.05}^R$.
4. 95% and 99% quantiles, $q_{0.95}^R$ and $q_{0.99}^R$.

Exercise 2.2. Let X denote the monthly return on Microsoft Stock and let Y denote the monthly return on Starbucks stock. Assume that $X \sim N(0.05, (0.10)^2)$ and $Y \sim N(0.025, (0.05)^2)$.

1. Using a grid of values between -0.25 and 0.35 , plot the normal curves for X and Y . Make sure that both normal curves are on the same plot.
2. Plot the points $(\sigma_X, \mu_X) = (0.10, 0.05)$ and $(\sigma_Y, \mu_Y) = (0.05, 0.025)$ in an x-y plot.
3. Comment on the risk-return tradeoffs for the two stocks.

Exercise 2.3. Let X_1 , X_2 , Y_1 , and Y_2 be random variables. Show that

$$\begin{aligned}\text{Cov}(X_1 + X_2, Y_1 + Y_2) &= \text{Cov}(X_1, Y_1) + \text{Cov}(X_1, Y_2) + \\ &\quad \text{Cov}(X_2, Y_1) + \text{Cov}(X_2, Y_2).\end{aligned}$$

Exercise 2.4. Let X be a random variable with $\text{var}(X) < \infty$. Show that $\text{var}(X) = E[(X - \mu_X)^2] = E[X^2] - \mu_X^2$.

Exercise 2.5. Let X and Y be a random variables such that $\text{cov}(X, Y) < \infty$.

1. If $Y = aX + b$ where $a > 0$ show that $\rho_{XY} = \text{cor}(X, Y) = 1$.
2. If $Y = aX + b$ where $a < 0$ show that $\rho_{XY} = \text{cor}(X, Y) = -1$.

Exercise 2.6. Let R denote the simple monthly return on Microsoft stock and let W_0 denote initial wealth to be invested over the month. Assume that $R \sim N(0.04, (0.09)^2)$ and that $W_0 = \$100,000$. Determine the 1% and 5% Value-at-Risk (VaR) over the month on the investment. That is, determine the loss in investment value that may occur over the next month with 1% probability and with 5% probability.

Exercise 2.7. Let R_t denote the simple monthly return and assume $R_t \sim iid N(\mu, \sigma^2)$. Consider the 2-period return $R_t(2) = (1 + R_t)(1 + R_{t-1}) - 1$.

1. Assuming that $\text{cov}(R_t, R_{t-1}) = 0$, show that $E[R_t R_{t-1}] = \mu^2$. Hint: use $\text{cov}(R_t, R_{t-1}) = E[R_t R_{t-1}] - E[R_t]E[R_{t-1}]$.
2. Show that $E[R_t(2)] = (1 + \mu^2) - 1$.
3. Is $R_t(2)$ normally distributed? Justify your answer.

Exercise 2.8. Let r denote the continuously compounded monthly return on Microsoft stock and let W_0 denote initial wealth to be invested over the month. Assume that $r \sim iid N(0.04, (0.09)^2)$ and that $W_0 = \$100,000$.

1. Determine the 1% and 5% value-at-risk (VaR) over the month on the investment. That is, determine the loss in investment value that may occur over the next month with 1% probability and with 5% probability. (Hint: compute the 1% and 5% quantile from the Normal distribution for r and then convert the continuously compounded return quantile to a simple return quantile using the transformation $R = e^r - 1$.)
2. Let $r(12) = r_1 + r_2 + \dots + r_{12}$ denote the 12-month (annual) continuously compounded return. Compute $E[r(12)]$, $\text{var}(r(12))$ and $\text{sd}(r(12))$. What is the probability distribution of $r(12)$?
3. Using the probability distribution for $r(12)$, determine the 1% and 5% value-at-risk (VaR) over the year on the investment.

Exercise 2.9. Let R_{VFINX} and R_{AAPL} denote the monthly simple returns on VFINX (Vanguard S&P 500 index) and AAPL (Apple stock). Suppose that $R_{VFINX} \sim i.i.d. N(0.013, (0.037)^2)$, $R_{AAPL} \sim i.i.d. N(0.028, (0.073)^2)$.

1. Sketch the normal distributions for the two assets on the same graph. Show the mean values and the ranges within 2 standard deviations of the mean. Which asset appears to be more risky?
2. Plot the risk-return tradeoff for the two assets. That is, plot the mean values of each asset on the y-axis and the standard deviations on the x-axis. Comment on the relationship.
3. Let $W_0 = \$1,000$ be the initial wealth invested in each asset. Compute the 1% monthly Value-at-Risk values for each asset. (Hint: $q_{0.01}^Z = -2.326$).
4. Continue with the above question, state in words what the 1% Value-at-Risk numbers represent (i.e., explain what 1% Value-at-Risk for a one month \$1,000 investment means).
5. The normal distribution can be used to characterize the probability distribution of monthly simple returns or monthly continuously compounded returns. What are two problems with using the normal distribution for simple returns? Given these two problems, why might it be better to use the normal distribution for continuously compounded returns?

Exercise 2.10. In this question, you will examine the chi-square and Student's t distributions. A chi-square random variable with n degrees of freedom, denoted $X \sim \chi_n^2$, is defined as $X = Z_1^2 + Z_2^2 + \dots + Z_n^2$ where $Z_1, Z_2, \dots, Z_n$ are $iid N(0, 1)$ random variables. Notice that X only takes positive values. A Student's t random variable with n degrees of freedom, denoted $t \sim t_n$, is

defined as $t = Z/\sqrt{X/n}$, where $Z \sim N(0, 1)$, $X \sim \chi_n^2$ and Z is independent of X . The Student's t distribution is similar to the standard normal except that it has fatter tails.

1. On the same graph, plot the probability curves of chi-squared distributed random variables with 1, 2, 5 and 10 degrees of freedom. Use different colors and line styles for each curve. Hint: In R the density of the chi-square distribution is computed using the function `dchisq()`.
2. On the same graph, plot the probability curves of Student's t distributed random variables with 1, 2, 5 and 10 degrees of freedom. Also include the probability curve for the standard normal distribution. Use different colors and line styles for each curve. Hint: In R the density of the chi-square distribution is computed using the function `dt()`.

Exercise 2.11. Consider the following joint distribution of X and Y :

X/Y	1	2	3
1	0.1	0.2	0
2	0.1	0	0.2
3	0	0.1	0.3

1. Find the marginal distributions of X and Y . Using these distributions, compute $E[X]$, $\text{var}(X)$, $\text{sd}(X)$, $E[Y]$, $\text{var}(Y)$ and $\text{sd}(Y)$.
2. Compute $\text{cov}(X, Y)$ and $\text{cor}(X, Y)$.
3. Find the conditional distributions $\Pr(X|Y = y)$ for $y = 1, 2, 3$ and the conditional distributions $\Pr(Y|X = x)$ for $x = 1, 2, 3$.
4. Using the conditional distributions $\Pr(X|Y = y)$ for $y = 1, 2, 3$ and $\Pr(Y|X = x)$ for $x = 1, 2, 3$ compute $E[X|Y = y]$ and $E[Y|X = x]$.
5. Using the conditional distributions $\Pr(X|Y = y)$ for $y = 1, 2, 3$ and $\Pr(Y|X = x)$ for $x = 1, 2, 3$ compute $\text{var}[X|Y = y]$ and $\text{var}[Y|X = x]$.
6. Are X and Y independent? Fully justify your answer.

Exercise 2.12. Let X and Y be distributed bivariate normal with $\mu_X = 0.05$, $\mu_Y = 0.025$, $\sigma_X = 0.10$, and $\sigma_Y = 0.05$.

1. Using R package **mvtnorm** function `rmvnorm()`, simulate 100 observations from the bivariate normal distribution with $\rho_{XY} = 0.9$. Using the `plot()` function create a scatterplot of the observations and comment on the direction and strength of the linear association. Using the function `pmvnorm()`, compute the joint probability $\Pr(X \leq, Y \leq 0)$.
2. Using R package **mvtnorm** function `rmvnorm()`, simulate 100 observations from the bivariate normal distribution with $\rho_{XY} = -0.9$. Using the `plot()` function create a scatterplot of the observations and comment on the direction and strength of the linear association. Using the function `pmvnorm()`, compute the joint probability $\Pr(X \leq, Y \leq 0)$.

3. Using R package **mvtnorm** function `rmvnorm()`, simulate 100 observations from the bivariate normal distribution with $\rho_{XY} = 0$. Using the `plot()` function create a scatterplot of the observations and comment on the direction and strength of the linear association. Using the function `pmvnorm()`, compute the joint probability $Pr(X \leq, Y \leq 0)$.

3 Matrix Algebra Review

Updated: September 3, 2026

Copyright © Eric Zivot 2026

This chapter reviews some basic matrix algebra concepts that we will use throughout the book. Many of the formulas that involve multiple variables that we derive in the book can be simplified using matrix algebra. This aids in understanding general concepts. For example, portfolio risk and return calculations for portfolios with more than two assets is easy to express using matrix algebra. If returns are jointly normally distributed, then this multivariate normal distribution can be expressed concisely using matrix algebra. The first order conditions involving the portfolio weights for mean-variance optimized portfolios is a system of linear equations that can be set-up and solved using matrix algebra. In addition, R is a matrix programming language. Pretty much anything that you can represent in matrix algebra you can evaluate in R using matrix calculations that have a syntax very much like the matrix algebra representation. Lastly, many R calculations can be efficiently evaluated if they are *vectorized* - that is, if they operate on vectors of elements instead of looping over individual elements.

This chapter is organized as follows. Section [3.1](#) gives definitions of matrices and vectors and shows how to represent these objects in R. Section [3.2](#) reviews basic matrix operations (addition, subtraction, multiplication), and section [3.3](#) shows how common summation calculations can be represented using matrix notation. Section [3.4](#) covers systems of linear equations and shows how the inverse of a matrix is related to the solutions to a system of linear equations. Section [3.5](#) discusses positive definite matrices and Cholesky decomposition. The use of matrix algebra for representing multivariate probability distributions is presented in section [3.6](#), and portfolio calculations using matrix algebra is presented in section [3.7](#). Section [3.8](#) concludes with a discussion of derivatives of some simple matrix functions.

The R packages used in this chapter are **Matrix** and **mvtnorm**. Make sure these packages are downloaded and installed prior to replicating the R examples in this chapter.

3.1 Matrices and Vectors

Definition 3.1 (Matrix). A *matrix* is just an array of numbers. The *dimension* of a matrix is determined by the number of its rows and columns.

For example, a matrix $\mathbf{A}$ with n rows and m columns is illustrated below:

$$\underset{(n \times m)}{\mathbf{A}} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix}$$

where a_{ij} denotes the i^{th} row and j^{th} column element of $\mathbf{A}$.

Definition 3.2 (Vector). A *vector* is simply a matrix with 1 column.

For example,

$$\underset{(n \times 1)}{\mathbf{x}} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

is an $n \times 1$ vector with elements $x_1, x_2, \dots, x_n$. Vectors and matrices are often written in bold type (or underlined) to distinguish them from scalars (single elements of vectors or matrices).

Example 3.1 (Matrix creation in R). In R, matrix objects are created using the `matrix()` function. For example, to create the 2×3 matrix:

$$\underset{(2 \times 3)}{\mathbf{A}} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

use,

```
matA = matrix(data=c(1,2,3,4,5,6),nrow=2,ncol=3,byrow=TRUE)
matA
#>      [,1] [,2] [,3]
#> [1,]    1    2    3
#> [2,]    4    5    6
class(matA)
#> [1] "matrix" "array"
```

The optional argument `byrow=TRUE` fills the matrix row-by-row.¹ The default is `byrow=FALSE` which fills the matrix column-by-column:

¹When specifying logical variables in R always spell out `TRUE` and `FALSE` instead of using `T` and `F`. Upon startup R defines the variables `T=TRUE` and `F=FALSE` so that `T` and `F` can be used as substitutes for `TRUE` and `FALSE`, respectively. However, this shortcut is not recommended because the variables `T` and `F` could be reassigned during subsequent programming.

```
matrix(data=c(1,2,3,4,5,6),nrow=2,ncol=3)
#>      [,1] [,2] [,3]
#> [1,]    1    3    5
#> [2,]    2    4    6
```

Matrix objects (i.e., objects of class "matrix") have row and column dimension attributes which can be examined with the `dim()` function:

```
dim(matA)
#> [1] 2 3
```

The rows and columns can be given names using:

```
dimnames(matA) = list(c("row1","row2"),c("col1","col2","col3"))
matA
#>      col1 col2 col3
#> row1    1    2    3
#> row2    4    5    6
```

or,

```
colnames(matA) = c("col1", "col2", "col3")
rownames(matA) = c("row1", "row2")
matA
#>      col1 col2 col3
#> row1    1    2    3
#> row2    4    5    6
```

The elements of a matrix can be extracted or subsetted as follows:

```
matA[1, 2]
#> [1] 2
matA["row1", "col1"]
#> [1] 1
matA[1, ]
#> col1 col2 col3
#>    1    2    3
matA[, 2]
#> row1 row2
#>    2    5
```

To preserve the dimension attributes when subsetting, use the `drop=FALSE` option:

```
matA[1, , drop=FALSE]
#>      col1 col2 col3
#> row1    1    2    3
matA[, 2, drop=FALSE]
#>      col2
#> row1     2
#> row2     5
```



Example 3.2 (Creating vectors in R). Vectors can be created in R using a variety of methods. The easiest way is to use the combine function `c()`:

```
xvec = c(1,2,3)
xvec
#> [1] 1 2 3
xvec = 1:3
xvec
#> [1] 1 2 3
xvec = seq(from=1,to=3,by=1)
xvec
#> [1] 1 2 3
```

Vectors of numbers in R are of class `"numeric"` and do not have a dimension attribute:

```
class(xvec)
#> [1] "numeric"
dim(xvec)
#> NULL
```

The elements of a vector can be assigned names using the `names()` function:

```
names(xvec) = c("x1", "x2", "x3")
xvec
#> x1 x2 x3
#>  1  2  3
```

To force a dimension attribute onto a vector, coerce it to a `"matrix"` object using `as.matrix()`:


```
as.matrix(xvec)
#>      [,1]
#> x1      1
#> x2      2
#> x3      3
```

■

Definition 3.3 (Transpose). The *transpose* of an $n \times m$ matrix $\mathbf{A}$ is a new matrix with the rows and columns of $\mathbf{A}$ interchanged, and is denoted $\mathbf{A}'$ or $\mathbf{A}^\top$.

For example,

$$\begin{aligned} \mathbf{A}_{(2 \times 3)} &= \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad \mathbf{A}'_{(3 \times 2)} = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} \\ \mathbf{x}_{(3 \times 1)} &= \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \mathbf{x}'_{(1 \times 3)} = [1 \quad 2 \quad 3]. \end{aligned}$$

Definition 3.4 (Symmetric matrix). A *symmetric* matrix $\mathbf{A}$ is such that $\mathbf{A} = \mathbf{A}'$.

Obviously, symmetry can only occur if $\mathbf{A}$ is a *square* matrix; i.e., the number of rows of $\mathbf{A}$ is equal to the number of columns. For example, consider the 2×2 matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}.$$

Then,

$$\mathbf{A}' = \mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}.$$

Example 3.3 (Transpose of a matrix in R). To take the transpose of a matrix or vector, use the `t()` function:

```
matA = matrix(data=c(1,2,3,4,5,6),nrow=2,ncol=3,byrow=TRUE)
matA
#>      [,1] [,2] [,3]
#> [1,]    1    2    3
#> [2,]    4    5    6
t(matA)
#>      [,1] [,2]
```

```
#> [1,] 1 4
#> [2,] 2 5
#> [3,] 3 6
xvec = c(1,2,3)
t(xvec)
#>      [,1] [,2] [,3]
#> [1,] 1 2 3
```

Notice that, when applied to a "numeric" vector with n elements, the `t()` function returns a "matrix" object with dimension $1 \times n$.

■

3.2 Basic Matrix Operations

In this section we review the basic matrix operations of addition, subtraction, scalar multiplication and multiplication.

3.2.1 Addition and subtraction

Definition 3.5 (Matrix addition). Matrix addition and subtraction are element by element operations and only apply to matrices of the same dimension.

For example, let,

$$\mathbf{A} = \begin{bmatrix} 4 & 9 \\ 2 & 1 \end{bmatrix}, \mathbf{B} = \begin{bmatrix} 2 & 0 \\ 0 & 7 \end{bmatrix}.$$

Then,

$$\begin{aligned} \mathbf{A} + \mathbf{B} &= \begin{bmatrix} 4 & 9 \\ 2 & 1 \end{bmatrix} + \begin{bmatrix} 2 & 0 \\ 0 & 7 \end{bmatrix} = \begin{bmatrix} 4+2 & 9+0 \\ 2+0 & 1+7 \end{bmatrix} = \begin{bmatrix} 6 & 9 \\ 2 & 8 \end{bmatrix}, \\ \mathbf{A} - \mathbf{B} &= \begin{bmatrix} 4 & 9 \\ 2 & 1 \end{bmatrix} - \begin{bmatrix} 2 & 0 \\ 0 & 7 \end{bmatrix} = \begin{bmatrix} 4-2 & 9-0 \\ 2-0 & 1-7 \end{bmatrix} = \begin{bmatrix} 2 & 9 \\ 2 & -6 \end{bmatrix}. \end{aligned}$$

Example 3.4 (Matrix addition and subtraction in R). Matrix addition and subtraction is straightforward in R:

```
matA = matrix(c(4,9,2,1),2,2,byrow=TRUE)
matB = matrix(c(2,0,0,7),2,2,byrow=TRUE)
matA
#>      [,1] [,2]
```

```

#> [1,] 4 9
#> [2,] 2 1
matB
#>      [,1] [,2]
#> [1,] 2 0
#> [2,] 0 7
# matrix addition
matC = matA + matB
matC
#>      [,1] [,2]
#> [1,] 6 9
#> [2,] 2 8
# matrix subtraction
matD = matA - matB
matD
#>      [,1] [,2]
#> [1,] 2 9
#> [2,] 2 -6

```



3.2.2 Scalar multiplication

Here we refer to the multiplication of a matrix by a scalar number. This is also an element-by-element operation. For example, let $c = 2$ and,

$$\mathbf{A} = \begin{bmatrix} 3 & -1 \\ 0 & 5 \end{bmatrix}.$$

Then,

$$c \cdot \mathbf{A} = \begin{bmatrix} 2 \cdot 3 & 2 \cdot (-1) \\ 2 \cdot (0) & 2 \cdot 5 \end{bmatrix} = \begin{bmatrix} 6 & -2 \\ 0 & 10 \end{bmatrix}.$$

Example 3.5 (Scalar multiplication in R).

```

matA = matrix(c(3,-1,0,5),2,2,byrow=TRUE)
matC = 2*matA
matC
#>      [,1] [,2]
#> [1,] 6 -2

```



3.2.3 Matrix multiplication

Matrix multiplication only applies to *conformable* matrices.

Definition 3.6 (Conformable matrices). **A** and **B** are conformable matrices if the number of columns in **A** is equal to the number of rows in **B**.

For example, if **A** is $n \times m$ and **B** is $m \times p$ then **A** and **B** are conformable and the matrix product of **A** and **B** has dimension $n \times p$. The mechanics of matrix multiplication is best explained by example. Let,

$$\underset{(2 \times 2)}{\mathbf{A}} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \text{ and } \underset{(2 \times 3)}{\mathbf{B}} = \begin{bmatrix} 1 & 2 & 1 \\ 3 & 4 & 2 \end{bmatrix}.$$

Then,

$$\begin{aligned} \underset{(2 \times 2)}{\mathbf{A}} \cdot \underset{(2 \times 3)}{\mathbf{B}} &= \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 1 \\ 3 & 4 & 2 \end{bmatrix} \\ &= \begin{bmatrix} 1 \cdot 1 + 2 \cdot 3 & 1 \cdot 2 + 2 \cdot 4 & 1 \cdot 1 + 2 \cdot 2 \\ 3 \cdot 1 + 4 \cdot 3 & 3 \cdot 2 + 4 \cdot 4 & 3 \cdot 1 + 4 \cdot 2 \end{bmatrix} \\ &= \begin{bmatrix} 7 & 10 & 5 \\ 15 & 22 & 11 \end{bmatrix} = \underset{(2 \times 3)}{\mathbf{C}} \end{aligned}$$

The resulting matrix **C** has 2 rows and 3 columns. The (1, 1) element of **C** is the *dot product* of the first row of **A** with the first column of **B**:

$$\begin{pmatrix} 1 & 2 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 3 \end{pmatrix} = 1 \cdot 1 + 2 \cdot 3 = 7.$$

The (1, 2) element of **C** is the dot product of the first row of **A** with the second column of **B**:

$$\begin{pmatrix} 1 & 2 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ 4 \end{pmatrix} = 1 \cdot 2 + 2 \cdot 4 = 10.$$

In general, the (i, j) element of **C** is the dot product of the i th row of **A** with the j th column of **B**. If **A** is $n \times m$ and **B** is $m \times p$ then **C** = **A** · **B** is $n \times p$.

As another example, let,

$$\underset{(2 \times 2)}{\mathbf{A}} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \text{ and } \underset{(2 \times 1)}{\mathbf{B}} = \begin{bmatrix} 2 \\ 6 \end{bmatrix}.$$

Then,

$$\begin{aligned} \underset{(2 \times 2)}{\mathbf{A}} \cdot \underset{(2 \times 1)}{\mathbf{B}} &= \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 6 \end{bmatrix} \\ &= \begin{bmatrix} 1 \cdot 2 + 2 \cdot 6 \\ 3 \cdot 2 + 4 \cdot 6 \end{bmatrix} \\ &= \begin{bmatrix} 14 \\ 30 \end{bmatrix}. \end{aligned}$$

As a final example, let,

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}.$$

Then

$$\mathbf{x}'\mathbf{y} = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$$

Example 3.6 (Matrix multiplication in R). In R, matrix multiplication is performed with the %*% operator. For example:

```
matA = matrix(1:4,2,2,byrow=TRUE)
matB = matrix(c(1,2,1,3,4,2),2,3,byrow=TRUE)
matA
#>      [,1] [,2]
#> [1,]    1    2
#> [2,]    3    4
matB
#>      [,1] [,2] [,3]
#> [1,]    1    2    1
#> [2,]    3    4    2
dim(matA)
#> [1] 2 2
dim(matB)
#> [1] 2 3
matC = matA%*%matB
matC
#>      [,1] [,2] [,3]
#> [1,]    7   10    5
#> [2,]   15   22   11
```

The next example shows that matrix multiplication in R also works on numeric vectors:

```

matA = matrix(c(1,2,3,4), 2, 2, byrow=TRUE)
vecB = c(2,6)
matA%%vecB
#>      [,1]
#> [1,]   14
#> [2,]   30
vecX = c(1,2,3)
vecY = c(4,5,6)
t(vecX)%%vecY
#>      [,1]
#> [1,]   32
crossprod(vecX, vecY)
#>      [,1]
#> [1,]   32

```

■

3.2.3.1 Miscellaneous properties

Matrix multiplication satisfies the distributive property: Let $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{C}$ be conformable matrices. Then

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}. \quad (3.1)$$

Also, the transpose of the product of two matrices is the product of the transposes in opposite order:

$$(\mathbf{AB})' = \mathbf{B}'\mathbf{A}'. \quad (3.2)$$

3.2.4 The identity matrix

The *identity matrix* plays a similar role as the number 1. Multiplying any number by 1 gives back that number. In matrix algebra, pre-multiplying or post-multiplying a matrix $\mathbf{A}$ by a conformable identity matrix $\mathbf{I}$ gives back the matrix $\mathbf{A}$. To illustrate, let

$$\mathbf{I}_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

denote the 2-dimensional identity matrix and let

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

denote an arbitrary 2×2 matrix. Then,

$$\begin{aligned}\mathbf{I}_2 \cdot \mathbf{A} &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \\ &= \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} = \mathbf{A},\end{aligned}$$

and,

$$\begin{aligned}\mathbf{A} \cdot \mathbf{I}_2 &= \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} = \mathbf{A}.\end{aligned}$$

Example 3.7 (The identity matrix in R). Use the `diag()` function to create an identity matrix:

```
matI = diag(2)
matI
#>      [,1] [,2]
#> [1,]    1    0
#> [2,]    0    1
matA = matrix(c(1,2,3,4), 2, 2, byrow=TRUE)
matI%%matA
#>      [,1] [,2]
#> [1,]    1    2
#> [2,]    3    4
matA%%matI
#>      [,1] [,2]
#> [1,]    1    2
#> [2,]    3    4
```

■

3.2.5 Diagonal, lower triangular and upper triangular matrices

Consider an $n \times n$ matrix $\mathbf{A}$

$$\mathbf{A} = \begin{bmatrix} d_1 & u_{12} & u_{13} & \cdots & u_{1n} \\ l_{21} & d_2 & u_{23} & \cdots & u_{2n} \\ l_{31} & l_{32} & d_3 & \cdots & u_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & l_{n3} & \cdots & d_n \end{bmatrix}.$$

The main diagonal consists of the n elements $\{d_1, d_2 \dots d_n\}$. The lower triangle consists of the $n(n-1)/2$ elements $\{l_{21}, l_{31}, l_{32}, \dots, l_{n(n-1)}\}$ below the main diagonal, and the upper triangle consists of the $n(n-1)/2$ elements $\{u_{12}, u_{13}, u_{23}, \dots, u_{(n-1)n}\}$ above the main diagonal.

An n -dimensional *diagonal matrix* $\mathbf{D}$ is an $n \times n$ square matrix with an $n \times 1$ vector $\mathbf{d} = (d_1, \dots, d_n)'$ along the main diagonal and zeros elsewhere:

$$\mathbf{D} = \text{diag}(\mathbf{d}) = \begin{bmatrix} d_1 & 0 & 0 & \cdots & 0 \\ 0 & d_2 & 0 & \cdots & 0 \\ 0 & 0 & d_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & d_n \end{bmatrix}.$$

An $n \times n$ *lower triangular matrix* $\mathbf{L}$ has all values above the main diagonal equal to zero:

$$\mathbf{L} = \begin{bmatrix} d_1 & 0 & 0 & \cdots & 0 \\ l_{21} & d_2 & 0 & \cdots & 0 \\ l_{31} & l_{32} & d_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & l_{n3} & \cdots & d_n \end{bmatrix}.$$

An $n \times n$ *upper triangular matrix* $\mathbf{U}$ has all values below the main diagonal equal to zero:

$$\mathbf{U} = \begin{bmatrix} d_1 & u_{12} & u_{13} & \cdots & u_{1n} \\ 0 & d_2 & u_{23} & \cdots & u_{2n} \\ 0 & 0 & d_3 & \cdots & u_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & d_n \end{bmatrix}.$$

Example 3.8 (Creating a diagonal matrix in R). Diagonal matrices can be created with the `diag()` function. To create a 3×3 diagonal matrix with $\mathbf{d} = (1, 2, 3)$ along the main diagonal, use

```
matD = diag(1:3)
matD
#>      [,1] [,2] [,3]
#> [1,]    1    0    0
#> [2,]    0    2    0
#> [3,]    0    0    3
```

■

Example 3.9 (Extracting lower and upper triangular elements of a square matrix). The R func-

tions `lower.tri()` and `upper.tri()` can be used to extract the lower and upper triangular parts of a square matrix:

```
matA = matrix(c(1,2,3,4,5,6,7,8,9), 3, 3)
matA
#>      [,1] [,2] [,3]
#> [1,]    1    4    7
#> [2,]    2    5    8
#> [3,]    3    6    9
lower.tri(matA)
#>      [,1] [,2] [,3]
#> [1,] FALSE FALSE FALSE
#> [2,]  TRUE FALSE FALSE
#> [3,]  TRUE  TRUE FALSE
upper.tri(matA)
#>      [,1] [,2] [,3]
#> [1,] FALSE  TRUE  TRUE
#> [2,] FALSE FALSE  TRUE
#> [3,] FALSE FALSE FALSE
matA[lower.tri(matA)]
#> [1] 2 3 6
matA[upper.tri(matA)]
#> [1] 4 7 8
```

■

3.3 Representing Summation Using Matrix Notation

Consider the sum:

$$\sum_{k=1}^n x_k = x_1 + \cdots + x_n.$$

Let $\mathbf{x} = (x_1, \dots, x_n)'$ be an $n \times 1$ vector and $\mathbf{1} = (1, \dots, 1)'$ be an $n \times 1$ vector of ones. Then,

$$\mathbf{x}'\mathbf{1} = \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix} \cdot \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} = x_1 + \cdots + x_n = \sum_{k=1}^n x_k,$$

and,

$$\mathbf{1}'\mathbf{x} = \begin{bmatrix} 1 & \dots & 1 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = x_1 + \dots + x_n = \sum_{k=1}^n x_k.$$

Next, consider the sum of squared x values,

$$\sum_{k=1}^n x_k^2 = x_1^2 + \dots + x_n^2.$$

This sum can be conveniently represented as,

$$\mathbf{x}'\mathbf{x} = \begin{bmatrix} x_1 & \dots & x_n \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = x_1^2 + \dots + x_n^2 = \sum_{k=1}^n x_k^2.$$

Last, consider the sum of cross products,

$$\sum_{k=1}^n x_k y_k = x_1 y_1 + \dots + x_n y_n.$$

This sum can be compactly represented by,

$$\mathbf{x}'\mathbf{y} = \begin{bmatrix} x_1 & \dots & x_n \end{bmatrix} \cdot \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = x_1 y_1 + \dots + x_n y_n = \sum_{k=1}^n x_k y_k.$$

Note that $\mathbf{x}'\mathbf{y} = \mathbf{y}'\mathbf{x}$.

Example 3.10 (Computing sums in R). In R, summing the elements in a vector can be done using matrix algebra.

```
# create vector of 1's and a vector of x
onevec = rep(1,3)
onevec
#> [1] 1 1 1
xvec = c(1,2,3)
# sum elements in x
t(xvec)%*%onevec
#>      [,1]
#> [1,]    6
```

The functions `crossprod()` and `sum()` are generally computationally more efficient:

```
crossprod(xvec,onevec)
#>      [,1]
#> [1,]    6
sum(xvec)
#> [1]  6
```

Sums of squares are best computed using:

```
crossprod(xvec)
#>      [,1]
#> [1,]   14
sum(xvec^2)
#> [1]  14
```

The dot-product two vectors can be conveniently computed using the `crossprod()` function:

```
yvec = 4:6
xvec
#> [1] 1 2 3
yvec
#> [1] 4 5 6
crossprod(xvec,yvec)
#>      [,1]
#> [1,]   32
crossprod(yvec,xvec)
#>      [,1]
#> [1,]   32
```



3.4 Systems of Linear Equations

Consider the system of two linear equations:

$$x + y = 1, \tag{3.3}$$

$$2x - y = 1. \tag{3.4}$$

As shown in Figure Figure 3.1, equations Equation 3.3 and Equation 3.4 represent two straight lines which intersect at the point $x = 2/3$ and $y = 1/3$. This point of intersection is determined by

solving for the values of x and y such that $x + y = 2x - y$ ²

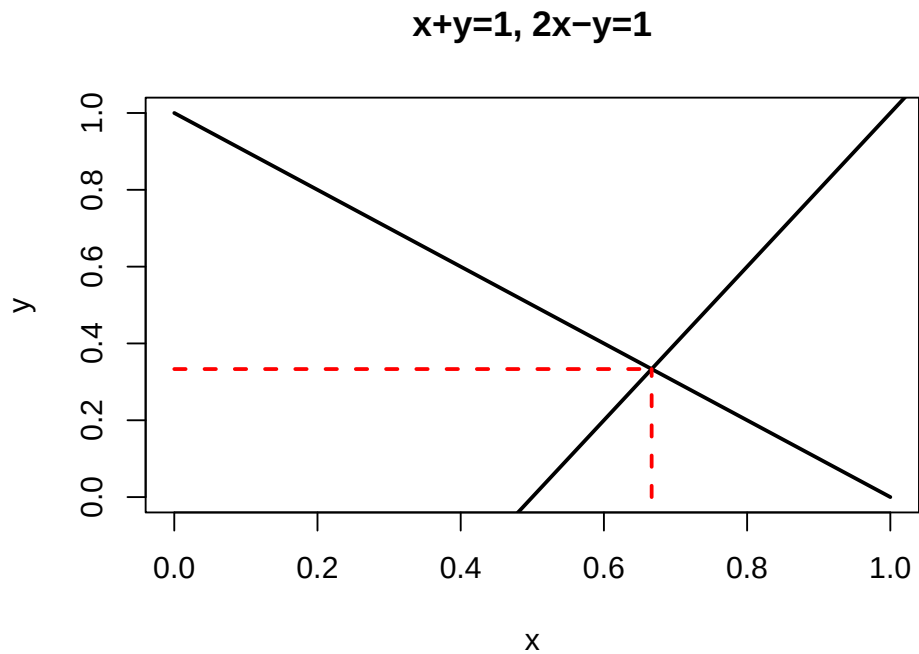


Figure 3.1: System of two linear equations.

The two linear equations can be written in matrix form as:

$$\begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

or,

$$\mathbf{A} \cdot \mathbf{z} = \mathbf{b},$$

where,

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix}, \mathbf{z} = \begin{bmatrix} x \\ y \end{bmatrix} \text{ and } \mathbf{b} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

If there was a (2×2) matrix $\mathbf{B}$, with elements b_{ij} , such that $\mathbf{B} \cdot \mathbf{A} = \mathbf{I}_2$, where $\mathbf{I}_2$ is the (2×2) identity matrix, then we could *solve* for the elements in $\mathbf{z}$ as follows. In the equation $\mathbf{A} \cdot \mathbf{z} = \mathbf{b}$, pre-multiply both sides by $\mathbf{B}$ to give:

$$\begin{aligned} \mathbf{B} \cdot \mathbf{A} \cdot \mathbf{z} &= \mathbf{B} \cdot \mathbf{b} \\ \implies \mathbf{I} \cdot \mathbf{z} &= \mathbf{B} \cdot \mathbf{b} \\ \implies \mathbf{z} &= \mathbf{B} \cdot \mathbf{b}, \end{aligned}$$

²Solving for x gives $x = 2y$. Substituting this value into the equation $x + y = 1$ gives $2y + y = 1$ and solving for y gives $y = 1/3$. Solving for x then gives $x = 2/3$.

or

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} b_{11} \cdot 1 + b_{12} \cdot 1 \\ b_{21} \cdot 1 + b_{22} \cdot 1 \end{bmatrix}.$$

If such a matrix $\mathbf{B}$ exists it is called the *inverse* of $\mathbf{A}$ and is denoted $\mathbf{A}^{-1}$. Intuitively, the inverse matrix $\mathbf{A}^{-1}$ plays a similar role as the inverse of a number. Suppose a is a number; e.g., $a = 2$. Then we know that $(1/a) \cdot a = a^{-1}a = 1$. Similarly, in matrix algebra $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_2$ where $\mathbf{I}_2$ is the identity matrix. Now, consider solving the equation $a \cdot x = b$. By simple division we have that $x = (1/a) \cdot b = a^{-1} \cdot b$. Similarly, in matrix algebra, if we want to solve the system of linear equations $\mathbf{Ax} = \mathbf{b}$ we pre-multiply by $\mathbf{A}^{-1}$ and get the solution $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$.

Using $\mathbf{B} = \mathbf{A}^{-1}$, we may express the solution for $\mathbf{z}$ as:

$$\mathbf{z} = \mathbf{A}^{-1}\mathbf{b}.$$

As long as we can determine the elements in $\mathbf{A}^{-1}$ then we can solve for the values of x and y in the vector $\mathbf{z}$. The system of linear equations has a solution as long as the two lines intersect, so we can determine the elements in $\mathbf{A}^{-1}$ provided the two lines are not parallel. If the two lines are parallel, then one of the equations is a multiple of the other. In this case, we say that $\mathbf{A}$ is *not invertible*.

There are general numerical algorithms for finding the elements of $\mathbf{A}^{-1}$ (e.g., Gaussian elimination) and R has these algorithms available. However, if $\mathbf{A}$ is a (2×2) matrix then there is a simple formula for $\mathbf{A}^{-1}$. Let $\mathbf{A}$ be a (2×2) matrix such that:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}.$$

Then,

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}, \quad (3.5)$$

where $\det(\mathbf{A}) = a_{11}a_{22} - a_{21}a_{12}$ denotes the determinant of $\mathbf{A}$ and is assumed to be not equal to

zero. By brute force matrix multiplication we can verify this formula:

$$\begin{aligned}
\mathbf{A}^{-1}\mathbf{A} &= \frac{1}{a_{11}a_{22} - a_{21}a_{12}} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \\
&= \frac{1}{a_{11}a_{22} - a_{21}a_{12}} \begin{bmatrix} a_{22}a_{11} - a_{12}a_{21} & a_{22}a_{12} - a_{12}a_{22} \\ -a_{21}a_{11} + a_{11}a_{21} & -a_{21}a_{12} + a_{11}a_{22} \end{bmatrix} \\
&= \frac{1}{a_{11}a_{22} - a_{21}a_{12}} \begin{bmatrix} a_{22}a_{11} - a_{12}a_{21} & 0 \\ 0 & -a_{21}a_{12} + a_{11}a_{22} \end{bmatrix} \\
&= \begin{bmatrix} \frac{a_{22}a_{11} - a_{12}a_{21}}{a_{11}a_{22} - a_{21}a_{12}} & 0 \\ 0 & \frac{-a_{21}a_{12} + a_{11}a_{22}}{a_{11}a_{22} - a_{21}a_{12}} \end{bmatrix} \\
&= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.
\end{aligned}$$

Let's apply the above rule to find the inverse of $\mathbf{A}$ in our example linear system Equation 3.3 and Equation 3.4:

$$\mathbf{A}^{-1} = \frac{1}{-1-2} \begin{bmatrix} -1 & -1 \\ -2 & 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} \\ \frac{2}{3} & \frac{-1}{3} \end{bmatrix}.$$

Notice that,

$$\mathbf{A}^{-1}\mathbf{A} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} \\ \frac{2}{3} & \frac{-1}{3} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Our solution for $\mathbf{z}$ is then,

$$\begin{aligned}
\mathbf{z} &= \mathbf{A}^{-1}\mathbf{b} \\
&= \begin{bmatrix} \frac{1}{3} & \frac{1}{3} \\ \frac{2}{3} & \frac{-1}{3} \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\
&= \begin{bmatrix} \frac{2}{3} \\ \frac{1}{3} \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix},
\end{aligned}$$

so that $x = 2/3$ and $y = 1/3$.

Example 3.11 (Solving systems of linear equations in R). In R, the `solve()` function is used to compute the inverse of a matrix and solve a system of linear equations. The linear system $x + y = 1$ and $2x - y = 1$ can be represented using:

```
matA = matrix(c(1,1,2,-1), 2, 2, byrow=TRUE)
vecB = c(1,1)
```

First we solve for $\mathbf{A}^{-1}$:³

³Notice that the calculations in R do not show $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$ exactly. The (1,2) element of $\mathbf{A}^{-1}\mathbf{A}$ is -5.552e-17, which for all practical purposes is zero. However, due to the limitations of machine calculations the result is not exactly

```

matA.inv = solve(matA)
matA.inv
#>      [,1]      [,2]
#> [1,] 0.333 0.333
#> [2,] 0.667 -0.333
matA.inv%%matA
#>      [,1]      [,2]
#> [1,] 1 -5.55e-17
#> [2,] 0 1.00e+00
matA%%matA.inv
#>      [,1]      [,2]
#> [1,] 1 5.55e-17
#> [2,] 0 1.00e+00

```

Then we solve the system $\mathbf{z} = \mathbf{A}^{-1}\mathbf{b}$:

```

z = matA.inv%%vecB
z
#>      [,1]
#> [1,] 0.667
#> [2,] 0.333

```

■

In general, if we have n linear equations in n unknown variables we may write the system of equations as

$$\begin{aligned}
 a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\
 a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\
 &\vdots \\
 a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n
 \end{aligned}$$

which we may then express in matrix form as

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

or

$$\underset{(n \times n)}{\mathbf{A}} \cdot \underset{(n \times 1)}{\mathbf{x}} = \underset{(n \times 1)}{\mathbf{b}}.$$

zero.

The solution to the system of equations is given by:

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b},$$

where $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n$ and $\mathbf{I}_n$ is the $(n \times n)$ identity matrix. If the number of equations is greater than two, then we generally use numerical algorithms to find the elements in $\mathbf{A}^{-1}$.

3.4.1 Rank of a matrix

Definition 3.7 (Linear independence of vectors). Consider an $n \times m$ matrix $\mathbf{A}$ with m columns $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_m$ where each column is an $n \times 1$ vector and n rows where each row is an $1 \times m$ row vector:

$$\underset{(n \times m)}{\mathbf{A}} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1 & \mathbf{a}_2 & \dots & \mathbf{a}_m \end{bmatrix}.$$

Two vectors $\mathbf{a}_1$ and $\mathbf{a}_2$ are linearly independent if $c_1\mathbf{a}_1 + c_2\mathbf{a}_2 = 0$ implies that $c_1 = c_2 = 0$. A set of vectors $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_m$ are linearly independent if $c_1\mathbf{a}_1 + c_2\mathbf{a}_2 + \dots + c_m\mathbf{a}_m = 0$ implies that $c_1 = c_2 = \dots = c_m = 0$. That is, no vector $\mathbf{a}_i$ can be expressed as a non-trivial linear combination of the other vectors.

Definition 3.8 (Rank of a matrix). The *column rank* of the $n \times m$ matrix $\mathbf{A}$, denoted $\text{rank}(\mathbf{A})$, is equal to the maximum number of linearly independent columns. The row rank of a matrix is equal to the maximum number of linearly independent rows, and is given by $\text{rank}(\mathbf{A}')$.

It turns out that the column rank and row rank of a matrix are the same. Hence, $\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A}') \leq \min(m, n)$. If $\text{rank}(\mathbf{A}) = m$ then $\mathbf{A}$ is called *full column rank*, and if $\text{rank}(\mathbf{A}) = n$ then $\mathbf{A}$ is called *full row rank*. If $\mathbf{A}$ is not full rank then it is called *reduced rank*.

Example 3.12 (Determining the rank of a matrix in R). Consider the 2×3 matrix

$$\mathbf{A} = \begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{pmatrix}$$

Here, $\text{rank}(\mathbf{A}) \leq \min(3, 2) = 2$, the number of rows. Now, $\text{rank}(\mathbf{A}) = 2$ since the rows of $\mathbf{A}$ are linearly independent. In R, the rank of a matrix can be found using the **Matrix** function `rankMatrix()`:

```
library(Matrix)
Amat = matrix(c(1,3,5,2,4,6), 2, 3, byrow=TRUE)
as.numeric(rankMatrix(Amat))
```




The rank of an $n \times n$ square matrix $\mathbf{A}$ is directly related to its invertibility. If $\text{rank}(\mathbf{A}) = n$ then $\mathbf{A}^{-1}$ exists. This result makes sense. Since $\mathbf{A}^{-1}$ is used to solve a system of n linear equations in n unknowns, it will have a unique solution as long as no equation in the system can be written as a linear combination of the other equations.

3.4.2 Partitioned matrices and partitioned inverses

Consider a general $n \times m$ matrix $\mathbf{A}$:

$$\underset{(n \times m)}{\mathbf{A}} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix}.$$

In some situations we might want to partition $\mathbf{A}$ into sub-matrices containing sub-blocks of the elements of $\mathbf{A}$:

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix},$$

where the sub-matrices $\mathbf{A}_{11}$, $\mathbf{A}_{12}$, $\mathbf{A}_{21}$, and $\mathbf{A}_{22}$ contain the appropriate sub-elements of $\mathbf{A}$. For example, consider

$$\mathbf{A} = \left[\begin{array}{cc|cc} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ \hline 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{array} \right] = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix},$$

where

$$\begin{aligned} \mathbf{A}_{11} &= \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix}, \mathbf{A}_{12} = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix}, \\ \mathbf{A}_{21} &= \begin{bmatrix} 9 & 10 \\ 13 & 14 \end{bmatrix}, \mathbf{A}_{22} = \begin{bmatrix} 11 & 12 \\ 15 & 16 \end{bmatrix}. \end{aligned}$$

Example 3.13 (Combining sub-matrices in R). In R sub-matrices can be combined column-wise using `cbind()`, and row-wise using `rbind()`. For example, to create the matrix $\mathbf{A}$ from the sub-matrices use

```

A11mat = matrix(c(1,2,5,6), 2, 2, byrow=TRUE)
A12mat = matrix(c(3,4,7,8), 2, 2, byrow=TRUE)
A21mat = matrix(c(9,10,13,14), 2, 2, byrow=TRUE)
A22mat = matrix(c(11,12,15,16), 2, 2, byrow=TRUE)
Amat = rbind(cbind(A11mat, A12mat), cbind(A21mat, A22mat))
Amat
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    2    3    4
#> [2,]    5    6    7    8
#> [3,]    9   10   11   12
#> [4,]   13   14   15   16

```

■

The basic matrix operations work on sub-matrices in the obvious way. To illustrate, consider another matrix $\mathbf{B}$ that is conformable with $\mathbf{A}$ and partitioned in the same way:

$$\mathbf{B} = \begin{bmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{bmatrix}.$$

Then

$$\mathbf{A} + \mathbf{B} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} + \begin{bmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_{11} + \mathbf{B}_{11} & \mathbf{A}_{12} + \mathbf{B}_{12} \\ \mathbf{A}_{21} + \mathbf{B}_{21} & \mathbf{A}_{22} + \mathbf{B}_{22} \end{bmatrix}.$$

If all of the sub-matrices of $\mathbf{A}$ and $\mathbf{B}$ are conformable then

$$\mathbf{AB} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} \begin{bmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{12}\mathbf{B}_{21} & \mathbf{A}_{11}\mathbf{B}_{12} + \mathbf{A}_{12}\mathbf{B}_{22} \\ \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{21} & \mathbf{A}_{21}\mathbf{B}_{12} + \mathbf{A}_{22}\mathbf{B}_{22} \end{bmatrix}.$$

The transpose of a partitioned matrix satisfies

$$\mathbf{A}' = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix}' = \begin{bmatrix} \mathbf{A}'_{11} & \mathbf{A}'_{21} \\ \mathbf{A}'_{12} & \mathbf{A}'_{22} \end{bmatrix}.$$

Notice the interchange of the two off-diagonal blocks.

A partitioned matrix $\mathbf{A}$ with appropriate invertible sub-matrices $\mathbf{A}_{11}$ and $\mathbf{A}_{22}$ has a partitioned inverse of the form

$$\mathbf{A}^{-1} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix}^{-1} = \begin{bmatrix} \mathbf{A}_{11}^{-1} + \mathbf{A}_{11}^{-1}\mathbf{A}_{12}\mathbf{C}^{-1}\mathbf{A}_{21}\mathbf{A}_{11}^{-1} & -\mathbf{A}_{11}^{-1}\mathbf{A}_{12}\mathbf{C}^{-1} \\ -\mathbf{C}^{-1}\mathbf{A}_{21}\mathbf{A}_{11}^{-1} & \mathbf{C}^{-1} \end{bmatrix},$$

where $\mathbf{C} = \mathbf{A}_{22} - \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12}$. This formula can be verified by direct calculation.

3.5 Positive Definite Matrices

Definition 3.9 (Positive definite matrix). Let $\mathbf{A}$ be an $n \times n$ symmetrix matrix. The matrix $\mathbf{A}$ is *positive definite* if for any $n \times 1$ vector $\mathbf{x} \neq \mathbf{0}$

$$\mathbf{x}'\mathbf{A}\mathbf{x} > 0.$$

This condition implies that there is no non-zero vector $\mathbf{x}$ such that $\mathbf{A}\mathbf{x} = \mathbf{0}$, which implies that $\mathbf{A}$ has full rank n .

Definition 3.10 (Positive semi-definite matrix). The matrix $\mathbf{A}$ is *positive semi-definite* if for any $n \times 1$ vector $\mathbf{x} \neq \mathbf{0}$

$$\mathbf{x}'\mathbf{A}\mathbf{x} \geq 0.$$

Hence, if $\mathbf{A}$ is positive semi-definite then there exists some $n \times 1$ vector $\mathbf{x}$ such that $\mathbf{A}\mathbf{x} = \mathbf{0}$, which implies that $\mathbf{A}$ does not have full rank n .

3.5.1 Matrix square root

To motivate the idea of the square root of a matrix, consider the square root factorization of a positive number a :

$$a = \sqrt{a} \times \sqrt{a} = a^{1/2} \times a^{1/2}.$$

That is, we can factorize any positive number into the product of its square root. It turns out we can do a similar factorization for a positive definite and symmetric matrix.

Proposition 3.1 (Cholesky factorization). *Let $\mathbf{A}$ be an $n \times n$ positive definite and symmetric matrix. Then it is possible to factor $\mathbf{A}$ as*

$$\mathbf{A} = \mathbf{C}'\mathbf{C},$$

where $\mathbf{C}$ is an $n \times n$ upper triangular matrix with non-negative diagonal elements called the Cholesky factor of $\mathbf{A}$. This factorization is also called a matrix square root factorization by defining the square root matrix $\mathbf{A}^{1/2} = \mathbf{C}$. Then we can write $\mathbf{A} = \mathbf{A}^{1/2'}\mathbf{A}^{1/2}$.⁴ If all of the diagonal elements of $\mathbf{C}$ are positive then $\mathbf{A}$ is positive definite. Otherwise, $\mathbf{A}$ is positive semi-definite.

Example 3.14 (Cholesky decomposition in R). The R function `chol()` computes the Cholesky factorization of a square symmetrix matrix. For example, consider the 2×2 correlation matrix:

⁴The Cholesky factorization is one way of defining the matrix square root, where the square root matrix is upper triangular. Other matrix square factorizations exist where the square root matrix is not necessarily upper triangular.

```
Rho = matrix(c(1, 0.75, 0.75, 1), 2, 2)
Rho
#>      [,1] [,2]
#> [1,] 1.00 0.75
#> [2,] 0.75 1.00
```

The Cholesky matrix square root factorization is

```
C = chol(Rho)
C
#>      [,1] [,2]
#> [1,]    1 0.750
#> [2,]    0 0.661
t(C)%*%C
#>      [,1] [,2]
#> [1,] 1.00 0.75
#> [2,] 0.75 1.00
```

Here, all the diagonal elements of **C** are positive so that covariance matrix is positive definite. Next, suppose the correlation matrix is

```
Rho = matrix(c(1, 1, 1, 1), 2, 2)
Rho
#>      [,1] [,2]
#> [1,]    1    1
#> [2,]    1    1
```

so that the two variables are perfectly correlated. Then

```
chol(Rho, pivot=TRUE)
#>      [,1] [,2]
#> [1,]    1    1
#> [2,]    0    0
#> attr("pivot")
#> [1] 1 2
#> attr("rank")
#> [1] 1
```

Here, we see that one of the diagonal elements of the Cholesky factor is zero and that the correlation matrix is rank deficient.

3.6 Multivariate Probability Distributions Using Matrix Algebra

In this section, we show how matrix algebra can be used to simplify many of the messy expressions concerning expectations and covariances between multiple random variables, and we show how certain multivariate probability distributions (e.g., the multivariate normal distribution) can be expressed using matrix algebra.

3.6.1 Random vectors

Let $X_1, \dots, X_n$ denote n random variables. For $i = 1, \dots, n$ let $\mu_i = E[X_i]$ and $\sigma_i^2 = \text{var}(X_i)$, and let $\sigma_{ij} = \text{cov}(X_i, X_j)$ for $i \neq j$. Define the $n \times 1$ random vector $\mathbf{X} = (X_1, \dots, X_n)'$. Associated with $\mathbf{X}$ is the $n \times 1$ vector of expected values:

$$\underset{n \times 1}{\boldsymbol{\mu}} = E[\mathbf{X}] = \begin{pmatrix} E[X_1] \\ \vdots \\ E[X_n] \end{pmatrix} = \begin{pmatrix} \mu_1 \\ \vdots \\ \mu_n \end{pmatrix}. \quad (3.6)$$

3.6.2 Covariance matrix

The covariance matrix, Σ , summarizes the variances and covariances of the elements of the random vector $\mathbf{X}$. In general, the covariance matrix of a random vector $\mathbf{X}$ (sometimes called the variance of the vector $\mathbf{X}$) with mean vector $\boldsymbol{\mu}$ is defined as:

$$\text{cov}(\mathbf{X}) = E[(\mathbf{X} - \boldsymbol{\mu})(\mathbf{X} - \boldsymbol{\mu})'] = \Sigma. \quad (3.7)$$

If $\mathbf{X}$ has n elements then Σ will be the symmetric and positive semi-definite $n \times n$ matrix,

$$\underset{n \times n}{\Sigma} = \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1n} \\ \sigma_{12} & \sigma_2^2 & \cdots & \sigma_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{1n} & \sigma_{2n} & \cdots & \sigma_n^2 \end{pmatrix}. \quad (3.8)$$

For the case $n = 2$, we have by direct calculation

$$\begin{aligned}
E[(\mathbf{X} - \mu)(\mathbf{X} - \mu)'] &= E \left[\begin{pmatrix} X_1 - \mu_1 \\ X_2 - \mu_2 \end{pmatrix} \cdot (X_1 - \mu_1, X_2 - \mu_2) \right] \\
&= E \left[\begin{pmatrix} (X_1 - \mu_1)^2 & (X_1 - \mu_1)(X_2 - \mu_2) \\ (X_2 - \mu_2)(X_1 - \mu_1) & (X_2 - \mu_2)^2 \end{pmatrix} \right] \\
&= \begin{pmatrix} E[(X_1 - \mu_1)^2] & E[(X_1 - \mu_1)(X_2 - \mu_2)] \\ E[(X_2 - \mu_2)(X_1 - \mu_1)] & E[(X_2 - \mu_2)^2] \end{pmatrix} \\
&= \begin{pmatrix} \text{var}(X_1) & \text{cov}(X_1, X_2) \\ \text{cov}(X_2, X_1) & \text{var}(X_2) \end{pmatrix} = \begin{pmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{12} & \sigma_2^2 \end{pmatrix} = \Sigma.
\end{aligned}$$

If $\rho_{12} \neq \pm 1$ then $\det(\Sigma) \neq 0$, Σ has rank 2 and is positive definite. However, if $\rho_{12} = \pm 1$ then $\det(\Sigma) = 0$, Σ has rank 1 and is positive semi-definite.

3.6.3 Correlation matrix

The *correlation matrix*, $\mathbf{C}$, summarizes all pairwise correlations between the elements of the $n \times 1$ random vector $\mathbf{X}$ and is given by

$$\mathbf{C} = \begin{pmatrix} 1 & \rho_{12} & \cdots & \rho_{1n} \\ \rho_{12} & 1 & \cdots & \rho_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \rho_{1n} & \rho_{2n} & \cdots & 1 \end{pmatrix} \quad (3.9)$$

Let Σ denote the covariance matrix of $\mathbf{X}$. Then the correlation matrix $\mathbf{C}$ can be computed using

$$\mathbf{C} = \mathbf{D}^{-1} \Sigma \mathbf{D}^{-1}, \quad (3.10)$$

where $\mathbf{D}$ is an $n \times n$ diagonal matrix with the standard deviations of the elements of $\mathbf{X}$ along the main diagonal:

$$\mathbf{D} = \begin{pmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n \end{pmatrix}.$$

Example 3.15 (Computing correlation matrix from covariance matrix in R). In R the `cov2cor()` function can be used to compute the correlation matrix given the covariance matrix using Equation 3.10:

```

sigma.1 = 1
sigma.2 = 2
rho.12 = 0.5
sigma.12 = rho.12*sigma.1*sigma.2
Sigma = matrix(c(sigma.1^2, sigma.12, sigma.12, sigma.2^2),
               2, 2, byrow=TRUE)

Sigma
#>      [,1] [,2]
#> [1,]    1    1
#> [2,]    1    4
cov2cor(Sigma)
#>      [,1] [,2]
#> [1,]  1.0  0.5
#> [2,]  0.5  1.0

```

■

3.6.4 Variance of linear combination of random vectors

Consider the $n \times 1$ random vector $\mathbf{X}$ with mean vector μ and covariance matrix Σ . Let $\mathbf{a} = (a_1, \dots, a_n)'$ be an $n \times 1$ vector of constants and consider the random variable $Y = \mathbf{a}'\mathbf{X} = a_1X_1 + \dots + a_nX_n$. The expected value of Y is:

$$\mu_Y = E[Y] = E[\mathbf{a}'\mathbf{X}] = \mathbf{a}'E[\mathbf{X}] = \mathbf{a}'\mu. \quad (3.11)$$

The variance of Y is

$$\text{var}(Y) = \text{var}(\mathbf{a}'\mathbf{X}) = E[(\mathbf{a}'\mathbf{X} - \mathbf{a}'\mu)^2] = E[(\mathbf{a}'(\mathbf{X} - \mu))^2] \quad (3.12)$$

since $Y = \mathbf{a}'\mathbf{X}$ is a scalar. Now we use a trick from matrix algebra. If z is a scalar (think of $z = 2$) then $z'z = z \cdot z' = z^2$. Let $z = \mathbf{a}'(\mathbf{X} - \mu)$ and so $z \cdot z' = \mathbf{a}'(\mathbf{X} - \mu)(\mathbf{X} - \mu)'\mathbf{a}$. Then,

$$\begin{aligned}
\text{var}(Y) &= E[z^2] = E[z \cdot z'] \\
&= E[\mathbf{a}'(\mathbf{X} - \mu)(\mathbf{X} - \mu)'\mathbf{a}] \\
&= \mathbf{a}'E[(\mathbf{X} - \mu)(\mathbf{X} - \mu)']\mathbf{a} \\
&= \mathbf{a}'\text{cov}(\mathbf{X})\mathbf{a} = \mathbf{a}'\Sigma\mathbf{a}.
\end{aligned} \quad (3.13)$$

3.6.5 Covariance between linear combination of two random vectors

Consider the $n \times 1$ random vector $\mathbf{X}$ with mean vector μ and covariance matrix Σ . Let $\mathbf{a} = (a_1, \dots, a_n)'$ and $\mathbf{b} = (b_1, \dots, b_n)'$ be $n \times 1$ vectors of constants, and consider the random variable $Y = \mathbf{a}'\mathbf{X} = a_1X_1 + \dots + a_nX_n$ and $Z = \mathbf{b}'\mathbf{X} = b_1X_1 + \dots + b_nX_n$. From the definition of covariance we have:

$$\text{cov}(Y, Z) = E[(Y - E[Y])(Z - E[Z])]$$

which may be rewritten in matrix notation as,

$$\begin{aligned} \text{cov}(\mathbf{a}'\mathbf{X}, \mathbf{b}'\mathbf{X}) &= E[(\mathbf{a}'\mathbf{X} - \mathbf{a}'\mu)(\mathbf{b}'\mathbf{X} - \mathbf{b}'\mu)] \\ &= E[\mathbf{a}'(\mathbf{X} - \mu)\mathbf{b}'(\mathbf{X} - \mu)] \\ &= E[\mathbf{a}'(\mathbf{X} - \mu)(\mathbf{X} - \mu)'\mathbf{b}] \\ &= \mathbf{a}'E[(\mathbf{X} - \mu)(\mathbf{X} - \mu)']\mathbf{b} \\ &= \mathbf{a}'\Sigma\mathbf{b}. \end{aligned} \tag{3.14}$$

Since $\mathbf{a}'(\mathbf{X} - \mu)$ and $\mathbf{b}'(\mathbf{X} - \mu)$ are scalars, we can use the trick:

$$\mathbf{a}'(\mathbf{X} - \mu)\mathbf{b}'(\mathbf{X} - \mu) = \mathbf{a}'(\mathbf{X} - \mu)(\mathbf{X} - \mu)'\mathbf{b}.$$

3.6.6 Variance of a vector of linear functions of a random vector

Consider the $n \times 1$ random vector $\mathbf{X}$ with mean vector μ and covariance matrix Σ . Let $\mathbf{A}$ be an $m \times n$ matrix. Then the $m \times 1$ random vector $\mathbf{Y} = \mathbf{A}\mathbf{X}$ represents m linear functions of $\mathbf{X}$. By the linearity of expectation, we have $E[\mathbf{Y}] = \mathbf{A}E[\mathbf{X}] = \mathbf{A}\mu$. The $m \times m$ covariance matrix of $\mathbf{Y}$ is given by

$$\begin{aligned} \text{var}(\mathbf{Y}) &= E[(\mathbf{Y} - E[\mathbf{Y}])(\mathbf{Y} - E[\mathbf{Y}])'] \\ &= E[(\mathbf{A}\mathbf{X} - \mathbf{A}\mu)(\mathbf{A}\mathbf{X} - \mathbf{A}\mu)'] \\ &= E[\mathbf{A}(\mathbf{X} - \mu)(\mathbf{A}(\mathbf{X} - \mu))'] \\ &= E[\mathbf{A}(\mathbf{X} - \mu)(\mathbf{X} - \mu)'\mathbf{A}'] \\ &= \mathbf{A}E[(\mathbf{X} - \mu)(\mathbf{X} - \mu)']\mathbf{A}' \\ &= \mathbf{A}\Sigma\mathbf{A}'. \end{aligned} \tag{3.15}$$

3.6.7 Multivariate normal distribution

Consider n random variables $X_1, \dots, X_n$ and assume they are jointly normally distributed. Define the $n \times 1$ vectors $\mathbf{X} = (X_1, \dots, X_n)'$, $\mathbf{x} = (x_1, \dots, x_n)'$ and $\mu = (\mu_1, \dots, \mu_n)'$, and the $n \times n$ covariance

matrix:

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1n} \\ \sigma_{12} & \sigma_2^2 & \cdots & \sigma_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{1n} & \sigma_{2n} & \cdots & \sigma_n^2 \end{pmatrix}.$$

Then $\mathbf{X} \sim N(\mu, \Sigma)$ means that the random vector $\mathbf{X}$ has a *multivariate normal distribution* with mean vector μ and covariance matrix Σ . The pdf of the multivariate normal distribution can be compactly expressed as:

$$\begin{aligned} f(\mathbf{x}) &= \frac{1}{2\pi^{n/2} \det(\Sigma)^{1/2}} e^{-\frac{1}{2}(\mathbf{x}-\mu)'\Sigma^{-1}(\mathbf{x}-\mu)} \\ &= (2\pi)^{-n/2} \det(\Sigma)^{-1/2} e^{-\frac{1}{2}(\mathbf{x}-\mu)'\Sigma^{-1}(\mathbf{x}-\mu)}. \end{aligned}$$

Example 3.16 (Creating a multivariate normal random vector from a vector of independent standard normal random variables). Let $\mathbf{Z} = (Z_1, \dots, Z_n)'$ denote a vector of n iid standard normal random variables. Then $\mathbf{Z} \sim N(\mathbf{0}, \mathbf{I}_n)$ where $\mathbf{I}_n$ denotes the n -dimensional identity matrix. Given $\mathbf{Z}$, we would like to create a random vector $\mathbf{X}$ such that $\mathbf{X} \sim N(\mu, \Sigma)$. Let $\Sigma^{1/2}$ denote the upper triangular Cholesky factor of Σ such that $\Sigma = \Sigma^{1/2'}\Sigma^{1/2}$. Define $\mathbf{X}$ using

$$\mathbf{X} = \mu + \Sigma^{1/2'}\mathbf{Z}.$$

Then

$$\begin{aligned} E[\mathbf{X}] &= \mu + \Sigma^{1/2'}E[\mathbf{Z}] = \mu, \\ \text{var}(\mathbf{X}) &= \text{var}(\Sigma^{1/2'}\mathbf{Z}) = \Sigma^{1/2'}\text{var}(\mathbf{Z})\Sigma^{1/2} = \Sigma^{1/2'}\mathbf{I}_n\Sigma^{1/2} = \Sigma. \end{aligned}$$

Hence, $\mathbf{X} \sim N(\mu, \Sigma)$. ■

Example 3.17 (Simulating multivariate normal random vectors in R). Here, we illustrate how to simulate $\mathbf{X} \sim N(\mu, \Sigma)$ where $\mu = (1, 1)'$ and

$$\Sigma = \begin{pmatrix} 1 & 1 \\ 1 & 4 \end{pmatrix}$$

First, specify the mean vector and covariance matrix for the the bivariate normal distribution:

```
mu.vec = c(1,1)
sigma.1 = 1
sigma.2 = 2
rho.12 = 0.5
```

```

sigma.12 = rho.12*sigma.1*sigma.2
Sigma = matrix(c(sigma.1^2, sigma.12, sigma.12, sigma.2^2),
               2, 2, byrow=TRUE)

mu.vec
#> [1] 1 1
Sigma
#>      [,1] [,2]
#> [1,]    1    1
#> [2,]    1    4

```

Second, compute the Cholesky factorization of Σ :

```

Sigma.5 = chol(Sigma)
Sigma.5
#>      [,1] [,2]
#> [1,]    1 1.00
#> [2,]    0 1.73

```

Third, simulate $\mathbf{Z} \sim N(\mathbf{0}, \mathbf{I}_2)$

```

n = 2
set.seed(123)
Z = rnorm(n)

```

Fourth, compute $\mathbf{X} = \mu + \Sigma^{1/2'} \mathbf{Z}$

```

X = mu.vec + t(Sigma.5)%*%Z
X
#>      [,1]
#> [1,] 0.4395
#> [2,] 0.0408

```

■

3.7 Portfolio Math Using Matrix Algebra

Let R_i denote the return on asset $i = A, B, C$ and assume that R_A, R_B and R_C are jointly normally distributed with means, variances and covariances:

$$\mu_i = E[R_i], \sigma_i^2 = \text{var}(R_i), \text{cov}(R_i, R_j) = \sigma_{ij}.$$

Let x_i denote the share of wealth invested in asset i ($i = A, B, C$), and assume that all wealth is invested in the three assets so that $x_A + x_B + x_C = 1$. The portfolio return, $R_{p,x}$, is the random variable:

$$R_{p,x} = x_A R_A + x_B R_B + x_C R_C. \quad (3.16)$$

The subscript “ x ” indicates that the portfolio is constructed using the x -weights x_A, x_B and x_C . The expected return on the portfolio is:

$$\mu_{p,x} = E[R_{p,x}] = x_A \mu_A + x_B \mu_B + x_C \mu_C, \quad (3.17)$$

and the variance of the portfolio return is:

$$\begin{aligned} \sigma_{p,x}^2 = \text{var}(R_{p,x}) &= x_A^2 \sigma_A^2 + x_B^2 \sigma_B^2 + x_C^2 \sigma_C^2 + 2x_A x_B \sigma_{AB} \\ &+ 2x_A x_C \sigma_{AC} + 2x_B x_C \sigma_{BC}. \end{aligned}$$

Notice that variance of the portfolio return depends on three variance terms and six covariance terms. Hence, with three assets there are twice as many covariance terms than variance terms contributing to portfolio variance. Even with three assets, the algebra representing the portfolio characteristics Equation 3.16 - Equation 3.18 is cumbersome. We can greatly simplify the portfolio algebra using matrix notation.

Define the following 3×1 column vectors containing the asset returns and portfolio weights:

$$\mathbf{R} = \begin{pmatrix} R_A \\ R_B \\ R_C \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_A \\ x_B \\ x_C \end{pmatrix}.$$

The probability distribution of the random return vector $\mathbf{R}$ is simply the joint distribution of the elements of $\mathbf{R}$. Here all returns are jointly normally distributed and this joint distribution is completely characterized by the means, variances and covariances of the returns. We can easily express these values using matrix notation as follows. The 3×1 vector of portfolio expected values is:

$$E[\mathbf{R}] = E \left[\begin{pmatrix} R_A \\ R_B \\ R_C \end{pmatrix} \right] = \begin{pmatrix} E[R_A] \\ E[R_B] \\ E[R_C] \end{pmatrix} = \begin{pmatrix} \mu_A \\ \mu_B \\ \mu_C \end{pmatrix} = \boldsymbol{\mu},$$

and the 3×3 covariance matrix of returns is,

$$\begin{aligned}\text{var}(\mathbf{R}) &= \begin{pmatrix} \text{var}(R_A) & \text{cov}(R_A, R_B) & \text{cov}(R_A, R_C) \\ \text{cov}(R_B, R_A) & \text{var}(R_B) & \text{cov}(R_B, R_C) \\ \text{cov}(R_C, R_A) & \text{cov}(R_C, R_B) & \text{var}(R_C) \end{pmatrix} \\ &= \begin{pmatrix} \sigma_A^2 & \sigma_{AB} & \sigma_{AC} \\ \sigma_{AB} & \sigma_B^2 & \sigma_{BC} \\ \sigma_{AC} & \sigma_{BC} & \sigma_C^2 \end{pmatrix} = \Sigma.\end{aligned}$$

Notice that the covariance matrix is symmetric (elements off the diagonal are equal so that $\Sigma = \Sigma'$, where Σ' denotes the transpose of Σ) since $\text{cov}(R_A, R_B) = \text{cov}(R_B, R_A)$, $\text{cov}(R_A, R_C) = \text{cov}(R_C, R_A)$ and $\text{cov}(R_B, R_C) = \text{cov}(R_C, R_B)$.

The return on the portfolio using vector notation is:

$$R_{p,x} = \mathbf{x}'\mathbf{R} = (x_A, x_B, x_C) \cdot \begin{pmatrix} R_A \\ R_B \\ R_C \end{pmatrix} = x_A R_A + x_B R_B + x_C R_C.$$

Similarly, the expected return on the portfolio is:

$$\begin{aligned}\mu_{p,x} &= E[\mathbf{x}'\mathbf{R}] = \mathbf{x}'E[\mathbf{R}] = \mathbf{x}'\boldsymbol{\mu} \\ &= (x_A, x_B, x_C) \cdot \begin{pmatrix} \mu_A \\ \mu_B \\ \mu_C \end{pmatrix} = x_A \mu_A + x_B \mu_B + x_C \mu_C.\end{aligned}$$

The variance of the portfolio is:

$$\begin{aligned}\sigma_{p,x}^2 &= \text{var}(\mathbf{x}'\mathbf{R}) = \mathbf{x}'\Sigma\mathbf{x} = (x_A, x_B, x_C) \cdot \begin{pmatrix} \sigma_A^2 & \sigma_{AB} & \sigma_{AC} \\ \sigma_{AB} & \sigma_B^2 & \sigma_{BC} \\ \sigma_{AC} & \sigma_{BC} & \sigma_C^2 \end{pmatrix} \begin{pmatrix} x_A \\ x_B \\ x_C \end{pmatrix} \\ &= x_A^2 \sigma_A^2 + x_B^2 \sigma_B^2 + x_C^2 \sigma_C^2 + 2x_A x_B \sigma_{AB} + 2x_A x_C \sigma_{AC} + 2x_B x_C \sigma_{BC}.\end{aligned}$$

Finally, the condition that the portfolio weights sum to one can be expressed as:

$$\mathbf{x}'\mathbf{1} = (x_A, x_B, x_C) \cdot \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = x_A + x_B + x_C = 1,$$

where $\mathbf{1}$ is a 3×1 vector with each element equal to 1.

Consider another portfolio with weights $\mathbf{y} = (y_A, y_B, y_C)'$. The return on this portfolio is:

$$R_{p,y} = \mathbf{y}'\mathbf{R} = y_A R_A + y_B R_B + y_C R_C.$$

We often need to compute the covariance between the return on portfolio $\mathbf{x}$ and the return on portfolio $\mathbf{y}$, $\text{cov}(R_{p,x}, R_{p,y})$. This can be easily expressed using matrix algebra:

$$\begin{aligned}\sigma_{xy} &= \text{cov}(R_{p,x}, R_{p,y}) = \text{cov}(\mathbf{x}'\mathbf{R}, \mathbf{y}'\mathbf{R}) \\ &= E[(\mathbf{x}'\mathbf{R} - \mathbf{x}'\mu)(\mathbf{y}'\mathbf{R} - \mathbf{y}'\mu)] = E[\mathbf{x}'(\mathbf{R} - \mu)(\mathbf{R} - \mu)'\mathbf{y}] \\ &= \mathbf{x}'E[(\mathbf{R} - \mu)(\mathbf{R} - \mu)']\mathbf{y} = \mathbf{x}'\Sigma\mathbf{y}.\end{aligned}$$

Notice that,

$$\begin{aligned}\sigma_{xy} &= \mathbf{x}'\Sigma\mathbf{y} = (x_A, x_B, x_C) \cdot \begin{pmatrix} \sigma_A^2 & \sigma_{AB} & \sigma_{AC} \\ \sigma_{AB} & \sigma_B^2 & \sigma_{BC} \\ \sigma_{AC} & \sigma_{BC} & \sigma_C^2 \end{pmatrix} \begin{pmatrix} y_A \\ y_B \\ y_C \end{pmatrix} \\ &= x_A y_A \sigma_A^2 + x_B y_B \sigma_B^2 + x_C y_C \sigma_C^2 \\ &\quad + (x_A y_B + x_B y_A) \sigma_{AB} + (x_A y_C + x_C y_A) \sigma_{AC} + (x_B y_C + x_C y_B) \sigma_{BC},\end{aligned}$$

which is quite a messy expression!

3.8 Derivatives of Simple Matrix Functions

Result: Let $\mathbf{A}$ be an $n \times n$ symmetric matrix, and let $\mathbf{x}$ and $\mathbf{y}$ be $n \times 1$ vectors. Then,

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{x}'\mathbf{y} = \begin{pmatrix} \frac{\partial}{\partial x_1} \mathbf{x}'\mathbf{y} \\ \vdots \\ \frac{\partial}{\partial x_n} \mathbf{x}'\mathbf{y} \end{pmatrix} = \mathbf{y}, \quad (3.18)$$

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{A}\mathbf{x} = \begin{pmatrix} \frac{\partial}{\partial x_1} (\mathbf{A}\mathbf{x})' \\ \vdots \\ \frac{\partial}{\partial x_n} (\mathbf{A}\mathbf{x})' \end{pmatrix} = \mathbf{A}, \quad (3.19)$$

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{x}'\mathbf{A}\mathbf{x} = \begin{pmatrix} \frac{\partial}{\partial x_1} \mathbf{x}'\mathbf{A}\mathbf{x} \\ \vdots \\ \frac{\partial}{\partial x_n} \mathbf{x}'\mathbf{A}\mathbf{x} \end{pmatrix} = 2\mathbf{A}\mathbf{x}. \quad (3.20)$$

We will demonstrate these results with simple examples. Let,

$$\mathbf{A} = \begin{pmatrix} a & b \\ b & c \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}.$$

First, consider Equation 3.18. Now,

$$\mathbf{x}'\mathbf{y} = x_1y_1 + x_2y_2.$$

Then,

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{x}'\mathbf{y} = \begin{pmatrix} \frac{\partial}{\partial x_1} \mathbf{x}'\mathbf{y} \\ \frac{\partial}{\partial x_2} \mathbf{x}'\mathbf{y} \end{pmatrix} = \begin{pmatrix} \frac{\partial}{\partial x_1} (x_1y_1 + x_2y_2) \\ \frac{\partial}{\partial x_2} (x_1y_1 + x_2y_2) \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \mathbf{y}.$$

Next, consider Equation 3.19. Note that,

$$\mathbf{A}\mathbf{x} = \begin{pmatrix} a & b \\ b & c \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} ax_1 + bx_2 \\ bx_1 + cx_2 \end{pmatrix},$$

and,

$$(\mathbf{A}\mathbf{x})' = (ax_1 + bx_2, bx_1 + cx_2).$$

Then,

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{A}\mathbf{x} = \begin{pmatrix} \frac{\partial}{\partial x_1} (ax_1 + bx_2, bx_1 + cx_2) \\ \frac{\partial}{\partial x_2} (ax_1 + bx_2, bx_1 + cx_2) \end{pmatrix} = \begin{pmatrix} a & b \\ b & c \end{pmatrix} = \mathbf{A}.$$

Finally, consider Equation 3.20. We have,

$$\mathbf{x}'\mathbf{A}\mathbf{x} = \begin{pmatrix} x_1 & x_2 \end{pmatrix} \begin{pmatrix} a & b \\ b & c \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = ax_1^2 + 2bx_1x_2 + cx_2^2.$$

Then,

$$\begin{aligned} \frac{\partial}{\partial \mathbf{x}} \mathbf{x}'\mathbf{A}\mathbf{x} &= \begin{pmatrix} \frac{\partial}{\partial x_1} (ax_1^2 + 2bx_1x_2 + cx_2^2) \\ \frac{\partial}{\partial x_2} (ax_1^2 + 2bx_1x_2 + cx_2^2) \end{pmatrix} = \begin{pmatrix} 2ax_1 + 2bx_2 \\ 2bx_1 + 2cx_2 \end{pmatrix} \\ &= 2 \begin{pmatrix} a & b \\ b & c \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 2\mathbf{A}\mathbf{x}. \end{aligned}$$

Example 3.18 (Calculating an asset's marginal contribution to portfolio volatility). In portfolio risk budgeting (see chapter ?@sec-Risk-Budgeting), asset i 's marginal contribution to portfolio volatility $\sigma_p = (\mathbf{x}'\Sigma\mathbf{x})^{1/2}$ is given by

$$\text{MCR}_i = \frac{\partial \sigma_p}{\partial x_i} = \frac{\partial (\mathbf{x}'\Sigma\mathbf{x})^{1/2}}{\partial x_i},$$

and approximates how much portfolio volatility changes when the allocation to asset i increases by a small amount. Using the chain rule and matrix derivatives we can compute the entire vector of asset marginal contributions at once:

$$\begin{aligned}\frac{\partial (\mathbf{x}'\Sigma\mathbf{x})^{1/2}}{\partial \mathbf{x}} &= \frac{1}{2} (\mathbf{x}'\Sigma\mathbf{x})^{-1/2} \frac{\partial \mathbf{x}'\Sigma\mathbf{x}}{\partial \mathbf{x}} = \frac{1}{2} (\mathbf{x}'\Sigma\mathbf{x})^{-1/2} 2\Sigma\mathbf{x} \\ &= (\mathbf{x}'\Sigma\mathbf{x})^{-1/2} \Sigma\mathbf{x} = \frac{\Sigma\mathbf{x}}{\sigma_p}.\end{aligned}$$

Then asset i 's marginal contribution is given by the i -th row of $\frac{\Sigma\mathbf{x}}{\sigma_p}$. ■

Example 3.19 (Finding the global minimum variance portfolio). Let $\mathbf{R}$ denote an $n \times 1$ random vector of asset returns with $E[\mathbf{R}] = \mu$ and $\text{var}(\mathbf{R}) = \Sigma$. The global minimum variance portfolio (see Chapter ?@sec-Introduction-to-Portfolio-Theory, Section ?@sec-Efficient-portfolios-with-two-risky-assets) $\mathbf{m}$ solves the constrained minimization problem:

$$\min_{\mathbf{m}} \sigma_{p,m}^2 = \mathbf{m}'\Sigma\mathbf{m} \text{ s.t. } \mathbf{m}'\mathbf{1} = 1. \quad (3.21)$$

The Lagrangian function is:

$$L(\mathbf{m}, \lambda) = \mathbf{m}'\Sigma\mathbf{m} + \lambda(\mathbf{m}'\mathbf{1} - 1).$$

The first order conditions can be expressed in matrix notation as,

$$\underset{(n \times 1)}{\mathbf{0}} = \frac{\partial L(\mathbf{m}, \lambda)}{\partial \mathbf{m}} = \frac{\partial}{\partial \mathbf{m}} \mathbf{m}'\Sigma\mathbf{m} + \frac{\partial}{\partial \mathbf{m}} \lambda(\mathbf{m}'\mathbf{1} - 1) = 2 \cdot \Sigma\mathbf{m} + \lambda \cdot \mathbf{1} \quad (3.22)$$

$$\underset{(1 \times 1)}{0} = \frac{\partial L(\mathbf{m}, \lambda)}{\partial \lambda} = \frac{\partial}{\partial \lambda} \mathbf{m}'\Sigma\mathbf{m} + \frac{\partial}{\partial \lambda} \lambda(\mathbf{m}'\mathbf{1} - 1) = \mathbf{m}'\mathbf{1} - 1 \quad (3.23)$$

These first order conditions represent a system of $n + 1$ linear equations in $n + 1$ unknowns ($\mathbf{m}$ and λ). These equations can be represented in matrix form as the system

$$\begin{bmatrix} 2\Sigma & \mathbf{1} \\ \mathbf{1}' & 0 \end{bmatrix} \begin{bmatrix} \mathbf{m} \\ \lambda \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ 1 \end{bmatrix},$$

which is of the form $\mathbf{A}\mathbf{z} = \mathbf{b}$ for

$$\mathbf{A} = \begin{bmatrix} 2\Sigma & \mathbf{1} \\ \mathbf{1}' & 0 \end{bmatrix}, \mathbf{z} = \begin{bmatrix} \mathbf{m} \\ \lambda \end{bmatrix}, \mathbf{b} = \begin{bmatrix} \mathbf{0} \\ 1 \end{bmatrix}.$$

The portfolio weight vector $\mathbf{m}$ can be found as the first n elements of $\mathbf{z} = \mathbf{A}^{-1}\mathbf{b}$. ■

3.9 Further Reading: Matrix Algebra Review

A classic textbook on linear algebra is (Strang 1980). A review of basic matrix algebra with applications in economics and finance is given in (Chiang 1984). Matrix algebra results that are useful for statistical analysis are presented in (Searle 1982), Gentile (2010) and (Banerjee and Roy 2014). (Lutkepohl 1997) and (Abadir and Magnus 2005) present advanced matrix algebra useful for econometrics and statistics. (Fieller 2015) overviews the basics of matrix algebra for statistics using R.

3.10 Problems: Matrix Algebra Review

Exercise 3.1. Use R to answer the following questions. 1. Create the following matrices and vectors

$$\mathbf{A} = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 4 & 8 \\ 6 & 1 & 3 \end{bmatrix}, \mathbf{B} = \begin{bmatrix} 4 & 4 & 0 \\ 5 & 9 & 1 \\ 2 & 2 & 5 \end{bmatrix}, \mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 5 \\ 2 \\ 7 \end{bmatrix}$$

2. Compute the transposes of the above matrices and vectors. 3. Compute $\mathbf{A} + \mathbf{B}$, $\mathbf{A} - \mathbf{B}$, $2\mathbf{A}$, $\mathbf{Ax}$, $\mathbf{AB}$, $\mathbf{y'Ax}$

Exercise 3.2. Consider the system of linear equations

$$\begin{aligned} x + y &= 1 \\ 2x + 4y &= 2 \end{aligned}$$

1. Plot the two lines and note the solution to the system of equations. Hint: you can use the R functions `abline()` or `curve()` to plot the lines.
2. Write the system in matrix notation as $\mathbf{Az} = \mathbf{b}$ and solve for $\mathbf{z}$ by computing $\mathbf{z} = \mathbf{A}^{-1}\mathbf{b}$.

Exercise 3.3. Consider creating an equally weighted portfolio of three assets denoted A, B, and C. Assume the following information

$$\mu = \begin{bmatrix} 0.01 \\ 0.04 \\ 0.02 \end{bmatrix}, \Sigma = \begin{bmatrix} 0.10 & 0.30 & 0.10 \\ 0.30 & 0.15 & -0.20 \\ 0.10 & -0.20 & 0.08 \end{bmatrix}.$$

Use R to answer the following questions. 1. Create the vector of portfolio weights. 2. Compute the expected return on the portfolio. 3. Compute the variance and standard deviation of the portfolio.

Exercise 3.4. Let R_i be a random variable denoting the simple return on asset i ($i = 1, \dots, N$) with $E[R_i] = \mu_i$, $\text{var}(R_i) = \sigma_i^2$ and $\text{cov}(R_i, R_j) = \sigma_{ij}$. Define the $N \times 1$ vectors $\mathbf{R} = (R_1, \dots, R_N)'$,

$\mu = (\mu_1, \dots, \mu_N)'$, $\mathbf{x} = (x_1, \dots, x_N)'$, $\mathbf{y} = (y_1, \dots, y_N)'$, and $\mathbf{1} = (1, \dots, 1)'$, and the $N \times N$ covariance matrix

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1N} \\ \sigma_{12} & \sigma_2^2 & \cdots & \sigma_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{1N} & \sigma_{2N} & \cdots & \sigma_N^2 \end{pmatrix}.$$

The vectors $\mathbf{x}$ and $\mathbf{y}$ contain portfolio weights (investment shares) that sum to one. Using simple matrix algebra, answer the following questions.

1. For the portfolios defined by the vectors $\mathbf{x}$ and $\mathbf{y}$ give the matrix algebra expressions for the portfolio returns $R_{p,x}$ and $R_{p,y}$ and the portfolio expected returns $\mu_{p,x}$ and $\mu_{p,y}$.
2. For the portfolios defined by the vectors $\mathbf{x}$ and $\mathbf{y}$ give the matrix algebra expressions for the constraint that the portfolio weights sum to one.
3. For the portfolios defined by the vectors $\mathbf{x}$ and $\mathbf{y}$ give the matrix algebra expressions for the portfolio variances $\sigma_{p,x}^2$ and $\sigma_{p,y}^2$, and the covariance between $R_{p,x}$ and $R_{p,y}$.
4. In the expression for the portfolio variance $\sigma_{p,x}^2$, how many variance terms are there? How many unique covariance terms are there?

4 Time Series Concepts

Last updated on September 4, 2026

Copyright © Eric Zivot 2026

Financial variables such as asset prices and returns are naturally ordered by time. That is, these variables are *time series* variables. When we construct returns, the time index or data frequency becomes the investment horizon associated with the return. Typical data frequencies are daily, weekly, monthly and annual. In building probability models for time series variables, the time ordering of the data matters because we may think there are important temporal dependencies among the variables. For example, we might have reason to believe that the return on an asset this month is correlated with the return on the same asset in the previous month. This autocorrelation can only be defined if the time ordering of the data is preserved. A major complication in analyzing time series data is that the usual assumption of random sampling from a common population is not appropriate because it does not allow for any kind of time dependence in the time series variables. We would like to retain the notion that the observed data come from some population model, perhaps with time-invariant parameters, but we would like to allow the variables to have time dependencies. Fortunately, we can do this if the time series data come from a *stationary* time series process.

This chapter reviews some basic times series concepts that are essential for describing and modeling financial time series. Section Section 4.1 defines univariate time series processes and introduces the important concepts of stationarity and ergodicity. Covariance stationary time series processes are defined, which gives meaning to measuring linear time dependence using autocorrelation. The benchmark Gaussian White Noise process and related processes are introduced and illustrated using R. Some common non-stationary time series processes are also discussed including the famous random walk model. Section Section 4.2 introduces covariance stationary multivariate time series process. Such processes allow for dynamic interactions among groups of time series variables. Section Section 4.3 discusses time series model building and introduces the class of univariate autoregressive-moving average time series models and multivariate vector autoregression models. The properties of some simple models are derived and it is shown how to simulate observations from these models using R. The chapter concludes with a brief discussion of forecasting from time series models.

The R packages used in this chapter are **mvtnorm** and **vars**. Make sure these packages are installed and loaded before running the R examples in this chapter.

4.1 Stochastic Processes

Definition 4.1 (Stochastic Process). A discrete-time *stochastic process* or *time series process*

$$\{\dots, Y_1, Y_2, \dots, Y_t, Y_{t+1}, \dots\} = \{Y_t\}_{t=-\infty}^{\infty}$$

is a sequence of random variables indexed by time t ¹.

In most applications, the time index is a regularly spaced index representing calendar time (e.g., days, months, years, etc.) but it can also be irregularly spaced representing event time (e.g., intra-day transaction times). In modeling time series data, the ordering imposed by the time index is important because we often would like to capture the temporal relationships, if any, between the random variables in the stochastic process. In random sampling from a population, the ordering of the random variables representing the sample does not matter because they are independent.

Definition 4.2 (Realization of Stochastic Process). A realization of a stochastic process with T observations is the sequence of observed data

$$\{Y_1 = y_1, Y_2 = y_2, \dots, Y_T = y_T\} = \{y_t\}_{t=1}^T.$$

The goal of time series modeling is to describe the probabilistic behavior of the underlying stochastic process that is believed to have generated the observed data in a concise way. In addition, we want to be able to use the observed sample to estimate important characteristics of a time series model such as measures of time dependence. In order to do this, we need to make a number of assumptions regarding the joint behavior of the random variables in the stochastic process such that we may treat the stochastic process in much the same way as we treat a random sample from a given population.

4.1.1 Stationary stochastic processes

We often describe random sampling from a population as a sequence of independent, and identically distributed (iid) random variables $X_1, X_2, \dots$ such that each X_i is described by the same probability distribution F_X , and write $X_i \sim F_X$. With a time series process, we would like to preserve the identical distribution assumption but we do not want to impose the restriction that each random variable in the sequence is independent of all of the other variables. In many contexts, we would expect some dependence between random variables close together in time (e.g., X_1 , and X_2) but little or no dependence between random variables far apart in time (e.g., X_1 and X_{100}). We can allow for this type of behavior using the concepts of *stationarity* and *ergodicity*.

We start with the definition of strict stationarity.

¹To conserve on notation, we will represent the stochastic process $\{Y_t\}_{t=-\infty}^{\infty}$ simply as $\{Y_t\}$.

Definition 4.3 (Strict stationarity). A stochastic process $\{Y_t\}$ is *strictly stationary* if, for any given finite integer r and for any set of subscripts $t_1, t_2, \dots, t_r$ the joint distribution of $(Y_{t_1}, Y_{t_2}, \dots, Y_{t_r})$ depends only on $t_1 - t, t_2 - t, \dots, t_r - t$ but not on t . In other words, the joint distribution of $(Y_{t_1}, Y_{t_2}, \dots, Y_{t_r})$ is the same as the joint distribution of $(Y_{t_1-t}, Y_{t_2-t}, \dots, Y_{t_r-t})$ for any value of t .

In simple terms, the joint distribution of random variables in a strictly stationary stochastic process is *time invariant*. For example, the joint distribution of (Y_1, Y_5, Y_7) is the same as the distribution of (Y_{12}, Y_{16}, Y_{18}) . Just like in an iid sample, in a strictly stationary process all of the individual random variables Y_t ($t = -\infty, \dots, \infty$) have the same marginal distribution F_Y . This means they all have the same mean, variance etc., assuming these quantities exist. However, assuming strict stationarity does not make any assumption about the correlations between $Y_t, Y_{t_1}, \dots, Y_{t_r}$ other than that the correlation between Y_t and Y_{t_r} only depends on $t - t_r$ (the time between Y_t and Y_{t_r}) and not on t . That is, strict stationarity allows for general temporal dependence between the random variables in the stochastic process.

A useful property of strict stationarity is that it is preserved under general transformations, as summarized in the following proposition.

Proposition 4.1. Let $\{Y_t\}$ be strictly stationary and let $g(\cdot)$ be any function of the elements in $\{Y_t\}$. Then $\{g(Y_t)\}$, is also strictly stationary.

For example, if $\{Y_t\}$ is strictly stationary then $\{Y_t^2\}$ and $\{Y_t Y_{t-1}\}$ are also strictly stationary.

The following are some simple examples of strictly stationary processes.

Example 4.1 (iid sequence). If $\{Y_t\}$ is an iid sequence, then it is strictly stationary. ■

Example 4.2 (Non iid sequence). Let $\{Y_t\}$ be an iid sequence and let $X \sim N(0, 1)$ independent of $\{Y_t\}$. Define $Z_t = Y_t + X$. The sequence $\{Z_t\}$ is not an independent sequence (because of the common X) but is an identically distributed sequence and is strictly stationary. ■

Strict stationarity places very strong restrictions on the behavior of a time series. A related concept that imposes weaker restrictions and is convenient for time series model building is *covariance stationarity* (sometimes called *weak stationarity*).

Definition 4.4 (Covariance stationarity). A stochastic process $\{Y_t\}$ is *covariance stationary* if

1. $E[Y_t] = \mu < \infty$ does not depend on t
2. $\text{var}(Y_t) = \sigma^2 < \infty$ does not depend on t
3. $\text{cov}(Y_t, Y_{t-j}) = \gamma_j < \infty$, and depends only on j but not on t for $j = 0, 1, 2, \dots$

The term γ_j is called the j^{th} order *autocovariance*. The j^{th} order *autocorrelation* is defined as:

$$\rho_j = \frac{\text{cov}(Y_t, Y_{t-j})}{\sqrt{\text{var}(Y_t)\text{var}(Y_{t-j})}} = \frac{\gamma_j}{\sigma^2}. \quad (4.1)$$

The autocovariances, γ_j , measure the direction of linear dependence between Y_t and Y_{t-j} . The autocorrelations, ρ_j , measure both the direction and strength of linear dependence between Y_t and Y_{t-j} . With covariance stationarity, instead of assuming the entire joint distribution of a collection of random variables is time invariant we make a weaker assumption that only the mean, variance and autocovariances of the random variables are time invariant. A strictly stationary stochastic process $\{Y_t\}$ such that μ , σ^2 , and γ_{ij} exist is a *covariance stationary* stochastic process. However, a covariance stationary process need not be strictly stationary.

The autocovariances and autocorrelations are measures of the linear temporal dependence in a covariance stationary stochastic process. A graphical summary of this temporal dependence is given by the plot of ρ_j against j , and is called the *autocorrelation function* (ACF). Figure Figure 4.1 illustrates an ACF for a hypothetical covariance stationary time series with $\rho_j = (0.9)^j$ for $j = 1, 2, \dots, 10$ created with

```
rho = 0.9
rhoVec = (rho)^(1:10)
ts.plot(rhoVec, type="h", lwd=2, col="blue", xlab="Lag j",
        ylab=expression(rho[j]))
```

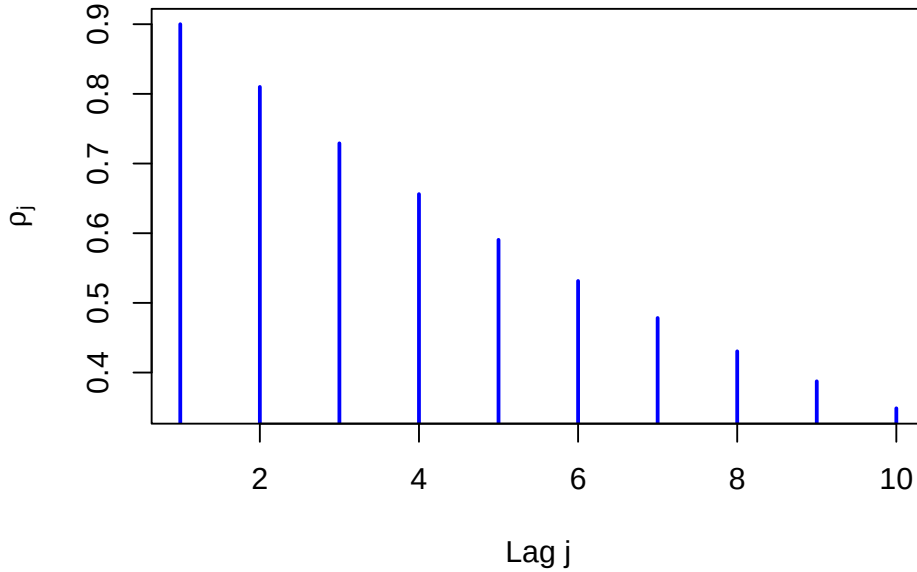


Figure 4.1: ACF for time series with $\rho_j = (0.9)^j$

For this process the strength of linear time dependence decays toward zero geometrically fast as j increases.

The definition of covariance stationarity requires that $E[Y_t] < \infty$ and $\text{var}(Y_t) < \infty$. That is, $E[Y_t]$ and $\text{var}(Y_t)$ must exist and be finite numbers. This is true if Y_t is normally distributed. However, it is not true if, for example, Y_t has a Student's t distribution with one degree of freedom.² Hence, a strictly stationary stochastic process $\{Y_t\}$ where the (marginal) pdf of Y_t (for all t) has very fat tails may not be covariance stationary.

Example 4.3 (Gaussian White Noise). Let $Y_t \sim iid N(0, \sigma^2)$. Then $\{Y_t\}$ is called a *Gaussian white noise* (GWN) process and is denoted $Y_t \sim \text{GWN}(0, \sigma^2)$. Notice that:

$$\begin{aligned} E[Y_t] &= 0 \text{ independent of } t, \\ \text{var}(Y_t) &= \sigma^2 \text{ independent of } t, \\ \text{cov}(Y_t, Y_{t-j}) &= 0 \text{ (for } j > 0) \text{ independent of } t \text{ for all } j, \end{aligned}$$

so that $\{Y_t\}$ satisfies the properties of a covariance stationary process. The defining characteristic of a GWN process is the lack of any predictable pattern over time in the realized values of the

²This is also called a Cauchy distribution. For this distribution $E[Y_t] = \text{var}(Y_t) = \text{cov}(Y_t, Y_{t-j}) = \infty$.

process. The term white noise comes from the electrical engineering literature and represents the absence of any signal.³

Simulating observations from a GWN process in R is easy: just simulate iid observations from a normal distribution. For example, to simulate $T = 250$ observations from the GWN(0,1) process use:

```
set.seed(123)
y = rnorm(250)
```

The simulated iid $N(0,1)$ values are generated using the `rnorm()` function. The command `set.seed(123)` initializes R's internal random number generator using the seed value 123. Every time the random number generator seed is set to a particular value, the random number generator produces the same set of random numbers. This allows different people to create the same set of random numbers so that results are reproducible. The simulated data is illustrated in Figure Figure 4.2 created using:

```
ts.plot(y,main="Gaussian White Noise Process",xlab="time",
        ylab="y(t)", col="blue", lwd=2)
abline(h=0)
```

³As an example of white noise, think of tuning an AM radio. In between stations there is no signal and all you hear is static. This is white noise.

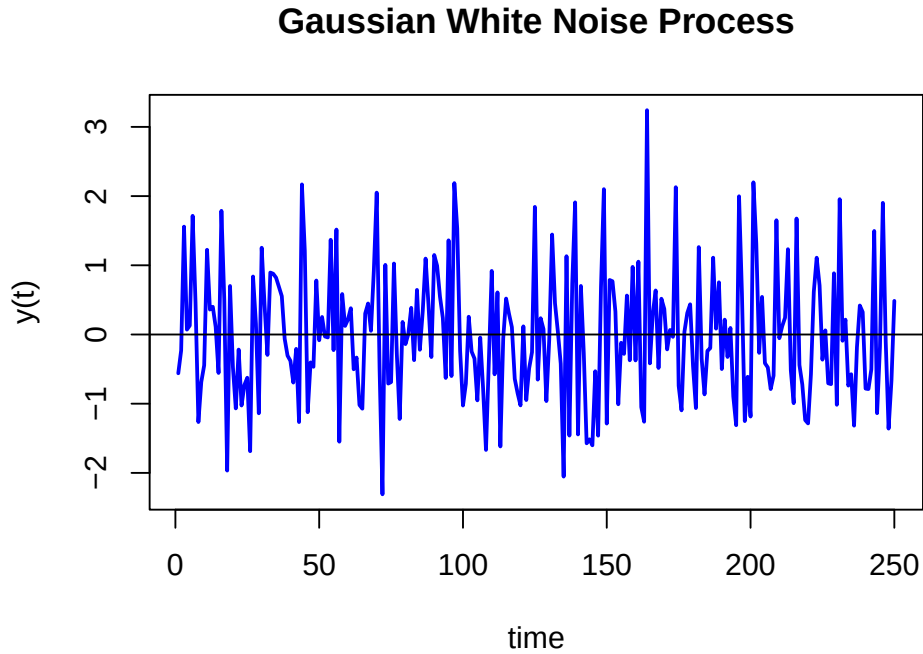


Figure 4.2: Realization of a GWN(0,1) process.

The function `ts.plot()` creates a time series line plot with a dummy time index. An equivalent plot can be created using the generic `plot()` function with optional argument `type="l"`. The data in Figure 4.2 fluctuate randomly about the mean value zero and exhibit a constant volatility of one (typical magnitude of a fluctuation about zero). There is no visual evidence of any predictable pattern in the data. ■

Example 4.4 (GWN model for continuously compounded returns). Let r_t denote the continuously compounded monthly return on Microsoft stock and assume that $r_t \sim \text{iid } N(0.01, (0.05)^2)$. We can represent this distribution in terms of a GWN process as follows

$$r_t = 0.01 + \varepsilon_t, \varepsilon_t \sim \text{GWN}(0, (0.05)^2).$$

Here, r_t is constructed as a $\text{GWN}(0, \sigma^2)$ process plus a constant. Hence, $\{r_t\}$ is a GWN process with a non-zero mean: $r_t \sim \text{GWN}(0.01, (0.05)^2)$. $T = 60$ simulated values of $\{r_t\}$ are computed using:

```
set.seed(123)
y = rnorm(60, mean=0.01, sd=0.05)
ts.plot(y, main="GWN Process for Monthly Continuously Compounded Returns",
        xlab="time", ylab="r(t)", col="blue", lwd=2)
```



```
abline(h=c(0,0.01,-0.04,0.06), lwd=2,
      lty=c("solid","solid","dotted","dotted"),
      col=c("black", "red", "red", "red"))
```

GWN Process for Monthly Continuously Compounded Retu

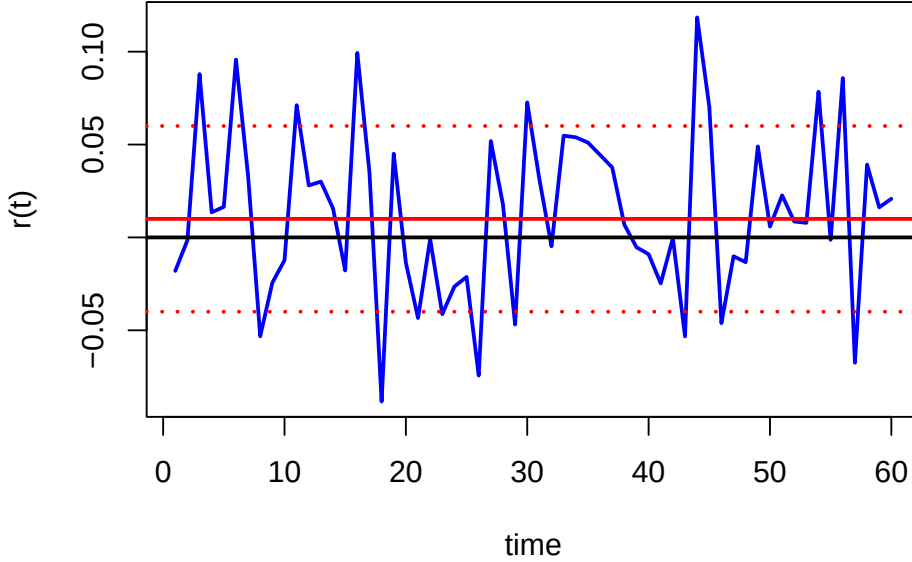


Figure 4.3: Simulated returns from $\text{GWN}(0.01, (0.05)^2)$.

and are illustrated in Figure Figure 4.3. Notice that the returns fluctuate around the mean value of 0.01 (solid red line) and the size of a typical deviation from the mean is about 0.05. The red dotted lines show the values 0.1 ± 0.05 .

An implication of the GWN assumption for monthly returns is that non-overlapping multi-period returns are also GWN. For example, consider the two-month return $r_t(2) = r_t + r_{t-1}$. The non-overlapping process $\{r_t(2)\} = \{\dots, r_{t-2}(2), r_t(2), r_{t+2}(2), \dots\}$ is GWN with mean $E[r_t(2)] = 2 \cdot \mu = 0.02$, variance $\text{var}(r_t(2)) = 2 \cdot \sigma^2 = 0.005$, and standard deviation $\text{sd}(r_t(2)) = \sqrt{2}\sigma = 0.071$. ■

Example 4.5 (Independent white noise). Let $Y_t \sim \text{iid}(0, \sigma^2)$. Then $\{Y_t\}$ is called an *independent white noise* (IWN) process and is denoted $Y_t \sim \text{IWN}(0, \sigma^2)$. The difference between GWN and IWN is that with IWN we don't specify that all random variables are normally distributed. The random variables can have any distribution with mean zero and variance σ^2 . To illustrate, suppose $Y_t = \frac{1}{\sqrt{3}} \times t_3$ where t_3 denotes a Student's t distribution with 3 degrees of freedom. This process has $E[Y_t] = 0$ and $\text{var}(Y_t) = 1$.⁴ Figure Figure 4.4 shows simulated observations from this process

⁴Recall, if $Y_t \sim t_v$ then $\text{var}(Y_t) = \frac{v}{v-2}$ for $v > 2$.

created using the R commands

```
set.seed(123)
y = (1/sqrt(3))*rt(250, df=3)
ts.plot(y, main="Independent White Noise Process", xlab="time", ylab="y(t)",
        col="blue", lwd=2)
abline(h=0)
```

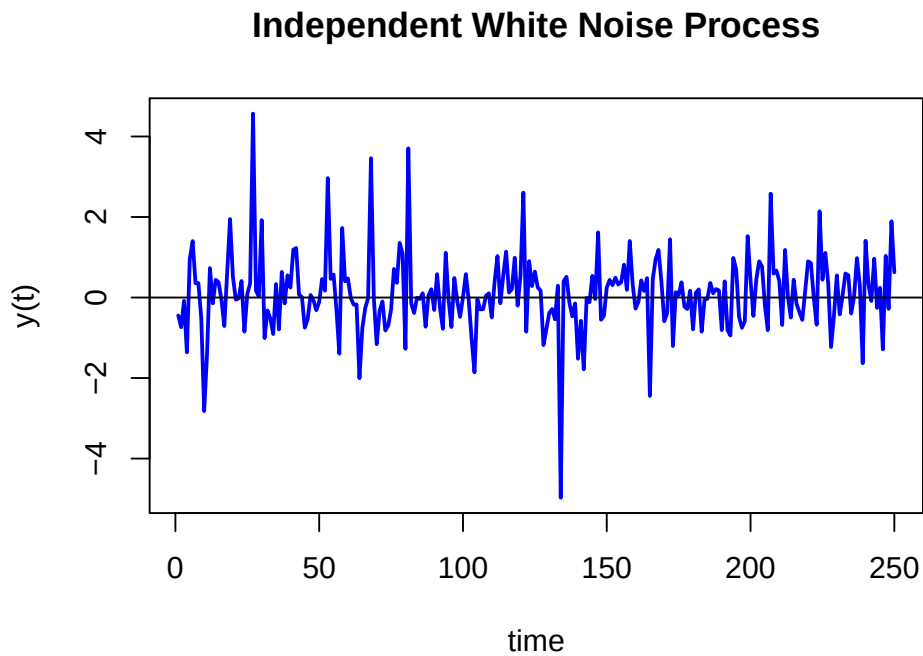


Figure 4.4: Simulation of IWN(0,1) process: $Y_t \sim \frac{1}{\sqrt{3}} \times t_3$

The simulated IWN process resembles the GWN in Figure Figure 4.4 but has more extreme observations. ■

Example 4.6 (Weak white noise). Let $\{Y_t\}$ be a sequence of uncorrelated random variables each with mean zero and variance σ^2 . Then $\{Y_t\}$ is called a *weak white noise* (WWN) process and is denoted $Y_t \sim \text{WWN}(0, \sigma^2)$. With a WWN process, the random variables are not independent, only uncorrelated. This allows for potential non-linear temporal dependence between the random variables in the process. ■

4.1.2 Non-Stationary processes

In a covariance stationary stochastic process it is assumed that the means, variances and autocovariances are independent of time. In a *non-stationary* process, one or more of these assumptions is not true. The following examples illustrate some typical non-stationary time series processes.

Example 4.7 (Deterministically trending process). Suppose $\{Y_t\}$ is generated according to the *deterministically trending process*:

$$Y_t = \beta_0 + \beta_1 t + \varepsilon_t, \quad \varepsilon_t \sim \text{GWN}(0, \sigma_\varepsilon^2), \\ t = 0, 1, 2, \dots$$

Then $\{Y_t\}$ is nonstationary because the mean of Y_t depends on t :

$$E[Y_t] = \beta_0 + \beta_1 t.$$

Figure 4.5 shows a realization of this process with $\beta_0 = 0, \beta_1 = 0.1$ and $\sigma_\varepsilon^2 = 1$ created using the R commands:

```
set.seed(123)
e = rnorm(250)
y.dt = 0.1*seq(1,250) + e
ts.plot(y.dt, lwd=2, col="blue", main="Deterministic Trend + Noise")
abline(a=0, b=0.1)
```

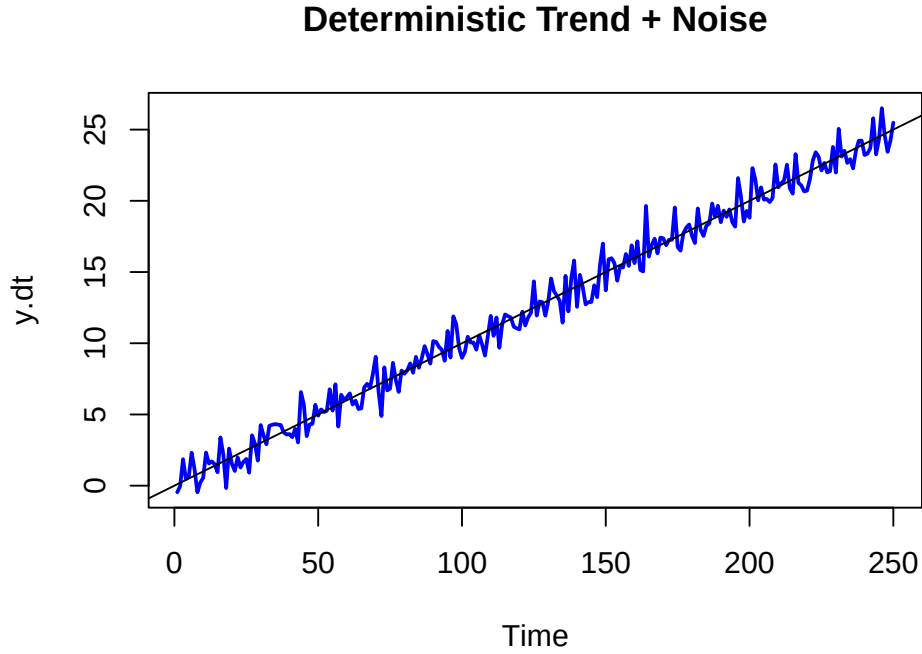


Figure 4.5: Deterministically trending nonstationary process $Y_t = 0.1 \times t + \varepsilon_t, \varepsilon_t \sim \text{GWN}(0, 1)$

Here the non-stationarity is created by the deterministic trend $\beta_0 + \beta_1 t$ in the data. The non-stationary process $\{Y_t\}$ can be transformed into a stationary process by simply subtracting off the trend:

$$X_t = Y_t - \beta_0 - \beta_1 t = \varepsilon_t \sim \text{GWN}(0, \sigma_\varepsilon^2).$$

The detrended process $X_t \sim \text{GWN}(0, \sigma_\varepsilon^2)$ is obviously covariance stationary. ■

Example 4.8 (Random walk). A random walk (RW) process $\{Y_t\}$ is defined as:

$$\begin{aligned} Y_t &= Y_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \text{GWN}(0, \sigma_\varepsilon^2), \\ Y_0 &\text{ is fixed (non-random).} \end{aligned}$$

By recursive substitution starting at $t = 1$, we have:

$$\begin{aligned} Y_1 &= Y_0 + \varepsilon_1, \\ Y_2 &= Y_1 + \varepsilon_2 = Y_0 + \varepsilon_1 + \varepsilon_2, \\ &\vdots \\ Y_t &= Y_{t-1} + \varepsilon_t = Y_0 + \varepsilon_1 + \cdots + \varepsilon_t \\ &= Y_0 + \sum_{j=1}^t \varepsilon_j. \end{aligned}$$

Now, $E[Y_t] = Y_0$ which is independent of t . However,

$$\text{var}(Y_t) = \text{var}\left(\sum_{j=1}^t \varepsilon_j\right) = \sum_{j=1}^t \sigma_\varepsilon^2 = \sigma_\varepsilon^2 \times t,$$

which depends on t , and so $\{Y_t\}$ is not stationary.

Figure 4.6 shows a realization of the RW process with $Y_0 = 0$ and $\sigma_\varepsilon^2 = 1$ created using the R commands:

```
set.seed(321)
e = rnorm(250)
y.rw = cumsum(e)
ts.plot(y.rw, lwd=2, col="blue", main="Random Walk")
abline(h=0)
```

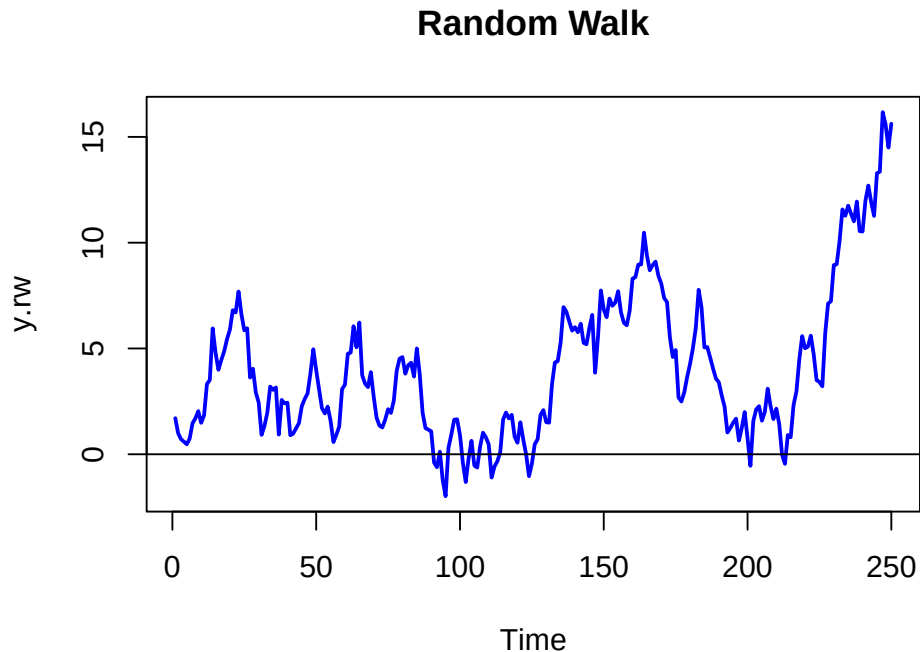


Figure 4.6: Random walk process: $Y_t = Y_{t-1} + \varepsilon_t, \varepsilon_t \sim \text{GWN}(0, 1)$.

The RW process looks much different from the GWN process in Figure 4.2. As the variance of the process increases linearly with time, the uncertainty about where the process will be at a given point in time increases with time.

Although $\{Y_t\}$ is non-stationary, a simple first-differencing transformation, however, yields a co-

variance stationary process:

$$X_t = Y_t - Y_{t-1} = \varepsilon_t \sim \text{GWN}(0, \sigma_\varepsilon^2).$$

■

Example 4.9 (Random walk with drift model for log stock prices). Let r_t denote the continuously compounded monthly return on Microsoft stock and assume that $r_t \sim \text{GWN}(\mu, \sigma^2)$. Since $r_t = \ln(P_t/P_{t-1})$ it follows that $\ln P_t = \ln P_{t-1} + r_t$. Now, re-express r_t as $r_t = \mu + \varepsilon_t$ where $\varepsilon_t \sim \text{GWN}(0, \sigma^2)$. Then $\ln P_t = \ln P_{t-1} + \mu + \varepsilon_t$. By recursive substitution we have $\ln P_t = \ln P_0 + \mu t + \sum_{i=1}^t \varepsilon_i$ and so $\ln P_t$ follows a random walk process with drift value μ . Here, $E[\ln P_t] = \mu t$ and $\text{var}(\ln P_t) = \sigma^2 t$ so $\ln P_t$ is non-stationary because both the mean and variance depend on t . In this model, prices, however, do not follow a random walk since $P_t = e^{\ln P_t} = P_{t-1} e^{r_t}$. ■

4.1.3 Ergodicity

In a strictly stationary or covariance stationary stochastic process no assumption is made about the strength of dependence between random variables in the sequence. For example, in a covariance stationary stochastic process it is possible that $\rho_1 = \text{cor}(Y_t, Y_{t-1}) = \rho_{100} = \text{cor}(Y_t, Y_{t-100}) = 0.5$, say. However, in many contexts it is reasonable to assume that the strength of dependence between random variables in a stochastic process diminishes the farther apart they become. That is, $\rho_1 > \rho_2 > \dots$ and that eventually $\rho_j = 0$ for j large enough. This diminishing dependence assumption is captured by the concept of *ergodicity*.

Definition 4.5 (Ergodicity (intuitive definition)). Intuitively, a stochastic process $\{Y_t\}$ is *ergodic* if any two collections of random variables partitioned far apart in the sequence are essentially independent.

The formal definition of ergodicity is highly technical and requires advanced concepts in probability theory. However, the intuitive definition captures the essence of the concept. The stochastic process $\{Y_t\}$ is ergodic if Y_t and Y_{t-j} are essentially independent if j is large enough.

If a stochastic process $\{Y_t\}$ is covariance stationary and ergodic then strong restrictions are placed on the joint behavior of the elements in the sequence and on the type of temporal dependence allowed.

Example 4.10 (White noise processes). If $\{Y_t\}$ is GWN or IWN then it is both covariance stationary and ergodic. ■

Example 4.11 (Covariance stationary but not ergodic process (White 1984 page 41)). Let $Y_t \sim \text{GWN}(0, 1)$ and let $X \sim N(0, 1)$ independent of $\{Y_t\}$. Define $Z_t = Y_t + X$. Then $\{Z_t\}$ is covariance stationary but not ergodic. To see why $\{Z_t\}$ is not ergodic, note that for all $j > 0$:

$$\begin{aligned}\text{var}(Z_t) &= \text{var}(Y_t + X) = 1 + 1 = 2, \\ \gamma_j &= \text{cov}(Y_t + X, Y_{t-j} + X) = \text{cov}(Y_t, Y_{t-j}) + \text{cov}(Y_t, X) + \text{cov}(Y_{t-j}, X) + \text{cov}(X, X) \\ &= \text{cov}(X, X) = \text{var}(X) = 1, \\ \rho_j &= \frac{1}{2} \text{ for all } j.\end{aligned}$$

Hence, the correlation between random variables separated far apart does not eventually go to zero and so $\{Z_t\}$ cannot be ergodic. ■

4.2 Multivariate Time Series

Consider n time series variables $\{Y_{1t}\}, \dots, \{Y_{nt}\}$. A *multivariate time series* is the $(n \times 1)$ vector time series $\{\mathbf{Y}_t\}$ where the i^{th} row of $\{\mathbf{Y}_t\}$ is $\{Y_{it}\}$. That is, for any time t , $\mathbf{Y}_t = (Y_{1t}, \dots, Y_{nt})'$. Multivariate time series analysis is used when one wants to model and explain the interactions and co-movements among a group of time series variables. In finance, multivariate time series analysis is used to model systems of asset returns, asset prices, exchange rates, the term structure of interest rates, and economic variables, etc.. Many of the time series concepts described previously for univariate time series carry over to multivariate time series in a natural way. Additionally, there are some important time series concepts that are particular to multivariate time series. The following sections give the details of these extensions.

4.2.1 Stationary and ergodic multivariate time series

A multivariate time series $\{\mathbf{Y}_t\}$ is covariance stationary and ergodic if all of its component time series are stationary and ergodic. The mean of $\mathbf{Y}_t$ is defined as the $(n \times 1)$ vector

$$E[\mathbf{Y}_t] = (\mu_1, \dots, \mu_n)' = \boldsymbol{\mu},$$

where $\mu_i = E[Y_{it}]$ for $i = 1, \dots, n$. The variance/covariance matrix of $\mathbf{Y}_t$ is the $(n \times n)$ matrix

$$\begin{aligned}\text{var}(\mathbf{Y}_t) &= \boldsymbol{\Sigma} = E[(\mathbf{Y}_t - \boldsymbol{\mu})(\mathbf{Y}_t - \boldsymbol{\mu})'] \\ &= \begin{pmatrix} \text{var}(Y_{1t}) & \text{cov}(Y_{1t}, Y_{2t}) & \cdots & \text{cov}(Y_{1t}, Y_{nt}) \\ \text{cov}(Y_{2t}, Y_{1t}) & \text{var}(Y_{2t}) & \cdots & \text{cov}(Y_{2t}, Y_{nt}) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(Y_{nt}, Y_{1t}) & \text{cov}(Y_{nt}, Y_{2t}) & \cdots & \text{var}(Y_{nt}) \end{pmatrix}.\end{aligned}$$

The matrix Σ has elements $\sigma_{ij} = \text{cov}(Y_{it}, Y_{jt})$ which measure the contemporaneous linear dependence between Y_{it} and Y_{jt} that is time invariant. The correlation matrix of $\mathbf{Y}_t$ is the $(n \times n)$ matrix

$$\text{cor}(\mathbf{Y}_t) = \mathbf{C} = \mathbf{D}^{-1}\Sigma\mathbf{D}^{-1},$$

where $\mathbf{D}$ is an $(n \times n)$ diagonal matrix with j^{th} diagonal element $\sigma_j = \text{sd}(Y_{jt})$.

4.2.1.1 Cross covariance and correlation matrices

For a univariate time series $\{Y_t\}$, the autocovariances, γ_k , and autocorrelations, ρ_k , summarize the linear time dependence in the data. With a multivariate time series $\{\mathbf{Y}_t\}$ each component has autocovariances and autocorrelations but there are also cross lead-lag covariances and correlations between all possible pairs of components. The lag k autocovariances and autocorrelations of Y_{jt} , for $j = 1, \dots, n$, are defined as

$$\begin{aligned}\gamma_{jj}^k &= \text{cov}(Y_{jt}, Y_{jt-k}), \\ \rho_{jj}^k &= \text{corr}(Y_{jt}, Y_{jt-k}) = \frac{\gamma_{jj}^k}{\sigma_j^2},\end{aligned}$$

and these are symmetric in k : $\gamma_{jj}^k = \gamma_{jj}^{-k}$, $\rho_{jj}^k = \rho_{jj}^{-k}$. The *cross lag- k covariances* and *cross lag- k correlations* between Y_{it} and Y_{jt} are defined as

$$\begin{aligned}\gamma_{ij}^k &= \text{cov}(Y_{it}, Y_{jt-k}), \\ \rho_{ij}^k &= \text{corr}(Y_{it}, Y_{jt-k}) = \frac{\gamma_{ij}^k}{\sqrt{\sigma_i^2 \sigma_j^2}},\end{aligned}$$

and they are not necessarily symmetric in k . In general,

$$\gamma_{ij}^k = \text{cov}(Y_{it}, Y_{jt-k}) \neq \text{cov}(Y_{jt}, Y_{it-k}) = \gamma_{ji}^k.$$

If $\gamma_{ij}^k \neq 0$ for some $k > 0$ then Y_{jt} is said to *lead* Y_{it} . This implies that past values of Y_{jt} are useful for predicting future values of Y_{it} . Similarly, if $\gamma_{ji}^k \neq 0$ for some $k > 0$ then Y_{it} is said to *lead* Y_{jt} . It is possible that Y_{it} leads Y_{jt} and vice-versa. In this case, there is said to be *dynamic feedback* between the two series.

All of the lag k cross covariances and correlations are summarized in the $(n \times n)$ lag k cross

covariance and lag k cross correlation matrices

$$\begin{aligned}\Gamma_k &= E[(\mathbf{Y}_t - \mu)(\mathbf{Y}_{t-k} - \mu)'] \\ &= \begin{pmatrix} \text{cov}(Y_{1t}, Y_{1t-k}) & \text{cov}(Y_{1t}, Y_{2t-k}) & \cdots & \text{cov}(Y_{1t}, Y_{nt-k}) \\ \text{cov}(Y_{2t}, Y_{1t-k}) & \text{cov}(Y_{2t}, Y_{2t-k}) & \cdots & \text{cov}(Y_{2t}, Y_{nt-k}) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(Y_{nt}, Y_{1t-k}) & \text{cov}(Y_{nt}, Y_{2t-k}) & \cdots & \text{cov}(Y_{nt}, Y_{nt-k}) \end{pmatrix}, \\ \mathbf{C}_k &= \mathbf{D}^{-1} \Gamma_k \mathbf{D}^{-1}.\end{aligned}$$

The matrices Γ_k and $\mathbf{C}_k$ are not symmetric in k but it is easy to show that $\Gamma_{-k} = \Gamma_k'$ and $\mathbf{C}_{-k} = \mathbf{C}_k'$.

Example 4.12 (Multivariate Gaussian white noise processes). Let $\{\mathbf{Y}_t\}$ be an $n \times 1$ vector time series process. If $\mathbf{Y}_t \sim iid N(\mathbf{0}, \Sigma)$ then $\{\mathbf{Y}_t\}$ is called *multivariate Gaussian white noise* and is denoted $\mathbf{Y}_t \sim \text{GWN}(\mathbf{0}, \Sigma)$. Notice that

$$\begin{aligned}E[\mathbf{Y}_t] &= \mathbf{0}, \\ \text{var}(\mathbf{Y}_t) &= \Sigma, \\ \text{cov}(Y_{jt}, Y_{jt-k}) &= \gamma_{jj}^k = 0 \text{ (for } k > 0) \\ \text{cov}(Y_{it}, Y_{jt-k}) &= \gamma_{ij}^k = 0 \text{ (for } k > 0)\end{aligned}$$

Hence, the elements of $\{\mathbf{Y}_t\}$ are contemporaneously correlated but exhibit no time dependence. That is, each element of $\mathbf{Y}_t$ exhibits no time dependence and there is no dynamic feedback between any two elements. Simulating observations from $\text{GWN}(\mathbf{0}, \Sigma)$ requires simulating from a multivariate normal distribution, which can be done using the **mvtnorm** function `rmvnorm()`. For example, to simulate and plot $T = 250$ observation from a bivariate $\text{GWN}(\mathbf{0}, \Sigma)$ process with

$$\Sigma = \begin{pmatrix} 4 & 1 \\ 1 & 1 \end{pmatrix} \Rightarrow \mathbf{C} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$$

use:

```
library(mvtnorm)
Sigma = matrix(c(4, 1, 1, 1), 2, 2)
set.seed(123)
Y = rmvnorm(250, sigma=Sigma)
colnames(Y) = c("Y1", "Y2")
ts.plot(Y, lwd=2, col=c("black", "blue"))
abline(h=0)
legend("topleft", legend=c("Y1", "Y2"),
```

```
lwd=2, col=c("black", "blue"), lty=1)
```

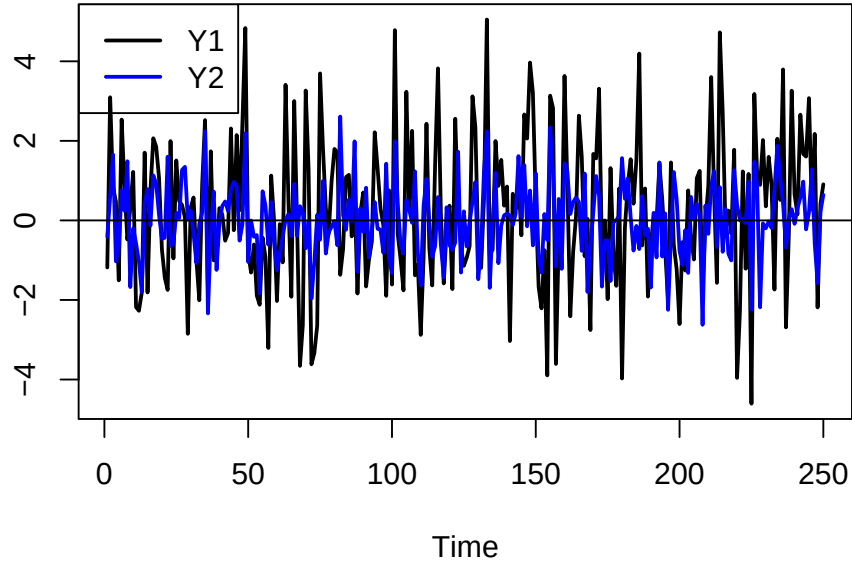


Figure 4.7: Simulated bivariate GWN process.

The simulated values are shown on the same plot in Figure Figure 4.7. Both series fluctuate randomly about zero, and the first series (black line) has larger fluctuations (volatility) than the second series (blue line). The two series are contemporaneously correlated ($\rho_{12} = 0.5$) but are both uncorrelated over time ($\rho_{11}^k = \rho_{22}^k = 0$, $k > 0$) and are not cross-lag correlated ($\rho_{12}^k = \rho_{21}^k = 0$, $k > 0$). ■

4.3 Time Series Models

Time series models are probability models that are used to describe the behavior of a stochastic process. In many cases of interest, it is assumed that the stochastic process to be modeled is covariance stationary and ergodic. Then, the main feature of the process to be modeled is the time dependence between the random variables. In this section, we illustrate some simple models for covariance stationary and ergodic time series that exhibit particular types of linear time dependence captured by autocorrelations. The univariate models, made popular originally by (Box and Jenkins 1976), are called *autoregressive moving average* (ARMA) models. The multivariate model, made

popular by (Sims 1980), is called the *vector autoregressive* (VAR) model. These models are used extensively in economics and finance for modeling univariate and multivariate time series.

4.3.1 Moving average models

Moving average models are simple covariance stationary and ergodic time series models built from linear functions of GWN that can capture time dependence between random variables that lasts only for a finite number of lags.

4.3.1.1 MA(1) Model

Suppose you want to create a covariance stationary and ergodic stochastic process $\{Y_t\}$ in which Y_t and Y_{t-1} are correlated but Y_t and Y_{t-j} are not correlated for $j > 1$. That is, the time dependence in the process only lasts for one period. Such a process can be created using the *first order moving average* (MA(1)) model:

$$Y_t = \mu + \varepsilon_t + \theta\varepsilon_{t-1}, \quad -1 < \theta < 1, \quad \varepsilon_t \sim \text{GWN}(0, \sigma_\varepsilon^2). \quad (4.2)$$

The MA(1) model is a simple linear function of the GWN random variables ε_t and ε_{t-1} . This linear structure allows for easy analysis of the model. The moving average parameter θ determines the sign and magnitude of the correlation between Y_t and Y_{t-1} . Clearly, if $\theta = 0$ then $Y_t = \mu + \varepsilon_t$ so that $\{Y_t\}$ is GWN with non-zero mean μ and exhibits no time dependence. As will be shown below, the MA(1) model produces a covariance stationary and ergodic process for any (finite) value of θ . The restriction $-1 < \theta < 1$ is called the *invertibility* restriction and will be explained below.

To verify that Equation 4.2 process is a covariance stationary process we must show that the mean, variance and autocovariances are time invariant. For the mean, we have:

$$E[Y_t] = \mu + E[\varepsilon_t] + \theta E[\varepsilon_{t-1}] = \mu,$$

because $E[\varepsilon_t] = E[\varepsilon_{t-1}] = 0$.

For the variance, we have

$$\begin{aligned} \text{var}(Y_t) &= \sigma^2 = E[(Y_t - \mu)^2] = E[(\varepsilon_t + \theta\varepsilon_{t-1})^2] \\ &= E[\varepsilon_t^2] + 2\theta E[\varepsilon_t \varepsilon_{t-1}] + \theta^2 E[\varepsilon_{t-1}^2] \\ &= \sigma_\varepsilon^2 + 0 + \theta^2 \sigma_\varepsilon^2 = \sigma_\varepsilon^2(1 + \theta^2). \end{aligned}$$

The term $E[\varepsilon_t \varepsilon_{t-1}] = \text{cov}(\varepsilon_t, \varepsilon_{t-1}) = 0$ because $\{\varepsilon_t\}$ is an independent process.

For $\gamma_1 = \text{cov}(Y_t, Y_{t-1})$, we have:

$$\begin{aligned}
\text{cov}(Y_t, Y_{t-1}) &= E[(Y_t - \mu)(Y_{t-1} - \mu)] \\
&= E[(\varepsilon_t + \theta\varepsilon_{t-1})(\varepsilon_{t-1} + \theta\varepsilon_{t-2})] \\
&= E[\varepsilon_t\varepsilon_{t-1}] + \theta E[\varepsilon_t\varepsilon_{t-2}] \\
&\quad + \theta E[\varepsilon_{t-1}^2] + \theta^2 E[\varepsilon_{t-1}\varepsilon_{t-2}] \\
&= 0 + 0 + \theta\sigma_\varepsilon^2 + 0 = \theta\sigma_\varepsilon^2.
\end{aligned}$$

Here, the sign of γ_1 is the same as the sign of θ .

For $\rho_1 = \text{cor}(Y_t, Y_{t-1})$ we have:

$$\rho_1 = \frac{\gamma_1}{\sigma^2} = \frac{\theta\sigma_\varepsilon^2}{\sigma_\varepsilon^2(1 + \theta^2)} = \frac{\theta}{(1 + \theta^2)}.$$

Clearly, $\rho_1 = 0$ if $\theta = 0$; $\rho_1 > 0$ if $\theta > 0$; $\rho_1 < 0$ if $\theta < 0$. Also, the largest value for $|\rho_1|$ is 0.5 which occurs when $|\theta| = 1$. Hence, a MA(1) model cannot describe a stochastic process that has $|\rho_1| > 0.5$. Also, note that there is more than one value of θ that produces the same value of ρ_1 . For example, θ and $1/\theta$ give the same value for ρ_1 . The *invertibility restriction* $-1 < \theta < 1$ provides a unique mapping between θ and ρ_1 .

For $\gamma_2 = \text{cov}(Y_t, Y_{t-2})$, we have:

$$\begin{aligned}
\text{cov}(Y_t, Y_{t-2}) &= E[(Y_t - \mu)(Y_{t-2} - \mu)] \\
&= E[(\varepsilon_t + \theta\varepsilon_{t-1})(\varepsilon_{t-2} + \theta\varepsilon_{t-3})] \\
&= E[\varepsilon_t\varepsilon_{t-2}] + \theta E[\varepsilon_t\varepsilon_{t-3}] \\
&\quad + \theta E[\varepsilon_{t-1}\varepsilon_{t-2}] + \theta^2 E[\varepsilon_{t-1}\varepsilon_{t-3}] \\
&= 0 + 0 + 0 + 0 = 0.
\end{aligned}$$

Similar calculations can be used to show that $\text{cov}(Y_t, Y_{t-j}) = \gamma_j = 0$ for $j > 1$. Hence, for $j > 1$ we have $\rho_j = 0$ and there is only time dependence between Y_t and Y_{t-1} but no time dependence between Y_t and Y_{t-j} for $j > 1$. Because $\rho_j = 0$ for $j > 1$ the MA(1) process is trivially ergodic.

Example 4.13 (Simulating values from MA(1) process). Consider simulating $T = 250$ observations from Equation 4.2 with $\mu = 1$, $\theta = 0.9$ and $\sigma_\varepsilon = 1$. When simulating an MA(1) process, you need to decide how to start the simulation. The value of Y_t at $t = 0$, y_0 , is called the *initial value* and is the starting value for the simulation. Now, the first two observations from Equation 4.2 starting

at $t = 0$ are

$$\begin{aligned}y_0 &= \mu + \varepsilon_0 + \theta\varepsilon_{-1}, \\y_1 &= \mu + \varepsilon_1 + \theta\varepsilon_0.\end{aligned}$$

It is common practice is to set $\varepsilon_{-1} = \varepsilon_0 = 0$ so that $y_0 = \mu$, $y_1 = \mu + \varepsilon_1 = y_0 + \varepsilon_1$ and ε_1 is the first simulated error term. The remaining observations for $t = 2, \dots, T$ are created from Equation 4.2. We can implement the simulation in a number of ways in R. The most straightforward way is to use a simple loop:

```
n.obs = 250
mu = 1
theta = 0.9
sigma.e = 1
set.seed(123)
e = rnorm(n.obs, sd = sigma.e)
y = rep(0, n.obs)
y[1] = mu + e[1]
for (i in 2:n.obs) {
  y[i] = mu + e[i] + theta*e[i-1]
}
head(y, n=3)
#> [1] 0.440 0.265 2.352
```

The “for loop” in R can be slow, however, especially for a very large number of simulations. The simulation can be more efficiently implemented using *vectorized* calculations as illustrated below:

```
set.seed(123)
e = rnorm(n.obs, sd = sigma.e)
em1 = c(0, e[1:(n.obs-1)])
y = mu + e + theta*em1
head(y, n=3)
#> [1] 0.440 0.265 2.352
```

The vectorized calculation avoids looping all together and computes all of the simulated values at the same time. This can be considerably faster than the “for loop” calculation.

The MA(1) model is a special case of the more general *autoregressive integrated moving average* (ARIMA) model. R has many built-in functions and several packages for working with ARIMA models. In particular, the R function `arima.sim()` can be used to simulate observations from a MA(1) process. It essentially implements the simulation loop described above. The arguments of

`arima.sim()` are:

```
args(arima.sim)
#> function (model, n, rand.gen = rnorm, innov = rand.gen(n, ...),
#>      n.start = NA, start.innov = rand.gen(n.start, ...), ...)
#> NULL
```

where `model` is a list with named components describing the ARIMA model parameters (excluding the mean value), `n` is the number of simulated observations, `rand.gen` specifies the pdf for ε_t , `innov` is a vector ε_t values of length `n`, `n.start` is the number of pre-simulation (burn-in) values for ε_t , `start.innov` is a vector of `n.start` pre-simulation values for ε_t , and `...` specify any additional arguments for `rand.gen`. For example, to perform the same simulations as above use:

```
ma1.model = list(ma=0.9)
set.seed(123)
y = mu + arima.sim(model=ma1.model, n=250,
                    n.start=1, start.innov=0)

head(y, n=3)
#> Time Series:
#> Start = 1
#> End = 3
#> Frequency = 1
#> [1] 0.440 0.265 2.352
```

The `ma` component of the "list" object `ma1.model` specifies the value of θ for the MA(1) model, and is used as an input to the function `arima.sim()`. The options `n.start = 1` and `start.innov = 0` sets the start-up initial value $\varepsilon_0 = 0$. By default, `arima.sim()` sets $\mu = 0$, specifies $\varepsilon_t \sim \text{GWN}(0, 1)$, and returns $\varepsilon_t + \theta\varepsilon_{t-1}$ for $t = 1, \dots, T$. The simulated value for Y_t is constructed by adding the value of `mu` ($\mu = 1$) to the output of `arima.sim()`.

The function `ARMAacf()` can be used to compute the theoretical autocorrelations, ρ_j , from the MA(1) model (recall, $\rho_1 = \theta/(1 + \theta^2)$ and $\rho_j = 0$ for $j > 1$). For example, to compute ρ_j for $j = 1, \dots, 10$ use:

```
ma1.acf = ARMAacf(ar=0, ma=0.9, lag.max=10)
ma1.acf
#>      0      1      2      3      4      5      6      7      8      9     10
#> 1.000 0.497 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
```

Figure Figure 4.8 shows the simulated data and the theoretical ACF created using:

```

par(mfrow=c(2,1))
ts.plot(y,main="MA(1) Process: mu=1, theta=0.9",
xlab="time",ylab="y(t)", col="blue", lwd=2)
abline(h=c(0,1))
plot(0:10, ma1.acf,type="h", col="blue", lwd=2,
main="ACF for MA(1): theta=0.9",xlab="lag",ylab="rho(j)")
abline(h=0)
par(mfrow=c(1,1))

```

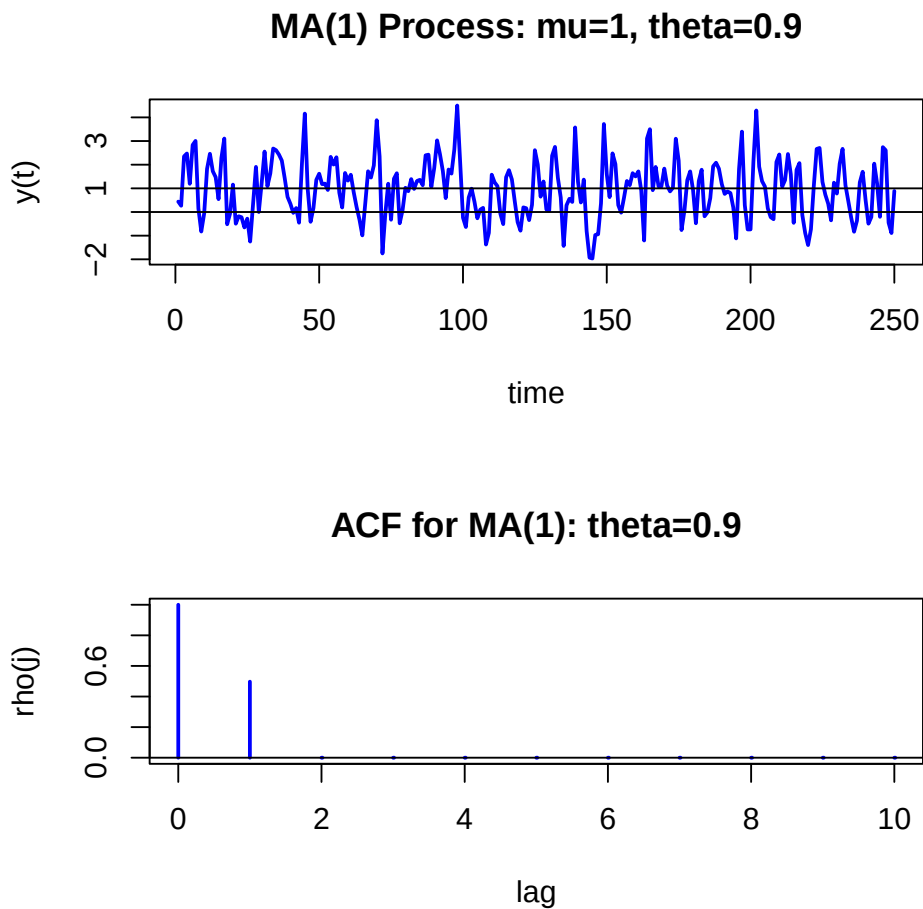


Figure 4.8: Simulated values and theoretical ACF from MA(1) process with $\mu = 1$, $\theta = 0.9$ and $\sigma_\varepsilon^2 = 1$.

Compared to the GWN process in Figure 4.2, the MA(1) process is a bit smoother in its appearance. This is due to the positive one-period time dependence captured by $\rho_1 = 0.4972$. ■

Example 4.14 (MA(1) model for overlapping continuously compounded returns). Let R_t denote

the one-month continuously compounded return and assume that:

$$R_t \sim \text{GWN}(\mu, \sigma^2).$$

Consider creating a monthly time series of two-month continuously compounded returns using:

$$R_t(2) = R_t + R_{t-1}.$$

The time series of these two-month returns, observed monthly, overlap by one month:

$$\begin{aligned} R_t(2) &= R_t + R_{t-1}, \\ R_{t-1}(2) &= R_{t-1} + R_{t-2}, \\ R_{t-2}(2) &= R_{t-2} + R_{t-3}, \\ &\vdots \end{aligned}$$

The one-month overlap in the two-month returns implies that $\{R_t(2)\}$ follows an MA(1) process. To show this, we need to show that the autocovariances of $\{R_t(2)\}$ behave like the autocovariances of an MA(1) process.

To verify that $\{R_t(2)\}$ follows an MA(1) process, first we have:

$$\begin{aligned} E[R_t(2)] &= E[R_t] + E[R_{t-1}] = 2\mu, \\ \text{var}(R_t(2)) &= \text{var}(R_t + R_{t-1}) = 2\sigma^2. \end{aligned}$$

Next, we have:

$$\text{cov}(R_t(2), R_{t-1}(2)) = \text{cov}(R_t + R_{t-1}, R_{t-1} + R_{t-2}) = \text{cov}(R_{t-1}, R_{t-1}) = \text{var}(R_{t-1}) = \sigma^2,$$

and,

$$\begin{aligned} \text{cov}(R_t(2), R_{t-2}(2)) &= \text{cov}(R_t + R_{t-1}, R_{t-2} + R_{t-3}) = 0, \\ \text{cov}(R_t(2), R_{t-j}(2)) &= 0 \text{ for } j > 1. \end{aligned}$$

Hence, the autocovariances of $\{R_t(2)\}$ are those of an MA(1) process.

Notice that

$$\rho_1 = \frac{\sigma^2}{2\sigma^2} = \frac{1}{2}.$$

What MA(1) process describes $\{R_t(2)\}$? Because $\rho_1 = \frac{\theta}{1+\theta^2} = 0.5$ it follows that $\theta = 1$. Hence, the

MA(1) process has mean 2μ and $\theta = 1$ and can be expressed as the MA(1) model:

$$\begin{aligned} R_t(2) &= 2\mu + \varepsilon_t + \varepsilon_{t-1}, \\ \varepsilon_t &\sim \text{GWN}(0, \sigma^2). \end{aligned}$$

Notice that this is a *non-invertible* MA(1) model. ■

4.3.1.2 MA(q) Model

The MA(q) model has the form

$$Y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}, \quad (4.3)$$

where

$$\varepsilon_t \sim \text{GWN}(0, \sigma_\varepsilon^2).$$

The MA(q) model is stationary and ergodic provided $\theta_1, \dots, \theta_q$ are finite. The moments of the MA(q) (see end-of-chapter exercises) are

$$\begin{aligned} E[Y_t] &= \mu, \\ \gamma_0 &= \sigma^2(1 + \theta_1^2 + \cdots + \theta_q^2), \\ \gamma_j &= \begin{cases} (\theta_j + \theta_{j+1}\theta_1 + \theta_{j+2}\theta_2 + \cdots + \theta_q\theta_{q-j})\sigma^2 & \text{for } j = 1, 2, \dots, q \\ 0 & \text{for } j > q \end{cases}. \end{aligned}$$

Hence, the ACF of an MA(q) is non-zero up to lag q and is zero afterward.

Example 4.15 (Overlapping returns and MA(q) models). MA(q) models often arise in finance through data aggregation transformations. For example, let $R_t = \ln(P_t/P_{t-1})$ denote the monthly continuously compounded return on an asset with price P_t . Define the annual return at time t using monthly returns as $R_t(12) = \ln(P_t/P_{t-12}) = \sum_{j=0}^{11} R_{t-j}$. Suppose $R_t \sim \text{GWN}(\mu, \sigma^2)$ and consider a sample of monthly returns of size T , $\{R_1, R_2, \dots, R_T\}$. A sample of annual returns may be created using *overlapping* or *non-overlapping* returns. Let $\{R_{12}(12), R_{13}(12), \dots, R_T(12)\}$ denote a sample of $T^* = T - 11$ monthly overlapping annual returns and $\{R_{12}(12), R_{24}(12), \dots, R_T(12)\}$ denote a sample of $T/12$ non-overlapping annual returns. Researchers often use overlapping returns in analysis due to the apparent larger sample size. One must be careful using overlapping returns because the monthly annual return sequence $\{R_t(12)\}$ is not a Gaussian white noise process even

if the monthly return sequence $\{R_t\}$ is. To see this, straightforward calculations give:

$$\begin{aligned} E[R_t(12)] &= 12\mu, \\ \gamma_0 &= \text{var}(R_t(12)) = 12\sigma^2, \\ \gamma_j &= \text{cov}(R_t(12), R_{t-j}(12)) = (12-j)\sigma^2 \text{ for } j < 12, \\ \gamma_j &= 0 \text{ for } j \geq 12. \end{aligned}$$

Since $\gamma_j = 0$ for $j \geq 12$ notice that $\{R_t(12)\}$ behaves like an MA(11) process:

$$\begin{aligned} R_t(12) &= 12\mu + \varepsilon_t + \theta_1\varepsilon_{t-1} + \cdots + \theta_{11}\varepsilon_{t-11}, \\ \varepsilon_t &\sim \text{GWN}(0, \sigma^2). \end{aligned}$$

■

4.3.2 Autoregressive Models

Moving average models can capture almost any kind of autocorrelation structure. However, this may require many moving average terms in Equation 4.3. Another type of simple time series model is the *autoregressive model*. This model can capture complex autocorrelation patterns with a small number of parameters and is used more often in practice than the moving average model.

4.3.2.1 AR(1) Model

Suppose you want to create a covariance stationary and ergodic stochastic process $\{Y_t\}$ in which Y_t and Y_{t-1} are correlated, Y_t and Y_{t-2} are slightly less correlated, Y_t and Y_{t-3} are even less correlated and eventually Y_t and Y_{t-j} are uncorrelated for j large enough. That is, the time dependence in the process decays to zero as the random variables in the process get farther and farther apart. Such a process can be created using the *first order autoregressive* (AR(1)) model:

$$Y_t - \mu = \phi(Y_{t-1} - \mu) + \varepsilon_t, \quad -1 < \phi < 1, \quad \varepsilon_t \sim \text{iid } N(0, \sigma_\varepsilon^2). \quad (4.4)$$

It can be shown that the AR(1) model is covariance stationary and ergodic provided $-1 < \phi < 1$. We will show that the AR(1) process has the following properties:

$$E[Y_t] = \mu, \quad (4.5)$$

$$\text{var}(Y_t) = \sigma^2 = \sigma_\varepsilon^2 / (1 - \phi^2), \quad (4.6)$$

$$\text{cov}(Y_t, Y_{t-1}) = \gamma_1 = \sigma^2 \phi, \quad (4.7)$$

$$\text{cor}(Y_t, Y_{t-1}) = \rho_1 = \gamma_1/\sigma^2 = \phi, \quad (4.8)$$

$$\text{cov}(Y_t, Y_{t-j}) = \gamma_j = \sigma^2 \phi^j, \quad (4.9)$$

$$\text{cor}(Y_t, Y_{t-j}) = \rho_j = \gamma_j/\sigma^2 = \phi^j. \quad (4.10)$$

Notice that the restriction $|\phi| < 1$ implies that:

$$\lim_{j \rightarrow \infty} \rho_j = \phi^j = 0,$$

so that Y_t is essentially independent of Y_{t-j} for large j and so $\{Y_t\}$ is ergodic. For example, if $\phi = 0.5$ then $\rho_{10} = (0.5)^{10} = 0.001$; if $\phi = 0.9$ then $\rho_{10} = (0.9)^{10} = 0.349$. Hence, the closer ϕ is to unity the stronger is the time dependence in the process. If $\phi = 1$, then Equation 4.4 becomes the random walk model $Y_t = Y_{t-1} + \varepsilon_t$ and is a non-stationary process.

Verifying covariance stationarity for the AR(1) model is more involved than for the MA(1) model, and establishing the properties Equation 4.5 - Equation 4.10 involves some tricks. In what follows, we will assume that $\{Y_t\}$ is a covariance stationary process and that $|\phi| < 1$. First, consider the derivation for Equation 4.5. We have:

$$\begin{aligned} E[Y_t] &= \mu + \phi(E[Y_{t-1}] - \mu) + E[\varepsilon_t] \\ &= \mu + \phi E[Y_{t-1}] - \phi\mu. \end{aligned}$$

Here, we use the first trick. Given that $\{Y_t\}$ is covariance stationary it follows that $E[Y_t] = E[Y_{t-1}]$. Substituting $E[Y_t] = E[Y_{t-1}]$ into the above and solving for $E[Y_t]$ gives Equation 4.5.

A similar trick can be used to derive Equation 4.6:

$$\text{var}(Y_t) = \phi^2(\text{var}(Y_{t-1})) + \text{var}(\varepsilon_t) = \phi^2(\text{var}(Y_t)) + \sigma_\varepsilon^2,$$

which uses the fact that Y_{t-1} is independent of ε_t (because Y_{t-1} only depends on $t-1$ values) and $\text{var}(Y_t) = \text{var}(Y_{t-1})$ given that $\{Y_t\}$ is covariance stationary. Solving for $\sigma^2 = \text{var}(Y_t)$ gives Equation 4.6.

To determine Equation 4.7, we use another trick. Multiply both sides of Equation 4.4 by $Y_{t-1} - \mu$ and take expectations to give:

$$\gamma_1 = E[(Y_t - \mu)(Y_{t-1} - \mu)] = \phi E[(Y_{t-1} - \mu)^2] + E[\varepsilon_t(Y_{t-1} - \mu)] = \phi\sigma^2,$$

which uses the fact that Y_{t-1} is independent of ε_t , and $\text{var}(Y_t) = \text{var}(Y_{t-1}) = \sigma^2$. Finally, to determine Equation 4.9, multiply both sides of Equation 4.4 by $Y_{t-j} - \mu$ and take expectations to

give:

$$\begin{aligned}\gamma_j &= E[(Y_t - \mu)(Y_{t-j} - \mu)] = \phi E[(Y_{t-1} - \mu)(Y_{t-j} - \mu)] + E[\varepsilon_t(Y_{t-j} - \mu)] \\ &= \phi \gamma_{j-1},\end{aligned}$$

which uses the fact that Y_{t-j} is independent of ε_t , and $E[(Y_{t-1} - \mu)(Y_{t-j} - \mu)] = \gamma_{j-1}$ provided $\{Y_t\}$ is covariance stationary. Using recursive substitution and $\gamma_0 = \sigma^2$ gives Equation 4.9.

The AR(1) model Equation 4.4 is written in what is called the *mean-adjusted form*. The mean-adjusted form can be re-expressed in the form of a linear regression model as follows:

$$\begin{aligned}Y_t - \mu &= \phi(Y_{t-1} - \mu) + \varepsilon_t \Rightarrow \\ Y_t &= \mu - \phi\mu + \phi Y_{t-1} + \varepsilon_t \\ &= c + \phi Y_{t-1} + \varepsilon_t,\end{aligned}\tag{4.11}$$

where $c = (1 - \phi)\mu \Rightarrow \mu = c/(1 - \phi)$ is the intercept of the linear regression. This regression model form is convenient for estimation by ordinary least squares.

Example 4.16 (Simulating values from AR(1) process). Consider simulating 250 observations from Equation 4.4 with $\mu = 1$, $\phi = 0.9$ and $\sigma_\varepsilon = 1$. To start the simulation, an initial value or start-up value for Y_0 is required. A commonly used initial value is the mean value μ so that $Y_1 = \mu + \varepsilon_1$. As with the MA(1) model, this can be performed using a simple “for loop” in R:

```
phi = 0.9
mu = 1
sigma.e = 1
n.obs = 250
y = rep(0, n.obs)
set.seed(123)
e = rnorm(n.obs, sd=sigma.e)
y[1] = mu + e[1]
for (i in 2:n.obs) {
  y[i] = mu + phi*(y[i-1] - mu) + e[i]
}
head(y, 3)
#> [1] 0.440 0.265 1.898
```

Unfortunately, there is no easy way to vectorize the loop calculation. However, the R function `filter()`, with optional argument `method = "recursive"`, implements the AR(1) recursion efficiently in C code and so is more efficient than the for loop code in R above:

```

y = mu + stats::filter(e, phi, method="recursive")
head(y, 3)
#> Time Series:
#> Start = 1
#> End = 3
#> Frequency = 1
#> [1] 0.440 0.265 1.898

```

The R function `arima.sim()`, which internally uses the `filter()` function, can also be used to simulate observations from an AR(1) process. For the AR(1) model, the function `arima.sim()` simulates the *components form* of the AR(1) model

$$\begin{aligned}
 Y_t &= \mu + u_t, \\
 u_t &= \phi u_{t-1} + \epsilon_t.
 \end{aligned}$$

Hence, to replicate the “for loop” simulation with `arima.sim()` use:

```

ar1.model = list(ar=0.9)
mu = 1
set.seed(123)
y = mu + arima.sim(model=ar1.model,n=250,n.start=1, start.innov=0)
head(y, 3)
#> Time Series:
#> Start = 1
#> End = 3
#> Frequency = 1
#> [1] 0.440 0.265 1.898

```

The R function `ARMAacf()` can be used to compute the theoretical ACF for an AR(1) model as follows

```

ar1.acf = ARMAacf(ar = 0.9, ma = 0, lag.max = 10)

```

The simulated AR(1) values and the ACF are shown in Figure Figure 4.9. Compared to the MA(1) process in Figure Figure 4.8, the realizations from the AR(1) process are much smoother. That is, when Y_t wanders high above its mean it tends to stay above the mean for a while and when it wanders low below the mean it tends to stay below for a while. ■

Example 4.17 (AR(1) models and the speed of mean reversion).

- insert example showing AR(1) models with different values of ρ

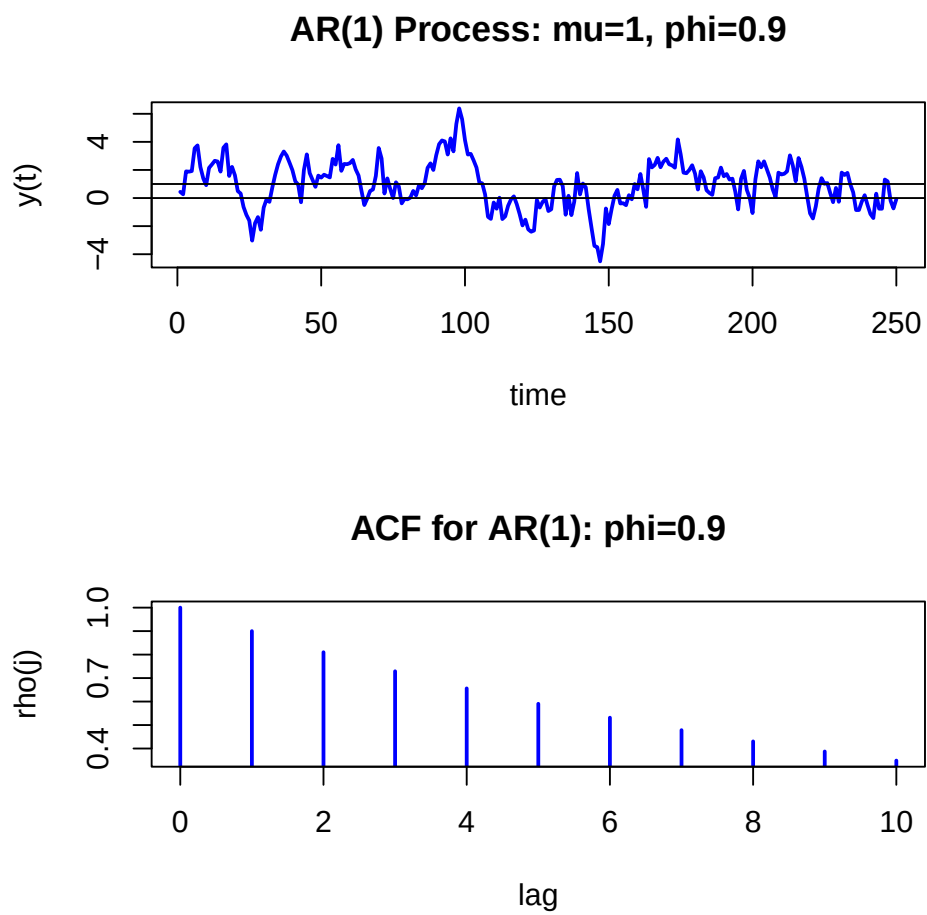


Figure 4.9: Simulated values and ACF from AR(1) model with $\mu = 1$, $\phi = 0.9$ and $\sigma_{\varepsilon}^2 = 1$

- discuss the concept of mean reversion
- discuss RW model as special case of AR(1)

■

4.3.2.2 AR(p) Model

The covariance stationary AR(p) model in mean-adjusted form is

$$\begin{aligned} Y_t - \mu &= \phi_1(Y_{t-1} - \mu) + \cdots + \phi_p(Y_{t-p} - \mu) + \varepsilon_t, \\ \varepsilon_t &\sim \text{GWN}(0, \sigma_\varepsilon^2), \end{aligned} \quad (4.12)$$

where $\mu = E[Y_t]$. Like the AR(1), restrictions on the autoregressive parameters $\phi_1, \dots, \phi_p$ are required for $\{Y_t\}$ to be covariance stationary and ergodic. A detailed treatment of these restrictions is beyond the scope of this book. However, one simple necessary condition for $\{Y_t\}$ to be covariance stationary is $|\phi| < 1$ where $\phi = \phi_1 + \cdots + \phi_p$. Hence, in the AR(p) model the sum of the autoregressive components ϕ has a similar interpretation as the single autoregressive coefficient in the AR(1) model.

The regression form of the AR(p) is

$$Y_t = c + \phi_1 Y_{t-1} + \cdots + \phi_p Y_{t-p} + \varepsilon_t,$$

where $c = \mu/(1 - \phi_1 - \cdots - \phi_p) = \mu/(1 - \phi)$. This form is convenient for estimation purposes because it is in the form of a linear regression.

The regression form of the AR(p) model is used very often in practice because of its simple linear structure and because it can capture a wide variety of autocorrelation patterns such as exponential decay, damped cyclical patterns, and oscillating damped cyclical patterns. Unfortunately, the mathematical derivation of the autocorrelations in the AR(p) model is complicated and tedious (and beyond the scope of this book). The exercises at the end of the chapter illustrate some of the calculations for the AR(2) model.

4.3.3 Autoregressive Moving Average Models

Autoregressive and moving average models can be combined into a general model called the *autoregressive moving average* (ARMA) model. The ARMA model with p autoregressive components and q moving average components, denoted ARMA(p, q) is given by

$$\begin{aligned} Y_t - \mu &= \phi_1(Y_{t-1} - \mu) + \cdots + \phi_p(Y_{t-p} - \mu) \\ &\quad + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q} \\ \varepsilon_t &\sim \text{GWN}(0, \sigma^2) \end{aligned} \quad (4.13)$$

The regression formulation is

$$Y_t = c + \phi_1 Y_{t-1} + \cdots + \phi_p Y_{t-p} + \varepsilon_t + \theta \varepsilon_{t-1} + \cdots + \theta \varepsilon_{t-q}$$

where $c = \mu/(1 - \phi_1 - \cdots - \phi_p) = \mu/(1 - \phi)$ and $\phi = \phi_1 + \cdots + \phi_p$. This model combines aspects of the pure moving average models and the pure autoregressive models and can capture many types of autocorrelation patterns. For modeling typical non-seasonal economic and financial data, it is seldom necessary to consider models in which $p > 2$ and $q > 2$. For example, the simple ARMA(1,1) model

$$Y_t - \mu = \phi_1(Y_{t-1} - \mu) + \varepsilon_t + \theta_1 \varepsilon_{t-1} \quad (4.14)$$

can capture many realistic autocorrelations observed in data.

4.3.4 Vector Autoregressive Models

The most popular multivariate time series model is the *vector autoregressive* (VAR) model. The VAR model is a multivariate extension of the univariate autoregressive model Equation 4.12. For example, a bivariate VAR(1) model for $\mathbf{Y}_t = (Y_{1t}, Y_{2t})'$ has the form

$$\begin{pmatrix} Y_{1t} \\ Y_{2t} \end{pmatrix} = \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} + \begin{pmatrix} a_{11}^1 & a_{12}^1 \\ a_{21}^1 & a_{22}^1 \end{pmatrix} \begin{pmatrix} Y_{1t-1} \\ Y_{2t-1} \end{pmatrix} + \begin{pmatrix} \varepsilon_{1t} \\ \varepsilon_{2t} \end{pmatrix},$$

or

$$\begin{aligned} Y_{1t} &= c_1 + a_{11}^1 Y_{1t-1} + a_{12}^1 Y_{2t-1} + \varepsilon_{1t}, \\ Y_{2t} &= c_2 + a_{21}^1 Y_{1t-1} + a_{22}^1 Y_{2t-1} + \varepsilon_{2t}, \end{aligned}$$

where

$$\begin{pmatrix} \varepsilon_{1t} \\ \varepsilon_{2t} \end{pmatrix} \sim \text{iid } N \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{pmatrix} \right).$$

In the equations for Y_1 and Y_2 , the lagged values of both Y_1 and Y_2 are present. Hence, the VAR(1) model allows for dynamic feedback between Y_1 and Y_2 and can capture cross-lag correlations between the variables. In matrix notation, the model is

$$\begin{aligned} \mathbf{Y}_t &= \mathbf{A} \mathbf{Y}_{t-1} + \varepsilon_t, \\ \varepsilon_t &\sim N(\mathbf{0}, \Sigma), \end{aligned}$$

where

$$\mathbf{A} = \begin{pmatrix} a_{11}^1 & a_{12}^1 \\ a_{21}^1 & a_{22}^1 \end{pmatrix}, \Sigma = \begin{pmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{pmatrix}.$$

The general VAR(p) model for $\mathbf{Y}_t = (Y_{1t}, Y_{2t}, \dots, Y_{nt})'$ has the form

$$\mathbf{Y}_t = \mathbf{c} + \mathbf{A}_1 \mathbf{Y}_{t-1} + \mathbf{A}_2 \mathbf{Y}_{t-2} + \dots + \mathbf{A}_p \mathbf{Y}_{t-p} + \varepsilon_t,$$

where $\mathbf{A}_i$ are $(n \times n)$ coefficient matrices and ε_t is an $(n \times 1)$ unobservable zero mean white noise vector process with covariance matrix Σ . VAR models are capable of capturing much of the complicated dynamics observed in stationary multivariate time series.

Example 4.18 (Simulated values from a bivariate VAR(1) process). To be completed

4.4 Forecasting

- One of the main practical uses of time series models is for forecasting future observations
- The existence of time dependence in a covariance stationary time series means that we can exploit this time dependence to obtain forecasts of future observations that are superior to the unconditional mean.
- Describe basic forecasting problem
- Show result that conditional mean is optimal mse forecast
- Show chain-rule of forecasting for AR(1) processes

4.5 Further Reading: Time Series Concepts

This chapter gives a very brief introduction to time series modeling. More thorough treatments of time series analysis with an orientation towards economics and finance with examples in R are given in (Ruppert and Matteson 2015), (Tsay 2010), and (Zivot 2016). The CRAN task view for Time Series is an excellent summary of R packages used for time series analysis.

4.6 Problems: Time Series Concepts

Exercise 4.1. Suppose the time series $\{X_t\}_{t=-\infty}^{\infty}$ is independent white noise. That is $X_t \sim i.i.d.(0, \sigma^2)$. Define two new time series $\{Y_t\}_{t=-\infty}^{\infty}$ and $\{Z_t\}_{t=-\infty}^{\infty}$, where $Y_t = X_t^2$ and $Z_t = |X_t|$. Are $\{Y_t\}_{t=-\infty}^{\infty}$ and $\{Z_t\}_{t=-\infty}^{\infty}$ also independent white noise processes? Why or why not?

Exercise 4.2. Realizations from four stochastic processes are given in Figure . Which processes appear to be covariance stationary and which processes appear to be non-stationary? Briefly justify your answers.

Exercise 4.3. Consider the MA(1) model

$$\begin{aligned}Y_t &= 0.05 + \varepsilon_t + \theta\varepsilon_{t-1}, \quad -1 < \theta < 1 \\ \varepsilon_t &\sim iid N(0, (0.10)^2).\end{aligned}$$

This process has mean $E[Y_t] = 0.05$.

1. Calculate $\text{var}(Y_t)$ and $\rho_1 = \text{cor}(Y_t, Y_{t-1})$ for $\theta = 0.5$ and $\theta = 0.9$.
2. Using the R function `arma.sim()`, simulate and plot 250 observations of the MA(1) process with $\theta = 0.5$ and $\theta = 0.9$. Briefly comment on the behavior of the simulated data series. Does it look covariance stationary? Does it show evidence of time dependence?

Exercise 4.4. Consider the AR(1) model

$$\begin{aligned}Y_t - 0.05 &= \phi(Y_{t-1} - 0.05) + \varepsilon_t, \quad -1 < \phi < 1 \\ \varepsilon_t &\sim iid N(0, (0.10)^2).\end{aligned}$$

This process has mean $E[Y_t] = 0.05$.

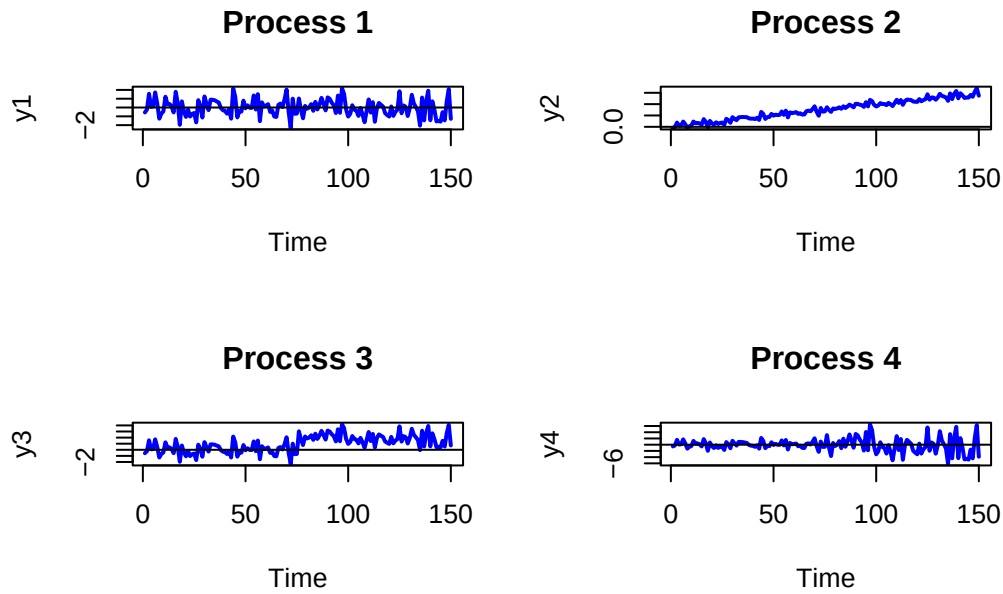
1. Calculate $\text{var}(Y_t)$ for $\phi = 0.5$ and $\phi = 0.9$.
2. Calculate $\rho_j = \text{cor}(Y_t, Y_{t-j})$ for $\phi = 0.5$ and $\phi = 0.9$ and for $j = 1, \dots, 5$.
3. Using the R function `arma.sim()`, simulate and plot 250 observations of the AR(1) process with $\theta = 0.5$ and $\theta = 0.9$. Briefly comment on the behavior of the simulated data series. Does it look covariance stationary? Does it show evidence of time dependence? How is it different from the MA(1) process

Exercise 4.5. Figure shows a realization of a stochastic process representing a monthly time series of overlapping 2-month continuously compounded returns $r_t(2) = r_t + r_{t-1}$, where the 1-month continuously compounded returns r_t follow a Gaussian White noise process with variance 1.

1. Based on the sample autocorrelations, which time series process is most appropriate for describing the series: MA(1) or AR(1)? Justify your answer.
2. If you think the process is an AR(1) process, what do you think is the value of the autoregressive parameter? If you think the process is a MA(1) process, what do you think is the value of the moving average parameter?

Exercise 4.6. Let Y_t represent a stochastic process. Under what conditions is Y_t covariance stationary?

Exercise 4.7. Realizations from four stochastic processes are given in the Figures below:



Which processes appear to be covariance stationary and which processes appear to be non-stationary? For those processes that you think are non-stationary, explain why the process is non-stationary.

Exercise 4.8. Consider the following model:

$$Y_t = 10 - 0.67Y_{t-1} + \epsilon_t, \quad \epsilon \sim N(0, 1)$$

1. Is it stationary? Why or why not?
2. Find the mean and the variance of this process.

5 Descriptive Statistics for Financial Data

Updated: September 4, 2026

Copyright © Eric Zivot 2026

In this chapter I use graphical and numerical descriptive statistics to study the distribution and dependence properties of daily and monthly asset returns on two representative assets representing a single stock and a large diversified portfolio of stocks. The purpose of this chapter is to introduce the techniques of exploratory data analysis for financial time series and to document a set of stylized facts for monthly and daily asset returns that will be used in later chapters to motivate probability models for asset returns.

This chapter is organized as follows. In Section 5.1 I introduce the example data and reviews graphical and numerical descriptive statistics for univariate data. Graphical descriptive statistics include time plots, histograms, QQ-plots and boxplots. Numerical statistics are the sample statistics associated with the analogous characteristics of univariate distributions reviewed in Chapter 2. In Section 5.2 I cover univariate time series descriptive statistics including sample autocovariances, autocorrelations and the sample autocorrelation function. In Section 5.3 I present bivariate descriptive statistics including scatterplots, sample covariance and correlation, and sample cross-lag covariances and correlations. In Section 5.5 I conclude with a summary of some stylized facts for monthly and daily returns.

The R packages I use in this chapter are **car**, **corrplot**, **IntroCompFinR**, **PerformanceAnalytics**, **sn**, **tseries**, **zoo** and **xts**. Make sure these packages are installed and loaded before running the R examples in the chapter.

5.1 Univariate Descriptive Statistics

Let $\{R_t\}$ denote a univariate time series of asset returns (simple or continuously compounded). Throughout this chapter I will assume that $\{R_t\}$ is a covariance stationary and ergodic stochastic

process such that:

$$\begin{aligned} E[R_t] &= \mu \text{ independent of } t, \\ \text{var}(R_t) &= \sigma^2 \text{ independent of } t, \\ \text{cov}(R_t, R_{t-j}) &= \gamma_j \text{ independent of } t, \\ \text{corr}(R_t, R_{t-j}) &= \gamma_j / \sigma^2 = \rho_j \text{ independent of } t. \end{aligned}$$

In addition, I will assume that each R_t is identically distributed with unknown pdf $f_R(r)$.

An observed sample of size T of historical asset returns $\{r_t\}_{t=1}^T$ is assumed to be a realization from the stochastic process $\{R_t\}$ for $t = 1, \dots, T$. That is,

$$\{r_t\}_{t=1}^T = \{R_1 = r_1, \dots, R_T = r_T\}$$

The goal of *exploratory data analysis* (eda) is to use the observed sample $\{r_t\}_{t=1}^T$ to learn about the unknown pdf $f_R(r)$ as well as the time dependence properties of $\{R_t\}$.

5.1.1 Example data

I illustrate the descriptive statistical analysis of financial data using daily and monthly adjusted closing prices on Microsoft stock (ticker symbol `msft`) and the S&P 500 index (ticker symbol `^gspc`) over the period January 2, 1998 and May 31, 2012.¹ These data are obtained from (finance.yahoo.com) and are available in the R package **introcompfinr**. I first use the daily and monthly data to illustrate descriptive statistical analysis and to establish a number of stylized facts about the distribution and time dependence in daily and monthly returns.

Example 5.1 (Constructing example data). Daily adjusted closing price data on Microsoft and the S&P 500 index over the period January 4, 1993 through December 31, 2014 are available in the **IntroCompFinR** package as the `xts` objects `msftDailyPrices` and `sp500DailyPrices`, respectively.

I restrict the sample period to January 2, 1998 and May 31, 2012 using:

```
smpl = "1998-01::2012-05"
msftDailyPrices = msftDailyPrices[smpl]
sp500DailyPrices = sp500DailyPrices[smpl]
```

¹An adjusted closing price is adjusted for dividend payments and stock splits. Any dividend payment received between closing dates are added to the close price. If a stock split occurs between the closing dates then the all past prices are divided by the split ratio. The ticker symbol `^gspc` refers to the actual S&P 500 index, which is not a tradeable security. There are several mutual funds (e.g., Vanguard's S&P 500 fund with ticker `VFINF`) and exchange traded funds (e.g., State Street's SPDR S&P 500 ETF with ticker `SPY`) which track the S&P 500 index that are investable.

End-of-month prices are extracted from the daily prices using the `xts` function `to.monthly()`:

```
msftMonthlyPrices = to.monthly(msftDailyPrices, OHLC=FALSE)
sp500MonthlyPrices = to.monthly(sp500DailyPrices, OHLC=FALSE)
```

It will also be convenient to create a merged `xts` object containing both the Microsoft and S&P 500 index prices:

```
msftSp500DailyPrices = merge(msftDailyPrices, sp500DailyPrices)
msftSp500MonthlyPrices = merge(msftMonthlyPrices, sp500MonthlyPrices)
```

I create `xts` objects containing simple returns using the `PerformanceAnalytics` function `Return.calculate()` (and remove the first NA value with `na.omit()`):

```
msftMonthlyRetS = na.omit(Return.calculate(msftMonthlyPrices,
                                           method="simple"))
msftDailyRetS = na.omit(Return.calculate(msftDailyPrices,
                                         method="simple"))
sp500MonthlyRetS = na.omit(Return.calculate(sp500MonthlyPrices,
                                           method="simple"))
sp500DailyRetS = na.omit(Return.calculate(sp500DailyPrices,
                                         method="simple"))
msftSp500MonthlyRetS = na.omit(Return.calculate(msftSp500MonthlyPrices,
                                              method="simple"))
msftSp500DailyRetS = na.omit(Return.calculate(msftSp500DailyPrices,
                                              method="simple"))
```

I also create `xts` objects containing monthly continuously compounded (cc) returns:

```
msftMonthlyRetC = log(1 + msftMonthlyRetS)
sp500MonthlyRetC = log(1 + sp500MonthlyRetS)
msftSp500MonthlyRetC = merge(msftMonthlyRetC, sp500MonthlyRetC)
```

■

5.1.2 Time plots

A natural graphical descriptive statistic for time series data is a *time plot*.

Definition 5.1 (Time Plot). A time plot is simply a line plot with the time series data on the y-axis and the time index on the x-axis.

Time plots are useful for quickly visualizing many features of the time series data.

Example 5.2 (Time plots of monthly prices and returns). A two-panel plot showing the monthly prices is given in Figure 5.1, and is created using the `plot` method for `xts` objects:

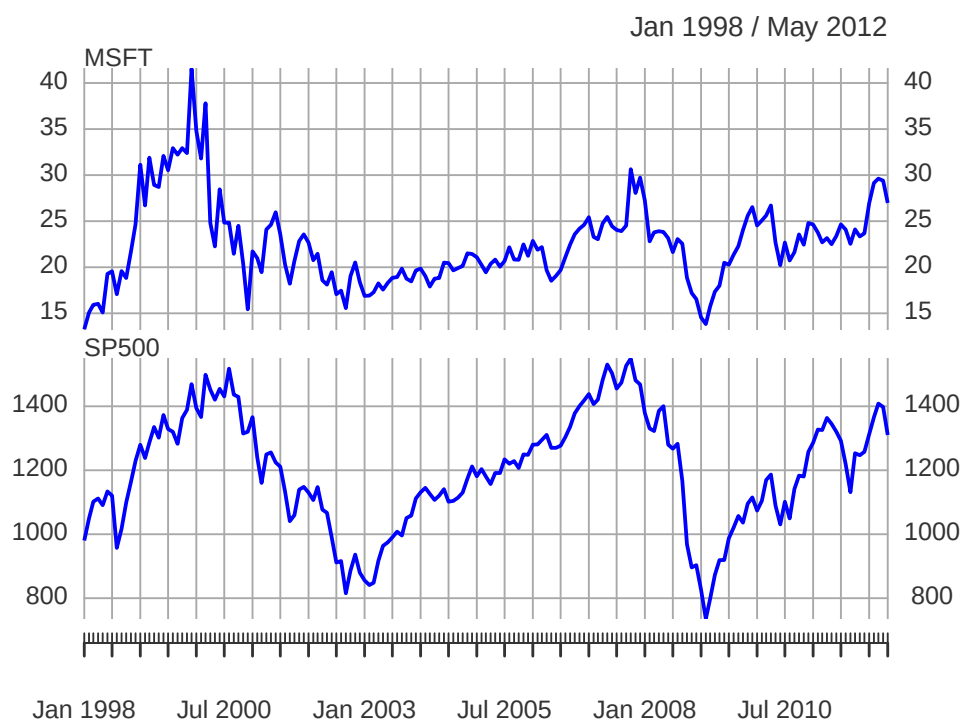


Figure 5.1: End-of-month closing prices on Microsoft stock and the S&P 500 index.

The prices exhibit random walk like behavior (no tendency to revert to a time independent mean) and appear to be non-stationary. Both prices show two large boom-bust periods associated with the dot-com period of the late 1990s and the run-up to the financial crisis of 2008 reflecting some common trend behavior of the two price series.

A time plot for the monthly simple returns is created using:

```
plot(msftSp500MonthlyRetS, multi.panel=TRUE, yaxis.same=FALSE,  
     main="", lwd=2, col="blue")
```

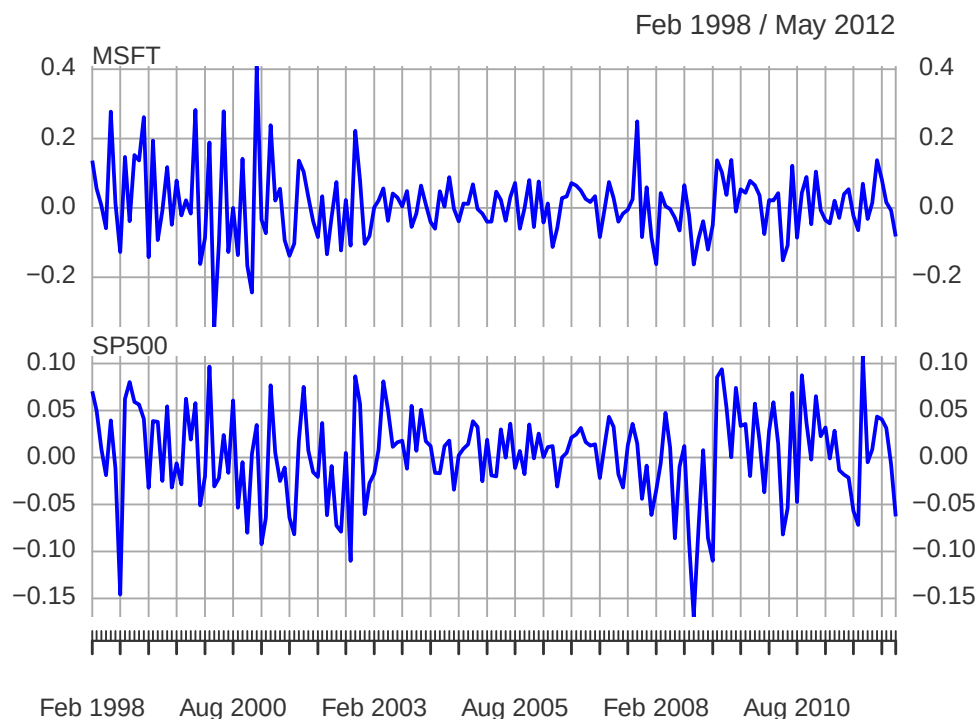


Figure 5.2: Monthly simple returns on Microsoft stock and the S&P 500 index.

and is given in Figure 5.2. In contrast to prices, returns show clear mean-reverting behavior and the common monthly mean values look to be very close to zero. Hence, the common mean value assumption of covariance stationarity looks to be satisfied. However, the volatility (i.e., fluctuation of returns about the mean) of both series appears to change over time. Both series show higher volatility over the periods 1998 - 2003 and 2008 - 2012 than over the period 2003 - 2008. This is an indication of possible non-stationarity in volatility.² Also, the coincidence of high and low volatility periods across assets suggests a common driver to the time varying behavior of volatility. There does not appear to be any visual evidence of systematic time dependence in the returns. Later on you will see that the estimated autocorrelations are very close to zero. The returns for Microsoft and the S&P 500 tend to go up and down together suggesting a positive correlation. ■

Example 5.3 (Plotting returns on the same graph). In Figure 5.2, the volatility of the returns on Microsoft and the S&P 500 looks to be similar but this is illusory. The y-axis scale for Microsoft is much larger than the scale for the S&P 500 index and so the volatility of Microsoft returns is actually much larger than the volatility of the S&P 500 returns. Figure 5.3 shows both returns series on the same time plot created using:

²The returns can still be covariance stationary and exhibit time varying *conditional* volatility. This is explored in chapter 10.


```
plot(msftSp500MonthlyRetS, main="", multi.panel=FALSE, lwd=2,
     col=c("red", "blue"), lty=c("dashed", "solid"),
     legend.loc = "topright")
```

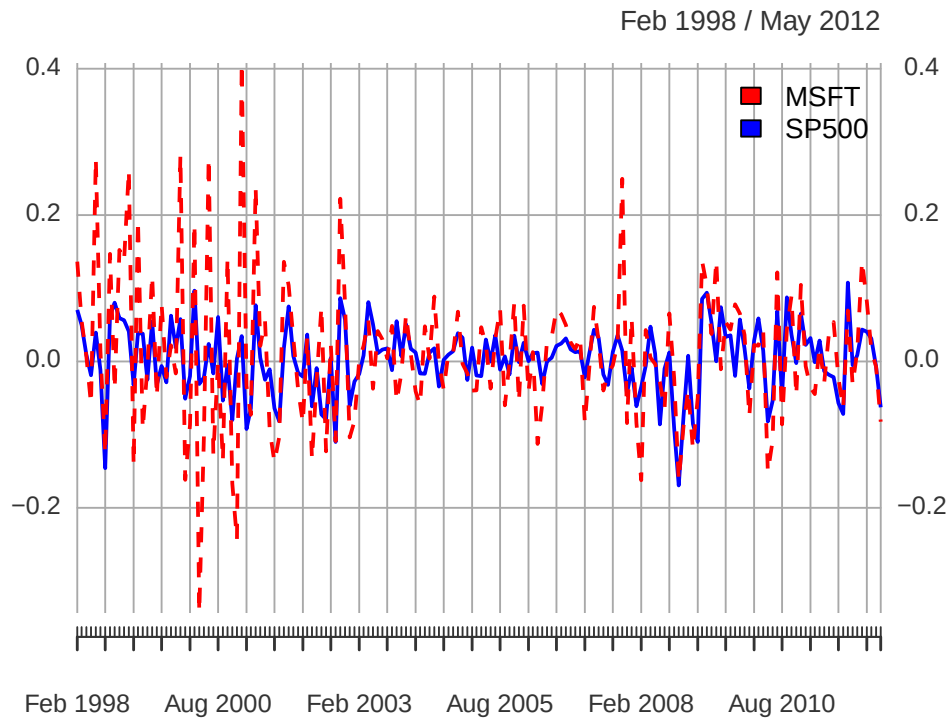


Figure 5.3: Monthly simple returns for Microsoft and S&P 500 index on the same graph.

Now the higher volatility of Microsoft returns, especially before 2003, is clearly visible. However, after 2008 the volatilities of the two series look quite similar. In general, the lower volatility of the S&P 500 index represents risk reduction due to holding a large diversified portfolio. ■

Example 5.4 (Comparing simple and continuously compounded returns). Figure 5.4 compares the simple and continuously compounded (cc) monthly returns for Microsoft created using:

```
retDiff = msftMonthlyRetS - msftMonthlyRetC
dataToPlot = merge(msftMonthlyRetS, msftMonthlyRetC, retDiff)
colnames(dataToPlot) = c("Simple", "cc", "Diff")
plot(dataToPlot, multi.panel=TRUE, yaxis.same=FALSE, main="",
     lwd=2, col=c("black", "blue", "red"))
```

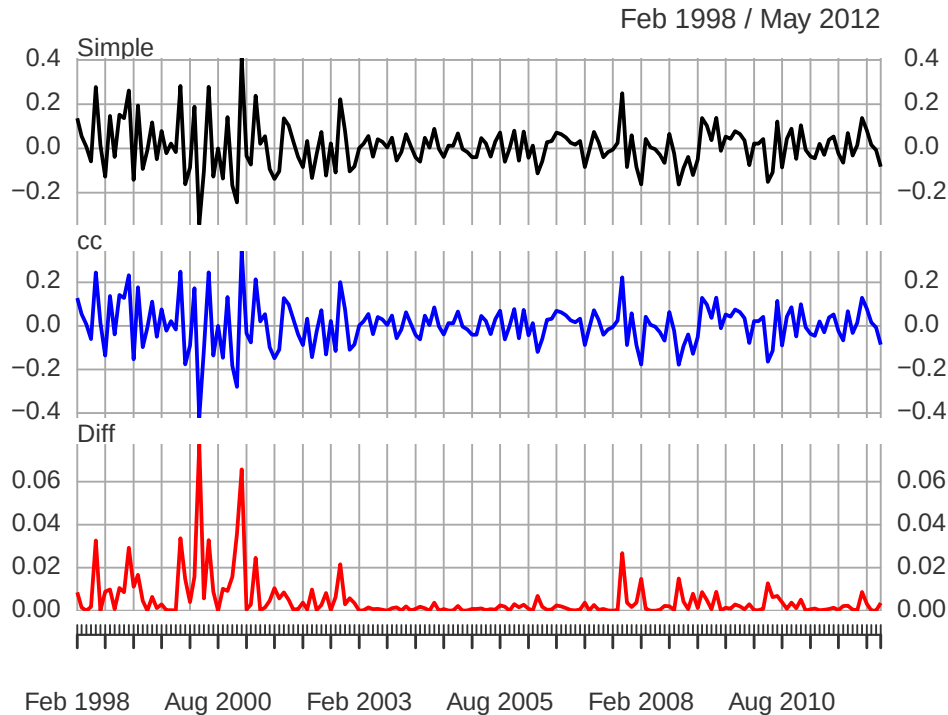


Figure 5.4: Monthly simple and cc returns on Microsoft. Top panel: simple returns; middle panel: cc returns; bottom panel: difference between simple and cc returns.

The top panel shows the simple returns, the middle panel shows the cc returns, and the bottom panel shows the difference between the simple and cc returns. Qualitatively, the simple and cc returns look almost identical. The main differences occur when the simple returns are large in absolute value (e.g., mid 2000, early 2001 and early 2008). In this case the difference between the simple and cc returns can be fairly large (e.g., as large as 0.08 in mid 2000). When the simple return is large and positive, the cc return is not quite as large; when the simple return is large and negative, the cc return is a larger negative number. ■

Example 5.5 (Plotting daily returns). Figure 5.5 shows the daily simple returns on Microsoft and the S&P 500 index created with:

```
plot(msftSp500DailyRetS, multi.panel=TRUE, yaxis.same=FALSE,
     main="", col="blue")
```

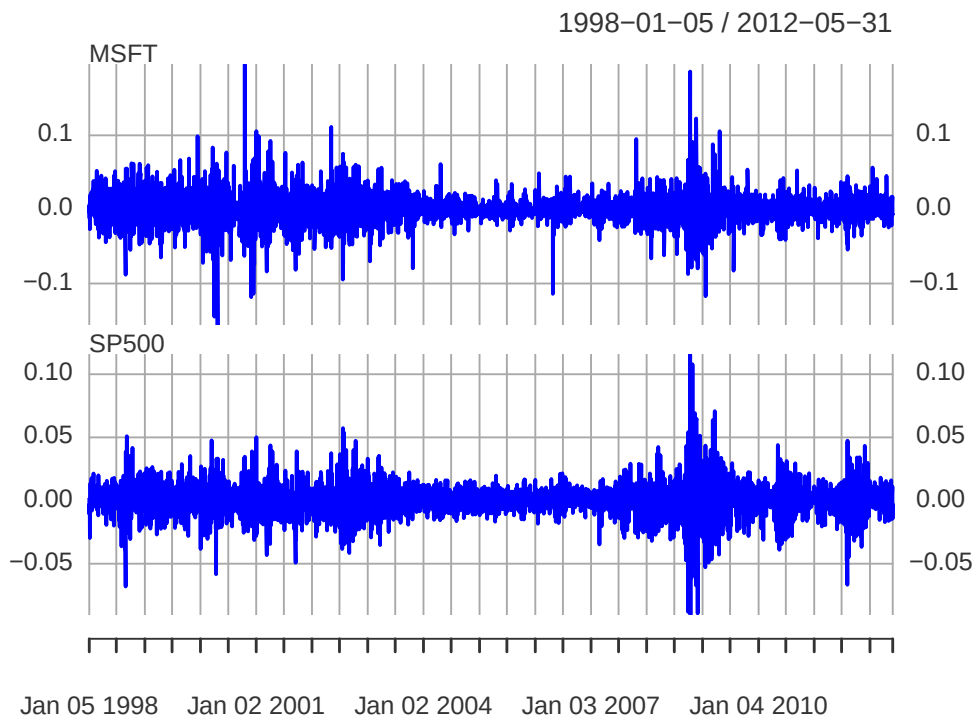


Figure 5.5: Daily returns on Microsoft and the S&P 500 index.

Compared with the monthly returns, the daily returns are closer to zero and the magnitude of the fluctuations (volatility) is smaller. However, the clustering of periods of high and low volatility is more pronounced in the daily returns. As with the monthly returns, the volatility of the daily returns on Microsoft is larger than the volatility of the S&P 500 returns. Also, the daily returns show some large positive and negative “spikes” that represent unusually large (in absolute value) daily movements (e.g., Microsoft up 20% in one day and down 15% on another day). The monthly returns do not exhibit such extreme movements relative to typical volatility. ■

5.1.3 Descriptive statistics for the distribution of returns

In this section, I consider graphical and numerical descriptive statistics for the unknown marginal pdf, $f_R(r)$, of returns. Recall, I assume that the observed sample $\{r_t\}_{t=1}^T$ is a realization from a covariance stationary and ergodic time series $\{R_t\}$ where each R_t is a continuous random variable with common pdf $f_R(r)$. The goal is to use $\{r_t\}_{t=1}^T$ to describe properties of $f_R(r)$.

I study returns and not prices because prices are random walk non-stationary. Sample descriptive statistics are only meaningful for covariance stationary and ergodic time series.

5.1.3.1 Histograms

Definition 5.2 (Histogram). A *histogram* of returns is a graphical summary used to describe the general shape of the unknown pdf $f_R(r)$. It is constructed as follows. Order returns from smallest to largest. Divide the range of observed values into N equally sized bins. Show the number or fraction of observations in each bin using a bar chart.

Example 5.6 (Histograms for the daily and monthly returns on Microsoft and the S&P 500 index). Figure 5.6 shows the histograms of the daily and monthly returns on Microsoft stock and the S&P 500 index created using the R function `hist()`:

```
par(mfrow=c(2,2))
hist(msftMonthlyRetS, main="", col="cornflowerblue")
hist(msftDailyRetS, main="", col="cornflowerblue")
hist(sp500MonthlyRetS, main="", col="cornflowerblue")
hist(sp500DailyRetS, main="", col="cornflowerblue")
```

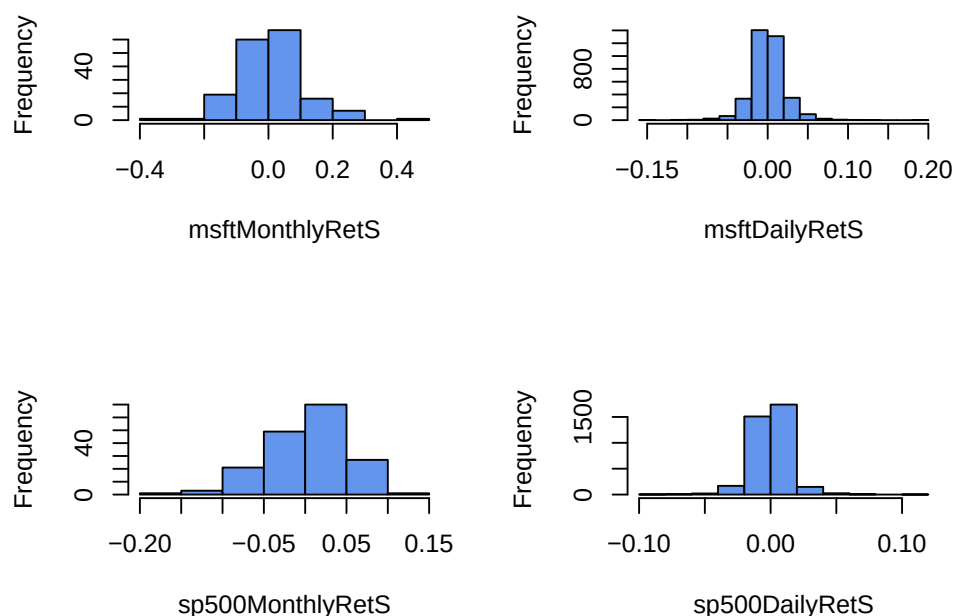


Figure 5.6: Histograms of monthly continuously compounded returns on Microsoft stock and S&P 500 index.

All histograms have a bell-shape like the normal distribution. The histograms of daily returns are centered around zero and those for the monthly return are centered around values slightly larger

than zero. The bulk of the daily (monthly) returns for Microsoft and the S&P 500 are between -5% and 5% (-20% and 20%) and -3% and 3% (-10% and 10%), respectively. The histogram for the S&P 500 monthly returns is slightly skewed left (long left tail) due to more large negative returns than large positive returns whereas the histograms for the other returns are roughly symmetric.

When comparing two or more return distributions, it is useful to use the same bins for each histogram. Figure 5.7 shows the histograms for Microsoft and S&P 500 returns using the same 15 bins, created with the R code:

```
msftHist = hist(msftMonthlyRetS, plot=FALSE, breaks=15)
par(mfrow=c(2,2))
hist(msftMonthlyRetS, main="", col="cornflowerblue")
hist(msftDailyRetS, main="", col="cornflowerblue",
     breaks=msftHist$breaks)
hist(sp500MonthlyRetS, main="", col="cornflowerblue",
     breaks=msftHist$breaks)
hist(sp500DailyRetS, main="", col="cornflowerblue",
     breaks=msftHist$breaks)
```

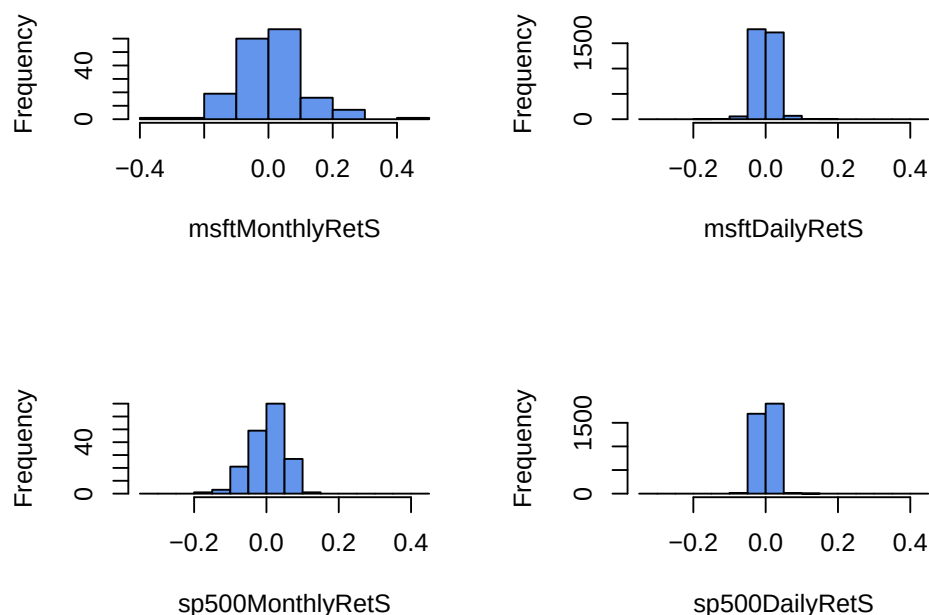


Figure 5.7: Histograms for Microsoft and S&P 500 returns using the same bins.

The common bins for each histogram are specified using the optional argument `breaks=msftHist$breaks`

in the call to `hist()`. Using the same bins for all histograms allows us to see more clearly that the distribution of monthly returns is more spread out than the distributions for daily returns, and that the distribution of S&P 500 returns is more tightly concentrated around zero than the distribution of Microsoft returns. ■

Example 5.7 (Are Microsoft returns normally distributed? A first look). The shape of the histogram for Microsoft returns suggests that a normal distribution might be a good candidate for the unknown distribution of Microsoft returns. To investigate this conjecture, I simulate random returns from a normal distribution with mean and standard deviation calibrated to the Microsoft daily and monthly returns using:

```
set.seed(123)
gwnDaily = rnorm(length(msftDailyRetS),
                 mean=mean(msftDailyRetS), sd=sd(msftDailyRetS))
gwnDaily = xts(gwnDaily, index(msftDailyRetS))
gwnMonthly = rnorm(length(msftMonthlyRetS),
                  mean=mean(msftMonthlyRetS), sd=sd(msftMonthlyRetS))
gwnMonthly = xts(gwnMonthly, index(msftMonthlyRetS))
```

Figure 5.8 shows the Microsoft monthly returns together with the simulated normal returns.

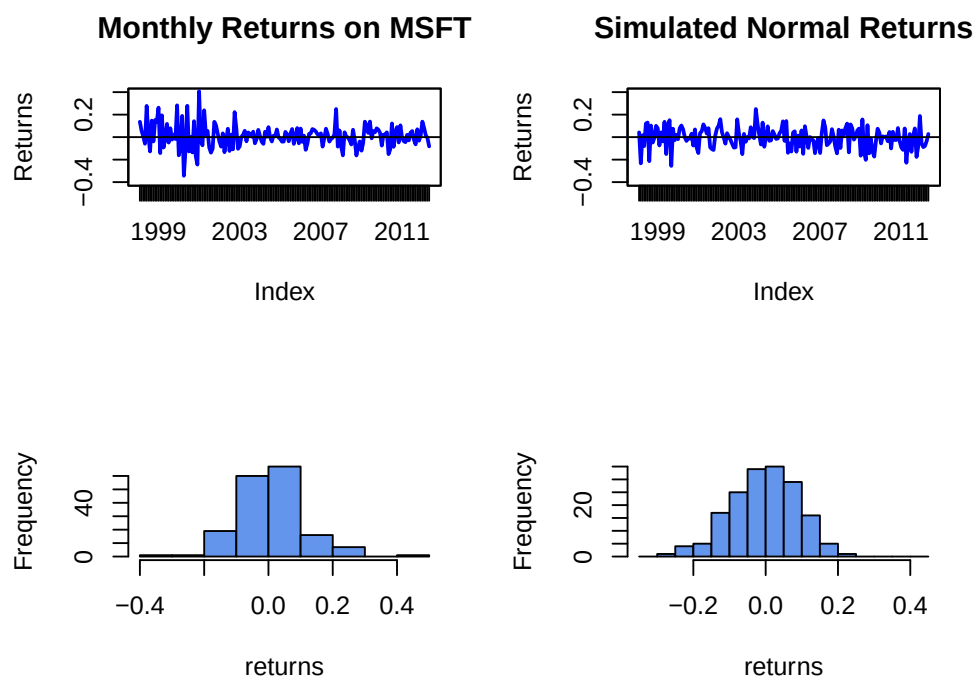


Figure 5.8: Comparison of Microsoft monthly returns with simulated normal returns with the same mean and standard deviation as the Microsoft returns.

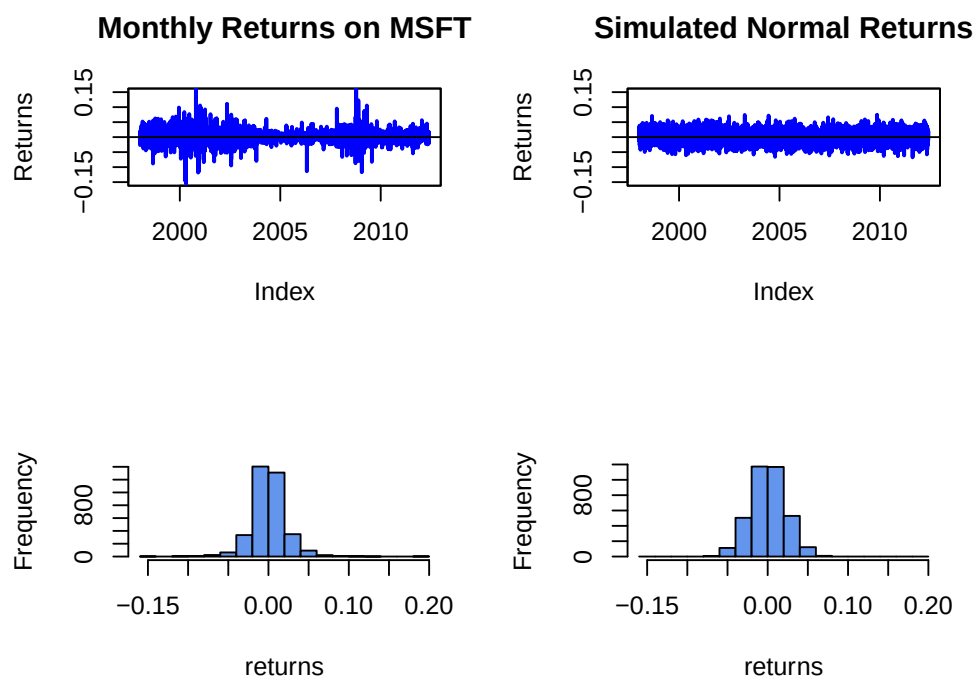


Figure 5.9: Comparison of Microsoft daily returns with simulated normal returns with the same mean and standard deviation as the Microsoft returns.

The simulated normal returns shares many of the same features as the Microsoft returns: both fluctuate randomly about zero. However, there are some important differences. In particular, the volatility of Microsoft returns appears to change over time (large before 2003, small between 2003 and 2008, and large again after 2008) whereas the simulated returns has constant volatility. Additionally, the distribution of Microsoft returns has fatter tails (more extreme large and small returns) than the simulated normal returns. Apart from these features, the simulated normal returns look remarkably like the Microsoft monthly returns.

Figure 5.9 shows the Microsoft daily returns together with the simulated normal returns. The daily returns look much less like GWN than the monthly returns. Here the constant volatility of simulated GWN does not match the volatility patterns of the Microsoft daily returns, and the tails of the histogram for the Microsoft returns are “fatter” than the tails of the GWN histogram. ■

5.1.3.2 Smoothed histogram

Histograms give a good visual representation of the data distribution. The shape of the histogram, however, depends on the number of bins used. With a small number of bins, the histogram often appears blocky and fine details of the distribution are not revealed. With a large number of bins,

the histogram might have many bins with very few observations. The `hist()` function in R smartly chooses the number of bins so that the resulting histogram typically looks good.

The main drawback of the histogram as a descriptive statistic for the underlying pdf of the data is that it is discontinuous. If it is believed that the underlying pdf is continuous, it is desirable to have a continuous graphical summary of the pdf. The *smoothed histogram* achieves this goal. Given a sample of data $\{x_t\}_{t=1}^T$ the R function `density()` computes a smoothed estimate of the underlying pdf $f_X(x)$ at each point x in the bins of the histogram using the formula:

$$\hat{f}_X(x) = \frac{1}{Tb} \sum_{t=1}^T k\left(\frac{x - x_t}{b}\right),$$

where $k(\cdot)$ is a continuous smoothing function (typically a standard normal distribution) and b is a bandwidth (or bin-width) parameter that determines the width of the bin around x in which the smoothing takes place. The resulting pdf estimate $\hat{f}_X(x)$ is a two-sided weighted average of the histogram values around x .³

Example 5.8 (Smoothed histogram for Microsoft monthly returns). Figure 5.10 shows the histogram of Microsoft returns overlaid with the smoothed histogram created using:

```
par(mfrow=c(1,1))
MSFT.density = density(msftMonthlyRetS)
hist(msftMonthlyRetS, main="", xlab="Microsoft Monthly Returns",
     col="cornflowerblue", probability=TRUE, ylim=c(0,5.5))
points(MSFT.density,type="l", col="orange", lwd=2)
```

³In this book, a “hat” on top of a symbol denotes that it is an estimate the underlying symbol.

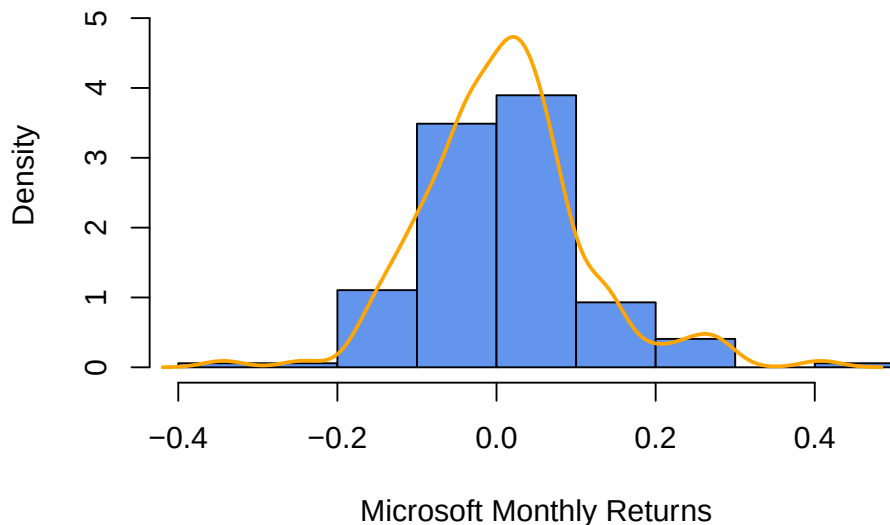


Figure 5.10: Histogram and smoothed density estimate for the monthly returns on Microsoft.

In Figure 5.10, the histogram is normalized (using the argument `probability=TRUE`), so that its total area is equal to one. The smoothed density estimate transforms the blocky shape of the histogram into a smooth continuous graph. ■

5.1.3.3 Empirical CDF

Recall, the CDF of a random variable X is the function $F_X(x) = \Pr(X \leq x)$. The empirical CDF $\hat{F}_X(x)$ is the sample analogue of $F_X(x)$.

Definition 5.3 (Empirical CDF). The *empirical CDF* of a data sample $\{x_t\}_{t=1}^T$ is the function that counts the fraction of observations less than or equal to x :

$$\begin{aligned}\hat{F}_X(x) &= \frac{1}{T}(\#x_i \leq x) \\ &= \frac{\text{number of } x_i \text{ values } \leq x}{\text{sample size}}\end{aligned}\tag{5.1}$$

Example 5.9 (Empirical CDF for monthly returns on Microsoft). Computing the empirical CDF of Microsoft monthly returns at a given point, say $r = 0$, is a straightforward calculation in R:

```
Fhat.0 = sum(msftMonthlyRetS <= 0)/nrow(msftMonthlyRetS)
Fhat.0
#> [1] 0.471
```

Here, the expression `msftMonthlyRetS <= 0` creates a logical vector the same length as `msftMonthlyRetS` that is equal to `TRUE` when returns are less than or equal to 0 and `FALSE` when returns are greater than zero. Then `Fhat.0` is equal to the fraction of `TRUE` values (returns less than or equal to zero) in the data, which gives Equation 5.1 evaluated at zero. The R function `ecdf()` can be used to compute and plot Equation 5.1 for all of the observed returns. For example, Figure 5.11 shows the empirical CDF for Microsoft monthly returns computed using:

```
plot(ecdf(coredata(msftMonthlyRetS)), main="", col="blue")
```

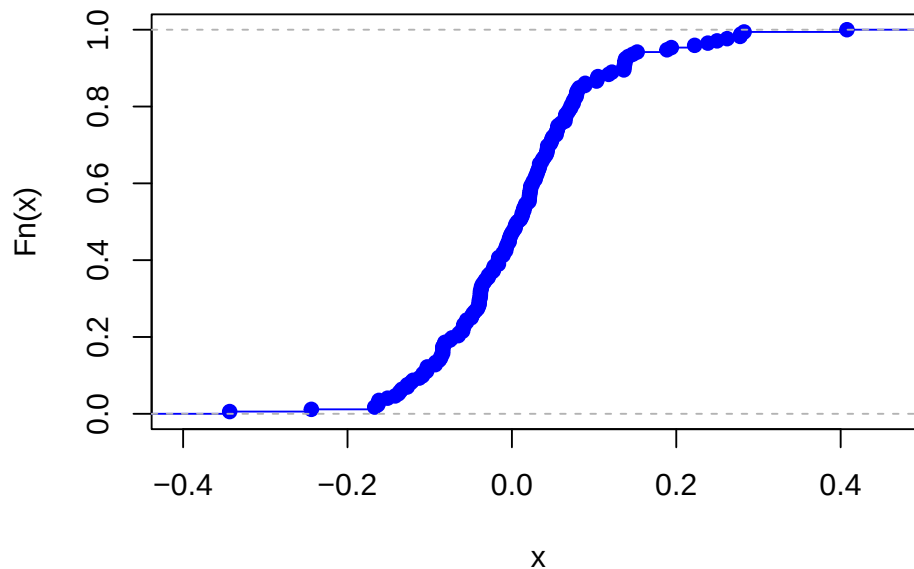


Figure 5.11: Empirical CDF of monthly returns on Microsoft

The “S” shape of the empirical CDF of Microsoft monthly returns is due to the symmetric shape of the histogram of monthly returns and resembles the CDF of a normal distribution. ■

5.1.3.4 Empirical quantiles/percentiles

Recall, for $\alpha \in (0, 1)$ the $\alpha \times 100\%$ quantile of the distribution of a continuous random variable X with CDF F_X is the point q_α^X such that $F_X(q_\alpha^X) = \Pr(X \leq q_\alpha^X) = \alpha$. The *empirical quantile* is defined similarly using the empirical CDF of the data.

Definition 5.4 (Empirical Quantile). The $\alpha \times 100\%$ *empirical quantile* (or $100 \times \alpha^{\text{th}}$ *percentile*) of a data sample $\{x_t\}_{t=1}^T$ is the data value $\hat{q}_\alpha$ such that $\alpha \cdot 100\%$ of the data are less than or equal to $\hat{q}_\alpha$.

Empirical quantiles can be easily determined by ordering the data from smallest to largest giving the ordered sample (also known as *order statistics*):

$$x_{(1)} < x_{(2)} < \cdots < x_{(T)}.$$

The empirical quantile $\hat{q}_\alpha$ is the order statistic closest to $\alpha \times T$.⁴

The *empirical quartiles* are the empirical quantiles for $\alpha = 0.25, 0.5$ and 0.75 , respectively. The second empirical quartile $\hat{q}_{.50}$ is called the *sample median* and is the data point such that half of the data is less than or equal to its value. The *interquartile range* (IQR) is the difference between the 3rd and 1st quartile:

$$\text{IQR} = \hat{q}_{.75} - \hat{q}_{.25},$$

and shows the size of the middle of the data distribution.

Example 5.10 (Empirical quantiles of the Microsoft and S&P 500 monthly returns). The R function `quantile()` computes empirical quantiles for a single data series. By default, `quantile()` returns the empirical quartiles as well as the minimum and maximum values:

```
apply(msftSp500MonthlyRetS, 2, quantile)
#>           MSFT      SP500
#> 0%    -0.34339 -0.16942
#> 25%   -0.04883 -0.02094
#> 50%    0.00854  0.00809
#> 75%    0.05698  0.03532
#> 100%   0.40765  0.10772
```

In the above code, the R function `apply()` is used to compute the empirical quantiles on each column of the `xts` object `msftSp500MonthlyRetS`. The syntax for `apply()` is:

⁴There is no unique way to determine the empirical quantile from a sample of size N for all values of α . The R function `quantile()` can compute empirical quantile using one of seven different definitions.

```
args(apply)
#> function (X, MARGIN, FUN, ..., simplify = TRUE)
#> NULL
```

where X is a multi-column data object, $MARGIN$ specifies looping over rows ($MARGIN=1$) or columns ($MARGIN=2$), FUN specifies the R function to be applied, and $\dots$ represents any optional arguments to be passed to FUN .

Here you see that the lower (upper) quartiles of the Microsoft monthly returns are smaller (larger) than the respective quartiles for the S&P 500 index.

To compute quantiles for a specified α use the `probs` argument. For example, to compute the 1% and 5% quantiles of the monthly returns use:

```
apply(msftSp500MonthlyRetS, 2, quantile, probs=c(0.01,0.05))
#>      MSFT    SP500
#> 1% -0.189 -0.1204
#> 5% -0.137 -0.0818
```

Historically, there is a 1% chance of losing more than 19% (12%) on Microsoft (S&P 500) in a given month, and a 5% chance of losing more than 14% (8%) on Microsoft (S&P 500) in a given month.

To compute the median and IQR values for monthly returns use the R functions `median()` and `IQR()`, respectively:

```
med = apply(msftSp500MonthlyRetS, 2, median)
iqr = apply(msftSp500MonthlyRetS, 2, IQR)
rbind(med, iqr)
#>      MSFT    SP500
#> med 0.00854 0.00809
#> iqr 0.10581 0.05626
```

The median returns are similar (about 0.8% per month) but the IQR for Microsoft is about twice as large as the IQR for the S&P 500 index. ■

5.1.3.5 Empirical/Historical VaR

Recall, the $\alpha \times 100\%$ value-at-risk (VaR) of an investment of $\$W$ is $VaR_\alpha = -\$W \times q_\alpha^R$, where q_α^R is the $\alpha \times 100\%$ quantile of the probability distribution of the investment simple rate of return R (recall, you multiply by -1 to represent loss as a positive number). The *empirical VaR* or *historical VaR* is defined similarly using the *empirical quantile*.

Definition 5.5 (Empirical/Historical VaR). The $\alpha \times 100\%$ *empirical VaR* (sometimes called *historical VaR* or *historical simulation VaR*) of an investment of $\$W$ is defined as:

$$\text{VaR}_\alpha^{HS} = -\$W \times \hat{q}_\alpha^R, \quad (5.2)$$

where $\hat{q}_\alpha^R$ is the empirical α quantile of a sample of simple returns $\{R_t\}_{t=1}^T$. For a sample of continuously compounded returns $\{r_t\}_{t=1}^T$ with empirical α quantile $\hat{q}_\alpha^r$,

$$\text{VaR}_\alpha^{HS} = -\$W \times (\exp(\hat{q}_\alpha^r) - 1). \quad (5.3)$$

Historical VaR is based on the distribution of the observed returns and not on any assumed distribution for returns (e.g., the normal distribution or the log-normal distribution).

Example 5.11 (Using empirical quantiles to compute historical VaR). Consider investing $W = \$100,000$ in Microsoft and the S&P 500 over a month. The 1% and 5% historical VaR values for these investments based on the historical samples of monthly returns are:

```
W = 100000
msftQuantiles = quantile(msftMonthlyRetS, probs=c(0.01, 0.05))
sp500Quantiles = quantile(sp500MonthlyRetS, probs=c(0.01, 0.05))
msftVaR = -W*msftQuantiles
sp500VaR = -W*sp500Quantiles
cbind(msftVaR, sp500VaR)
#>      msftVaR sp500VaR
#> 1%      18929      12040
#> 5%      13694       8184
```

Based on the empirical distribution of the monthly returns, a \$100,000 monthly investment in Microsoft will lose \$13,694 or more with 5% probability and will lose \$18,929 or more with 1% probability. The corresponding values for the S&P 500 are \$8,183 and \$12,039, respectively. The historical VaR values for the S&P 500 are considerably smaller than those for Microsoft. In this sense, investing in Microsoft is riskier than investing in the S&P 500 index. ■

5.1.4 QQ-plots

Often it is of interest to see if a given data sample could be viewed as a random sample from a specified probability distribution. One easy and effective way to do this is to compare the empirical quantiles of a data sample to the quantiles from a reference probability distribution. If the quantiles match up, then this provides strong evidence that the reference distribution is appropriate for describing the distribution of the observed data. If the quantiles do not match

up, then the observed differences between the empirical quantiles and the reference quantiles can be used to determine a more appropriate reference distribution. It is common to use the normal distribution as the reference distribution, but any distribution can in principle be used.

Definition 5.6 (Quantile-Quantile Plot). The *quantile-quantile plot* (QQ-plot) gives a graphical comparison of the empirical quantiles of a data sample to those from a specified reference distribution. The QQ-plot is an xy-plot with the reference distribution quantiles on the x-axis and the empirical quantiles on the y-axis. If the quantiles exactly match up then the QQ-plot is a straight line. If the quantiles do not match up, then the shape of the QQ-plot indicates which features of the data are not captured by the reference distribution.

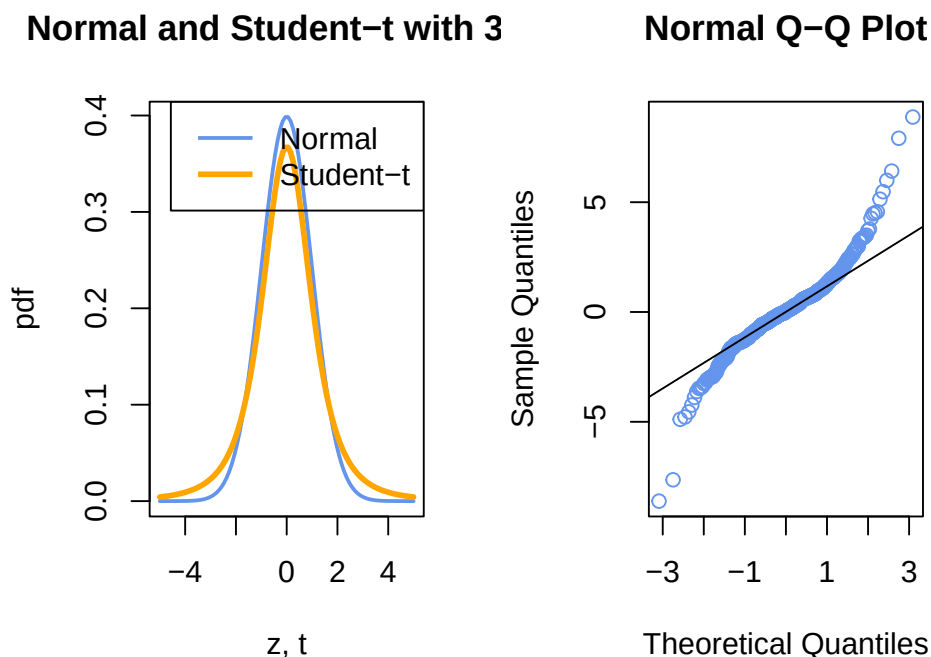


Figure 5.12: Normal QQ-Plot for Fat-tailed Data

Example 5.12 (QQ-plots for non-normal data). Figure 5.12 shows a normal QQ-plot for data simulated from a Student's t distribution with 3 degrees-of-freedom which has fatter tails than the normal distribution. Notice how the normal QQ-plot deviates from the straight line for both large and small quantiles of the normal distribution. This S-shape tells us that both extremely small and extremely large empirical quantiles (on the vertical axis) are larger (in absolute value) than the corresponding theoretical quantiles of the normal distribution.

Figure 5.13 shows a normal QQ-plot for data simulated from a distribution with positive skewness which has more density in the right tail than in the left tail. Here the normal QQ-plot has a convex

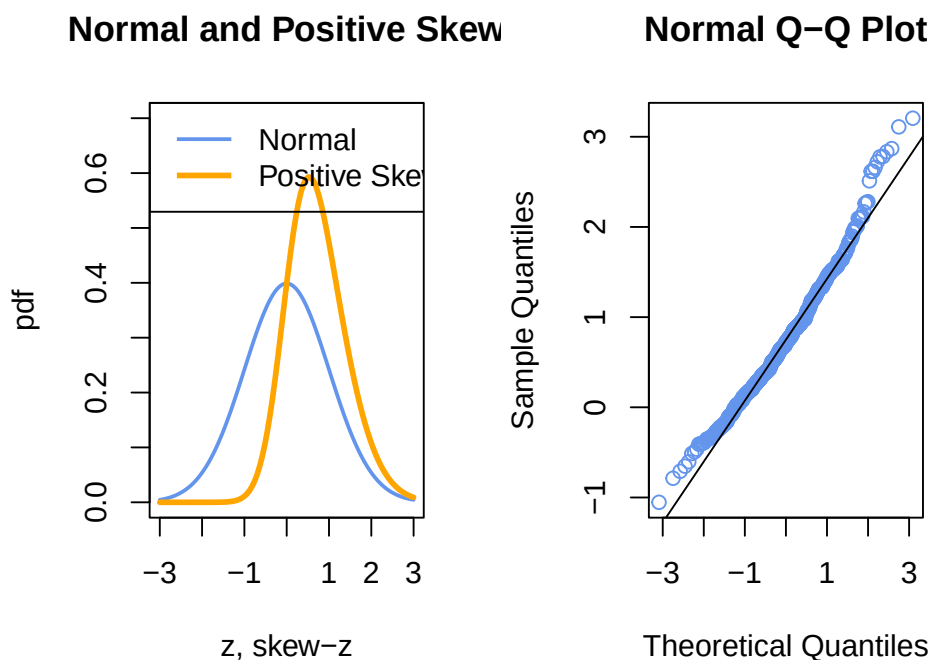


Figure 5.13: Normal QQ-Plot for Positively Skewed Data

shape relative to the straight line indicating that the lower (upper) empirical quantiles are closer to (farther from) zero than the corresponding theoretical quantiles of the normal distribution.

Figure 5.14 shows a normal QQ-plot for data simulated from a distribution with negative skewness which has more density in the left tail than in the right tail. Here the normal QQ-plot has a concave shape relative to the straight line indicating that the lower (upper) empirical quantiles are farther from (closer to) zero than the corresponding theoretical quantiles of the normal distribution. ■

...

Example 5.13 (Normal QQ-plots for GWN Microsoft and S&P 500 returns). The R function `qqnorm()` creates a QQ-plot for a data sample using the normal distribution as the reference distribution. Figure 5.15 shows normal QQ-plots for the simulated GWN data, Microsoft returns and S&P 500 returns.

The normal QQ-plot for the simulated GWN data is very close to a straight line, as it should be since the data are simulated from a normal distribution.⁵ The `qqline()` function draws a straight line through the points to help determine if the quantiles match up. The normal QQ-plots for the Microsoft and S&P 500 monthly returns are linear in the middle of the distribution but

⁵Even with data simulated from a normal distribution, the empirical quantiles in the extreme left and right tails can deviate a bit from the straight line due to estimation error. For this reason, small deviations from linearity in the tails should be ignored.

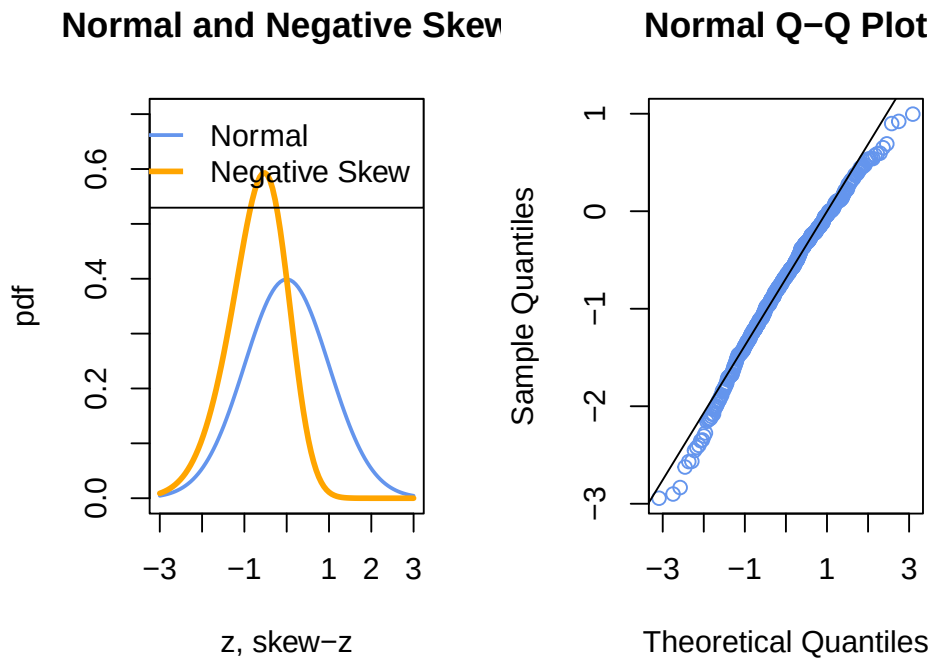


Figure 5.14: Normal Q-Q-Plot for Negatively Skewed Data



Figure 5.15: Normal QQ-plots for GWN, Microsoft returns and S&P 500 returns.

deviate from linearity in the tails of the distribution. In the normal QQ-plot for Microsoft monthly returns, the theoretical normal quantiles on the x-axis are too small in both the left and right tails because the points fall below the straight line in the left tail and fall above the straight line in the right tail. Hence, the normal distribution does not match the empirical distribution of Microsoft monthly returns in the extreme tails of the distribution. In other words, the Microsoft returns have *fatter tails* than the normal distribution. For the S&P 500 monthly returns, the theoretical normal quantiles are too small only for the left tail of the empirical distribution of returns (points fall below the straight line in the left tail only). This reflects the long left tail (negative skewness) of the empirical distribution of S&P 500 returns. The normal QQ-plots for the daily returns on Microsoft and the S&P 500 index both exhibit a pronounced tilted S-shape with extreme departures from linearity in the left and right tails of the distributions. The daily returns clearly have much fatter tails than the normal distribution. ■

Example 5.14 (Student's t QQ-plot for Microsoft returns). The function `qqPlot()` from the **car** package or the function `chart.QQPlot()` from the **PerformanceAnalytics** package can be used to create a QQ-plot against any reference distribution that has a corresponding quantile function implemented in R. For example, a QQ-plot for the Microsoft returns using a Student's t reference distribution with 5 degrees of freedom can be created using:

```
qqPlot(coredata(msftMonthlyRetS), distribution="t",  
       df=5, ylab="MSFT quantiles", envelope=FALSE, id=FALSE)
```

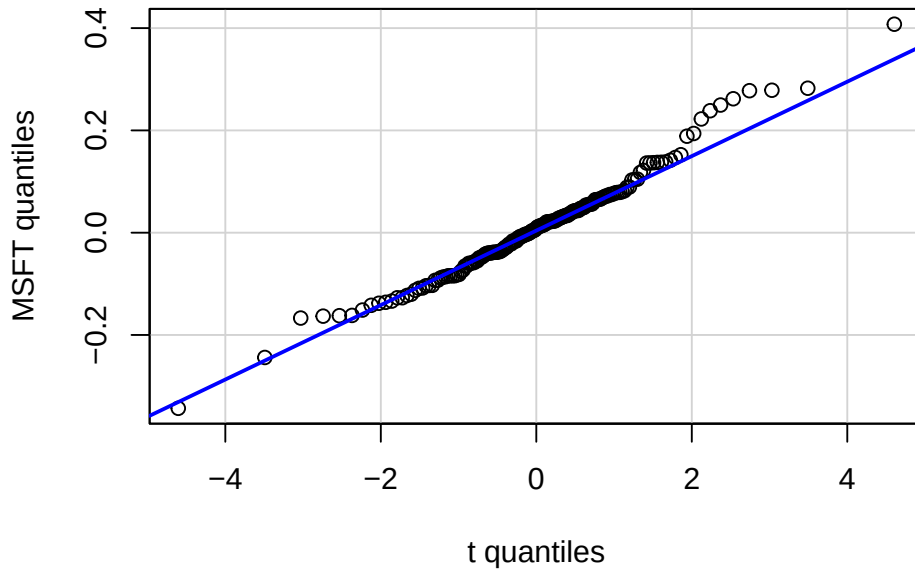


Figure 5.16: QQ-plot of Microsoft returns using Student's t distribution with 5 degrees of freedom as the reference distribution.

In the function `qqPlot()`, the argument `distribution= "t"` specifies that the quantiles are to be computed using the R function `qt()`.⁶ Figure 5.16 shows the resulting graph. Here, with a reference distribution with fatter tails than the normal distribution the QQ-plot for Microsoft returns is closer to a straight line. This indicates that the Student's t distribution with 5 degrees of freedom is a better reference distribution for Microsoft returns than the normal distribution. ■

5.1.5 Shape characteristics of the empirical distribution

Recall, for a random variable X the shape measures of center, spread, asymmetry and tail thickness of the pdf are:

$$\begin{aligned}
 \text{center} : \mu_X &= E[X], \\
 \text{spread} : \sigma_X^2 &= \text{var}(X) = E[(X - \mu_X)^2], \\
 \text{spread} : \sigma_X &= \sqrt{\text{var}(X)} \\
 \text{asymmetry} : \text{skew}_X &= E[(X - \mu_X)^3]/\sigma^3, \\
 \text{tail thickness} : \text{kurt}_X &= E[(X - \mu_X)^4]/\sigma^4.
 \end{aligned}$$

⁶The `coredata()` function is used to extract the data as a "numeric" object because `qqPlot()` does not work correctly with "zoo" or "xts" objects.

Definition 5.7 (Sample Statistics). The shape measures for the empirical distribution (e.g., as measured by the histogram) of a data sample $\{x_t\}_{t=1}^T$ are the *sample statistics*:

$$\hat{\mu}_x = \bar{x} = \frac{1}{T} \sum_{t=1}^T x_t, \quad (5.4)$$

$$\hat{\sigma}_x^2 = s_x^2 = \frac{1}{T-1} \sum_{t=1}^T (x_t - \bar{x})^2, \quad (5.5)$$

$$\hat{\sigma}_x = \sqrt{\hat{\sigma}_x^2}, \quad (5.6)$$

$$\widehat{\text{skew}}_x = \frac{\frac{1}{T-1} \sum_{t=1}^T (x_t - \bar{x})^3}{s_x^3}, \quad (5.7)$$

$$\widehat{\text{kurt}}_x = \frac{\frac{1}{T-1} \sum_{t=1}^T (x_t - \bar{x})^4}{s_x^4}. \quad (5.8)$$

The sample mean, $\hat{\mu}_x$, measures the center of the histogram; the sample standard deviation, $\hat{\sigma}_x$, measures the spread of the data about the mean in the same units as the data; the sample skewness, $\widehat{\text{skew}}_x$, measures the asymmetry of the histogram; the sample kurtosis, $\widehat{\text{kurt}}_x$, measures the tail-thickness of the histogram. The sample excess kurtosis, defined as the sample kurtosis minus 3:

$$\widehat{\text{ekurt}}_x = \widehat{\text{kurt}}_x - 3, \quad (5.9)$$

measures the tail thickness of the data sample relative to that of a normal distribution.⁷

Notice that the divisor in Equation 5.5 - Equation 5.8 is $T-1$ and not T . This is called a *degrees-of-freedom* correction. In computing the sample variance, skewness and kurtosis, one degree-of-freedom in the sample is used up in the computation of the sample mean so that there are effectively only $T-1$ observations available to compute the statistics.⁸

Example 5.15 (Sample shape statistics for the returns on Microsoft and S&P 500). The R functions for computing Equation 5.4 - Equation 5.8 are `mean()`, `var()` and `sd()`, respectively. There are no functions for computing Equation 5.7 and Equation 5.8 in base R. The functions `skewness()` and `kurtosis()` in the **PerformanceAnalytics** package compute the sample skewness Equation 5.7 and the sample excess kurtosis Equation 5.9, respectively.⁹ The sample statistics for the Microsoft and S&P 500 monthly returns are computed using:

⁷Values with hats, " $\hat{\cdot}$ ", denote sample estimates of the corresponding population quantity. For example, the sample mean $\hat{\mu}_x$ is the sample estimate of the population expected value μ_x .

⁸If there is only one observation in the sample then it is impossible to create a measure of spread in the sample. You need at least two observations to measure deviations from the sample average. Hence the effective sample size for computing the sample variance is $T-1$.

⁹Similar functions are available in the **moments** package.

```

statsMonthly = rbind(apply(msftSp500MonthlyRetS, 2, mean),
                        apply(msftSp500MonthlyRetS, 2, var),
                        apply(msftSp500MonthlyRetS, 2, sd),
                        apply(msftSp500MonthlyRetS, 2, skewness),
                        apply(msftSp500MonthlyRetS, 2, kurtosis))
rownames(statsMonthly) = c("Mean", "Variance", "Std Dev",
                           "Skewness", "Excess Kurtosis")
round(statsMonthly, digits=4)
#>           MSFT    SP500
#> Mean          0.0092  0.0028
#> Variance       0.0103  0.0023
#> Std Dev        0.1015  0.0478
#> Skewness       0.4853 -0.5631
#> Excess Kurtosis 1.9148  0.6545

```

The mean and standard deviation for Microsoft monthly returns are 0.9% and 10%, respectively. Annualized, these values are approximately 10.8% (0.009×12) and 34.6% ($0.10 \times \sqrt{12}$), respectively. The corresponding monthly and annualized values for S&P 500 returns are .3% and 4.7%, and 4.8% and 16.6%, respectively. Microsoft has a higher mean and volatility than S&P 500. The lower volatility for the S&P 500 reflects risk reduction due to *diversification* (holding many assets in a portfolio). The sample skewness for Microsoft, 0.485, is slightly positive and reflects the approximate symmetry in the histogram in Figure 5.6. The skewness for S&P 500, however, is moderately negative at -0.563 which reflects the somewhat long left tail of the histogram in Figure 5.6. The sample excess kurtosis values for Microsoft and S&P 500 are 1.915 and 0.654, respectively, and indicate that the tails of the histograms are slightly fatter than the tails of a normal distribution.

The sample statistics for the daily returns are:

```

statsDaily = rbind(apply(msftSp500DailyRetS, 2, mean),
                      apply(msftSp500DailyRetS, 2, var),
                      apply(msftSp500DailyRetS, 2, sd),
                      apply(msftSp500DailyRetS, 2, skewness),
                      apply(msftSp500DailyRetS, 2, kurtosis))
rownames(statsDaily) = rownames(statsMonthly)
round(statsDaily, digits=4)
#>           MSFT    SP500
#> Mean          0.0005  0.0002
#> Variance       0.0005  0.0002
#> Std Dev        0.0217  0.0135
#> Skewness       0.2506  0.0012

```

```
#> Excess Kurtosis 7.3915 6.9971
```

For a daily horizon, the sample mean for both series is zero (to three decimals). As with the monthly returns, the sample standard deviation for Microsoft, 2.2%, is about twice as big as the sample standard deviation for the S&P 500 index, 1.4%. Neither daily return series exhibits much skewness, but the excess kurtosis values are much larger than the corresponding monthly values indicating that the daily returns have much fatter tails and are more non-normally distributed than the monthly return series.

The daily simple returns are very close to the daily cc returns so the square-root-of-time rule (with 20 trading days within the month) can be applied to the sample mean and standard deviation of the simple returns:

```
20*statsDaily["Mean", ]
#>    MSFT    SP500
#> 0.00933 0.00345
sqrt(20)*statsDaily["Std Dev", ]
#>    MSFT    SP500
#> 0.0971 0.0603
```

Comparing these values to the corresponding monthly sample statistics shows that the square-root-of-time-rule applied to the daily mean and standard deviation gives results that are fairly close to the actual monthly mean and standard deviation. ■

5.1.6 Outliers

Figure 5.17 nicely illustrates the concept of an outlier in a data sample. All of the points are following a nice systematic relationship except one - the red dot “outlier”. Outliers can be thought of in two ways. First, an outlier can be the result of a data entry error. In this view, the outlier is not a valid observation and should be removed from the data sample. Second, an outlier can be a valid data point whose behavior is seemingly unlike the other data points. In this view, the outlier provides important information and should not be removed from the data sample. For financial market data, outliers are typically extremely large or small values that could be the result of a data entry error (e.g. price entered as 10 instead of 100) or a valid outcome associated with some unexpected bad or good news. Outliers are problematic for data analysis because they can greatly influence the value of certain sample statistics.

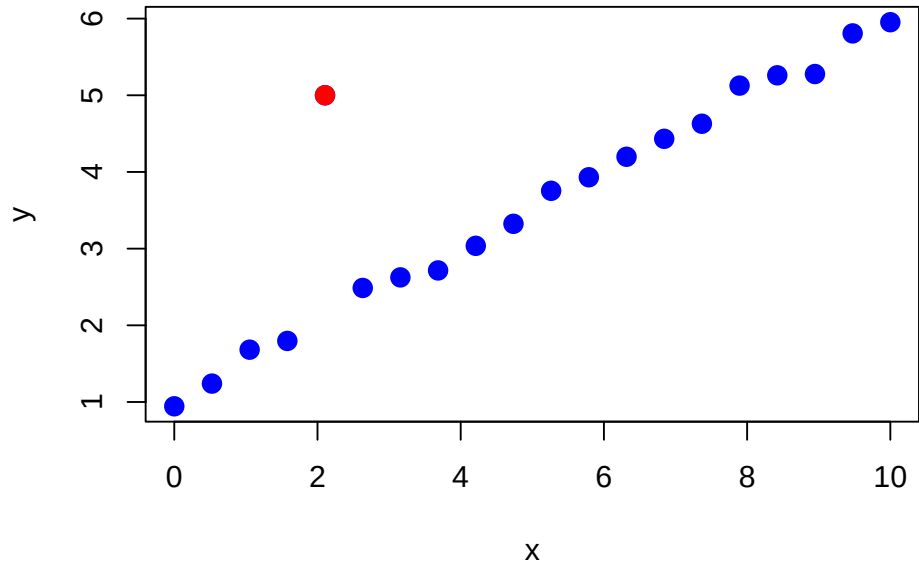


Figure 5.17: Illustration of an “outlier” in a data sample.

Table 5.1: Sample statistics for GWN with and without outlier.

Statistic	GWN	GWN with Outlier	% Δ
Mean	-0.0054	-0.0072	32%
Variance	0.0086	0.0101	16%
Std. Deviation	0.0929	0.1003	8%
Skewness	-0.2101	-1.0034	378%
Kurtosis	2.7635	7.1530	159%
Median	-0.0002	-0.0002	0%
IQR	0.1390	0.1390	0%

Example 5.16 (Effect of outliers on sample statistics). To illustrate the impact of outliers on sample statistics, the simulated monthly GWN data is polluted by a single large negative outlier:

```
gwnMonthlyOutlier = gwnMonthly
gwnMonthlyOutlier[20] = -sd(gwnMonthly)*6
```

Figure 5.18 shows the resulting data. Visually, the outlier is much smaller than a typical negative observation and creates a pronounced asymmetry in the histogram. Table 5.1 compares the sample statistics Equation 5.4 - Equation 5.8 of the unpolluted and polluted data. All of the

sample statistics are influenced by the single outlier with the skewness and kurtosis being influenced the most. The mean decreases by 32% and the variance and standard deviation increase by 16% and 8%, respectively. The skewness changes by 378% and the kurtosis by 159%. Table Table 5.1 also shows the median and the IQR, which are quantile-based statistics for the center and spread, respectively. Notice that these statistics are unaffected by the outlier.

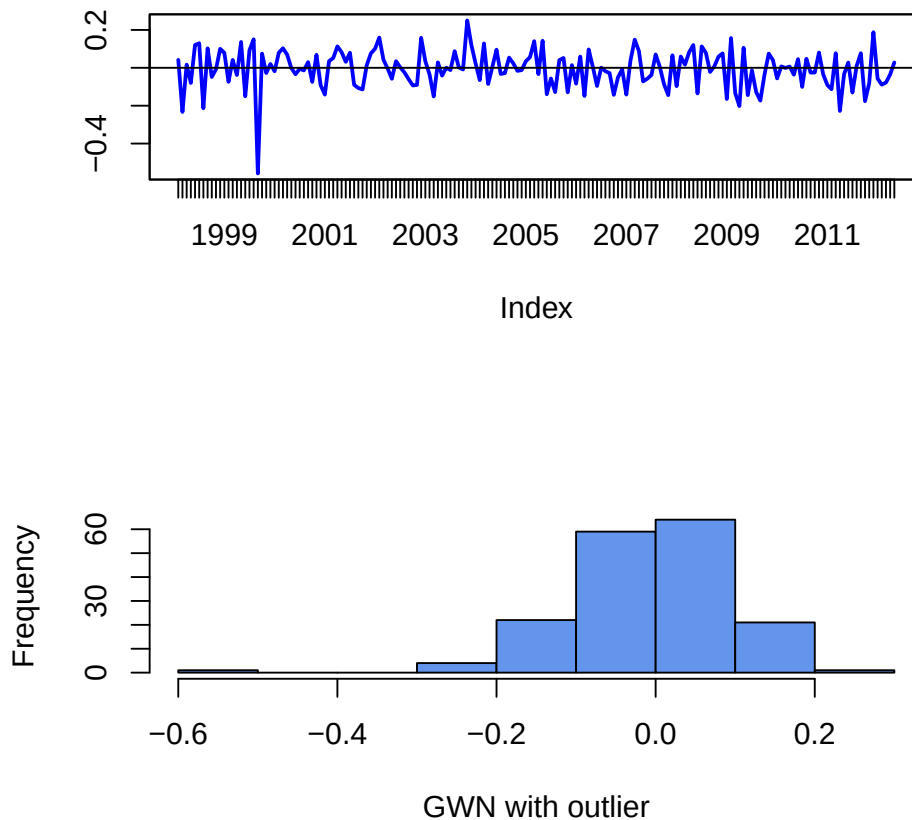


Figure 5.18: Monthly GWN data polluted by a single outlier.

■

The previous example shows that the common sample statistics Equation 5.4 - Equation 5.8 based on the sample average and deviations from the sample average can be greatly influenced by a single outlier, whereas quantile-based sample statistics are not. Sample statistics that are not greatly influenced by a single outlier are called (outlier) *robust statistics*.

5.1.6.1 Defining outliers

A commonly used rule-of-thumb defines an outlier as an observation that is beyond the sample mean plus or minus three times the sample standard deviation. The intuition for this rule comes from the fact that if $X \sim N(\mu, \sigma^2)$ then $\Pr(\mu - 3\sigma \leq X \leq \mu + 3\sigma) \approx 0.99$. While the intuition for this definition is clear, the practical implementation of the rule using the sample mean, $\hat{\mu}$, and sample standard deviation, $\hat{\sigma}$, is problematic because these statistics can be heavily influenced by the presence of one or more outliers (as illustrated in the previous sub-section). In particular, both $\hat{\mu}$ and $\hat{\sigma}$ become larger (in absolute value) when there are outliers and this may cause the rule-of-thumb rule to miss identifying an outlier or to identify a non-outlier as an outlier. That is, the rule-of-thumb outlier detection rule is not robust to outliers!

A natural fix to the above outlier detection rule-of-thumb is to use outlier robust statistics, such as the median and the IQR, instead of the mean and standard deviation. To this end, a commonly used definition of a moderate outlier in the right tail of the distribution (value above the median) is a data point x such that:

$$\hat{q}_{.75} + 1.5 \cdot \text{IQR} < x < \hat{q}_{.75} + 3 \cdot \text{IQR}. \quad (5.10)$$

To understand this rule, recall that the IQR is where the middle 50% of the data lies. If the data were normally distributed then $\hat{q}_{.75} \approx \hat{\mu} + 0.67 \times \hat{\sigma}$ and $\text{IQR} \approx 1.34 \times \hat{\sigma}$. Then Equation 5.10 is approximately:

$$\hat{\mu} + 2.67 \times \hat{\sigma} < x < \hat{\mu} + 4.67 \times \hat{\sigma}$$

Similarly, a moderate outlier in the left tail of the distribution (value below the median) is a data point such that,

$$\hat{q}_{.25} - 3 \cdot \text{IQR} < x < \hat{q}_{.25} - 1.5 \cdot \text{IQR}.$$

Extreme outliers are those observations x even further out in the tails of the distribution and are defined using:

$$\begin{aligned} x &> \hat{q}_{.75} + 3 \cdot \text{IQR}, \\ x &< \hat{q}_{.25} - 3 \cdot \text{IQR}. \end{aligned}$$

For example, with normally distributed data an extreme outlier would be an observation that is more than 4.67 standard deviations above the mean.

5.1.7 Box plots

A boxplot of a data series describes the distribution using outlier robust statistics. The basic boxplot is illustrated in Figure 5.19. The middle of the distribution is shown by a rectangular box whose top and bottom sides are approximately the first and third quartiles, $\hat{q}_{.25}$ and $\hat{q}_{.75}$,

respectively. The line roughly in the middle of the box is the sample median, $\hat{q}_{.5}$. The top whisker is a horizontal line that represents either the largest data value or $\hat{q}_{.75} + 1.5 \cdot IQR$, whichever is smaller. Similarly, the bottom whisker is located at either the smallest data value or $\hat{q}_{.25} - 1.5 \cdot IQR$, whichever is closer to $\hat{q}_{.25}$. Data values above the top whisker and below the bottom whisker are displayed as circles and represent extreme observations. If a data distribution is symmetric then the median is in the center of the box and the top and bottom whiskers are approximately the same distance from the box. Fat tailed distributions will have observations above and below the whiskers.

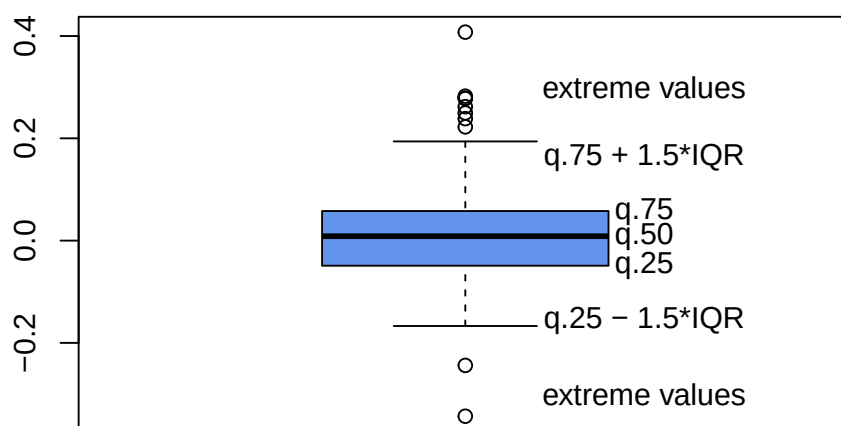


Figure 5.19: Boxplot of return distribution.

Example 5.17 (Boxplots of return distributions). Boxplots are a great tool for visualizing the return distribution of one or more assets. Figure 5.20 shows the boxplots of the monthly continuously compounded and simple returns on Microsoft and the S&P 500 index computed using:¹⁰

```
dataToPlot = merge(msftMonthlyRetS, msftMonthlyRetC,
                   sp500MonthlyRetS, sp500MonthlyRetC)
colnames(dataToPlot) = c("msftRetS", "msftRetC",
                        "sp500RetS", "sp500RetC")
boxplot(coredata(dataToPlot), main="Simple vs. CC Monthly Returns",
```

¹⁰The `coredata()` function is used to extract the data from the zoo object because `boxplot()` doesn't like "xts" or "zoo" objects.

```
col="cornflowerblue")
```

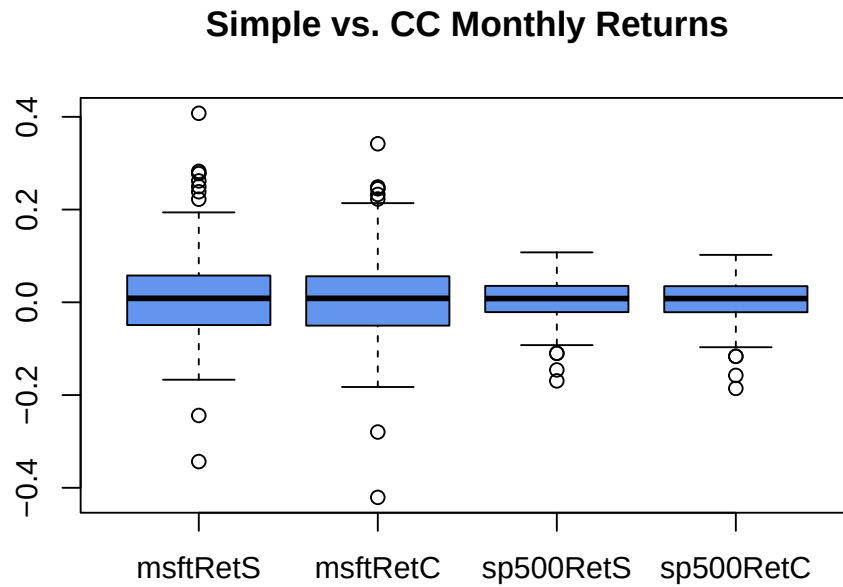


Figure 5.20: Boxplots of the continuously compounded and simple monthly returns on Microsoft stock and the S&P 500 index.

The Microsoft returns have both positive and negative outliers whereas the S&P 500 index returns only have negative outliers. The boxplots also clearly show that the distributions of the continuously compounded and simple returns are very similar except for the extreme tails of the distributions.

■

5.2 Time Series Descriptive Statistics

In this section, I consider graphical and numerical descriptive statistics for summarizing the linear time dependence in a time series.

5.2.1 Sample autocovariances and autocorrelations

For a covariance stationary time series process $\{X_t\}$, the autocovariances $\gamma_j = \text{cov}(X_t, X_{t-j})$ and autocorrelations $\rho_j = \text{cov}(X_t, X_{t-j}) / \sqrt{\text{var}(X_t)\text{var}(X_{t-j})} = \gamma_j / \sigma^2$ describe the linear time dependencies in the process. For a sample of data $\{x_t\}_{t=1}^T$, the linear time dependencies are captured by

the *sample autocovariances* and *sample autocorrelations*:

$$\hat{\gamma}_j = \frac{1}{T-1} \sum_{t=j+1}^T (x_t - \bar{x})(x_{t-j} - \bar{x}), \quad j = 1, 2, \dots, T - J + 1,$$

$$\hat{\rho}_j = \frac{\hat{\gamma}_j}{\hat{\sigma}^2}, \quad j = 1, 2, \dots, T - J + 1,$$

where $\hat{\sigma}^2$ is the sample variance Equation 5.5. The *sample autocorrelation function* (SACF) is a plot of $\hat{\rho}_j$ vs. j , and gives a graphical view of the linear time dependencies in the observed data.

Example 5.18 (SACF for the Microsoft and S&P 500 returns). The R function `acf()` computes and plots the sample autocorrelations $\hat{\rho}_j$ of a time series for a specified number of lags. For example, to compute $\hat{\rho}_j$ for $j = 1, \dots, 5$ for the monthly and daily returns on Microsoft use:

```
acf(coredata(msftMonthlyRetS), lag.max=5, plot=FALSE)
#>
#> Autocorrelations of series 'coredata(msftMonthlyRetS)', by lag
#>
#>      0      1      2      3      4      5
#> 1.000 -0.193 -0.114  0.193 -0.139 -0.023
acf(coredata(msftDailyRetS), lag.max=5, plot=FALSE)
#>
#> Autocorrelations of series 'coredata(msftDailyRetS)', by lag
#>
#>      0      1      2      3      4      5
#> 1.000 -0.044 -0.018 -0.010 -0.042  0.018
```

None of these sample autocorrelations are very big and indicate lack of linear time dependence in the monthly and daily returns.

Sample autocorrelations can also be visualized using the function `acf()`. Figure 5.21 shows the SACF for the monthly and daily returns on Microsoft and the S&P 500 created using:

```
par(mfrow=c(2,2))
acf(coredata(msftMonthlyRetS), main="msftMonthlyRetS", lwd=2)
acf(coredata(sp500MonthlyRetS), main="sp500MonthlyRetS", lwd=2)
acf(coredata(msftDailyRetS), main="msftDailyRetS", lwd=2)
acf(coredata(sp500DailyRetS), main="sp500DailyRetS", lwd=2)
```

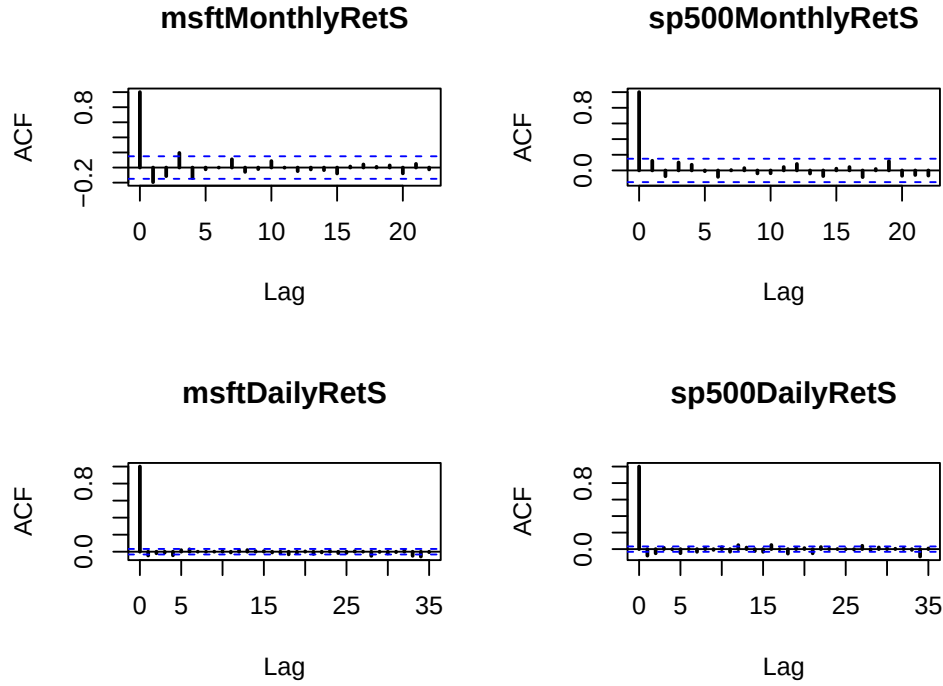


Figure 5.21: SACFs for the monthly and daily returns on Microsoft and the S&P 500 index.

None of the series show strong evidence for linear time dependence. The monthly returns have slightly larger sample autocorrelations at low lags than the daily returns. The dotted lines on the SACF plots are 5% critical values for testing the null hypothesis that the true autocorrelations ρ_j are equal to zero (see chapter Chapter 9). If $\hat{\rho}_j$ crosses the dotted line then the null hypothesis that $\rho_j = 0$ can be rejected at the 5% significance level. ■

5.2.2 Time dependence in volatility

The daily and monthly returns of Microsoft and the S&P 500 index do not exhibit much evidence for linear time dependence. Their returns are essentially uncorrelated over time. However, the lack of autocorrelation does not mean that returns are independent over time. There can be nonlinear time dependence. Indeed, there is strong evidence for a particular type of nonlinear time dependence in the daily returns of Microsoft and the S&P 500 index that is related to time varying volatility. This nonlinear time dependence, however, is less pronounced in the monthly returns.

Example 5.19 (GWN and lack of time independence). Let $y_t \sim \text{GWN}(0, \sigma^2)$. Then y_t is, by construction, independent over time. This implies that any function of y_t is also independent over time. To illustrate, Figure 5.22 shows y_t , y_t^2 and $|y_t|$ together with their SACFs where $y_t \sim \text{GWN}(0, 1)$ for $t = 1, \dots, 500$. Notice that all of the sample autocorrelations are essentially zero which confirms the lack of linear time dependence in y_t , y_t^2 and $|y_t|$.

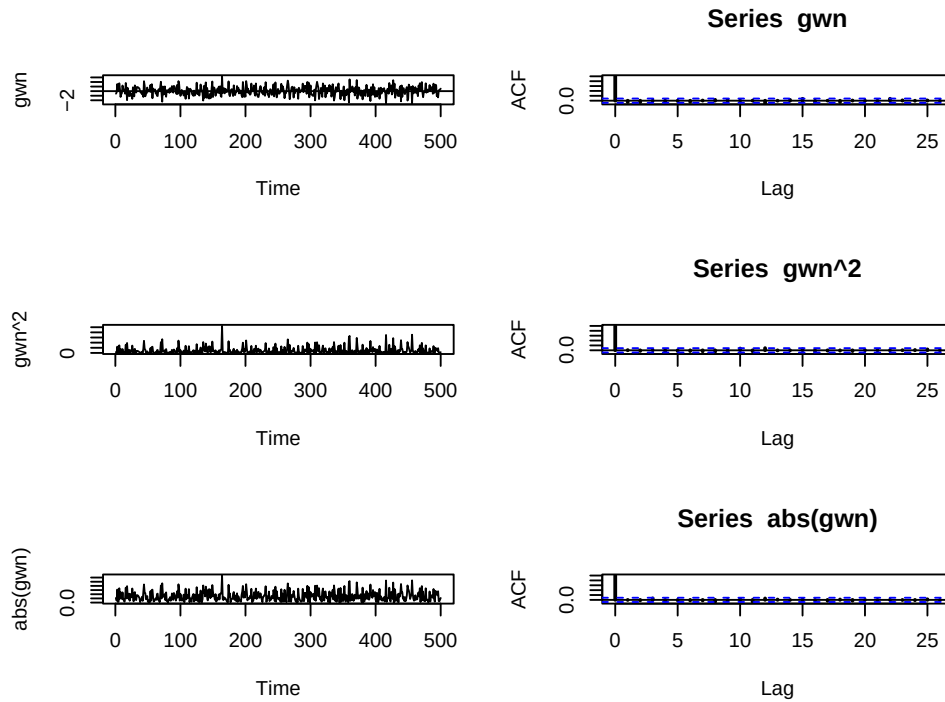


Figure 5.22: The left panel (top to bottom) shows y_t , y_t^2 and $|y_t|$ where y_t is similar to $\text{GWN}(0,1)$ for $t = 1, \dots, 500$. The right panel shows the corresponding SACFs.

■

5.2.2.1 Nonlinear time dependence in asset returns

Figure 5.5 and Figure 5.2 show that the daily and monthly returns on Microsoft and the S&P 500 index appear to exhibit periods of volatility clustering. That is, high periods of volatility tend to be followed by periods of high volatility and periods of low volatility appear to be followed by periods of low volatility. In other words, the volatility of the returns appears to exhibit some time dependence. This time dependence in volatility causes linear time dependence in the absolute value and squared value of returns.

Example 5.20 (Nonlinear time dependence in the S&P 500 returns). To illustrate nonlinear time dependence in asset returns, Figure 5.23 shows the daily returns, absolute returns and squared returns for the S&P 500 index. Notice how the absolute and squared returns are large (small) when the daily returns are more (less) volatile. Figure 5.24 shows the SACFs for the absolute and squared daily and monthly returns. There is clear evidence of time dependence in the daily absolute and squared returns while there is some evidence of time dependence in the monthly absolute and squared returns. Since the absolute and squared daily returns reflect the volatility of the daily returns, the SACFs show a strong positive linear time dependence in daily return volatility.

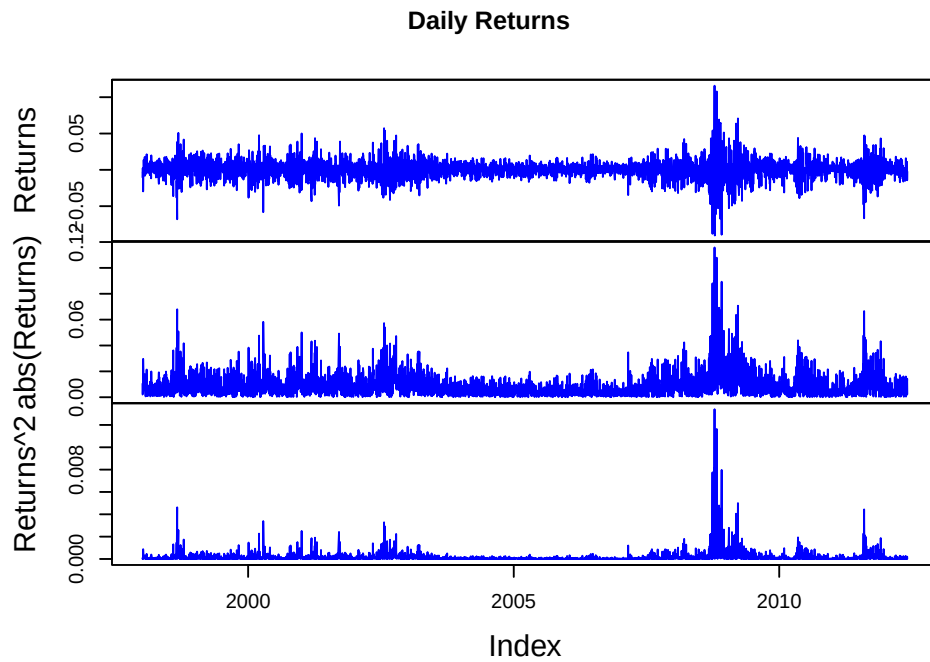


Figure 5.23: Daily returns, absolute returns, and squared returns on the S&P 500 index.

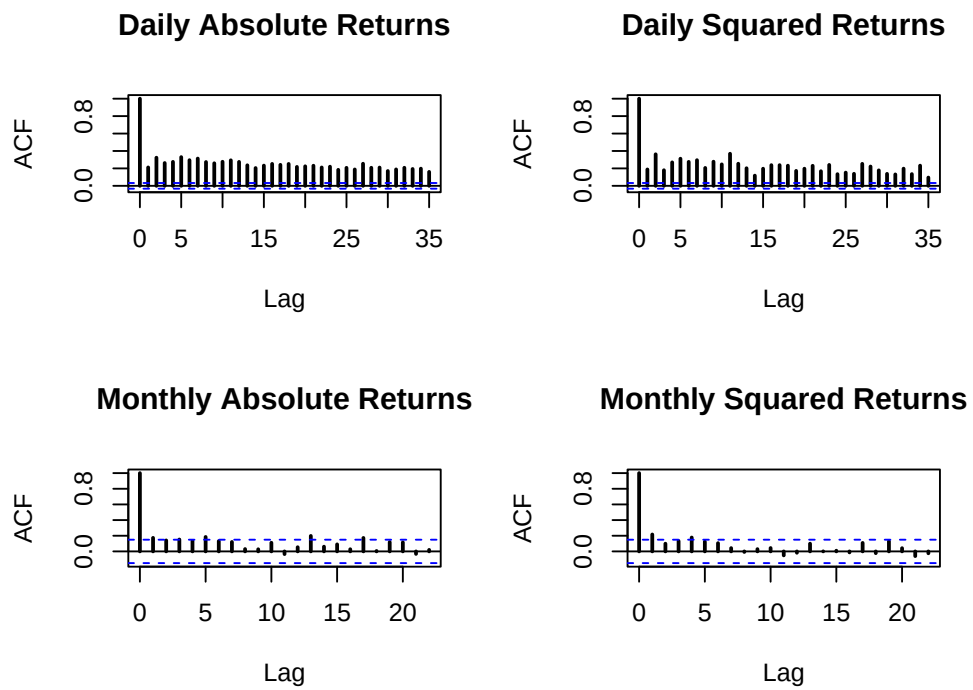


Figure 5.24: SACFs of daily and monthly absolute and squared returns on the S&P 500 index.

5.3 Bivariate Descriptive Statistics

In this section, I consider graphical and numerical descriptive statistics for summarizing two or more data series.

5.3.1 Scatterplots

The contemporaneous dependence properties between two data series $\{x_t\}_{t=1}^T$ and $\{y_t\}_{t=1}^T$ can be displayed graphically in a *scatterplot*, which is simply an xy-plot of the bivariate data.

Example 5.21 (Scatterplot of Microsoft and S&P 500 returns). Figure 5.25 shows the scatterplots between the Microsoft and S&P 500 monthly and daily returns created using:

```
par(mfrow=c(1,2))
plot(coredata(sp500MonthlyRetS),coredata(msftMonthlyRetS),
     main="Monthly returns", xlab="S&P500", ylab="MSFT", lwd=2,
     pch=16, cex=1.25, col="blue")
abline(v=mean(sp500MonthlyRetS))
abline(h=mean(msftMonthlyRetS))
plot(coredata(sp500DailyRetS),coredata(msftDailyRetS),
     main="Daily returns", xlab="S&P500", ylab="MSFT", lwd=2,
     pch=16, cex=1.25, col="blue")
abline(v=mean(sp500DailyRetS))
abline(h=mean(msftDailyRetS))
```

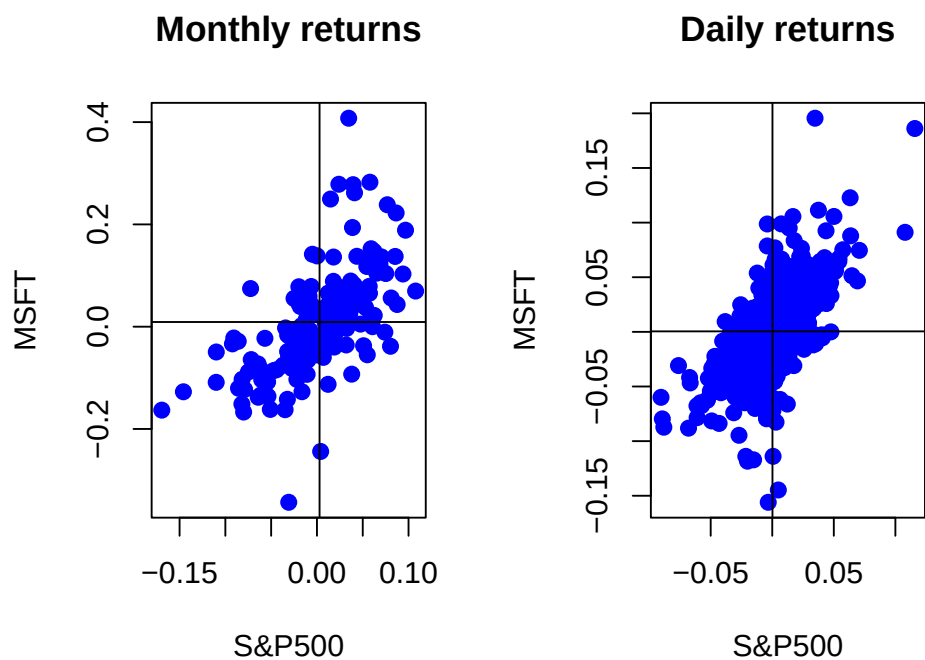


Figure 5.25: Scatterplot of Monthly returns on Microsoft and the S&P 500 index.

The S&P 500 returns are put on the x-axis and the Microsoft returns on the y-axis because the “market”, as proxied by the S&P 500, is often thought as an independent variable driving individual asset returns. The upward sloping orientation of the scatterplots indicate a positive linear dependence between Microsoft and S&P 500 returns at both the monthly and daily frequencies. ■

Example 5.22 (Pair-wise scatterplots for multiple series). For more than two data series, the R function `pairs()` plots all pair-wise scatterplots in a single plot. For example, to plot all pair-wise scatterplots for the GWN, Microsoft returns and S&P 500 returns use:

```
dataToPlot = merge(gwnMonthly,msftMonthlyRetS,sp500MonthlyRetS)
pairs(coredata(dataToPlot), col="blue", pch=16, cex=1.25, cex.axis=1.25)
```

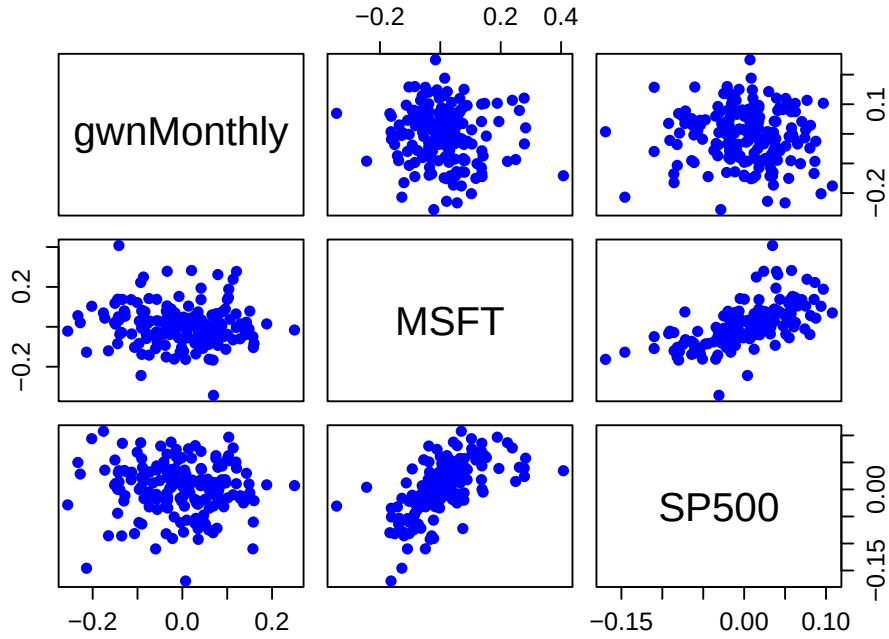



Figure 5.26: Pair-wise scatterplots between simulated GWN, Microsoft returns and S&P 500 returns.

The top row of Figure 5.26 shows the scatterplots between the pairs (MSFT, GWN) and (SP500, GWN), the second row shows the scatterplots between the pairs (GWN, MSFT) and (SP500, MSFT), the third row shows the scatterplots between the pairs (GWN, SP500) and (MSFT, SP500). The plots in the lower triangle are the same as the plots in the upper triangle except the axes are reversed. ■

5.3.2 Sample covariance and correlation

For two random variables X and Y , the direction of linear dependence is captured by the covariance, $\sigma_{XY} = E[(X - \mu_X)(Y - \mu_Y)]$, and the direction and strength of linear dependence is captured by the correlation, $\rho_{XY} = \sigma_{XY} / \sigma_X \sigma_Y$.

Definition 5.8 (Sample Covariance and Correlation). For two data series $\{x_t\}_{t=1}^T$ and $\{y_t\}_{t=1}^T$, the *sample covariance*,

$$\hat{\sigma}_{xy} = \frac{1}{T-1} \sum_{t=1}^T (x_t - \bar{x})(y_t - \bar{y}), \quad (5.11)$$

measures the direction of linear dependence, and the *sample correlation*,

$$\hat{\rho}_{xy} = \frac{\hat{\sigma}_{xy}}{\hat{\sigma}_x \hat{\sigma}_y}, \quad (5.12)$$

measures the direction and strength of linear dependence. In Equation 5.12, $\hat{\sigma}_x$ and $\hat{\sigma}_y$ are the sample standard deviations of $\{x_t\}_{t=1}^T$ and $\{y_t\}_{t=1}^T$, respectively, defined by Equation 5.6.

When more than two data series are being analyzed, it is often convenient to compute all pair-wise covariances and correlations at once using matrix algebra. Recall, for a vector of N random variables $\mathbf{X} = (X_1, \dots, X_N)'$ with mean vector $\mu = (\mu_1, \dots, \mu_N)'$ the $N \times N$ variance-covariance matrix is defined as:

$$\Sigma = \text{var}(\mathbf{X}) = E[(\mathbf{X} - \mu)(\mathbf{X} - \mu)'] = \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1N} \\ \sigma_{12} & \sigma_2^2 & \cdots & \sigma_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{1N} & \sigma_{2N} & \cdots & \sigma_N^2 \end{pmatrix}.$$

Definition 5.9 (Sample Covariance and Correlation Matrices). For N data series $\{\mathbf{x}_t\}_{t=1}^T$, where $x_t = (x_{1t}, \dots, x_{Nt})'$, the *sample covariance matrix* is computed using:

$$\hat{\Sigma} = \frac{1}{T-1} \sum_{t=1}^T (\mathbf{x}_t - \hat{\mu})(\mathbf{x}_t - \hat{\mu})' = \begin{pmatrix} \hat{\sigma}_1^2 & \hat{\sigma}_{12} & \cdots & \hat{\sigma}_{1N} \\ \hat{\sigma}_{12} & \hat{\sigma}_2^2 & \cdots & \hat{\sigma}_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{\sigma}_{1N} & \hat{\sigma}_{2N} & \cdots & \hat{\sigma}_N^2 \end{pmatrix}, \quad (5.13)$$

where $\hat{\mu}$ is the $N \times 1$ sample mean vector. Define the $N \times N$ diagonal matrix:

$$\hat{\mathbf{D}} = \begin{pmatrix} \hat{\sigma}_1 & 0 & \cdots & 0 \\ 0 & \hat{\sigma}_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \hat{\sigma}_N \end{pmatrix}. \quad (5.14)$$

Then the $N \times N$ *sample correlation matrix* $\hat{\mathbf{C}}$ is computed as:

$$\hat{\mathbf{C}} = \hat{\mathbf{D}}^{-1} \hat{\Sigma} \hat{\mathbf{D}}^{-1} = \begin{pmatrix} 1 & \hat{\rho}_{12} & \cdots & \hat{\rho}_{1N} \\ \hat{\rho}_{12} & 1 & \cdots & \hat{\rho}_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{\rho}_{1N} & \hat{\rho}_{2N} & \cdots & 1 \end{pmatrix}. \quad (5.15)$$

Example 5.23 (Sample covariance and correlation between Microsoft and S&P 500 returns). The scatterplots of Microsoft and S&P 500 returns in Figure 5.25 suggest positive linear relationships

in the data. You can confirm this by computing the sample covariance and correlation using the R functions `cov()` and `cor()`. For the monthly returns, you get:

```
cov(sp500MonthlyRetS, msftMonthlyRetS)
#>           MSFT
#> SP500  0.00298
cor(sp500MonthlyRetS, msftMonthlyRetS)
#>           MSFT
#> SP500  0.614
```

Indeed, the sample covariance is positive and the sample correlation shows a moderately strong linear relationship. For the daily returns you get:

```
cov(sp500DailyRetS, msftDailyRetS)
#>           MSFT
#> SP500  0.000196
cor(sp500DailyRetS, msftDailyRetS)
#>           MSFT
#> SP500  0.671
```

Here, the daily sample covariance is about twenty times smaller than the monthly covariance (recall the square-root-of time rule), but the daily sample correlation is similar to the monthly sample correlation.

When passed a matrix of data, the `cov()` and `cor()` functions can also be used to compute the sample covariance and correlation matrices $\hat{\Sigma}$ and $\hat{\mathbf{C}}$, respectively. For example,

```
cov(msftSp500MonthlyRetS)
#>           MSFT   SP500
#> MSFT  0.01030 0.00298
#> SP500 0.00298 0.00228
cor(msftSp500MonthlyRetS)
#>           MSFT SP500
#> MSFT  1.000 0.614
#> SP500 0.614 1.000
```

The function `cov2cor()` transforms a sample covariance matrix to a sample correlation matrix using Equation 5.15:

```
cov2cor(cov(msftSp500MonthlyRetS))
#>           MSFT SP500
```

```
#> MSFT  1.000 0.614
#> SP500 0.614 1.000
```

■

Example 5.24 (Visualizing correlation matrices). The R package **corrplot** contains functions for visualizing correlation matrices. This is particularly useful for summarizing the linear dependencies among many data series. For example, Figure 5.27 shows the correlation plot from the **corrplot** function `corrplot.mixed()` created using:

```
dataToPlot = merge(gwnMonthly, msftMonthlyRetS, sp500MonthlyRetS)
cor.mat = cor(dataToPlot)
corrplot.mixed(cor.mat, lower="number", upper="ellipse")
```

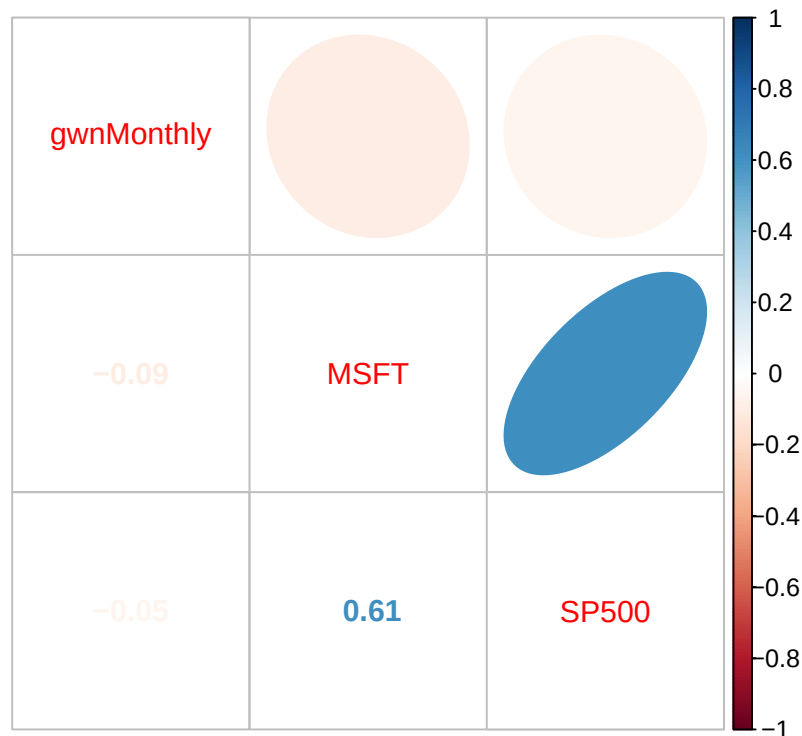


Figure 5.27: Correlation plot created with `corrplot()`.

The color scheme shows the magnitudes of the correlations (blue for positive and red for negative) and the orientation of the ellipses show the magnitude and direction of the linear associations. ■

5.3.3 Sample cross-lag covariances and correlations

The dynamic interactions between two observed time series $\{x_t\}_{t=1}^T$ and $\{y_t\}_{t=1}^T$ can be measured using the *sample cross-lag covariances* and *correlations* defined by:

$$\begin{aligned}\hat{\gamma}_{xy}^k &= \widehat{\text{cov}}(X_t, Y_{t-k}), \\ \hat{\rho}_{xy}^k &= \widehat{\text{corr}}(X_t, Y_{t-k}) = \frac{\hat{\gamma}_{xy}^k}{\sqrt{\hat{\sigma}_x^2 \hat{\sigma}_y^2}}.\end{aligned}$$

When more than two data series are being analyzed, all pairwise cross-lag sample covariances and sample correlations can be computed at once using matrix algebra. For a time series of N data series $\{\mathbf{x}_t\}_{t=1}^T$, where $\mathbf{x}_t = (x_{1t}, \dots, x_{Nt})'$, the sample lag k cross-lag covariance and correlation matrices are computed using:

$$\hat{\Gamma}_k = \frac{1}{T-1} \sum_{t=k+1}^T (\mathbf{x}_t - \hat{\mu})(\mathbf{x}_{t-k} - \hat{\mu})', \quad (5.16)$$

$$\hat{\mathbf{C}}_k = \hat{\mathbf{D}}^{-1} \hat{\Gamma}_k \hat{\mathbf{D}}^{-1}, \quad (5.17)$$

where $\hat{\mathbf{D}}$ is defined in Equation 5.14.

Example 5.25 (Sample cross-lag covariances and correlations between Microsoft and S&P 500 returns). Consider computing the cross-lag covariance and correlation matrices Equation 5.16 and Equation 5.17 for $k = 0, 1, \dots, 5$ between Microsoft and S&P 500 monthly returns. These matrices may be computed using the R function `acf()` as follows:

```
Ghat = acf(coredata(msftSp500MonthlyRetS), type="covariance",
           lag.max=5, plot=FALSE)
Chat = acf(coredata(msftSp500MonthlyRetS), type="correlation",
           lag.max=5, plot=FALSE)
names(Ghat)
#> [1] "acf"      "type"     "n.used"  "lag"     "series"  "snames"
```

Here, `Ghat` and `Chat` are objects of class "acf" for which there are print and plot methods. The `acf` components of `Ghat` and `Chat` are 3-dimensional arrays containing the cross lag matrices Equation 5.16 and Equation 5.17, respectively. For example, to extract $\hat{\mathbf{C}}_0$ and $\hat{\mathbf{C}}_1$ use:

```
# Chat0
Chat$acf[1,,]
#>      [,1] [,2]
#> [1,] 1.000 0.614
```

```
#> [2,] 0.614 1.000
# Chat1
Chat$acf[2,,]
#>      [,1]      [,2]
#> [1,] -0.1931 -0.0135
#> [2,] -0.0307  0.1232
```

The print method shows the sample autocorrelations of each variable as well as the pairwise cross-lag correlations:

```
Chat
#>
#> Autocorrelations of series 'coredata(msftSp500MonthlyRetS)', by lag
#>
#> , , MSFT
#>
#>  MSFT      SP500
#>  1.000 ( 0)  0.614 ( 0)
#> -0.193 ( 1) -0.031 (-1)
#> -0.114 ( 2) -0.070 (-2)
#>  0.193 ( 3)  0.137 (-3)
#> -0.139 ( 4) -0.030 (-4)
#> -0.023 ( 5)  0.032 (-5)
#>
#> , , SP500
#>
#>  MSFT      SP500
#>  0.614 ( 0)  1.000 ( 0)
#> -0.014 ( 1)  0.123 ( 1)
#> -0.043 ( 2) -0.074 ( 2)
#>  0.112 ( 3)  0.101 ( 3)
#> -0.105 ( 4)  0.073 ( 4)
#> -0.055 ( 5) -0.010 ( 5)
```

These values can also be visualized using the plot method:

```
plot(Chat, lwd=2)
```

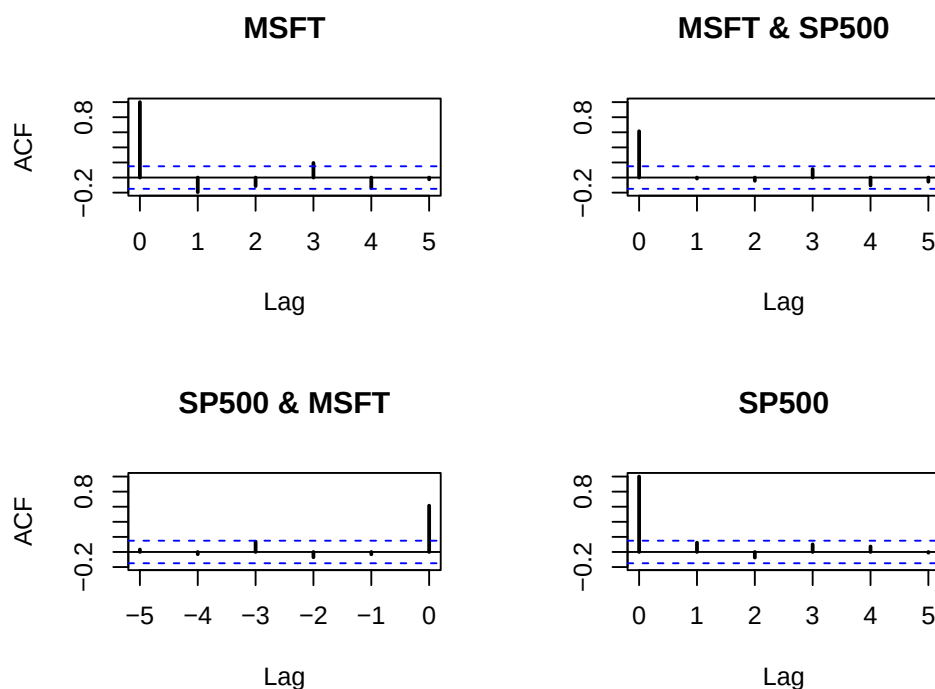


Figure 5.28: Sample cross-lag correlations between Microsoft and S&P 500 returns.

Figure 5.28 shows the resulting four-panel plot. The top-left and bottom-right (diagonal) panels give the SACFs for Microsoft and S&P 500 returns. The top-right plot gives the cross-lag correlations $\hat{\rho}_{msft,sp500}^k$ for $k = 0, 1, \dots, 5$ and the bottom-left panel gives the cross-lag correlations $\hat{\rho}_{sp500,msft}^k$ for $k = 0, 1, \dots, 5$. The plots show no evidence of any dynamic feedback between the return series.

■

5.4 Rolling descriptive statistics

The assumption that asset returns are covariance stationary implies that the asset return means (μ_i), variances (σ_i^2), covariances (σ_{ij}), and correlations (ρ_{ij}) are constant over time for all assets i and j . Essentially, this means that the economic environment remains constant over time. Clearly, this is a strong assumption and you have seen evidence in the graphical descriptive statistics that this assumption might be suspect. For example, you have seen that the volatility of asset returns appears to change over time. In this section, I introduce *rolling descriptive statistics* to further investigate the appropriateness of the covariance stationary assumption of asset returns.

A commonly used technique to investigate whether certain asset return characteristics are constant over time is to compute sample descriptive statistics over rolling sub-sample windows of fixed length. To illustrate, suppose one is interested in determining if the expected return parameter of an asset

return μ is constant over time. Assume returns are observed over a sample of size T . An informal way to do this is to compute estimates of μ over rolling windows of length $n < T$:

$$\hat{\mu}_t(n) = \frac{1}{n} \sum_{k=0}^{n-1} r_{t-k} = \frac{1}{n} (r_t + r_{t-1} + \dots + r_{t-n+1}), \quad (5.18)$$

for $t = n, \dots, T$. Here $\hat{\mu}_n(n)$ is the sample mean of the returns $\{r_t\}_{t=1}^n$ over the first sub-sample window of size n . Then $\hat{\mu}_{n+1}(n)$ is the sample mean of the returns $\{r_t\}_{t=2}^{n+1}$ over the second sub-sample window of size n . Continuing in this fashion I roll the sample window forward one observation at a time and compute $\hat{\mu}_t(n)$ until I arrive at the last sub-sample window of size n . The end result is $T - n + 1$ rolling estimates $\{\hat{\mu}_n(n), \hat{\mu}_{n+1}(n), \dots, \hat{\mu}_T(n)\}$.

If returns are covariance stationary then μ is constant over time and you would expect that $\hat{\mu}_n(n) \approx \hat{\mu}_{n+1}(n) \approx \dots \approx \hat{\mu}_T(n)$. That is, the estimates of μ over each rolling window should be similar. I would not expect them to be exactly equal due to sampling uncertainty but they should not be drastically different. In contrast, if returns are not covariance stationary due to time variation in μ then you would expect to see meaningful time variation in the rolling estimates $\{\hat{\mu}_n(n), \hat{\mu}_{n+1}(n), \dots, \hat{\mu}_T(n)\}$.

A natural way to see the time variation in $\{\hat{\mu}_t(n)\}_{t=n}^T$ is to make a time series plot of these rolling estimates as illustrated in the next example.

Example 5.26 (Rolling mean estimates for Microsoft S&P 500 Index and simulated GWN). Consider computing rolling estimates of μ for Microsoft, Starbucks and the S&P 500 index returns. You can easily do this using the **zoo** function `rollapply()`, which applies a user-specified function to a time series over rolling windows of fixed size. The arguments to `rollapply()` are:

```
args(zoo:::rollapply.zoo)
#> function (data, width, FUN, ..., by = 1, by.column = TRUE, fill = if (na.pad) NA,
#>      na.pad = FALSE, partial = FALSE, align = c("center", "left",
#>      "right"), coredata = TRUE)
#> NULL
```

where `data` is either a "zoo" or "xts" object, `width` is the window width, `FUN` is the R function to be applied over the rolling windows, `...` are any optional arguments to be supplied to `FUN`, `by` specifies how many observations to increment the rolling windows, `by.column` specifies if `FUN` is to be applied to each column of data or to be applied to all columns of data, and `align` specifies the location of the time index in the window. The function returns an "xts/zoo" object containing the rolling estimates.

For example, to compute 24-month rolling estimates of μ for Microsoft, S&P 500 index, and simulated GWN data (that matches the mean and volatility of Microsoft returns), incremented by one

month with the time index aligned to the end of the window use:

```
roll.data = merge(msftMonthlyRetS,sp500MonthlyRetS,gwnMonthly)
colnames(roll.data) = c("MSFT", "SP500", "GWN")
roll.muhat = rollapply(roll.data, width=24, by=1, by.column=TRUE,
                       FUN=mean, align="right")

class(roll.muhat)
#> [1] "xts" "zoo"
```

The object `roll.muhat` is an “xts” object with three columns giving the rolling mean estimates for the GWN data, Microsoft, and the S&P 500 index. The first 23 months of data are NA as you need at least 24 months of data to compute the first rolling estimates:

```
head(roll.muhat, n=3)
#>           MSFT SP500 GWN
#> Feb 1998   NA    NA  NA
#> Mar 1998   NA    NA  NA
#> Apr 1998   NA    NA  NA
```

The first rolling estimates are for the 24-months just prior to January, 2000 and the last rolling estimates are for the 24-months just prior to May 2012.

```
head(na.omit(roll.muhat), n=3)
#>           MSFT SP500 GWN
#> Jan 2000 0.0487 0.0161 -1.68e-03
#> Feb 2000 0.0394 0.0124 -9.85e-05
#> Mar 2000 0.0450 0.0143 1.39e-02
tail(na.omit(roll.muhat), n=3)
#>           MSFT SP500 GWN
#> Mar 2012 0.00869 0.00894 -0.0307
#> Apr 2012 0.00661 0.00801 -0.0324
#> May 2012 0.00948 0.00882 -0.0298
```

The rolling means are shown in Figure 5.29 created using:

```
plot(roll.muhat, main="", multi.panel=FALSE, lwd=2,
     col=c("black", "red", "green"), lty=c("solid", "solid", "solid"),
     major.ticks="years", grid.ticks.on="years",
     legend.loc = "topright")
```

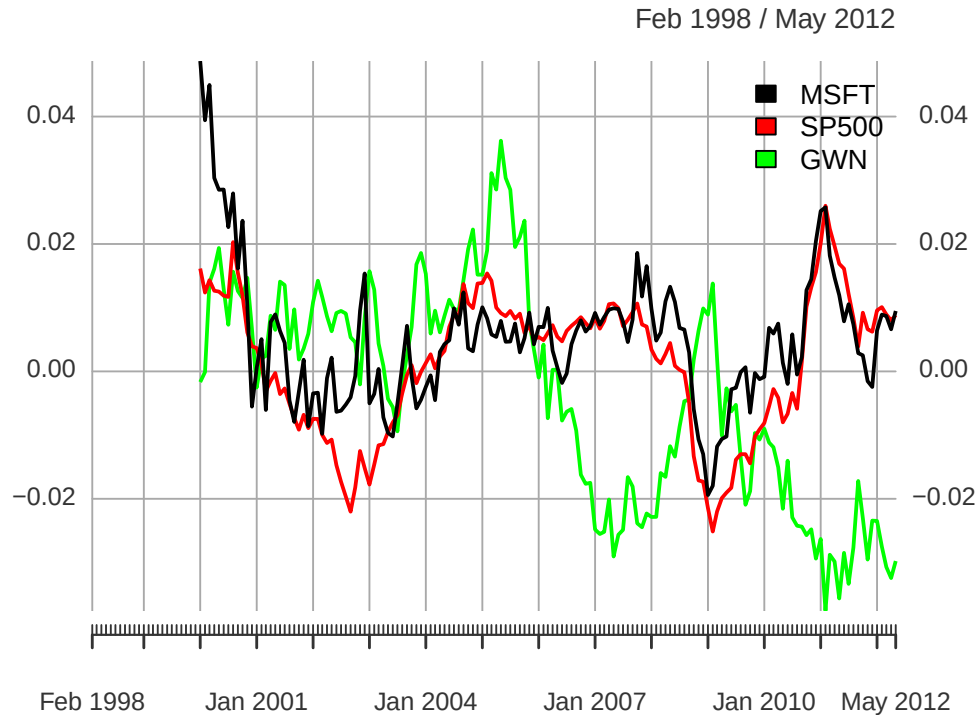


Figure 5.29: 24-month rolling estimates of μ

All of the rolling estimates show time variation over the sample, including the simulated GWN returns which have a constant mean by construction. The ranges of the rolling estimates are:

```
ans = apply(na.omit(roll.muhat),2,range)
rownames(ans) = c("Min", "Max")
ans
#>      MSFT   SP500   GWN
#> Min -0.0194 -0.0251 -0.0376
#> Max  0.0487  0.0260  0.0362
```

Interestingly, the rolling estimates for Microsoft and the S&P 500 index exhibit a similar pattern of time variation. The estimates start positive at the end of the dot-com boom, decrease from 2000 to mid 2002 (Microsoft and the S&P 500 index become negative), increase from 2003 to 2006 during the booming economy, fall sharply during the financial crisis period 2007 to 2009, and increase from 2009 to 2011, and decrease afterward. The Microsoft and S&P 500 estimates follow each other very closely due to their moderately strong positive correlation of 0.614. Somewhat surprisingly, the rolling means for the simulated GWN data also show substantial time variation over the sample. Since the true mean of the simulated GWN data is constant, the variation observed in the rolling means represents sampling uncertainty due to the small sample sizes of the rolling means. The magnitude of the variation in the rolling mean estimates for the GWN data should make us pause

in concluding there is substantial change in the mean estimates of Microsoft (and possibly the S&P 500). However, the fact that the rolling estimates for Microsoft and the S&P 500 index behave as expected during the booms and busts strongly suggests that the underlying mean values for these assets are not constant over the full sample. ■

To investigate time variation in return volatility, I compute rolling estimates of σ^2 and σ over fixed windows of size $n < T$:

$$\hat{\sigma}_t^2(n) = \frac{1}{n-1} \sum_{k=0}^{n-1} (r_{t-k} - \hat{\mu}_t(n))^2, \quad (5.19)$$

$$\hat{\sigma}_t(n) = \sqrt{\hat{\sigma}_t^2(n)}. \quad (5.20)$$

If return volatility is constant over time, then you would expect that $\hat{\sigma}_n(n) \approx \hat{\sigma}_{n+1}(n) \approx \dots \approx \hat{\sigma}_T(n)$. That is, the estimates of σ over each rolling window should be similar. In contrast, if volatility is not constant over time then you would expect to see meaningful time variation in the rolling estimates $\{\hat{\sigma}_n(n), \hat{\sigma}_{n+1}(n), \dots, \hat{\sigma}_T(n)\}$.

Example 5.27 (Rolling volatility estimates for Microsoft Starbucks and the S&P 500 Index). To compute and plot 24-month rolling estimates of σ for Microsoft, S&P 500 index, and GWN returns incremented by one month with the time index aligned to the beginning of the window use:

```
roll.sigmahat = rollapply(roll.data,width=24, by=1, by.column=TRUE,
                          FUN=sd, align="right")
```

The rolling estimates are illustrated in Figure 5.30.

The rolling volatilities for Microsoft and the S&P 500 index show a similar pattern of time variation over the sample but the Microsoft volatilities are considerably more variable:

```
ans = apply(na.omit(roll.sigmahat),2,range)
rownames(ans) = c("Min", "Max")
ans
#>      MSFT  SP500    GWN
#> Min 0.0402 0.0186 0.0716
#> Max 0.1863 0.0706 0.1192
```

The rolling volatilities for Microsoft peak at about 18% per month at the end of the dot-com boom, fall sharply to about 4% per month at the beginning of 2005, then level off until the beginning of the financial crisis, and increase during the financial crisis to about 10% per month and fall slightly afterward. The S&P 500 volatilities have a similar pattern as the Microsoft volatilities but their movements are not as extreme. In contrast, the rolling volatilities of the GWN series are fairly stable bouncing between 7% and 12%. The time variation exhibited by the rolling volatilities of

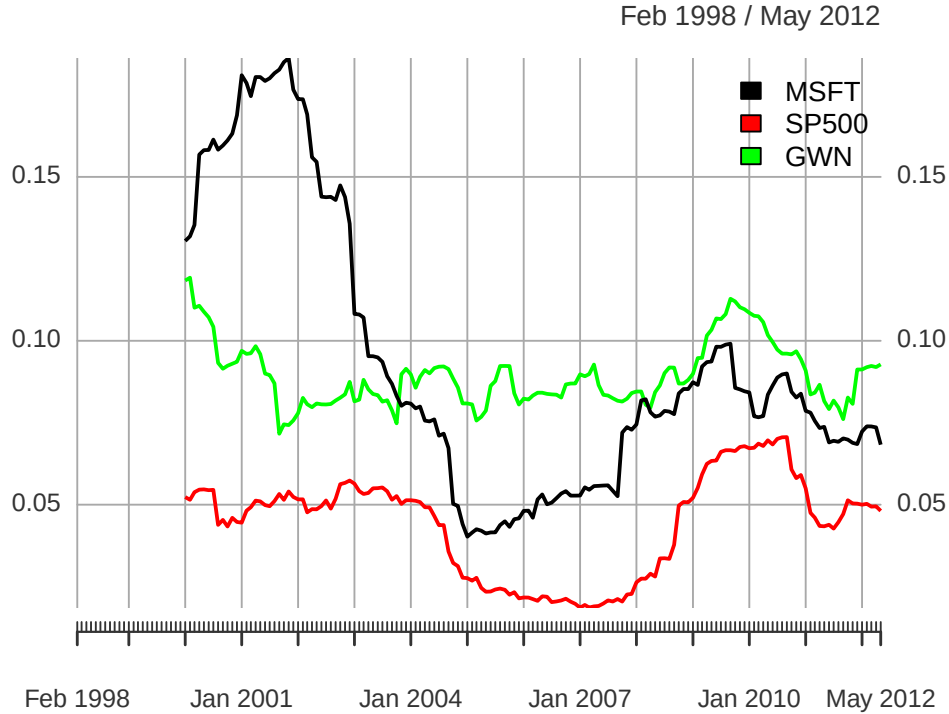


Figure 5.30: 24-month rolling estimates of σ .

Microsoft and the S&P 500 index places additional doubt on the covariance stationarity assumption for returns. ■

To investigate time variation in the correlation between two asset returns, ρ_{ij} , one can compute the rolling covariances and correlations over fixed windows of size $n < T$:

$$\hat{\sigma}_{ij,t}(n) = \frac{1}{n-1} \sum_{k=0}^{n-1} (r_{i,t-k} - \hat{\mu}_i(n))(r_{j,t-k} - \hat{\mu}_j(n)), \quad (5.21)$$

$$\hat{\rho}_{ij,t}(n) = \frac{\hat{\sigma}_{ij,t}(n)}{\hat{\sigma}_{i,t}(n)\hat{\sigma}_{j,t}(n)}. \quad (5.22)$$

Example 5.28 (Rolling correlation estimates for Microsoft Starbucks and the S&P 500 index). A function to compute the pair-wise sample correlations between Microsoft, Starbucks and the S&P 500 index is

```
rhohat = function(x) {
  corhat = cor(x)
  corhat.vals = corhat[lower.tri(corhat)]
  names(corhat.vals) = c("MSFT.SP500", "MSFT.GWN", "SP500.GWN")
  corhat.vals
}
```

```
}
```

Using the function `rhohat()`, 24-month rolling estimates of ρ_{ij} for Microsoft, Starbucks and the S&P 500 index incremented by one month with the time index aligned to the beginning of the window can be computed using

```
roll.rhohat = rollapply(roll.data, width=24, FUN=rhohat,  
  by=1, by.column=FALSE, align="right")  
head(na.omit(roll.rhohat),n=3)  
#>      MSFT.SP500 MSFT.GWN SP500.GWN  
#> Jan 2000      0.638   0.392   0.278  
#> Feb 2000      0.637   0.345   0.244  
#> Mar 2000      0.663   0.415   0.377
```

Notice that `by.column=FALSE` in the call to `rollapply()`. This is necessary as the function `rhohat()` operates on all columns of the data object `roll.data`. The rolling estimates are illustrated in Figure 5.31.

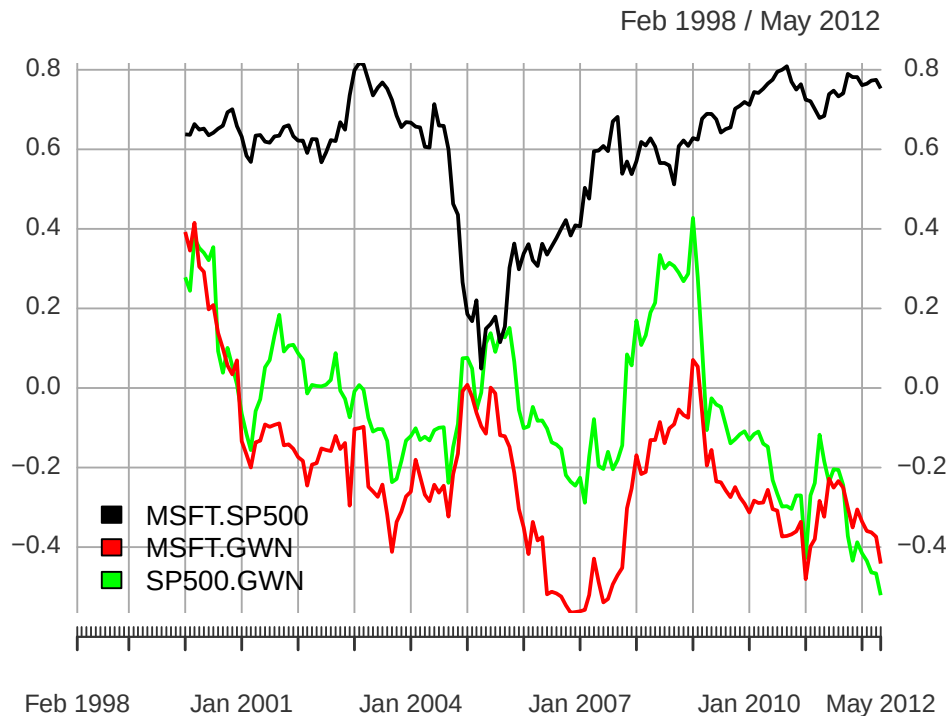


Figure 5.31: 24-month rolling estimates of ρ_{ij} for Microsoft, Starbucks and the S&P 500 index.

The ranges of the rolling correlations are:

```
ans = apply(na.omit(roll.rhohat),2,range)
rownames(ans) = c("Min", "Max")
ans
#>      MSFT.SP500 MSFT.GWN SP500.GWN
#> Min      0.0487  -0.565   -0.521
#> Max      0.8169   0.415    0.428
```

The rolling correlations between Microsoft and the S&P 500 index show an interesting pattern. They start moderately high, around 0.6, then plunge to about 0.05 in early 2005, and then gradually pick back up to about 0.8 at the end of the sample. The rolling correlations are highest during the financial crisis periods and lowest during the calm periods. Interestingly, the rolling correlations between the GWN series and Microsoft, and the GWN series and the S&P 500 index fluctuate somewhat randomly between -0.5 and 0.5 and show no particular pattern. There is compelling evidence that the correlation between Microsoft and the S&P 500 index is not constant over time.



It must be kept in mind that the rolling descriptive statistics are *informal diagnostics* for evaluating the assumption of covariance stationarity. As you have seen, there is considerably sampling variability in the rolling statistics due to the small window size which can make it difficult to untangle statistical noise from true changes in the underlying parameters. In Chapter 9 I will consider formal statistical tests for constant parameters that can give better evidence for or against the assumption of covariance stationarity.

5.4.1 Practical issues associated with rolling estimates

To be completed

- Discuss choice of window width. Small window widths tend to produce choppy estimates with a lot of estimation error. Large window widths produce smooth estimates with less estimation error
- Discuss choice of increment. For a given window width, the roll-by increment determines the number of overlapping returns in each window. Number of overlapping returns is $n - incr$ where n is the window width and $incr$ is the roll-by increment. Incrementing the window by 1 gives the largest number of overlapping returns. If $incr \geq n$ then the rolling windows are non-overlapping.
- A large positive or negative return in a window can greatly influence the estimate and cause a drop-off effect as the window rolls past the large observation.

5.5 Stylized facts for daily and monthly asset returns

A *stylized fact* is something that is generally true but not always. From the data analysis of the daily and monthly returns on Microsoft and the S&P 500 index I observe a number of stylized facts that tend to be true for other individual assets and portfolios of assets. For monthly data I observe the following stylized facts:

1. M1. Prices appear to be random walk non-stationary and returns appear to be mostly covariance stationary. There is evidence that return volatility changes over time.
2. M2. Returns appear to be approximately normally distributed. There is some negative skewness and excess kurtosis.
3. M3. Assets that have high average returns tend to have high standard deviations (volatilities) and vice-versa. This is the *no free lunch* principle.
4. M4. Returns on individual assets (stocks) have higher standard deviations than returns on diversified portfolios of assets (stocks).
5. M5. Returns on different assets tend to be positively correlated. It is unusual for returns on different assets to be negatively correlated.
6. M6. Returns are approximately uncorrelated over time. That is, there is little evidence of linear time dependence in asset returns.
7. M7. Returns do not exhibit dynamic feedback. That is, there are no strong lead-lag effects between pairs of returns.
8. M8 There is some informal evidence of non-constant volatilities and correlations, particularly during financial crisis periods.

Daily returns have some features in common with monthly returns and some not. The common features are M1 and M3-M7 above. The stylized facts that are specific to daily returns are:

1. D2. Returns are not normally distributed. Empirical distributions have much fatter tails than the normal distribution (excess kurtosis).
2. D8. Returns are not independent over time. Absolute and squared returns are positively auto correlated and the correlation dies out very slowly. Volatility appears to be auto correlated and, hence, predictable.

These stylized facts of daily and monthly asset returns are the main features of returns that models of assets returns should capture. A *good model* is one that can explain many stylized facts of the data. A bad model does not capture important stylized facts.

5.6 Further Reading: Descriptive Statistics

(Fama 1976) is the first book that provides a comprehensive descriptive statistical analysis of asset returns. Much of the material in this chapter, and this book, is motivated by Fama's book. Several

recent textbooks discuss exploratory data analysis of finance time series. This discussion in this chapter is similar to that presented in (Carmona 2014), (Jondeau, Poon, and Rockinger 2007), (Ruppert and Matteson 2015), (Tsay 2010), (Tsay 2014), and (Zivot 2016).

- Mention books by Carol Alexander
- Mention R packages for descriptive statistics: `dataExplorer`,

5.7 Problems: Descriptive Statistics

Exercise 5.1. Compute time plots with *ggplot2* function `autoplot()`

Exercise 5.2. In this question, you will compute descriptive statistics for the continuously compounded monthly return data on the Vanguard long term bond index fund (VBLTX), Fidelity Magellan stock mutual fund (FMAGX), and Starbucks stock (SBUX). You will use the R packages **corrplot**, **IntroCompFinR**, **PerformanceAnalytics**, **tseries** and **zoo**.

1. Downloading data and return calculations.
 - a. Use the **tseries** function `get.hist.quote()` to download from Yahoo! the monthly adjusted closing prices on VBLTX, FMAGX and SBUX over the period 1998-01-01 through 2009-12-31. Download these prices as "zoo" objects.
 - b. Use the **PerformanceAnalytics** function `Return.calculate()` to compute continuously compounded monthly returns and remove the first NA observation.
2. Univariate graphical analysis.
 - a. Make time plots of the return data using the **zoo** function `plot.zoo()`. Comment on any relationships between the returns suggested by the plots. Pay particular attention to the behavior of returns toward the end of 2008 at the beginning of the financial crisis.
 - b. Make a equity curve/cumulative return plot (future of \$1 invested in each asset) and comment. Which assets gave the best and worst future values over the investment horizon?
 - c. For each return series, use the **IntroCompFinR** function `fourPanelPlot()` to make a four-panel plot containing a histogram, boxplot, normal QQ-plot, and SACF plot. Do the return series look normally distributed? Briefly compare the return distributions. Do you see any evidence of time dependence in the returns?
3. Univariate numerical summary statistics and historical VaR.
 - a. Compute numerical descriptive statistics for all assets using the R functions `summary()`, `mean()`, `var()`, `stdev()`, and the **PerformanceAnalytics** functions `skewness()` and `kurtosis()`. Compare and contrast the descriptive statistics for the three assets. Which asset appears to be the riskiest asset?

- b. Using the mean monthly return for each asset, compute an estimate of the annual continuously compounded return (i.e., recall the relationship between the expected monthly cc return and the expected annual cc return). Convert this annual continuously compounded return into a simple annual return. Are there any surprises?
 - c. Using the estimate of the monthly return standard deviation for each asset, compute an estimate of the annual return standard deviation. Briefly comment on the magnitude of the annual standard deviations.
 - d. For each asset compute the empirical 1% and 5% quantiles of the cc returns. Using these quantiles compute the 1% and 5% historical (monthly) VaR values based on an initial \$100,000 investment. Which asset has the highest and lowest VaR values? Are you surprised?
4. Bivariate graphical analysis and numerical summary statistics
- a. Use the R `pairs()` function to create all pair-wise scatterplots of returns. Comment on the direction and strength of the linear relationships in these plots.
 - b. Use the **corrplot** functions `corrplot()` and `corrplot.mixed()` to plot the correlation matrix of the returns on the three assets.
 - c. Use the R functions `var()`, `cov()`, and `cor()` to compute the sample covariance matrix and sample correlation matrix of the returns. Comment on the direction and strength of the linear relationships suggested by the values of the covariances and correlations.

Exercise 5.3. In this question, you will compute descriptive statistics for the continuously compounded weekly return data on the Vanguard long term bond index fund (VBLTX), Fidelity Magellan stock mutual fund (FMAGX), and Starbucks stock (SBUX). Repeat the analysis for the monthly returns but use instead weekly returns computed from end-of-week adjusted closing prices.

Exercise 5.4. In this question, you will compute descriptive statistics for the continuously compounded daily return data on the Vanguard long term bond index fund (VBLTX), Fidelity Magellan stock mutual fund (FMAGX), and Starbucks stock (SBUX). Repeat the analysis for the monthly returns but use instead daily returns computed from end-of-day adjusted closing prices.

Exercise 5.5. In this problem you will analyze the monthly simple returns on the Vanguard S&P 500 index (VFINX) and Amazon stock (AMZN) over the 5-year period January 2010, through January 2015. For this period there are $T=60$ monthly returns. The monthly prices for VFINX are in the **IntroCompFinR** object `VanguardPrices` and the daily prices for AMZN are in the **IntroCompFinR** object `amznDailyPrices`.

1. Compute simple monthly returns on AMZN and VFINX and plot these returns together in a single plot. Do the monthly returns from the two assets look like realizations from a covariance stationary stochastic process? Why or why not?

2. Compare and contrast the return characteristics of the two assets. In addition, comment on any common features, if any, of the two return series.
3. Plot the cumulative simple returns (equity curve) for each fund, which represents the growth of \$1 invested in each fund over the sample period, and plot these returns in the same graph. Which fund performed better over the sample?
4. Using the **IntroCompFinR** function `fourPanelPlot()`, create descriptive statistics plots for each asset. In addition, compute Based on statistics, do the returns on VFINX and AMZN look normally distributed? Is there any evidence of linear time dependence? Briefly justify your answer.
5. Which asset appears to be riskier? Briefly justify your answer.

Exercise 5.6 (Descriptive statistics for weekly data).

- Create weekly time series for MSFT, SP500
- Repeat graphical and numerical descriptive statistics
- Deduce stylized facts for weekly data

Exercise 5.7 (Rolling descriptive statistics for daily data). TBD

Exercise 5.8. Loading data and computing returns

Load the daily price data from **IntroCompFinR**, and create monthly returns over the period Jan 1998 through Dec 2014:

```
data(amznDailyPrices, baDailyPrices, costDailyPrices, jwnDailyPrices, sbuxDailyPrices)
fiveStocks = merge(amznDailyPrices, baDailyPrices, costDailyPrices, jwnDailyPrices, sbuxDailyPrices)
fiveStocks = to.monthly(fiveStocks, OHLC=FALSE)
```

Let's look at the data

```
head(fiveStocks, n=3)
#>           AMZN   BA COST   JWN SBUX
#> Jan 1998 4.92 34.0 17.4   9.4 4.24
#> Feb 1998 6.42 38.9 19.6  10.6 4.59
#> Mar 1998 7.13 37.4 21.4  11.8 5.25
```

```
tail(fiveStocks, n=3)
#>           AMZN   BA COST   JWN SBUX
#> Oct 2014  305 123  128   72 75.0
#> Nov 2014  339 134  137   76 80.9
#> Dec 2014  310 129  137   79 81.8
```

Next, let's compute monthly simple returns using the **PerformanceAnalytics** function `Return.Calculate()`

```
fiveStocksRet = na.omit(Return.calculate(fiveStocks, method = "simple"))
head(fiveStocksRet, n=3)
#>           AMZN      BA  COST    JWN   SBUX
#> Feb 1998 0.3049  0.1425 0.1267 0.1298 0.0825
#> Mar 1998 0.1106 -0.0393 0.0920 0.1121 0.1438
#> Apr 1998 0.0729 -0.0396 0.0468 0.0262 0.0629
```

I removed the missing January return using the function `na.omit()`.

Part I: Univariate Graphical Analysis

1. Make time plots of the return data using the **zoo** function `plot.zoo()`. Comment on any relationships between the returns suggested by the plots.
2. Make a cumulative return plot (equity curve) showing the future value of \$1 invested in each asset. Which assets gave the best and worst future values over the investment horizon? Hint: you can use the `cumprod()` function or the **PerformanceAnalytics** function `chart.CumReturns()`.
3. For each return series, make a four panel plot containing a histogram, boxplot, normal QQ-plot and SACF. Do the return series look normally distributed? Is there any evidence of time dependence? Briefly compare the return distributions. Hint: the `fourPanelPlot()` function only works on one asset at a time. So call the function for each asset. For example, `fourPanelPlot(fiveStocksRet[, "AMZN"])`

Part II: Univariate Numerical Summaries

1. Compute numerical descriptive statistics for all assets using the R functions `mean()`, `var()`, `stdev()`, and the **PerformanceAnalytics** functions `skewness()` and `kurtosis()`. Compare and contrast the descriptive statistics for the five assets. Which asset appears to be the riskiest asset?
2. Using the monthly mean return for each asset, compute an estimate of the mean annual return. Hint: use the square root of time rule for cc returns. (The square root of time rule is exact for continuously compounded returns but can be applied as an approximation to simple returns.)
3. Using the monthly return standard deviation for each asset, compute an estimate of the annual return standard deviation. Hint: use the square root of time rule for cc returns. (The square root of time rule is exact for continuously compounded returns but can be applied as an approximation to simple returns.)

Part III: Historical VaR

1. For each asset compute the empirical 1% and 5% quantiles of the returns. Using these quantiles compute the 1% and 5% historical (monthly) VaR values based on an initial \$100,000 investment. Which asset has the highest and lowest VaR values? Are you surprised?

Part IV. Bivariate Graphical Analysis

1. Use the `pairs()` function to create all pair-wise scatter plots of returns. Comment on the direction and strength of the linear relationships in these plots.
2. Use the **corrplot** function `corrplot.mixed()` to plot the correlation matrix of the returns on the five assets. Comment on what you see.

Part V. Bivariate Numerical Summary Statistics

Use the R functions `cov()`, and `cor()` to compute the sample covariance matrix and sample correlation matrix of the returns. Comment on the direction and strength of the linear relationships suggested by the values of the covariances and correlations.

Part VI. Stylized Facts

1. Based on your analysis of the five Northwest stocks, what stylized facts do you see?

6 The Gaussian White Noise Return Model

Updated: September 4, 2026

Copyright © Eric Zivot 2026

The first model of asset returns I consider is the very simple *Gaussian White Noise* (GWN) model. This model is motivated by the stylized facts for monthly asset returns discussed in the previous chapter. The GWN model assumes that an asset's return (simple or continuously compounded) over time is independent and identically normally distributed. The model allows for the returns on different assets to be contemporaneously correlated but does not allow for any lead-lag cross correlations. The GWN model is widely used in finance. For example, it is used in risk analysis (e.g., for computing Value-at-Risk) for assets and portfolios, in mean-variance portfolio analysis, in the *Capital Asset Pricing Model* (CAPM), and in the *Black-Scholes* option pricing model. Although this model is very simple, it provides important intuition about the statistical behavior of asset returns and prices and serves as a benchmark against which more sophisticated models can be compared and evaluated. It allows me to discuss and develop several important econometric topics such as Monte Carlo simulation, estimation, bootstrapping, hypothesis testing, forecasting, and model evaluation that are discussed in later chapters.

The outline of this chapter is as follows. In Section 6.1 I review the assumptions of the GWN model and presents several equivalent specifications of the model. I also discuss the application of the model to continuously compounded returns, simple returns, and portfolios. In Section 6.2 I illustrate Monte Carlo simulation of the GWN model as a first-step reality check of the model.

The R packages I use in this chapter are **introcompfinr**, **mvtnorm**, and **PerformanceAnalytics**. Make sure these packages are installed and loaded before running the R examples in the chapter.

6.1 GWN Model Assumptions

Let R_{it} denote the simple or continuously compounded (cc) return on asset i over the investment horizon between times $t - 1$ and t (e.g., monthly returns). I make the following assumptions regarding the probability distribution of R_{it} for $i = 1, \dots, N$ assets for all times t .

Assumption GWN

1. *Covariance stationarity and ergodicity*: $\{R_{i1}, \dots, R_{iT}\} = \{R_{it}\}_{t=1}^T$ is a covariance stationary and ergodic stochastic process with $E[R_{it}] = \mu_i$, $\text{var}(R_{it}) = \sigma_i^2$, $\text{cov}(R_{it}, R_{jt}) = \sigma_{ij}$ and $\text{cor}(R_{it}, R_{jt}) = \rho_{ij}$.
2. *Normality*: $R_{it} \sim N(\mu_i, \sigma_i^2)$ for all i and t , and all joint distributions are normal.
3. *No serial correlation*: $\text{cov}(R_{it}, R_{js}) = \text{cor}(R_{it}, R_{js}) = 0$ for $t \neq s$ and $i, j = 1, \dots, N$.

Assumption GWN states that in every time period asset returns are jointly (multivariate) normally distributed, that the means and the variances of all asset returns, and all of the pairwise contemporaneous covariances and correlations between assets are constant over time. In addition, all of the asset returns are *serially uncorrelated*:

$$\text{cor}(R_{it}, R_{is}) = \text{cov}(R_{it}, R_{is}) = 0 \text{ for all } i \text{ and } t \neq s,$$

and the returns on all possible pairs of assets i and j are *cross lead-lag uncorrelated*:

$$\text{cor}(R_{it}, R_{js}) = \text{cov}(R_{it}, R_{js}) = 0 \text{ for all } i \neq j \text{ and } t \neq s.$$

In addition, under the normal distribution assumption lack of serial and cross lead-lag correlation implies time independence of returns. Clearly, these are very strong assumptions. However, they allow us to develop a straightforward probabilistic model for asset returns as well as statistical tools for estimating the parameters of the model, testing hypotheses about the parameter values and assumptions.

6.1.1 Regression model representation

A convenient mathematical representation or *model* of asset returns can be given based on Assumption GWN. This is the *GWN regression model*.

Definition 6.1 (GWN Regression Model). For assets $i = 1, \dots, N$ and time periods $t = 1, \dots, T$, the GWN regression model is:

$$\begin{aligned} R_{it} &= \mu_i + \varepsilon_{it}, \\ \{\varepsilon_{it}\}_{t=1}^T &\sim \text{GWN}(0, \sigma_i^2), \\ \text{cov}(\varepsilon_{it}, \varepsilon_{js}) &= \begin{cases} \sigma_{ij} & t = s \\ 0 & t \neq s \end{cases}. \end{aligned} \tag{6.1}$$

The notation $\varepsilon_{it} \sim \text{GWN}(0, \sigma_i^2)$ stipulates that the stochastic process $\{\varepsilon_{it}\}_{t=1}^T$ is a GWN process with $E[\varepsilon_{it}] = 0$ and $\text{var}(\varepsilon_{it}) = \sigma_i^2$. In addition, the random error term ε_{it} is independent of ε_{js} for all assets $i \neq j$ and all time periods $t \neq s$.

Using the basic properties of expectation, variance and covariance, you can derive the following properties of returns in the GWN model:

$$\begin{aligned}
E[R_{it}] &= E[\mu_i + \varepsilon_{it}] = \mu_i + E[\varepsilon_{it}] = \mu_i, \\
\text{var}(R_{it}) &= \text{var}(\mu_i + \varepsilon_{it}) = \text{var}(\varepsilon_{it}) = \sigma_i^2, \\
\text{cov}(R_{it}, R_{jt}) &= \text{cov}(\mu_i + \varepsilon_{it}, \mu_j + \varepsilon_{jt}) = \text{cov}(\varepsilon_{it}, \varepsilon_{jt}) = \sigma_{ij}, \\
\text{cov}(R_{it}, R_{js}) &= \text{cov}(\mu_i + \varepsilon_{it}, \mu_j + \varepsilon_{js}) = \text{cov}(\varepsilon_{it}, \varepsilon_{js}) = 0, \quad t \neq s.
\end{aligned}$$

Given that covariances and variances of returns are constant over time implies that the correlations between returns over time are also constant:

$$\begin{aligned}
\text{cor}(R_{it}, R_{jt}) &= \frac{\text{cov}(R_{it}, R_{jt})}{\sqrt{\text{var}(R_{it})\text{var}(R_{jt})}} = \frac{\sigma_{ij}}{\sigma_i \sigma_j} = \rho_{ij}, \\
\text{cor}(R_{it}, R_{js}) &= \frac{\text{cov}(R_{it}, R_{js})}{\sqrt{\text{var}(R_{it})\text{var}(R_{js})}} = \frac{0}{\sigma_i \sigma_j} = 0, \quad i \neq j, t \neq s.
\end{aligned}$$

Finally, since $\{\varepsilon_{it}\}_{t=1}^T \sim \text{GWN}(0, \sigma_i^2)$ it follows that $\{R_{it}\}_{t=1}^T \sim \text{iid } N(\mu_i, \sigma_i^2)$. Hence, the GWN regression model Equation 6.1 for R_{it} is equivalent to the model implied by Assumption GWN.

6.1.1.1 Interpretation of the GWN regression model

The GWN model has a very simple form and is identical to the *measurement error model* in the statistics literature.¹ In words, the model states that each asset return is equal to a constant μ_i (the expected return) plus an *i.i.d.* normally distributed random variable ε_{it} with mean zero and constant variance. The random variable ε_{it} can be interpreted as representing the *unexpected news* concerning the value of the asset that arrives between time $t-1$ and time t . To see this, Equation 6.1 implies that:

$$\varepsilon_{it} = R_{it} - \mu_i = R_{it} - E[R_{it}],$$

so that ε_{it} is defined as the deviation of the random return from its expected value. If the news between times $t-1$ and t is good, then the realized value of ε_{it} is positive and the observed return is above its expected value μ_i . If the news is bad, then ε_{it} is negative and the observed return is less than expected. The assumption $E[\varepsilon_{it}] = 0$ means that news, on average, is neutral; neither good nor bad. The assumption that $\text{sd}(\varepsilon_{it}) = \sigma_i$ can be interpreted as saying that the volatility, or typical magnitude, of news arrival is constant over time. The random news variable affecting asset i , ε_{it} , is allowed to be contemporaneously correlated with the random news variable affecting asset j , ε_{jt} , to capture the idea that news about one asset may spill over and affect another asset. For

¹In the measurement error model, r_{it} represents the t^{th} measurement of some physical quantity μ_i and ε_{it} represents the random measurement error associated with the measurement device. The value σ_i represents the typical size of a measurement error.

example, if asset i is Microsoft stock and asset j is Apple Computer stock, then one interpretation of news in this context is general news about the computer industry and technology. Good news should lead to positive values of both ε_{it} and ε_{jt} . Hence these variables will be positively correlated due to a positive reaction to a common news component. Finally, the news on asset j at time s is unrelated to the news on asset i at time t for all times $t \neq s$. For example, this means that the news for Apple in January is not related to the news for Microsoft in February.

6.1.2 Location-Scale model representation

Sometimes it is convenient to re-express the regression form of the GWN model Equation 6.1 in *location-scale* form:

$$R_{it} = \mu_i + \varepsilon_{it} = \mu_i + \sigma_i \cdot Z_{it}, \quad \{Z_{it}\}_{t=1}^T \sim \text{GWN}(0, 1), \quad (6.2)$$

where I use the decomposition $\varepsilon_{it} = \sigma_i \cdot Z_{it}$. In this form, the random news shock is the *i.i.d.* standard normal random variable Z_{it} scaled by the “news” volatility σ_i .

This form is particularly convenient for Value-at-Risk calculations because the $\alpha \times 100\%$ quantile of the return distribution has the simple form:

$$q_\alpha^{R_i} = \mu_i + \sigma_i \times q_\alpha^Z,$$

where q_α^Z is the $\alpha \times 100\%$ quantile of the standard normal random distribution. Let W_0 be the initial amount of wealth to be invested from time $t - 1$ to t . If R_{it} is the simple return then,

$$\text{VaR}_\alpha = W_0 \times q_\alpha^{R_i} = W_0 \times (\mu_i + \sigma_i \times q_\alpha^Z),$$

whereas if R_{it} is the continuously compounded return then,

$$\text{VaR}_\alpha = W_0 \times (e^{q_\alpha^{R_i}} - 1) = W_0 \times (e^{\mu_i + \sigma_i \times q_\alpha^Z} - 1).$$

6.1.3 The GWN model in matrix notation

Define the $N \times 1$ vectors $\mathbf{R}_t = (R_{1t}, \dots, R_{Nt})'$, $\boldsymbol{\mu} = (\mu_1, \dots, \mu_N)'$, $\boldsymbol{\varepsilon}_t = (\varepsilon_{1t}, \dots, \varepsilon_{Nt})'$ and the $N \times N$ symmetric covariance matrix:

$$\text{var}(\boldsymbol{\varepsilon}_t) = \Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1N} \\ \sigma_{12} & \sigma_2^2 & \cdots & \sigma_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{1N} & \sigma_{2N} & \cdots & \sigma_N^2 \end{pmatrix}.$$

Definition 6.2 (GWN Regression Model in Matrix Form). The regression form of the GWN model in matrix notation is:

$$\mathbf{R}_t = \mu + \varepsilon_t, \varepsilon_t \sim \text{iid } N(\mathbf{0}, \Sigma), \quad (6.3)$$

which implies that $\mathbf{R}_t \sim \text{iid } N(\mu, \Sigma)$.

The location-scale form of the GWN model in matrix notation makes use of the matrix square root factorization $\Sigma = \Sigma^{1/2} \Sigma^{1/2'}$ where $\Sigma^{1/2}$ is the lower-triangular matrix square root (usually the Cholesky factorization). Then Equation 6.3 can be rewritten as:

$$\mathbf{R}_t = \mu + \Sigma^{1/2} \mathbf{Z}_t, \mathbf{Z}_t \sim \text{GWN}(\mathbf{0}, \mathbf{I}_N), \quad (6.4)$$

where $\mathbf{I}_N$ denotes the N -dimensional identity matrix.

6.1.4 The GWN model for continuously compounded returns

The GWN model is often used to describe continuously compounded (cc) returns defined as $R_{it} = \ln(P_{it}/P_{it-1})$ where P_{it} is the price of asset i at time t . An advantage of the GWN model for cc returns is that the model aggregates to any time horizon because multi-period cc returns are additive. This is particularly convenient for investment risk analysis. The GWN model for cc returns also gives rise to the *random walk model* for the logarithm of asset prices. The normal distribution assumption of the GWN model for cc returns implies that single-period simple returns are log-normally distributed.

A disadvantage of the GWN model for cc returns is that the model has some limitations for the analysis of portfolios because the cc return on a portfolio of assets is not a weighted average of the cc returns on the individual assets. As a result, for portfolio analysis the GWN model is typically applied to simple returns. However, if the investment horizon is short (e.g. daily, weekly or monthly) then simple returns are typically close to zero on average and there is little difference between simple and cc returns. Hence, the time aggregation results of the GWN model for cc returns can be applied to simple returns and the portfolio results of the GWN model for simple returns can be applied to cc returns.

6.1.4.1 Time Aggregation and the GWN Model

The GWN model for cc returns has the following nice aggregation property with respect to the interpretation of ε_{it} as news. For illustration purposes, suppose that t represents months so that R_{it} is the cc monthly return on asset i . Now, instead of the monthly return, suppose you are interested in the annual cc return $R_{it} = R_{it}(12)$. Since multi-period cc returns are additive, $R_{it}(12)$ is the

sum of 12 monthly cc returns:

$$R_{it} = R_{it}(12) = \sum_{k=0}^{11} R_{it-k} = R_{it} + R_{it-1} + \cdots + R_{it-11}.$$

Using the GWN regression model Equation 6.1 for the monthly return R_{it} , you may express the annual return $R_{it}(12)$ as:

$$R_{it}(12) = \sum_{t=0}^{11} (\mu_i + \varepsilon_{it}) = 12 \cdot \mu_i + \sum_{t=0}^{11} \varepsilon_{it} = \mu_i(12) + \varepsilon_{it}(12),$$

where $\mu_i(12) = 12 \cdot \mu_i$ is the annual expected return on asset i , and $\varepsilon_{it}(12) = \sum_{k=0}^{11} \varepsilon_{it-k}$ is the annual random news component. The annual expected return, $\mu_i(12)$, is simply 12 times the monthly expected return, μ_i . The annual random news component, $\varepsilon_{it}(12)$, is the accumulation of news over the year. As a result, the variance of the annual news component, $\sigma_i^2(12)$, is 12 times the variance of the monthly news component:

$$\begin{aligned} \text{var}(\varepsilon_{it}(12)) &= \text{var} \left(\sum_{k=0}^{11} \varepsilon_{it-k} \right) \\ &= \sum_{k=0}^{11} \text{var}(\varepsilon_{it-k}) \text{ since } \varepsilon_{it} \text{ is uncorrelated over time} \\ &= \sum_{k=0}^{11} \sigma_i^2 \text{ since } \text{var}(\varepsilon_{it}) \text{ is constant over time} \\ &= 12 \cdot \sigma_i^2 = \sigma_i^2(12). \end{aligned}$$

It follows that the standard deviation of the annual news is equal to $\sqrt{12}$ times the standard deviation of monthly news:

$$\text{sd}(\varepsilon_{it}(12)) = \sqrt{12} \times \text{sd}(\varepsilon_{it}) = \sqrt{12} \times \sigma_i.$$

Similarly, due to the additivity of covariances, the covariance between $\varepsilon_{it}(12)$ and $\varepsilon_{jt}(12)$ is 12 times the monthly covariance:

$$\begin{aligned} \text{cov}(\varepsilon_{it}(12), \varepsilon_{jt}(12)) &= \text{cov} \left(\sum_{k=0}^{11} \varepsilon_{it-k}, \sum_{k=0}^{11} \varepsilon_{jt-k} \right) \\ &= \sum_{k=0}^{11} \text{cov}(\varepsilon_{it-k}, \varepsilon_{jt-k}) \text{ since } \varepsilon_{it} \text{ and } \varepsilon_{jt} \text{ are uncorrelated over time} \\ &= \sum_{k=0}^{11} \sigma_{ij} \text{ since } \text{cov}(\varepsilon_{it}, \varepsilon_{jt}) \text{ is constant over time} \\ &= 12 \cdot \sigma_{ij} = \sigma_{ij}(12). \end{aligned}$$

The above results imply that the correlation between the annual errors $\varepsilon_{it}(12)$ and $\varepsilon_{jt}(12)$ is the same as the correlation between the monthly errors ε_{it} and ε_{jt} :

$$\begin{aligned}\text{cor}(\varepsilon_{it}(12), \varepsilon_{jt}(12)) &= \frac{\text{cov}(\varepsilon_{it}(12), \varepsilon_{jt}(12))}{\sqrt{\text{var}(\varepsilon_{it}(12)) \cdot \text{var}(\varepsilon_{jt}(12))}} \\ &= \frac{12 \cdot \sigma_{ij}}{\sqrt{12\sigma_i^2 \cdot 12\sigma_j^2}} \\ &= \frac{\sigma_{ij}}{\sigma_i \sigma_j} = \rho_{ij} = \text{cor}(\varepsilon_{it}, \varepsilon_{jt}).\end{aligned}$$

The above results generalize to aggregating returns to arbitrary time horizons as summarized in the following Proposition.

Proposition 6.1 (Square Root of Time Rule). *Let R_{it} denote the cc return on asset i between times $t - 1$ and t , where t represents the general investment horizon, and let $R_{it}(k) = \sum_{j=0}^{k-1} R_{it-j}$ denote the k -period cc return. Then the GWN model for $R_{it}(k)$ has the form:*

$$R_{it}(k) = \mu_i(k) + \varepsilon_{it}(k), \varepsilon_{it}(k) \sim \text{GWN}(0, \sigma_i^2(k)), \quad (6.5)$$

where $\mu_i(k) = k \times \mu_i$ is the k -period expected return, $\varepsilon_{it}(k) = \sum_{j=0}^{k-1} \varepsilon_{it-j}$ is the k -period error term, and $\sigma_i^2(k) = k \times \sigma_i^2$ is the k -period variance. The k -period volatility follows the square-root-of-time rule: $\sigma_i(k) = \sqrt{k} \times \sigma_i$. The k -period covariance between asset i and asset j is $\sigma_{ij}(k) = k \times \sigma_{ij}$, and the k -period correlation is $\rho_{ij}(k) = \rho_{ij}$. This aggregation result is exact for cc returns but it is often used as an approximation for simple returns.

6.1.4.2 The random walk model of asset prices

The GWN model for cc returns Equation 6.1 gives rise to the so-called *random walk* (RW) model for the *logarithm* of asset prices. To see this, recall that the continuously compounded return, R_{it} , is defined from asset prices via $R_{it} = \ln\left(\frac{P_{it}}{P_{it-1}}\right) = \ln(P_{it}) - \ln(P_{it-1})$. Letting $p_{it} = \ln(P_{it})$ and using the representation of R_{it} in the GWN model Equation 6.1, you can express the log-price as:

$$p_{it} = p_{it-1} + \mu_i + \varepsilon_{it}. \quad (6.6)$$

The representation in Equation 6.6 is known as the RW model for log-prices.² It is a representation of the GWN model in terms of log-prices.

²The model Equation 6.6 is technically a random walk with drift μ_i . A pure random walk has zero drift ($\mu_i = 0$).

In the RW model Equation 6.6, μ_i represents the expected change in the log-price (cc return) between months $t - 1$ and t , and ε_{it} represents the unexpected change in the log-price. That is,

$$\begin{aligned} E[\Delta p_{it}] &= E[R_{it}] = \mu_i, \\ \varepsilon_{it} &= \Delta p_{it} - E[\Delta p_{it}]. \end{aligned}$$

where $\Delta p_{it} = p_{it} - p_{it-1}$. Further, in the RW model, the unexpected changes in the log-price, $\{\varepsilon_{it}\}$, are uncorrelated over time ($\text{cov}(\varepsilon_{it}, \varepsilon_{is}) = 0$ for $t \neq s$) so that future changes in the log-price cannot be predicted from past changes in the log-price.³

The RW model gives the following interpretation for the evolution of log prices. Let p_{i0} denote the initial log price of asset i . The RW model says that the log-price at time $t = 1$ is:

$$p_{i1} = p_{i0} + \mu_i + \varepsilon_{i1},$$

where ε_{i1} is the value of random news that arrives between times 0 and 1. The expected log-price at time $t = 1$ is:

$$E[p_{i1}] = p_{i0} + \mu_i + E[\varepsilon_{i1}] = p_{i0} + \mu_i,$$

which is the initial price plus the expected return between times 0 and 1. Similarly, by recursive substitution the log-price at time $t = 2$ is:

$$\begin{aligned} p_{i2} &= p_{i1} + \mu_i + \varepsilon_{i2} \\ &= p_{i0} + \mu_i + \mu_i + \varepsilon_{i1} + \varepsilon_{i2} \\ &= p_{i0} + 2 \cdot \mu_i + \sum_{t=1}^2 \varepsilon_{it} \\ &= p_{i0} + 2 \cdot \mu_i + \varepsilon_{it}(2), \end{aligned}$$

which is equal to the initial log-price, p_{i0} , plus the two period expected return, $2 \cdot \mu_i$, plus the accumulated random news over the two periods, $\sum_{t=1}^2 \varepsilon_{it} = \varepsilon_{it}(2)$. By repeated recursive substitution, the log price at time $t = T$ is,

$$p_{iT} = p_{i0} + T \cdot \mu_i + \sum_{t=1}^T \varepsilon_{it} = p_{i0} + T \cdot \mu_i + \varepsilon_{it}(T).$$

At time $t = 0$, the expected log-price at time $t = T$ is,

$$E[p_{iT}] = p_{i0} + T \cdot \mu_i,$$

which is the initial price plus the expected growth in prices over T periods. The actual price, p_{iT} ,

³The notion that future changes in asset prices cannot be predicted from past changes in asset prices is often referred to as the weak form of the efficient markets hypothesis.

deviates from the expected price by the accumulated random news:

$$p_{iT} - E[p_{iT}] = \sum_{t=1}^T \varepsilon_{it} = \varepsilon_{it}(T).$$

At time $t = 0$, the variance of the log-price at time T is,

$$\text{var}(p_{iT}) = \text{var} \left(\sum_{t=1}^T \varepsilon_{it} \right) = T \cdot \sigma_i^2$$

Hence, the RW model implies that the stochastic process of log-prices $\{p_{it}\}$ is non-stationary because the variance of p_{it} increases with t . Finally, because $\varepsilon_{it} \sim (0, \sigma_i^2)$ it follows that (conditional on p_{i0}) $p_{iT} \sim N(p_{i0} + T\mu_i, T\sigma_i^2)$.

The term *random walk* was originally used to describe the unpredictable movements of a drunken sailor staggering down the street. The sailor starts at an initial position, p_0 , outside the bar. The sailor generally moves in the direction described by μ but randomly deviates from this direction after each step t by an amount equal to ε_t . After T steps the sailor ends up at position $p_T = p_0 + \mu \cdot T + \sum_{t=1}^T \varepsilon_t$. The sailor is expected to be at location μT , but where he actually ends up depends on the accumulation of the random changes in direction $\sum_{t=1}^T \varepsilon_t$. Because $\text{var}(p_T) = \sigma^2 T$, the uncertainty about where the sailor will be increases with each step.

The RW model for log-prices implies the following model for prices:

$$P_{it} = e^{p_{it}} = P_{i0} e^{\mu_i \cdot t + \sum_{s=1}^t \varepsilon_{is}} = P_{i0} e^{\mu_i t} e^{\sum_{s=1}^t \varepsilon_{is}},$$

where $p_{it} = p_{i0} + \mu_i t + \sum_{s=1}^t \varepsilon_{is}$. Since $P_{it} = e^{p_{it}}$, $P_{it} > 0$. The term $e^{\mu_i t}$ represents the expected exponential growth rate in prices between times 0 and time t , and the term $e^{\sum_{s=1}^t \varepsilon_{is}}$ represents the unexpected exponential growth in prices. Here, conditional on P_{i0} , P_{it} is log-normally distributed because $p_{it} = \ln P_{it}$ is normally distributed.

6.1.5 GWN model for simple returns

For simple returns, defined as $R_{it} = (P_{it} - P_{it-1})/P_{it-1}$, the GWN model is often used for the analysis of portfolios as discussed in chapters [?@sec-Introduction-to-Portfolio-Theory - ?@sec-Statistical-Analysis-of-Portfolios](#). The reason is that the simple return on a portfolio of N assets is weighted average of the simple returns on the individual assets. Hence, the GWN model for simple returns extends naturally to portfolios of assets.

6.1.5.1 GWN Model and Portfolios

Consider the GWN model in matrix form Equation 6.3 for the $N \times 1$ vector of simple returns $\mathbf{R}_t = (R_{1t}, \dots, R_{Nt})'$. For a vector of portfolio weights $\mathbf{x} = (x_1, \dots, x_N)$ such that $\mathbf{x}'\mathbf{1} = \sum_{i=1}^N x_i = 1$, the simple return on the portfolio is:

$$R_{pt} = \mathbf{x}'\mathbf{R}_t = \sum_{i=1}^N x_i R_{it}.$$

Substituting in Equation 6.1 gives the GWN model for the portfolio returns:

$$R_{pt} = \mathbf{x}'(\mu + \varepsilon_t) = \mathbf{x}'\mu + \mathbf{x}'\varepsilon_t = \mu_p + \varepsilon_{pt} \quad (6.7)$$

where $\mu_p = \mathbf{x}'\mu = \sum_{i=1}^N x_i \mu_i$ is the portfolio expected return, and $\varepsilon_{pt} = \mathbf{x}'\varepsilon_t = \sum_{i=1}^N x_i \varepsilon_{it}$ is the portfolio error. The variance of R_{pt} is given by:

$$\text{var}(R_{pt}) = \text{var}(\mathbf{x}'\mathbf{R}_t) = \mathbf{x}'\Sigma\mathbf{x} = \sigma_p^2.$$

Therefore, the distribution of portfolio returns is normal,

$$R_{pt} \sim N(\mu_p, \sigma_p^2).$$

This result is exact for simple returns but is often used as an approximation for cc returns.

6.1.5.2 GWN Model for Multi-Period Simple Returns

The GWN model for single period simple returns does not extend exactly to multi-period simple returns because multi-period simple returns are not additive. Recall, the k -period simple return has a multiplicative relationship to single period returns:

$$\begin{aligned} R_t(k) &= (1 + R_t)(1 + R_{t-1}) \times \dots \times (1 + R_{t-k+1}) - 1 \\ &= R_t + R_{t-1} + \dots + R_{t-k+1} \\ &\quad + R_t R_{t-1} + R_t R_{t-2} + \dots + R_{t-k+2} R_{t-k+1}. \end{aligned}$$

Even though single period returns are normally distributed in the GWN model, multi-period returns are not normally distributed because the product of two normally distributed random variables (e.g., $R_t R_{t-1}$) is not normally distributed. Hence, the GWN model does not exactly generalize to multi-period simple returns. However, if single period returns are small then all of the cross products of

returns are approximately zero ($R_t R_{t-1} \approx \dots \approx R_{t-k+2} R_{t-k+1} \approx 0$) and,

$$\begin{aligned} R_t(k) &\approx R_t + R_{t-1} + \dots + R_{t-k+1} \\ &\approx \mu(k) + \varepsilon_t(k). \end{aligned}$$

where $\mu(k) = k\mu$ and $\varepsilon_t(k) = \sum_{j=0}^{k-1} \varepsilon_{t-j}$. Hence, the GWN model is approximately true for multi-period simple returns when single period simple returns are not too big.

Some exact returns can be derived for the mean and variance of multi-period simple returns. For simplicity, let $k = 2$ so that,

$$R_t(2) = (1 + R_t)(1 + R_{t-1}) - 1 = R_t + R_{t-1} + R_t R_{t-1}.$$

Substituting in Equation 6.1 then gives,

$$\begin{aligned} R_t(2) &= (\mu + \varepsilon_t) + (\mu + \varepsilon_{t-1}) + (\mu + \varepsilon_t)(\mu + \varepsilon_{t-1}) \\ &= 2\mu + \varepsilon_t + \varepsilon_{t-1} + \mu^2 + \mu\varepsilon_t + \mu\varepsilon_{t-1} + \varepsilon_t\varepsilon_{t-1} \\ &= 2\mu + \mu^2 + \varepsilon_t(1 + \mu) + \varepsilon_{t-1}(1 + \mu) + \varepsilon_t\varepsilon_{t-1}. \end{aligned}$$

The result for the expected return is easy,

$$\begin{aligned} E[R_t(2)] &= 2\mu + \mu^2 + (1 + \mu)E[\varepsilon_t] + (1 + \mu)E[\varepsilon_{t-1}] + E[\varepsilon_t\varepsilon_{t-1}] \\ &= 2\mu + \mu^2 = (1 + \mu)^2 - 1, \end{aligned}$$

The result uses the independence of ε_t and ε_{t-1} to get $E[\varepsilon_t\varepsilon_{t-1}] = E[\varepsilon_t]E[\varepsilon_{t-1}] = 0$. The result for the variance, however, is more work:

$$\begin{aligned} \text{var}(R_t(2)) &= \text{var}(\varepsilon_t(1 + \mu) + \varepsilon_{t-1}(1 + \mu) + \varepsilon_t\varepsilon_{t-1}) \\ &= (1 + \mu)^2 \text{var}(\varepsilon_t) + (1 + \mu)^2 \text{var}(\varepsilon_{t-1}) + \text{var}(\varepsilon_t\varepsilon_{t-1}) \\ &\quad + 2(1 + \mu)^2 \text{cov}(\varepsilon_t, \varepsilon_{t-1}) + 2(1 + \mu) \text{cov}(\varepsilon_t, \varepsilon_t\varepsilon_{t-1}) \\ &\quad + 2(1 + \mu) \text{cov}(\varepsilon_{t-1}, \varepsilon_t\varepsilon_{t-1}) \end{aligned}$$

Now, $\text{var}(\varepsilon_t) = \text{var}(\varepsilon_{t-1}) = \sigma^2$ and $\text{cov}(\varepsilon_t, \varepsilon_{t-1}) = 0$. Next, note that,

$$\text{var}(\varepsilon_t\varepsilon_{t-1}) = E[\varepsilon_t^2\varepsilon_{t-1}^2] - (E[\varepsilon_t\varepsilon_{t-1}])^2 = E[\varepsilon_t^2]E[\varepsilon_{t-1}^2] - (E[\varepsilon_t]E[\varepsilon_{t-1}])^2 = 2\sigma^2.$$

Finally,

$$\begin{aligned} \text{cov}(\varepsilon_t, \varepsilon_t\varepsilon_{t-1}) &= E[\varepsilon_t(\varepsilon_t\varepsilon_{t-1})] - E[\varepsilon_t]E[\varepsilon_t\varepsilon_{t-1}] \\ &= E[\varepsilon_t^2]E[\varepsilon_{t-1}] - E[\varepsilon_t]E[\varepsilon_t]E[\varepsilon_{t-1}] \\ &= 0. \end{aligned}$$

Then,

$$\begin{aligned}\text{var}(R_t(2)) &= (1 + \mu)^2 \sigma^2 + (1 + \mu)^2 \sigma^2 + 2\sigma^2 \\ &= 2\sigma^2[(1 + \mu)^2 + 1].\end{aligned}$$

If μ is close to zero then $E[R_t(2)] \approx 2\mu$ and $\text{var}(R_t(2)) \approx 2\sigma^2$ and so the square-root-of-time rule holds approximately.

6.1.5.3 Implied model for prices

Since $R_{it} = (P_{it} - P_{it-1})/P_{it-1}$, you can write

$$P_{it} = P_{it-1}(1 + R_{it})$$

Starting at $t = 1$ and assuming $P_{i0} > 0$ is fixed, by recursive substitution you get

$$P_{it} = P_{i0}(1 + R_{i1})(1 + R_{i2}) \dots (1 + R_{it-1})$$

Prices will be positive provided all returns are greater than -1 . This model for prices is not a RW model but behaves similarly to the RW model if all simple returns are close to zero.

6.2 Monte Carlo Simulation of the GWN Model

A simple technique that can be used to understand the probabilistic behavior of a model involves using computer simulation methods to create pseudo data from the model. The process of creating such pseudo data is called *Monte Carlo simulation*.⁴ Monte Carlo simulation of a model can be used as a *first step reality check* of the model. If simulated data from the model do not look like the data that the model is supposed to describe, then serious doubt is cast on the model. However, if simulated data look reasonably close to the actual data then the first step reality check is passed. Ideally, one should consider many simulated samples from the model because it is possible for a given simulated sample to look strange simply because of an unusual set of random numbers.

Monte Carlo simulations can be used to reveal properties of a model that are hard to derive analytically.

Monte Carlo simulation can also be used to create “what if?” type *scenarios* for a model. Different scenarios typically correspond with different model parameter values.

⁴Monte Carlo refers to the famous city in Monaco where gambling is legal.

Finally, Monte Carlo simulation can be used to study properties of statistics computed from the pseudo data from the model. For example, Monte Carlo simulation can be used to illustrate the concepts of estimator bias and confidence interval coverage probabilities (see Chapter Chapter 9).

To illustrate the use of Monte Carlo simulation, consider creating pseudo return data from the GWN model Equation 6.1 for a single asset. The steps to create a Monte Carlo simulation from the GWN model are:

Monte Carlo Steps: Single Asset

1. Fix values for the GWN model parameters μ and σ .
2. Determine the number of simulated values, T , to create.
3. Use a computer random number generator to simulate T *iid* values of ε_t from a $N(0, \sigma^2)$ distribution. Denote these simulated values as $\tilde{\varepsilon}_1, \dots, \tilde{\varepsilon}_T$.
4. Create the simulated return data $\tilde{R}_t = \mu + \tilde{\varepsilon}_t$ for $t = 1, \dots, T$.

Example 6.1 (Microsoft data to calibrate univariate Monte Carlo simulation of GWN model). To motivate plausible values for μ and σ in the simulation, Figure 6.1 shows the monthly continuously compounded (cc) returns on Microsoft stock over the period January 1998 through May 2012. The data is the same as that used in Chapter Chapter 5 and is constructed from the **IntroCompFinR** "xts" object `msftDailyPrices` as follows:

```
data(msftDailyPrices)
msftPrices = to.monthly(msftDailyPrices, OHLC=FALSE)
smp1 = "1998-01::2012-05"
msftPrices = msftPrices[smp1]
msftRetC = na.omit(Return.calculate(msftPrices, method="log"))
mu.msftRetC = mean(msftRetC)
sigma.msftRetC = sd(msftRetC)
c(mu.msftRetC, sigma.msftRetC)
#> [1] 0.00413 0.10021
```

The parameter $\mu = E[R_t]$ in the GWN model is the expected monthly return, and σ represents the typical size of a deviation about μ . In Figure 6.1, the returns seem to fluctuate up and down about a central value near 0 and the typical size of a return deviation about 0 is roughly 0.10, or 10% (see dashed lines in Figure 6.1). The sample mean turns out to be $\hat{\mu} = 0.004$ (0.4%) and the sample standard deviation is $\hat{\sigma} = 0.100$ (10%). Figure 6.2 shows three distribution summaries (histogram, boxplot and normal qq-plot) and the SACF computed from the sample returns. The returns look to have slightly fatter tails than the normal distribution and show little evidence of linear time dependence (autocorrelation).

■

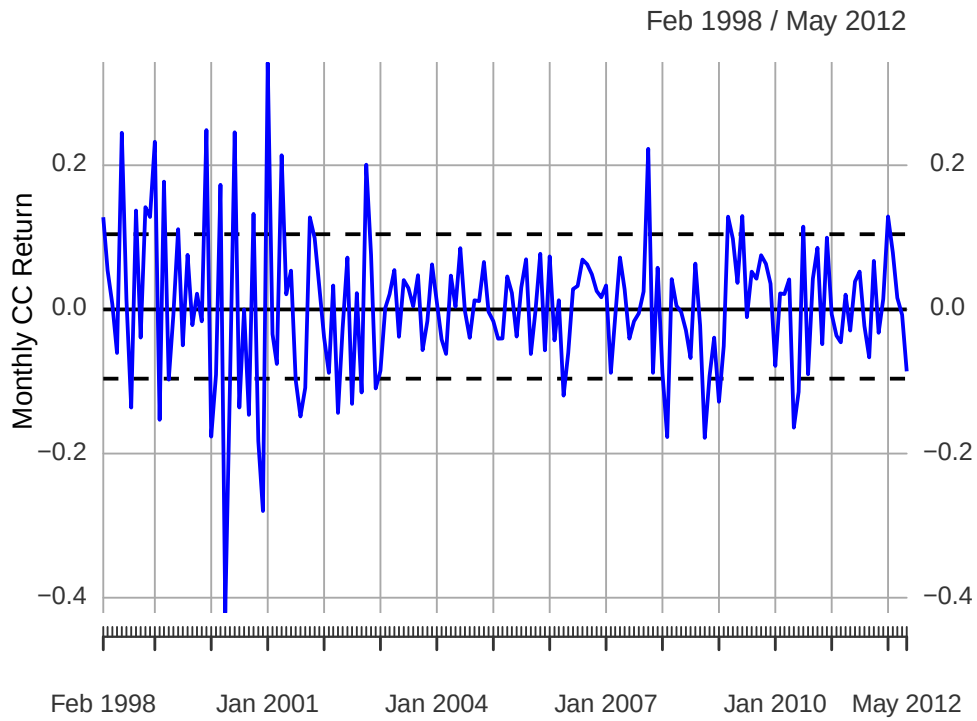


Figure 6.1: Monthly cc returns on Microsoft. Dashed lines indicate $\hat{\mu} \pm \hat{\sigma}$.

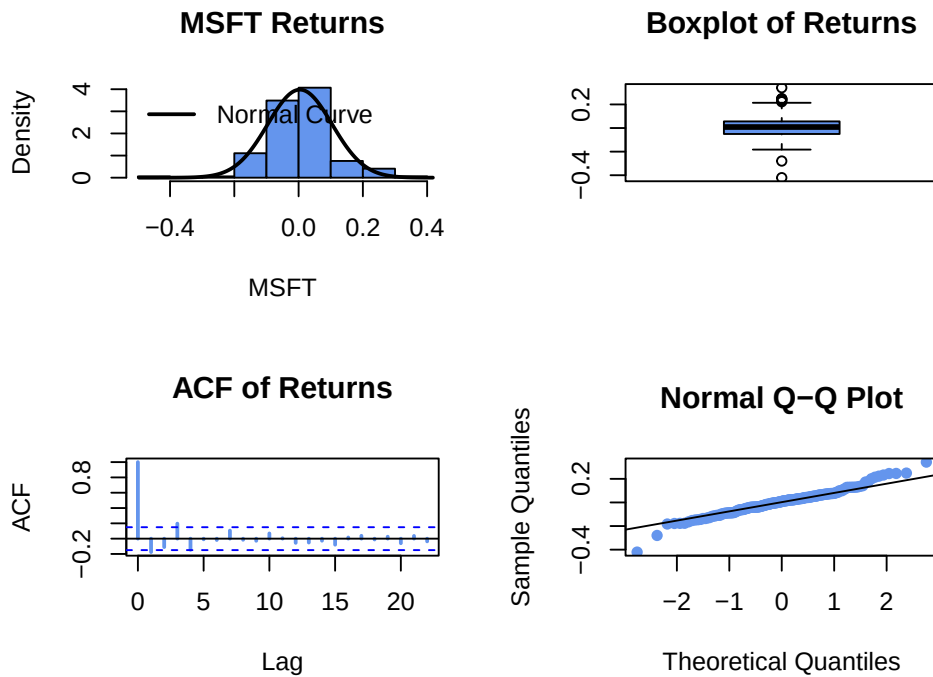


Figure 6.2: Graphical descriptive statistics for the monthly cc returns on Microsoft.

Example 6.2 (Simulating observations from the GWN model). To mimic the monthly return data on Microsoft in the Monte Carlo simulation, the values $\mu = 0.004$ and $\sigma = 0.10$ are used as the model's true parameters and $T = 172$ is the number of simulated values (sample size of actual data). Let $\{\tilde{\varepsilon}_1, \dots, \tilde{\varepsilon}_{172}\}$ denote the 172 simulated values of the random news variable $\varepsilon_t \sim \text{GWN}(0, (0.10)^2)$. The simulated returns are then computed using:⁵

$$\tilde{R}_t = 0.004 + \tilde{\varepsilon}_t, \quad t = 1, \dots, 172 \quad (6.8)$$

To create the simulated returns from Equation 6.8 use:

```
mu = 0.004
sd.e = 0.10
n.obs = length(msftRetC)
set.seed(111)
sim.e = rnorm(n.obs, mean=0, sd=sd.e)
sim.ret = mu + sim.e
sim.ret = xts(sim.ret, index(msftRetC))
```

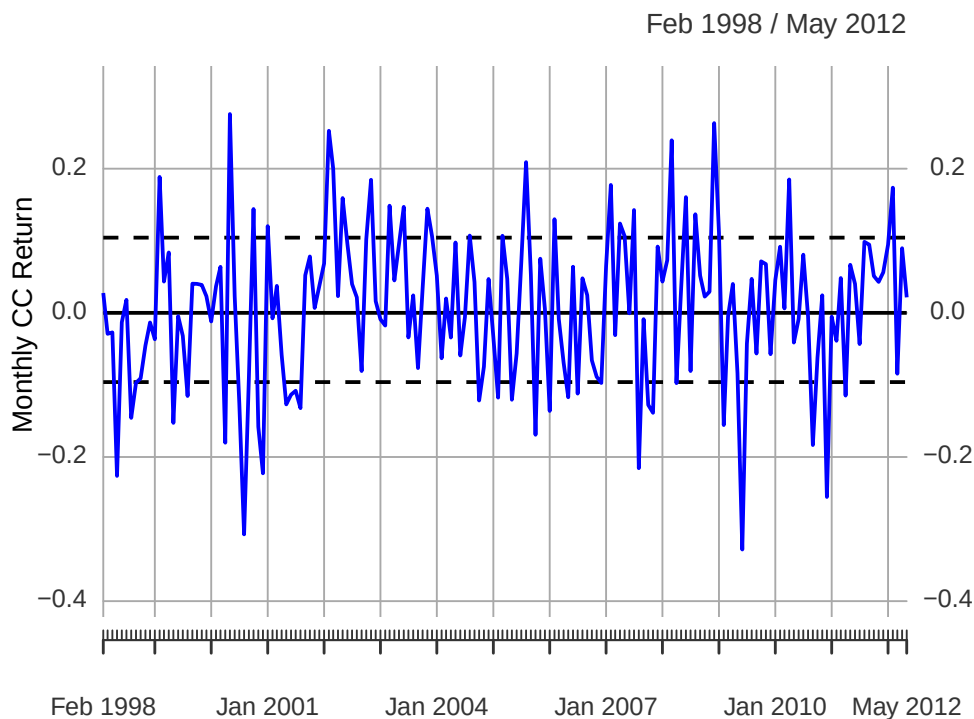


Figure 6.3: Monte Carlo simulated cc returns from the GWN model for Microsoft.

The simulated returns $\{\tilde{R}_t\}_{t=1}^{172}$ (with the same time index as the Microsoft returns) are shown in

⁵Alternatively, the returns can be simulated directly by simulating observations from a normal distribution with mean 0.0 and standard deviation 0.10.

Figure 6.3. The simulated return data fluctuate randomly about $\mu = 0.004$, and the typical size of the fluctuation is approximately equal to $\sigma = 0.10$. The simulated return data look somewhat like the actual monthly return data for Microsoft. The main difference is that the return volatility for Microsoft appears to have decreased in the latter part of the sample whereas the simulated data has constant volatility over the entire sample. Figure 6.4 shows the distribution summaries (histogram, boxplot and normal qq-plot) and the SACF for the simulated returns. The simulated returns are normally distributed and show thinner tails than the actual returns. The simulated returns also show no evidence of linear time dependence (autocorrelation). ■

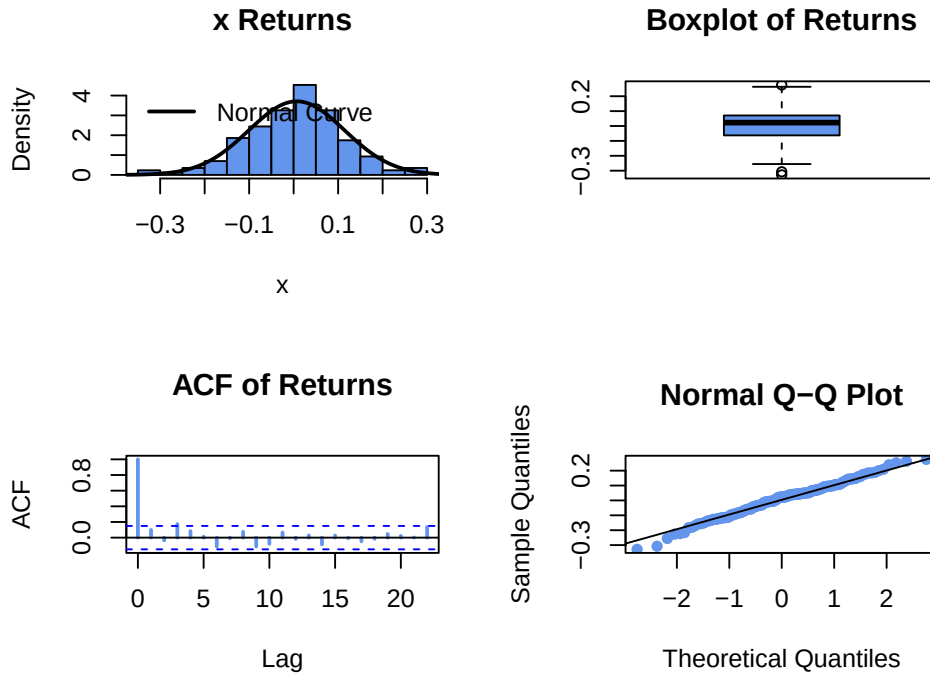


Figure 6.4: Graphical descriptive statistics for the Monte Carlo simulated returns on Microsoft.

Example 6.3 (Simulating log-prices and prices from the RW model). The RW model for log-price based on the GWN model Equation 6.8 calibrated to Microsoft log prices is:

$$p_t = 2.592 + 0.004 \cdot t + \sum_{j=1}^t \varepsilon_j, \varepsilon_j \sim \text{GWN}(0, (0.10)^2).$$

where $p_0 = 2.586 = \ln(13.27)$ is the log of Microsoft Price at the end of January 1998. Given p_t , the price level is determined using $P_t = e^{p_t}$. A Monte Carlo simulation of this RW model with drift can be created in R using:

```

p0 = 2.58
sim.p = p0 + mu*seq(n.obs) + cumsum(sim.e)
sim.p = c(p0, sim.p)
sim.P = exp(sim.p)
sim.p = xts(sim.p, index(msftPrices))
sim.P = xts(sim.P, index(msftPrices))

```

Figure 6.5 shows the simulated log prices and prices. The top panel shows the simulated log price, $\tilde{p}_t$, and the bottom panel shows the simulated price levels $\tilde{P}_t = e^{\tilde{p}_t}$. For comparison, Figure 6.6 shows the actual log prices and price levels for Microsoft stock. Notice the similar behavior of the simulated random walk data and the actual data. ■

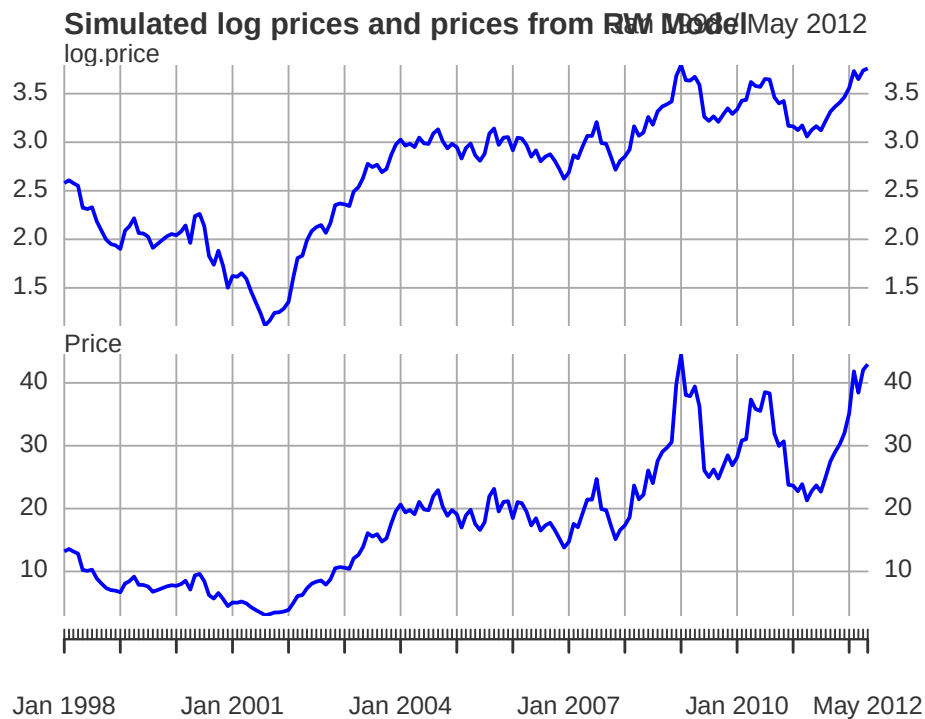


Figure 6.5: Top panel: Simulated log monthly prices of Microsoft from RW model. Bottom panel: Simulated monthly prices of Microsoft stock from RW model.

The previous examples show one Monte Carlo simulation from the GWN model for cc returns. The simulated returns were compared to the actual returns to do the first stage reality check. Here, the simulated returns looked similar to actual returns so the GWN model passed the first stage reality check. However, keep in mind that the simulated returns represent only *one random draw* of size T from the GWN model for cc returns. It is possible that you can, by chance, get a strange set of simulated returns which would lead you to reject the first stage reality check even if the GWN model is a good model for actual returns. To guard against this possibility, you should always

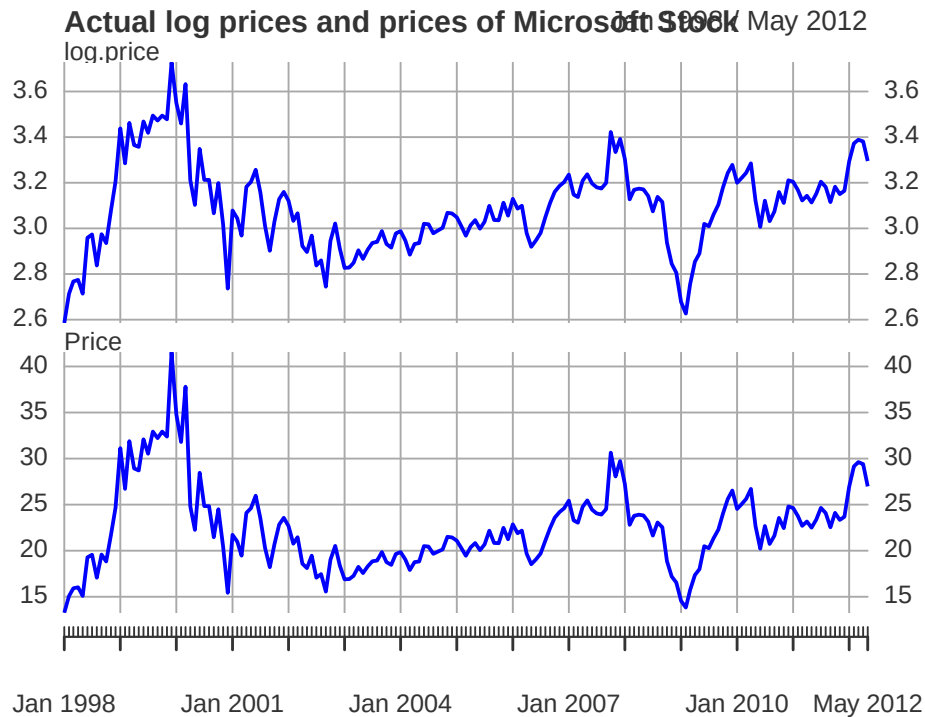


Figure 6.6: Top panel: Actual log monthly prices of Microsoft. Bottom panel: Actual monthly prices of Microsoft stock.

create many Monte Carlo simulations from the proposed model and use the typical behavior from the simulations to do the first step reality check.

Example 6.4 (Creating many simulations from the GWN model). To create ten simulated samples of size $T = 132$ from the GWN model for Microsoft cc returns use:

```
sim.e = matrix(0, n.obs, 10)
set.seed(111)
for (i in 1:10) {
  sim.e[,i] = rnorm(n.obs, mean=0, sd=sd.e)
}
sim.ret = mu + sim.e
sim.ret = xts(sim.ret, index(msftRetC))
colnames(sim.ret) = paste("sim", 1:10, sep=".")
```

The ten simulations are shown in Figure 6.7.

Each panel shows an *alternative reality* for returns over the period February 1998 through May 2012. Each one of these alternative realities is equally likely to happen, and the typical behavior of the simulated returns is similar to actual returns.

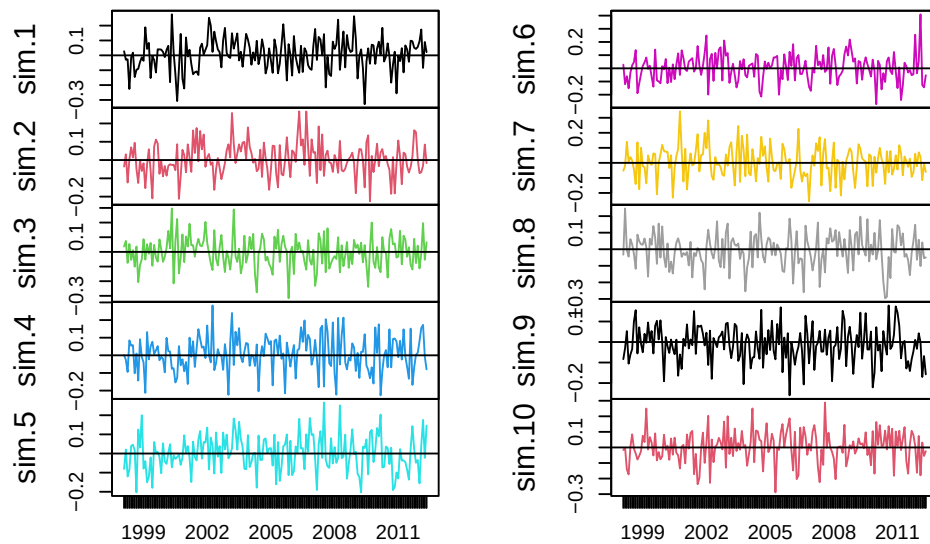


Figure 6.7: Ten Simulations from GWN Model for Microsoft CC Returns.

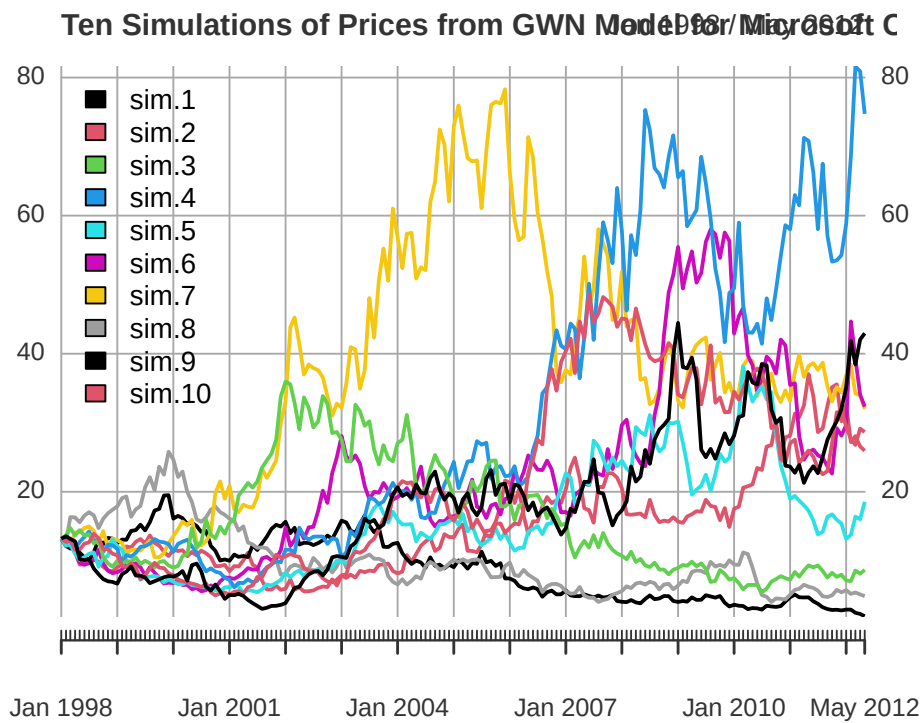


Figure 6.8: Ten Price Simulations from GWN Model for Microsoft CC Returns.

Figure 6.8 shows the same ten simulations from the GWN model in terms of prices. Notice the large variability in prices at the end of the sample. ■

6.2.1 Simulating returns on more than one asset

Creating a Monte Carlo simulation of more than one return from the GWN model requires simulating observations from a multivariate normal distribution. This follows from the matrix representation of the GWN model given in Equation 6.3. The steps required to create a multivariate Monte Carlo simulation are:

Monte Carlo Steps: Multiple Assets

1. Fix values for the $N \times 1$ mean vector μ and the $N \times N$ covariance matrix Σ .
2. Determine the number of simulated values, T , to create.
3. Use a computer random number generator to simulate T *iid* values of the $N \times 1$ random vector ε_t from the multivariate normal distribution $N(\mathbf{0}, \Sigma)$. Denote these simulated vectors as $\tilde{\varepsilon}_1, \dots, \tilde{\varepsilon}_T$.
4. Create the $N \times 1$ simulated return vector $\tilde{\mathbf{R}}_t = \mu + \tilde{\varepsilon}_t$ for $t = 1, \dots, T$.
5. Repeat steps 1-4 to create multiple simulations (alternative realities).

Example 6.5 (Microsoft Starbucks and S&P 500 data to calibrate multivariate Monte Carlo simulation of GWN model). To motivate the parameters for a multivariate simulation of the GWN model, consider the monthly cc returns for Microsoft, Starbucks and the S&P 500 index over the period January 1998 through May 2012 illustrated in Figures Figure 6.9 and Figure 6.10. The data is assembled using the R commands:

```
data(sp500DailyPrices, sbuxDailyPrices)
sp500Prices = to.monthly(sp500DailyPrices, OHLC=FALSE)
sbuxPrices = to.monthly(sbuxDailyPrices, OHLC=FALSE)
sp500Prices = sp500Prices[smp1]
sbuxPrices = sbuxPrices[smp1]
sp500RetC = na.omit(Return.calculate(sp500Prices, method="log"))
sbuxRetC = na.omit(Return.calculate(sbuxPrices, method="log"))
gwnRetC = merge(msftRetC, sbuxRetC, sp500RetC)
colnames(gwnRetC) = c("MSFT", "SBUX", "SP500")
```

The multivariate sample descriptive statistics (mean vector, standard deviation vector, covariance matrix and correlation matrix) are:

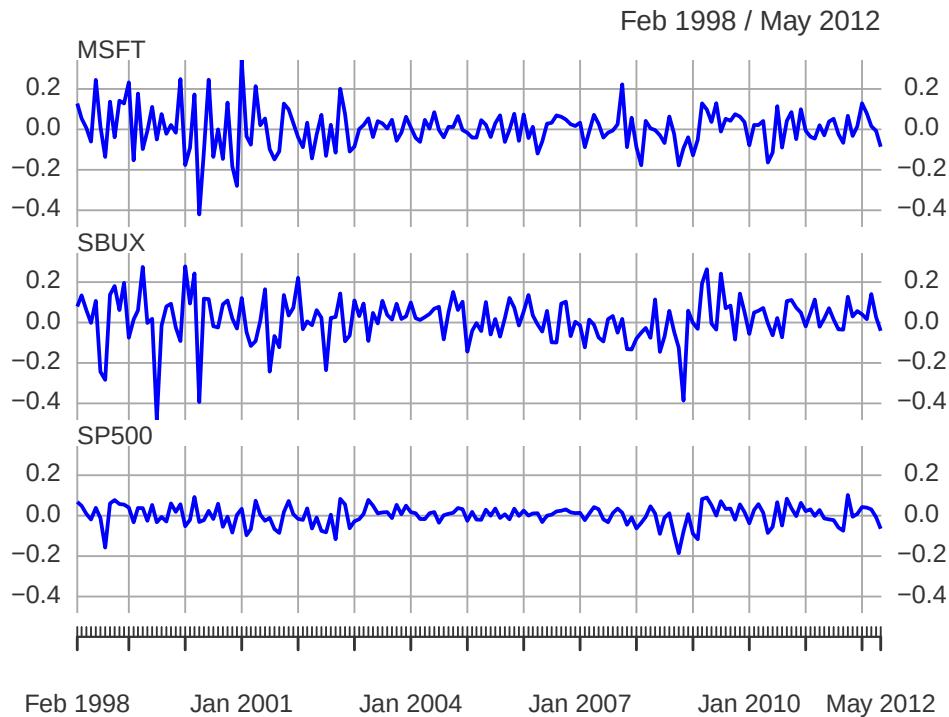


Figure 6.9: Monthly cc returns on Microsoft, Starbucks, and the S&P 500 Index.

```
mean.vals = apply(gwnRetC, 2, mean)
sd.vals = apply(gwnRetC, 2, sd)
rbind(mean.vals, sd.vals)
#>           MSFT  SBUX  SP500
#> mean.vals 0.00413 0.0147 0.00169
#> sd.vals   0.10021 0.1116 0.04847
cov(gwnRetC)
#>           MSFT  SBUX  SP500
#> MSFT  0.01004 0.00381 0.00300
#> SBUX  0.00381 0.01246 0.00248
#> SP500 0.00300 0.00248 0.00235
cor(gwnRetC)
#>           MSFT  SBUX  SP500
#> MSFT  1.000 0.341 0.617
#> SBUX  0.341 1.000 0.457
#> SP500 0.617 0.457 1.000
```

All returns fluctuate around mean values close to zero. The volatilities of Microsoft and Starbucks are similar with typical magnitudes around 0.10, or 10%. The volatility of the S&P 500 index is considerably smaller at about 0.05, or 5%. The pairwise scatterplots show that all returns are

positively related. The pairs (MSFT, SP500) and (SBUX, SP500) are the most correlated with sample correlation values around 0.5. The pair (MSFT, SBUX) has a moderate positive correlation around 0.3. ■

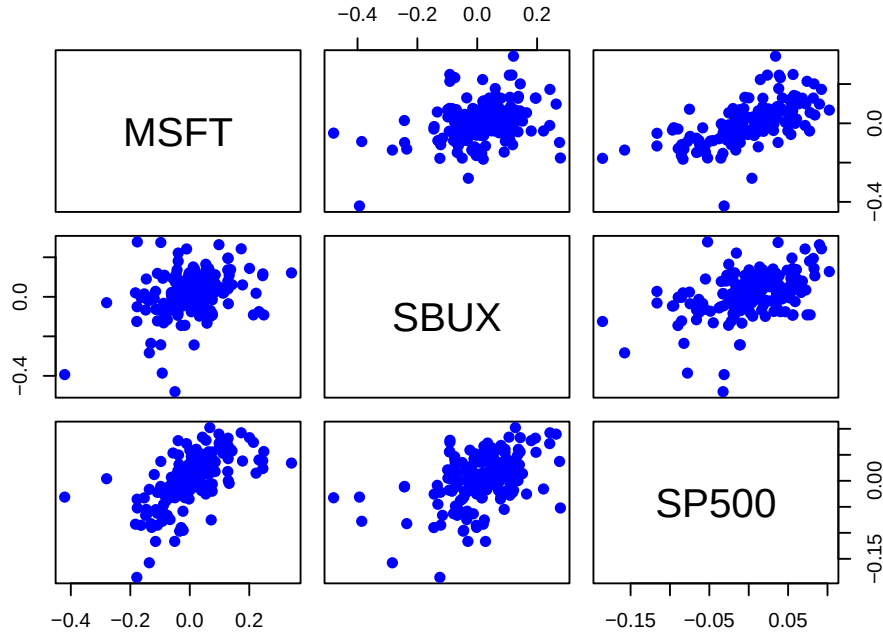


Figure 6.10: Pairwise scatterplots of the monthly returns on Microsoft, Starbucks, and the S&P 500 Index.

Example 6.6 (Monte Carlo simulation of GWN model for three assets). Simulating values from the multivariate GWN model Equation 6.3 requires simulating multivariate normal random variables. In R, this can be done using the function `rmvnorm()` from the package `mvtnorm`. The function `rmvnorm()` requires a vector of mean values and a covariance matrix. Define:

$$\mathbf{R}_t = \begin{pmatrix} R_{\text{msft},t} \\ R_{\text{sbux},t} \\ R_{\text{sp500},t} \end{pmatrix}, \quad \mu = \begin{pmatrix} \mu_{\text{msft},t} \\ \mu_{\text{sbux},t} \\ \mu_{\text{sp500},t} \end{pmatrix}, \quad \Sigma = \begin{pmatrix} \sigma_{\text{msft}}^2 & \sigma_{\text{msft},\text{sbux}} & \sigma_{\text{msft},\text{sp500}} \\ \sigma_{\text{msft},\text{sbux}} & \sigma_{\text{sbux}}^2 & \sigma_{\text{sbux},\text{sp500}} \\ \sigma_{\text{msft},\text{sp500}} & \sigma_{\text{sbux},\text{sp500}} & \sigma_{\text{sp500}}^2 \end{pmatrix}$$

The parameters μ and Σ of the multivariate GWN model are set equal to the sample mean vector μ and sample covariance matrix Σ :

```
muVec = apply(gwnRetC, 2, mean)
covMat = cov(gwnRetC)
```

To create a Monte Carlo simulation from the GWN model calibrated to the month continuously

returns on Microsoft, Starbucks and the S&P 500 index use:⁶

```
set.seed(123)
e.sim = rmvnorm(n.obs, sigma=covMat)
returns.sim = muVec + t(e.sim)
returns.sim = t(returns.sim)
colnames(returns.sim) = colnames(gwnRetC)
returns.sim = xts(returns.sim, index(gwnRetC))
```

The simulated returns are shown in Figure 6.11 and Figure 6.12. They look similar to the actual returns shown in Figure 6.9 and Figure 6.10. The actual returns show periods of high and low volatility that the simulated returns do not. The sample statistics from the simulated returns, however, are close to the sample statistics of the actual data:

```
mean.vals = apply(returns.sim, 2, mean)
sd.vals = apply(returns.sim, 2, sd)
rbind(mean.vals, sd.vals)
#>           MSFT    SBUX    SP500
#> mean.vals 0.00671 0.0138 0.00508
#> sd.vals   0.09509 0.1051 0.04601
cov(returns.sim)
#>           MSFT    SBUX    SP500
#> MSFT  0.00904 0.00353 0.00246
#> SBUX  0.00353 0.01105 0.00194
#> SP500 0.00246 0.00194 0.00212
cor(returns.sim)
#>           MSFT    SBUX    SP500
#> MSFT  1.000 0.354 0.563
#> SBUX  0.354 1.000 0.402
#> SP500 0.563 0.402 1.000
```

■

Example 6.7 (Multiple Monte Carlo simulations of GWN model for three assets). TBD

■

⁶`rmvnorm` returns a matrix of dimension `n.obs` by 3. The simulated news shocks for each asset are in the columns of `e.sim`. Transposing `e.sim` allows the mean vector to be added row-wise. Transposing again creates a column vector of simulated returns.

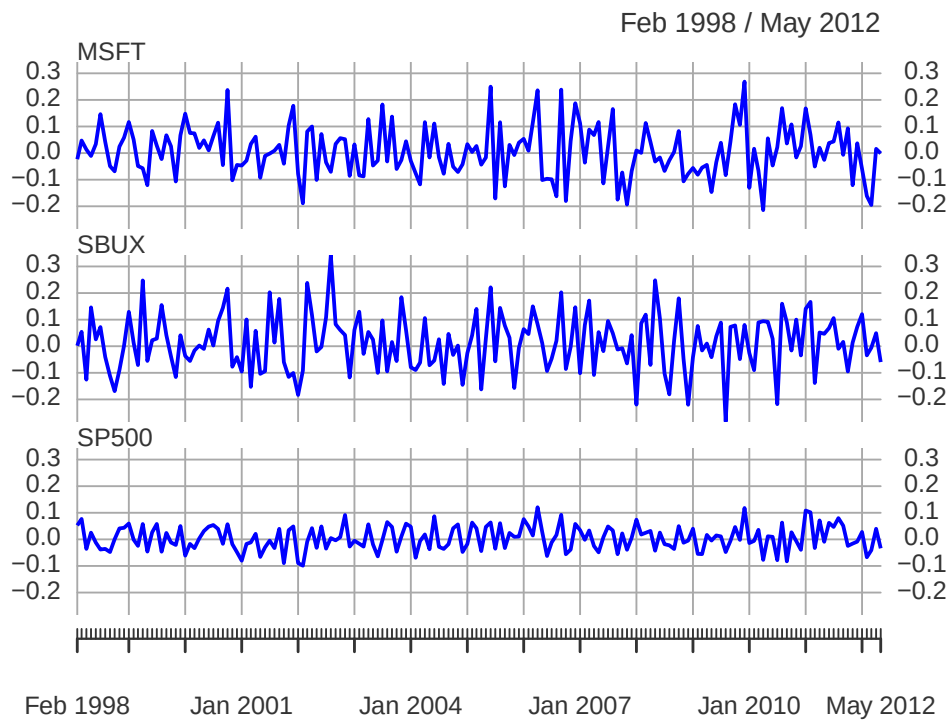


Figure 6.11: Simulated GWN model returns on Microsoft, Starbucks and the S&P 500 Index.

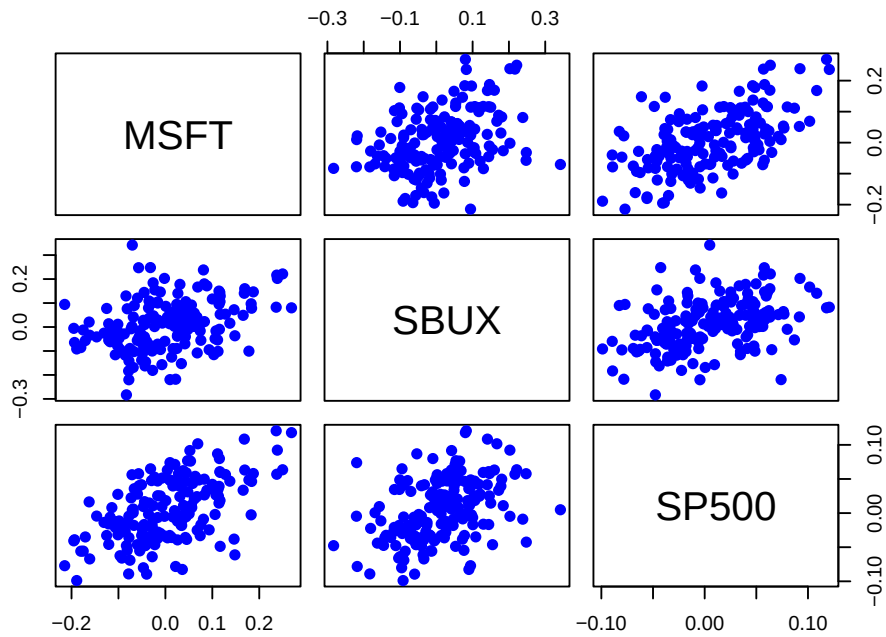


Figure 6.12: Pairwise scatterplot of simulated returns

6.3 Scenario Analysis

TBD

6.4 Further Reading: The GWN Return Model

The location-scale form of the GWN model Equation 6.2 is commonly used as a starting point for the distributional analysis of returns. In this chapter, and throughout most of the book, I use the standard normal distribution for the standardized news shock. However, many other standardized distributions that are additive (e.g., skew-normal, Student's t, skewed-t, etc.) can also be used as illustrated in (Jondeau, Poon, and Rockinger 2007), (Pfaff 2013), (Carmona 2014), and (Ruppert and Matteson 2015).

xxx gives as comprehensive non-technical description of the RW model for stock prices.

6.5 Exercises

Exercise 6.1. Consider the GWN model for cc returns:

$$r_t = \mu + \epsilon_t, \epsilon_t \sim \text{GWN}(0, \sigma^2).$$

It implies that the log price follows a random walk with drift:

$$\ln P_t = \ln P_{t-1} + r_t = \ln P_0 + \mu t + \sum_{s=1}^t \epsilon_s.$$

Show that $E[\ln P_t]$ and $\text{var}[\ln P_t]$ depend on t so that $\ln P_t$ is non-stationary.

Exercise 6.2. Consider using Monte Carlo simulation to evaluate the GWN model for the monthly simple returns on Amazon stock and the S&P 500 index over the 5 year period January 2009, through January 2014. 1. Daily adjusted closing prices on Amazon stock and the S&P 500 index are in the **introcompfinr** "xts" objects `amznDailyPrices` and `sp500DailyPrices`, respectively. Using these prices, create monthly returns over the period January 2009, through January 2014. 2. Create a time plot showing the monthly returns on the two assets. Do the monthly returns from the two assets look like realizations from a covariance stationary stochastic process? Why or why not? 3. Compare and contrast the return characteristics of the two assets. In addition, comment on any common features, if any, of the two return series. 4. Using the **introcompfinr** function `four_panel_plot()`, create graphical descriptive statistics for the monthly returns. Also, create the scatterplot between the two return series. Do the returns look normally distributed? Is there any evidence for linear time dependence? Do the return appear to contemporaneously

correlated? 5. Compute sample descriptive statistics (mean, standard deviation, skewness, kurtosis, and correlation between Amazon and the S&P 500 index). 6. In the GWN model for simple returns prices do not follow a random walk but instead evolve over time according to

$$P_t = P_{t-1}(1 + R_t)$$

7 Estimation of The GWN Model

Updated: September 5, 2026

Copyright © Eric Zivot 2026

The GWN model of asset returns presented in the previous chapter gives you a simple framework for interpreting the time series behavior of asset returns and prices. At the beginning of time $t - 1$, $\mathbf{R}_t$ is an $N \times 1$ random vector representing the returns (simple or continuously compounded) on assets $i = 1, \dots, N$ to be realized at time t . The GWN model states that $\mathbf{R}_t \sim iid N(\mu, \Sigma)$. Your best guess for the return at t on asset i is $E[R_{it}] = \mu_i$, your measure of uncertainty about our best guess is captured by $sd(R_{it}) = \sigma_i$, and your measures of the direction and strength of linear association between R_{it} and R_{jt} are $\sigma_{ij} = \text{cov}(R_{it}, R_{jt})$ and $\rho_{ij} = \text{cor}(R_{it}, R_{jt})$, respectively. The GWN model assumes that the economic environment is constant over time (i.e., covariance stationary) so that the multivariate normal distribution characterizing returns is the same for all time periods t .

Your life would be very easy if you knew the exact values of $\mu_i, \sigma_i^2, \sigma_{ij}$ and ρ_{ij} : the parameters of the GWN model. Then you could use the GWN model for risk and portfolio analysis. In actuality, however, you do not know these values with certainty. Therefore, a key task in *financial econometrics* is estimating these values from a history of observed return data. Given estimates of the GWN model parameters you can then apply the model to risk and portfolio analysis.

Suppose you observe returns on N different assets over the sample $t = 1, \dots, T$. Denote this sample $\{\mathbf{r}_1, \dots, \mathbf{r}_T\} = \{\mathbf{r}_t\}_{t=1}^T$, where $\mathbf{r}_t = (r_{1t}, \dots, r_{Nt})'$ is the $N \times 1$ vector of returns on N assets observed at time t . It is assumed that the observed returns are realizations of the random variables $\{\mathbf{R}_t\}_{t=1}^T$, where $\mathbf{R}_t = (R_{1t}, \dots, R_{Nt})'$ is a vector of N asset returns described by the GWN model:

$$\mathbf{R}_t = \mu + \varepsilon_t, \quad \varepsilon_t \sim \text{GWN}(\mathbf{0}, \Sigma). \quad (7.1)$$

Under these assumptions, you can use the observed returns $\{\mathbf{r}_t\}_{t=1}^T$ to estimate the unknown parameters in μ and Σ of the GWN model. However, before you describe the estimation of the GWN model in detail, it is necessary to review some fundamental concepts in the statistical theory of estimation. This is important because estimates of parameters have *estimation error* and statistical theory allows you to evaluate characteristics of estimation error and to develop quantitative measures of the typical magnitude of estimation error. As you shall see, some parameters of the

GWN model are precisely estimated (have small estimation error) and some parameters are not (have large estimation error).

The chapter is outlined as follows. In Section 7.1 I review some of the statistical theory of estimation and discusses some important finite sample properties of estimators such as bias and precision. For many estimators it is difficult to derive exact finite sample properties which motivates the need to look at asymptotic properties of estimators. These are properties that hold exactly as the sample size goes to infinity but are used as approximations for a finite sample size. I present estimators of the GWN model parameters in Section 7.4 and I investigate the statistical properties of these estimators in Section 7.5. In Section 7.6 I illustrates how Monte Carlo simulation can be used to evaluate and understand the statistical properties of the GWN model estimators.

To be added:

- Which asymptotic distribution formulas depend on the normality assumption? Make clear which results depend on GWN model and which results do not.

In this chapter I use the R packages **introcompfinr**, **mvtnorm**, and **PerformanceAnalytics**. Make sure these packages are installed and loaded prior to running the R examples in the chapter.

7.1 Estimators and Estimates

Let R_t be the return on a single asset (simple or continuously compounded) described by the GWN model and let θ denote some characteristic (parameter) of the GWN model you are interested in estimating. For simplicity, assume that $\theta \in \mathbb{R}$ is a single parameter. For example, if you are interested in the expected return on the asset, then $\theta = \mu$; if you are interested in the variance of the asset returns, then $\theta = \sigma^2$; if you are interested in the first lag autocorrelation of returns then $\theta = \rho_1$. The goal is to estimate θ based on a sample of size T of the observed data when you believe the data is generated from the GWN return model.

Definition 7.1 (Estimator). Let $\{R_1, \dots, R_T\}$ denote a collection of T random returns from the GWN model, and let θ denote some characteristic of the model. An *estimator* of θ , denoted $\hat{\theta}$, is a rule or algorithm for estimating θ as a function of the random variables $\{R_1, \dots, R_T\}$. Here, $\hat{\theta}$ is a random variable.

Definition 7.2 (Estimate). Let $\{r_1, \dots, r_T\}$ denote an observed sample of size T from the GWN model, and let θ denote some characteristic of the model. An *estimate* of θ , denoted $\hat{\theta}$, is simply the value of the estimator for θ based on the observed sample $\{r_1, \dots, r_T\}$. Here, $\hat{\theta}$ is a number.

Example 7.1 (The sample average as an estimator and an estimate). Let R_t be the return on a single asset described by the GWN model, and suppose you are interested in estimating $\theta =$

$\mu = E[R_t]$ from the sample of observed returns $\{r_t\}_{t=1}^T$. The sample average is an algorithm for computing an estimate of the expected return μ . Before the sample is observed, you can think of $\hat{\mu} = \frac{1}{T} \sum_{t=1}^T R_t$ as a simple linear function of the random variables $\{R_t\}_{t=1}^T$ and so is itself a random variable. You can study the properties of $\hat{\mu}$ using the tools of probability theory presented in chapter 2.

After the sample is observed, the sample average can be evaluated using the observed data $\{r_t\}_{t=1}^T$ giving $\hat{\mu} = \frac{1}{T} \sum_{t=1}^T r_t$, which produces the numerical estimate of μ . For example, suppose $T = 5$ and the realized values of the returns are $r_1 = 0.1, r_2 = 0.05, r_3 = 0.025, r_4 = -0.1, r_5 = -0.05$. Then the estimate of μ using the sample average is:

$$\hat{\mu} = \frac{1}{5}(0.1 + 0.05 + 0.025 + -0.1 + -0.05) = 0.005.$$

■

The example above illustrates the distinction between an estimator and an estimate of a parameter θ . However, typically in the statistics literature you use the same symbol, $\hat{\theta}$, to denote both an estimator and an estimate. When $\hat{\theta}$ is treated as a function of the random returns it denotes an estimator and is a random variable. When $\hat{\theta}$ is evaluated using the observed data it denotes an estimate and is simply a number. The context in which you discuss $\hat{\theta}$ will determine how to interpret it.

7.2 Finite Sample Properties of Estimators

Consider $\hat{\theta}$ as a random variable. In general, the pdf of $\hat{\theta}$, $f(\hat{\theta})$, depends on the pdf's of the random variables $\{R_t\}_{t=1}^T$. The exact form of $f(\hat{\theta})$ may be very complicated. Sometimes you can use analytic calculations to determine the exact form of $f(\hat{\theta})$. This can be done, for example, when $\hat{\theta} = \hat{\mu}$. In general, the exact form of $f(\hat{\theta})$ is often too difficult to derive exactly. When $f(\hat{\theta})$ is too difficult to compute you can often approximate $f(\hat{\theta})$ using either *Monte Carlo simulation* techniques or the *Central Limit Theorem* (CLT). In Monte Carlo simulation, you use the computer to simulate many different realizations of the random returns $\{R_t\}_{t=1}^T$ and on each simulated sample you evaluate the estimator $\hat{\theta}$. The Monte Carlo approximation of $f(\hat{\theta})$ is the empirical distribution of $\hat{\theta}$ over the different simulated samples. For a given sample size T , Monte Carlo simulation gives a very accurate approximation to $f(\hat{\theta})$ if the number of simulated samples is very large. The CLT approximation of $f(\hat{\theta})$ is a normal distribution approximation that becomes more accurate as the sample size T gets very large. An advantage of the CLT approximation is that it is often easy to compute. The disadvantage is that the accuracy of the approximation depends on the estimator $\hat{\theta}$ and sample size T .

For analysis purposes, you often focus on certain characteristics of $f(\hat{\theta})$, like its expected value (center), variance and standard deviation (spread about expected value). The expected value of

an estimator is related to the concept of *estimator bias*, and the variance/standard deviation of an estimator is related to the concept of *estimator precision*. Different realizations of the random variables $\{R_t\}_{t=1}^T$ will produce different values of $\hat{\theta}$. Some values of $\hat{\theta}$ will be bigger than θ and some will be smaller. Intuitively, a *good estimator* of θ is one that is on average correct (unbiased) and never gets too far away from θ (small variance). That is, a good estimator will have small bias and high precision.

7.2.1 Bias

Bias concerns the location or center of $f(\hat{\theta})$ in relation to θ . If $f(\hat{\theta})$ is centered away from θ , then you say $\hat{\theta}$ is a *biased* estimator of θ . If $f(\hat{\theta})$ is centered at θ , then you say that $\hat{\theta}$ is an *unbiased* estimator of θ . Formally, you have the following definitions:

Definition 7.3 (Estimation Error). The *estimation error* is the difference between the estimator and the parameter being estimated:

$$\text{error}(\hat{\theta}, \theta) = \hat{\theta} - \theta. \quad (7.2)$$

Definition 7.4 (bias). The *bias* of an estimator $\hat{\theta}$ of θ is the expected estimation error:

$$\text{bias}(\hat{\theta}, \theta) = E[\text{error}(\hat{\theta}, \theta)] = E[\hat{\theta}] - \theta. \quad (7.3)$$

Definition 7.5 (Unbiased Estimator). An estimator $\hat{\theta}$ of θ is *unbiased* if $\text{bias}(\hat{\theta}, \theta) = 0$; i.e., if $E[\hat{\theta}] = \theta$ or $E[\text{error}(\hat{\theta}, \theta)] = 0$.

Unbiasedness is a desirable property of an estimator. It means that the estimator produces the correct answer “on average”, where “on average” means over many hypothetical realizations of the random variables $\{R_t\}_{t=1}^T$. It is important to keep in mind that an unbiased estimator for θ may not be very close to θ for a particular sample, and that a biased estimator may actually be quite close to θ . For example, consider two estimators of θ , $\hat{\theta}_1$ and $\hat{\theta}_2$. The pdfs of $\hat{\theta}_1$ and $\hat{\theta}_2$ are illustrated in Figure 7.1. $\hat{\theta}_1$ is an unbiased estimator of θ with a large variance, and $\hat{\theta}_2$ is a biased estimator of θ with a small variance. Consider first, the pdf of $\hat{\theta}_1$. The center of the distribution is at the true value $\theta = 0$, $E[\hat{\theta}_1] = 0$, but the distribution is very widely spread out about $\theta = 0$. That is, $\text{var}(\hat{\theta}_1)$ is large. On average (over many hypothetical samples) the value of $\hat{\theta}_1$ will be close to θ , but in any given sample the value of $\hat{\theta}_1$ can be quite a bit above or below θ . Hence, unbiasedness by itself does not guarantee a good estimator of θ . Now consider the pdf for $\hat{\theta}_2$. The center of the pdf is slightly higher than $\theta = 0$, i.e., $\text{bias}(\hat{\theta}_2, \theta) > 0$, but the spread of the distribution is small. Although the value of $\hat{\theta}_2$ is not equal to 0 *on average* you might prefer the estimator $\hat{\theta}_2$ over $\hat{\theta}_1$ because it is generally closer to $\theta = 0$ *on average* than $\hat{\theta}_1$.

While unbiasedness is a desirable property of an estimator $\hat{\theta}$ of θ , it, by itself, is not enough to determine if $\hat{\theta}$ is a good estimator. Being correct on average means that $\hat{\theta}$ is seldom exactly correct for any given sample. In some samples $\hat{\theta}$ is less than θ , and some samples $\hat{\theta}$ is greater than θ . More importantly, you need to know how far $\hat{\theta}$ typically is from θ . That is, you need to know about the magnitude of the spread of the distribution of $\hat{\theta}$ about its average value. This will tell you the precision of $\hat{\theta}$.

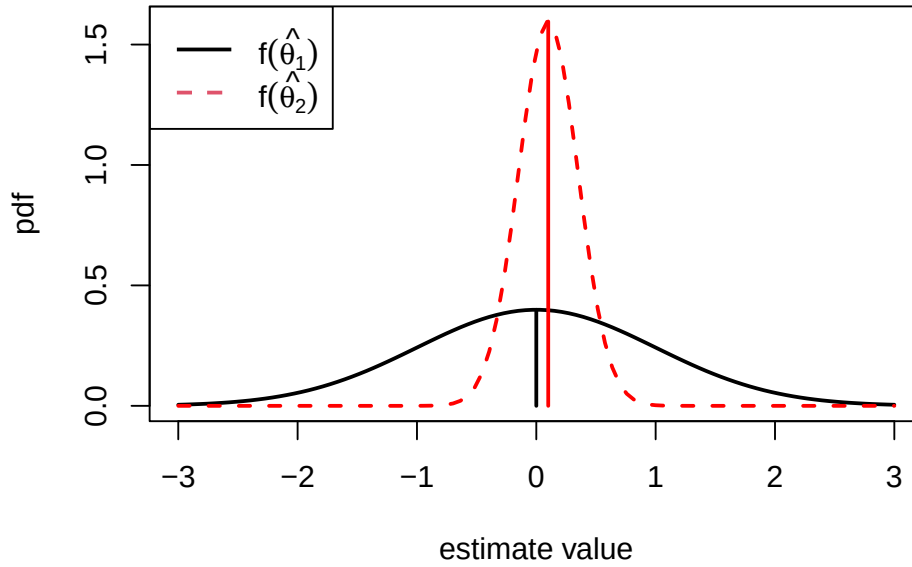


Figure 7.1: Distributions of competing estimators for $\theta = 0$. $\hat{\theta}_1$ is unbiased but has high variance, and $\hat{\theta}_2$ is biased but has low variance.

7.2.2 Precision

An estimate is, hopefully, our best guess of the true (but unknown) value of θ . Our guess most certainly will be wrong, but you hope it will not be too far off. A *precise estimate* is one in which the variability of the estimation error is small. The variability of the estimation error is captured by the *mean squared error*.

Definition 7.6 (Mean Squared Error). The *mean squared error* of an estimator $\hat{\theta}$ of θ is given by:

$$\text{mse}(\hat{\theta}, \theta) = E[(\hat{\theta} - \theta)^2] = E[\text{error}(\hat{\theta}, \theta)^2] \quad (7.4)$$

The mean squared error measures the expected squared deviation of $\hat{\theta}$ from θ . If this expected

deviation is small, then you know that $\hat{\theta}$ will almost always be close to θ . Alternatively, if the mean squared is large then it is possible to see samples for which $\hat{\theta}$ is quite far from θ . A useful decomposition of $\text{mse}(\hat{\theta}, \theta)$ is:

$$\text{mse}(\hat{\theta}, \theta) = E[(\hat{\theta} - E[\hat{\theta}])^2] + (E[\hat{\theta}] - \theta)^2 = \text{var}(\hat{\theta}) + \text{bias}(\hat{\theta}, \theta)^2$$

The derivation of this result is straightforward. Use the *add and subtract trick* and write

$$\hat{\theta} - \theta = \hat{\theta} - E[\hat{\theta}] + E[\hat{\theta}] - \theta.$$

Then square both sides giving

$$(\hat{\theta} - \theta)^2 = (\hat{\theta} - E[\hat{\theta}])^2 + 2(\hat{\theta} - E[\hat{\theta}])(E[\hat{\theta}] - \theta) + (E[\hat{\theta}] - \theta)^2.$$

Taking expectations of both sides yields,

$$\begin{aligned} \text{mse}(\hat{\theta}, \theta) &= E[(\hat{\theta} - E[\hat{\theta}])^2] + 2(E[\hat{\theta}] - E[\hat{\theta}])(E[\hat{\theta}] - \theta) + E[(E[\hat{\theta}] - \theta)^2] \\ &= E[(\hat{\theta} - E[\hat{\theta}])^2] + E[(E[\hat{\theta}] - \theta)^2] \\ &= \text{var}(\hat{\theta}) + \text{bias}(\hat{\theta}, \theta)^2. \end{aligned}$$

The result states that for any estimator $\hat{\theta}$ of θ , $\text{mse}(\hat{\theta}, \theta)$ can be split into a variance component, $\text{var}(\hat{\theta})$, and a squared bias component, $\text{bias}(\hat{\theta}, \theta)^2$. Clearly, $\text{mse}(\hat{\theta}, \theta)$ will be small only if both components are small. If an estimator is unbiased then $\text{mse}(\hat{\theta}, \theta) = \text{var}(\hat{\theta}) = E[(\hat{\theta} - \theta)^2]$ is just the squared deviation of $\hat{\theta}$ about θ . Hence, an unbiased estimator $\hat{\theta}$ of θ is *good*, if it has a small variance.

The $\text{mse}(\hat{\theta}, \theta)$ and $\text{var}(\hat{\theta})$ are based on squared deviations and so are not in the same units of measurement as θ . Measures of precision that are in the same units as θ are the *root mean squared error* and the *standard error*.

Definition 7.7 (Root Mean Squared Error and Standard Error). The *root mean squared error* and the *standard error* of an estimator $\hat{\theta}$ are:

$$\text{rmse}(\hat{\theta}, \theta) = \sqrt{\text{mse}(\hat{\theta}, \theta)}, \tag{7.5}$$

$$\text{se}(\hat{\theta}) = \sqrt{\text{var}(\hat{\theta})}. \tag{7.6}$$

If $\text{bias}(\hat{\theta}, \theta) \approx 0$ then the precision of $\hat{\theta}$ is typically measured by $\text{se}(\hat{\theta})$.

With the concepts of bias and precision in hand, I can state what defines a good estimator.

Definition 7.8 (Good Estimator). A good estimator $\hat{\theta}$ of θ has a small bias Equation 7.3 and a small standard error Equation 7.6.

7.2.3 Estimated standard errors

Standard error formulas $\text{se}(\hat{\theta})$ that I will derive in this chapter often depend on the unknown values of the GWN model parameters. For example, later on I will show that $\text{se}(\hat{\mu}) = \sigma^2/T$ which depends on the GWN parameter σ^2 which is unknown. Hence, the formula for $\text{se}(\hat{\mu})$ is a *practically useless* quantity as I cannot compute its value because I don't know the value of σ^2 . Fortunately, I can create a *practically useful* formula by replacing the unknown quantity σ^2 with a good estimate $\hat{\sigma}^2$ that I compute from the data. This gives rise the *estimated standard error* $\widehat{\text{se}}(\hat{\mu}) = \hat{\sigma}^2/T$.

7.3 Asymptotic Properties of Estimators

Estimator bias and precision are finite sample properties. That is, they are properties that hold for a fixed sample size T . Very often you are also interested in properties of estimators when the sample size T gets very large. For example, analytic calculations may show that the bias and mse of an estimator $\hat{\theta}$ depend on T in a decreasing way. That is, as T gets very large the bias and mse approach zero. So for a very large sample, $\hat{\theta}$ is effectively unbiased with high precision. In this case I say that $\hat{\theta}$ is a *consistent* estimator of θ . In addition, for large samples the *Central Limit Theorem* (CLT) says that $f(\hat{\theta})$ can often be well approximated by a normal distribution. In this case, I say that $\hat{\theta}$ is *asymptotically normally distributed*. The word “asymptotic” means “in an infinitely large sample” or “as the sample size T goes to infinity”. Of course, in the real world I don't have an infinitely large sample and so the asymptotic results are only approximations. How good these approximations are for a given sample size T depends on the context. Monte Carlo simulations (see Section 7.6) can often be used to evaluate asymptotic approximations in a given context.

7.3.1 Consistency

Let $\hat{\theta}$ be an estimator of θ based on the random returns $\{R_t\}_{t=1}^T$.

Definition 7.9 (Consistent Estimator). $\hat{\theta}$ is consistent for θ (converges in probability to θ) if for any $\varepsilon > 0$:

$$\lim_{T \rightarrow \infty} \Pr(|\hat{\theta} - \theta| > \varepsilon) = 0.$$

Intuitively, consistency says that as you get enough data then $\hat{\theta}$ will eventually equal θ . In other words, if you have enough data then you know the truth.

Theorems in probability theory known as *Laws of Large Numbers* are used to determine if an estimator is consistent or not. In general, you have the following result.

Proposition 7.1 (Estimator Consistency). *An estimator $\hat{\theta}$ is consistent for θ if:*

- $\text{bias}(\hat{\theta}, \theta) = 0$ as $T \rightarrow \infty$.
- $\text{se}(\hat{\theta}) = 0$ as $T \rightarrow \infty$.

Equivalently, $\hat{\theta}$ is consistent for θ if $\text{mse}(\hat{\theta}, \theta) \rightarrow 0$ as $T \rightarrow \infty$.

Intuitively, if $f(\hat{\theta})$ collapses to θ as $T \rightarrow \infty$ then $\hat{\theta}$ is consistent for θ .

7.3.2 Asymptotic Normality

Let $\hat{\theta}$ be an estimator of θ based on the random returns $\{R_t\}_{t=1}^T$.

Definition 7.10 (Asymptotically Normal Estimator). *An estimator $\hat{\theta}$ is asymptotically normally distributed if:*

$$\hat{\theta} \sim N(\theta, \text{se}(\hat{\theta})^2) \quad (7.7)$$

for large enough T .

Asymptotic normality means that $f(\hat{\theta})$ is well approximated by a normal distribution with mean θ and variance $\text{se}(\hat{\theta})^2$. The justification for asymptotic normality comes from the famous *Central Limit Theorem*.

In the definition of an asymptotically normal estimator, the variance of the normal distribution, $\text{se}(\hat{\theta})^2$, often depends on unknown GWN model parameters and so is practically useless. Fortunately, you can create a practically useful result if you replace the unknown parameters in $\text{se}(\hat{\theta})^2$ with consistent estimates.

Proposition 7.2 (Practically Useful Asymptotic Normality). *Suppose an estimator $\hat{\theta}$ is asymptotically normally distributed with variance $\text{se}(\hat{\theta})^2$ that depends on unknown GWN model parameters. If you replace these unknown parameters with consistent estimates, creating the estimated variance $\widehat{\text{se}}(\hat{\theta})^2$, then*

$$\hat{\theta} \sim N(\theta, \widehat{\text{se}}(\hat{\theta})^2) \quad (7.8)$$

for large enough T .

7.3.3 Central Limit Theorem

There are actually many versions of the CLT with different assumptions.¹ In its simplest form, the CLT says that the sample average of a collection of iid random variables $X_1, \dots, X_T$ with $E[X_i] = \mu$ and $\text{var}(X_i) = \sigma^2$ is asymptotically normal with mean μ and variance σ^2/T . In particular, the CDF of the standardized sample mean:

$$\frac{\bar{X} - \mu}{\text{se}(\bar{X})} = \frac{\bar{X} - \mu}{\sigma/\sqrt{T}} = \sqrt{T} \left(\frac{\bar{X} - \mu}{\sigma} \right),$$

converges to the CDF of a standard normal random variable Z as $T \rightarrow \infty$. This result can be expressed as:

$$\sqrt{T} \left(\frac{\bar{X} - \mu}{\sigma} \right) \sim Z \sim N(0, 1),$$

for large enough T . Equivalently,

$$\bar{X} \sim \mu + \frac{\sigma}{\sqrt{T}} \times Z \sim N \left(\mu, \frac{\sigma^2}{T} \right) = N(\mu, \text{se}(\bar{X})^2),$$

for large enough T . This form shows that $\bar{X}$ is asymptotically normal with mean μ and variance $\text{se}(\bar{X})^2 = \sigma^2/T$.

i Remarks

1. The CLT result is truly remarkable because the iid random variables X_i can come from *any distribution* that has mean μ and variance σ^2 . For example, X_i could be binomial, Student's t with $\text{df} > 2$, chi-square, etc.
2. While the basic CLT shows that the sample average $\bar{X}$ is asymptotically normally distributed, it can be extended to general estimators of parameters of the GWN model because, as you shall see, these estimators are essentially averages of iid random variables.
3. The iid assumption of the basic CLT can be relaxed to allow $\{X_i\}$ to be covariance stationary and ergodic. This allows the asymptotic normality result to be extended to estimators computed from time series data that could be serially correlated.
4. The CLT result depends on knowing σ^2 , which is practically useless because you almost never know σ^2 . However, the CLT result holds if you replace σ^2 with a consistent estimate $\hat{\sigma}^2$.

¹White (1984) gives a comprehensive discussion of CLTs useful in econometrics.

7.3.4 Asymptotic Confidence Intervals

For an asymptotically normal estimator $\hat{\theta}$ of θ , the precision of $\hat{\theta}$ is measured by $\widehat{\text{se}}(\hat{\theta})$ but is best communicated by computing a (asymptotic) *confidence interval* for the unknown value of θ . A confidence interval is an *interval estimate* of θ such that you can put an explicit probability statement about the likelihood that the interval covers θ .

The construction of an asymptotic confidence interval for θ uses the asymptotic normality result:

$$\frac{\hat{\theta} - \theta}{\widehat{\text{se}}(\hat{\theta})} = Z \sim N(0, 1). \quad (7.9)$$

Then, for $\alpha \in (0, 1)$, you compute a $(1 - \alpha) \cdot 100\%$ confidence interval for θ using Equation 7.9 and the $1 - \alpha/2$ standard normal quantile (critical value) $q_{(1-\alpha/2)}^Z$ to give:

$$\Pr \left(-q_{(1-\alpha/2)}^Z \leq \frac{\hat{\theta} - \theta}{\widehat{\text{se}}(\hat{\theta})} \leq q_{(1-\alpha/2)}^Z \right) = 1 - \alpha,$$

which can be rearranged as,

$$\Pr \left(\hat{\theta} - q_{(1-\alpha/2)}^Z \cdot \widehat{\text{se}}(\hat{\theta}) \leq \mu_i \leq \hat{\theta} + q_{(1-\alpha/2)}^Z \cdot \widehat{\text{se}}(\hat{\theta}) \right) = 1 - \alpha.$$

Hence, the random interval,

$$[\hat{\theta} - q_{(1-\alpha/2)}^Z \cdot \widehat{\text{se}}(\hat{\theta}), \hat{\theta} + q_{(1-\alpha/2)}^Z \cdot \widehat{\text{se}}(\hat{\theta})] = \hat{\theta} \pm q_{(1-\alpha/2)}^Z \cdot \widehat{\text{se}}(\hat{\theta}) \quad (7.10)$$

covers the true unknown value of θ with probability $1 - \alpha$.

In practice, typical values for α are 0.05 and 0.01 for which $q_{(0.975)}^Z = 1.96$ and $q_{(0.995)}^Z = 2.58$. Then, approximate 95% and 99% asymptotic confidence intervals for θ have the form $\hat{\theta} \pm 2 \cdot \widehat{\text{se}}(\hat{\theta})$ and $\hat{\theta} \pm 2.5 \cdot \widehat{\text{se}}(\hat{\theta})$, respectively.

7.4 Estimators for the Parameters of the GWN Model

Let $\{\mathbf{r}_t\}_{t=1}^T$ denote a sample of size T of observed returns on N assets from the GWN model Equation 7.1. To estimate the unknown GWN model parameters μ_i , σ_i^2 , σ_{ij} and ρ_{ij} from $\{\mathbf{r}_t\}_{t=1}^T$ you can use the *plug-in principle* from statistics:

Definition 7.11 (Plug-in-Principle). The *plug-in-principle* says to estimate model parameters using corresponding sample statistics.

Proposition 7.3 (Plug-in Principle Estimates for GWN Model Parameters). *For the GWN model parameters, the plug-in principle estimates are the following sample descriptive statistics discussed in Chapter Chapter 5:*

$$\hat{\mu}_i = \frac{1}{T} \sum_{t=1}^T r_{it}, \quad (7.11)$$

$$\hat{\sigma}_i^2 = \frac{1}{T-1} \sum_{t=1}^T (r_{it} - \hat{\mu}_i)^2, \quad (7.12)$$

$$\hat{\sigma}_i = \sqrt{\hat{\sigma}_i^2}, \quad (7.13)$$

$$\hat{\sigma}_{ij} = \frac{1}{T-1} \sum_{t=1}^T (r_{it} - \hat{\mu}_i)(r_{jt} - \hat{\mu}_j), \quad (7.14)$$

$$\hat{\rho}_{ij} = \frac{\hat{\sigma}_{ij}}{\hat{\sigma}_i \hat{\sigma}_j}. \quad (7.15)$$

The plug-in principle is appropriate because the GWN model parameters $\mu_i, \sigma_i^2, \sigma_{ij}$ and ρ_{ij} are characteristics of the underlying distribution of returns that are naturally estimated using sample statistics.

The plug-in principle sample statistics Equation 7.11 - Equation 7.15 are given for a single asset and the statistics Equation 7.14 - Equation 7.15 are given for one pair of assets. However, these statistics can be computed for a collection of N assets using the matrix sample statistics:

$$\hat{\boldsymbol{\mu}}_{(N \times 1)} = \frac{1}{T} \sum_{t=1}^T \mathbf{r}_t = \begin{pmatrix} \hat{\mu}_1 \\ \vdots \\ \hat{\mu}_N \end{pmatrix}, \quad (7.16)$$

$$\hat{\hat{\Sigma}}_{(N \times N)} = \frac{1}{T-1} \sum_{t=1}^T (\mathbf{r}_t - \hat{\boldsymbol{\mu}})(\mathbf{r}_t - \hat{\boldsymbol{\mu}})' = \begin{pmatrix} \hat{\sigma}_1^2 & \hat{\sigma}_{12} & \cdots & \hat{\sigma}_{1N} \\ \hat{\sigma}_{12} & \hat{\sigma}_2^2 & \cdots & \hat{\sigma}_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{\sigma}_{1N} & \hat{\sigma}_{2N} & \cdots & \hat{\sigma}_N^2 \end{pmatrix}. \quad (7.17)$$

Here $\hat{\boldsymbol{\mu}}$ is called the *sample mean vector* and $\hat{\hat{\Sigma}}$ is called the *sample covariance matrix*. The sample variances are the diagonal elements of $\hat{\hat{\Sigma}}$, and the sample covariances are the off diagonal elements of $\hat{\hat{\Sigma}}$. To get the sample correlations, define the $N \times N$ diagonal matrix:

$$\hat{\mathbf{D}} = \begin{pmatrix} \hat{\sigma}_1 & 0 & \cdots & 0 \\ 0 & \hat{\sigma}_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \hat{\sigma}_N \end{pmatrix}.$$

Then the sample correlation matrix $\hat{\mathbf{C}}$ is computed as:

$$\hat{\mathbf{C}} = \hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{D}}^{-1} = \begin{pmatrix} 1 & \hat{\rho}_{12} & \cdots & \hat{\rho}_{1N} \\ \hat{\rho}_{12} & 1 & \cdots & \hat{\rho}_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{\rho}_{1N} & \hat{\rho}_{2N} & \cdots & 1 \end{pmatrix}. \quad (7.18)$$

Here, the sample correlations are the off diagonal elements of $\hat{\mathbf{C}}$.

Example 7.2 (Estimating the GWN model parameters for Microsoft Starbucks and the S&P500 index). To illustrate typical estimates of the GWN model parameters, I use data on monthly simple and continuously compounded (cc) returns for Microsoft, Starbucks and the S & P 500 index over the period January 1998 through May 2012 from the **introcompfinr** package. This is same data used in Chapter Chapter 5 and is constructed as follows:

```
msftPrices = to.monthly(msftDailyPrices, OHLC=FALSE)
sp500Prices = to.monthly(sp500DailyPrices, OHLC=FALSE)
sbuxPrices = to.monthly(sbuxDailyPrices, OHLC=FALSE)
# set sample to match examples in other chapters
smpl = "1998-01::2012-05"
msftPrices = msftPrices[smpl]
sp500Prices = sp500Prices[smpl]
sbuxPrices = sbuxPrices[smpl]
# calculate returns
msftRetS = na.omit(Return.calculate(msftPrices, method="simple"))
sp500RetS = na.omit(Return.calculate(sp500Prices, method="simple"))
sbuxRetS = na.omit(Return.calculate(sbuxPrices, method="simple"))
msftRetC = log(1 + msftRetS)
sp500RetC = log(1 + sp500RetS)
sbuxRetC = log(1 + sbuxRetS)
# merged data set
gwnRetS = merge(msftRetS, sbuxRetS, sp500RetS)
gwnRetC = merge(msftRetC, sbuxRetC, sp500RetC)
colnames(gwnRetS) = colnames(gwnRetC) = c("MSFT", "SBUX", "SP500")
```

These data are illustrated in Figure 7.2 and Figure 7.3.

The estimates of μ_i ($i = \text{msft}, \text{sbux}, \text{sp500}$) using Equation 7.11 or Equation 7.16 can be computed using the R functions `apply()` and `mean()`:

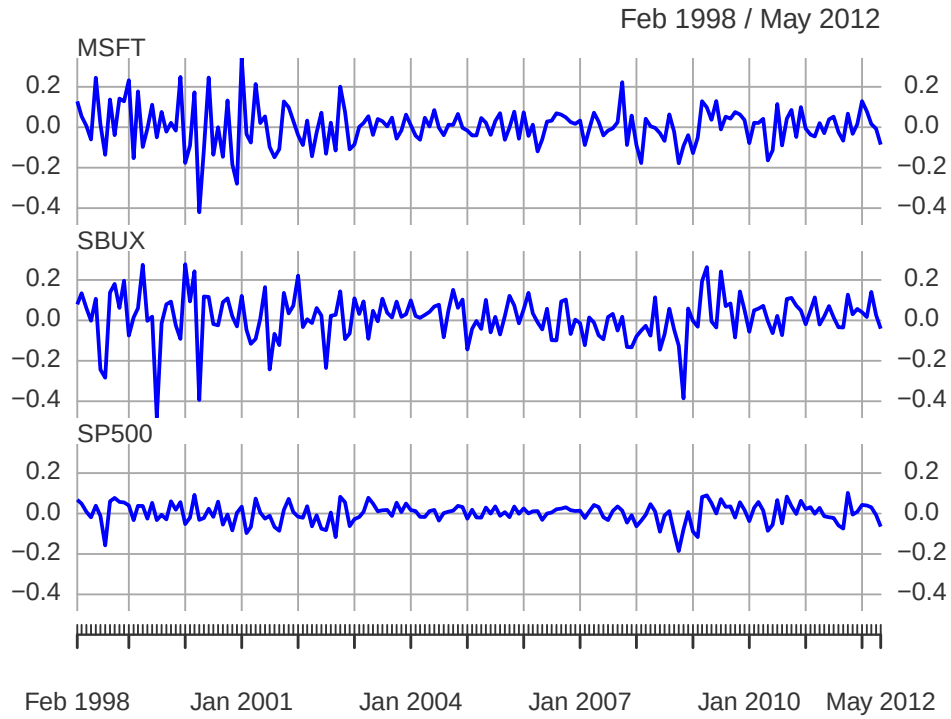


Figure 7.2: Monthly cc returns on Microsoft stock, Starbucks stock, and the S&P 500 index, over the period January 1998 through May 2012.

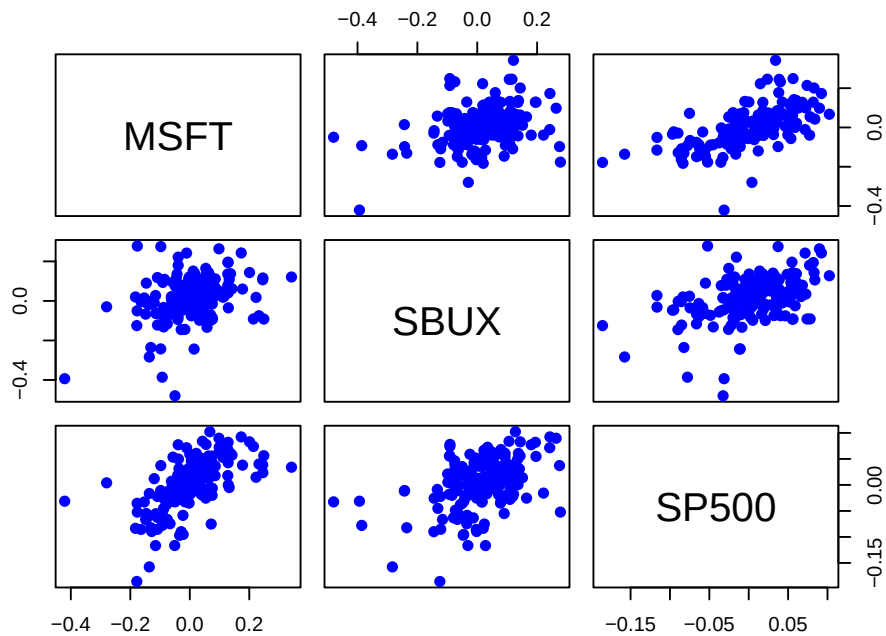


Figure 7.3: Monthly cc returns on Microsoft stock, Starbucks stock, and the S&P 500 index, over the period January 1998 through May 2012.

```

muhat = apply(gwnRetC,2,mean)
muhat
#>      MSFT      SBUX      SP500
#> 0.00413 0.01466 0.00169

```

Starbucks has the highest average monthly return at 1.5% and the S&P 500 index has the lowest at 0.2%.

The estimates of the parameters σ_i^2, σ_i , using Equation 7.12 and Equation 7.13 can be computed using `apply()`, `var()` and `sd()`:

```

sigma2hat = apply(gwnRetC,2,var)
sigmahat = apply(gwnRetC,2,sd)
rbind(sigma2hat, sigmahat)
#>           MSFT      SBUX      SP500
#> sigma2hat 0.01 0.0125 0.00235
#> sigmahat  0.10 0.1116 0.04847

```

Starbucks has the most variable monthly returns at 11%, and the S&P 500 index has the smallest at 5%.

The scatterplots of the returns are illustrated in Figure 7.3. All returns appear to be positively related. The covariance and correlation matrix estimates using Equation 7.17 and Equation 7.18 can be computed using the functions `var()` (or `cov()`) and `cor()`:

```

covmat = var(gwnRetC)
covmat
#>           MSFT      SBUX      SP500
#> MSFT  0.01004 0.00381 0.00300
#> SBUX  0.00381 0.01246 0.00248
#> SP500 0.00300 0.00248 0.00235
cormat = cor(gwnRetC)
cormat
#>           MSFT      SBUX      SP500
#> MSFT  1.000 0.341 0.617
#> SBUX  0.341 1.000 0.457
#> SP500 0.617 0.457 1.000

```

To extract the unique pairwise values of σ_{ij} and ρ_{ij} from the matrix objects `covmat` and `cormat` use:

```

covhat = covmat[lower.tri(covmat)]
rhohat = cormat[lower.tri(cormat)]
names(covhat) <- names(rhohat) <-
c("msft,sbux","msft,sp500","sbux,sp500")
covhat
#>  msft,sbux msft,sp500 sbux,sp500
#>    0.00381    0.00300    0.00248
rhohat
#>  msft,sbux msft,sp500 sbux,sp500
#>    0.341     0.617     0.457

```

The pairs (MSFT, SP500) and (SBUX, SP500) are the most correlated. These estimates confirm the visual results from the scatterplot matrix in Figure 7.3.

■

7.5 Statistical Properties of the GWN Model Estimates

To determine the statistical properties of plug-in principle estimators $\hat{\mu}_i, \hat{\sigma}_i^2, \hat{\sigma}_i, \hat{\sigma}_{ij}$ and $\hat{\rho}_{ij}$ in the GWN model, you treat them as functions of the random variables $\{\mathbf{R}_t\}_{t=1}^T$ where $\mathbf{R}_t$ is assumed to be generated by the GWN model Equation 7.1.

7.5.1 Bias

Proposition 7.4 (Bias of GWN Model Estimators). *Assume that returns are generated by the GWN model Equation 7.1. Then the estimators $\hat{\mu}_i, \hat{\sigma}_i^2$ and $\hat{\sigma}_{ij}$ are unbiased estimators:*²

$$\begin{aligned}
 E[\hat{\mu}_i] &= \mu_i \\
 E[\hat{\sigma}_i^2] &= \sigma_i^2, \\
 E[\hat{\sigma}_{ij}] &= \sigma_{ij}.
 \end{aligned}$$

The estimators $\hat{\sigma}_i$ and $\hat{\rho}_{ij}$ are biased estimators:

$$\begin{aligned}
 E[\hat{\sigma}_i] &\neq \sigma_i, \\
 E[\hat{\rho}_{ij}] &\neq \rho_{ij}.
 \end{aligned}$$

It can be shown that the biases in $\hat{\sigma}_i$ and $\hat{\rho}_{ij}$, are very small and decreasing in T such that $\text{bias}(\hat{\sigma}_i, \sigma_i) = \text{bias}(\hat{\rho}_{ij}, \rho_{ij}) = 0$ as $T \rightarrow \infty$. The proofs of these results are beyond the scope of this

²The matrix sample statistics Equation 7.16 and Equation 7.17 are unbiased estimators of μ and Σ , respectively.

book and may be found, for example, in Goldberger (1991). As you shall see, these results about bias can be easily verified using Monte Carlo methods.

It is instructive to illustrate how to derive the result $E[\hat{\mu}_i] = \mu_i$. Using $\hat{\mu}_i = \frac{1}{T} \sum_{t=1}^T R_{it}$ and results about the expectation of a linear combination of random variables, it follows that:

$$\begin{aligned} E[\hat{\mu}_i] &= E \left[\frac{1}{T} \sum_{t=1}^T R_{it} \right] \\ &= \frac{1}{T} \sum_{t=1}^T E[R_{it}] \text{ (by the linearity of } E[\cdot] \text{)} \\ &= \frac{1}{T} \sum_{t=1}^T \mu_i \text{ (since } E[R_{it}] = \mu_i, t = 1, \dots, T \text{)} \\ &= \frac{1}{T} T \cdot \mu_i = \mu_i. \end{aligned}$$

The derivation of the results $E[\hat{\sigma}_i^2] = \sigma_i^2$ and $E[\hat{\sigma}_{ij}] = \sigma_{ij}$ are similar but are considerably more involved and so are omitted.

7.5.2 Precision

Because the GWN model estimators are either unbiased or the bias is very small, the precision of these estimators is measured by their standard errors. Here, I give the mathematical formulas for their standard errors.

Proposition 7.5 (Standard error for $\hat{\mu}_i$). *The standard error for $\hat{\mu}_i$, $\text{se}(\hat{\mu}_i)$, can be calculated exactly and is given by:*

$$\text{se}(\hat{\mu}_i) = \frac{\sigma_i}{\sqrt{T}}. \quad (7.19)$$

The derivation of this result is straightforward. Using the results for the variance of a linear combination of uncorrelated random variables, you have:

$$\begin{aligned} \text{var}(\hat{\mu}_i) &= \text{var} \left(\frac{1}{T} \sum_{t=1}^T R_{it} \right) \\ &= \frac{1}{T^2} \sum_{t=1}^T \text{var}(R_{it}) \text{ (since } R_{it} \text{ is independent over time)} \\ &= \frac{1}{T^2} \sum_{t=1}^T \sigma_i^2 \text{ (since } \text{var}(R_{it}) = \sigma_i^2, t = 1, \dots, T \text{)} \\ &= \frac{1}{T^2} T \sigma_i^2 \\ &= \frac{\sigma_i^2}{T}. \end{aligned}$$

Then $\text{se}(\hat{\mu}_i) = \text{sd}(\hat{\mu}_i) = \frac{\sigma_i}{\sqrt{T}}$.

i Remarks

1. The value of $\text{se}(\hat{\mu}_i)$ is in the same units as $\hat{\mu}_i$ and measures the precision of $\hat{\mu}_i$ as an estimate. If $\text{se}(\hat{\mu}_i)$ is small relative to $\hat{\mu}_i$ then $\hat{\mu}_i$ is a relatively precise estimate of μ_i because $f(\hat{\mu}_i)$ will be tightly concentrated around μ_i ; if $\text{se}(\hat{\mu}_i)$ is large relative to μ_i then $\hat{\mu}_i$ is a relatively imprecise estimate of μ_i because $f(\hat{\mu}_i)$ will be spread out about μ_i .
2. The magnitude of $\text{se}(\hat{\mu}_i)$ depends positively on the volatility of returns, $\sigma_i = \text{sd}(R_{it})$. For a given sample size T , assets with higher return volatility have larger values of $\text{se}(\hat{\mu}_i)$ than assets with lower return volatility. In other words, estimates of expected return for high volatility assets are less precise than estimates of expected returns for low volatility assets.
3. For a given return volatility σ_i , $\text{se}(\hat{\mu}_i)$ is smaller for larger sample sizes T . In other words, $\hat{\mu}_i$ is more precisely estimated for larger samples.
4. $\text{se}(\hat{\mu}_i) \rightarrow 0$ as $T \rightarrow \infty$. This together with $E[\hat{\mu}_i] = \mu_i$ shows that $\hat{\mu}_i$ is consistent for μ_i .

The derivations of the standard errors for $\hat{\sigma}_i^2$, $\hat{\sigma}_i$, $\hat{\sigma}_{ij}$ and $\hat{\rho}_{ij}$ are complicated, and the exact results are extremely messy and hard to work with. However, there are simple approximate formulas for the standard errors of $\hat{\sigma}_i^2$, $\hat{\sigma}_i$, $\hat{\sigma}_{ij}$, and $\hat{\rho}_{ij}$ based on the CLT under the GWN model that are valid if the sample size, T , is reasonably large.

Proposition 7.6 (Approximate standard error formulas for $\hat{\sigma}_i^2$, $\hat{\sigma}_i$, $\hat{\sigma}_{ij}$, and $\hat{\rho}_{ij}$). *The approximate standard error formulas for $\hat{\sigma}_i^2$, $\hat{\sigma}_i$, $\hat{\sigma}_{ij}$, and $\hat{\rho}_{ij}$ are given by:*

$$\text{se}(\hat{\sigma}_i^2) \approx \frac{\sqrt{2}\sigma_i^2}{\sqrt{T}} = \frac{\sigma_i^2}{\sqrt{T/2}}, \quad (7.20)$$

$$\text{se}(\hat{\sigma}_i) \approx \frac{\sigma_i}{\sqrt{2T}}, \quad (7.21)$$

$$\text{se}(\hat{\sigma}_{ij}) \approx \frac{\sqrt{\sigma_i^2\sigma_j^2 + \sigma_{ij}^2}}{\sqrt{T}} = \frac{\sqrt{\sigma_i^2\sigma_j^2(1 + \rho_{ij}^2)}}{\sqrt{T}} \quad (7.22)$$

$$\text{se}(\hat{\rho}_{ij}) \approx \frac{1 - \rho_{ij}^2}{\sqrt{T}}, \quad (7.23)$$

where “ $\approx$ ” denotes approximately equal. The approximations are such that the approximation error goes to zero as the sample size T gets very large.

I make the following remarks:

1. As with the formula for the standard error of the sample mean, the formulas for $\text{se}(\hat{\sigma}_i^2)$ and $\text{se}(\hat{\sigma}_i)$ depend on σ_i^2 . Larger values of σ_i^2 imply less precise estimates of $\hat{\sigma}_i^2$ and $\hat{\sigma}_i$.

2. The formula for $\text{se}(\hat{\sigma}_{ij})$ depends on σ_i^2 , σ_j^2 , and ρ_{ij}^2 . Given σ_i^2 and σ_j^2 , the standard error is smallest when $\rho_{ij} = 0$.
3. The formula for $\text{se}(\hat{\rho}_{ij})$, does not depend on σ_i^2 but rather depends on ρ_{ij}^2 and is smaller the closer ρ_{ij}^2 is to unity. Intuitively, this makes sense because as ρ_{ij}^2 approaches one the linear dependence between R_{it} and R_{jt} becomes almost perfect and this will be easily recognizable in the data (scatterplot will almost follow a straight line).
4. The formulas for the standard errors above are inversely related to the square root of the sample size, $\sqrt{T}$, which means that larger sample sizes imply smaller values of the standard errors.
5. Interestingly, $\text{se}(\hat{\sigma}_i)$ goes to zero the fastest and $\text{se}(\hat{\sigma}_i^2)$ goes to zero the slowest. Hence, for a fixed sample size, these formulas suggest that σ_i is generally estimated more precisely than σ_i^2 and ρ_{ij} , and ρ_{ij} is estimated generally more precisely than σ_i^2 .

The above formulas Equation 7.20 - Equation 7.23 are not practically useful, however, because they depend on the unknown quantities σ_i^2 , σ_i and ρ_{ij} . Practically useful formulas make use of the plug-in principle and replace σ_i^2 , σ_i and ρ_{ij} by the estimates $\hat{\sigma}_i^2$, $\hat{\sigma}_i$ and $\hat{\rho}_{ij}$ and give rise to the *estimated standard errors*:

$$\widehat{\text{se}}(\hat{\mu}_i) = \frac{\hat{\sigma}_i}{\sqrt{T}} \quad (7.24)$$

$$\widehat{\text{se}}(\hat{\sigma}_i^2) \approx \frac{\hat{\sigma}_i^2}{\sqrt{T/2}}, \quad (7.25)$$

$$\widehat{\text{se}}(\hat{\sigma}_i) \approx \frac{\hat{\sigma}_i}{\sqrt{2T}}, \quad (7.26)$$

$$\widehat{\text{se}}(\hat{\sigma}_{ij}) \approx \frac{\sqrt{\hat{\sigma}_i^2 \hat{\sigma}_j^2 (1 + \hat{\rho}_{ij}^2)}}{\sqrt{T}} \quad (7.27)$$

$$\widehat{\text{se}}(\hat{\rho}_{ij}) \approx \frac{1 - \hat{\rho}_{ij}^2}{\sqrt{T}}. \quad (7.28)$$

It is good practice to report estimates together with their estimated standard errors. In this way the precision of the estimates is transparent to the user. Typically, estimates are reported in a table with the estimates in one column and the estimated standard errors in an adjacent column.

Example 7.3 ($\widehat{\text{se}}(\hat{\mu}_i)$ values for Microsoft Starbucks and the S&P 500 index). For Microsoft, Starbucks and S&P 500, the values of $\widehat{\text{se}}(\hat{\mu}_i)$ are easily computed in R using:

```
n.obs = nrow(gwnRetC)
seMuhat = sigmahat/sqrt(n.obs)
```

The values of $\hat{\mu}_i$ and $\widehat{\text{se}}(\hat{\mu}_i)$ shown together are:


```

cbind(muhat, seMuhat)
#>           muhat seMuhat
#> MSFT    0.00413 0.00764
#> SBUX    0.01466 0.00851
#> SP500   0.00169 0.00370

```

For Microsoft and Starbucks, the values of $\widehat{\text{se}}(\hat{\mu}_i)$ are similar because the values of $\hat{\sigma}_i$ are similar, and $\widehat{\text{se}}(\hat{\mu}_i)$ is smallest for the S&P 500 index. This occurs because $\hat{\sigma}_{\text{sp500}}$ is much smaller than $\hat{\sigma}_{\text{msft}}$ and $\hat{\sigma}_{\text{sbux}}$. Hence, $\hat{\mu}_i$ is estimated more precisely for the S&P 500 index (a highly diversified portfolio) than it is for Microsoft and Starbucks stock (individual assets).

It is tempting to compare the magnitude of $\hat{\mu}_i$ to $\widehat{\text{se}}(\hat{\mu}_i)$ to evaluate if $\hat{\mu}_i$ is a precise estimate. A common way to do this is to compute the so-called *t-ratio*:

```

muhat/seMuhat
#> MSFT SBUX SP500
#> 0.540 1.722 0.457

```

The t-ratio shows the number of $\widehat{\text{se}}(\hat{\mu}_i)$ values $\hat{\mu}_i$ is from zero. For example, $\hat{\mu}_{\text{msft}} = 0.004$ is 0.54 values of $\widehat{\text{se}}(\hat{\mu}_{\text{msft}}) = 0.008$ above zero. This is not very far from zero. In contrast, $\hat{\mu}_{\text{sbux}} = 0.015$ is 1.72 values of $\widehat{\text{se}}(\hat{\mu}_{\text{sbux}}) = 0.008$ above zero. This is moderately far from zero. Because, $\hat{\mu}_i$ represents an average rate of return on an asset it informative to know how far above zero it is likely to be. The farther $\hat{\mu}_i$ is from zero, in units of $\widehat{\text{se}}(\hat{\mu}_i)$, the more sure you are that the asset has a positive average rate of return. ■

Example 7.4 (Computing $\widehat{\text{se}}(\hat{\sigma}_i^2)$, $\widehat{\text{se}}(\hat{\sigma}_i)$, $\widehat{\text{se}}(\hat{\sigma}_{ij})$, and $\widehat{\text{se}}(\hat{\rho}_{ij})$ for Microsoft Starbucks and the S&P 500). For Microsoft, Starbucks and S&P 500, the values of $\widehat{\text{se}}(\hat{\sigma}_i^2)$ and $\widehat{\text{se}}(\hat{\sigma}_i)$ (together with the estimates $\hat{\sigma}_i^2$ and $\hat{\sigma}_i$ are:

```

seSigma2hat = sigma2hat/sqrt(n.obs/2)
seSigmahat = sigmahat/sqrt(2*n.obs)
cbind(sigma2hat, seSigma2hat, sigmahat, seSigmahat)
#>           sigma2hat seSigma2hat sigmahat seSigmahat
#> MSFT      0.01004      0.001083  0.1002      0.00540
#> SBUX      0.01246      0.001344  0.1116      0.00602
#> SP500     0.00235      0.000253  0.0485      0.00261

```

Notice that σ^2 and σ for the S&P 500 index are estimated much more precisely than the values for Microsoft and Starbucks. Also notice that σ_i is estimated more precisely than μ_i for all assets: the values of $\widehat{\text{se}}(\hat{\sigma}_i)$ relative to $\hat{\sigma}_i$ are much smaller than the values of $\widehat{\text{se}}(\hat{\mu}_i)$ to $\hat{\mu}_i$.

The values of $\widehat{\text{se}}(\hat{\sigma}_{ij})$ and $\widehat{\text{se}}(\hat{\rho}_{ij})$ (together with the values of σ_{ij} and $\hat{\rho}_{ij}$) are:

```

seCovhat = sqrt(sigma2hat[c("MSFT", "MSFT", "SBUX")]*sigma2hat[c("SBUX", "SP500", "SP500")]*(1 +
seRhohat = (1-rhohat^2)/sqrt(n.obs)
cbind(covhat, seCovhat, rhohat, seRhohat)
#>               covhat seCovhat rhohat seRhohat
#> msft,sbux   0.00381 0.000901  0.341   0.0674
#> msft,sp500 0.00300 0.000435  0.617   0.0472
#> sbux,sp500 0.00248 0.000454  0.457   0.0603

```

The values of $\widehat{\text{se}}(\hat{\sigma}_{ij})$ and $\widehat{\text{se}}(\hat{\rho}_{ij})$ are moderate in size (relative to $\hat{\sigma}_{ij}$ and $\hat{\rho}_{ij}$). Notice that $\hat{\rho}_{\text{msft,sp500}}$ has the smallest estimated standard error because $\hat{\rho}_{\text{msft,sp500}}^2$ is closest to one. ■

7.5.3 Sampling Distributions and Confidence Intervals

7.5.3.1 Sampling Distribution for $\hat{\mu}_i$

In the GWN model, $R_{it} \sim iid N(\mu_i, \sigma_i^2)$ and since $\hat{\mu}_i = \frac{1}{T} \sum_{t=1}^T R_{it}$ is an average of these normal random variables, it is also normally distributed. Previously I showed that $E[\hat{\mu}_i] = \mu_i$ and $\text{var}(\hat{\mu}_i) = \frac{\sigma_i^2}{T}$. Therefore, the exact probability distribution of $\hat{\mu}_i$, $f(\hat{\mu}_i)$, for a fixed sample size T is the normal distribution $\hat{\mu}_i \sim N\left(\mu_i, \frac{\sigma_i^2}{T}\right)$. Therefore, you have an exact formula for $f(\hat{\mu}_i)$:

$$f(\hat{\mu}_i) = \left(\frac{2\pi\sigma_i^2}{T}\right)^{-1/2} \exp\left\{-\frac{1}{2\sigma_i^2/T}(\hat{\mu}_i - \mu_i)^2\right\}. \quad (7.29)$$

The probability curve $f(\hat{\mu}_i)$ is centered at the true value μ_i , and the spread about μ_i depends on the magnitude of σ_i^2 , the variability of R_{it} , and the sample size, T . For a fixed sample size T , the uncertainty in $\hat{\mu}_i$ is larger (smaller) for larger (smaller) values of σ_i^2 . Notice that the variance of $\hat{\mu}_i$ is inversely related to the sample size T . Given σ_i^2 , $\text{var}(\hat{\mu}_i)$ is smaller for larger sample sizes than for smaller sample sizes. This makes sense since you expect to have a more precise estimator when you have more data. If the sample size is very large (as $T \rightarrow \infty$) then $\text{var}(\hat{\mu}_i)$ will be approximately zero and the normal distribution of $\hat{\mu}_i$ given by Equation 7.29 will be essentially a spike at μ_i . In other words, if the sample size is very large then you essentially know the true value of μ_i . Hence, you have established that $\hat{\mu}_i$ is a *consistent* estimator of μ_i as the sample size goes to infinity.

The exact sampling distribution for $\hat{\mu}$ assumes that σ^2 is known, which is not practically useful. Below, I provide a practically useful result which holds when you use $\hat{\sigma}^2$.

Example 7.5 (Sampling distribution of $\hat{\mu}$ with different sample sizes). The distribution of $\hat{\mu}_i$, with $\mu_i = 0$ and $\sigma_i^2 = 1$ for various sample sizes is illustrated in Figure 7.4. Notice how fast the distribution collapses at $\mu_i = 0$ as T increases. ■

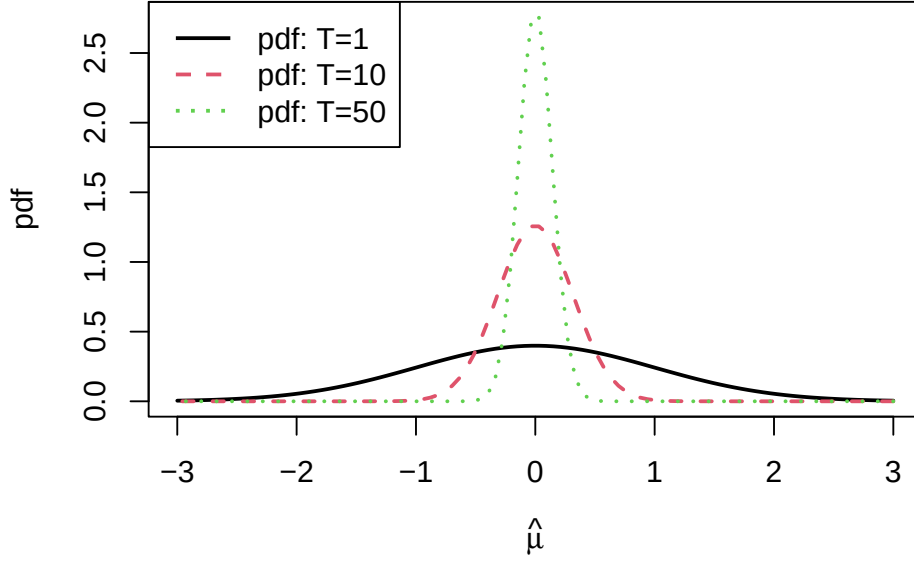


Figure 7.4: $N(0, 1/T)$ sampling distributions for $\hat{\mu}$ for $T = 1, 10$ and 50 .

7.5.3.2 Confidence Intervals for μ_i

In practice, the precision of $\hat{\mu}_i$ is measured by $\widehat{\text{se}}(\hat{\mu}_i)$ but is best communicated by computing a *confidence interval* for the unknown value of μ_i . A confidence interval is an *interval estimate* of μ_i such that you can put an explicit probability statement about the likelihood that the interval covers μ_i .

Because you know the exact finite sample distribution of $\hat{\mu}_i$ in the GWN return model, you can construct an exact confidence interval for μ_i .

Proposition 7.7 (t-ratio for sample mean). *Let $\{R_{it}\}_{t=1}^T$ be generated from the GWN model Equation 7.1. Define the t-ratio as:*

$$t_i = \frac{\hat{\mu}_i - \mu_i}{\widehat{\text{se}}(\hat{\mu}_i)} = \frac{\hat{\mu}_i - \mu_i}{\hat{\sigma}/\sqrt{T}}, \quad (7.30)$$

Then $t_i \sim t_{T-1}$ where t_{T-1} denotes a Student's t random variable with $T - 1$ degrees of freedom.

The Student's t distribution with $v > 0$ degrees of freedom is a symmetric distribution centered at zero, like the standard normal. The tail-thickness (kurtosis) of the distribution is determined by the degrees of freedom parameter v . For values of v close to zero, the tails of the Student's t

distribution are much fatter than the tails of the standard normal distribution. As v gets large, the Student's t distribution approaches the standard normal distribution.

For $\alpha \in (0, 1)$, you compute a $(1 - \alpha) \cdot 100\%$ confidence interval for μ_i using Equation 7.30 and the $1 - \alpha/2$ quantile (critical value) $t_{T-1}(1 - \alpha/2)$ to give:

$$\Pr \left(-t_{T-1}(1 - \alpha/2) \leq \frac{\hat{\mu}_i - \mu_i}{\widehat{\text{se}}(\hat{\mu}_i)} \leq t_{T-1}(1 - \alpha/2) \right) = 1 - \alpha,$$

which can be rearranged as,

$$\Pr (\hat{\mu}_i - t_{T-1}(1 - \alpha/2) \cdot \widehat{\text{se}}(\hat{\mu}_i) \leq \mu_i \leq \hat{\mu}_i + t_{T-1}(1 - \alpha/2) \cdot \widehat{\text{se}}(\hat{\mu}_i)) = 1 - \alpha.$$

Hence, the interval,

$$[\hat{\mu}_i - t_{T-1}(1 - \alpha/2) \cdot \widehat{\text{se}}(\hat{\mu}_i), \hat{\mu}_i + t_{T-1}(1 - \alpha/2) \cdot \widehat{\text{se}}(\hat{\mu}_i)] = \hat{\mu}_i \pm t_{T-1}(1 - \alpha/2) \cdot \widehat{\text{se}}(\hat{\mu}_i) \quad (7.31)$$

covers the true unknown value of μ_i with probability $1 - \alpha$.

Example 7.6 (Computing 95% confidence intervals for μ_i). Suppose you want to compute a 95% confidence interval for μ_i . In this case $\alpha = 0.05$ and $1 - \alpha = 0.95$. Suppose further that $T - 1 = 60$ (e.g., five years of monthly return data) so that $t_{T-1}(1 - \alpha/2) = t_{60}(0.975) = 2$. This can be verified in R using the function `qt()`.

Then the 95% confidence for μ_i is given by:

$$\hat{\mu}_i \pm 2 \cdot \widehat{\text{se}}(\hat{\mu}_i). \quad (7.32)$$

The above formula for a 95% confidence interval is often used as a rule of thumb for computing an approximate 95% confidence interval for moderate sample sizes. It is easy to remember and does not require the computation of the quantile $t_{T-1}(1 - \alpha/2)$ from the Student's t distribution. It is also an approximate 95% confidence interval that is based the asymptotic normality of $\hat{\mu}_i$. Recall, for a normal distribution with mean μ and variance σ^2 approximately 95% of the probability lies between $\mu \pm 2\sigma$. ■

The coverage probability associated with the confidence interval for μ_i is based on the fact that the estimator $\hat{\mu}_i$ is a random variable. Since the confidence interval is constructed as $\hat{\mu}_i \pm t_{T-1}(1 - \alpha/2) \cdot \widehat{\text{se}}(\hat{\mu}_i)$ it is also a random variable. An intuitive way to think about the coverage probability associated with the confidence interval is to think about the game of “horseshoes”³. The horseshoe is the confidence interval and the parameter μ_i is the post at which the horse shoe is tossed. Think of playing the game 100 times. If the thrower is 95% accurate (if the coverage probability is 0.95)

³Horse shoes is a game commonly played at county fairs. See (<http://en.wikipedia.org/wiki/Horseshoes>) for a complete description of the game.

then 95 of the 100 tosses should ring the post (95 of the constructed confidence intervals should contain the true value μ_i).

Example 7.7 (95% confidence intervals for μ_i for Microsoft Starbucks and the S&P 500 index). Consider computing 95% confidence intervals for μ_i using Equation 7.31 based on the estimated results for the Microsoft, Starbucks and S&P 500 data. The degrees of freedom for the Student's t distribution is $T - 1 = 171$. The 97.5% quantile, $t_{99}(0.975)$, can be computed using the R function `qt()`:

```
t.975 = qt(0.975, df=(n.obs-1))
t.975
#> [1] 1.97
```

Notice that this quantile is very close to 2. Then the exact 95% confidence intervals are given by:

```
lower = muhat - t.975*seMuhat
upper = muhat + t.975*seMuhat
width = upper - lower
ans= cbind(lower, upper, width)
colnames(ans) = c("2.5%", "97.5%", "Width")
ans
#>           2.5%   97.5%  Width
#> MSFT  -0.01096 0.01921 0.0302
#> SBUX  -0.00215 0.03146 0.0336
#> SP500 -0.00561 0.00898 0.0146
```

With probability 0.95, the above intervals will contain the true mean values assuming the GWN model is valid. The 95% confidence intervals for Microsoft and Starbucks are fairly wide (about 3%) and contain both negative and positive values. The confidence interval for the S&P 500 index is tighter but also contains negative and positive values. For Microsoft, the confidence interval is $[-1.1\% \ 1.9\%]$. This means that with probability 0.95, the true monthly expected return is somewhere between -1.1% and 1.9%. The economic implications of a -1.1% expected monthly return and a 1.9% expected return are vastly different. In contrast, the 95% confidence interval for the SP500 is about half the width of the intervals for Microsoft or Starbucks. The lower limit is near -0.5% and the upper limit is near 1%. This result clearly shows that the monthly mean return for the S&P 500 index is estimated much more precisely than the monthly mean returns for Microsoft or Starbucks. ■

7.5.3.3 Sampling Distributions for $\hat{\sigma}_i^2$, $\hat{\sigma}_i$, $\hat{\sigma}_{ij}$, and $\hat{\rho}_{ij}$

The exact distributions of $\hat{\sigma}_i^2$, $\hat{\sigma}_i$, $\hat{\sigma}_{ij}$, and $\hat{\rho}_{ij}$ based on a fixed sample size T are not normal distributions and are difficult to derive⁴. However, approximate normal distributions of the form Equation 7.7 based on the CLT are readily available:

$$\hat{\sigma}_i^2 \sim N(\sigma_i^2, \widehat{\text{se}}(\hat{\sigma}_i^2)^2) = N\left(\sigma_i^2, \frac{4\hat{\sigma}_i^4}{T}\right), \quad (7.33)$$

$$\hat{\sigma}_i \sim N(\sigma_i, \widehat{\text{se}}(\hat{\sigma}_i)^2) = N\left(\sigma_i, \frac{\hat{\sigma}_i^2}{2T}\right), \quad (7.34)$$

$$\hat{\sigma}_{ij} \sim N(\sigma_{ij}, \widehat{\text{se}}(\hat{\sigma}_{ij})^2) = N\left(\sigma_{ij}, \frac{\hat{\sigma}_i^2 \hat{\sigma}_j^2 (1 + \hat{\rho}_{ij}^2)}{T}\right) \quad (7.35)$$

$$\hat{\rho}_{ij} \sim N(\rho_{ij}, \widehat{\text{se}}(\hat{\rho}_{ij})^2) = N\left(\rho_{ij}, \frac{(1 - \hat{\rho}_{ij}^2)^2}{T}\right). \quad (7.36)$$

These approximate normal distributions can be used to compute approximate confidence intervals for σ_i^2 , σ_i , σ_{ij} and ρ_{ij} .

7.5.3.4 Approximate Confidence Intervals for σ_i^2 , σ_i , σ_{ij} , and ρ_{ij}

Approximate 95% confidence intervals for σ_i^2 , σ_i and ρ_{ij} are given by:

$$\hat{\sigma}_i^2 \pm 2 \cdot \widehat{\text{se}}(\hat{\sigma}_i^2) = \hat{\sigma}_i^2 \pm 2 \cdot \frac{\hat{\sigma}_i^2}{\sqrt{T/2}}, \quad (7.37)$$

$$\hat{\sigma}_i \pm 2 \cdot \widehat{\text{se}}(\hat{\sigma}_i) = \hat{\sigma}_i \pm 2 \cdot \frac{\hat{\sigma}_i}{\sqrt{2T}}, \quad (7.38)$$

$$\hat{\sigma}_{ij} \pm 2 \cdot \widehat{\text{se}}(\hat{\sigma}_{ij}) = \hat{\sigma}_{ij} \pm 2 \cdot \frac{\sqrt{\hat{\sigma}_i^2 \hat{\sigma}_j^2 (1 + \hat{\rho}_{ij}^2)}}{\sqrt{T}}. \quad (7.39)$$

$$\hat{\rho}_{ij} \pm 2 \cdot \widehat{\text{se}}(\hat{\rho}_{ij}) = \hat{\rho}_{ij} \pm 2 \cdot \frac{(1 - \hat{\rho}_{ij}^2)}{\sqrt{T}}. \quad (7.40)$$

Example 7.8 (Approximate 95% confidence intervals for σ_i^2 , σ_i , σ_{ij} and ρ_{ij} for Microsoft Starbucks and the S&P 500). Using Equation 7.37 - Equation 7.38, the approximate 95% confidence intervals for σ_i^2 and σ_i (i = Microsoft, Starbucks, S&P 500) are:

⁴For example, the exact sampling distribution of $(T-1)\hat{\sigma}_i^2/\sigma_i^2$ is chi-square with $T-1$ degrees of freedom.

```
# 95% confidence interval for variance
lowerSigma2 = sigma2hat - 2*seSigma2hat
upperSigma2 = sigma2hat + 2*seSigma2hat
widthSigma2 = upperSigma2 - lowerSigma2
ans = cbind(lowerSigma2, upperSigma2, widthSigma2)
colnames(ans) = c("2.5%", "97.5%", "Width")
ans
#>           2.5%   97.5%   Width
#> MSFT  0.00788 0.01221 0.00433
#> SBUX  0.00978 0.01515 0.00538
#> SP500 0.00184 0.00286 0.00101
```

```
# 95% confidence interval for volatility
lowerSigma = sigmahat - 2*seSigmahat
upperSigma = sigmahat + 2*seSigmahat
widthSigma = upperSigma - lowerSigma
ans = cbind(lowerSigma, upperSigma, widthSigma)
colnames(ans) = c("2.5%", "97.5%", "Width")
ans
#>           2.5%   97.5%   Width
#> MSFT  0.0894 0.1110 0.0216
#> SBUX  0.0996 0.1237 0.0241
#> SP500 0.0432 0.0537 0.0105
```

The 95% confidence intervals for σ and σ^2 are larger for Microsoft and Starbucks than for the S&P 500 index. For all assets, the intervals for σ are fairly narrow (2% for Microsoft and Starbucks and 1% for S&P 500 index) indicating that σ is precisely estimated.

The approximate 95% confidence intervals for σ_{ij} are:

```
lowerCov = covhat - 2*seCovhat
upperCov = covhat + 2*seCovhat
widthCov = upperCov - lowerCov
ans = cbind(lowerCov, upperCov, widthCov)
colnames(ans) = c("2.5%", "97.5%", "Width")
ans
#>           2.5%   97.5%   Width
#> msft,sbux  0.00201 0.00562 0.00361
#> msft,sp500 0.00213 0.00387 0.00174
#> sbux,sp500 0.00157 0.00338 0.00181
```

The approximate 95% confidence intervals for ρ_{ij} are:

```
lowerRho = rhohat - 2*seRhohat
upperRho = rhohat + 2*seRhohat
widthRho = upperRho - lowerRho
ans = cbind(lowerRho, upperRho, widthRho)
colnames(ans) = c("2.5%", "97.5%", "Width")
ans
#>           2.5% 97.5% Width
#> msft,sbux  0.206 0.476 0.270
#> msft,sp500 0.523 0.712 0.189
#> sbux,sp500 0.337 0.578 0.241
```

The 95% confidence intervals for σ_{ij} ρ_{ij} are not too wide and all contain just positive values away from zero. The smallest interval is for $\rho_{\text{msft,sp500}}$ because $\hat{\rho}_{\text{msft,sp500}}$ is closest to 1. ■

7.6 Using Monte Carlo Simulation to Understand the Statistical Properties of Estimators

Let R_t be the return on a single asset described by the GWN model, let θ denote some characteristic (parameter) of the GWN model you are interested in estimating, and let $\hat{\theta}$ denote an estimator for θ based on a sample of size T . The exact meaning of estimator bias, $\text{bias}(\hat{\theta}, \theta)$, the interpretation of $\text{se}(\hat{\theta})$ as a measure of precision, the sampling distribution $f(\hat{\theta})$, and the interpretation of the coverage probability of a confidence interval for θ , can all be a bit hard to grasp at first. If $\text{bias}(\hat{\theta}, \theta) = 0$ so that $E[\hat{\theta}] = \theta$ then over an infinite number of repeated samples of $\{R_{it}\}_{t=1}^T$ the average of the $\hat{\theta}$ values computed over the infinite samples is equal to the true value θ . The value of $\text{se}(\hat{\theta})$ represents the standard deviation of these $\hat{\theta}$ values. The sampling distribution $f(\hat{\theta})$ is the smoothed histogram of these $\hat{\theta}$ values. And the 95% confidence intervals for θ will actually contain θ in 95% of the samples. You can think of these hypothetical samples as different Monte Carlo simulations of the GWN model. In this way you can approximate the computations involved in evaluating $E[\hat{\theta}]$, $\text{se}(\hat{\theta})$, $f(\hat{\theta})$, and the coverage probability of a confidence interval using a large, but finite, number of Monte Carlo simulations.

7.6.1 Evaluating the Statistical Properties of $\hat{\mu}$ Using Monte Carlo Simulation

Consider the GWN model:

$$\begin{aligned} R_t &= 0.05 + \varepsilon_t, t = 1, \dots, 100 \\ \varepsilon_t &\sim \text{GWN}(0, (0.10)^2). \end{aligned} \tag{7.41}$$

Here, the true parameter values are $\mu = 0.05$ and $\sigma = 0.10$. Using Monte Carlo simulation, you can simulate $N = 1000$ different samples of size $T = 100$ from Equation 7.41 giving the sample realizations $\{r_t^j\}_{t=1}^{100}$ for $j = 1, \dots, 1000$. The R code to perform the Monte Carlo simulation is:

```
mu = 0.05
sigma = 0.10
n.obs = 100
n.sim = 1000
set.seed(111)
sim.means = rep(0,n.sim)
mu.lower = rep(0,n.sim)
mu.upper = rep(0,n.sim)
qt.975 = qt(0.975, n.obs-1)
for (sim in 1:n.sim) {
  sim.ret = rnorm(n.obs,mean=mu,sd=sigma)
  sim.means[sim] = mean(sim.ret)
  se.muhat = sd(sim.ret)/sqrt(n.obs)
  mu.lower[sim] = sim.means[sim]-qt.975*se.muhat
  mu.upper[sim] = sim.means[sim]+qt.975*se.muhat
}
```

The first 10 of these simulated samples are illustrated in Figure 7.5. Notice that there is considerable variation in the appearance of the simulated samples, but that all of the simulated samples fluctuate about the true mean value of $\mu = 0.05$ and have a typical deviation from the mean of about 0.10.

For each of the 1000 simulated samples you can estimate $\hat{\mu}$ giving 1000 mean estimates $\{\hat{\mu}^1, \dots, \hat{\mu}^{1000}\}$. A histogram of these 1000 mean values is illustrated in Figure 7.6.

The histogram of the estimated means, $\hat{\mu}^j$, can be thought of as an estimate of the underlying pdf, $f(\hat{\mu})$, of the estimator $\hat{\mu}$ which you know from Equation 7.29 is a normal pdf centered at $E[\hat{\mu}] = \mu = 0.05$ with $\text{se}(\hat{\mu}_i) = 0.10/\sqrt{100} = 0.01$. This normal curve (solid orange line) is superimposed on the histogram in Figure 7.6. Notice that the center of the histogram (white dashed vertical line) is very close to the true mean value $\mu = 0.05$. That is, on average over the 1000 Monte Carlo samples the value of $\hat{\mu}$ is about 0.05. In some samples, the estimate is too big and in some samples the estimate is too small but on average the estimate is correct. In fact, the average value of $\{\hat{\mu}^1, \dots, \hat{\mu}^{1000}\}$ from the 1000 simulated samples is:

$$\bar{\hat{\mu}} = \frac{1}{1000} \sum_{j=1}^{1000} \hat{\mu}^j = 0.0497,$$

which is very close to the true value 0.05. If the number of simulated samples is allowed to go to

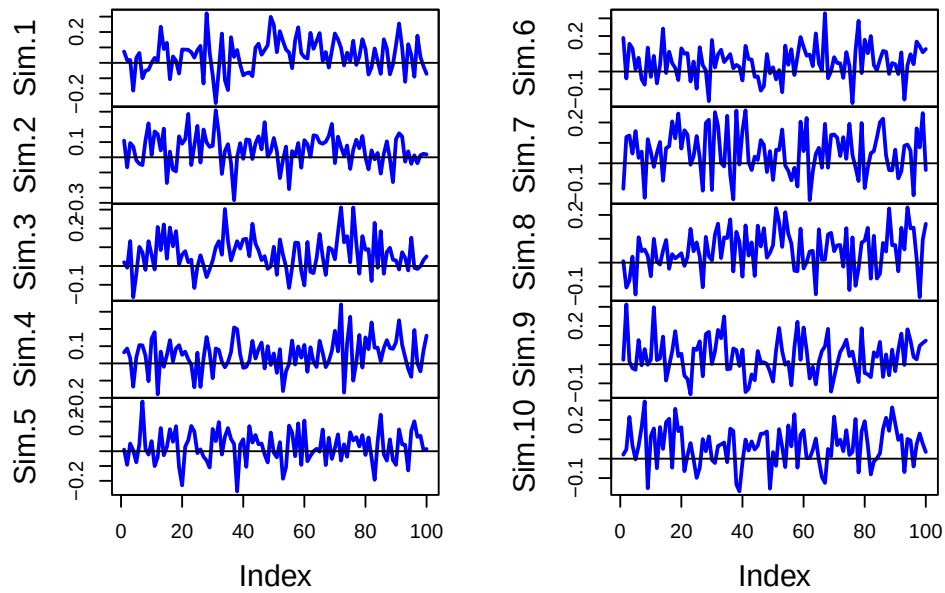


Figure 7.5: Ten simulated samples of size $T = 100$ from the GWN model $R_t = 0.05 + \varepsilon_t$, $\varepsilon_t \sim iid N(0, (0.10)^2)$.

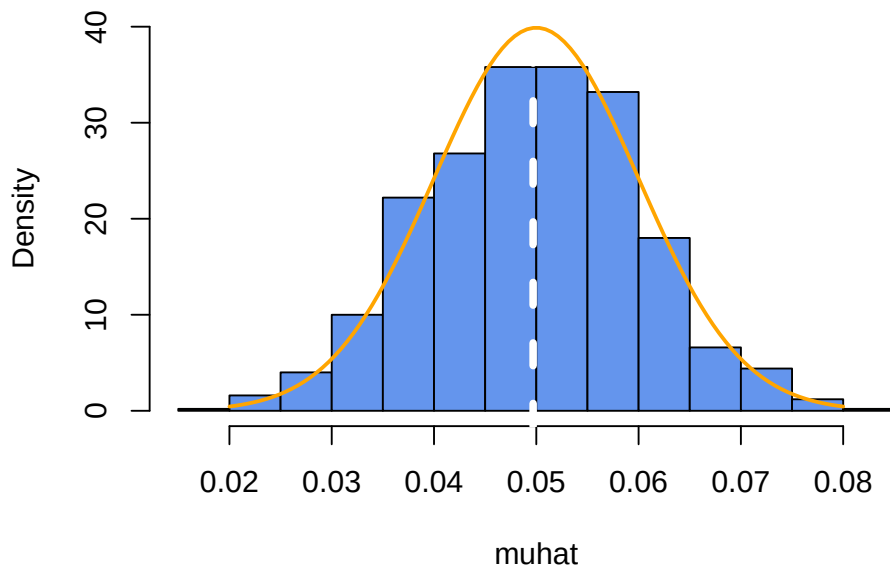


Figure 7.6: Distribution of $\hat{\mu}$ computed from 1000 Monte Carlo simulations from the GWN model. White dashed line is the average of the μ values, and orange curve is the true $f(\hat{\mu})$.

infinity then the sample average $\bar{\hat{\mu}}$ will be exactly equal to $\mu = 0.05$:

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{j=1}^N \hat{\mu}^j = E[\hat{\mu}] = \mu = 0.05.$$

The typical size of the spread about the center of the histogram represents $\text{se}(\hat{\mu}_i)$ and gives an indication of the precision of $\hat{\mu}_i$. The value of $\text{se}(\hat{\mu}_i)$ may be approximated by computing the sample standard deviation of the 1000 $\hat{\mu}^j$ values:

$$\hat{\sigma}_{\hat{\mu}} = \sqrt{\frac{1}{999} \sum_{j=1}^{1000} (\hat{\mu}^j - 0.04969)^2} = 0.0104.$$

Notice that this value is very close to the exact standard error: $\text{se}(\hat{\mu}_i) = 0.10/\sqrt{100} = 0.01$. If the number of simulated sample goes to infinity then:

$$\lim_{N \rightarrow \infty} \sqrt{\frac{1}{N-1} \sum_{j=1}^N (\hat{\mu}^j - \bar{\hat{\mu}})^2} = \text{se}(\hat{\mu}_i) = 0.01.$$

The coverage probability of the 95% confidence interval for μ can also be illustrated using Monte Carlo simulation. For each simulation j , the interval $\hat{\mu}^j \pm t_{100}(0.975) \times \widehat{\text{se}}(\hat{\mu}^j)$ is computed. The coverage probability is approximated by the fraction of intervals that contain (cover) the true $\mu = 0.05$. For the 1000 simulated samples, this fraction turns out to be 0.931. As the number of simulations goes to infinity, the Monte Carlo coverage probability will be equal to 0.95.

Here are the R results to support the above calculations. The mean and standard deviation of $\{\hat{\mu}^1, \dots, \hat{\mu}^{1000}\}$ are:

```
ans = c(mean(sim.means), sd(sim.means))
names(ans) = c("Mu", "Sigma")
ans
#>      Mu  Sigma
#> 0.0497 0.0104
```

To evaluate the coverage probability of the 95% confidence intervals, you count the number of times each interval actually contains the true value of μ :

```
in.interval = mu >= mu.lower & mu <= mu.upper
sum(in.interval)/n.sim
#> [1] 0.931
```

7.6.2 Evaluating the Statistical Properties of $\hat{\sigma}_i^2$ and $\hat{\sigma}_i$ Using Monte Carlo simulation

You can evaluate the statistical properties of $\hat{\sigma}_i^2$ and $\hat{\sigma}_i$ by Monte Carlo simulation in the same way that you evaluated the statistical properties of $\hat{\mu}_i$. You use the simulation model Equation 7.41 and $N = 1000$ simulated samples of size $T = 100$ to compute the estimates $\{(\hat{\sigma}^2)^1, \dots, (\hat{\sigma}^2)^{1000}\}$ and $\{\hat{\sigma}^1, \dots, \hat{\sigma}^{1000}\}$. The R code for the simulation loop is:

```
mu = 0.05
sigma = 0.10
n.obs = 100
n.sim = 1000
set.seed(111)
sim.sigma2 = rep(0,n.sim)
sigma2.lower = rep(0,n.sim)
sigma2.upper = rep(0,n.sim)
sim.sigma = rep(0,n.sim)
sigma.lower = rep(0,n.sim)
sigma.upper = rep(0,n.sim)
for (sim in 1:n.sim) {
  sim.ret = rnorm(n.obs,mean=mu,sd=sigma)
  sim.sigma2[sim] = var(sim.ret)
  sim.sigma[sim] = sqrt(sim.sigma2[sim])
  sigma2.lower[sim] = sim.sigma2[sim] - 2*sim.sigma2[sim]/sqrt(n.obs/2)
  sigma2.upper[sim] = sim.sigma2[sim] + 2*sim.sigma2[sim]/sqrt(n.obs/2)
  sigma.lower[sim] = sim.sigma[sim] - 2*sim.sigma[sim]/sqrt(n.obs*2)
  sigma.upper[sim] = sim.sigma[sim] + 2*sim.sigma[sim]/sqrt(n.obs*2)
}
```

The histograms of these values, with the asymptotic normal distributions overlayed, are displayed in Figure 7.7. The histogram for the $\hat{\sigma}^2$ values is bell-shaped and slightly right skewed but is centered very close to $\sigma^2 = 0.010$. The histogram for the $\hat{\sigma}$ values is more symmetric and is centered near $\sigma = 0.10$.

The average values of $\hat{\sigma}^2$ and $\hat{\sigma}$ from the 1000 simulations are:

$$\begin{aligned}\overline{\hat{\sigma}^2} &= \frac{1}{1000} \sum_{j=1}^{1000} (\hat{\sigma}^2)^j = 0.00999, \\ \overline{\hat{\sigma}} &= \frac{1}{1000} \sum_{j=1}^{1000} \hat{\sigma}^j = 0.0997.\end{aligned}$$

The Monte Carlo estimate of the bias for $\hat{\sigma}^2$ is $0.00999 - 0.01 = -0.0000$, and the estimate of bias for

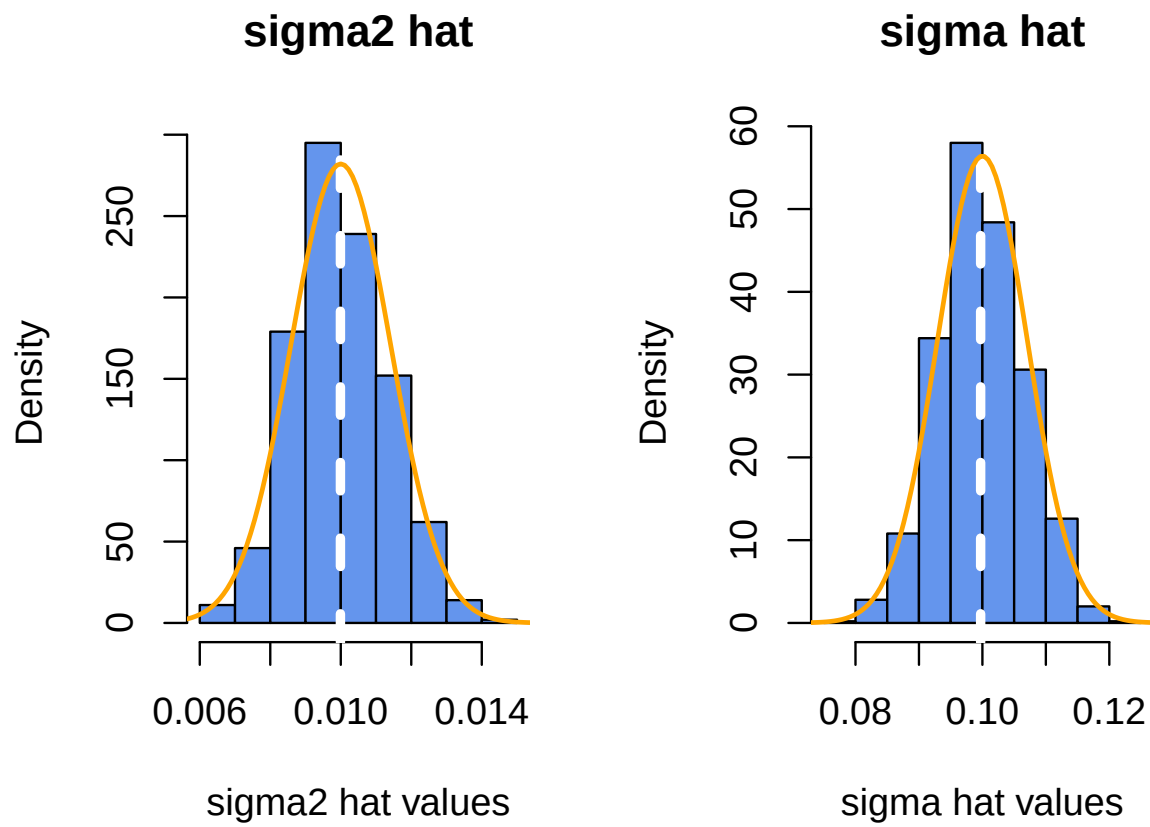


Figure 7.7: Histograms of $\hat{\sigma}^2$ and $\hat{\sigma}$ computed from $N = 1000$ Monte Carlo samples from GWN model.

$\hat{\sigma}$ is $0.0997 - 0.010 = -0.0003$. This confirms that $\hat{\sigma}^2$ is unbiased and that the bias in $\hat{\sigma}$ is extremely small. If the number of simulated samples, N , goes to infinity then $\overline{\hat{\sigma}^2} \rightarrow E[\hat{\sigma}^2] = \sigma^2 = 0.01$, and $\overline{\hat{\sigma}} \rightarrow E[\hat{\sigma}] = \sigma + \text{bias}(\hat{\sigma}, \sigma)$.

The sample standard deviation values of the Monte Carlo estimates of σ^2 and σ give approximations to $\text{se}(\hat{\sigma}^2)$ and $\text{se}(\hat{\sigma})$:

$$\hat{\sigma}_{\hat{\sigma}^2} = \sqrt{\frac{1}{999} \sum_{j=1}^{1000} ((\hat{\sigma}^2)^j - 0.00999)^2} = 0.00135,$$

$$\hat{\sigma}_{\hat{\sigma}} = \sqrt{\frac{1}{999} \sum_{j=1}^{1000} (\hat{\sigma}^j - 0.0997)^2} = 0.00676.$$

The approximate values for $\text{se}(\hat{\sigma}^2)$ and $\text{se}(\hat{\sigma})$ based on the CLT are:

$$\text{se}(\hat{\sigma}^2) = \frac{(0.10)^2}{\sqrt{100/2}} = 0.00141,$$

$$\text{se}(\hat{\sigma}) = \frac{0.10}{\sqrt{2 \times 100}} = 0.00707.$$

Notice that the Monte Carlo estimates of $\text{se}(\hat{\sigma}^2)$ and $\text{se}(\hat{\sigma})$ are a bit different from the CLT based estimates. The reason is that the CLT based estimates are approximations that hold when the sample size T is large. Because $T = 100$ is not too large, the Monte Carlo estimates of $\text{se}(\hat{\sigma}^2)$ and $\text{se}(\hat{\sigma})$ are likely more accurate (and will be more accurate if the number of simulations is larger).

For each simulation j , the approximate 95% confidence intervals $(\hat{\sigma}^2)^j \pm 2 \times \widehat{\text{se}}((\hat{\sigma}^2)^j)$ and $\hat{\sigma}^j \pm 2 \times \widehat{\text{se}}(\hat{\sigma}^j)$ are computed. The coverage probabilities of these intervals is approximated by the fractions of intervals that contain (cover) the true values $\sigma^2 = 0.01$ and $\sigma = 0.10$ respectively. For the 1000 simulated samples, these fractions turn out to be 0.951 and 0.963, respectively. As the number of simulations and the sample size go to infinity, the Monte Carlo coverage probability will be equal to 0.95.

Here is the R code to support the above calculation. The mean values and biases of the estimates are:

```
ans = rbind(c(sigma^2, mean(sim.sigma2), mean(sim.sigma2) - sigma^2),
            c(sigma, mean(sim.sigma), mean(sim.sigma) - sigma))
colnames(ans) = c("True Value", "Mean", "Bias")
rownames(ans) = c("Variance", "Volatility")
round(ans, digits=5)
#>           True Value      Mean      Bias
#> Variance           0.01 0.00999 -0.00001
#> Volatility          0.10 0.09972 -0.00028
```

The Monte Carlo and analytic standard error estimates are:

```
ans = rbind(c(sd(sim.sigma2),sigma^2/sqrt(n.obs/2)),
            c(sd(sim.sigma), sigma/sqrt(2*n.obs)))
colnames(ans) = c("MC Std Error", "Analytic Std Error")
rownames(ans) = c("Variance", "Volatility")
ans
#>           MC Std Error Analytic Std Error
#> Variance      0.00135      0.00141
#> Volatility    0.00676      0.00707
```

The coverage probabilities of the Monte Carlo 95% Confidence intervals are:

```
in.interval.sigma2 = sigma^2 >= sigma2.lower & sigma^2 <= sigma2.upper
in.interval.sigma = sigma >= sigma.lower & sigma <= sigma.upper
ans = rbind(c(sum(in.interval.sigma2)/n.sim,0.95),
            c(sum(in.interval.sigma)/n.sim,0.95))
colnames(ans) = c("MC Coverage", "True Coverage")
rownames(ans) = c("Variance", "Volatility")
ans
#>           MC Coverage True Coverage
#> Variance      0.951      0.95
#> Volatility    0.963      0.95
```

7.6.3 Evaluating the Statistical Properties of $\hat{\sigma}_{ij}$ and $\hat{\rho}_{ij}$ by Monte Carlo simulation

To evaluate the statistical properties of $\hat{\sigma}_{ij}$ and $\hat{\rho}_{ij} = \text{cor}(R_{it}, R_{jt})$, you must simulate from the GWN model in matrix form Equation 7.1. For example, consider the bivariate GWN model:

$$\begin{pmatrix} R_{1t} \\ R_{2t} \end{pmatrix} = \begin{pmatrix} 0.05 \\ 0.03 \end{pmatrix} + \begin{pmatrix} \varepsilon_{1t} \\ \varepsilon_{2t} \end{pmatrix}, \quad t = 1, \dots, 100, \quad (7.42)$$

$$\begin{pmatrix} \varepsilon_{1t} \\ \varepsilon_{2t} \end{pmatrix} \sim iid N \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} (0.10)^2 & (0.75)(0.10)(0.05) \\ (0.75)(0.10)(0.05) & (0.05)^2 \end{pmatrix} \right), \quad (7.43)$$

where $\mu_1 = 0.05$, $\mu_2 = 0.03$, $\sigma_1 = 0.10$, $\sigma_2 = 0.05$, and $\rho_{12} = 0.75$. You use the simulation model Equation 7.42 - Equation 7.43 with $N = 1000$ simulated samples of size $T = 100$ to compute the estimates $\{\hat{\rho}_{12}^1, \dots, \hat{\rho}_{12}^{1000}\}$. The R code to perform the Monte Carlo simulation is:

```

library(mvtnorm)
n.obs = 100
n.sim = 1000
set.seed(111)
sim.corrs = rep(0, n.sim) # initialize vectors
sim.covs = rep(0, n.sim)
muvec = c(0.05, 0.03)
sigmavec = c(0.1, 0.05)
names(muvec) = names(sigmavec) = c("Asset.1", "Asset.2")
rho12 = 0.75
cov12 = rho12*sigmavec[1]*sigmavec[2]
Sigma = diag(sigmavec^2)
Sigma[1,2] = Sigma[2,1] = cov12
rho.lower = rep(0,n.sim)
rho.upper = rep(0,n.sim)
cov.lower = rep(0,n.sim)
cov.upper = rep(0,n.sim)
for (sim in 1:n.sim) {
  sim.ret = rmvnorm(n.obs,mean=muvec, sigma=Sigma)
  sim.corrs[sim] = cor(sim.ret)[1,2]
  sim.covs[sim] = cov(sim.ret)[1,2]
  sig1hat2 = cov(sim.ret)[1,1]
  sig2hat2 = cov(sim.ret)[2,2]
  se.rhohat = (1 - sim.corrs[sim]^2)/sqrt(n.obs)
  se.covhat = sqrt((sig1hat2*sig2hat2)*(1+sim.corrs[sim]^2))/sqrt(n.obs)
  rho.lower[sim] = sim.corrs[sim] - 2*se.rhohat
  rho.upper[sim] = sim.corrs[sim] + 2*se.rhohat
  cov.lower[sim] = sim.covs[sim] - 2*se.covhat
  cov.upper[sim] = sim.covs[sim] + 2*se.covhat
}

```

The histograms of these values, with the asymptotic normal distributions overlaid, is displayed in Figure 7.8.

The histogram for the $\hat{\sigma}_{12}$ values is bell-shaped with and centered at the true value $\sigma_{12} = 0.00375$. The sample mean and standard deviation values of $\hat{\sigma}_{12}$ across the 1000 simulations are, respec-

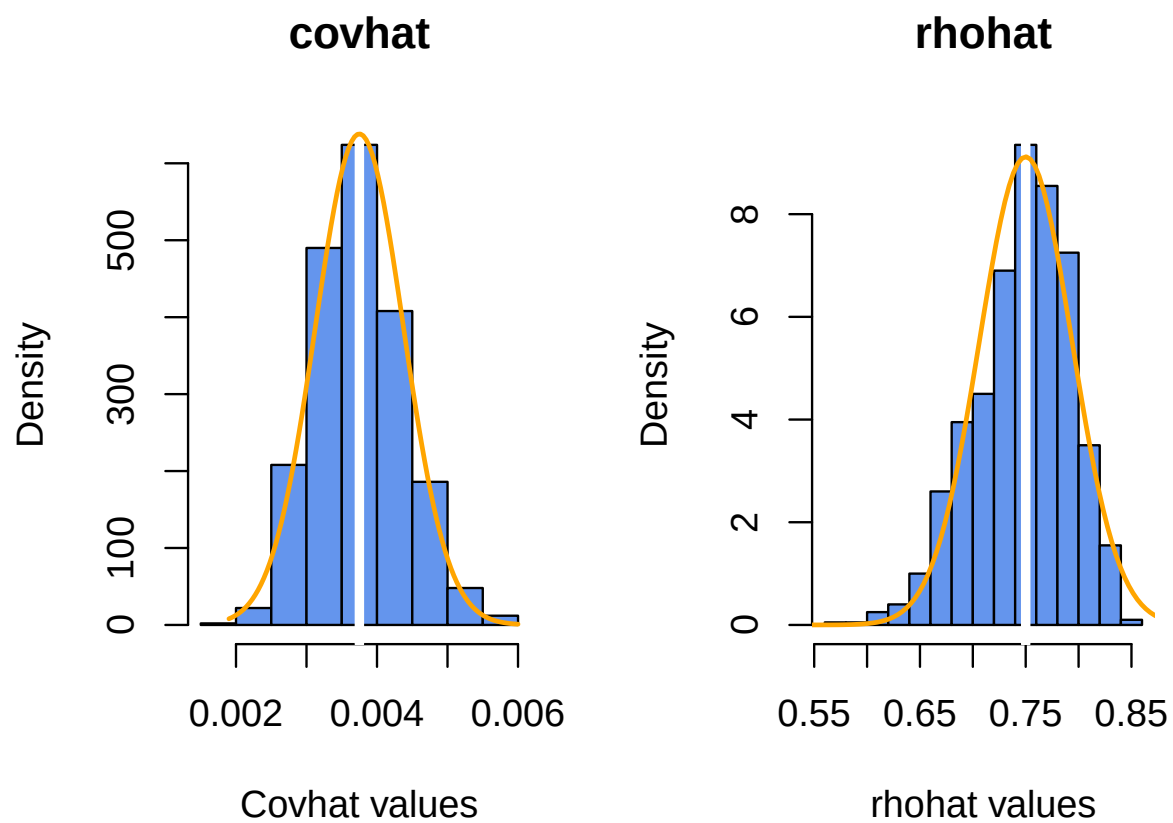


Figure 7.8: Histograms of $\hat{\rho}_{12}$ computed from $N = 1000$ Monte Carlo samples from the bivariate GWN model

tively,

$$\begin{aligned}\bar{\hat{\sigma}}_{12} &= \frac{1}{1000} \sum_{j=1}^{1000} \hat{\sigma}_{12}^j = 0.00376, \\ \hat{\sigma}_{\hat{\sigma}_{12}} &= \sqrt{\frac{1}{999} \sum_{j=1}^{1000} (\hat{\sigma}_{12}^j - 0.00376)^2} = 0.000628.\end{aligned}$$

Here, you see that $\hat{\sigma}_{12}$ is essentially unbiased and the Monte Carlo standard error is almost identical to the analytic standard based the asymptotic distribution of $\hat{\sigma}_{12}$.

The histogram for the $\hat{\rho}_{12}$ values is bell-shaped with a mild left skewness and centered close to $\rho_{12} = 0.75$. The sample mean and standard deviation values of $\hat{\rho}_{12}$ across the 1000 simulations are, respectively,

$$\begin{aligned}\bar{\hat{\rho}}_{12} &= \frac{1}{1000} \sum_{j=1}^{1000} \hat{\rho}_{12}^j = 0.747, \\ \hat{\sigma}_{\hat{\rho}_{12}} &= \sqrt{\frac{1}{999} \sum_{j=1}^{1000} (\hat{\rho}_{12}^j - 0.747)^2} = 0.045.\end{aligned}$$

There is very slight downward bias in $\hat{\rho}_{12}$. The Monte Carlo standard deviation, $\hat{\sigma}_{\hat{\rho}_{12}}$, is very close to the approximate standard error $\text{se}(\hat{\rho}_{12}) = (1 - 0.75^2)/\sqrt{100} = 0.044$.

For each simulation j , the approximate 95% confidence intervals

$$\begin{aligned}\hat{\sigma}_{12}^j \pm 2 \times \widehat{\text{se}}(\hat{\rho}_{12}^j), \\ \hat{\rho}_{12}^j \pm 2 \times \widehat{\text{se}}(\hat{\rho}_{12}^j),\end{aligned}$$

are computed. The coverage probabilities of these interval is approximated by the fractions of intervals that contain (cover) the true values $\sigma_{12} = 0.00375$ and $\rho_{12} = 0.75$, respectively. For the 1000 simulated samples, this fractions turns out to be 0.942 adn 0.952, respectively.

Here is the R code to support the above calculations. The Monte Carlo means, standard deviations, and analytic standard errors are:

```
secovhat.a = sqrt((sigmavec[1]^2*sigmavec[2]^2)*(1+rho12^2))/sqrt(n.obs)
serhohat.a = (1 - rho12^2)/sqrt(n.obs)
ans = rbind(c(cov12, mean(sim.covs),sd(sim.covs),secovhat.a),
             c(rho12, mean(sim.corr), sd(sim.corr), serhohat.a))
colnames(ans) = c("True Value", "MC Mean", "MC SE", "Analytic SE")
rownames(ans) = c("Covariance", "Correlation")
ans
```

```
#>           True Value MC Mean    MC SE Analytic SE
#> Covariance    0.00375 0.00376 0.000628    0.000625
#> Correlation    0.75000 0.74685 0.045036    0.043750
```

The fractions of confidence intervals that cover the true parameters are:

```
in.interval.rho = (rho12 >= rho.lower) & (rho12 <= rho.upper)
in.interval.cov = (cov12 >= cov.lower) & (cov12 <= cov.upper)
ans = rbind(c(sum(in.interval.cov)/n.sim,0.95),
            c(sum(in.interval.rho)/n.sim,0.95))
colnames(ans) = c("MC Coverage", "True Coverage")
rownames(ans) = c("Covariance", "Correlation")
ans
#>           MC Coverage True Coverage
#> Covariance    0.942            0.95
#> Correlation    0.952            0.95
```

7.7 Sampling distributions of $\hat{\mu}$ and $\hat{\Sigma}$

Consider the GWN model in matrix form Equation 7.1. The estimator of the $N \times 1$ matrix μ is the sample mean vector Equation 7.16 and the estimator of the $N \times N$ matrix Σ is the sample covariance matrix `?@eq-SigmaHat`. In this section I give the sampling distributions for the mean vector $\hat{\mu}$ and for the elements of the covariance matrix $\hat{\Sigma}$.

7.7.1 Sampling distribution of $\hat{\mu}$

The following proposition gives the sampling distribution of the $N \times 1$ vector $\hat{\mu}$.

Proposition 7.8 (Sampling distribution of $\hat{\mu}$). *Under the GWN model and for large enough T , the sampling distribution for $\hat{\mu}$ is multivariate normal:*

$$\hat{\mu} \sim N\left(\mu, \frac{1}{T}\hat{\Sigma}\right).$$

This result shows that the elements of the vector $\hat{\mu}$ are correlated, with the correlations being the same as the asset return correlations.

Example 7.9 (Joint distribution of $\hat{\mu}_1$ and $\hat{\mu}_2$). Consider the bivariate case ($N = 2$). Proposition 7.8 tells us the joint distribution between $\hat{\mu}_1$ and $\hat{\mu}_2$ is bivariate normal:

$$\begin{pmatrix} \hat{\mu}_1 \\ \hat{\mu}_2 \end{pmatrix} \sim N \left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \frac{1}{T} \begin{pmatrix} \hat{\sigma}_1^2 & \hat{\sigma}_{12} \\ \hat{\sigma}_{12} & \hat{\sigma}_2^2 \end{pmatrix} \right).$$

Hence, $\widehat{\text{cov}}(\hat{\mu}_1, \hat{\mu}_2) = \frac{1}{T} \hat{\sigma}_{12}$ and $\widehat{\text{cor}}(\hat{\mu}_1, \hat{\mu}_2) = \hat{\rho}_{12}$.

The fact that the elements of $\hat{\mu}$ are correlated and the correlations are equal to the correlations between the corresponding assets is a useful result. For example, if assets 1 and 2 are strongly positively (negatively) correlated, then $\hat{\mu}_1$ and $\hat{\mu}_2$ are strongly positively (negatively) correlated. As a result, a positive (negative) estimate of $\hat{\mu}_1$ will tend to be associated with a positive (negative) estimate of $\hat{\mu}_2$. ■

7.7.2 Sampling distribution of the elements of $\hat{\Sigma}$

You can also determine the sampling distribution for the vector of unique elements of $\hat{\Sigma}$. This requires a bit of new matrix notation because the joint distribution of a $N \times N$ symmetric matrix of random variables is defined as the joint distribution of the $N(N+1)/2$ elements of the matrix stacked into a $N(N+1)/2 \times 1$ vector.

Define $\hat{\theta}$ as the $N(N+1)/2 \times 1$ vector of unique elements of Σ . These elements can be extracted using the $\text{vech}(\cdot)$ operator, which extracts the elements on and below the main diagonal of $\hat{\Sigma}$ column-wise and stacks them into a $N(N+1)/2$ vector. For example, if $N = 3$ then $\hat{\theta}$ is the 6×1 vector

$$\hat{\theta} = \text{vech}(\hat{\Sigma}) = \text{vech} \begin{pmatrix} \hat{\sigma}_1^2 & \hat{\sigma}_{12} & \hat{\sigma}_{13} \\ \hat{\sigma}_{12} & \hat{\sigma}_2^2 & \hat{\sigma}_{23} \\ \hat{\sigma}_{13} & \hat{\sigma}_{23} & \hat{\sigma}_3^2 \end{pmatrix} = \begin{pmatrix} \hat{\sigma}_1^2 \\ \hat{\sigma}_{12} \\ \hat{\sigma}_{13} \\ \hat{\sigma}_2^2 \\ \hat{\sigma}_{23} \\ \hat{\sigma}_3^2 \end{pmatrix}.$$

Proposition 7.9 (Sampling distribution of the unique elements of $\hat{\Sigma}$). *Under the GWN model and for large enough T , the sampling distribution for the $N(N+1)/2 \times 1$ vector $\hat{\theta} = \text{vech}(\hat{\Sigma})$ is multivariate normal:*

$$\hat{\theta} \sim N \left(\theta, \frac{1}{T} \hat{\Omega} \right).$$

where $\frac{1}{T} \hat{\Omega}$ is an $N(N+1)/2 \times N(N+1)/2$ matrix giving the estimated variances and covariances of the unique elements of $\hat{\Sigma}$. The element of $\frac{1}{T} \hat{\Omega}$ corresponding to $\widehat{\text{cov}}(\hat{\sigma}_{ij}, \hat{\sigma}_{lm})$ is given by $T^{-1}(\hat{\sigma}_{il}\hat{\sigma}_{jm} + \hat{\sigma}_{im}\hat{\sigma}_{jl})$ for all $i, j, l, m = 1, 2, \dots, N$, including $i = j = l = m$. If $i = j$ and $l = m$ then

$$\begin{aligned}\widehat{\text{cov}}(\hat{\sigma}_{ij}, \hat{\sigma}_{lm}) &= \widehat{\text{cov}}(\hat{\sigma}_{ii}, \hat{\sigma}_{ll}) = \widehat{\text{cov}}(\hat{\sigma}_i^2, \hat{\sigma}_l^2) \\ &= \frac{\hat{\sigma}_{il}\hat{\sigma}_{il} + \hat{\sigma}_{il}\hat{\sigma}_{il}}{T} = \frac{\hat{\sigma}_{il}^2 + \hat{\sigma}_{il}^2}{T} = \frac{2\hat{\sigma}_{il}^2}{T}\end{aligned}$$

If $i = j = l = m$ then

$$\begin{aligned}\widehat{\text{cov}}(\hat{\sigma}_{ij}, \hat{\sigma}_{lm}) &= \widehat{\text{cov}}(\hat{\sigma}_i^2, \hat{\sigma}_i^2) = \widehat{\text{var}}(\hat{\sigma}_i^2) \\ &= \frac{\hat{\sigma}_i^2\hat{\sigma}_i^2 + \hat{\sigma}_i^2\hat{\sigma}_i^2}{T} = \frac{2\hat{\sigma}_i^4}{T}\end{aligned}$$

In Proposition 7.9, it is not easy to see what are the elements of $\hat{\Omega}$. A simple example can make things more clear. Consider the bivariate case $N = 2$. Then the result in Proposition 7.9 gives:

$$\begin{aligned}\hat{\theta} = \begin{pmatrix} \hat{\sigma}_1^2 \\ \hat{\sigma}_{12} \\ \hat{\sigma}_2^2 \end{pmatrix} &\sim N \left(\begin{pmatrix} \sigma_1^2 \\ \sigma_{12} \\ \sigma_2^2 \end{pmatrix}, \frac{1}{T} \begin{pmatrix} \hat{\omega}_{11} & \hat{\omega}_{12} & \hat{\omega}_{13} \\ \hat{\omega}_{21} & \hat{\omega}_{22} & \hat{\omega}_{23} \\ \hat{\omega}_{31} & \hat{\omega}_{32} & \hat{\omega}_{33} \end{pmatrix} \right) \\ &\sim N \left(\begin{pmatrix} \sigma_1^2 \\ \sigma_{12} \\ \sigma_2^2 \end{pmatrix}, \begin{pmatrix} \widehat{\text{var}}(\hat{\sigma}_1^2) & \widehat{\text{cov}}(\hat{\sigma}_1^2, \hat{\sigma}_{12}) & \widehat{\text{cov}}(\hat{\sigma}_1^2, \hat{\sigma}_2^2) \\ \widehat{\text{cov}}(\hat{\sigma}_{12}, \hat{\sigma}_1^2) & \widehat{\text{var}}(\hat{\sigma}_{12}) & \widehat{\text{cov}}(\hat{\sigma}_{12}, \hat{\sigma}_2^2) \\ \widehat{\text{cov}}(\hat{\sigma}_1^2, \hat{\sigma}_2^2) & \widehat{\text{cov}}(\hat{\sigma}_2^2, \hat{\sigma}_{12}) & \widehat{\text{var}}(\hat{\sigma}_2^2) \end{pmatrix} \right) \\ &\sim N \left(\begin{pmatrix} \sigma_1^2 \\ \sigma_{12} \\ \sigma_2^2 \end{pmatrix}, \frac{1}{T} \begin{pmatrix} 2\hat{\sigma}_1^4 & 2\hat{\sigma}_1^2\hat{\sigma}_{12} & 2\hat{\sigma}_{12}^2 \\ 2\hat{\sigma}_1^2\hat{\sigma}_{12} & \hat{\sigma}_1^2\hat{\sigma}_2^2 + \hat{\sigma}_{12}^2 & 2\hat{\sigma}_{12}\hat{\sigma}_2^2 \\ 2\hat{\sigma}_{12} & 2\hat{\sigma}_{12}\hat{\sigma}_2^2 & 2\hat{\sigma}_2^4 \end{pmatrix} \right).\end{aligned}$$

Notice that the results for the diagonal elements of $\frac{1}{T}\hat{\Omega}$ match those given earlier (see equations Equation 7.20 and Equation 7.22). What is new here are formulas for the off-diagonal elements.

7.7.3 Joint sampling distribution between $\hat{\mu}$ and $\text{vech}(\hat{\Sigma})$

When you study hypothesis testing in Chapter Chapter 9 and the statistical analysis of portfolios in Chapter ?@sec-Statistical-Analysis-of-Portfolios you will need to use results on the joint sampling distribution between $\hat{\mu}$ and $\text{vech}(\hat{\Sigma})$. The follow proposition states the result:

Proposition 7.10 (Joint sampling distribution between $\hat{\mu}$ and $\text{vech}(\hat{\Sigma})$). *Under the GWN model and for large enough T , the joint sampling distribution for the $N + N(N+1)/2 \times 1$ vector $(\hat{\mu}', \hat{\theta}')$, where $\hat{\theta} = \text{vech}(\hat{\Sigma})$ is multivariate normal*

$$\begin{pmatrix} \hat{\mu} \\ \hat{\theta} \end{pmatrix} \sim N \left(\begin{pmatrix} \mu \\ \theta \end{pmatrix}, \frac{1}{T} \begin{pmatrix} \hat{\Sigma} & 0 \\ 0 & \hat{\Omega} \end{pmatrix} \right).$$

Proposition 7.10 tells us that the vectors $\hat{\mu}$ and $\hat{\theta}$ are jointly multivariate normally distributed and that all elements of $\hat{\mu}$ are independent of all the elements of $\hat{\theta}$.

Example 7.10 (Joint distribution of $\hat{\mu}$, and $\hat{\sigma}$). Consider the GWN model for a single asset and suppose you are interested in the joint distribution of the vector $(\hat{\mu}, \hat{\sigma}^2)'$. Then Proposition 7.10 gives us the result:

$$\begin{pmatrix} \hat{\mu} \\ \hat{\sigma}^2 \end{pmatrix} \sim N \left(\begin{pmatrix} \mu \\ \sigma^2 \end{pmatrix}, \begin{pmatrix} \frac{\hat{\sigma}^2}{T} & 0 \\ 0 & \frac{2\hat{\sigma}^4}{T} \end{pmatrix} \right). \quad (7.44)$$

Here, you see that $\text{cov}(\hat{\mu}, \hat{\sigma}^2) = 0$, which implies that $\hat{\mu}$ and $\hat{\sigma}^2$ are independent since they are jointly normally distributed. Furthermore, since $\hat{\mu}$ and $\hat{\sigma}^2$ are independent and any function of $\hat{\mu}$ is independent of any function of $\hat{\sigma}^2$. For example, $\hat{\sigma} = \sqrt{\hat{\sigma}^2}$ is independent of $\hat{\mu}$. Using Equation 7.34 you can then deduce the joint distribution of $(\hat{\mu}, \hat{\sigma}^2)'$:

$$\begin{pmatrix} \hat{\mu} \\ \hat{\sigma} \end{pmatrix} \sim N \left(\begin{pmatrix} \mu \\ \sigma \end{pmatrix}, \begin{pmatrix} \frac{\hat{\sigma}^2}{T} & 0 \\ 0 & \frac{\hat{\sigma}^2}{2T} \end{pmatrix} \right). \quad (7.45)$$

■

7.8 Further Reading: Estimation of The GWN Model

- Good introductory statistical theory is given in (DeGroot and Schervish 2011). For an econometrics focus see (Goldberger 1991). More advanced books - (Casella and Berger 2002), Bickell and Dokson.
- R books on statistics, Monte Carlo and bootstrapping: (Vinod 2010) (bootstrapping)
- Surprisingly, the statistical properties of VaR are often not discussed much in standard textbook treatments. See, for example (Jorion 1997).

7.9 Problems: Estimation of The GWN Model

Exercise 7.1. Show that the matrix sample statistics Equation 7.16 and Equation 7.17 are unbiased estimators for μ and Σ , respectively.

8 Estimating Functions of GWN Model Parameters

Updated: September 5, 2026

Copyright © Eric Zivot 2026

In the previous chapter, I considered estimation of the parameters of the GWN model for asset returns and the construction of estimated standard errors and confidence intervals for the estimated parameters. In this chapter, I consider estimation of functions of the parameters of the GWN model, and the construction of estimated standard errors and confidence intervals for these estimates. This is an important topic as you are often more interested in a function of the GWN model parameters (e.g., VaR_α) than the GWN model parameters themselves. In order to do this, I introduce some new statistical tools including the analytic delta method based on asymptotic distribution theory, and two computer intensive resampling methods called the jackknife and the bootstrap.

The R packages I use in this chapter are **boot**, **bootstrap**, **car**, **introcompfinr**, and **PerformanceAnalytics**. Have these packages installed and loaded in R before replicating the chapter examples.

8.1 Functions of GWN Model Parameters

Consider the GWN model for returns (simple or continuously compounded):

$$\begin{aligned} R_{it} &= \mu_i + \varepsilon_{it}, \\ \{\varepsilon_{it}\}_{t=1}^T &\sim \text{GWN}(0, \sigma^2), \\ \text{cov}(\varepsilon_{it}, \varepsilon_{js}) &= \begin{cases} \sigma_{ij} & t = s \\ 0 & t \neq s \end{cases}. \end{aligned}$$

Let θ denote a $k \times 1$ vector of GWN model parameters from which you want to compute a function of interest (I drop the asset subscript i to simplify notation). Let $f : R^k \rightarrow R$ be a continuous and

differentiable function of θ , and define $\eta = f(\theta)$ as the scalar parameter of interest. To make things concrete, let $\theta = (\mu, \sigma)'$. Consider the following functions of θ :

$$f_1(\theta) = \mu + \sigma \times q_\alpha^Z = q_\alpha^R \quad (8.1)$$

$$f_2(\theta) = -W_0 \times (\mu + \sigma \times q_\alpha^Z) = W_0 \times f_1(\theta) = \text{VaR}_\alpha^N \quad (8.2)$$

$$f_3(\theta) = -W_0 \times (\exp(\mu + \sigma \times q_\alpha^Z) - 1) = W_0 \times (\exp(f_1(\theta)) - 1) = \text{VaR}_\alpha^{LN} \quad (8.3)$$

The first function Equation 8.1 is the α -quantile, q_α^R , of the $N(\mu, \sigma^2)$ distribution for simple returns. The second function Equation 8.2 is VaR_α of an initial investment of W_0 based on the normal distribution for simple returns. The third function Equation 8.3 is VaR_α based on the log-normal distribution for simple returns (normal distribution for continuously compounded returns). The first two functions are linear functions of the elements of θ , and the third function is a nonlinear function of the elements of θ .

Given the plug-in estimate of θ , $\hat{\theta} = (\hat{\mu}, \hat{\sigma})'$, I want to estimate each of the above functions of θ , and construct estimated standard errors and confidence intervals.

Example 8.1 (Data for examples). For the examples in this chapter, I use data on monthly continuously compounded and simple returns for Microsoft over the period January 1998, through May 2012, from the **introcompfinr** package. The data is the same as that used in Chapter 5 and is constructed as follows:

```
msftPrices = to.monthly(msftDailyPrices, OHLC=FALSE)
smp1 = "1998-01::2012-05"
msftPrices = msftPrices[smp1]
msftRetS = na.omit(Return.calculate(msftPrices, method="discrete"))
msftRetC = log(1 + msftRetS)
```

The GWN model estimates for μ_{msft} and σ_{msft} , with estimated standard errors, are:

```
n.obs = nrow(msftRetC)
muhatC = mean(msftRetC)
sigmahatC = sd(msftRetC)
muhatS = mean(msftRetS)
sigmahatS = sd(msftRetS)
estimates = c(muhatC, sigmahatC, muhatS, sigmahatS)
se.muhatC = sigmahatC/sqrt(n.obs)
se.muhatS = sigmahatS/sqrt(n.obs)
se.sigmahatC = sigmahatC/sqrt(2*n.obs)
```

```

se.sigmahatS = sigmahatS/sqrt(2*n.obs)
stdErrors = c(se.muhatC, se.sigmahatC, se.muhatS, se.sigmahatS)
ans = rbind(estimates, stdErrors)
colnames(ans) = c("mu.cc", "sigma.cc", "mu.simple", "sigma.simple")
ans
#>           mu.cc sigma.cc mu.simple sigma.simple
#> estimates 0.00413   0.1002   0.00915      0.10150
#> stdErrors 0.00764   0.0054   0.00774      0.00547

```

■

8.2 Estimation of Functions of GWN Model Parameters

In Chapter Chapter 7, I used the *plug-in principle* to motivate estimators of the GWN model parameters μ_i , σ_i^2 , σ_{ij} , and ρ_{ij} . I can also use the plug-in principle to estimate *functions* of the GWN model parameters.

Definition 8.1 (Plug-in principle for functions of GWN model parameters). Let θ denote a $k \times 1$ vector of GWN model parameters, and let $\eta = f(\theta)$ where $f : \mathbb{R}^k \rightarrow \mathbb{R}$ is a scalar function of θ . Let $\hat{\theta}$ denote the plug-in estimator of θ . Then the plug-in estimator of η is $\hat{\eta} = f(\hat{\theta})$.

Example 8.2 (Plug-in estimates of example functions for example data). Let $W_0 = \$100,000$, and $r_f = 0.03/12 = 0.0025$. Using the plug-in estimates of the GWN model parameters, the plug-in estimates of the functions Equation 8.1 - Equation 8.3 are:

```

W0 = 100000
r.f = 0.03/12
f1.hat = muhatS + sigmahatS*qnorm(0.05)
f2.hat = -W0*f1.hat
f3.hat = -W0*(exp(muhatC + sigmahatC*qnorm(0.05)) - 1)
fhat.vals = cbind(f1.hat, f2.hat, f3.hat)
colnames(fhat.vals) = c("f1", "f2", "f3")
rownames(fhat.vals) = "Estimate"
fhat.vals
#>           f1      f2      f3
#> Estimate -0.158 15780 14846

```

■

Plug-in estimators of functions are random variables and are subject to estimation error. Just as you studied the statistical properties of the plug-in estimators of the GWN model parameters you can study the statistical properties of the plug-in estimators of functions of GWN model parameters.

8.2.1 Bias

As discussed in Chapter Chapter 7, a desirable finite sample property of an estimator $\hat{\theta}$ of θ is *unbiasedness*: on average over many hypothetical samples $\hat{\theta}$ is equal to θ . When estimating $f(\theta)$, unbiasedness of $\hat{\theta}$ may or may not carry over to $f(\hat{\theta})$.

Suppose $f(\theta)$ is a linear function of the elements of θ so that

$$f(\theta) = a + b_1\theta_1 + b_2\theta_2 + \cdots + b_k\theta_k.$$

where $a, b_1, b_2, \dots, b_k$ are constants. For example, the functions Equation 8.1 and Equation 8.2 are linear functions of the elements of $\theta = (\mu, \sigma)'$. Let $\hat{\theta}$ denote an estimator of θ and let $f(\hat{\theta})$ denote the plug-in estimator of $f(\theta)$. If $E[\hat{\theta}] = \theta$ for all elements of θ ($\hat{\theta}_i$ is unbiased for θ_i , $i = 1, \dots, k$), then

$$\begin{aligned} E[f(\hat{\theta})] &= a + b_1E[\hat{\theta}_1] + b_2E[\hat{\theta}_2] + \cdots + b_kE[\hat{\theta}_k] \\ &= a + b_1\theta_1 + b_2\theta_2 + \cdots + b_k\theta_k = f(\theta) \end{aligned}$$

and so $f(\hat{\theta})$ is unbiased for $f(\theta)$.

Now suppose $f(\theta)$ is a nonlinear function of the elements of θ . For example, the function Equation 8.3 is a nonlinear function of the elements of $\theta = (\mu, \sigma)'$. Then, in general, $f(\hat{\theta})$ is not unbiased for $f(\theta)$ even if $\hat{\theta}$ is unbiased for θ . The direction of the bias depends on the properties of $f(\theta)$. If $f(\cdot)$ is convex at the scalar parameter θ or concave at θ then you can say something about the direction of the bias in $f(\hat{\theta})$.

Definition 8.2 (Convex function). Let θ be a scalar parameter. A scalar function $f(\theta)$ is *convex* (in two dimensions) if all points on a straight line connecting any two points on the graph of $f(\theta)$ is above or on that graph. More formally, $f(\theta)$ is convex if for all θ_1 and θ_2 and for all $0 \leq \alpha \leq 1$

$$f(\alpha\theta_1 + (1 - \alpha)\theta_2) \leq \alpha f(\theta_1) + (1 - \alpha)f(\theta_2)$$

For example, the function $f(\theta) = \theta^2$ is a convex function. Another common convex function is $\exp(\theta)$.

Definition 8.3 (Concave function). Let θ be a scalar parameter. A scalar function $f(\theta)$ is *concave* if $-f(\theta)$ is convex.

For example, the function $f(\theta) = -\theta^2$ is concave. Other common concave functions are $\log(\theta)$ and $\sqrt{\theta}$.

Proposition 8.1 (Jensen's inequality). Let θ be a scalar parameter and let $\hat{\theta}$ be an unbiased estimate of θ so that $E[\hat{\theta}] = \theta$. Let $f(\theta)$ be a convex function of θ . Then

$$E[f(\hat{\theta})] \geq f(E[\hat{\theta}]) = f(\theta).$$

Jensen's inequality tells you that if f is convex (concave) and $\hat{\theta}$ is unbiased then $f(\hat{\theta})$ is positively (negatively) biased. For example, you know from Chapter 7 that $\hat{\sigma}^2$ is unbiased for σ^2 . Now, $\hat{\sigma} = f(\hat{\sigma}^2) = \sqrt{\hat{\sigma}^2}$ is a concave function of $\hat{\sigma}^2$. From Jensen's inequality, you know that $\hat{\sigma}$ is negatively biased (i.e., too small). ■

8.2.2 Consistency

An important property of the plug-in estimators of the GWN model parameters is that they are *consistent*: as the sample size gets larger and larger the estimators get closer and closer to their true values and eventually equal their true values for an infinitely large sample.

Consistency also holds for plug-in estimators of functions of GWN model parameters. The justification comes from *Slutsky's Theorem*.

Proposition 8.2 (Slutsky's Theorem). Let $\hat{\theta}$ be consistent for θ , and let $f : \mathbb{R}^k \rightarrow \mathbb{R}$ be continuous at θ . Then $f(\hat{\theta})$ is consistent for $f(\theta)$.

The key condition on $f(\theta)$ is continuity at θ .¹

Example 8.3 (Consistency of functions of GWN model parameters). All of the functions Equation 8.1 - Equation 8.3 are continuous at $\theta = (\mu, \sigma)'$. By Slutsky's Theorem, all of the plug-in estimators of the functions are also consistent. ■

¹Recall, the intuitive definition of continuity is that you can draw the function on paper without lifting up your pen.

8.3 Asymptotic normality and the delta method

Let θ denote a scalar parameter of the GWN model for returns and let $\hat{\theta}$ denote the plug-in estimator of θ . In Chapter Chapter 7 you learned that $\hat{\theta}$ is asymptotically normally distributed

$$\hat{\theta} \sim N(\theta, \widehat{\text{se}}(\hat{\theta})^2),$$

for large enough T , where $\widehat{\text{se}}(\hat{\theta})$ denotes the estimated standard error of $\hat{\theta}$. This result is justified by the CLT. The following proposition establishes the asymptotic distribution of the plug-in estimator of a function of θ using a technique called the *delta method*.

Proposition 8.3 (The delta method). *Let $f(\theta)$ denote a real-valued continuous and differentiable scalar function of θ . Using a technique called the delta method, you can show that the plug-in estimator $f(\hat{\theta})$ is also asymptotically normally distributed:*

$$f(\hat{\theta}) \sim N(f(\theta), \widehat{\text{se}}(f(\hat{\theta}))^2),$$

for large enough T , where $\widehat{\text{se}}(f(\hat{\theta}))$ is the estimated standard error for $f(\hat{\theta})$.

The delta method shows you how to compute $\widehat{\text{se}}(f(\hat{\theta}))^2$:

$$\widehat{\text{se}}(f(\hat{\theta}))^2 = f'(\hat{\theta})^2 \times \widehat{\text{se}}(\hat{\theta})^2,$$

where $f'(\hat{\theta})$ denotes the derivative of $f(\theta)$ evaluated at $\hat{\theta}$. Then,

$$\widehat{\text{se}}(f(\hat{\theta})) = \sqrt{f'(\hat{\theta})^2 \times \widehat{\text{se}}(\hat{\theta})^2}. \quad (8.4)$$

Example 8.4 (Delta method standard error for $\hat{\sigma}$). The results from Chapter Chapter 7 showed, for large enough T ,

$$\hat{\sigma}^2 \sim N(\sigma^2, \text{se}(\hat{\sigma}^2)^2), \quad \hat{\sigma} \sim N(\sigma, \text{se}(\hat{\sigma})^2),$$

where $\text{se}(\sigma^2)^2 = 2\sigma^4/T$ and $\text{se}(\sigma)^2 = \sigma^2/2T$. The formula for $\text{se}(\hat{\sigma})^2$ is the result of the delta method for the function $f(\sigma^2) = \sqrt{\sigma^2} = \sigma$ applied to the asymptotic distribution of $\hat{\sigma}^2$. To see this, first note that by the chain rule

$$f'(\sigma^2) = \frac{1}{2}(\sigma^2)^{-1/2},$$

so that $f'(\sigma^2)^2 = \frac{1}{4}(\sigma^2)^{-1}$. Then,

$$\begin{aligned}\text{se}(f(\hat{\theta}))^2 &= f'(\sigma^2)^2 \times \text{se}(\hat{\sigma}^2)^2 = \frac{1}{4}(\sigma^2)^{-1} \times 2\sigma^4/T \\ &= \frac{1}{2}\sigma^2/T = \sigma^2/2T \\ &= \text{se}(\hat{\sigma})^2.\end{aligned}$$

The estimated squared standard error replaces σ^2 with its estimate $\hat{\sigma}^2$ and is given by

$$\widehat{\text{se}}(\hat{\sigma}^2)^2 = \hat{\sigma}^2/2T.$$

■

Now suppose θ is a $k \times 1$ vector of GWN model parameters, and $f : \mathbb{R}^k \rightarrow \mathbb{R}$ is a continuous and differentiable function of θ . Define the $k \times 1$ *gradient function* $\mathbf{g}(\theta)$ as:

$$\mathbf{g}(\theta) = \frac{\partial f(\theta)}{\partial \theta} = \begin{pmatrix} \frac{\partial f(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial f(\theta)}{\partial \theta_k} \end{pmatrix}.$$

Assume that $\hat{\theta}$ is asymptotically normally distributed:

$$\hat{\theta} \sim N(\theta, \widehat{\text{var}}(\hat{\theta})),$$

where $\widehat{\text{var}}(\hat{\theta})$ is the $k \times k$ estimated variance-covariance matrix of $\hat{\theta}$. Then the delta method shows you that $f(\hat{\theta})$ is asymptotically normally distributed:

$$f(\hat{\theta}) \sim N(f(\theta), \widehat{\text{se}}(f(\hat{\theta}))^2),$$

where

$$\widehat{\text{se}}(f(\hat{\theta}))^2 = \mathbf{g}(\hat{\theta})' \widehat{\text{var}}(\hat{\theta}) \mathbf{g}(\hat{\theta}).$$

Example 8.5 (Delta method standard error for $\hat{q}_\alpha^Z$). Let $\theta = (\mu, \sigma)'$ and consider $f_1(\theta) = q_\alpha^R = \mu + \sigma q_\alpha^Z$ (normal simple return quantile). The gradient function $\mathbf{g}_1(\theta)$ for $f_1(\theta)$ is:

$$\mathbf{g}_1(\theta) = \frac{\partial f_1(\theta)}{\partial \theta} = \begin{pmatrix} \frac{\partial f_1(\theta)}{\partial \mu} \\ \frac{\partial f_1(\theta)}{\partial \sigma} \end{pmatrix} = \begin{pmatrix} 1 \\ q_\alpha^Z \end{pmatrix}.$$

Regarding $\text{var}(\hat{\theta})$, recall from Chapter Chapter 7, in the GWN model:

$$\begin{pmatrix} \hat{\mu} \\ \hat{\sigma}^2 \end{pmatrix} \sim N \left(\begin{pmatrix} \mu \\ \sigma^2 \end{pmatrix}, \begin{pmatrix} \text{se}(\hat{\mu})^2 & 0 \\ 0 & \text{se}(\hat{\sigma}^2)^2 \end{pmatrix} \right)$$

for large enough T . I show below, using the delta method, that

$$\begin{pmatrix} \hat{\mu} \\ \hat{\sigma} \end{pmatrix} \sim N \left(\begin{pmatrix} \mu \\ \sigma \end{pmatrix}, \begin{pmatrix} \text{se}(\hat{\mu})^2 & 0 \\ 0 & \text{se}(\hat{\sigma})^2 \end{pmatrix} \right)$$

Hence,

$$\text{var}(\hat{\theta}) = \begin{pmatrix} \text{se}(\hat{\mu})^2 & 0 \\ 0 & \text{se}(\hat{\sigma})^2 \end{pmatrix} = \begin{pmatrix} \frac{\sigma^2}{T} & 0 \\ 0 & \frac{\sigma^2}{2T} \end{pmatrix}$$

Then,

$$\begin{aligned} \mathbf{g}_1(\theta)' \text{var}(\hat{\theta}) \mathbf{g}_1(\theta) &= \begin{pmatrix} 1 & q_\alpha^Z \end{pmatrix} \begin{pmatrix} \text{se}(\hat{\mu})^2 & 0 \\ 0 & \text{se}(\hat{\sigma})^2 \end{pmatrix} \begin{pmatrix} 1 \\ q_\alpha^Z \end{pmatrix} \\ &= \text{se}(\hat{\mu})^2 + (q_\alpha^Z)^2 \text{se}(\hat{\sigma})^2 \\ &= \frac{\sigma^2}{T} + \frac{(q_\alpha^Z)^2 \sigma^2}{2T} \\ &= \frac{\sigma^2}{T} \left[1 + \frac{1}{2} (q_\alpha^Z)^2 \right] \\ &= \text{se}(\hat{q}_\alpha^Z)^2. \end{aligned}$$

The standard error for $\hat{q}_\alpha^Z$ is:

$$\text{se}(\hat{q}_\alpha^R) = \sqrt{\text{var}(\hat{q}_\alpha^R)} = \frac{\sigma}{\sqrt{T}} \left[1 + \frac{1}{2} (q_\alpha^Z)^2 \right]^{1/2}. \quad (8.5)$$

The formula Equation 8.6 shows that $\text{se}(\hat{q}_\alpha^R)$ increases with σ and q_α^Z , and decreases with T . In addition, for fixed T , $\text{se}(\hat{q}_\alpha^R) \rightarrow \infty$ as $\alpha \rightarrow 0$ or as $\alpha \rightarrow 1$ because $q_\alpha^Z \rightarrow -\infty$ as $\alpha \rightarrow 0$ and $q_\alpha^Z \rightarrow \infty$ as $\alpha \rightarrow 1$. Hence, there is very large estimation error in $\hat{q}_\alpha^R$ when α is very close to zero or one

(for fixed values of σ and T). This makes intuitive sense as you have very few observations in the extreme tails of the empirical distribution of the data for a fixed sample size and so you expect a larger estimation error.

Replacing σ with its estimate $\hat{\sigma}$ gives the *practically useful* estimated standard error for $\hat{q}_\alpha^R$:

$$\widehat{\text{se}}(\hat{q}_\alpha^R) = \frac{\hat{\sigma}}{\sqrt{T}} \left[1 + \frac{1}{2} (q_\alpha^Z)^2 \right]^{1/2}. \quad (8.6)$$

Using the above results, the sampling distribution of $\hat{q}_\alpha^R$ can be approximated by the normal distribution:

$$\hat{q}_\alpha^R \sim N(q_\alpha^R, \widehat{\text{se}}(\hat{q}_\alpha^R)^2), \quad (8.7)$$

for large enough T . ■

Example 8.6 (Applying the delta method to $\hat{q}_\alpha^R$ for Microsoft). The estimates of q_α^R , $\text{se}(\hat{q}_\alpha^R)$, and 95% confidence intervals, for $\alpha = 0.05, 0.01$ and 0.001 , from the simple monthly returns for Microsoft are:

```
qhat = muhatS + sigmahatS*qnorm(c(0.05,0.01,0.001))
seQhat = (sigmahatS/sqrt(n.obs))*sqrt(1 + 0.5*qnorm(c(0.05,0.01,0.001))^2)
lowerQhat = qhat - 2*seQhat
upperQhat = qhat + 2*seQhat
width = upperQhat - lowerQhat
ans = cbind(qhat,seQhat,lowerQhat,upperQhat,width)
colnames(ans) = c("Estimates", "Std Errors", "2.5%", "97.5%", "Width")
rownames(ans) = c("q.05", "q.01", "q.001")
ans
```

#>	Estimates	Std Errors	2.5%	97.5%	Width
#> q.05	-0.158	0.0119	-0.182	-0.134	0.0475
#> q.01	-0.227	0.0149	-0.257	-0.197	0.0596
#> q.001	-0.304	0.0186	-0.342	-0.267	0.0744

For Microsoft the values of $\widehat{\text{se}}(\hat{q}_\alpha^R)$ are about 1.2%, 1.5%, and 1.9% for $\alpha = 0.05$, $\alpha = 0.01$ and $\alpha = 0.001$, respectively. The 95% confidence interval widths increase with decreases in α indicating more uncertainty about the true value of q_α^R for smaller values of α . For example, the 95% confidence for interval $q_{0.01}^R$ indicates that the true loss with 1% probability (one month out of 100 months) is between 19.7% and 25.7%. ■

Example 8.7 (Delta method standard errors for $\widehat{\text{VaR}}_\alpha^N$ and $\widehat{\text{VaR}}_\alpha^{LN}$). Let $\theta = (\mu, \sigma)'$ and consider the functions $f_2(\theta) = \text{VaR}_\alpha^N = -W_0(\mu + \sigma q_\alpha^Z)$ (normal VaR), and $f_3(\theta) = \text{VaR}_\alpha^{LN} = -W_0(\exp((\mu + \sigma q_\alpha^Z)) - 1)$ (log-normal VaR). It is straightforward to show (see end-of-chapter exercises) that the gradient functions are:

$$\begin{aligned}\mathbf{g}_2(\theta) &= \frac{\partial f_2(\theta)}{\partial \theta} = \begin{pmatrix} -W_0 \\ -W_0 q_\alpha^Z \end{pmatrix}, \\ \mathbf{g}_3(\theta) &= \frac{\partial f_3(\theta)}{\partial \theta} = \begin{pmatrix} -W_0 \exp(\mu + \sigma q_\alpha^Z) \\ -W_0 \exp(\mu + \sigma q_\alpha^Z) q_\alpha^Z \end{pmatrix}.\end{aligned}$$

The delta method variances are:

$$\begin{aligned}\mathbf{g}_2(\theta)' \text{var}(\hat{\theta}) \mathbf{g}_2(\theta) &= W_0^2 \text{se}(\hat{q}_\alpha^R)^2 = \text{se}\left(\widehat{\text{VaR}}_\alpha^N\right)^2 \\ \mathbf{g}_3(\theta)' \text{var}(\hat{\theta}) \mathbf{g}_3(\theta) &= W_0^2 \exp(q_\alpha^R)^2 \text{se}(\hat{q}_\alpha^R)^2 = \text{se}\left(\widehat{\text{VaR}}_\alpha^{LN}\right)^2.\end{aligned}$$

The corresponding standard errors are:

$$\begin{aligned}\text{se}\left(\widehat{\text{VaR}}_\alpha^N\right) &= W_0 \text{se}(\hat{q}_\alpha^R) \\ \text{se}\left(\widehat{\text{VaR}}_\alpha^{LN}\right) &= W_0 \exp(q_\alpha^R) \text{se}(\hat{q}_\alpha^R).\end{aligned}$$

where

$$\text{se}(\hat{q}_\alpha^Z) = \sqrt{\frac{\sigma^2}{T} \left[1 + \frac{1}{2} (q_\alpha^Z)^2\right]}.$$

Remarks:

1. $\text{se}(\widehat{\text{VaR}}_\alpha^N)$ and $\text{se}(\widehat{\text{VaR}}_\alpha^{LN})$ increase with W_0 , σ , and q_α^Z , and decrease with T .
2. For fixed values of W_0 , σ , and T , $\text{se}(\widehat{\text{VaR}}_\alpha^N)$ and $\text{se}(\widehat{\text{VaR}}_\alpha^{LN})$ become very large as α approaches zero. That is, for very small loss probabilities you will have very imprecise estimates of VaR_α^N and VaR_α^{LN} .
3. The practically useful estimated standard errors replace the unknown value of q_α^R with its estimated value $\hat{q}_\alpha^R$ and are given by:

$$\widehat{\text{se}}\left(\widehat{\text{VaR}}_{\alpha}^N\right) = W_0 \widehat{\text{se}}(\hat{q}_{\alpha}^R) \quad (8.8)$$

$$\widehat{\text{se}}\left(\widehat{\text{VaR}}_{\alpha}^{LN}\right) = W_0 \exp(\hat{q}_{\alpha}^R) \widehat{\text{se}}(\hat{q}_{\alpha}^R) \quad (8.9)$$

■

Example 8.8 (Applying the delta method to $\widehat{\text{VaR}}_{\alpha}^N$ and $\widehat{\text{VaR}}_{\alpha}^{LN}$ for Microsoft). Consider a \$100,000 investment for one month in Microsoft. The estimates of VaR_{α}^N , its standard error, and 95% confidence intervals, for $\alpha = 0.05$, $\alpha = 0.01$, and $\alpha = 0.001$ are:

```
VaR.N = -qhat*W0
seVaR.N = W0*seQhat
lowerVaR.N = VaR.N - 2*seVaR.N
upperVaR.N = VaR.N + 2*seVaR.N
width = upperVaR.N - lowerVaR.N
ans = cbind(VaR.N, seVaR.N, lowerVaR.N, upperVaR.N, width)
colnames(ans) = c("Estimate", "Std Error", "2.5%", "97.5%", "Width")
rownames(ans) = c("VaR.N.05", "VaR.N.01", "VaR.N.001")
ans
```

#>	Estimate	Std Error	2.5%	97.5%	Width
#> VaR.N.05	15780	1187	13405	18154	4748
#> VaR.N.01	22696	1490	19717	25676	5959
#> VaR.N.001	30450	1860	26730	34169	7439

The estimated values of $\text{VaR}_{.05}^N$, $\text{VaR}_{.01}^N$, and $\text{VaR}_{.001}^N$ are \$15,780, \$22,696, and \$30,450 respectively, with estimated standard errors of \$1,187, \$1,490, and \$1,860. The estimated standard errors are fairly small compared with the estimated VaR values, so you can say that VaR is estimated fairly precisely here. Notice that the estimated standard error for $\text{VaR}_{.001}^N$ is about 56% larger than the estimated standard error for $\text{VaR}_{.05}^N$ indicating you have much less precision for estimating VaR for very small values of α .

The estimates of VaR_{α}^{LN} and its standard error for $\alpha = 0.05$ and $\alpha = 0.01$ are:

```
qhat.c = muhatC + sigmahatC*qnorm(c(0.05,0.01,0.001))
seQhat.c = (sigmahatC/sqrt(n.obs))*sqrt(1 + 0.5*qnorm(c(0.05,0.01,0.001))^2)
VaR.LN = -W0*(exp(qhat.c)-1)
seVaR.LN = W0*exp(qhat.c)*seQhat.c
lowerVaR.LN = VaR.LN - 2*seVaR.LN
upperVaR.LN = VaR.LN + 2*seVaR.LN
width = upperVaR.LN - lowerVaR.LN
```

```

ans = cbind(VaR.LN, seVaR.LN, lowerVaR.LN, upperVaR.LN, width)
colnames(ans) = c("Estimate", "Std Error", "2.5%", "97.5%", "Width")
rownames(ans) = c("VaR.LN.05", "VaR.LN.01", "VaR.LN.001")
ans
#>           Estimate Std Error  2.5% 97.5% Width
#> VaR.LN.05      14846      998 12850 16842  3992
#> VaR.LN.01      20467     1170 18128 22807  4680
#> VaR.LN.001     26329     1353 23623 29034  5411

```

The estimates of VaR_α^{LN} (and its standard error) are slightly smaller than the estimates of VaR_α^N (and its standard error) due to the positive skewness in the log-normal distribution. ■

8.3.1 The numerical delta method

The delta method is an advanced statistical technique and requires a bit of calculus to implement, especially for nonlinear vector-valued functions. It can be easy to make mistakes when working out the math! Fortunately, in R there is an easy way to implement the delta method that does not require you to work out the math. The **car** function `deltaMethod` implements the delta method for scalar valued functions *numerically*, utilizing the **stats** function `D()` which implements symbolic derivatives. The arguments expected by `deltaMethod` are:

```

args(car::deltaMethod.default)
#> function (object, g., vcov., func = g., constants, level = 0.95,
#>      rhs = NULL, singular.ok = FALSE, ..., envir = parent.frame())
#> NULL

```

Here, `object` is the vector of named estimated parameters, $\hat{\theta}$, `g.` is a quoted string that is the function of the parameter estimates to be evaluated, $f(\theta)$, and `vcov` is the named estimated covariance matrix of the coefficient estimates, $\widehat{\text{var}}(\hat{\theta})$. The function returns an object of class “deltaMethod” for which there is a `print` method. The “deltaMethod” object is essentially a `data.frame` with columns giving the parameter estimate(s), the delta method estimated standard error(s), and lower and upper confidence limits. The following example illustrates how to use `deltaMethod` for the example functions.

Example 8.9 (Numerical delta method standard errors for example data). I first create the named inputs $\hat{\theta}$ and $\widehat{\text{var}}(\hat{\theta})$ from the GWN model for simple returns:

```

thetahatS = c(muhatS, sigmahatS)
names(thetahatS) = c("mu", "sigma")
var.thetahatS = matrix(c(se.muhatS^2, 0, 0, se.sigmahatS^2), 2, 2, byrow=TRUE)

```

```

rownames(var.thetahatS) = colnames(var.thetahatS) = names(thetahatS)
thetahatS
#>      mu    sigma
#> 0.00915 0.10150
var.thetahatS
#>           mu    sigma
#> mu    5.99e-05 0.00e+00
#> sigma 0.00e+00 2.99e-05

```

I do the same for the inputs from the GWN model for continuously compounded returns:

```

thetahatC = c(muhatC, sigmahatC)
names(thetahatC) = c("mu", "sigma")
var.thetahatC = matrix(c(se.muhatC^2, 0, 0, se.sigmahatC^2), 2, 2, byrow=TRUE)
rownames(var.thetahatC) = colnames(var.thetahatC) = names(thetahatC)

```

It is important to name the elements in `thetahat` and `var.thetahat` as these names will be used in defining $f(\theta)$. To implement the numerical delta method for the normal quantile, you need to supply a quoted string specifying the function $f_1(\theta) = \mu + \sigma q_\alpha^Z$. Here I use the string `"mu+sigma*q.05"`, where `q.05 = qnorm(0.05)`. For, $\alpha = 0.05$ the R code is:

```

q.05 = qnorm(0.05)
dm1 = deltaMethod(thetahatS, "mu+sigma*q.05", var.thetahatS)
class(dm1)
#> [1] "deltaMethod" "data.frame"

```

The returned object `dm1` is of class “deltaMethod”, inheriting from “data.frame”, and has named columns:

```

names(dm1)
#> [1] "Estimate" "SE"      "2.5 %"    "97.5 %"

```

The `print` method shows the results of applying the delta method:

```

dm1
#>           Estimate      SE    2.5 % 97.5 %
#> mu + sigma * q.05 -0.1578 0.0119 -0.1811 -0.13

```

The results match the analytic calculations performed above.²

²The 95% confidence interval computed in `deltaMethod` uses 1.96 instead of the rule-of-thumb value of 2 used in the analytic calculations.

The delta method for the normal VaR function $f_2(\theta)$ is:

```
dm2 = deltaMethod(thetahatS, "-W0*(mu+sigma*q.05)", var.thetahatS)
dm2
#>                                Estimate      SE 2.5 % 97.5 %
#> -W0 * (mu + sigma * q.05)      15780   1187 13453  18106
```

The delta method for the log-normal VaR function $f_3(\theta)$ is:

```
dm3 = deltaMethod(thetahatC, "-W0*(exp(mu+sigma*q.05)-1)", var.thetahatC)
dm3
#>                                Estimate      SE 2.5 % 97.5 %
#> -W0 * (exp(mu + sigma * q.05) - 1)  14846   998 12890  16802
```

8.3.2 The delta method for vector valued functions (advanced)

Let θ denote a $k \times 1$ vector of GWN model parameters. For example, $\theta = (\mu, \sigma^2)'$. Assume that $\hat{\theta}$ is asymptotically normally distributed:

$$\hat{\theta} \sim N(\theta, \widehat{\text{var}}(\theta)),$$

for large enough T where $\widehat{\text{var}}(\theta)$ is the $k \times k$ variance-covariance matrix of $\hat{\theta}$. Sometimes you need to consider the asymptotic distribution of a vector-valued function of θ . Let $\mathbf{f}: \mathbb{R}^k \rightarrow \mathbb{R}^j$ where $j \leq k$ be a continuous and differentiable function, and define the $j \times 1$ vector $\eta = \mathbf{f}(\theta)$. It is useful to express η as:

$$\eta = \begin{pmatrix} \eta_1 \\ \eta_2 \\ \vdots \\ \eta_j \end{pmatrix} = \begin{pmatrix} f_1(\theta) \\ f_2(\theta) \\ \vdots \\ f_j(\theta) \end{pmatrix}.$$

Example 8.10 (Example vector functions of GWN model parameters). Let $\theta = (\sigma_1^2, \sigma_2^2)'$, and define the 2×1 vector of volatilities: $\eta = \mathbf{f}(\theta) = (\sigma_1, \sigma_2)'$. Then, $\eta_1 = f_1(\theta) = \sqrt{\sigma_1^2}$ and $\eta_2 = f_2(\theta) = \sqrt{\sigma_2^2}$. ■

Define the $j \times k$ Jacobian matrix:

$$\mathbf{G}(\theta)' = \frac{\partial \mathbf{f}(\theta)}{\partial \theta'} = \begin{pmatrix} \frac{\partial f_1(\theta)}{\partial \theta_1} & \frac{\partial f_1(\theta)}{\partial \theta_2} & \dots & \frac{\partial f_1(\theta)}{\partial \theta_k} \\ \frac{\partial f_2(\theta)}{\partial \theta_1} & \frac{\partial f_2(\theta)}{\partial \theta_2} & \dots & \frac{\partial f_2(\theta)}{\partial \theta_k} \\ \vdots & \vdots & \dots & \vdots \\ \frac{\partial f_j(\theta)}{\partial \theta_1} & \frac{\partial f_j(\theta)}{\partial \theta_2} & \dots & \frac{\partial f_j(\theta)}{\partial \theta_k} \end{pmatrix} \quad (8.10)$$

Then, by the delta method, the asymptotic normal distribution of $\hat{\eta} = \mathbf{f}(\hat{\theta})$ is:

$$\hat{\eta} \sim N(\eta, \widehat{\text{var}}(\hat{\eta})),$$

where

$$\widehat{\text{var}}(\hat{\eta}) = \mathbf{G}(\hat{\theta})' \widehat{\text{var}}(\hat{\theta}) \mathbf{G}(\hat{\theta}). \quad (8.11)$$

Example 8.11 (Asymptotic distributions of example functions). Consider the first example function where $\theta = (\sigma_1^2, \sigma_2^2)'$ and $\eta = \mathbf{f}(\theta) = (\sigma_1, \sigma_2)'$. Now, using the chain-rule:

$$\begin{aligned} \frac{\partial f_1(\theta)}{\partial \sigma_1^2} &= \frac{1}{2} \sigma_1^{-1}, \quad \frac{\partial f_1(\theta)}{\partial \sigma_2^2} = 0 \\ \frac{\partial f_2(\theta)}{\partial \sigma_1^2} &= 0, \quad \frac{\partial f_2(\theta)}{\partial \sigma_2^2} = \frac{1}{2} \sigma_1^{-1} \end{aligned}$$

The Jacobian matrix is

$$\mathbf{G}(\theta)' = \begin{pmatrix} \frac{1}{2} \sigma_1^{-1} & 0 \\ 0 & \frac{1}{2} \sigma_2^{-1} \end{pmatrix}$$

From Chapter Chapter 7, Proposition 7.9,

$$\widehat{\text{var}}(\hat{\theta}) = \frac{1}{T} \begin{pmatrix} 2\hat{\sigma}_1^4 & 2\hat{\sigma}_{12}^2 \\ 2\hat{\sigma}_{12}^2 & 2\hat{\sigma}_2^4 \end{pmatrix}$$

Then

$$\begin{aligned} \widehat{\text{var}}(\hat{\eta}) &= \mathbf{G}(\hat{\theta})' \widehat{\text{var}}(\hat{\theta}) \mathbf{G}(\hat{\theta}) \\ &= \begin{pmatrix} \frac{1}{2} \hat{\sigma}_1^{-1} & 0 \\ 0 & \frac{1}{2} \hat{\sigma}_2^{-1} \end{pmatrix} \frac{1}{T} \begin{pmatrix} 2\hat{\sigma}_1^4 & 2\hat{\sigma}_{12}^2 \\ 2\hat{\sigma}_{12}^2 & 2\hat{\sigma}_2^4 \end{pmatrix} \begin{pmatrix} \frac{1}{2} \hat{\sigma}_1^{-1} & 0 \\ 0 & \frac{1}{2} \hat{\sigma}_2^{-1} \end{pmatrix} \\ &= \frac{1}{T} \begin{pmatrix} \frac{1}{2} \hat{\sigma}_1^2 & \frac{1}{2} \hat{\sigma}_{12}^2 \hat{\sigma}_1^{-1} \hat{\sigma}_2^{-1} \\ \frac{1}{2} \hat{\sigma}_{12}^2 \hat{\sigma}_1^{-1} \hat{\sigma}_2^{-1} & \frac{1}{2} \hat{\sigma}_2^2 \end{pmatrix} = \frac{1}{T} \begin{pmatrix} \frac{1}{2} \hat{\sigma}_1^2 & \frac{1}{2} \hat{\sigma}_{12} \hat{\rho}_{12} \\ \frac{1}{2} \hat{\sigma}_{12} \hat{\rho}_{12} & \frac{1}{2} \hat{\sigma}_2^2 \end{pmatrix}. \end{aligned}$$



8.3.3 The numerical delta method for vector valued functions (advanced)

- TBD
- Use the delta method implemented in **metafor** and **msm**

8.3.4 The delta method explained

The delta method deduces the asymptotic distribution of $f(\hat{\theta})$ using a first order Taylor series approximation of $f(\hat{\theta})$ evaluated at the true value θ . For simplicity, assume that θ is a scalar parameter. Then, the first order Taylor series expansion gives

$$\begin{aligned} f(\hat{\theta}) &= f(\theta) + f'(\theta)(\hat{\theta} - \theta) + \text{remainder} \\ &= f(\theta) + f'(\theta)\hat{\theta} + f'(\theta)\theta + \text{remainder} \\ &\approx f(\theta) + f'(\theta)\hat{\theta} + f'(\theta)\theta \end{aligned}$$

The first order Taylor series expansion shows that $f(\hat{\theta})$ is an approximate linear function of the random variable $\hat{\theta}$. The values θ , $f(\theta)$, and $f'(\theta)$ are constants. Assuming that, for large enough T ,

$$\hat{\theta} \sim N(\theta, \widehat{\text{se}}(\hat{\theta})^2),$$

it follows that

$$f(\hat{\theta}) \sim N(f(\theta), f'(\theta)^2 \widehat{\text{se}}(\hat{\theta})^2)$$

Since $\hat{\theta}$ is consistent for θ you can replace $f'(\theta)^2$ with $f'(\hat{\theta})^2$ giving the practically useful result

$$f(\hat{\theta}) \sim N(f(\theta), f'(\hat{\theta})^2 \widehat{\text{se}}(\hat{\theta})^2)$$

8.4 Resampling

As described in the previous sections, the delta method can be used to produce analytic expressions for standard errors of functions of estimated parameters that are asymptotically normally distributed. The formulas are specific to each function and are approximations that rely on large samples and the validity of the Central Limit Theorem to be accurate.

Alternatives to the analytic asymptotic distributions of estimators and the delta method are computer intensive *resampling methods* such as the *jackknife* and the *bootstrap*. These resampling methods can be used to produce numerical estimated standard errors for estimators and functions of estimators without the use of mathematical formulas. Most importantly, these jackknife and bootstrap standard errors are: (1) easy to compute, and (2) often numerically very close to the analytic standard errors based on CLT approximations. In some cases the bootstrap standard errors can even be more reliable than asymptotic approximations.

There are many advantages of the jackknife and bootstrap over analytic formulas. The most important are the following:

- *Fewer assumptions.* The jackknife and bootstrap procedures typically requires fewer assumptions than are needed to derive analytic formulas. In particular, they do not require the data to be normally distributed or the sample size to be large enough so that the Central Limit Theorem holds.
- *Greater accuracy.* In small samples the bootstrap is often more reliable than analytic approximations based on the Central Limit Theorem. In large samples when the Central Limit Theorem holds the bootstrap can be even be more accurate.
- *Generality.* In many situations, the jackknife and bootstrap procedures are applied in the same way regardless of the estimator under consideration. That is, you don't need a different jackknife or bootstrap procedure for each estimator.

8.5 The Jackknife

Let θ denote a scalar parameter of interest. It could be a scalar parameter of the GWN model (e.g. μ or σ) or it could be a scalar function of the parameters of the GWN model (e.g., VaR). Let $\hat{\theta}$ denote the plug-in estimator of θ . The goal is to compute a numerical estimated standard error for $\hat{\theta}$, $\widehat{\text{se}}(\hat{\theta})$, using the jackknife resampling method.

The jackknife resampling method utilizes T resamples of the original data $\{R_t\}_{t=1}^T$. The resamples are created by leaving out a single observation. For example, the first jackknife resample leaves out the first observation and is given by $\{R_t\}_{t=2}^T$. This sample has $T-1$ observations. In general, the t^{th} jackknife resample leaves out the t^{th} return R_t for $t \leq T$. Now, on each of the $t = 1, \dots, T$ jackknife resamples you compute an estimate of θ . Denote this jackknife estimate $\hat{\theta}_{-t}$ to indicate that θ is

estimated using the resample that omits observation t . The estimator $\hat{\theta}_{-t}$ is called the *leave-one-out* estimator of θ . Computing $\hat{\theta}_{-t}$ for $t = 1, \dots, T$ gives T leave-one-out estimators $\{\hat{\theta}_{-t}\}_{t=1}^T$.

The *jackknife estimator* of $\widehat{\text{se}}(\hat{\theta})$ is the scaled standard deviation of $\{\hat{\theta}_{-t}\}_{t=1}^T$:

$$\widehat{\text{se}}_{jack}(\hat{\theta}) = \sqrt{\frac{T-1}{T} \sum_{t=1}^T (\hat{\theta}_{-t} - \bar{\theta})^2}, \quad (8.12)$$

where $\bar{\theta} = T^{-1} \sum_{t=1}^T \hat{\theta}_{-t}$ is the sample mean of $\{\hat{\theta}_{-t}\}_{t=1}^T$. The estimator Equation 8.12 is easy to compute and does not require any analytic calculations from the asymptotic distribution of $\hat{\theta}$.

The intuition for the jackknife standard error formula can be illustrated by considering the jackknife standard error estimate for the sample mean $\hat{\theta} = \hat{\mu}$. You know that the analytic estimated squared standard error for $\hat{\mu}$ is

$$\widehat{\text{se}}(\hat{\mu})^2 = \frac{\hat{\sigma}^2}{T} = \frac{1}{T} \frac{1}{T-1} \sum_{t=1}^T (R_t - \hat{\mu})^2.$$

Now consider the jackknife squared standard error for $\hat{\mu}$:

$$\widehat{\text{se}}_{jack}(\hat{\mu})^2 = \frac{T-1}{T} \sum_{t=1}^T (\hat{\mu}_{-t} - \bar{\mu})^2,$$

where $\bar{\mu} = T^{-1} \sum_{t=1}^T \hat{\mu}_{-t}$. Here, I will show that $\widehat{\text{se}}(\hat{\mu})^2 = \widehat{\text{se}}_{jack}(\hat{\mu})^2$.

First, note that

$$\begin{aligned} \hat{\mu}_{-t} &= \frac{1}{T-1} \sum_{j \neq t} R_j = \frac{1}{T-1} \left(\sum_{j=1}^T R_j - R_t \right) \\ &= \frac{1}{T-1} (T\hat{\mu} - R_t) \\ &= \frac{T}{T-1} \hat{\mu} - \frac{1}{T-1} R_t. \end{aligned}$$

Next, plugging in the above to the definition of $\bar{\mu}$ I get

$$\begin{aligned}
\bar{\mu} &= \frac{1}{T} \sum_{j=1}^T \hat{\mu}_{-t} = \frac{1}{T} \sum_{t=1}^T \left(\frac{T}{T-1} \hat{\mu} - \frac{1}{T-1} R_t \right) \\
&= \frac{1}{T-1} \sum_{t=1}^T \hat{\mu} - \frac{1}{T} \frac{1}{T-1} \sum_{t=1}^T R_t \\
&= \frac{T}{T-1} \hat{\mu} - \frac{1}{T-1} \hat{\mu} \\
&= \hat{\mu} \left(\frac{T}{T-1} - \frac{1}{T-1} \right) \\
&= \hat{\mu}.
\end{aligned}$$

Hence, the sample mean of $\hat{\mu}_{-t}$ is $\hat{\mu}$. Then I can write

$$\begin{aligned}
\hat{\mu}_{-t} - \bar{\mu} &= \hat{\mu}_{-t} - \hat{\mu} \\
&= \frac{T}{T-1} \hat{\mu} - \frac{1}{T-1} R_t - \hat{\mu} \\
&= \frac{T}{T-1} \hat{\mu} - \frac{1}{T-1} R_t - \frac{T-1}{T-1} \hat{\mu} \\
&= \frac{1}{T-1} \hat{\mu} - \frac{1}{T-1} R_t \\
&= \frac{1}{T-1} (\hat{\mu} - R_t).
\end{aligned}$$

Using the above I can write

$$\begin{aligned}
\widehat{\text{se}}_{jack}(\hat{\mu})^2 &= \frac{T-1}{T} \sum_{t=1}^T (\hat{\mu}_{-t} - \bar{\mu})^2 \\
&= \frac{T-1}{T} \left(\frac{1}{T-1} \right)^2 \sum_{t=1}^T (\hat{\mu} - R_t)^2 \\
&= \frac{1}{T} \frac{1}{T-1} \sum_{t=1}^T (\hat{\mu} - R_t)^2 \\
&= \frac{1}{T} \hat{\sigma}^2 \\
&= \widehat{\text{se}}(\hat{\mu})^2.
\end{aligned}$$

Example 8.12 (Jackknife standard error for $\hat{\mu}$ for Microsoft). You can compute the leave-one-out estimates $\{\hat{\mu}_{-t}\}_{t=1}^T$ using a simple `for` loop as follows:

```

n.obs = length(msftRetS)
muhat.loo = rep(0, n.obs)
for(i in 1:n.obs) {
  muhat.loo[i] = mean(msftRetS[-i])
}

```

In the above loop, `msftRetS[-i]` extracts all observations except observation i . The mean of the leave-one-out estimates is

```

mean(muhat.loo)
#> [1] 0.00915

```

which is the sample mean of `msftRetS`. It is informative to plot a histogram of the leave-one-out estimates:

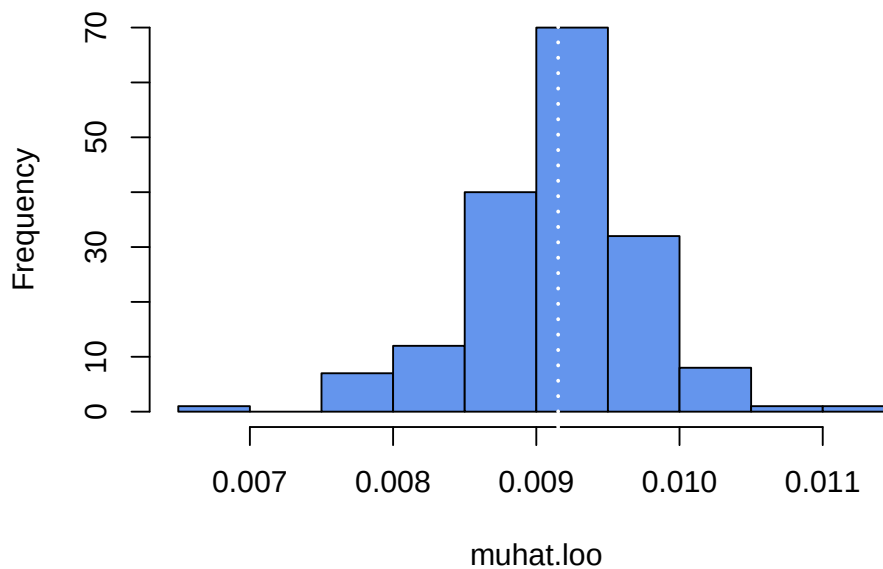


Figure 8.1: Histogram of leave-one-out estimator of μ for Microsoft.

The white dotted line is at the point $\hat{\mu} = 0.00915$. The jackknife estimated standard error is the scaled standard deviation of the $T \hat{\mu}_{-t}$ values:³

³the scaling of the standard deviation of $\hat{\mu}_{-t}$ is approximately $\sqrt{T}$.

```
se.jack.muhat = sqrt(((n.obs - 1)^2)/n.obs)*sd(muhat.loo)
se.jack.muhat
#> [1] 0.00774
```

This value is exactly the same as the analytic standard error:

```
se.muhatS
#> [1] 0.00774
```

You can also compute jackknife estimated standard errors using the **bootstrap** function **jackknife**:

```
library(bootstrap)
muhat.jack = jackknife(coredata(msftRetS), mean)
names(muhat.jack)
#> [1] "jack.se"      "jack.bias"      "jack.values" "call"
```

By default, **jackknife()** takes as input a vector of data and the name of an R function and returns a list with components related to the jackknife procedure. The component **jack.values** contains the leave-one-out estimators, and the component **jack.se** gives you the jackknife estimated standard error:

```
muhat.jack$jack.se
#> [1] 0.00774
```

■

The fact that the jackknife standard error estimate for $\hat{\mu}$ equals the analytic standard error is motivates using the jackknife to compute standard errors for general estimators. For estimated parameters other than $\hat{\mu}$, and general estimated functions of GWN model parameters, the jackknife standard errors are often numerically very close to the delta method standard errors. The following example illustrates this point for the example functions Equation 8.1 - Equation 8.3.

Example 8.13 (Jackknife standard errors for plug-in estimators of example functions). Here, I use the **bootstrap** function **jackknife()** to compute jackknife estimated standard errors for the plug-in estimates of the example functions Equation 8.1 - Equation 8.3 and I compare the jackknife standard errors with the delta method standard errors. To use **jackknife()** with user-defined functions involving multiple parameters you must create the function such that the first input is an integer vector of index observations and the second input is the data. For example, a function for computing Equation 8.1 to be passed to **jackknife()** is:

```

theta.f1 = function(x, xdata) {
  mu.hat = mean(xdata[x, ])
  sigma.hat = sd(xdata[x, ])
  fun.hat = mu.hat + sigma.hat*qnorm(0.05)
  fun.hat
}

```

To compute the jackknife estimated standard errors for the estimate of Equation 8.1 use:

```

f1.jack = jackknife(1:n.obs, theta=theta.f1, coredata(msftRetS))
ans = cbind(f1.jack$jack.se, dm1$SE)
colnames(ans) = c("Jackknife", "Delta Method")
rownames(ans) = "f1"
ans
#>      Jackknife Delta Method
#> f1      0.0133      0.0119

```

Here, you see that the estimated jackknife standard error for $f_1(\hat{\theta})$ is very close to the delta method standard error computed earlier.

To compute the jackknife estimated standard error for $f_2(\hat{\theta})$ use:

```

theta.f2 = function(x, xdata, W0) {
  mu.hat = mean(xdata[x, ])
  sigma.hat = sd(xdata[x, ])
  fun.hat = -W0*(mu.hat + sigma.hat*qnorm(0.05))
  fun.hat
}
f2.jack = jackknife(1:n.obs, theta=theta.f2, coredata(msftRetS), W0)
ans = cbind(f2.jack$jack.se, dm2$SE)
colnames(ans) = c("Jackknife", "Delta Method")
rownames(ans) = "f2"
ans
#>      Jackknife Delta Method
#> f2      1329      1187

```

Again, the jackknife standard error is close to the delta method standard error.

To compute the jackknife estimated standard error for $f_3(\hat{\theta})$ use:

```

theta.f3 = function(x, xdata, W0) {
  mu.hat = mean(xdata[x, ])
  sigma.hat = sd(xdata[x, ])
  fun.hat = -W0*(exp(mu.hat + sigma.hat*qnorm(0.05)) - 1)
  fun.hat
}
f3.jack = jackknife(1:n.obs, theta=theta.f3, coredata(msftRetC), W0)
ans = cbind(f3.jack$jack.se, dm3$SE)
colnames(ans) = c("Jackknife", "Delta Method")
rownames(ans) = "f3"
ans
#>      Jackknife Delta Method
#> f3      1315      998

```

For log-normal VaR, the jackknife standard error is a bit larger than the delta method standard error.

Here, the jackknife and delta method standard errors are very close. ■

8.5.1 The jackknife for vector-valued estimates

The jackknife can also be used to compute an estimated variance matrix of a vector-valued estimate of parameters. Let θ denote a $k \times 1$ vector of GWN model parameters (e.g., $\theta = (\mu, \sigma)'$), and let $\hat{\theta}$ denote the plug-in estimate of θ . Let $\hat{\theta}_{-t}$ denote the leave-one-out estimate of θ , and $\bar{\theta} = T^{-1} \sum_{t=1}^T \hat{\theta}_{-t}$. Then the jackknife estimate of the $k \times k$ variance matrix $\widehat{\text{var}}(\hat{\theta})$ is the scaled sample covariance of $\{\hat{\theta}_{-t}\}_{t=1}^T$:

$$\widehat{\text{var}}_{\text{jack}}(\hat{\theta}) = \frac{T-1}{T} \sum_{t=1}^T (\hat{\theta}_{-t} - \bar{\theta})(\hat{\theta}_{-t} - \bar{\theta})'. \quad (8.13)$$

The jackknife estimated standard errors for the elements of $\hat{\theta}$ are given by the square root of the diagonal elements of $\widehat{\text{var}}_{\text{jack}}(\hat{\theta})$.

The same principle works for computing the estimated variance matrix of a vector of functions of GWN model parameters. Let $\mathbf{f} : R^k \rightarrow R^l$ with $l \leq k$ and define $\eta = \mathbf{f}(\theta)$ denote a $l \times 1$ vector of functions of GWN model parameters. Let $\hat{\eta} = \mathbf{f}(\hat{\theta})$ denote the plug-in estimator of η . Let $\hat{\eta}_{-t}$ denote the leave-one-out estimate of η , and $\bar{\eta} = T^{-1} \sum_{t=1}^T \hat{\eta}_{-t}$. Then the jackknife estimate of the $l \times l$ variance matrix $\widehat{\text{var}}(\hat{\eta})$ is the scaled sample covariance of $\{\hat{\eta}_{-t}\}_{t=1}^T$:

$$\widehat{\text{var}}_{jack}(\hat{\eta}) = \frac{T-1}{T} \sum_{t=1}^T (\hat{\eta}_{-t} - \bar{\eta})(\hat{\eta}_{-t} - \bar{\eta})'. \quad (8.14)$$

Example 8.14 (Jackknife variance estimate for $\hat{\theta} = (\hat{\mu}, \hat{\sigma})'$ for Microsoft simple returns). The **bootstrap** function `jackknife()` function only works for scalar-valued parameter estimates, so you have to compute the jackknife variance estimate by hand using a `for` loop. The R code is:

```
n.obs = length(msftRetS)
muhat.loo = rep(0, n.obs)
sigmahat.loo = rep(0, n.obs)
for(i in 1:n.obs) {
  muhat.loo[i] = mean(msftRetS[-i])
  sigmahat.loo[i] = sd(msftRetS[-i])
}
thetahat.loo = cbind(muhat.loo, sigmahat.loo)
colnames(thetahat.loo) = c("mu", "sigma")
varhat.jack = (((n.obs-1)^2)/n.obs)*var(thetahat.loo)
varhat.jack
#>           mu      sigma
#> mu      5.99e-05 1.49e-05
#> sigma 1.49e-05 6.12e-05
```

The jackknife variance estimate is somewhat close to the analytic estimate:

```
var.thetahatS
#>           mu      sigma
#> mu      5.99e-05 0.00e+00
#> sigma 0.00e+00 2.99e-05
```

Since the first element of $\hat{\theta}$ is $\hat{\mu}$, the (1,1) elements of the two variance matrix estimates match exactly. However, the `varhat.jack` has non-zero off diagonal elements and the (2,2) element corresponding to $\hat{\sigma}$ is considerably bigger than (2,2) element of the analytic estimate. Converting `var.thetahatS` to an estimated correlation matrix gives:

```
cov2cor(varhat.jack)
#>           mu sigma
#> mu      1.000 0.246
#> sigma 0.246 1.000
```

Here, you see that the jackknife shows a small positive correlation between $\hat{\mu}$ and $\hat{\sigma}$ whereas the analytic matrix shows a zero correlation. ■

8.5.2 Pros and Cons of the Jackknife

The jackknife gets its name because it is a useful statistical device.⁴ The main advantages (pros) of the jackknife are:

- It is a good “quick and dirty” method for computing numerical estimated standard errors of estimates that does not rely on asymptotic approximations.
- The jackknife estimated standard errors are often close to the delta method standard errors.

This jackknife, however, has some limitations (cons). The main limitations are:

- It can be computationally burdensome for very large samples (e.g., if used with ultra high frequency data).
- It does not work well for non-smooth functions such as empirical quantiles.
- It is not well suited for computing confidence intervals.
- It is not as accurate as the bootstrap.

8.6 The Nonparametric Bootstrap

In this section, I describe the easiest and most common form of the bootstrap: the *nonparametric bootstrap*. As you shall see, the nonparametric bootstrap procedure is very similar to a Monte Carlo simulation experiment. The main difference is how the random data is generated. In a Monte Carlo experiment, the random data are computer simulated observations from an assumed model (e.g., the GWN model). In the nonparametric bootstrap, the random data is created by resampling with replacement from the original data.

The procedure for the nonparametric bootstrap of an estimate $\hat{\theta}$ of a scalar parameter θ is as follows:

⁴A jackknife is also the name of a very useful folding knife.

1. *Resample.* Create B bootstrap samples by sampling with replacement from the original data $\{r_1, \dots, r_T\}$.⁵ Each bootstrap sample has T observations (same as the original sample)

$$\begin{aligned}\{r_{11}^*, r_{12}^*, \dots, r_{1T}^*\} &= \text{1st bootstrap sample} \\ &\vdots \\ \{r_{B1}^*, r_{B2}^*, \dots, r_{BT}^*\} &= \text{Bth bootstrap sample}\end{aligned}$$

2. *Estimate θ .* From each bootstrap sample estimate θ and denote the resulting estimate $\hat{\theta}^*$. There will be B values of $\hat{\theta}^*$: $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$.
3. *Compute statistics.* Using $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$ compute estimate of bias, standard error, and approximate 95% confidence interval.

This procedure is very similar to the procedure used to perform a Monte Carlo experiment described in Chapter Chapter 7. The main difference is how the hypothetical samples are created. With the nonparametric bootstrap, the hypothetical samples are created by resampling with replacement from the original data. In this regard the bootstrap treats the sample as if it were the population. This ensures that the bootstrap samples inherit the same distribution as the original data – *whatever that distribution may be*. If the original data is normally distributed, then the nonparametric bootstrap samples will also be normally distributed. If the data is Student's t distributed, then the nonparametric bootstrap samples will be Student's t distributed. With Monte Carlo simulation, the hypothetical samples are simulated under an assumed model and distribution. This requires one to specify values for the model parameters and the distribution from which to simulate.

8.6.1 Bootstrap bias estimate

The nonparametric bootstrap can be used to estimate the bias of an estimator $\hat{\theta}$ using the bootstrap estimates $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$. The key idea is to treat the empirical distribution (i.e., histogram) of the bootstrap estimates $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$ as an approximate to the unknown distribution of $\hat{\theta}$.

Recall, the bias of an estimator is defined as

$$E[\hat{\theta}] - \theta.$$

The bootstrap estimate of bias is given by

$$\widehat{\text{bias}}_{\text{boot}}(\hat{\theta}, \theta) = \bar{\theta}^* - \hat{\theta} = \frac{1}{B} \sum_{j=1}^B \hat{\theta}_j^* - \hat{\theta}. \quad (8.15)$$

(bootstrap mean - estimate)

⁵You can also bootstrap data from multiple assets by defining r_t as a vector of asset returns.

The bootstrap bias estimate Equation 8.15 is the difference between the mean of the bootstrap estimates of θ and the sample estimate of θ . This is similar to the Monte Carlo estimate of bias discussed in Chapter Chapter 7. However, the Monte Carlo estimate of bias is the difference between the mean of the Monte Carlo estimates of θ and the true value of θ . The bootstrap estimate of bias does not require knowing the true value of θ . Effectively, the bootstrap treats the sample estimate $\hat{\theta}$ as the population value θ and the bootstrap mean $\bar{\theta}^* = \frac{1}{B} \sum_{j=1}^B \hat{\theta}_j^*$ as an approximation to $E[\hat{\theta}]$. Here, $\widehat{\text{bias}}_{boot}(\hat{\theta}, \theta) = 0$ if the center of the histogram of $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$ is at $\hat{\theta}$.

Given the bootstrap estimate of bias Equation 8.15, you can compute a *bias-adjusted* estimate:

$$\hat{\theta}_{adj} = \hat{\theta} - (\bar{\theta}^* - \hat{\theta}) = 2\hat{\theta} - \bar{\theta}^* \quad (8.16)$$

if the bootstrap estimate of bias is large, it may be tempting to use the bias-adjusted estimate Equation 8.16 in place of the original estimate. This is generally not done in practice because the bias adjustment introduces extra variability into the estimator. So what use is the bootstrap bias estimate? It provides information to you that your estimate contains bias (or not) and this information can influence your decision making based on the estimate.

8.6.2 Bootstrap standard error estimate

for a scalar estimate $\hat{\theta}$, the bootstrap estimate of $\widehat{\text{se}}(\hat{\theta})$ is given by the sample standard deviation of the bootstrap estimates $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$:

$$\widehat{\text{se}}_{boot}(\hat{\theta}) = \sqrt{\frac{1}{B-1} \sum_{j=1}^B (\hat{\theta}_j^* - \bar{\theta}^*)^2}. \quad (8.17)$$

where $\bar{\theta}^* = \frac{1}{B} \sum_{j=1}^B \hat{\theta}_j^*$. Here, $\widehat{\text{se}}_{boot}(\hat{\theta})$ is the size of a typical deviation of a bootstrap estimate, $\hat{\theta}^*$, from the mean of the bootstrap estimates (typical deviation from the middle of the histogram of the bootstrap estimates). This is very closely related to the Monte Carlo estimate of $\widehat{\text{se}}(\hat{\theta})$, which is the sample standard deviation of the estimates of θ from the Monte Carlo samples.

For a $k \times 1$ vector estimate $\hat{\theta}$, the bootstrap estimate of $\widehat{\text{var}}(\hat{\theta})$ is

$$\widehat{\text{var}}(\hat{\theta}) = \frac{1}{B-1} \sum_{j=1}^B (\hat{\theta}_j^* - \bar{\theta}^*)(\hat{\theta}_j^* - \bar{\theta}^*)'. \quad (8.18)$$

8.6.3 Bootstrap confidence intervals

In addition to providing standard error estimates, the bootstrap is commonly used to compute confidence intervals for scalar parameters. There are several ways of computing confidence intervals with the bootstrap and which confidence interval to use in practice depends on the characteristics of the distribution of the bootstrap estimates $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$.

If the bootstrap distribution appears to be normal then you can compute a 95% confidence interval for θ using two simple methods. The first method mimics the rule-of-thumb that is justified by the CLT to compute a 95% confidence interval but uses the bootstrap standard error estimate Equation 8.17:

$$\hat{\theta} \pm 2 \times \widehat{\text{se}}_{boot}(\hat{\theta}) = [\hat{\theta} - 2 \times \widehat{\text{se}}_{boot}(\hat{\theta}), \hat{\theta} + 2 \times \widehat{\text{se}}_{boot}(\hat{\theta})]. \quad (8.19)$$

The 95% confidence interval Equation 8.19 is called the *normal approximation* 95% confidence interval.

The second method directly uses the distribution of the bootstrap estimates $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$ to form a 95% confidence interval:

$$[\hat{q}_{.025}^*, \hat{q}_{.975}^*], \quad (8.20)$$

where $\hat{q}_{.025}^*$ and $\hat{q}_{.975}^*$ are the 2.5% and 97.5% empirical quantiles of $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$, respectively. By construction, 95% of the bootstrap estimates lie in the quantile-based interval Equation 8.20. This 95% confidence interval is called the *bootstrap percentile* 95% confidence interval.

The main advantage of Equation 8.20 over Equation 8.19 is *transformation invariance*. This means that if Equation 8.20 is a 95% confidence interval for θ and $f(\theta)$ is a monotonic function of θ then $[f(\hat{q}_{.025}^*), f(\hat{q}_{.975}^*)]$ is a 95% confidence interval for $f(\theta)$. The interval Equation 8.20 can also be asymmetric whereas Equation 8.19 is symmetric.

The bootstrap normal approximation and percentile 95% confidence intervals Equation 8.19 and Equation 8.20 will perform well (i.e., will have approximately correct coverage probability) if the bootstrap distribution looks normally distributed. However, if the bootstrap distribution is asymmetric then these 95% confidence intervals may have coverage probability different from 95%, especially if the asymmetry is substantial. The basic shape of the bootstrap distribution can be visualized by the histogram of $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$, and the normality of the distribution can be visually evaluated using the normal QQ-plot of $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$. If the histogram is asymmetric and/or not bell-shaped or if the normal QQ-plot is not linear then normality is suspect and the confidence intervals Equation 8.20 and Equation 8.19 may be inaccurate.

If the bootstrap distribution is asymmetric then it is recommended to use Efron's *bias and skewness adjusted bootstrap percentile confidence* (BCA) interval. The details of how to compute the BCA interval are a bit complicated, and so are omitted, and the interested reader is referred to Chapter 10 of Hansen (2011) for a good explanation. Suffice it to say that the 95% BCA method adjusts the 2.5% and 97.5% quantiles of the bootstrap distribution for bias and skewness. The BCA confidence interval can be computed using the **boot** function `boot.ci()` with optional argument `type="bca"`.⁶

8.6.4 The bootstrap for vector-valued estimates

- TBD

8.6.5 Performing the nonparametric bootstrap in R

The nonparametric bootstrap procedure is easy to perform in R. You can implement the procedure by “brute force” in very much the same way as you perform a Monte Carlo experiment. In this approach you program all parts of the bootstrapping procedure. Alternatively, you can use the R package **boot** which contains functions for automating certain parts of the bootstrapping procedure.

8.6.5.1 Brute force implementation

In the brute force implementation you program all parts of the bootstrap procedure. This typically involves three steps:

8.7 Brute force nonparametric bootstrap procedure

1. Sample with replacement from the original data using the R function `sample()` to create $\{r_1^*, r_2^*, \dots, r_T^*\}$. Do this B times.
2. Compute B values of the statistic of interest $\hat{\theta}$ from each bootstrap sample giving $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$. $B = 1,000$ for routine applications of the bootstrap. To minimize simulation noise it may be required to have $B = 10,000$ or higher.
3. Compute the bootstrap bias and SE values from $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$ using the R functions `mean()` and `sd()`, respectively.
4. Compute the histogram and normal QQ-plot of $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$ using the R functions `hist()` and `qqnorm()`, respectively, to see if the bootstrap distribution looks normal.

⁶The bootstrap percentile-t confidence interval is another bias and skewness adjusted confidence interval.

Typically steps 1 and 2 are performed inside a `for` loop or within the R function `apply()`.

In step 1 sampling with replacement for the original data is performed with the R function `sample()`. To illustrate, consider sampling with replacement from the 5×1 return vector:

```
r = c(0.1,0.05,-0.02,0.03,-0.04)
r
#> [1] 0.10 0.05 -0.02 0.03 -0.04
```

Recall, in R you can extract elements from a vector by subsetting based on their location, or index, in the vector. Since the vector `r` has five elements, its index is the integer vector `1:5`. Then to extract the first, second and fifth elements of `r` use the index vector `c(1, 2, 5)`:

```
r[c(1, 2, 5)]
#> [1] 0.10 0.05 -0.04
```

Using this idea, you can extract a random sample (of any given size) with replacement from `r` by creating a random sample with replacement of the integers $\{1, 2, \dots, 5\}$ and using this set of integers to extract the sample from `r`. The R function `sample()` can be used to do this process. When you pass a positive integer value `n` to `sample()`, with the optional argument `replace=TRUE`, it returns a random sample from the set of integers from 1 to `n`. For example, to create a random sample with replacement of size 5 from the integers $\{1, 2, \dots, 5\}$ use:⁷

```
set.seed(123)
idx = sample(5, replace=TRUE)
idx
#> [1] 3 3 2 2 3
```

You can then get a random sample with replacement from the vector `r` by subsetting using the index vector `idx`:

```
r[idx]
#> [1] -0.02 -0.02 0.05 0.05 -0.02
```

This two step process is automated when you pass a vector of observations to `sample()`:

```
set.seed(123)
sample(r, replace=TRUE)
#> [1] -0.02 -0.02 0.05 0.05 -0.02
```

⁷I use `set.seed(123)` to initialize the random number generator in R and allow for reproducible results.

Example 8.15 (Brute force nonparametric bootstrap of GWN model in R). Consider using the nonparametric bootstrap to compute estimates of the bias and standard error for $\hat{\mu}$ in the GWN model for Microsoft. The R code for the brute force “for loop” to implement the nonparametric bootstrap is:

```
B = 1000
muhat.boot = rep(0, B)
n.obs = nrow(msftRetS)
set.seed(123)
for (i in 1:B) {
  boot.data = sample(msftRetS, n.obs, replace=TRUE)
  muhat.boot[i] = mean(boot.data)
}
```

The bootstrap bias estimate is:

```
mean(muhat.boot) - muhatS
#> [1] 0.000483
```

which is very close to zero and confirms that $\hat{\mu}$ is unbiased. The bootstrap standard error estimate is

```
sd(muhat.boot)
#> [1] 0.00796
```

and is equal (to three decimals) to the analytic standard error estimate computed earlier. This confirms that the nonparametric bootstrap accurately computes $\widehat{se}(\hat{\mu})$. Figure 8.2 shows the histogram and normal QQ-plot of the bootstrap estimates created with:

```
par(mfrow=c(1,2))
hist(muhat.boot, col="cornflowerblue")
abline(v=muhatS, col="white", lwd=2)
qqnorm(muhat.boot, col="cornflowerblue")
qqline(muhat.boot)
```

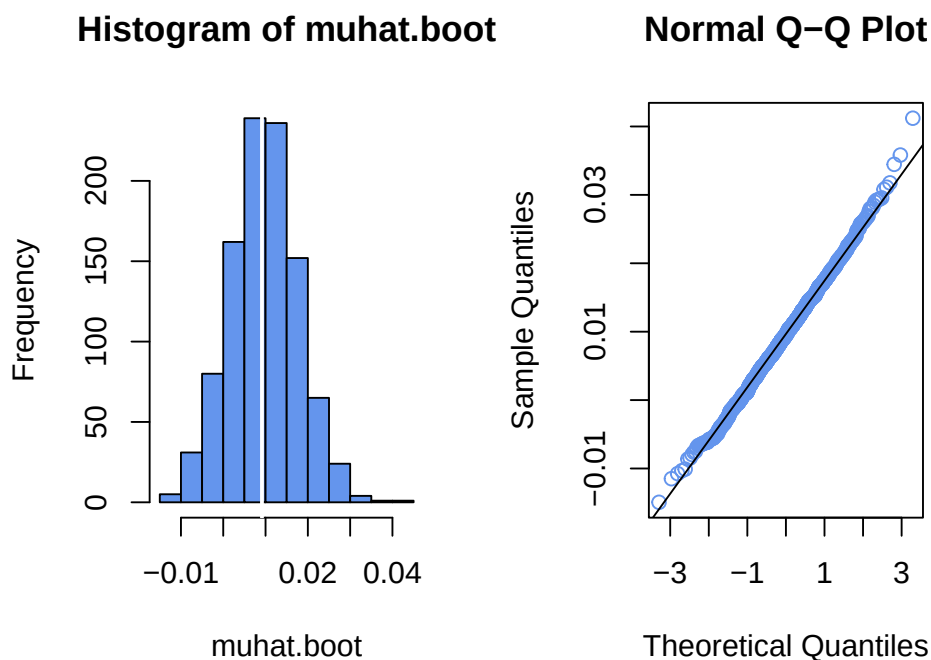


Figure 8.2: Histogram and normal Q-Q-plot of bootstrap estimates of μ for Microsoft.

The distribution of the bootstrap estimates looks normal. As a result, the bootstrap 95% confidence interval for μ has the form:

```
se.boot = sd(muhat.boot)
lower = muhatS + se.boot
upper = muhatS - se.boot
ans = cbind(muhatS, se.boot, lower, upper)
colnames(ans)=c("Estimate", "Std Error", "2.5%", "97.5%")
rownames(ans) = "mu"
ans
#>   Estimate Std Error   2.5%  97.5%
#> mu  0.00915   0.00796 0.0171 0.0012
```

■

It is important to keep in mind that the bootstrap resampling algorithm draws *random samples* from the observed data. In the example above, the random number generator in R was initially set with `set.seed(123)`. If the seed is set to a different integer, say 345, then the $B = 1,000$ bootstrap samples will be different and you will get different numerical results for the bootstrap bias and estimated standard error. How different these results will be depends on number of bootstrap

samples B and the statistic $\hat{\theta}$ being bootstrapped. Generally, the smaller is B the more variable the bootstraps results will be. It is good practice to repeat the bootstrap analysis with several different random number seeds to see how variable the results are. If the results vary considerably for different random number seeds then it is advised to increase the number of bootstrap simulations B . It is common to use $B = 1,000$ for routine calculations, and $B = 10,000$ for final results.

8.7.0.1 R package boot

The R package **boot** implements a variety of bootstrapping techniques including the basic non-parametric bootstrap described above. The **boot** package was written to accompany the textbook *Bootstrap Methods and Their Application* by (Davison and Hinkley 1997). The two main functions in **boot** are `boot()` and `boot.ci()`, respectively. The `boot()` function implements the bootstrap for a statistic computed from a user-supplied function. The `boot.ci()` function computes bootstrap confidence intervals given the output from the `boot()` function.

The arguments to `boot()` are:

```
library(boot)
args(boot)
#> function (data, statistic, R, sim = "ordinary", stype = c("i",
#>      "f", "w"), strata = rep(1, n), L = NULL, m = 0, weights = NULL,
#>      ran.gen = function(d, p) d, mle = NULL, simple = FALSE, ...,
#>      parallel = c("no", "multicore", "snow"), ncpus = getOption("boot.ncpus",
#>      1L), cl = NULL)
#> NULL
```

where `data` is a data object (typically a vector or matrix but can be a time series object too), `statistic` is a user-specified function to compute the statistic of interest, and `R` is the number of bootstrap replications. The remaining arguments are not important for the basic non-parametric bootstrap. The function assigned to the argument `statistic` has to be written in a specific form, which is illustrated in the next example. The `boot()` function returns an object of class `boot` for which there are print and plot methods.

The arguments to `boot.ci()` are

```
args(boot.ci)
#> function (boot.out, conf = 0.95, type = "all", index = 1L:min(2L,
#>      length(boot.out$t0)), var.t0 = NULL, var.t = NULL, t0 = NULL,
#>      t = NULL, L = NULL, h = function(t) t, hdot = function(t) rep(1,
#>      length(t)), hinv = function(t) t, ...)
#> NULL
```


where `boot.out` is an object of class `boot`, `conf` specifies the confidence level, and `type` is a subset from `c("norm", "basic", "stud", "perc", "bca")` indicating the type of confidence interval to compute. The choices `norm`, `perc`, and `bca` compute the normal confidence interval Equation 8.19, the percentile confidence interval Equation 8.20, and the BCA confidence interval, respectively. The remaining arguments are not important for the computation of these bootstrap confidence intervals.

Example 8.16 (Nonparametric bootstrap of GWN model using the R package **boot**). To use the `boot()` function to implement the bootstrap for $\hat{\mu}$, a function must be specified to compute $\hat{\mu}$ for each bootstrap sample. The function must have two arguments: `x` and `idx`. Here, `x` represents the original data and `idx` represents the random integer index (created internally by `boot()`) to subset `x` for each bootstrap sample. For example, a function to be passed to `boot()` for $\hat{\mu}$ is

```
mean.boot = function(x, idx) {
  # arguments:
  # x      data to be resampled
  # idx    vector of scrambled indices created by boot() function
  # value:
  # ans    mean value computed using resampled data
  ans = mean(x[idx])
  ans
}
```

To implement the nonparametric bootstrap for $\hat{\mu}$ with 999 samples use

```
set.seed(123)
muhat.boot = boot(msftRetS, statistic = mean.boot, R=999)
class(muhat.boot)
#> [1] "boot"
names(muhat.boot)
#> [1] "t0"      "t"      "R"      "data"    "seed"    "statistic"
#> [7] "sim"     "call"   "stype"  "strata"  "weights"
```

The returned object `muhat.boot` is of class `boot`. The component `t0` is the sample estimate $\hat{\mu}$, and the component `t` is a 999×1 matrix containing the bootstrap estimates $\{\hat{\mu}_1^*, \dots, \hat{\mu}_{999}^*\}$. The print method shows the sample estimate, the bootstrap bias and the bootstrap standard error:

```
muhat.boot
#>
#> ORDINARY NONPARAMETRIC BOOTSTRAP
#>
```

```
#>
#> Call:
#> boot(data = msftRetS, statistic = mean.boot, R = 999)
#>
#>
#> Bootstrap Statistics :
#>      original    bias      std. error
#> t1*  0.00915 0.000482      0.00758
```

These statistics can be computed directly from the components of `muhat.boot`

```
c(muhat.boot$t0, mean(muhat.boot$t) - muhat.boot$t0, sd(muhat.boot$t))
#> [1] 0.009153 0.000482 0.007584
```

A histogram and normal QQ-plot of the bootstrap values, shown in Figure 8.3, can be created using the `plot` method

```
plot(muhat.boot)
```

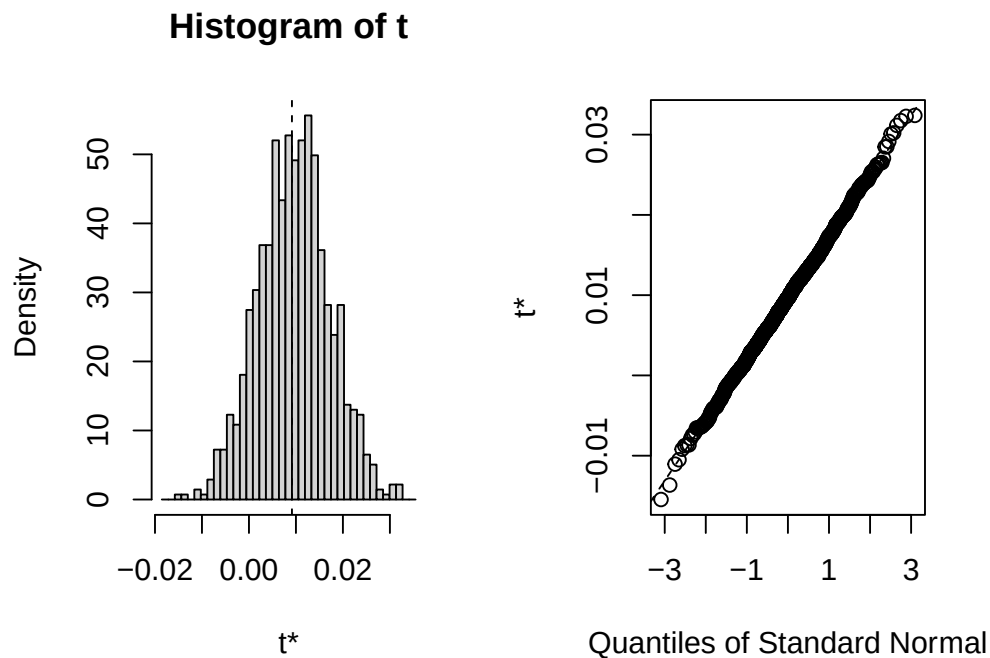


Figure 8.3: `plot` method for objects of class `boot`

Normal, percentile, and bias-adjusted (bca) percentile 95% confidence intervals can be computed

using

```
boot.ci(muhat.boot, conf=0.95, type=c("norm", "perc", "bca"))
#> BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
#> Based on 999 bootstrap replicates
#>
#> CALL :
#> boot.ci(boot.out = muhat.boot, conf = 0.95, type = c("norm",
#> "perc", "bca"))
#>
#> Intervals :
#> Level      Normal          Percentile          BCa
#> 95%  (-0.0062,  0.0235 )  (-0.0057,  0.0245 )  (-0.0062,  0.0239 )
#> Calculations and Intervals on Original Scale
```

Because the bootstrap distribution looks normal, the normal, percentile, and bca confidence intervals are very similar.

The GWN model estimate $\hat{\sigma}$ can be “bootstrapped” in a similar fashion. First, I write a function to compute $\hat{\sigma}$ for each bootstrap sample:

```
sd.boot = function(x, idx) {
  ans = sd(x[idx])
  ans
}
```

Then I call `boot()` with `statistic=sd.boot`

```
set.seed(123)
sigmahat.boot = boot(msftRetS, statistic = sd.boot, R=999)
sigmahat.boot
#>
#> ORDINARY NONPARAMETRIC BOOTSTRAP
#>
#>
#> Call:
#> boot(data = msftRetS, statistic = sd.boot, R = 999)
#>
#>
#> Bootstrap Statistics :
#>   original      bias    std. error
```

```
#> t1*      0.101 -0.000783      0.00764
```

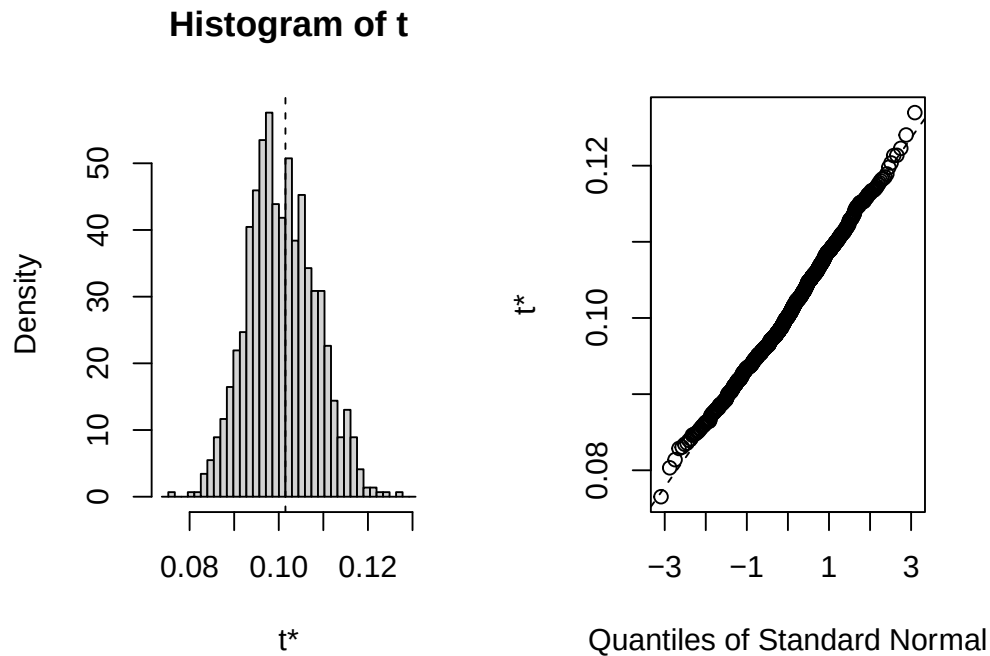


Figure 8.4: Bootstrap distribution for $\hat{\sigma}$

The bootstrap distribution, shown in Figure Figure 8.4, looks a bit non-normal. The 95% confidence intervals are computed using

```
boot.ci(sigmahat.boot, conf=0.95, type=c("norm", "perc", "bca"))
#> BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
#> Based on 999 bootstrap replicates
#>
#> CALL :
#> boot.ci(boot.out = sigmahat.boot, conf = 0.95, type = c("norm",
#>      "perc", "bca"))
#>
#> Intervals :
#> Level      Normal          Percentile          BCa
#> 95%   ( 0.0873, 0.1173 )   ( 0.0864, 0.1162 )   ( 0.0892, 0.1200 )
#> Calculations and Intervals on Original Scale
#> Some BCa intervals may be unstable
```

You see that all of these intervals are quite similar. ■

Example 8.17 (Nonparametric bootstrap for example functions). The real power of the bootstrap procedure comes when you apply it to plug-in estimates of functions like Equation 8.1-Equation 8.3 for which there are no easy analytic formulas for estimating bias and standard errors. In this situation, the bootstrap procedure easily computes numerical estimates of the bias, standard error, and 95% confidence interval. All you need is an R function for computing the plug-in function estimate in a form suitable for `boot()`. The R functions for the example functions are:

```
f1.boot = function(x, idx, alpha=0.05) {
  q = mean(x[idx]) + sd(x[idx])*qnorm(alpha)
  q
}

f2.boot = function(x, idx, alpha=0.05, w0=100000) {
  q = mean(x[idx]) + sd(x[idx])*qnorm(alpha)
  VaR = -w0*q
  VaR
}

f3.boot = function(x, idx, alpha=0.05, w0=100000) {
  q = mean(x[idx]) + sd(x[idx])*qnorm(alpha)
  VaR = -(exp(q) - 1)*w0
  VaR
}
```

The functions f_1 , f_2 , and f_3 have the additional argument `alpha` which specifies the tail probability for the quantile, and the functions f_2 , and f_3 have the additional argument `w0` for the initial wealth invested.

To compute the nonparametric bootstrap for f_1 with $\alpha = 0.05$ use:

```
set.seed(123)
f1.bs = boot(msftRetS, statistic = f1.boot, R=999)
f1.bs
#>
#> ORDINARY NONPARAMETRIC BOOTSTRAP
#>
#>
#> Call:
#> boot(data = msftRetS, statistic = f1.boot, R = 999)
#>
```

```
#>
#> Bootstrap Statistics :
#>      original    bias    std. error
#> t1*      -0.158 0.00177      0.0124
```

Here, the bootstrap estimate of bias is small and the bootstrap estimated standard error is similar to the delta method and jackknife standard errors.

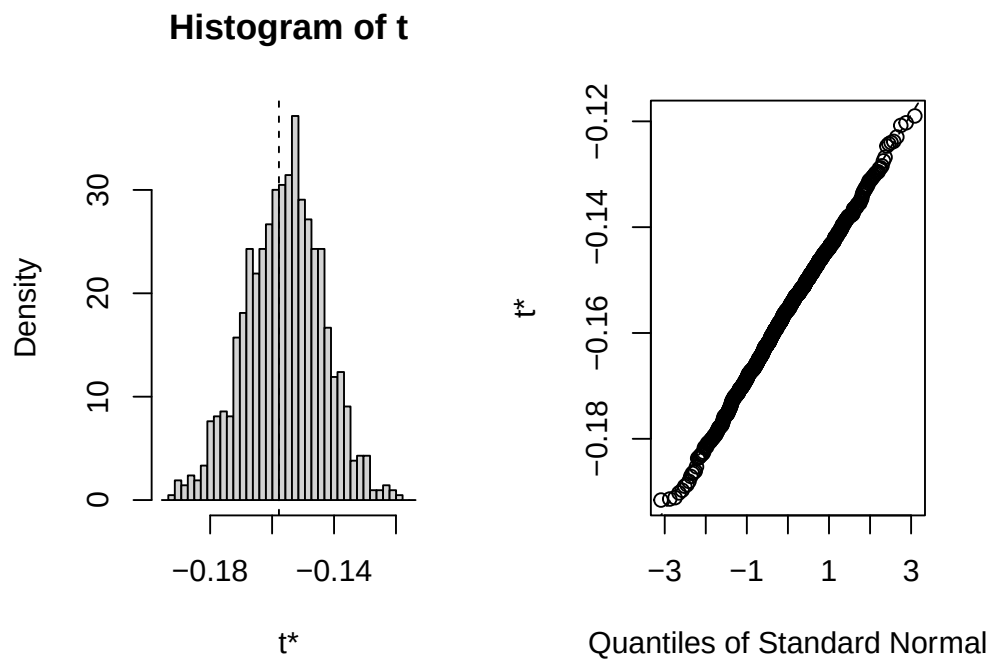


Figure 8.5: Bootstrap distribution for $\hat{f}_1$

The bootstrap distribution looks normal, and the three methods for computing 95% confidence intervals are essentially the same:

```
boot.ci(f1.bs, type=c("norm", "perc", "bca"))
#> BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
#> Based on 999 bootstrap replicates
#>
#> CALL :
#> boot.ci(boot.out = f1.bs, type = c("norm", "perc", "bca"))
#>
#> Intervals :
#> Level      Normal      Percentile      BCa
```

```
#> 95%   (-0.184, -0.135 )   (-0.181, -0.132 )   (-0.189, -0.138 )
#> Calculations and Intervals on Original Scale
#> Some BCa intervals may be unstable
```

To compute the nonparametric bootstrap for f_2 with $\alpha = 0.05$ and $W_0 = \$100,000$ use:

```
set.seed(123)
f2.bs = boot(msftRetS, statistic = f2.boot, R=999)
f2.bs
#>
#> ORDINARY NONPARAMETRIC BOOTSTRAP
#>
#>
#> Call:
#> boot(data = msftRetS, statistic = f2.boot, R = 999)
#>
#>
#> Bootstrap Statistics :
#>      original    bias      std. error
#> t1*      15780     -177         1244
```

The bootstrap estimate of bias is small and the bootstrap estimated standard error is similar to the delta method and jackknife standard errors.

The bootstrap distribution looks normal, and the three bootstrap 95% confidence intervals are all very similar:

```
boot.ci(f2.bs, type=c("norm", "perc", "bca"))
#> BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
#> Based on 999 bootstrap replicates
#>
#> CALL :
#> boot.ci(boot.out = f2.bs, type = c("norm", "perc", "bca"))
#>
#> Intervals :
#> Level      Normal      Percentile      BCa
#> 95%   (13518, 18395 )   (13161, 18089 )   (13801, 18940 )
#> Calculations and Intervals on Original Scale
#> Some BCa intervals may be unstable
```

To compute the nonparametric bootstrap for f_3 with $\alpha = 0.05$ and $W_0 = \$100,000$ use:

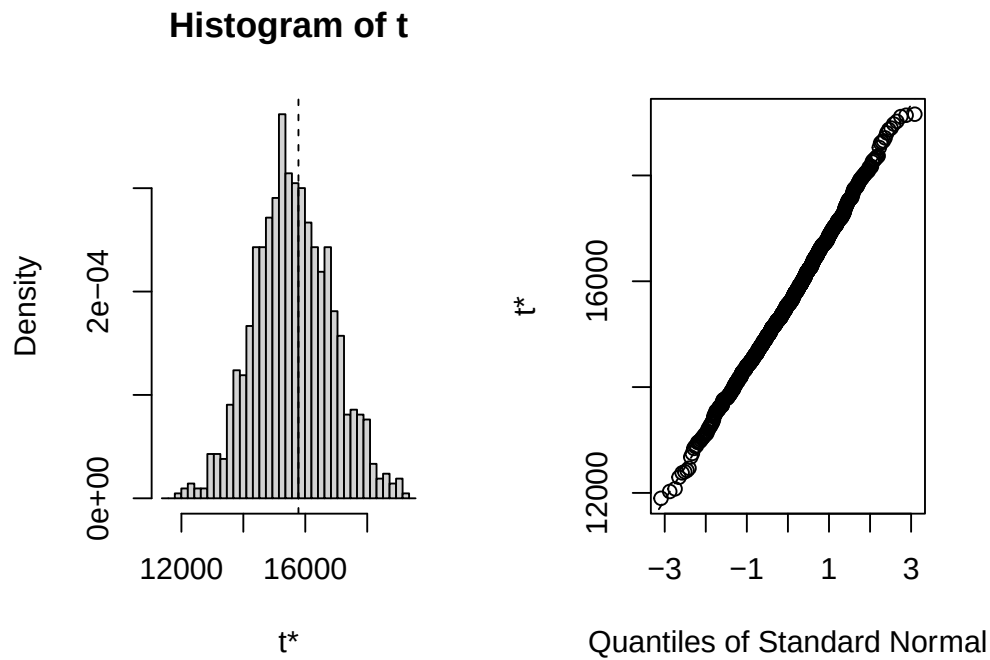


Figure 8.6: Bootstrap distribution for $\hat{f}_2$

```
set.seed(123)
f3.bs = boot(msftRetC, statistic = f3.boot, R=999)
f3.bs
#>
#> ORDINARY NONPARAMETRIC BOOTSTRAP
#>
#>
#> Call:
#> boot(data = msftRetC, statistic = f3.boot, R = 999)
#>
#>
#> Bootstrap Statistics :
#>      original    bias    std. error
#> t1*      14846     -182       1223
```

The bias is small and the bootstrap estimated standard error is larger than the delta method standard error and is closer to the jackknife standard error.

The bootstrap distribution looks a little asymmetric and the BCA confidence interval is a bit different from the normal and percentile intervals:

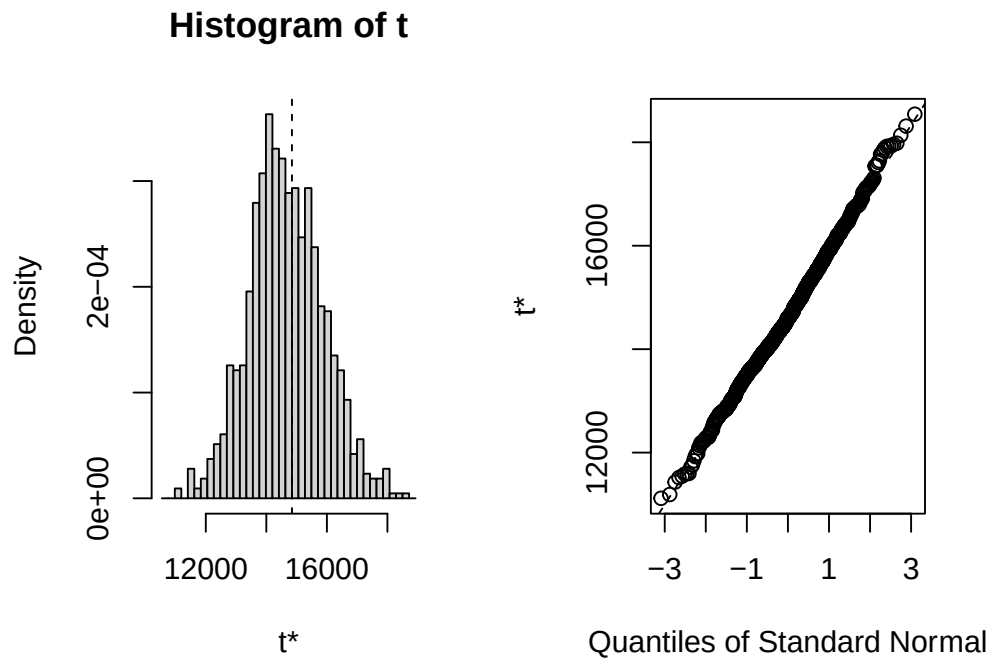


Figure 8.7: Bootstrap distribution for $\hat{f}_3$

```
boot.ci(f3.bs, type=c("norm", "perc", "bca"))
#> BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
#> Based on 999 bootstrap replicates
#>
#> CALL :
#> boot.ci(boot.out = f3.bs, type = c("norm", "perc", "bca"))
#>
#> Intervals :
#> Level      Normal          Percentile          BCa
#> 95%  (12631, 17426 )  (12295, 17155 )  (12994, 18061 )
#> Calculations and Intervals on Original Scale
#> Some BCa intervals may be unstable
```

■

8.7.1 Pros and cons of the nonparametric bootstrap

The nonparametric bootstrap is extremely useful and powerful statistical technique. The main advantages (pros) are:

- General procedure to estimate bias and standard errors, and to compute confidence intervals, that does not rely on asymptotic distributions.
- Can be used for scalar and vector estimates.
- Under general conditions, bootstrap distributions of estimators converge to the asymptotic distribution of the estimators.
- In certain situations, bootstrap results can be more accurate than analytic results computed from asymptotic distributions.

There are some things to watch out for when using the bootstrap (cons):

- It be computational intensive but this is really not much of an issue with today's fast computers.
- With small B , bootstrap results can vary substantially across simulations with different random number seeds.
- There are situations where the bootstrap does not work. A leading case is when the bootstrap is applied to a function that can become unbounded (e.g. a ratio of means when the denominator mean is close to zero). In these cases, a trimmed version of the bootstrap is recommended. See Hansen (2011) for a discussion of this issue.
- You have to be careful applying the bootstrap to time series data. If there is serial correlation in the data then the nonparametric bootstrap may give poor results. There are versions of the bootstrap (e.g. the block bootstrap) that work with serial correlated data. For example, the *tseries* function `tsbootstrap()` is designed to work with serially correlated time series data.

8.8 Using Monte Carlo Simulation to Evaluate Delta Method, Jackknife and Bootstrap Standard Errors

- You have studied three ways of computing standard errors for functions of GWN model parameters
- Give similar results for the example functions
- What are the preferred methods in practice?
- Which methods are most accurate?
- Use MC Simulation to evaluate the three methods in stylized model in which the GWN model is true. MC can give the best numerical standard error with which you can compare to the delta method, jackknife and bootstrap standard errors.

You can evaluate the statistical properties of $f_j(\hat{\theta})$, for $j = 1, \dots, 3$, by Monte Carlo simulation in the same way that you evaluated the statistical properties of $\hat{\mu}$ in Chapter Chapter 7. You use the simulation model Equation 7.41 and $N = 1000$ simulated samples of size $T = 100$ to compute the estimates $\{(f_j(\hat{\theta}))^1, \dots, (f_j(\hat{\theta}))^{1000}\}$. The true values for $f_j(\theta)$ are:

$$\begin{aligned} f_1(\theta) &= 0.05 + (0.10) \times (-1.645) = -0.114, \\ f_2(\theta) &= -\$100,000 \times f_1(\theta) = \$11,449, \\ f_3(\theta) &= \$100,000 \times (\exp(f_1(\theta)) - 1) = \$10,818. \end{aligned}$$

The R code for the simulation loop is:

```
mu = 0.05
sigma = 0.10
W0 = 100000
f1 = mu + sigma*qnorm(0.05)
f2 = -W0*f1
f3 = -W0*(exp(f1) - 1)
n.obs = 172
n.sim = 1000
set.seed(111)
sim.f = matrix(0,n.sim,3)
dm.f = matrix(0, n.sim, 3)
jack.f = matrix(0, n.sim, 3)
boot.f = matrix(0, n.sim, 3)
boot.bias = matrix(0, n.sim, 3)
colnames(sim.f) = colnames(dm.f) = colnames(jack.f) = c("f1", "f2","f3")
colnames(boot.f) = colnames(boot.bias) = c("f1", "f2","f3")

for (sim in 1:n.sim) {
  sim.ret = rnorm(n.obs,mean=mu,sd=sigma)
  muhat = mean(sim.ret)
  sigmahat = sd(sim.ret)
  sim.f[sim, "f1"] = muhat + sigmahat*qnorm(0.05)
  sim.f[sim, "f2"] = -W0*sim.f[sim, "f1"]
  sim.f[sim, "f3"] = -W0*(exp(sim.f[sim, "f1"]) - 1)
  # delta method se
  dm.f[sim, "f1"] = (sigmahat/sqrt(n.obs))*sqrt(1 + 0.5*qnorm(0.05)^2)
  dm.f[sim, "f2"] = W0*dm.f[sim, "f1"]
  dm.f[sim, "f3"] = W0*exp(sim.f[sim, "f1"])*dm.f[sim, "f1"]
}
```

```

# jackknife se
f1.jack = jackknife(1:n.obs, theta=theta.f1, as.matrix(sim.ret))
jack.f[sim, "f1"] = f1.jack$jack.se
f2.jack = jackknife(1:n.obs, theta=theta.f2, as.matrix(sim.ret), W0)
jack.f[sim, "f2"] = f2.jack$jack.se
f3.jack = jackknife(1:n.obs, theta=theta.f3, as.matrix(sim.ret), W0)
jack.f[sim, "f3"] = f3.jack$jack.se
# bootstrap se
f1.bs = boot(sim.ret, statistic = f1.boot, R=999)
boot.f[sim, "f1"] = sd(f1.bs$t)
f2.bs = boot(sim.ret, statistic = f2.boot, R=999)
boot.f[sim, "f2"] = sd(f2.bs$t)
f3.bs = boot(sim.ret, statistic = f3.boot, R=999)
boot.f[sim, "f3"] = sd(f3.bs$t)
# bootstrap bias
boot.bias[sim, "f1"] = summary(f1.bs)$bootBias
boot.bias[sim, "f2"] = summary(f2.bs)$bootBias
boot.bias[sim, "f3"] = summary(f3.bs)$bootBias
}

```

The histograms of $f_j(\hat{\theta})$ values are displayed in Figure Figure 8.8. The dotted white lines indicate the true values, $f_j(\theta)$.

The histograms for the $f_j(\hat{\theta})$ values are bell-shaped for $j = 1, 2, 3$ and the histogram for $j = 4$ is slightly right skewed. All of the histograms are centered very close to the respective true values. Recall, the standard deviation of the Monte Carlo estimates (spread of the histograms about true value) approximates the true standard error of the plug-in estimates.

The Monte Carlo estimates of bias are:

```

mc.bias = colMeans(sim.f) - c(f1, f2, f3)
round(mc.bias, digits=3)
#>    f1    f2    f3
#> 0.00 10.79 3.52

```

which confirm that f_j are essentially unbiased estimators. It is often useful to also look at the bias relative to the true value to eliminate units:

```

mc.bias/c(f1, f2, f3)
#>      f1      f2      f3
#> 0.000943 0.000943 0.000326

```

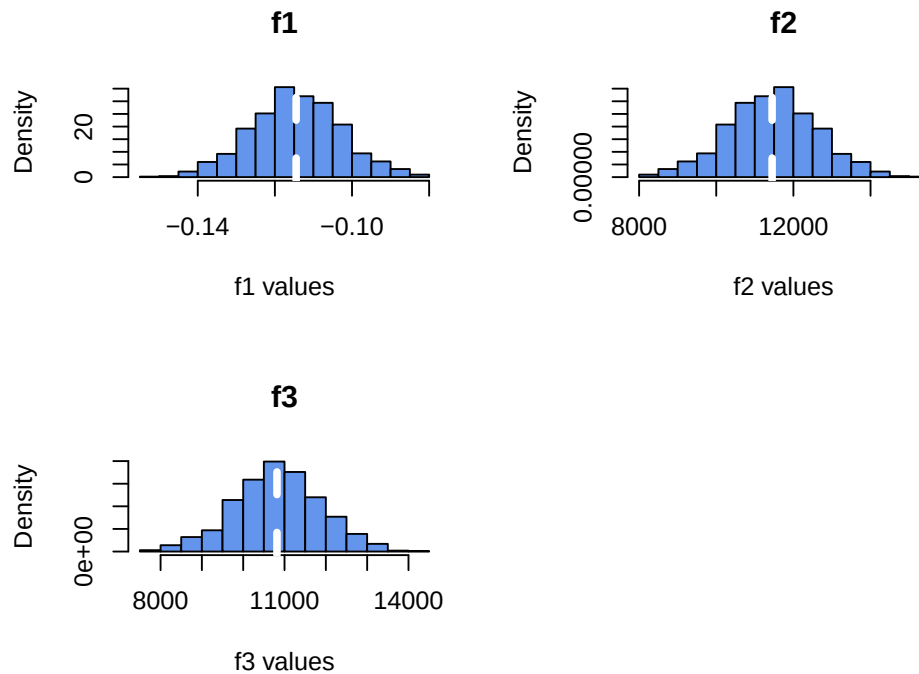


Figure 8.8: Histograms of $\hat{f}_j$ (for $j = 1, \dots, 4$) computed from $N = 1000$ Monte Carlo samples from the GWN model.

In relative terms, the largest bias is xx%, which occurs for the estimate of xxx.

The average bootstrap estimate of bias is:

```
boot.bias.vals = colMeans(boot.bias)
round(boot.bias.vals, digits=3)
#>      f1      f2      f3
#>  0.001 -69.435 -68.570
```

The bootstrap bias values are very close to the Monte Carlo estimates of bias which indicates that the bootstrap estimate of bias is very accurate in this context.

The sample standard deviations of the Monte Carlo estimates approximate the true standard errors of the estimated functions:

```
mc.se = apply(sim.f, 2, sd)
round(mc.se, digits=3)
#>      f1      f2      f3
#>  0.012 1170.205 1043.851
```

The sample means of the delta method, jackknife, and bootstrap standard errors give the average

performance of these estimated standard errors. It is informative to compare these average standard errors to the Monte Carlo standard errors:

```
dm.se = colMeans(dm.f)
jack.se = colMeans(jack.f)
boot.se = colMeans(boot.f)
ans = rbind(dm.se/mc.se, jack.se/mc.se, boot.se/mc.se)
rownames(ans) = c("Delta Method", "Jackknife", "Bootstrap")
ans
#>           f1      f2      f3
#> Delta Method 0.998 0.998 0.998
#> Jackknife    0.997 0.997 0.997
#> Bootstrap    0.987 0.988 0.987
```

In the table above, a value of 1 means the estimated standard error is the same as the Monte Carlo standard error (on average), and a value greater than 1 means that the estimated standard error is larger than the Monte Carlo standard error (on average). Here, you see that all of the estimated standard errors are very close to the Monte Carlo standard errors, and overall the bootstrap estimated standard errors are the closest. The worst performance is for $f_x(\hat{\theta})$ where the estimated standard errors are about xx% to xx% larger than the Monte Carlo standard errors.

It is also useful to look at the standard deviations of the estimated standard errors over the Monte Carlo simulations. This tells you how variable the estimated standard errors are across different simulations:

```
dm.sd = apply(dm.f,2,sd)
jack.sd = apply(jack.f,2,sd)
boot.sd = apply(boot.f,2,sd)
ans = rbind(dm.sd, jack.sd, boot.sd)
rownames(ans) = c("Delta Method", "Jackknife", "Bootstrap")
ans
#>           f1      f2      f3
#> Delta Method 0.000637 63.7 48.1
#> Jackknife    0.001148 114.8 98.1
#> Bootstrap    0.001145 115.1 96.8
```

Here, you see that, across the simulations, the delta method is the least variable and the jackknife and the bootstrap have similar variability that is about twice as large as the delta method variability. The variability of the bootstrap can be reduced by increasing the number of bootstrap simulations (here it is 1000).

Overall, the Monte Carlo experiment confirms that the delta method, jackknife and bootstrap all

produce accurate standard error estimates for the example plug-in functions.

8.9 Further Reading: Delta Method and Resampling

Chernick and LaBudde (2011) give a comprehensive overview of the bootstrap with applications in R.

8.10 Exercises

Exercise 8.1 (Delta method).

- TBD

Exercise 8.2 (Gradients for example functions).

- TBD

Exercise 8.3 (Standard error of Sharpe ratio).

- Look at determinants of se of Sharpe ratio. See discussion from Lo's paper

Exercise 8.4 (Correlation between estimated Sharpe ratios).

- Look at the formula for the covariance between two Sharpe ratios

Exercise 8.5 (Bootstrap percentile confidence interval). Illustrate the transformation preserving property of percentile interval using the example of computing the confidence interval for σ based on the percentile interval for σ^2 . Show that asymptotic confidence interval is not transformation preserving.

Exercise 8.6 (Bootstrap variability). Repeat the bootstrap simulations from Example 8.16 using 5 different random number seeds: 124, 125, 126, 127 and 128. Make a table to compare the bootstrap estimated standard errors for the example functions across the 5 bootstrap simulations. Briefly comment on your results.

Exercise 8.7 (Failure of the bootstrap).

- Run the bootstrap for calculating a standard error for μ_i/μ_j .

Exercise 8.8 (Bootstrap accuracy). Repeat the Monte Carlo simulation in Section 8.8 using 1,000 simulated samples and 10,000 bootstrap simulations for the bootstrap calculations. That is, set `R=10000` in the call to `boot()` inside the Monte Carlo simulation loop.

1. How does the increase in simulations impact the bootstrap bias estimates?
2. How does the increase in simulations impact the bootstrap standard error estimates?

9 Hypothesis Testing in the GWN Model

Updated: September 7, 2026

Copyright © Eric Zivot 2026

The core of statistical analysis involves estimation and inference. The previous chapters considered estimation of the GWN model parameters, and the computation of standard error estimates and confidence intervals. In this chapter I discuss issues of statistical inference within the GWN model. Statistical inference is about asking questions about a model and using statistical procedures to obtain answers to these questions with a given level of confidence. For example, you might find that our estimate of the monthly mean of an asset return is positive but due to large estimation error we might ask the question: is the true monthly mean return positive? As another example, you might find that a normal qq-plot of returns shows some evidence of non-normality and you ask the question: are returns normally distributed? As a third example, you might find that rolling estimates of the mean and volatility show some evidence of non-constant behavior and you might ask the question: are returns covariance stationary? Statistical hypothesis testing gives us a rigorous framework for answering such questions with a given level of confidence.

The R packages used in this chapter are **introcompfinr**, **PerformanceAnalytics**, **tseries**, and **zoo**. Make sure this packages are installed and loaded before replicating the examples.

1. Hypothesis Testing in the GWN Model

a. Specification tests

i. Normal distribution

A. Tests for skewness and kurtosis

B. JB test

C. What to do if returns are not normally distributed?

ii. No autocorrelation A. individual and joint tests

iii. Constant parameters (covariance stationarity) A. Formal tests: testing same mean, volatility in sub-samples B. Informal diagnostics - standard error bands on rolling estimators

b. Using Monte Carlo Simulation to Understand Hypothesis tests

i. Size

ii. Power

- c. Using the bootstrap for hypothesis tests
 - a. exploit duality between hypothesis tests and confidence intervals
- 2. Appendix: Distributions for test statistics
 - a. Student's t distribution
 - b. Chi-square distribution

9.1 Review of General Principles

Classical hypothesis testing is discussed in almost all introductory textbooks on statistics, and the reader is assumed be familiar with the basic concepts.¹ In this section I provide a brief and concise review of the general principles of classical hypothesis testing. In the following sections I apply these principles to answer questions about the GWN model parameters and assumptions.

9.1.1 Steps for hypothesis testing

The main steps for conducting a classical hypothesis test are as follows:

1. Specify the hypotheses to be tested:

$$H_0 : \text{null hypothesis vs. } H_1 : \text{alternative hypothesis.}$$

The *null hypothesis* is the maintained hypothesis (what is assumed to be true) and is the hypothesis to be tested against the data. The *alternative hypothesis* specifies what is assumed to be true if the null hypothesis is false. This can be specific or vague depending on the context.

2. Specify the significance level of the test:

$$\alpha = \text{significance level} = \Pr(\text{Reject } H_0 | H_0 \text{ is true}).$$

The *significance level* of a test specifies the probability of making a certain type of decision error: rejecting the null hypothesis when the null hypothesis is, in fact, true. In practice, the significance level α is chosen to be a small number like 0.01 or 0.05.

3. Construct a *test statistic*, S , from the observed data whose probability distribution is known when H_0 is true.
4. Use the test statistic S to evaluate the data evidence regarding the validity of H_0 . Typically, if S is a big number then there is strong data evidence against H_0 and H_0 should be rejected; otherwise, there is not strong data evidence against H_0 and H_0 should not be rejected.

¹In this book, I do not consider evaluating hypotheses from a Bayesian perspective.

- Decide to reject H_0 at the specified significance level α if the value of the test statistic S falls in the *rejection region* of the test. Usually, the rejection region for S is determined by a critical value cv_α such that:

$$\begin{aligned} S &> cv_\alpha \Rightarrow \text{reject } H_0, \\ S &\leq cv_\alpha \Rightarrow \text{do not reject } H_0. \end{aligned}$$

Smaller values of α typically make cv_α larger and require more data evidence (i.e., larger S) to reject H_0 .

- Alternatively, decide to reject H_0 at the specified significance level α if the p-value of the test statistic S is less than α . The *p-value* of the statistic S is defined as the significance level at which the test is just rejected.

9.1.2 Hypothesis tests and decisions

Hypothesis testing involves making a decision: reject H_0 or do not reject H_0 . Notice that if the data evidence strongly favors H_0 then you say “I do not reject H_0 ” instead of “accept H_0 ”. You can rarely know the truth with absolute certainty with a finite amount of data so it is more appropriate to say “do not reject H_0 ” than to say “accept H_0 ”.²

Table 9.1: Decision table for hypothesis testing

		Reality	
		H_0 is true	H_0 is false
Decision	Reject H_0	Type I error	No error
	Do not reject H_0	No error	Type II error

Table 9.1 shows the 2×2 decision table associated with hypothesis testing. In reality there are two states of the world: either H_0 is true or it is not.³ There are two decisions to make: reject H_0 or don’t reject H_0 . If the decision corresponds with reality then the correct decision is made and there is no decision error. This occurs in the off-diagonal elements of the table. If the decision and reality disagree then there is a decision error. These errors are in the diagonal elements of the table. Type I error results when the decision to reject H_0 occurs when, in fact, H_0 is true. Type II error happens when the decision not to reject H_0 occurs when H_0 is false. To put these types

²As an extreme example, consider testing H_0 : “all swans are white”. To logically accept this hypothesis you would have to show that every swan that ever existed is white. In contrast, if you have a data set that only contains white swans then saying you cannot reject the null hypothesis that all swans are white is consistent with your data set.

³It is important to emphasize that there is a “true state of the world” in this context. That is, the null hypothesis H_0 is either true or it is not. As a result, $\Pr\{H_0 \text{ is true}\} = 1$ or 0 depending on whether H_0 is true or not.

of errors in context, consider a jury in a capital murder trial in the US.⁴ In the US, a defendant is considered innocent until proven guilty. Here, the null hypothesis to be tested is:

$$H_0 : \text{defendant is innocent.}$$

The alternative is:

$$H_1 : \text{defendant is guilty.}$$

There is a true state of the world here: the defendant is either innocent or guilty. The jury must decide what the state of the world is based on evidence presented to them. The decision table for the jury is shown in Table 9.2. Type I error occurs when the jury convicts an innocent defendant and puts that person on “death row”. Clearly, this is a terrible mistake. To avoid this type of mistake the jury should have a very small (close to zero) significance level for evaluating evidence so that Type I error very rarely occurs. This is why typical jury instructions are to convict only if evidence of guilt is presented beyond reasonable doubt. Type II error happens when the jury does not convict (acquits) a guilty defendant and sets that person free. This is also a terrible mistake, but perhaps it is not as terrible as convicting the innocent person. Notice that there is a conflict between Type I error and Type II error. In order to completely eliminate Type I error, you can never reject H_0 . That is, to avoid ever sending innocent people to “death row” you must never convict anyone. But if you never convict anyone then you never convict guilty people either and this increases the occurrences of Type II errors.

9.1.3 Significance level and power

The *significance level* of a test is the probability of Type I error:

$$\alpha = \Pr(\text{Type I error}) = \Pr(\text{Reject } H_0 | H_0 \text{ is true}).$$

The *power* of a test is one minus the probability of Type II error:

$$\pi = 1 - \Pr(\text{Type II error}) = \Pr(\text{Reject } H_0 | H_0 \text{ is false}).$$

In classical hypothesis tests, the goal is to construct a test that has a small significance level (α close to zero) that you can control and has high power (π close to one). That is, a good test is one that has a small and controllable probability of rejecting the null when it is true and, at the same time, has a very high probability of rejecting the null when it is false. In general, an *optimal test* is one that has the highest possible power for a given significance level. In the jury example, an optimal test would be the one in which the jury has the highest probability of convicting a guilty defendant while at the same time has a very low probability of convicting an innocent defendant. A difficult problem indeed!

⁴A capital murder trial is one in which the defendant is eligible for the death penalty.

Table 9.2: Decision table for jury on capital murder trial

		Reality	
		Defendant is innocent	Defendant is guilty
Decision	Convict	Type I error	No error
	Acquit	No error	Type II error

9.2 Hypothesis Testing in the GWN Model

In this section, I discuss hypothesis testing in the context of the GWN model for asset returns:

$$\begin{aligned}
 R_{it} &= \mu_i + \varepsilon_{it}, \quad i = 1 \dots, N; \quad t = 1, \dots, T \\
 \{\varepsilon_{it}\}_{t=1}^T &\sim \text{GWN}(0, \sigma_i^2), \\
 \text{cov}(\varepsilon_{it}, \varepsilon_{js}) &= \begin{cases} \sigma_{ij} & t = s \\ 0 & t \neq s \end{cases}.
 \end{aligned} \tag{9.1}$$

In Equation 9.1, R_{it} is the monthly return (continuously compounded or simple) on asset i in month t . Define $\mathbf{R}_t = (R_{1t}, \dots, R_{Nt})'$. The GWN model in matrix form is:

$$\begin{aligned}
 \mathbf{R}_t &= \boldsymbol{\mu} + \boldsymbol{\varepsilon}_t, \\
 \boldsymbol{\varepsilon}_t &\sim \text{iid } N(\mathbf{0}, \boldsymbol{\Sigma}).
 \end{aligned} \tag{9.2}$$

I consider two types of hypothesis tests: (1) tests for model coefficients or functions of model coefficients; and (2) tests for model assumptions.⁵ These tests can be specified for the GWN model for a single asset using Equation 9.1, or they can be specified for the GWN model for all assets using Equation 9.2.

9.2.1 Coefficient tests

Here, I assume that the GWM is the true model for our observed return data and I are interested in asking questions about the parameters of the model. The most common tests for model coefficients and functions of model coefficients for a single asset are tests for a specific value. Some typical hypotheses to be tested are:

⁵Tests for model assumptions are sometimes called model specification tests.

$$\begin{aligned}
H_0 : \mu_i &= \mu_i^0 \text{ vs. } H_1 : \mu_i \neq \mu_i^0, \\
H_0 : \sigma_i &= \sigma_i^0 \text{ vs. } H_1 : \sigma_i \neq \sigma_i^0, \\
H_0 : \rho_{ij} &= \rho_{ij}^0 \text{ vs. } H_1 : \rho_{ij} \neq \rho_{ij}^0, \\
H_0 : \text{SR}_i &= \text{SR}_i^0 \text{ vs. } H_1 : \text{SR}_i \neq \text{SR}_i^0,
\end{aligned}$$

where $\text{SR}_i = \frac{\mu_i - r_f}{\sigma_i}$ is the Sharpe ratio of asset i and r_f is the risk-free rate of interest. A hypothesis about the Sharpe ratio is a hypothesis about the ratio of two GWN model parameters. In the above hypotheses, the alternative hypotheses are *two-sided*. That is, under H_1 the model parameters are either larger or smaller than what is specified under H_0 .

Also common are tests for specific sign. For example:

$$\begin{aligned}
H_0 : \mu_i &= 0 \text{ vs. } H_1 : \mu_i > 0 \text{ or } H_1 : \mu_i < 0, \\
H_0 : \rho_{ij} &= 0 \text{ vs. } H_1 : \rho_{ij} > 0 \text{ or } H_1 : \rho_{ij} < 0, \\
H_0 : \text{SR}_i &= 0 \text{ vs. } H_1 : \text{SR}_i > 0 \text{ or } H_1 : \text{SR}_i < 0,
\end{aligned}$$

Here, the alternative hypotheses are *one-sided*. That is, under H_1 the model parameter is either greater than zero or less than zero.

Sometimes you are interested in testing hypotheses about the relationship between model coefficients for two or more assets. For example:

$$\begin{aligned}
H_0 : \mu_1 &= \mu_2 \text{ vs. } H_1 : \mu_1 \neq \mu_2, \\
H_0 : \text{SR}_1 &= \text{SR}_2 \text{ vs. } H_1 : \text{SR}_1 \neq \text{SR}_2.
\end{aligned}$$

It is often convenient to re-express these hypotheses in terms of differences of coefficients under the null and alternative hypotheses:

$$\begin{aligned}
H_0 : \mu_1 - \mu_2 &= 0 \text{ vs. } H_1 : \mu_1 - \mu_2 \neq 0, \\
H_0 : \text{SR}_1 - \text{SR}_2 &= 0 \text{ vs. } H_1 : \text{SR}_1 - \text{SR}_2 \neq 0.
\end{aligned}$$

9.2.2 Model specification tests

In the GWN model Equation 9.1, asset returns are assumed to be normally distributed. The hypotheses to be tested for this assumption are:

$$H_0 : R_{it} \sim N(\mu_i, \sigma_i^2) \text{ vs. } H_1 : R_{it} \sim \text{not normal.}$$

Notice that the distribution of R_t under H_1 is not specified (e.g., Student's t with 5 degrees of freedom).

In addition, it is assumed that asset returns are uncorrelated over time. One can test the hypothesis that each autocorrelation (up to some maximum lag J) is zero:

$$\begin{aligned} H_0 : \rho_j &= \text{cor}(r_{it}, r_{i,t-j}) = 0, j = 1, \dots, J \text{ vs.} \\ H_1 : \rho_j &= \text{cor}(r_{it}, r_{i,t-j}) \neq 0, \end{aligned}$$

or one can test the joint hypothesis that all autocorrelation coefficients (up to some maximum lag J) are zero:

$$\begin{aligned} H_0 : \rho_1 &= \rho_2 = \dots = \rho_J = 0 \text{ vs.} \\ H_1 : \rho_j &\neq 0 \text{ for some } j \leq J \end{aligned}$$

Lastly, it is assumed that asset returns are covariance stationary. This assumption implies that all model parameters are constant over time. The hypotheses of interest are then:

$$\begin{aligned} H_0 : \mu_i, \sigma_i \text{ and } \rho_{ij} &\text{ are constant over entire sample, vs.} \\ H_1 : \mu_i, \sigma_i \text{ or } \rho_{ij} &\text{ changes in some sub-sample.} \end{aligned}$$

The following sections describe test statistics for the above listed hypotheses.

9.2.3 Data for hypothesis testing examples

The examples in this chapter use the monthly continuously compounded returns on Microsoft, Starbucks and the S&P 500 index over the period January 1998 through May 2012 created using:

```
gwnDailyPrices = merge(msftDailyPrices, sbuxDailyPrices, sp500DailyPrices)
gwnMonthlyPrices = to.monthly(gwnDailyPrices, OHLC=FALSE)
smpl = "1998-01::2012-05"
gwnMonthlyPrices = gwnMonthlyPrices[smpl]
gwnMonthlyRetC = na.omit(Return.calculate(gwnMonthlyPrices, method="log"))
```

The monthly continuously compounded returns, with a common y-axis, are illustrated in Figure 9.1 created using:

```
plot(gwnMonthlyRetC, multi.panel=TRUE, main="",
     col="blue", lwd=2, yaxis.same=TRUE,
     major.ticks="years", grid.ticks.on="years")
```

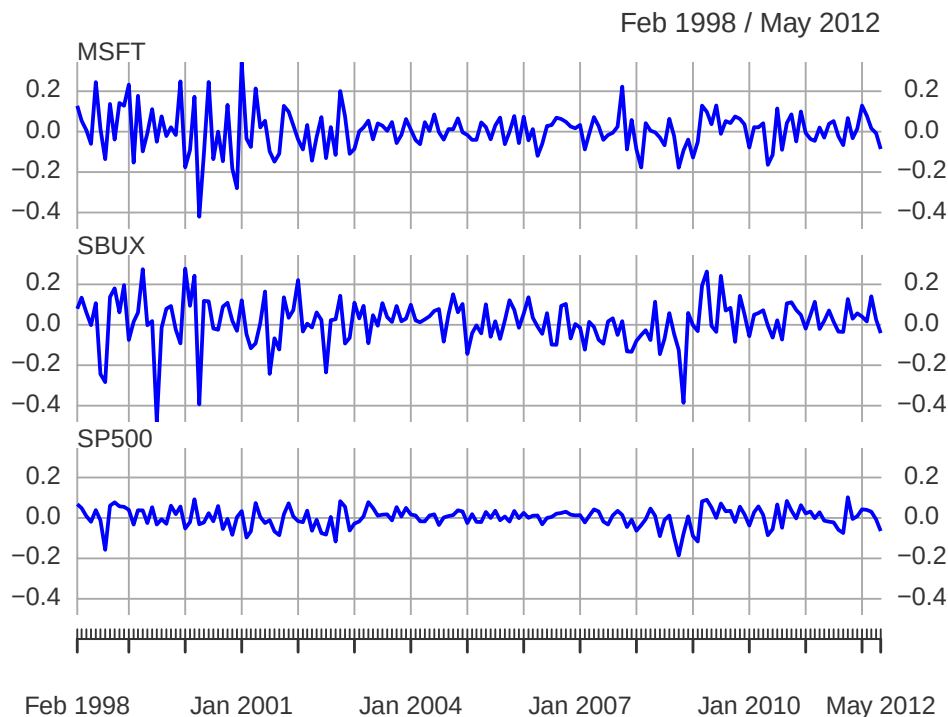


Figure 9.1: Monthly cc returns on Microsoft, Starbucks and the S&P 500 Index.

9.3 Tests for Individual Parameters: t-tests and z-scores

In this section, I present test statistics for testing hypotheses that certain model coefficients equal specific values. In some cases, I can derive *exact tests*. For an exact test, the pdf of the test statistic assuming the null hypothesis is true is known exactly for a finite sample size T . More generally, however, I rely on *asymptotic tests*. For an asymptotic test, the pdf of the test statistic assuming the null hypothesis is true is not known exactly for a finite sample size T , but can be approximated by a known pdf. The approximation is justified by the Central Limit Theorem (CLT) and the approximation becomes exact as the sample size becomes infinitely large.

9.3.1 Exact tests under normality of data

In the GWN model Equation 9.1, consider testing the hypothesis that the mean return, μ_i , is equal to a specific value μ_i^0 :

$$H_0 : \mu_i = \mu_i^0. \quad (9.3)$$

For example, an investment analyst may have provided an expected return forecast of μ_i^0 and you would like to see if past data was consistent with such a forecast.

The alternative hypothesis can be either two-sided or one-sided. The two-sided alternative is:

$$H_1 : \mu_i \neq \mu_i^0. \quad (9.4)$$

Two-sided alternatives are used when you don't care about the sign of $\mu_i - \mu_i^0$ under the alternative. With one-sided alternatives, you care about the sign of $\mu_i - \mu_i^0$ under the alternative:

$$H_1 : \mu_i > \mu_i^0 \text{ or } H_1 : \mu_i < \mu_i^0. \quad (9.5)$$

How do you come up with a test statistic S for testing Equation 9.3 against Equation 9.4 or Equation 9.5? You generally use two criteria: (1) you know the pdf of S assuming Equation 9.3 is true; and (2) the value of S should be big if the alternative hypotheses Equation 9.4 or Equation 9.5 are true.

9.3.1.1 Two-sided test

To simplify matters, let's assume that the value of σ_i in Equation 9.1 is known and does not need to be estimated. This assumption is unrealistic and will be relaxed later. Consider testing Equation 9.3 against the two-sided alternative Equation 9.4 using a 5% significance level. Two-sided

alternatives are typically more commonly used than one-sided alternatives. Let $\{R_t\}_{t=1}^T$ denote a random sample from the GWN model. You estimate μ_i using the sample mean:

$$\hat{\mu}_i = \frac{1}{T} \sum_{t=1}^T R_{it}. \quad (9.6)$$

Under the null hypothesis Equation 9.3, it is assumed that $\mu_i = \mu_i^0$. Hence, if the null hypothesis is true then $\hat{\mu}_i$ computed from data should be close to μ_i^0 . If $\hat{\mu}_i$ is far from μ_i^0 (either above or below μ_i^0) then this evidence casts doubt on the validity of the null hypothesis. Now, there is estimation error in Equation 9.6 which is measured by:

$$\text{se}(\hat{\mu}_i) = \frac{\sigma_i}{\sqrt{T}}. \quad (9.7)$$

Because you assume σ_i is known, you also know $\text{se}(\hat{\mu}_i)$. Because of estimation error, you don't expect $\hat{\mu}_i$ to equal μ_i^0 even if the null hypothesis Equation 9.3 is true. How far from μ_i^0 can $\hat{\mu}_i$ be if the null hypothesis is true? To answer this question, recall from Chapter Chapter 7 that the exact (finite sample) pdf of $\hat{\mu}_i$ is the normal distribution:

$$\hat{\mu}_i \sim N(\mu_i, \text{se}(\hat{\mu}_i)^2) = N\left(\mu_i, \frac{\sigma_i^2}{T}\right). \quad (9.8)$$

Under the null hypothesis Equation 9.3, the normal distribution for $\hat{\mu}_i$ is centered at μ_i^0 :

$$\hat{\mu}_i \sim N(\mu_i^0, \text{se}(\hat{\mu}_i)^2) = N\left(\mu_i^0, \frac{\sigma_i^2}{T}\right). \quad (9.9)$$

From properties of the normal distribution you have:

$$\Pr(\mu_i^0 - 1.96 \times \text{se}(\hat{\mu}_i) \leq \hat{\mu}_i \leq \mu_i^0 + 1.96 \times \text{se}(\hat{\mu}_i)) = 0.95. \quad (9.10)$$

Hence, if the null hypothesis Equation 9.3 is true you only expect to see $\hat{\mu}_i$ more than 1.96 values of $\text{se}(\hat{\mu}_i)$ away from μ_i^0 with probability 0.05.⁶ Therefore, it makes intuitive sense to base a test statistic for testing Equation 9.3 on a measure of distance between $\hat{\mu}_i$ and μ_i^0 relative to $\text{se}(\hat{\mu}_i)$. Such a statistic is the *z-score*:

$$z_{\mu=\mu^0} = \frac{\hat{\mu}_i - \mu_i^0}{\text{se}(\hat{\mu}_i)} = \frac{\hat{\mu}_i - \mu_i^0}{\sigma/\sqrt{T}}. \quad (9.11)$$

⁶This means that in an infinite number of hypothetical samples from the GWN model in which H_0 is true only 5 percent of the samples produce an estimate $\hat{\mu}_i$ that is more than 1.96 values of $\text{se}(\hat{\mu}_i)$ away from μ_i^0 .

Assuming the null hypothesis Equation 9.3 is true, from Equation 9.9 it follows that:

$$z_{\mu=\mu^0} \sim N(0, 1).$$

The intuition for using the z-score Equation 9.11 to test Equation 9.3 is straightforward. If $z_{\mu=\mu^0} \approx 0$ then $\hat{\mu}_i \approx \mu_i^0$ and Equation 9.3 should not be rejected. In contrast, if $z_{\mu=\mu^0} > 1.96$ or $z_{\mu=\mu^0} < -1.96$ then $\hat{\mu}_i$ is more than 1.96 values of $\text{se}(\hat{\mu}_i)$ away from μ_i^0 . From Equation 9.10, this is very unlikely (less than 5% probability) if Equation 9.3 is true. In this case, there is strong data evidence against Equation 9.3 and you should reject it. Notice that the condition $z_{\mu=\mu^0} > 1.96$ or $z_{\mu=\mu^0} < -1.96$ can be simplified as the condition $|z_{\mu=\mu^0}| > 1.96$. When $|z_{\mu=\mu^0}|$ is big, larger than 1.96, you have data evidence against Equation 9.3. Using the value 1.96 to determine the rejection region for the test ensures that the significance level (probability of Type I error) of the test is exactly 5%:

$$\begin{aligned} \Pr(\text{Reject } H_0 | H_0 \text{ is true}) &= \Pr(S > 1.96 | \mu_i = \mu_i^0) \\ &= \Pr(|z_{\mu=\mu^0}| > 1.96 | \mu_i = \mu_i^0) = .05 \end{aligned}$$

Hence, our formal test statistic for testing Equation 9.3 against Equation 9.4 is

$$S = |z_{\mu=\mu^0}|. \quad (9.12)$$

The 5% critical value is $cv_{.05} = 1.96$, and you reject Equation 9.3 at the 5% significance level if $S > 1.96$.

In summary, the steps for using $S = |z_{\mu=\mu^0}|$ to test the Equation 9.3 against Equation 9.4 are:

1. Set the significance level $\alpha = \Pr(\text{Reject } H_0 | H_0 \text{ is true})$ and determine the critical value cv_α such that $\Pr(S > cv_\alpha) = \alpha$. Now,

$$\begin{aligned} \Pr(S > cv_\alpha) &= \Pr(|z_{\mu=\mu^0}| > cv_\alpha) \\ &= \Pr(z_{\mu=\mu^0} > cv_\alpha) + \Pr(z_{\mu=\mu^0} < -cv_\alpha) = \alpha \end{aligned}$$

which implies that $cv_\alpha = -q_{\alpha/2}^Z = q_{1-\alpha/2}^Z$, where $q_{\alpha/2}^Z$ denotes the $\frac{\alpha}{2}$ -quantile of $Z \sim N(0, 1)$. For example, if $\alpha = .05$ then $cv_{.05} = -q_{.025}^Z = 1.96$.

2. Reject Equation 9.3 at the $\alpha \times 100\%$ significance level if $S > cv_\alpha$. For example, if $\alpha = .05$ then reject Equation 9.3 at the 5% level if $S > 1.96$.

3. Equivalently, reject Equation 9.3 at the $\alpha \times 100\%$ significance level if the p-value for S is less than α . Here, the *p-value* is defined as the significance level at which the test is just rejected. Let $Z \sim N(0, 1)$. The p-value is computed as:

$$\text{p-value} = \Pr(|Z| > S) = \Pr(Z > S) + \Pr(Z < -S) = 2 \times \Pr(Z > S) = 2 \times (1 - \Pr(Z < S)). \quad (9.13)$$

Example 9.1 (Testing hypothesis with the z-score using simulated data: two-sided tests). Assume returns follow the GWN model

$$\begin{aligned} R_t &= 0.05 + \epsilon_t, \quad t = 1, \dots, 60, \\ \epsilon_t &\sim GWN(0, 0.10). \end{aligned}$$

and you are interested in testing the following hypotheses

$$H_0 : \mu = 0.05 \text{ vs. } H_1 : \mu \neq .05$$

$$H_0 : \mu = 0.06 \text{ vs. } H_1 : \mu \neq .06$$

$$H_0 : \mu = 0.10 \text{ vs. } H_1 : \mu \neq .10$$

using the test statistic Equation 9.12 with a 5% significance level. The 5% critical value is $cv_{.05} = 1.96$. One hypothetical sample from the hypothesized model is simulated using:

```
set.seed(123)
n.obs = 60
mu = 0.05
sigma = 0.10
ret.sim = mu + rnorm(n.obs, sd = sigma)
```

The estimate of μ and the value of $se(\hat{\mu})$ are:

```
muhat = mean(ret.sim)
se.muhat = sigma/sqrt(n.obs)
c(muhat, se.muhat)
#> [1] 0.0566 0.0129
```

The test statistic Equation 9.12 for testing $H_0 : \mu = 0.05$ computed from this sample is:

```

z.score.05 = (muhat - 0.05)/se.muhat
S = abs(z.score.05)
S
#> [1] 0.508

```

The z-score tells us that the estimated mean, $\hat{\mu} = 0.0566$, is 0.58 values of $se(\hat{\mu})$ away from the hypothesized value $\mu^0 = 0.05$. This evidence does not contradict $H_0 : \mu = 0.05$. Since $S = 0.508 < 1.96$, you do not reject $H_0 : \mu = 0.05$ at the 5% significance level. The p-value of the test using Equation 9.13 is:

```

p.value = 2*(1 - pnorm(S))
p.value
#> [1] 0.611

```

Here, the p-value of 0.611 is less than the significance level $\alpha = 0.05$ so you do not reject the null at the 5% significance level. The p-value tells us that you would reject $H_0 : \mu = 0.05$ at the 61.1% significance level.

The z-score information for testing $H_0 : \mu = 0.06$ is:

```

z.score.06 = (muhat - 0.06)/se.muhat
S = abs(z.score.06)
S
#> [1] 0.266

```

Here, the z-score indicates that $\hat{\mu} = 0.0566$ is just 0.266 values of $se(\hat{\mu})$ away from the hypothesized value $\mu^0 = 0.06$. This evidence also does not contradict $H_0 : \mu = 0.06$. Since $S = 0.226 < 1.96$, you do not reject $H_0 : \mu = 0.06$ at the 5% significance level. The p-value of the test is:

```

p.value = 2*(1 - pnorm(S))
p.value
#> [1] 0.79

```

The large p-value shows that there is insufficient data to reject $H_0 : \mu = 0.06$.

Last, the z-score information for testing $H_0 : \mu = 0.10$ is:

```

z.score.10 = (muhat - 0.10)/se.muhat
S = abs(z.score.10)
S
#> [1] 3.36

```

Now, the z-score indicates that $\hat{\mu} = 0.0566$ is 3.36 values of $\text{se}(\hat{\mu})$ away from the hypothesized value $\mu^0 = 0.10$. This evidence is not in support of $H_0 : \mu = 0.10$. Since $S = 3.36 > 1.96$, you reject $H_0 : \mu = 0.06$ at the 5% significance level. The p-value of the test is:

```
p.value = 2*(1 - pnorm(S))
p.value
#> [1] 0.000766
```

Here, the p-value is much less 5% supporting the rejection of $H_0 : \mu = 0.06$.

■

9.3.1.2 One-sided test

Here, I consider testing the null Equation 9.3 against the one-sided alternative Equation 9.5. For expositional purposes, consider $H_1 : \mu_i > \mu_i^0$. This alternative can be equivalently represented as $H_1 : \mu_i - \mu_i^0 > 0$. That is, under the alternative hypothesis the sign of the difference $\mu_i - \mu_i^0$ is positive. This is why a test against a one-sided alternative is sometimes called a *test for sign*. As in the previous sub-section, assume that σ_i is known and let the significance level be 5%.

The natural test statistic is the z-score Equation 9.11. The intuition is straightforward. If $z_{\mu=\mu^0} \approx 0$ then $\hat{\mu}_i \approx \mu_i^0$ and the null should not be rejected. However, if the one-sided alternative is true then you would expect to see $\hat{\mu}_i > \mu_i^0$ and $z_{\mu=\mu^0} > 0$. How big $z_{\mu=\mu^0}$ needs to be for us to reject the null depends on the significance level. Since under the null $z_{\mu=\mu^0} \sim N(0, 1) = Z$, with a 5% significance level

$$\Pr(\text{Reject } H_0 | H_0 \text{ is true}) = \Pr(z_{\mu=\mu^0} > 1.645) = \Pr(Z > 1.645) = 0.05.$$

Hence, our 5% one-sided critical value is $cv_{.05} = q_{.95}^Z = 1.645$, and you reject the null at the 5% significance level if $z_{\mu=\mu^0} > 1.645$.

In general, the steps for using $z_{\mu=\mu^0}$ to test Equation 9.3 against the one-sided alternative $H_1 : \mu_i > \mu_i^0$ are:

1. Set the significance level $\alpha = \Pr(\text{Reject } H_0 | H_0 \text{ is true})$ and determine the critical value cv_α such that $\Pr(z_{\mu=\mu^0} > cv_\alpha) = \alpha$. Since under the null $z_{\mu=\mu^0} \sim N(0, 1) = Z$, $cv_\alpha = q_{1-\alpha}^Z$.
2. Reject the null hypothesis Equation 9.3 at the $\alpha \times 100\%$ significance level if $z_{\mu=\mu^0} > cv_\alpha$.
3. Equivalently, reject Equation 9.3 at the $\alpha \times 100\%$ significance level if the one-sided p-value is less than α , where

$$\text{p-value} = \Pr(Z > z_{\mu=\mu^0}) = 1 - \Pr(Z \leq z_{\mu=\mu^0})$$

Example 9.2 (Testing hypothesis with the z-score using simulated data: one-sided tests). Here, I use the simulated GWN return data from the previous example to test Equation 9.3 against the one-sided alternative $H_1 : \mu_i > \mu_i^0$ using a 5% significance level.

The z-scores and one-sided p-values for testing $H_0 : \mu = 0.05$, $H_0 : \mu = 0.06$, and $H_0 : \mu = 0.10$, computed from the simulated sample are:

```
z.score.05 = (muhat - 0.05)/se.muhat
p.value.05 = 1 - pnorm(z.score.05)
z.score.06 = (muhat - 0.06)/se.muhat
p.value.06 = 1 - pnorm(z.score.06)
z.score.10 = (muhat - 0.10)/se.muhat
p.value.10 = 1 - pnorm(z.score.10)
ans = rbind(c(z.score.05, p.value.05),
            c(z.score.06, p.value.06),
            c(z.score.10, p.value.10))
colnames(ans) = c("Z-score", "P-value")
rownames(ans) = c("H0:mu=0.05", "H0:mu=0.06", "H0:mu=0.10")
ans
```

#>	Z-score	P-value
#> H0:mu=0.05	0.508	0.306
#> H0:mu=0.06	-0.266	0.605
#> H0:mu=0.10	-3.365	1.000

Here, you do not reject any of the null hypotheses in favor of the one-sided alternative at the 5% significance level. ■

9.3.2 Exact tests with σ unknown

In practice, σ^2 is unknown and is estimated with $\hat{\sigma}^2$, and so the exact tests described above are not feasible. Fortunately, an exact test is still available. Instead of using the z-score Equation 9.11, we use the t-ratio (or t-score)

$$t_{\mu=\mu^0} = \frac{\hat{\mu}_i - \mu_i^0}{\widehat{\text{se}}(\hat{\mu}_i)} = \frac{\hat{\mu}_i - \mu_i^0}{\hat{\sigma}/\sqrt{T}}. \quad (9.14)$$

Assuming the null hypothesis Equation 9.3 is true, Proposition 7.7 tells us that Equation 9.14 is distributed Student's t with $T - 1$ degrees of freedom and is denoted by the random variable t_{T-1} . The steps for using the z-score and the t-ratio are the same for evaluating Equation 9.3, but now we use critical values and p-values from t_{T-1} . Our test statistic for the two-sided alternative

$H_0 : \mu_i \neq \mu_i^0$ is:

$$S = |t_{\mu=\mu^0}|, \quad (9.15)$$

and our test statistic for the one-sided alternative $H_0 : \mu_i > \mu_i^0$ is

$$S = t_{\mu=\mu^0}, \quad (9.16)$$

Our critical values are determined from the quantiles of the Student's t with $T - 1$ degrees of freedom, $t_{T-1}(1 - \alpha/2)$. For example, if $\alpha = 0.05$ and $T - 1 = 60$ then the two-sided critical value is $cv_{.05} = t_{60}(0.975) = 2$ (which can be verified using the R function `qt()`). The two-sided p-value is computed as:

$$\begin{aligned} \text{p-value} &= \Pr(|t_{T-1}| > S) = \Pr(t_{T-1} > S) + \Pr(t_{T-1} < -S) \\ &= 2 \times \Pr(t_{T-1} > S) = 2 \times (1 - \Pr(t_{T-1} < S)). \end{aligned}$$

The one-sided critical value is $cv_{.05} = t_{60}(0.95) = 1.67$, and the one-sided p-value is computed as:

$$\text{p-value} = \Pr(t_{T-1} > S) = 1 - \Pr(t_{T-1} \leq S).$$

As the sample size gets larger $\hat{\sigma}_i$ gets closer to σ_i and the Student's t distribution gets closer to the normal distribution. Decisions using the t-ratio and the z-score are almost the same for $T \geq 60$.

Example 9.3 (Testing hypothesis with the t-ratio using simulated data). I repeat the hypothesis testing from the previous example this time use the t-ratio Equation 9.14 instead of the z-score. Using $T - 1 = 59$ the 5% critical value for the two-sided test is

```
qt(0.975, df=59)
#> [1] 2
```

To compute the t-ratios we first estimate $se(\hat{\mu})$:

```
sigmahat = sd(ret.sim)
sehat.muhat = sigmahat/sqrt(n.obs)
c(sehat.muhat, se.muhat)
#> [1] 0.0118 0.0129
```

Here, $\widehat{se}(\hat{\mu}) = 0.0118 < 0.0129 = se(\hat{\mu})$ and so the t-ratios will be slightly larger than the z-scores. The t-ratios and test statistics for the three hypotheses are:


```

t.ratio.05 = (muhat - 0.05)/sehat.muhat
t.ratio.06 = (muhat - 0.06)/sehat.muhat
t.ratio.10 = (muhat - 0.10)/sehat.muhat
S.05 = abs(t.ratio.05)
S.06 = abs(t.ratio.06)
S.10 = abs(t.ratio.10)
ans = c(S.05, S.06, S.10)
names(ans) = c("S.05", "S.06", "S.07")
ans
#>  S.05  S.06  S.07
#> 0.558 0.293 3.696

```

As expected the test statistics computed from the t-ratios are slightly larger than the tests computed from the z-scores. The first two statistics are less than 2, and the third statistic is bigger than 2 and so we reach the same decisions as before. The p-values of the three tests are:

```

S.vals = c(S.05, S.06, S.10)
p.values = 2*(1 - pt(S.vals, df=59))
p.values
#> [1] 0.578747 0.770895 0.000482

```

Here, the p-values computed from t_{59} are very similar to those computed from $Z \sim N(0, 1)$. ■

You can derive an exact test for testing hypothesis about the value of μ_i based on the z-score or the t-ratio, but you cannot derive exact tests for the values of σ_i or for the values of ρ_{ij} based on z-scores. Exact tests for these parameters are much more complicated. While t-ratios for the values of σ_i or for the values of ρ_{ij} do not have exact t-distributions in finite samples, as the sample size gets large the distributions of the t-ratios get closer and closer to the normal distribution due to the CLT. This motivates the use of so-called asymptotic z-scores discussed in the next sub-section.

9.3.3 Z-scores under asymptotic normality of estimators

Let $\hat{\theta}$ denote an estimator for θ . Here, I allow θ to be a GWN model parameter or a function of GWN model parameters. For example, in the GWN model θ could be μ_i , σ_i , ρ_{ij} , q_α^R , VaR_α or SR_i . As you have seen, the CLT (and the delta method if needed) justifies the asymptotic normal distribution:

$$\hat{\theta} \sim N(\theta, \widehat{\text{se}}(\hat{\theta})^2), \quad (9.17)$$

for large enough sample size T , where $\widehat{\text{se}}(\hat{\theta})$ is the estimated standard error for $\hat{\theta}$. Consider testing:

$$H_0 : \theta = \theta_0 \text{ vs. } H_1 : \theta \neq \theta_0. \quad (9.18)$$

Under H_0 , the asymptotic normality result Equation 9.17 implies that the z-score for testing Equation 9.18 has a standard normal distribution for large enough sample size T :

$$z_{\theta=\theta_0} = \frac{\hat{\theta} - \theta_0}{\widehat{\text{se}}(\hat{\theta})} \sim N(0, 1) = Z. \quad (9.19)$$

The intuition for using the z-score Equation 9.19 is straightforward. If $z_{\theta=\theta_0} \approx 0$ then $\hat{\theta} \approx \theta_0$, and $H_0 : \theta = \theta_0$ should not be rejected. On the other hand, if $|z_{\theta=\theta_0}| > 2$, say, then $\hat{\theta}$ is more than 2 values of $\widehat{\text{se}}(\hat{\theta})$ away from θ_0 . This is very unlikely if $\theta = \theta_0$ because $\hat{\theta} \sim N(\theta_0, \widehat{\text{se}}(\hat{\theta})^2)$, so $H_0 : \theta \neq \theta_0$ should be rejected. Therefore, the test statistic for testing Equation 9.19 is $S = |z_{\theta=\theta_0}|$.

The steps for using the z-score Equation 9.19 with its critical value to test the hypotheses Equation 9.18 are:

1. Set the significance level α of the test and determine the two-sided critical value $cv_{\alpha/2}$. Using Equation 9.17, the critical value, $cv_{\alpha/2}$, is determined using:

$$\begin{aligned} \Pr(|Z| \geq cv_{\alpha/2}) &= \alpha \\ \Rightarrow cv_{\alpha/2} &= -q_{\alpha/2}^Z = q_{1-\alpha/2}^Z, \end{aligned}$$

where $q_{\alpha/2}^Z$ denotes the $\frac{\alpha}{2}$ -quantile of $N(0, 1)$. A commonly used significance level is $\alpha = 0.05$ and the corresponding critical value is $cv_{.025} = -q_{.025}^Z = q_{.975}^Z = 1.96 \approx 2$.

2. Reject Equation 9.18 at the $100 \times \alpha\%$ significance level if:

$$S = |z_{\theta=\theta_0}| = \left| \frac{\hat{\theta} - \theta_0}{\widehat{\text{se}}(\hat{\theta})} \right| > cv_{\alpha/2}.$$

If the significance level is $\alpha = 0.05$, then reject Equation 9.18 at the 5% level using the rule-of-thumb:

$$S = |z_{\theta=\theta_0}| > 2.$$

The steps for using the z-score Equation 9.19 with its p-value to test the hypotheses Equation 9.18 are:

1. Determine the two-sided p-value. The p-value of the two-sided test is the significance level at which the test is just rejected. From Equation 9.17, the two-sided p-value is defined by

$$\text{p-value} = \Pr(|Z| > |z_{\theta=\theta_0}|) = 2 \times (1 - \Pr(Z \leq |z_{\theta=\theta_0}|)). \quad (9.20)$$

2. Reject Equation 9.18 at the $100 \times \alpha\%$ significance level if the p-value Equation 9.20 is less than α .

Example 9.4 (Using the z-score to test hypothesis about () in the GWN model). Consider using the z-score Equation 9.19 to test $H_0 : \mu_i = 0$ vs. $H_1 : \mu_i \neq 0$ (i = Microsoft, Starbucks, S&P 500) using a 5% significance level. First, calculate the GWN model estimates for μ_i :

```
n.obs = nrow(gwnMonthlyRetC)
muhat.vals = apply(gwnMonthlyRetC, 2, mean)
muhat.vals
#>    MSFT    SBUX    SP500
#> 0.00413 0.01466 0.00169
```

Next, calculate the estimated standard errors:

```
sigmahat.vals = apply(gwnMonthlyRetC, 2, sd)
se.muhat = sigmahat.vals/sqrt(n.obs)
se.muhat
#>    MSFT    SBUX    SP500
#> 0.00764 0.00851 0.00370
```

Then calculate the z-scores and test statistics:

```
z.scores = muhat.vals/se.muhat
S.vals = abs(z.scores)
S.vals
#>    MSFT    SBUX    SP500
#> 0.540 1.722 0.457
```

Since the absolute value of all of the z-scores are less than two, you do not reject $H_0 : \mu_i = 0$ at the 5% level for all assets.

The p-values for all of the test statistics are computed using:

```
2*(1-pnorm(S.vals))
#>    MSFT    SBUX    SP500
#> 0.5893 0.0851 0.6480
```

Since all p-values are greater than $\alpha = 0.05$, you reject $H_0 : \mu_i = 0$ at the 5% level for all assets. The p-value for Starbucks is the smallest at 0.0851. Here, you can reject $H_0 : \mu_{SBUX} = 0$ at the 8.51% level. ■

Example 9.5 (Using the z-score to test hypothesis about ρ in the GWN model). Consider using the z-score Equation 9.19 to test the hypotheses:

$$H_0 : \rho_{ij} = 0.5 \text{ vs. } H_1 : \rho_{ij} \neq 0.5. \quad (9.21)$$

using a 5% significance level. Here, you use the result from the GWN model that for large enough T :

$$\hat{\rho}_{ij} \sim N(\rho_{ij}, \widehat{\text{se}}(\hat{\rho}_{ij})^2), \widehat{\text{se}}(\hat{\rho}_{ij}) = \frac{1 - \hat{\rho}_{ij}^2}{\sqrt{T}}.$$

Then the z-score for testing Equation 9.21 has the form:

$$z_{\rho_{ij}=0.5} = \frac{\hat{\rho}_{ij} - 0.5}{\widehat{\text{se}}(\hat{\rho}_{ij})} = \frac{\hat{\rho}_{ij} - 0.5}{(1 - \hat{\rho}_{ij}^2) / \sqrt{T}}. \quad (9.22)$$

To compute the z-scores, first, calculate the GWN model estimates for ρ_{ij} :

```
corhat.mat = cor(gwnMonthlyRetC)
rhohat.vals = corhat.mat[lower.tri(corhat.mat)]
names(rhohat.vals) = c("MSFT.SBUX", "MSFT.SP500", "SBUX.SP500")
rhohat.vals
#> MSFT.SBUX MSFT.SP500 SBUX.SP500
#>      0.341      0.617      0.457
```

Next, calculate estimated standard errors:

```
se.rhohat = (1 - rhohat.vals^2)/sqrt(n.obs)
se.rhohat
#> MSFT.SBUX MSFT.SP500 SBUX.SP500
#>      0.0674      0.0472      0.0603
```

Then calculate the z-scores Equation 9.22 and test statistics:

```
z.scores = (rhohat.vals - 0.5)/se.rhohat
S.vals = abs(z.scores)
S.vals
#> MSFT.SBUX MSFT.SP500 SBUX.SP500
#>      2.361      2.482      0.706
```

Here, the absolute value of the z-scores for $\rho_{MSFT,SBUX}$ and $\rho_{MSFT,SP500}$ are greater than 2 whereas the absolute value of the z-score for $\rho_{SBUX,SP500}$ is less than 2. Hence, for the pairs (MSFT, SBUX) and (MSFT, SP500) you cannot reject the null Equation 9.21 at the 5% level but

for the pair (SBUX, SP500) you can reject the null at the 5% level. The p-values for the test statistics are:

```
2*(1-pnorm(S.vals))
#> MSFT.SBUX MSFT.SP500 SBUX.SP500
#> 0.0182 0.0130 0.4804
```

Here, the p-values for the pairs (MSFT,SBUX) and (MSFT,SP500) are less than 0.05 and the p-value for the pair (SBUX, SP500) is much greater than 0.05. ■

Example 9.6 (Using the z-score to test hypothesis about asset Sharpe ratios in the GWN model). Consider using the z-score to test the hypotheses:

$$H_0 : \text{SR}_i = \frac{\mu_i - r_f}{\sigma_i} = 0 \text{ vs. } H_1 : \text{SR}_i > 0$$

using a 5% significance level. In Chapter 8 I used the delta method to show that, for large enough T :

$$\widehat{\text{SR}}_i \sim N \left(\text{SR}_i, \widehat{\text{se}}(\widehat{\text{SR}}_i)^2 \right), \quad \widehat{\text{se}}(\widehat{\text{SR}}_i) = \frac{1}{\sqrt{T}} \sqrt{1 + \frac{1}{2} \widehat{\text{SR}}_i^2}$$

Then, the z-score has the form

$$z_{\text{SR}=0} = \frac{\widehat{\text{SR}}_i - 0}{\widehat{\text{se}}(\widehat{\text{SR}}_i)} = \frac{\widehat{\text{SR}}_i}{\frac{1}{\sqrt{T}} \sqrt{1 + \frac{1}{2} \widehat{\text{SR}}_i^2}} = \frac{\sqrt{T} \times \widehat{\text{SR}}_i}{\sqrt{1 + \frac{1}{2} \widehat{\text{SR}}_i^2}}$$

■

9.3.4 Relationship between hypothesis tests and confidence intervals

Consider testing the hypotheses Equation 9.18 at the 5% significance level using the z-score Equation 9.19. The rule-of-thumb decision rule is to reject $H_0 : \theta = \theta_0$ if $|z_{\theta=\theta_0}| > 2$. This implies that:

$$\frac{\hat{\theta} - \theta_0}{\widehat{\text{se}}(\hat{\theta})} > 2 \text{ or } \frac{\hat{\theta} - \theta_0}{\widehat{\text{se}}(\hat{\theta})} < -2,$$

which further implies that:

$$\theta_0 < \hat{\theta} - 2 \times \widehat{\text{se}}(\hat{\theta}) \text{ or } \theta_0 > \hat{\theta} + 2 \times \widehat{\text{se}}(\hat{\theta}).$$

Recall the definition of an approximate 95% confidence interval for θ :

$$\hat{\theta} \pm 2 \times \widehat{\text{se}}(\hat{\theta}) = [\hat{\theta} - 2 \times \widehat{\text{se}}(\hat{\theta}), \hat{\theta} + 2 \times \widehat{\text{se}}(\hat{\theta})].$$

Notice that if $|z_{\theta=\theta_0}| > 2$ then θ_0 does not lie in the approximate 95% confidence interval for θ . Hence, you can reject $H_0 : \theta = \theta_0$ at the 5% significance level if θ_0 does not lie in the 95% confidence interval for θ . This result allows you to have a deeper understanding of the 95% confidence interval for θ : it contains all values of θ_0 for which you cannot reject $H_0 : \theta = \theta_0$ at the 5% significance level.

This duality between two-sided hypothesis tests and confidence intervals makes hypothesis testing for individual parameters particularly simple. As part of estimation, you calculate estimated standard errors and form approximate 95% confidence intervals. Then when you look at the approximate 95% confidence interval, it gives us all values of θ_0 for which you cannot reject $H_0 : \theta = \theta_0$ at the 5% significance level (approximately).

Example 9.7 (Hypothesis testing for μ_i , ρ_{ij} , and SR_i using 95% confidence intervals). For the example data, the approximate 95% confidence intervals for μ_i for Microsoft, Starbucks and the S&P 500 index are:

```
lower = muhat.vals - 2*se.muhat
upper = muhat.vals + 2*se.muhat
cbind(lower, upper)
#>           lower    upper
#> MSFT    -0.01116  0.01941
#> SBUX    -0.00237  0.03168
#> SP500   -0.00570  0.00908
```

Consider testing $H_0 : \mu_i = 0$ at the 5% level. Here you see that $\mu_i^0 = 0$ lies in all of the 95% confidence intervals and so you do not reject $H_0 : \mu_i = 0$ at the 5% level for any asset. Each interval gives the values of μ_i^0 for which you cannot reject $H_0 : \mu_i = \mu_i^0$ at the 5% level. For example, for Microsoft you cannot reject the hypothesis (at the 5% level) that μ_{MSFT} is as small as -0.011 or as large as 0.019.

Next, the approximate 95% confidence intervals for ρ_{ij} for the pairs $\rho_{MSFT,SBUX}$, $\rho_{MSFT,SP500}$, and $\rho_{SBUX,SP500}$ are:

```
lower = rhohat.vals - 2*se.rhohat
upper = rhohat.vals + 2*se.rhohat
cbind(lower, upper)
#>           lower upper
#> MSFT.SBUX   0.206  0.476
#> MSFT.SP500  0.523  0.712
```

```
#> SBUX.SP500 0.337 0.578
```

Consider testing $H_0 : \rho_{ij} = 0.5$ for all pairs of assets. Here, you see that $\rho_{ij}^0 = 0.5$ is not in the approximate 95% confidence intervals for $\rho_{MSFT,SBUX}$ and $\rho_{MSFT,SP500}$ but is in the approximate 95% confidence interval for $\rho_{SBUX,SP500}$. Hence, you can reject $H_0 : \rho_{ij} = 0.5$ at the 5% significance level for $\rho_{MSFT,SBUX}$ and $\rho_{MSFT,SP500}$ but not for $\rho_{SBUX,SP500}$.

- Add SR info here

■

9.4 Coefficient tests between model coefficients for two assets

In this section I consider hypothesis tests for coefficients in the bivariate GWN model for asset returns.

9.4.1 Test for equal means between two assets

Often it is of interest to test if the expected returns on two assets are the same:

$$H_0 : \mu_1 = \mu_2 \text{ vs. } H_1 : \mu_1 \neq \mu_2.$$

Notice that this hypothesis is not of the form that the two coefficients equal a specific value (e.g. $\mu_1 = \mu_2 = 0.05$). However, you can transform the hypothesis into one in which a coefficient equals a specific value by considering the differences in the expected returns. In particular, the relation $\mu_1 = \mu_2$ is equivalent to the relation $\delta = \mu_1 - \mu_2 = 0$. Hence, you can reformulate the hypothesis of equal expected returns to the hypothesis that the difference in expected returns is equal to zero:

$$H_0 : \delta = \mu_1 - \mu_2 = 0 \text{ vs. } H_1 : \delta = \mu_1 - \mu_2 \neq 0. \quad (9.23)$$

Now, you have test in the form of the coefficient tests you considered earlier.

The parameter $\delta = \mu_1 - \mu_2$ can be estimated with the plug-in estimator:

$$\hat{\delta} = \hat{\mu}_1 - \hat{\mu}_2.$$

To form a z-score, you require an estimate of $\text{se}(\hat{\delta})$. Now,

$$\text{var}(\hat{\delta}) = \text{var}(\hat{\mu}_1 - \hat{\mu}_2) = \text{var}(\hat{\mu}_1) + \text{var}(\hat{\mu}_2) - 2\text{cov}(\hat{\mu}_1, \hat{\mu}_2). \quad (9.24)$$

To evaluate Equation 9.24, you use the result from Chapter 7 that the joint distribution of the vector $(\hat{\mu}_1, \hat{\mu}_2)'$ is multivariate normal:

$$\begin{pmatrix} \hat{\mu}_1 \\ \hat{\mu}_2 \end{pmatrix} = N \left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \frac{1}{T} \begin{pmatrix} \hat{\sigma}_1^2 & \hat{\sigma}_{12} \\ \hat{\sigma}_{12} & \hat{\sigma}_2^2 \end{pmatrix} \right),$$

for large enough T . Then

$$\widehat{\text{var}}(\hat{\delta}) = \frac{1}{T}(\hat{\sigma}_1^2 + \hat{\sigma}_2^2 - 2\hat{\sigma}_{12}),$$

and

$$\widehat{\text{se}}(\hat{\delta}) = \sqrt{\frac{1}{T}(\hat{\sigma}_1^2 + \hat{\sigma}_2^2 - 2\hat{\sigma}_{12})}.$$

you can test Equation 9.23 using the z-score:

$$z_{\delta=0} = \frac{\hat{\delta}}{\widehat{\text{se}}(\hat{\delta})} = \frac{\hat{\mu}_1 - \hat{\mu}_2}{\sqrt{\frac{1}{T}(\hat{\sigma}_1^2 + \hat{\sigma}_2^2 - 2\hat{\sigma}_{12})}}, \quad (9.25)$$

with the test statistic $S = |z_{\delta=0}|$.

The estimate $\widehat{\text{var}}(\hat{\delta})$ can also be computed using the delta method. Define $\theta = (\mu_1, \mu_2)'$ and $f(\theta) = \delta = \mu_1 - \mu_2$. The gradient of $f(\theta)$ is:

$$g(\theta) = \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

and $\widehat{\text{var}}(\hat{\delta})$ can be computed as:

$$\begin{aligned} \widehat{\text{var}}(\hat{\delta}) &= g(\hat{\theta})' \widehat{\text{var}}(\hat{\theta}) g(\hat{\theta}) = (1, -1)' \begin{pmatrix} \frac{\hat{\sigma}_1^2}{T} & \frac{\hat{\sigma}_{12}}{T} \\ \frac{\hat{\sigma}_{12}}{T} & \frac{\hat{\sigma}_2^2}{T} \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix} \\ &= \frac{1}{T}(\hat{\sigma}_1^2 + \hat{\sigma}_2^2 - 2\hat{\sigma}_{12}). \end{aligned}$$

Example 9.8 (Testing that Microsoft and Starbucks have the same expected return.). Let's use the z-score Equation 9.25 to test the hypothesis that the expected returns on Microsoft and Starbucks are the same using a 5% significance level. The R code to compute the z-score and test statistic is:


```

delta.hat = muhat.vals["MSFT"] - muhat.vals["SBUX"]
sig2hat.vals = sigmahat.vals^2
covhat = cov(gwnMonthlyRetC[, c("MSFT", "SBUX")])[1,2]
sehat.deltahat = sqrt((sig2hat.vals["MSFT"]+sig2hat.vals["SBUX"]-2*covhat)/n.obs)
z.score.delta = delta.hat/sehat.deltahat
S= abs(z.score.delta)
ans = c(delta.hat, sehat.deltahat, S)
names(ans) = c("Estimate", "Std Error", "Test Statistic")
ans
#>      Estimate      Std Error Test Statistic
#>      -0.0105      0.0093      1.1323

```

Here, $\hat{\delta} = -0.0105$, $\widehat{\text{se}}(\hat{\delta}) = 0.0093$ and $S = 1.1323$. Since $S < 2$ you do not reject $H_0 : \delta = \mu_{msft} - \mu_{sbux} = 0$ at the 5% level. ■

9.4.2 Test for equal Sharpe ratios between two assets

In the previous section, I considered testing if two assets have the same expected return. It is important to recognize that even if two assets have the same mean return they may have different standard deviations (volatilities) and, hence, different risks. Often, it is more interesting to test the hypothesis that two assets have the same reward-to-risk (i.e., Sharpe) ratios:

$$H_0 : \text{SR}_1 = \frac{\mu_1 - r_f}{\sigma_1} = \text{SR}_2 = \frac{\mu_2 - r_f}{\sigma_2} \text{ vs. } H_1 : \text{SR}_1 \neq \text{SR}_2.$$

where r_f denotes the risk-free rate over the investment horizon. As with testing the equality between two means, it is convenient to re-formulate the hypothesis of equal Sharpe ratio to the hypothesis that the difference in the Sharpe ratios is equal to zero:

$$H_0 : \delta = \text{SR}_1 - \text{SR}_2 = 0 \text{ vs. } H_1 : \delta = \text{SR}_1 - \text{SR}_2 \neq 0. \quad (9.26)$$

The plug-in estimator for δ is:

$$\hat{\delta} = \widehat{\text{SR}}_1 - \widehat{\text{SR}}_2 = \frac{\hat{\mu}_1 - r_f}{\hat{\sigma}_1} - \frac{\hat{\mu}_2 - r_f}{\hat{\sigma}_2}$$

To determine $\widehat{\text{se}}(\hat{\delta})$ from Equation 9.24, I utilize the result from Chapter Chapter 8 that the joint distribution of the vector $(\widehat{\text{SR}}_1, \widehat{\text{SR}}_2)$ is multivariate normal:

$$\begin{pmatrix} \widehat{\text{SR}}_1 \\ \widehat{\text{SR}}_2 \end{pmatrix} \sim N \left(\begin{pmatrix} \text{SR}_1 \\ \text{SR}_2 \end{pmatrix}, \begin{pmatrix} \frac{1}{T}(1 + \frac{1}{2}\widehat{\text{SR}}_1^2) & \frac{\hat{\rho}_{12}}{T}(1 + \frac{1}{2}\hat{\rho}_{12}\widehat{\text{SR}}_1\widehat{\text{SR}}_2) \\ \frac{\hat{\rho}_{12}}{T}(1 + \frac{1}{2}\hat{\rho}_{12}\widehat{\text{SR}}_1\widehat{\text{SR}}_2) & \frac{1}{T}(1 + \frac{1}{2}\widehat{\text{SR}}_2^2) \end{pmatrix} \right),$$

for large enough T . Then,

$$\begin{aligned} \widehat{\text{var}}(\hat{\delta}) &= \widehat{\text{var}}(\widehat{\text{SR}}_1) + \widehat{\text{var}}(\widehat{\text{SR}}_2) - 2\widehat{\text{cov}}(\widehat{\text{SR}}_1, \widehat{\text{SR}}_2) \\ &= \frac{1}{T}(1 + \frac{1}{2}\widehat{\text{SR}}_1^2) + \frac{1}{T}(1 + \frac{1}{2}\widehat{\text{SR}}_2^2) \\ &\quad - 2\frac{\hat{\rho}_{12}}{T}(1 + \frac{1}{2}\hat{\rho}_{12}\widehat{\text{SR}}_1\widehat{\text{SR}}_2) \\ &= \frac{1}{T} \left(2 + \frac{1}{2}(\widehat{\text{SR}}_1^2 + \widehat{\text{SR}}_2^2) - 2\hat{\rho}_{12}(1 + \frac{1}{2}\hat{\rho}_{12}\widehat{\text{SR}}_1\widehat{\text{SR}}_2) \right), \end{aligned}$$

and,

$$\widehat{\text{se}}(\hat{\delta}) = \sqrt{\frac{1}{T} \left(2 + \frac{1}{2}(\widehat{\text{SR}}_1^2 + \widehat{\text{SR}}_2^2) - 2\hat{\rho}_{12}(1 + \frac{1}{2}\hat{\rho}_{12}\widehat{\text{SR}}_1\widehat{\text{SR}}_2) \right)}.$$

You can test Equation 9.26 using the z-score:

$$z_{\delta=0} = \frac{\hat{\delta}}{\widehat{\text{se}}(\hat{\delta})} = \frac{\widehat{\text{SR}}_1 - \widehat{\text{SR}}_2}{\sqrt{\frac{1}{T} \left(2 + \frac{1}{2}(\widehat{\text{SR}}_1^2 + \widehat{\text{SR}}_2^2) - 2\hat{\rho}_{12}(1 + \frac{1}{2}\hat{\rho}_{12}\widehat{\text{SR}}_1\widehat{\text{SR}}_2) \right)}}, \quad (9.27)$$

with the test statistic $S = |z_{\delta=0}|$.

Example 9.9 (Testing equal Sharpe ratios between Microsoft and Starbucks). Let's use the z-score Equation 9.27 to test the hypothesis of equal Sharpe ratios between Microsoft and Starbucks using a 5% significance level. The R code to compute the z-score and test statistic is:

```
r.f = 0.03/12
SR.msft = (muhat.vals["MSFT"]-r.f)/sigmahat.vals["MSFT"]
SR.sbx = (muhat.vals["SBUX"]-r.f)/sigmahat.vals["SBUX"]
corhat = cor(gwnMonthlyRetC[, c("MSFT", "SBUX")])[1,2]
delta.hat = SR.msft - SR.sbx
var.deltahat = (2 + 0.5*(SR.msft^2 + SR.sbx^2)
               -2*corhat*(1+0.5*SR.msft*SR.sbx))
se.deltahat = sqrt(var.deltahat)
z.score.delta = delta.hat/se.deltahat
S = abs(z.score.delta)
```

```
ans = c(SR.msft, SR.sbox, delta.hat, se.deltahat, S)
names(ans) = c("SR.msft", "S.sbox", "deltahat", "se.deltahat", "S")
ans
#>      SR.msft      S.sbox    deltaxat se.deltahat      S
#>      0.0162      0.1089    -0.0927      1.1505      0.0805
```

Although the Sharpe ratio for Starbucks is much larger than the Sharpe ratio for Microsoft, there is substantial estimation error in $\hat{\delta} = \text{SR}_{msft} - \text{SR}_{sbox}$ which produces a small test statistic $S = 0.0805$. Since $S < 2$ you do not reject the null hypothesis of equal Sharpe ratios at the 5% level of significance. ■

9.5 Wald tests for general linear hypotheses (advanced)

- testing equal means across many assets
- testing equal Sharpe ratios across many assets (mention result from Jobson and Korke etc here)

9.6 Model Specification Tests

- Review GWN model assumptions
- Discuss why it is interesting and useful to test these assumptions
- What to do if certain assumptions are violated?

9.6.1 Tests for normality

- Why do you care about normality?
 - Return distribution only depends on mean and volatility (and correlation in multivariate case).
 - Justifies volatility as an appropriate measure of risk.
 - Gives formula for Value-at-risk calculations.
 - Justifies mean-variance portfolio theory and Black-Scholes option pricing formula.
 - GWN model parameter estimates and standard error formula do not require the data to be normally distributed. These estimates are consistent and asymptotically normally distributed for many non-normal distributions. There may be estimators that are more accurate (i.e., have smaller standard errors)
- What to do if returns are found to not be normally distributed?

- Return distribution characteristics in addition to mean and volatility may be important to investors. Volatility may not be the best measure of asset return risk. For example, skewness and kurtosis. If many returns are non-normal then this casts doubt on the appropriateness of mean-variance portfolio theory.
- Standard practice is to compute VaR based on a normal distribution. If returns are not normally distributed this is an incorrect way to compute VaR. Instead of using the normal distribution quantile to compute VaR you can use the empirical quantile to compute VaR. If returns are substantially non-normal in the tails of the distribution then the empirical quantile will be more appropriate than the normal quantile for computing VaR
- How are returns non-normal? This is important.

In chapter Chapter 5, I used histograms, normal QQ-plots, and boxplots to assess whether the normal distribution is a good characterization of the empirical distribution of daily and monthly returns. For example, Figure Figure 9.2 shows the four-panel plot for the monthly returns on Microsoft that I used to summarize the empirical distribution of returns. The histogram and smoothed histogram show that the empirical distribution of returns is bell-shaped like the normal distribution, whereas the boxplot and normal QQ-plot shows that returns have fatter tails than the normal distribution. Overall, the graphical descriptive statistics suggest that distribution of monthly Microsoft returns is not normally distributed. However, graphical diagnostics are not formal statistical tests. There are estimation errors in the statistics underlying the graphical diagnostics and it is possible that the observed non-normality of the data is due to sampling uncertainty. To be more definitive, I should use an appropriate test statistic to test the null hypothesis that the random return, R_t , is normally distributed. Formally, the null hypothesis to be tested is:

$$H_0 : R_t \text{ is normally distributed} \quad (9.28)$$

The alternative hypothesis is that R_t is not normally distributed:

$$H_1 : R_t \text{ is not normally distributed} \quad (9.29)$$

Here, I do not specify a particular distribution under the alternative (e.g., a Student's t distribution with a 5 degrees of freedom). The goal is to have a test statistic that will reject the null hypothesis Equation 9.28 for a wide range of non-normal distributions.

9.6.1.1 JB test based on skewness and kurtosis

In Chapter Chapter 2, I showed that the skewness and kurtosis of a normally distributed random variable are equal to zero and three, respectively. I can use this information to develop a simple

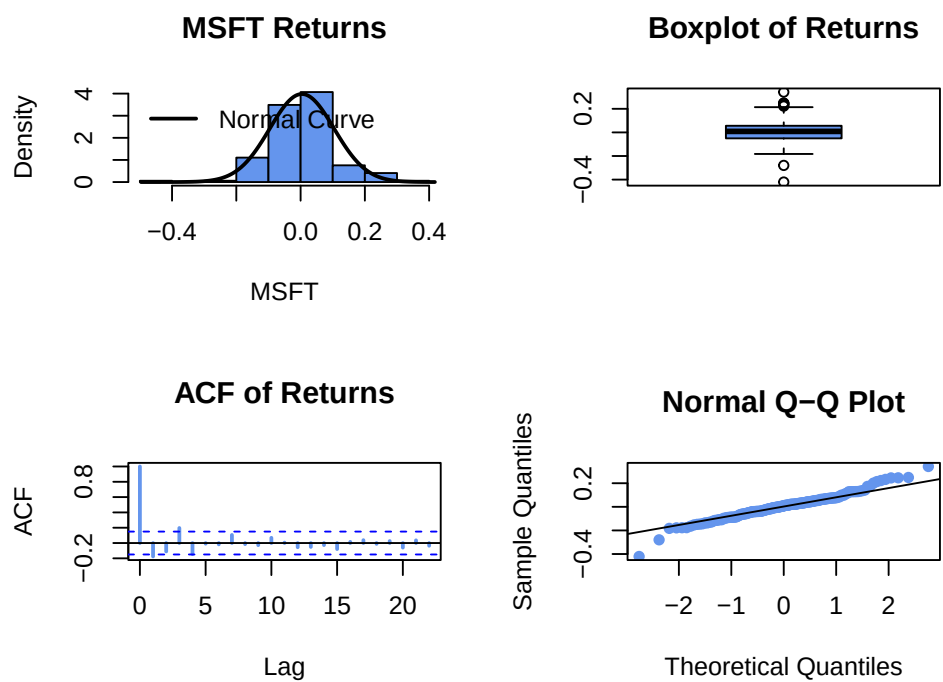


Figure 9.2: Four panel distribution plot for Microsoft monthly returns.

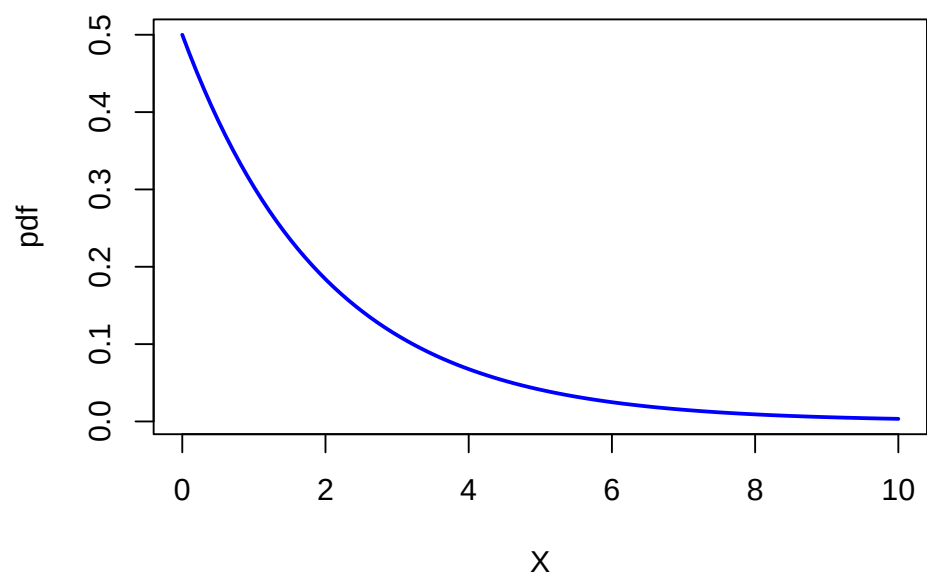


Figure 9.3: pdf of $\chi^2(2)$

test of Equation 9.28 against Equation 9.29 based on testing the hypotheses:

$$H_0 : \text{skew}(R_t) = 0 \text{ and } \text{kurt}(R_t) = 3 \quad (9.30)$$

versus

$$H_1 : \text{skew}(R_t) \neq 0 \text{ or } \text{kurt}(R_t) \neq 3 \text{ or both.} \quad (9.31)$$

To develop a test statistic for testing Equation 9.30 against Equation 9.31, Jarque and Bera (1981) used the result (see Kendall and Stuart, 1977) that if $\{R_t\} \sim \text{iid } N(\mu, \sigma^2)$ then by the CLT it can be shown that:

$$\begin{pmatrix} \widehat{\text{skew}}_r \\ \widehat{\text{kurt}}_r - 3 \end{pmatrix} \sim N \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \frac{6}{T} & 0 \\ 0 & \frac{24}{T} \end{pmatrix} \right),$$

for large enough T , where $\widehat{\text{skew}}_r$ and $\widehat{\text{kurt}}_r$ denote the sample skewness and kurtosis of the returns $\{r_t\}_{t=1}^T$, respectively. This motivates Jarque and Bera's *JB statistic*:

$$\text{JB} = T \times \left[\frac{(\widehat{\text{skew}}_r)^2}{6} + \frac{(\widehat{\text{kurt}}_r - 3)^2}{24} \right], \quad (9.32)$$

to test Equation 9.28 against Equation 9.29. If returns are normally distributed then $\widehat{\text{skew}}_r \approx 0$ and $\widehat{\text{kurt}}_r - 3 \approx 0$ so that $\text{JB} \approx 0$. In contrast, if returns are not normally distributed then $\widehat{\text{skew}}_r \neq 0$ or $\widehat{\text{kurt}}_r - 3 \neq 0$ or both so that $\text{JB} > 0$. Hence, if JB is big then there is data evidence against the null hypothesis that returns are normally distributed.

To determine the critical value for testing Equation 9.28 at a given significance level I need the distribution of the JB statistic Equation 9.32 under the null hypothesis Equation 9.28. To this end, Jarque and Bera show, assuming Equation 9.28 and large enough T , that:

$$\text{JB} \sim \chi^2(2),$$

where $\chi^2(2)$ denotes a chi-square distribution with two degrees of freedom (see Appendix). Hence, the critical values for the JB statistic are the upper quantiles of $\chi^2(2)$. Figure 9.3 shows the pdf of $\chi^2(2)$. The 90%, 95% and 99% quantiles can be computed using the R function `qchisq()`:

```
qchisq(c(0.90, 0.95, 0.99), 2)
#> [1] 4.61 5.99 9.21
```

Since the 95% quantile of $\chi^2(2)$ is $5.99 \approx 6$, a simple rule of thumb is to reject the null hypothesis Equation 9.28 at the 5% significance level if the JB statistic Equation 9.32 is greater than 6.

Let $X \sim \chi^2(2)$. The p-value of the JB test is computed using:

$$p\text{-value} = \Pr(X > \text{JB}).$$

If the `p-value()` is less than α then the null () is rejected at the

Example 9.10 (Using the JB statistic to test the normality assumption for returns on Microsoft Starbucks and the S&P 500 index). The sample skewness and excess kurtosis values for the monthly returns on Microsoft, Starbucks, and the S&P 500 index are

```
skewwhat = skewness(gwnMonthlyRetC)
ekurthat = kurtosis(gwnMonthlyRetC)
rbind(skewwhat, ekurthat)
#>           MSFT   SBUX SP500
#> Skewness    -0.0901 -0.987 -0.74
#> Excess Kurtosis 2.0802 3.337 1.07
```

The sample skewness values for the assets are not too large but the sample excess kurtosis values are fairly large. The JB statistics for the three assets are easy to compute

```
n.obs = nrow(gwnMonthlyRetC)
JB = n.obs*((as.numeric(skewwhat)^2)/6 + (as.numeric(ekurthat)^2)/24)
names(JB) = colnames(gwnMonthlyRetC)
JB
#> MSFT SBUX SP500
#> 31.2 107.7 23.9
```

Since all JB values are greater than 6, you reject the null hypothesis that the monthly returns are normally distributed.

You can also use the `tseries` function `jarque.bera.test()` to compute the JB statistic:

```
jarque.bera.test(gwnMonthlyRetC[, 1])
#>
#> Jarque Bera Test
#>
#> data: gwnMonthlyRetC[, 1]
#> X-squared = 31, df = 2, p-value = 2e-07
jarque.bera.test(gwnMonthlyRetC[, 2])
#>
#> Jarque Bera Test
#>
#> data: gwnMonthlyRetC[, 2]
#> X-squared = 108, df = 2, p-value <2e-16
jarque.bera.test(gwnMonthlyRetC[, 3])
```

```
#>
#>  Jarque Bera Test
#>
#> data:  gwnMonthlyRetC[, 3]
#> X-squared = 24, df = 2, p-value = 7e-06
```

The test statistics are slightly different from the direct calculation above due to differences in how the sample skewness and sample kurtosis is computed within the function `jarque.bera.test()`.

■

9.6.1.2 Shipiro Wilks test

To be complete...

9.6.2 Tests for no autocorrelation

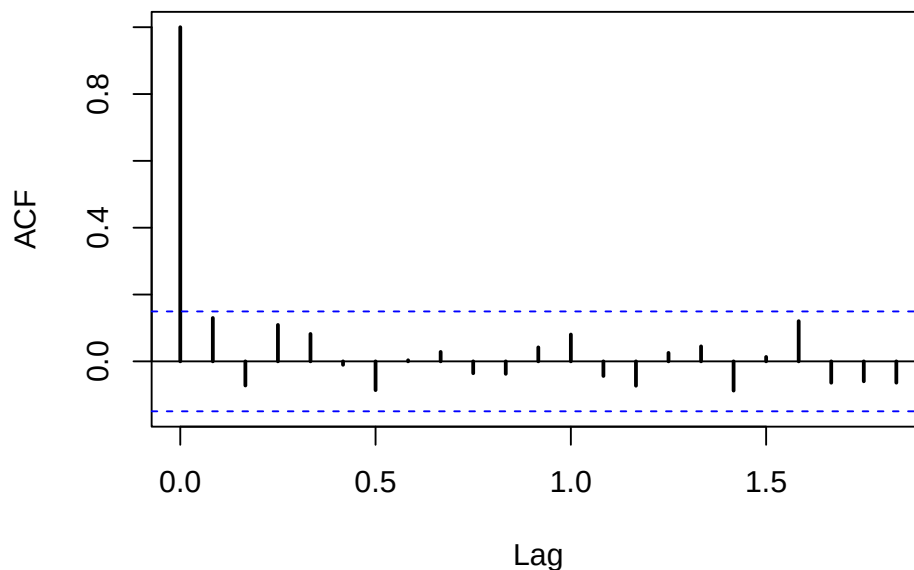


Figure 9.4: SACF for monthly returns on the S&P 500 index.

A key assumption of the GWN model is that returns are uncorrelated over time (not autocorrelated). In efficient financial markets where prices fully reflect true asset value, returns of actively traded assets should not be autocorrelated. In addition, if returns are autocorrelated then they

are predictable. That is, future returns can be predicted from the past history of returns. If this is the case then trading strategies can be easily developed to exploit this predictability. But if many market participants act on these trading strategies then prices will adjust to eliminate any predictability in returns. Hence, for actively traded assets in well functioning markets I expect to see that the returns on these assets are uncorrelated over time.

For less frequently traded assets returns may appear to be autocorrelated due to stale pricing.

As you saw in Chapter Chapter 6, this allows us to aggregate the GWN model over time and, in particular, derive the square-root-of time rule for aggregating return volatility. If returns are substantially correlated over time (i.e., autocorrelated) then the square-root-of-time rule will not be an accurate way to aggregate volatility.

To measure autocorrelation in asset returns $\{R_t\}$ I use the j^{th} lag autocorrelation coefficient

$$\rho_j = \text{cor}(R_t, R_{t-j}) = \frac{\text{cov}(R_t, R_{t-j})}{\text{var}(R_t)},$$

which is estimated using the sample autocorrelation

$$\hat{\rho}_j = \frac{\frac{1}{T-1} \sum_{t=j+1}^T (r_t - \hat{\mu})(r_{t-j} - \hat{\mu})}{\frac{1}{T-1} \sum_{t=1}^T (r_t - \hat{\mu})^2}.$$

In chapter Chapter 5, I used the sample autocorrelation function (SACF) to graphically evaluate the data evidence on autocorrelation for daily and monthly returns. For example, Figure Figure 9.4 shows the SACF for the monthly returns on the S&P 500 index. Here you see that none of the $\hat{\rho}_j$ values are very large. You also see a pair of blue dotted lines on the graph. As you shall see, these dotted lines are related to critical values for testing the null hypothesis that $\rho_j = 0$. Hence, if

9.6.2.1 Individual tests

Individual tests for autocorrelation are similar to coefficient tests discussed earlier and are based on testing the hypotheses

$$H_0 : \rho_j = 0 \text{ vs. } H_1 : \rho_j \neq 0,$$

for a given lag j . The tests rely on the result, justified by the CLT if T is large enough, that if returns are covariance stationary and uncorrelated then

$$\hat{\rho}_j \sim N\left(0, \frac{1}{T}\right) \text{ for all } j \geq 1 \text{ and } \text{se}(\hat{\rho}_j) = \frac{1}{\sqrt{T}}.$$

This motivates the test statistics

$$t_{\rho_{j=0}} = \frac{\hat{\rho}_j}{\text{se}(\hat{\rho}_j)} = \frac{\hat{\rho}_j}{1/\sqrt{T}} = \sqrt{T}\hat{\rho}_j, j \geq 1$$

and the 95% confidence intervals

$$\hat{\rho}_j \pm 2 \cdot \frac{1}{\sqrt{T}}.$$

If you use a 5% significance level, then you reject the null hypothesis $H_0 : \rho_j = 0$ if

$$|t_{\rho_{j=0}}| = |\sqrt{T}\hat{\rho}_j| > 2.$$

That is, reject if

$$\hat{\rho}_j > \frac{2}{\sqrt{T}} \text{ or } \hat{\rho}_j < \frac{-2}{\sqrt{T}}.$$

Here, note that the dotted lines on the SACF are at the points $\pm 2 \cdot \frac{1}{\sqrt{T}}$. If the sample autocorrelation $\hat{\rho}_j$ crosses either above or below the dotted lines then you can reject the null hypothesis $H_0 : \rho_j = 0$ at the 5% level.

9.6.2.2 Joint tests

- Have to be careful how to interpret the individual tests. Cannot simply use 20 individual tests at the 5% level to test a joint hypothesis at the 5% level. With independent individual tests each test has a 5% probability of rejecting the null when it is true. So if you do 20 individual tests it would not be surprising if one tests rejects and this could be purely by chance (a type I error) if the null of no autocorrelation was true.

To be complete

9.6.3 Tests for constant parameters

A key assumption of the GWN model is that returns are generated from a covariance stationary time series process. As a result, all of the parameters of the GWN model are constant over time. However, as shown in Chapter Chapter 5 there is evidence to suggest that the assumption of covariance stationarity may not always be appropriate. For example, the volatility of returns for some assets does not appear to be constant over time.

A natural null hypothesis associated with the covariance stationarity assumption for the GWN model for an individual asset return has the form:

$$H_0 : \mu_i, \sigma_i, \rho_{ij} \text{ are constant over time.}$$

To reject this null implied by covariance stationarity, one only needs to find evidence that at least one parameter is not constant over time.

9.6.3.1 Formal tests (two parameter structural change)

To be completed

- Do two-sample structural change tests based on known segments. Analysis is very similar to testing equality of parameters across two assets.

9.6.4 Tests for constant parameters based on rolling estimators

In Chapter Chapter 5 I introduced rolling descriptive statistics to informally investigate the assumption of covariance stationarity. Given any parameter θ of the GWN model (or any function of the parameters), you can compute a set of rolling estimates of θ over windows of length $n < T$: $\{\hat{\theta}_t(n)\}_{t=n}^T$. A time plot of these rolling estimates may show evidence of non-stationary behavior in returns.

However, you must always keep in mind that estimates have estimation error and part of the observed time variation in the rolling estimates $\{\hat{\theta}_t(n)\}_{t=n}^T$ is due to random estimation error. To account for this estimation error, rolling estimates are often displayed with estimated standard error bands of the form:

$$\hat{\theta}_t(n) \pm c \times \widehat{\text{se}}(\hat{\theta}_t(n)), \quad (9.33)$$

where c is a constant representing some multiple of the estimated standard error. Typically, c is chosen so that Equation 9.33 represents an approximate confidence interval for θ . For example, setting $c = 2$ gives an approximate 95% confidence interval.

Example 9.11. As in Chapter Chapter 5, you can compute 24-month rolling estimates of μ and σ for Microsoft, Starbucks and the S&P 500 index using the **zoo** function `rollapply()`:

```
roll.muhat = rollapply(gwnMonthlyRetC, width = 24, by = 1,
                      by.column = TRUE, FUN = mean, align = "right")
roll.sigmahat = rollapply(gwnMonthlyRetC, width = 24, by = 1,
                          by.column = TRUE, FUN = sd, align = "right")
```

Estimated standard errors for $\hat{\mu}_t(n)$ and $\hat{\sigma}_t(n)$ are computed using:

$$\widehat{\text{se}}(\hat{\mu}_t(n)) = \frac{\hat{\sigma}_t(n)}{\sqrt{T}}, \quad \widehat{\text{se}}(\hat{\sigma}_t(n)) = \frac{\hat{\sigma}_t(n)}{\sqrt{2T}}$$

Figure 9.5 shows the rolling estimates of μ (top panel) and σ (bottom panel) for Starbucks together with 95% confidence intervals ($\pm 2 \times$ standard error bands) computed using:

```
se.muhat.SBUX = roll.sigmahat[, "SBUX"]/sqrt(24)
se.sigmahat.SBUX = roll.sigmahat[, "SBUX"]/sqrt(2 * 24)
lower.muhat.SBUX = roll.muhat[, "SBUX"] - 2 * se.muhat.SBUX
upper.muhat.SBUX = roll.muhat[, "SBUX"] + 2 * se.muhat.SBUX
lower.sigmahat.SBUX = roll.sigmahat[, "SBUX"] - 2 * se.sigmahat.SBUX
upper.sigmahat.SBUX = roll.sigmahat[, "SBUX"] + 2 * se.sigmahat.SBUX
# plot estimates with standard error bands
dataToPlot = merge(roll.muhat[, "SBUX"], lower.muhat.SBUX,
                   upper.muhat.SBUX, roll.sigmahat[, "SBUX"],
                   lower.sigmahat.SBUX, upper.sigmahat.SBUX)
my.panel <- function(...) {
  lines(...)
  abline(h = 0)
}
plot.zoo(dataToPlot, main = "", screens = c(1, 1, 1, 2, 2, 2),
         lwd = 2, col = c("black", "red", "red", "black", "red", "red"),
         lty = c("solid", "dashed", "dashed", "solid", "dashed", "dashed"),
         panel = my.panel, ylab = list("muhat", "sigmahat"))
```

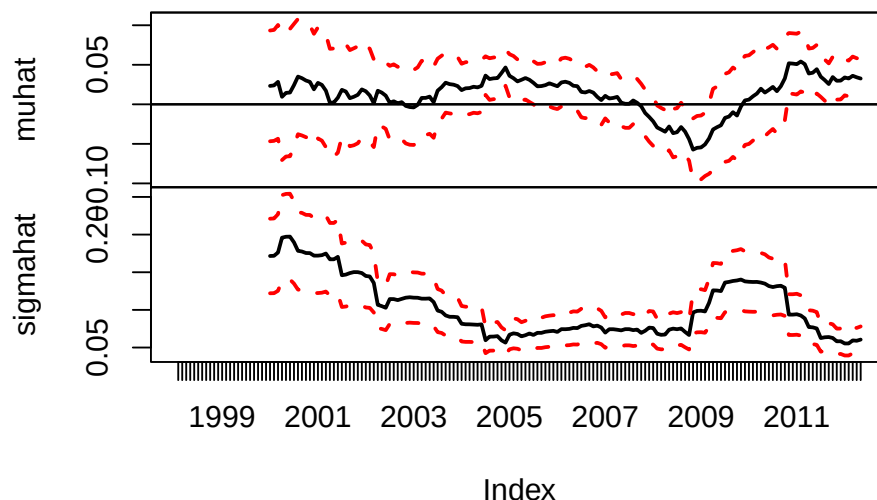


Figure 9.5: 24-month rolling estimates of μ and σ for Starbucks with estimated standard error bands

The standard error bands for $\hat{\mu}_t(n)$ are fairly wide, especially during the high volatility periods of the dot-com boom-bust and the financial crisis. During the dot-com period the 95% confidence intervals for μ are wide and contain both positive and negative values reflecting a high degree of uncertainty about the value of μ . However, during the economic boom around 2005, where $\hat{\mu}_t(n)$ is large and $\hat{\sigma}_t(n)$ is small, the 95% confidence interval for μ does not contain zero and one can infer that the mean is significantly positive. Also, at the height of the financial crisis, where $\hat{\mu}_t(n)$ is the largest negative value, the 95% confidence interval for μ contains all negative values and one can infer that the mean is significantly negative. At the end of the sample, the 95% confidence intervals for μ contain all positive values.⁷

The standard error bands for $\hat{\sigma}_t(n)$ are smaller than the bands for $\hat{\mu}_t(n)$ and one can see more clearly that $\hat{\sigma}_t(n)$ is significantly larger during the dot-com period and the financial crisis period than during the economic boom period and the period after the financial crisis. ■

Plotting rolling estimates with estimated standard error bands is good practice as it reminds you that there is estimation error in the rolling estimates and part of any observed time variation is

⁷You have to be a little careful about interpreting the implicit hypotheses tests underlying the rolling 95% confidence intervals. Each confidence interval represents a single hypothesis test with a 5% significance level. However, we have many confidence intervals and so we are really doing many hypothesis tests. The joint significance level for all of these tests will be larger than the significance level for a single test.

this estimation error.

9.7 Using Monte Carlo Simulation to Understand Hypothesis Testing

In chapter Chapter 7 I used Monte Carlo simulation to understand the statistical properties of estimators. Now, I will use Monte Carlo Simulation to understand hypothesis testing. To fix ideas, let R_t be the return on a single asset described by the GWN model, let θ denote some parameter of the GWN model you are interested in testing an hypothesis about, let $\hat{\theta}$ denote an estimator for θ based on a sample of size T , and let $\widehat{\text{se}}(\hat{\theta})$ denote the estimated standard error for $\hat{\theta}$. Suppose you are interested in testing $H_0 : \theta = \theta_0$ against $H_1 : \theta \neq \theta_0$ using the z-score $z_{\theta=\theta_0} = \frac{\hat{\theta} - \theta_0}{\widehat{\text{se}}(\hat{\theta})}$.

To be completed

9.8 Using the Bootstrap for Hypothesis Testing

To be completed

- duality between HT and CIs can be exploited by the bootstrap. Use bootstrap to compute CI and see if hypothesized values lies in the interval. Illustrate with testing equality of 2 Sharpe ratios

9.9 Further Reading: Hypothesis Testing in the GWN Model

- Mention books by Carol Alexander for discussion of rolling and EWMA estimates
- Formal tests for structural change require advanced probability and statistics. See (Zivot 2016) for discussion. Many tests are implemented in the **strucchange** package.

9.10 Exercises

To be completed...

References

- Abadir, K. M., and J. R. Magnus. 2005. *Matrix Algebra (Econometric Exercises, Vol. 1)*. Cambridge University Press.
- Amemiya, T. 1994. *Introduction to Statistics and Econometrics*. Cambridge, MA.: Harvard University Press.
- Ang, A. 2014. *Asset Management: A Systematic Approach to Factor Investing*. New York: Oxford University Press.
- Bacon, C. 2008. *Practical Portfolio Performance Measurement and Attribution, Second Edition*. Chichester, England: John Wiley & Sons.
- Banerjee, S., and A. Roy. 2014. *Linear Algebra and Matrix Analysis for Statistics*. London: Chapman; Hall/CRC.
- Box, G., and G. M. Jenkins. 1976. *Time Series Analysis : Forecasting and Control*. San Francisco: Holden-Day.
- Carmona, R. 2014. *Statistical Analysis of Financial Data in R, Second Edition*. New York: Springer.
- Casella, G., and R. L. Berger. 2002. *Statistical Inference. 2nd Ed.* Pacific Grove, CA: Thomson Learning.
- Chiang, A. 1984. *Fundamental Methods of Mathematical Economics, Third Edition*. New York: McGraw-Hill.
- Christopherson, J. A., D. R. Carino, and W. E. Ferson. 2009. *Portfolio Performance Measurement and Benchmarking*. McGraw-Hill.
- Dalgaard, P. 2002. *Introductory Statistics with R*. Springer.
- Davison, A., and D. Hinkley. 1997. *Bootstrap Methods and Their Application*. Cambridge: Cambridge University Press.
- DeGroot, M. H., and M. J. Schervish. 2011. *Probability and Statistics, 4th Edition*. Pearson.
- Fama, E. F. 1976. *Foundations of Finance*. New York: Basic Books.
- Fieller, N. 2015. *Basics of Matrix Algebra for Statistics with R*. Boca Raton: Taylor & Francis.
- Goldberger, A. S. 1991. *A Course in Econometrics*. Cambridge, MA.: Harvard University Press.
- Jondeau, E., S.-H. Poon, and M. Rockinger. 2007. *Financial Modeling Under Non-Gaussian Distributions*. New York: Springer.
- Jorion, P. 1997. *Value at Risk: The New Benchmark for Controlling Market Risk*. New York: McGraw-Hill.
- Lutkepohl, H. 1997. *Handbook of Matrices*. Wiley.
- Pfaff, B. 2013. *Financial Risk Modelling and Portfolio Optimization with R*. Chichester, UK.:

- Wiley.
- Qian, E. E. 2019. *Portfolio Rebalancing*. Boca Raton, FL: Chapman; Hall/CRC.
- Ruppert, D., and D. S. Matteson. 2015. *Statistics and Data Analysis for Financial Engineering with *r* Examples*. New York: Springer.
- Searle, S. 1982. *Matrix Algebra Useful for Statistics*. New York: Wiley.
- Sims, C. A. 1980. *Macroeconomics and Reality*. Econometrica.
- Strang, G. 1980. *Linear Algebra and Its Applications*. Academic Press.
- Tsay, R. 2010. *Analysis of Financial Time Series*. Wiley.
- . 2014. *An Introduction to Analysis of Financial Time Series*. Wiley.
- Verzani, J. 2005. *Using *r* for Introductory Statistics*. Boca Raton: Chapman & Hall/CRC.
- Vinod, H. D. 2010. *Advances in Social Science Research Using *r**. New York, NY: Springer.
- Zivot, E. 2016. *Modeling Financial Time Series with *r**. New York: Springer.